Psychological Statistics

Stats for Kids Who Don't Read Good

Alexander P. Demos, PhD

University of Illinois Chicago

Department of Psychology

Version 0.5 - August 2026

Psychological Statistics
Stats for Kids Who Don't Read Good

Copyright © 2026 Alexander P. Demos, PhD

Except where otherwise noted, the original text, figures, data, and example code in this work are licensed under the Creative Commons Attribution–NonCommercial 4.0 International License (CC BY-NC 4.0).

You may share and adapt the licensed material with appropriate attribution for noncommercial purposes. You must indicate whether changes were made.

https://creativecommons.org/licenses/by-nc/4.0/

Third-party images, quotations, datasets, trademarks, and other material are excluded from this license and remain subject to their respective rights.

No warranties are given. The license may not provide every permission needed for a particular use of third-party material.

First digital edition, 2026.

Table of contents

Preface 5

I The Necessary Slog 7

1 Why You Have to Learn Statistics (Sorry not sorry) 8

2 R: The Extremely Short Survival Guide 20

3 Probability: The Gods Are Fickle 30

4 Distributions and their Moments 41

5 Standard Error 63

6 One Sample t-Test 76

7 Power and Effect Size 91

8 Independent Samples t-Test 112

9 Paired-Samples t-Test 124

II Putting Lines Through Things 136

10 Covariance and Correlation 137

11 Linear Regression 160

12 Linear Regression with Categorical Variables 176

13 Multiple Regression: Statistical Control 195

14 Hierarchical Regression: Testing Whether Your Model Improves 212

15 Statistical Control: What Are We Removing? 225

16 Continuous-by-Categorical Interactions: When the Effect of One Variable Depends on Another 240

17 Categorical-by-Categorical Interactions: The 2-by-2 Design 261

18 Mixed Regression: Between People and Within People 280

III Okay, But What If It's Complicated? 300

19 Advanced: Regression Diagnostics and Influential Weirdos 301

20 P-Values and Multiple Comparisons 320

21 Advanced: Continuous and Higher-Order Interactions 337

22 Generalized Linear Models 356

23 Mediation 372

24 Moderated Mediation 384

IV ANOVA is dead, Long Live ANOVA 391

25 Historical ANOVA Lectures 392

26 Introduction to Analysis of Variance (ANOVA) 393

27 Follow-Ups to One-Way ANOVAs 406

28 Calculating the Two-Way Between-Subjects ANOVA 423

29 Following Up the Two-Way Between-Subjects ANOVA 432

30 From the Paired *t*-Test to the One-Way Within-Subjects ANOVA 443

31 Two-Way Within-Subjects ANOVA: Analysis and Graphing 453

32 Follow-Ups to the Two-Way Within-Subjects ANOVA 461

33 Two-Way Mixed-Design ANOVA: Analysis and Graphing 469

V Other Statistical Creatures 478

34 Pearson's Chi-Square 479

35 Nonparametric Tests: When the Data Refuse to Behave 496

36 Advanced: Bootstrapping, the Jackknife, and Resampling 506

VI When R Inevitably Betrays You 520

37 Reporting Models Without Making the Reader Solve a Puzzle 521

38 R Help and Review: Tidyverse Without Tears 527

References 542

Preface

I'm dyslexic, I don't like reading, and I wrote a stats textbook. Make it make sense.

Here's the deal: I learn by *doing* things and *seeing* things. Dense paragraphs of text make my brain go on vacation unless they are funny, interesting, or use analogies. So that's what this book is: nothing abstract is allowed to stay abstract. Every idea gets a picture, a story, or R code you can run to watch it happen. If I ever tell you something is true without showing you, I have failed and you should email me about it.

Fair warning, though. I said I would keep this short and then I wrote seventy thousand words and counting, plus more code than you can shake a stick at. I plan to add chapters, so sadly there will be more of both. What I can promise is that no long stretch goes by without a simulation, an analogy, or a joke at my own expense.

This book teaches **statistics and R at the same time**—because honestly, you learn both better together. Stats without code is just memorizing formulas. Code without stats is just typing. Together they're actually useful.

A few things you should know going in:

I hate ANOVA. It's old. It's clunky. Psychologists have been running ANOVAs since the Eisenhower administration and it needs to stop. Everything you can do with an ANOVA you can do with regression, and regression scales up to the real world in ways ANOVA never will. So we're going full regression in this book. I have added my old ANOVA lectures as a section in the book for when someone inflicts it on you.

The undergraduate chapters assume you have zero statistics background and approximately zero R experience. That's fine. We start from scratch.

Most of these chapters are my deeply misunderstood summaries of *Applied Multiple Regression/Correlation Analysis for the Behavioral Sciences* by Cohen, Cohen, West, and Aiken (Cohen et al. 2003). That book is the holy bible of regression. This book isn't trying to replace it. The Cohen book gives you more theory, more detail, more qualifications, and a lot more words. If you need the full argument, or you're planning advanced work, go read the relevant chapters there.

This book has a smaller job. It's a free knockoff assembled from a decade-plus of me ranting about statistics: lectures, class materials, examples, mistakes, revised examples, and explanations that finally worked after the first three explanations failed. The goal is to get you up to speed on the statistical theory you need to survive, do research responsibly, and recognize when an analysis has gone feral.

> **About the AI Editor Standing Behind the Curtain**
>
> AI was the editor for this book. Claude and ChatGPT helped clean up my grammar and spelling, merge material from my different courses, modernize examples because I'm old and I bet half of you don't even know the title is a *Zoolander* reference, fix the formatting, fix my stupid R code errors, check consistency across chapters, and reword explanations so they make sense to people who aren't dyslexic and can't automatically translate Alex into English.
>
> Lots of people have negative feelings about AI. I don't. This technology opens doors for people like me who'd never otherwise dream of turning a decade-plus of lectures into one coherent, free book. A traditional publisher would've been amazing, but what they'd charge you for it could be astronomical. That's not my goal. My goal is to make statistics accessible, and I'll use every tool I've got to do it.
>
> AI also produces confident nonsense. I've tried to catch its hallucinations, check the code, keep my own voice when it keeps trying to make me sound like a robot, and stay responsible for what shows up here. But as I already warned you, I don't like to read, and even when I check, I might not notice it changed my meaning when I asked it to help translate me to neurotypicals. If you find something wrong, tell me before I confuse people more than I usually do.

None of this comes together without the many students who've suffered in my classes, especially my Spring 2026 class, who gave me feedback on drafts of several chapters. I owe thanks to my graduate-student TAs and my many colleagues. I also need to thank Stephanie Del Tufo, who uses me as a pilot subject for her dyslexia research and who pushed me to add sections on writing up results in APA style. And I should apologize in advance if I borrowed material from my super-smart and awesome colleagues Ryne Estabrook and Linda Skitka, or saw something on the web, thought, *Wow, that looks cool for my lecture*, and promptly forgot where I found it. I'm 1000% sure Ryne contributed to many places in this book because we've edited each other's R Markdown lectures over the years. Thanks, Ryne. I'll make sure to share all the massive profits with you. Oh wait, it's free!

Anyway, I have the memory of a goldfish. If I forgot to acknowledge you someplace, tell me and I'll happily fix it. Thanks as well to everyone who believes in sharing teaching materials on the web. Borrow whatever you want from me, but do try to cite me if you can remember; I can show it to my department to remind them not to get rid of me, since I'm often more trouble than I'm worth.

The goal is that by the end, you can look at a dataset, run an appropriate model in R, make a plot that actually communicates something, and write up the results without crying.

Happy coding!

Please Note: I'll be expanding the advanced chapters to include more of my graduate-level material.

Part I

The Necessary Slog

1 Why You Have to Learn Statistics (Sorry not sorry)

1.1 Why Should You Care?

You are taking a statistics class. I assume this was always your dream.

Maybe you woke up as a child, looked at the ceiling, and thought, *One day I hope to calculate a standard error.* More likely, you have to read this book because someone has blocked your graduation until you pass a statistics class. Either way, statistics matters because psychology runs on data, and data are messy!

Before we go any farther, here are three phrases researchers use constantly and explain approximately never:

- **Statistically significant** means something like, "This result probably, maybe did not happen just because chance was being weird." That definition is deliberately incomplete. We will give it a real one in the one-sample *t*-test chapter, after you learn some probability theory and are emotionally prepared for it.
- An **asterisk** is the little star researchers sometimes put after a number to show that it was statistically significant: for example, `r=.32*`. More stars may indicate smaller *p*-values, depending on the researchers' chosen rules, but more does not mean better or "more better," as I say constantly. Asterisks do not mean the effect is large, important, true, or blessed by the Statistics Gods.
- An **effect size** asks how strong or large an effect is. Think of a small effect as being hit over the head with a coffee stirrer, a medium effect as a Nerf bat, a large effect as a wooden bat, and a huge effect as a metal bat. A **gargantuan effect**—a made-up but extremely technical term—is your mother hitting you over the head with her slipper or the hanger she happens to be holding. The effect is so devastating that you would have preferred to have taken the metal bat.

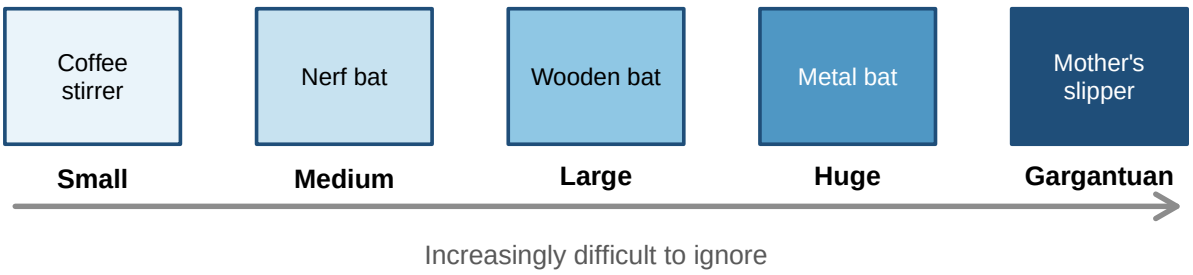


Figure 1.1: The official effect-size scale, ordered by regret.

Statistics does four jobs for us:

The Job	Psychological Example	Why You Care
Analyze and interpret data	Two thousand students answer a mental-health survey.	Statistics turns 2,000 separate cries for help into a pattern we can understand.
Make decisions	A therapy group improves more than a control group.	We need to decide whether the difference looks like a real effect or the kind of nonsense random samples produce.
Design research	We want to test whether sleep improves memory.	Statistics helps us choose the sample size, variables, comparisons, and analysis *before* we start throwing undergraduates at the problem.
Communicate findings	We found a relationship between stress and exam performance.	A graph, effect size, and confidence interval tell people how large and uncertain the result is. "Trust me, bro, it looked huge in Excel" does not cut it.

These are not four separate activities. Your design determines the data you get. Your measurement determines what the numbers mean. Your analysis determines what claims are defensible. Then your graph determines whether anyone understands you before they wander away.

Statistics is therefore not the decorative math placed at the end of a study. It is part of the logic of the entire study.

Abelson called statistics a form of **principled argument** (Abelson 1995). The software will calculate whatever you ask it to calculate, including complete garbage. Your job is to build an argument in which the design, measurement, analysis, and conclusion are all talking about the same thing.

That argument usually asks whether **chance is enough** to explain what happened or whether we need some systematic explanation on top of it. Unfortunately, chance is fickle. Random data do not arrive evenly spaced, alphabetized, and wearing little name tags that say, "Hello, I am meaningless noise." They form streaks, clusters, suspicious-looking differences, and occasionally an image of a saint on a piece of toast. Statistics helps us judge how much of that weirdness a reasonable chance process could produce.

Abelson argued that doing this well requires the combined skills of an honest lawyer, a detective, and a storyteller (Abelson 1995). The lawyer builds a defensible case. The detective looks for alternative explanations and evidence that does not fit. The storyteller explains why the comparison matters. The word **honest** is doing considerable work here. Statistics does not let us torture the data until they confess to whatever we hoped was true.

1.2 The Replication Crisis: Our Extremely Rude Wake-Up Call

A **replication** repeats a study to see whether the result appears again. If a finding is "real," we should be able to find it more than once. Not every single time—chance is fickle—but often enough that the result becomes more trustworthy.

In 2015, the Open Science Collaboration repeated 100 published psychology studies. Ninety-seven of the original studies reported statistically significant results. Only 36 replications did, and the replication effects were, on average, about half the original effect size (Open Science Collaboration 2015). Psychology—well, mostly social psychology—responded with the scientific equivalent of waking up to find the house on fire and tried to do something about it. *They are still sort of overcorrecting and have totally lost the forest for the trees, but we will love them for caring, as some other areas of psychology and neuroscience still think this is not a me problem.*

This was not proof that 64% of psychology is fake. Replication is not a divine courtroom where one failed study gets to bang a tiny gavel from the Statistics Gods and declare a theory dead. The projects differed in methods, samples, measures, and statistical power. Still, the result was far worse than psychologists expected.

1.2.1 Why Results Fail to Replicate

The problem is partly misunderstanding statistics:

- Treating the **null hypothesis** as if it must be exactly true, even though tiny relationships are almost everywhere (Meehl 1978).
- Treating a **p-value** as the probability that the theory is wrong. It is not (Cohen 1994).
- Using small samples that can only detect cartoonishly large effects (Cohen 1992; Button et al. 2013).
- Ignoring effect sizes and power, then acting shocked when a weak study produces unstable results.
- Calculating **post hoc power** from the result we already observed, which mostly turns the p-value into a new number wearing a fake mustache (Hoenig and Heisey 2001).
- Publishing exciting positive results while null results live forever in someone's forgotten `Final_Final_REAL_Results.docx` file (Ioannidis 2005).

The problem is also systemic. Journals want novelty. Universities want publications. Funders want breakthroughs. Nobody gets a parade for carefully discovering that the exciting effect is probably 0.08 instead of 0.80. Those incentives create publication bias and reward researchers for finding asterisks rather than finding the truth (Ioannidis 2012). You cannot personally repair every journal, university, and funding agency this semester. I checked, and maybe wrote, your syllabus. We do not have time.

But you *can* stop contributing avoidable statistical nonsense by learning how probability drives the stability of results.

For example, suppose you put 20 people in therapy. The Probability Gods might hand you 20 unusually cooperative people who respond beautifully. Or they might give you 20 people who hate your face, refuse to listen to anything you say, and make the treatment look useless. With 200 people, it becomes much harder for the Probability Gods to find enough people who all hate your face. The oddballs begin to balance one another out, so the estimated effects tend to cluster closer to the true effect. Probability is always fickle; it just has less room to wreck the study.

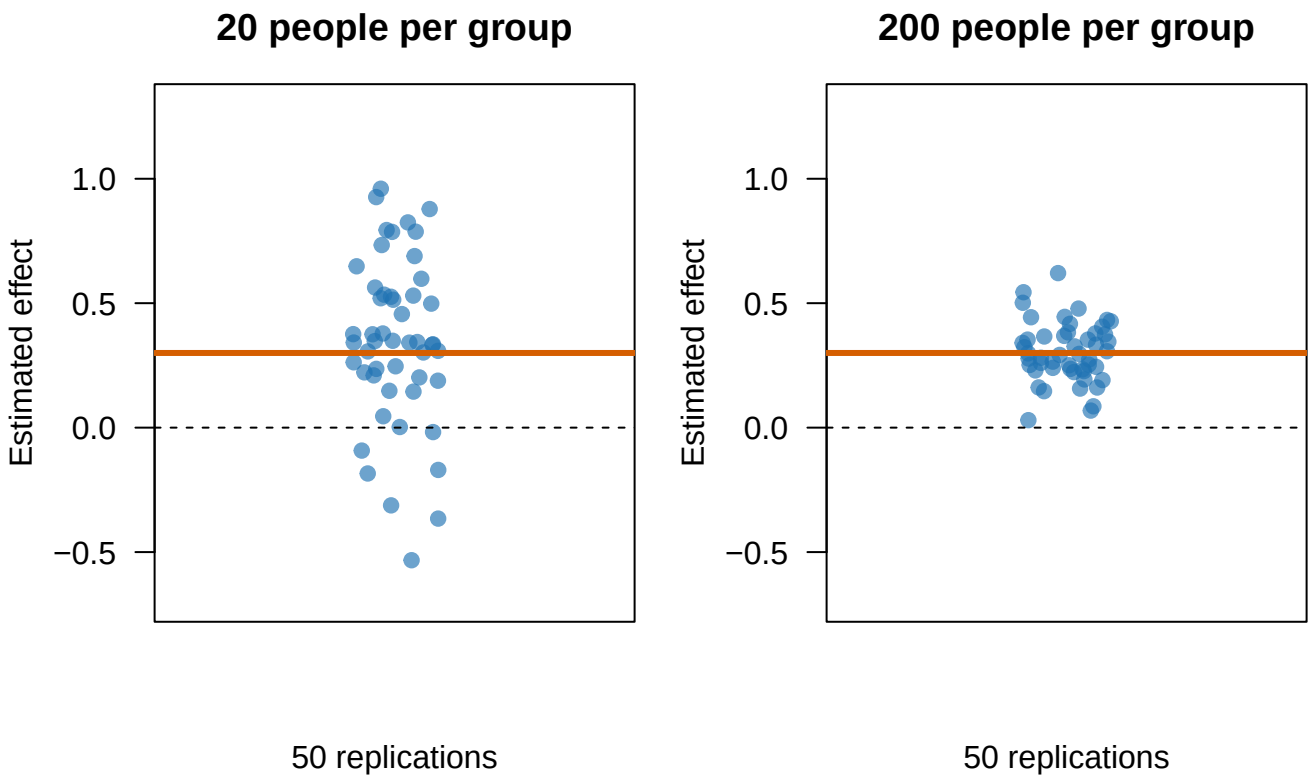


Figure 1.2: A real effect can produce wildly different estimates in small studies. Each point is one simulated replication; the orange line is the true effect.

We can see from the graph that collecting more people makes our estimate more stable. Small samples can produce wildly inaccurate and unstable guesses because probability is fickle. What does *probability is fickle* mean? It means the Gods who govern probability decide whether to do helpful or unhelpful things because they feel like it or maybe they also don't like your face.

1.2.2 This Still Affects Your Life in a Serious Way

The crisis did not end in 2015, and it is not confined to social-psychology trivia, as some psychologists still sadly believe.

Field	What they redid	What happened
Preclinical cancer biology (Errington et al. 2021)	50 experiments from 23 influential papers	Replicated effects were typically much smaller, with a median effect size **85% below** the originals.
Brain-behavior imaging (Marek et al. 2022)	Brain-wide association studies	Below 500 people, replication rates ran around **5%**. Stable associations often required **thousands** of participants.
COVID-era social claims (Marcoci et al. 2025)	29 sufficiently powered replications	**19 succeeded (about 65%)** — better than 2015, but different claims tested in different ways.

Cancer studies help determine which treatments deserve more research. This is not an argument about whether people unconsciously prefer the color blue on Tuesdays. And when a headline announces that scientists found "the brain region for procrastination" after scanning 23 sophomores, perhaps continue procrastinating before sharing it.

There has also been real progress. Researchers preregister hypotheses and analysis plans, share data and code, conduct large multi-lab studies, and use **Registered Reports**, in which journals judge the question and method before knowing whether the result is exciting. More than 300 journals had adopted Registered Reports by 2024 (Nature Neuroscience 2024). A large preregistered national study of growth-mindset training, for example, found a small benefit for lower-achieving students and showed that the effect depended on the school context (Yeager et al. 2019). That is a much more useful conclusion than either "growth mindset fixes education" or "growth mindset is fake."

The lesson is not that science is broken beyond repair. The lesson is that science can repair itself.

1.3 Do Not Just Apply Statistics Like a Monkey Hoping to Type Hamlet Perfectly by Chance

You could memorize a decision tree:

> Two groups? Push the t-test button. Three groups? Push the ANOVA button. More variables? Panic, then push buttons until Reviewer 2 stops rejecting your paper.

That approach can generate output. It cannot tell you whether the output answers your question.

To use statistics responsibly, you need to understand **why** a method works, what assumptions it makes, what uncertainty it leaves behind, and what your design allows you to claim. A perfectly executed analysis of a badly measured variable is still a beautifully formatted mistake.

1.4 Paradigms and the Stories We Tell About Causes

Thomas Kuhn used **paradigm** to describe the shared assumptions that tell scientists what counts as a sensible question, acceptable evidence, and a reasonable explanation (Kuhn 1962). Most of the time, scientists work inside a paradigm without discussing it. Fish probably spend very little time defining water. **Also, did you know there are no such things as fish?** (Miller 2020)

Psychology usually relies on **one-way causal logic**: $X \longrightarrow Y$

Sleep affects memory. Stress affects grades. A treatment affects symptoms. This does **not** mean every relationship must be a straight line. It means our explanation has a direction: X does something to Y.

But human behavior also contains feedback. Stress disrupts sleep, poor sleep increases stress, both encourage caffeine, and caffeine returns at 2:00 a.m. to finish the job. In a dynamical-systems perspective, causes can be circular, reciprocal, and dependent on what the system was doing a moment ago. Asking "Which came first?" can be a stupid question when each variable keeps causing the other one.

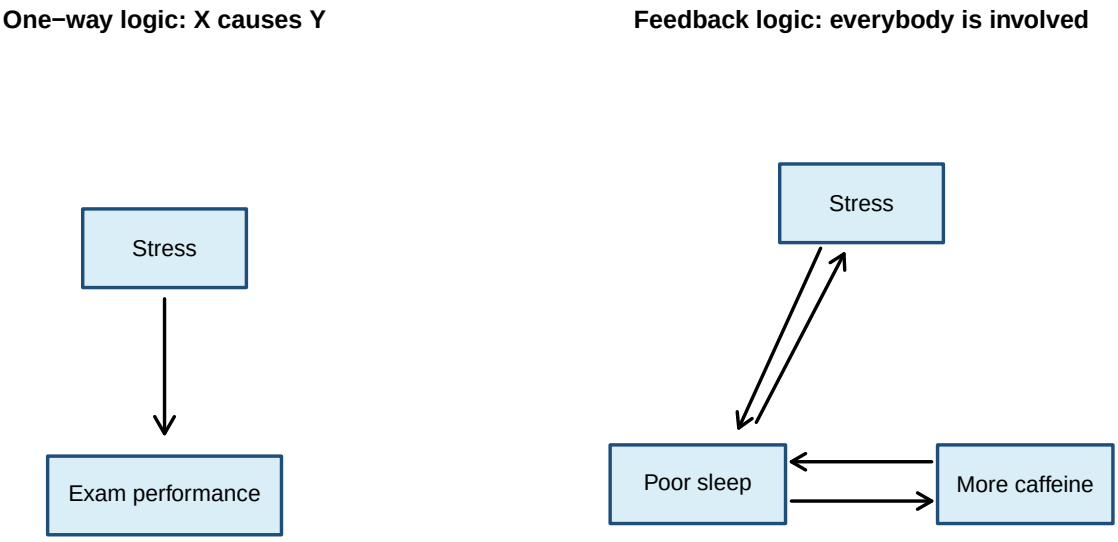


Figure 1.3: One-way causal logic gives us a direction. Feedback logic gives us a small committee of variables making one another worse.

I study systems like the second picture. Most statistics in this book will use the cleaner X-predicts-Y structure because that is how classical statistical models and most psychological research questions are organized. The dynamical-systems perspective does not make those statistics useless. It changes the questions I ask, the boundaries I place around an answer, and how suspicious I become when someone claims to have found the one true cause of a human behavior.

1.5 Measurement: Numbers Are Not Magic

Before analyzing a variable, we must decide what our numbers mean.

Stevens defined measurement broadly as assigning numbers to objects or events according to rules. He organized measurements into four scales (Stevens 1946):

Scale	What the numbers tell us	Psychology example	A better example
Nominal	Different categories; no order	Therapy condition: CBT, exposure, or waitlist	Volcano A, B, or C into which an undergraduate is dropped
Ordinal	Categories have an order; gaps between categories may be unequal	Mild, moderate, or severe symptoms	How much the student regrets volunteering for "easy" extra credit
Interval	Equal differences are meaningful; zero is arbitrary	Temperature in the overheated laboratory	Years on a calendar counting down to when Reviewer 2 retires
Ratio	Equal differences plus a meaningful zero	Heart rate after a student realizes the exam is today and they forgot to study	Number of chainsaw-carrying research assistants chasing the participant: ideally zero

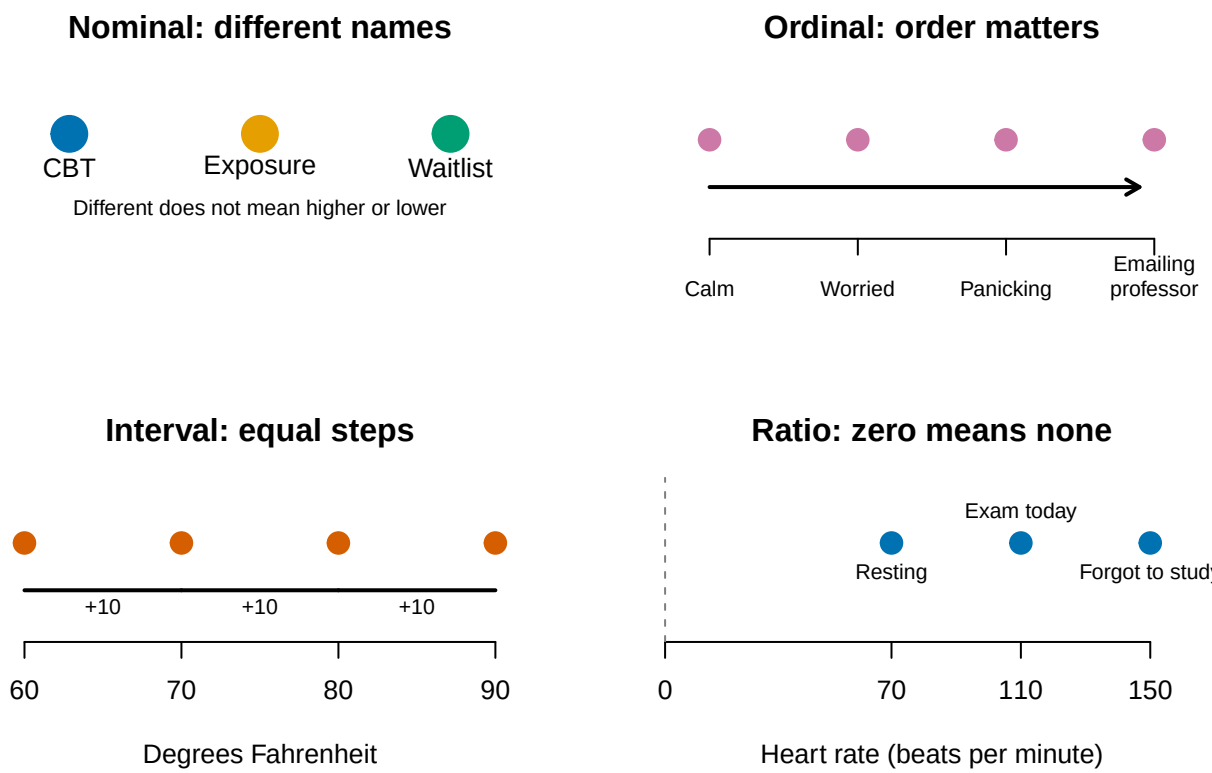


Figure 1.4: Four scales of measurement. The pictures are simple; the assumptions hiding underneath them are not.

That classification is useful, but Joel Michell points out a rather large problem: assigning a number does not prove that the attribute itself is quantitative (Michell 1997). I can assign you a student ID number. That does not mean a student with ID 800000 is twice as much student as someone with ID 400000.

Advanced: Do Our Numbers Even Measure Anything? (Michell's Complaint)

Michell distinguishes two jobs. The **instrumental task** creates a procedure that produces numbers. The **scientific task** asks whether the attribute actually has the structure those numbers assume. Psychologists are often excellent at the first job and quietly assume the second one has been handled by someone else. Possibly an undergrad. Probably for extra credit.

This matters because psychological constructs are abstractions. Anxiety is not sitting inside the skull like a kidney waiting to be weighed. We infer it from behavior, self-report, physiology, and context. Those measurements may be useful without being perfect, but their meaning depends on the theory and the measurement process. Numbers do not rescue a vague construct. Sometimes they merely give the vagueness decimal places.

The Institutional Review Board has entered the chat

Undergraduates may volunteer for ordinary research or earn reasonable extra credit. We cannot dissect their brains, drop them into volcanoes to appease Dorkaos (Ancient Greek God of Statistics), or chase them with chainsaws to increase the relationship between fear and exercise. "The effect size would be enormous" is not an ethical argument, or so I am told.

1.6 Research Design: The Extremely Short Version

Here are the terms the rest of the book will use constantly. Do not memorize this table. Bookmark it. You will be back, and the forgetting is normal—I planned for it.

Term	Meaning
Theory	An organized explanation of how or why variables are related.
Hypothesis	A specific, testable prediction derived from a theory.
Operational definition	The exact procedure used to manipulate or measure a construct.
Independent variable (IV)	The proposed cause or predictor. In an experiment, the researcher manipulates it.
Dependent variable (DV)	The measured outcome we are trying to explain or predict.
Population	Everyone or everything we want our conclusion to describe.
Sample	The smaller group we actually observe because surveying all humans is expensive and unpleasant.
Parameter	A numerical description of the population, usually unknown.
Statistic	A numerical description calculated from the sample, which we use to guess the parameter.
Random sampling	Selecting people so members of the population have a known chance of entering the sample. This helps generalization.
Random assignment	Assigning sampled participants to conditions by chance. This helps causal inference. It is not the same as random sampling.
Experimental design	The researcher manipulates an IV and uses control plus random assignment to test a causal claim.
Correlational design	Variables are observed as they naturally occur. We can estimate relationships, but causal direction remains unresolved.
Confound	An uncontrolled alternative explanation that changes systematically with the IV.
Internal validity	How confidently we can say the IV, rather than a confound, caused the DV.
External validity	How confidently the result can generalize beyond this study to other people, settings, situations, or times.

The basic research sequence is not a list with an end. It is a loop, and the last step feeds the first one.

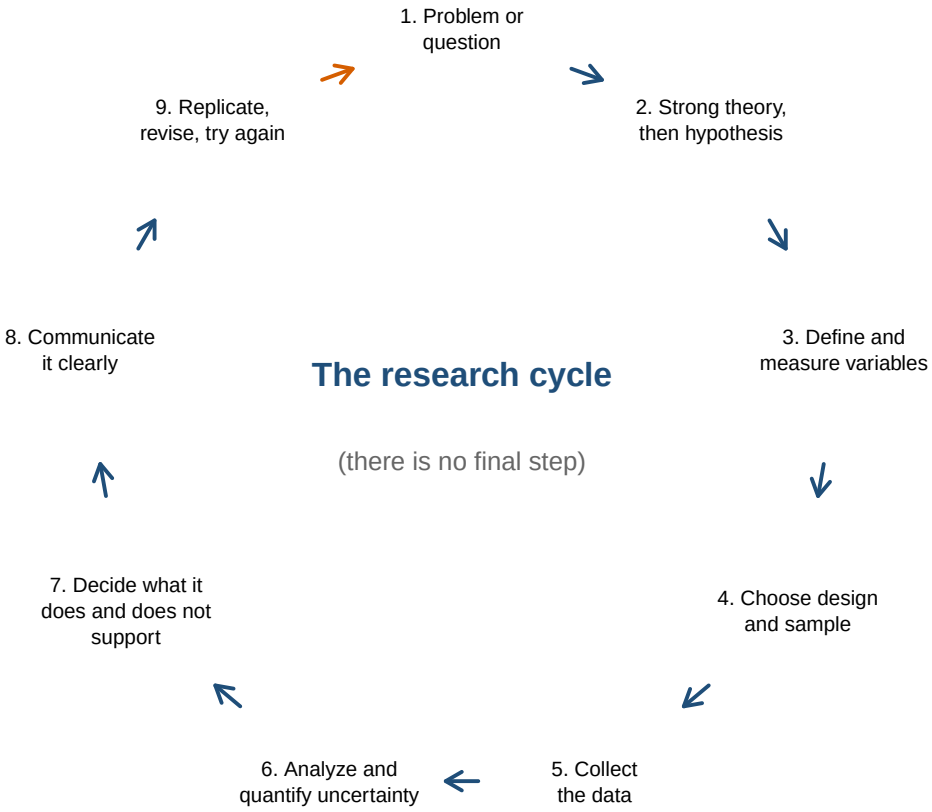


Figure 1.5: The research cycle. Step 9 is an arrow back to step 1, which is why science is a repair process rather than a finish line.

If the hypothesis fails, the theory may be wrong. The operational definition may be bad. The manipulation may have failed. The sample may be weird. Or the data may simply have had a day. Statistics cannot automatically tell us which explanation is correct. It gives us disciplined ways to narrow the possibilities.

A Warning about Soft Theories

Meehl argued that theories in "soft" psychology often do not win, lose, or improve. They become fashionable, collect a mixture of significant and nonsignificant results, acquire emergency auxiliary explanations, and eventually fade away when everyone gets bored (Meehl 1978). The theory is not killed. It is placed in an attic with several dissertations and a box of unidentified power cords.

His deeper complaint was that a significant result is usually a weak test of a theory. Predicting "the groups will differ" is easy because groups almost always differ a little. A stronger theory predicts **how much**, **in which direction**, **under what conditions**, and **what result would make us abandon or revise it**. Good science takes theoretical risks. Collecting asterisks is not the same as understanding behavior.

Graduate students: read the actual Meehl paper (Meehl 1978). It is nearly fifty years old and still describes your literature review with uncomfortable precision.

That is why this book will not only show you **how** to run statistics. You will learn what the statistics are doing, why they behave that way, and how to notice when your analysis is answering a question no sensible person asked.

Statistics does not manufacture truth. It just makes your guesses defensible.

The important truth is you will always be wrong. Your job is to be less wrong.

1.7 How to Read This Book Without Reading All of It, Cause Yuck

This book serves two groups cause I am lazy and tired.

The main chapters are written for undergraduates with little or no statistics background. Read those chapters mostly in order, unless I change my mind—you can read my mind, right? Each chapter assumes you survived the chapters before it, although it may occasionally provide a short recap because forgetting is one of the brain's most important skills—look up the people who never forget; they are sad, sad people.

The book also contains material labeled **Advanced**. Those sections and chapters are intended for:

- Graduate students who need a compact statistics and regression reference.
- Advanced undergraduates who want to understand what is hiding in the closet.
- Researchers who remember learning the topic but cannot remember where, when, or why someone decided there should be different kinds of residuals.

Skip the advanced material unless your instructor assigns it, or unless curiosity has defeated your instinct for self-preservation. Skipping an advanced chapter will not prevent you from understanding the next undergraduate chapter. Graduate students should treat the advanced material as a rapid overview, not a replacement for a full course or a detailed reference such as Cohen (Cohen et al. 2003).

A Sensible Way to Use Each Chapter

1. Read the explanation and inspect the figures before running code.
2. Run the R code yourself. Looking at code is not the same as discovering that you replaced a comma with a semicolon and angered R.
3. Change one value and predict what will happen before rerunning it.
4. Use the callout boxes to find warnings, common mistakes, and the short version of an argument.
5. Return to the mathematics after the example makes sense. Symbols are much friendlier once you know what they are trying to describe.

> **Advanced Does Not Mean Optional Assumptions**
>
> An advanced model does not rescue a weak design, a bad measure, or a causal claim assembled from wishful thinking. More complicated software merely allows you to be wrong with additional parameters.

1.8 Why Almost All the Data in This Book Is Fake

Nearly every dataset you are about to meet is simulated. I made it up with code, in front of you, on purpose.

That will look advanced at first, and probably confusing. Stay with it, because it is the whole point. When you simulate data you are not reading about statistics, you are *building* the thing the statistic describes. You can change one number, run it again, and watch what happens. You can construct a world where the effect is real and see how often the test misses it. You cannot do that with a table of somebody else's numbers, and you will never learn as much from one.

So do not be scared of the code. It will feel like a lot early on. That is not a sign you are behind, it is a sign you are at the beginning.

Here is the good news about learning statistics: **the effort is front-loaded.** It is hardest at the start and it gets easier, which is the opposite of what most people expect. Once you understand the guts of what is happening, every new technique turns out to be one more step bolted onto something you already know. Regression is not a new subject after t-tests, it is the same subject wearing a bigger coat.

Skip the guts and the opposite happens. Every new method arrives as an unrelated set of buttons to memorize, and it gets harder forever. That is like being handed *Crime and Punishment* in Russian after being taught only to sound out the letters. You can pronounce every word on the page and understand none of them.

Learn the guts. It costs more now and less every week after.

> **You Can Download This Book and Run It**
>
> Every chapter in this book is a `.qmd` file, which is a plain text file containing all the prose you are reading plus every line of R code that produced the numbers and figures around it. You can download them and open them in RStudio, and then run the chapter line by line, one chunk at a time, and watch each result appear.
>
> Do that. Especially with the simulations. The whole argument of the section above is that you learn more from data you built than from data you were handed, and that only works if you actually run it and start changing numbers to see what breaks.
>
> One thing that will confuse you if nobody warns you: **some of the code in the files does not appear in the book.** That is deliberate. If a chunk is fifteen lines of fiddling with axis labels and legend positions, you do not need to learn it, you just need to see the plot it made, so I hid it. Everything you are actually meant to be able to write yourself is shown. If you open the `.qmd` and find extra code that was never printed in the chapter, it is scaffolding, not a secret.

1.9 The Short Story

- Statistics is not decorative math bolted onto the end of a study. It is the logic connecting the design, the measurement, the analysis, and the claim you are allowed to make.
- The replication crisis is what happens when that logic is ignored at scale. You cannot fix the whole system this semester. You can stop adding to it.
- Numbers mean only what the measurement lets them mean. Assigning a number to an attribute does not prove the attribute is quantitative; sometimes it merely gives the vagueness decimal places.

- Design decides what you are permitted to claim. Random sampling helps you generalize, random assignment helps you argue about causes, and a confound undoes both without asking your permission.
- The vocabulary table in this chapter is the grammar of every chapter that follows. Bookmark it rather than memorizing it. You will be back.
- Next, we teach R to do the arithmetic. R is fast, tireless, and completely unable to notice when the question was not worth asking. That part stays your job—along with being less wrong.

2 R: The Extremely Short Survival Guide

2.1 Why R?

R is a free, open-source programming language built for statistics, data analysis, and graphics. It was developed by people who enjoy statistics enough to create an entire language for it. We should probably be grateful and slightly concerned.

Why are we using it?

- **It is free.** You do not lose access when you graduate, change jobs, or cause the university licensing office to hunt you down.
- **It is powerful.** R can run the analyses in this book, make publication-quality figures, simulate entire populations, and automate repetitive work.
- **It is reproducible.** Your script records what you did. Six months later, you can rerun the analysis instead of trying to remember which buttons you clicked.
- **It is extensible.** Researchers build packages that add new methods. This is wonderful, although packages sometimes change, break, or disappear into the ether.
- **It is used by actual researchers.** Learning R gives you a skill that goes beyond your class.

R changes more frequently than a pirate changes their underwear. Packages change too. This is the price of living in a flexible, community-driven ecosystem. When old code breaks, it does not mean you are stupid. It means you have joined the rest of us in the never-ending struggle to be hip with what the kids are doing today—like skibidi or whatever nonsense you all say now.

2.2 R Is Not RStudio

R is the language and the statistical engine. **RStudio** is an integrated development environment, or IDE, that gives R a useful place to live. R does the calculations; RStudio provides the script editor, console, plots, files, help, and many little buttons you will ignore until the day they save you.

Think of R as a temperamental but Michelin-starred chef. RStudio is their kitchen. Installing only RStudio is like building a professional kitchen and forgetting the chef.

Here is the kitchen. You will spend a semester in it, so it is worth thirty seconds of looking.

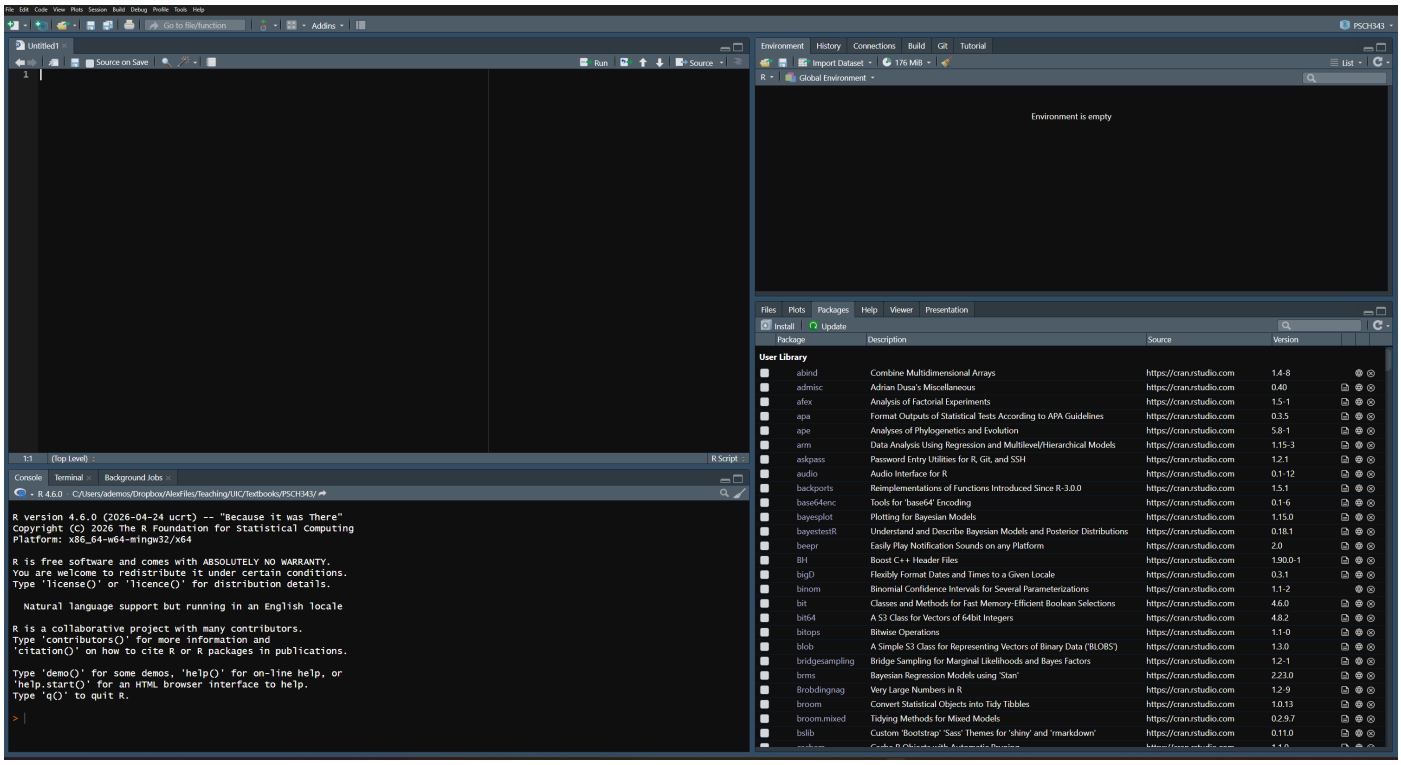


Figure 2.1: The four panes. **Top left** is the script: the work you keep. **Bottom left** is the Console, where R talks back. **Top right** is the Environment, which lists every object you have made—empty here, because nothing has happened yet. **Bottom right** holds Files, Plots, Packages, and Help.

You will ignore most of the buttons. That is fine. The four panes are the whole map. The name in the far top-right corner is the Project that is open, which will matter in a moment.

2.3 Install Them in This Order

1. Download and install the current version of **R** from CRAN.
2. Download and install the free **RStudio Desktop** from Posit.
3. Open RStudio. In the Console, run `R.version.string`. If R replies with a version, the creature is alive.

Windows users can usually keep multiple versions of R. On any operating system, install a version that matches your computer. For example, Apple Silicon Macs use the ARM64 build. The download pages provide the current choices, which is safer than this textbook pretending the year will remain 2026 forever.

Give yourself time to install software. Starting 30 minutes before class is not "giving yourself time." That is creating a hair-on-fire emergency and assigning it to Future You. *Don't burn all your hair off, or you will end up a bald ogre like me.*

If the install fails, or your laptop is old enough to remember the Obama administration, use Posit Cloud instead. It runs RStudio in a browser, the free tier might be enough for this course, and you can install locally later when you have an afternoon and a better mood.

Imprecations to the Statistics Gods

If RStudio behaves strangely after an update, close it completely and open it again. This solves most problems. Sometimes it fails for you, you bring the computer to me, and it works in my mere presence. The explanation is simple: I am a very sophisticated android. If you do not have access to me, find someone you suspect is also a sophisticated android.

2.4 The Console Is for Experiments; Scripts Are for Work

You can type directly into the **Console**, but console commands are easy to lose. Put work you care about in an R script: a plain-text file ending in `.R`.

In RStudio, choose **File** > **New File** > **R Script**. Type the following into the script:

```
# This is a comment. R ignores it. Future You should not.
2 + 2
```

```
[1] 4
```

Run a line with **Ctrl** + **Enter** on Windows or **Command** + **Enter** on a Mac. You may also highlight several lines and run them together.

Comments begin with `#`. Write comments that explain *why* the code exists. `# calculate the mean` above `mean(scores)` is not a revelation. `# exclude the practice trial before calculating each person's mean` may prevent a future disaster.

For this book, make an **RStudio Project** for your work. Projects keep scripts, data, and output together and reduce the need for `setwd()`, which is the command people use to make code work only on their own computer.

2.5 PEMDAS

I regret to report that middle-school mathematics has returned!

R follows the usual order of operations:

1. **P**arentheses: `()`
2. **E**xponents: `^`
3. **M**ultiplication and **D**ivision: `*` and `/`
4. **A**ddition and **S**ubtraction: `+` and `-`

Operations at the same level are generally handled in their defined order. The humane solution is to use parentheses whenever a tired person might have to stare at the expression.

```
2 + 3 * 4
```

```
[1] 14
```

```
(2 + 3) * 4
```

```
[1] 20
```

```
10 / 2 * 5
```

```
[1] 25
```

```
10 / (2 * 5)
```

```
[1] 1
```

```
(5 + 3)^2 / 4
```

```
[1] 16
```

```
-2^2      # surprise: the exponent goes first, so this is -(2^2)
```

```
[1] -4
```

```
(-2)^2    # write it this way if you meant "negative two, squared"
```

```
[1] 4
```

Those expressions are not equivalent. Parentheses are cheap.

That last example matters sooner than you think. In a few chapters we start squaring how far each person sits from the mean, and half of those distances are negative. Write the parentheses. Asking your instructor or TA to hunt down that bug will make them question their life choices.

Suppose a stress score is 10, and our terrible theory predicts performance using

$$\widehat{Y} = 80 - 2(\text{Stress}).$$

In R:

```
stress <- 10
predicted_performance <- 80 - 2 * stress
predicted_performance
```

```
[1] 60
```

R does not understand implied multiplication. Write `2 * stress`, not `2stress`. R is powerful, not psychic.

2.6 Objects: Put a Name on the Thing

The assignment arrow `<-` stores a value inside an **object**.

```
coffee_cups <- 4
hours_of_sleep <- 3
regret <- coffee_cups^2 / hours_of_sleep
regret
```

```
[1] 5.333333
```

Read `<-` as "gets." `coffee_cups <- 4` means *coffee_ cups gets 4*. You may see `=` used for assignment. It often works, but this book will generally use `<-` because it makes assignment easier to recognize.

Object names cannot contain spaces. Use names such as `exam_score`, `ExamScore`, or `exam.score`. Pick a system and stick to it.

2.7 Vectors and Functions

A **vector** stores several values of the same basic type. The function `c()` combines values into a vector:

```
anxiety <- c(4, 7, 6, 9, 3)
anxiety
```

```
[1] 4 7 6 9 3
```

```
mean(anxiety)
```

```
[1] 5.8
```

```
sd(anxiety)
```

```
[1] 2.387467
```

```
length(anxiety)
```

```
[1] 5
```

A **function** performs a job. Its arguments go inside parentheses. In `mean(anxiety)`, the function is `mean()` and the object we hand it is `anxiety`.

Arguments can change what a function does:

```
anxiety_with_missing <- c(4, 7, NA, 9, 3)

mean(anxiety_with_missing)
```

```
[1] NA
```

```r
mean(anxiety_with_missing, na.rm = TRUE)
```

```
[1] 5.75
```

`NA` means a value is missing. R refuses to quietly pretend it is not there. The argument `na.rm = TRUE` tells `mean()` to remove missing values for this calculation.

2.8 Data Frames: Where the Undergraduates Are Stored for Later Consumption

A **data frame** is a rectangular data set. Rows usually represent cases or observations; columns represent variables.

```r
study <- data.frame(
  student = c("A", "B", "C", "D"),
  therapy = c("CBT", "CBT", "Control", "Control"),
  anxiety = c(4, 7, 8, 6)
)

study
```

```
  student therapy anxiety
1       A     CBT       4
2       B     CBT       7
3       C Control       8
4       D Control       6
```

```r
summary(study)
```

```
      student       therapy     anxiety
 Length   :4   Length   :4   Min.   :4.00
 N.unique :4   N.unique :2   1st Qu.:5.50
 N.blank  :0   N.blank  :0   Median :6.50
 Min.nchar:1   Min.nchar:3   Mean   :6.25
 Max.nchar:1   Max.nchar:7   3rd Qu.:7.25
                             Max.   :8.00
```

In a real study, those rows represent participants who consented to research. They are not a convenient supply of organs, volcano offerings, or targets for a chainsaw-based manipulation of fear. The Institutional Review Board remains stubborn about this.

2.9 Getting Data Into R

Typing a data frame by hand was fine for four students. Real data arrives as a spreadsheet, usually a **CSV** file—comma-separated values, which is a spreadsheet with the formatting boiled off.

Reading one in takes a single line:

```
study <- read.csv("anxiety_study.csv")
```

R looks for that file in your **working directory**. This is why we made a Project: open the Project first, keep the CSV in the Project folder, and R already knows where to look. Skip the Project and you end up typing paths like `C:/Users/you/Desktop/stuff/final_FINAL/data.csv`, which works perfectly on exactly one computer in the world.

2.9.1 Look at What Arrived

Never trust a file you have not looked at.

```
head(study)
```

```
  student therapy anxiety
1       A     CBT       4
2       B     CBT       7
3       C Control       8
4       D Control       6
```

```
str(study)
```

```
'data.frame':   4 obs. of  3 variables:
 $ student: chr  "A" "B" "C" "D"
 $ therapy: chr  "CBT" "CBT" "Control" "Control"
 $ anxiety: num  4 7 8 6
```

`head()` prints the first few rows. `str()` prints the structure: how many rows, how many columns, and what R thinks each column *is*. `num` means numbers. `chr` means text.

2.9.2 Tell R Which Columns Are Categories

R read `therapy` as plain text. It does not know those words are conditions. A **factor** is R's word for a categorical variable—a fixed set of groups.

```
study$therapy <- factor(study$therapy)
str(study)
```

```
'data.frame':   4 obs. of  3 variables:
 $ student: chr  "A" "B" "C" "D"
 $ therapy: Factor w/ 2 levels "CBT","Control": 1 1 2 2
 $ anxiety: num  4 7 8 6
```

The `$` pulls one column out of a data frame. Now `therapy` is a factor with two levels instead of loose text.

Do this every time. When you run a *t*-test or a regression later, R has to know that CBT and Control are two groups being compared, not two words it happened to find lying around.

2.9.3 Count Them

```
table(study$therapy)
```

```
    CBT Control
      2       2
```

`table()` counts how many cases fall into each category. Run it on every categorical variable before you analyze anything. It is the fastest way to discover that you have 47 people in one condition and 3 in the other, or that somebody typed "Contol."

2.10 Random Numbers and the Magic Word `set.seed()`

This book fakes a lot of data. Almost every chapter simulates a study rather than making you go collect one, and simulated data starts with random numbers.

R can produce them on demand:

```
runif(5)   # five random numbers between 0 and 1
```

```
[1] 0.4753604 0.4805481 0.2836487 0.5277388 0.6324330
```

Run that line again and you get five different numbers. Run it tomorrow, different again. That is the point of random.

It is also a problem for a textbook. If my numbers change every time the book is built, nothing I write about them stays true, and worse, you would run my code and get a different answer than the one printed on the page, and reasonably conclude you had broken something.

So we cheat, on purpose:

```
set.seed(42)
runif(5)
```

```
[1] 0.9148060 0.9370754 0.2861395 0.8304476 0.6417455
```

```
set.seed(42)
runif(5)   # the exact same five numbers
```

```
[1] 0.9148060 0.9370754 0.2861395 0.8304476 0.6417455
```

`set.seed()` fixes R's starting point for generating randomness. Same seed, same sequence, every time, on every computer. The number inside is arbitrary and means nothing. People use 42, 123, their birthday, or in my case whatever I typed without thinking.

What set.seed() Is and Is Not Doing

It is **not** making the numbers less random. The sequence is just as unpredictable as before. You have only told R which unpredictable sequence to use.

Two habits worth forming now:

- When you see `set.seed()` in this book, it is there so your output matches mine. If your numbers differ from the page, check that you ran the seed line too.
- In real analysis, set a seed any time chance is involved: random assignment, bootstrapping, cross-validation, simulations. It is the difference between "I got $p = .04$" and "I got $p = .04$ and here is how you get it too." Future You will need that, and so will Reviewer 2.

2.11 Packages: Install Once, Load Each Session

Packages add functions to R. Install a package once:

```
install.packages("tidyverse")
```

Load it each time you start a new R session and need it:

```
library(tidyverse)
```

The quotation marks belong in `install.packages("tidyverse")`, but not in `library(tidyverse)`. This distinction exists because the Probability Gods were briefly allowed to help design software.

2.12 The Pipe: |>

You will start seeing this symbol in a couple of chapters, so meet it now. The **pipe** `|>` takes the thing on its left and hands it to the function on its right as the first argument. Read it out loud as the word **"then."**

```
study |>
  head(2)
```

```
  student therapy anxiety
1       A     CBT       4
2       B     CBT       7
```

That says "take `study`, *then* show me the first two rows," and it does exactly what `head(study, 2)` does. With one step the pipe buys you nothing. With five steps stacked up it is the difference between code you can read top to bottom and code buried inside nested parentheses, which is why the tidyverse chapter at the end of the book runs on it.

One warning, because you will meet it on Stack Overflow within a week. Older R code uses `%>%` instead, a pipe that came from the `magrittr` package. For everyday work the two mean the same thing, so when you find `%>%` in somebody's code you can read it as `|>` and carry on. This book uses `|>` throughout. The difference is that `|>` is built into R itself while `%>%` needs a package loaded first, which is a good reason to prefer the one that always works.

2.13 Asking R for Help

R ships with a manual for every function. Put a `?` in front of the name:

```
?mean
```

The help page opens in the Help pane. Most of it is written for people who already know the answer, so do not read it top to bottom. Read two parts:

- **Arguments** tells you what you can hand the function and what each thing does. This is where you would have found `na.rm`.
- **Examples**, at the very bottom, is code you can copy and run right now. It is the only part of the page guaranteed to work.

2.14 Errors Are Information, Delivered Rudely

An **error** means R stopped. A **warning** means R finished but wants you to know something suspicious happened. A **message** is usually informational. Read the first error carefully, because the next 47 errors may merely be its panicked descendants.

When code fails:

1. Read the error.
2. Check spelling, capitalization, commas, quotation marks, and parentheses.
3. Run the code in smaller pieces.
4. Confirm that the objects and packages it needs exist.
5. Restart R and try again.
6. Ask for help and show the code **plus the complete error**. "It hates me" is emotionally valid but diagnostically weak.

That is enough R to begin. You do not need to memorize the language before doing statistics. You need to keep your work in scripts, run small pieces, inspect the result, and understand what you asked R to do.

2.15 The Short Story

- Keep your work in a script. The Console is for things you do not mind losing.
- `<-` makes an object. `c()` makes a vector out of several values.
- Functions take arguments, and the arguments change the answer: `mean(x, na.rm = TRUE)`.
- `read.csv()` brings data in. `str()` and `head()` tell you what actually showed up.
- Make your categorical variables factors, then `table()` them before you trust anything.
- Install a package once. Load it with `library()` every session, forever.
- When it breaks: read the *first* error, check your commas and capitals, then `?` the function.

R will calculate nonsense with breathtaking speed. The thinking remains your job.

3 Probability: The Gods Are Fickle

3.1 A Car, Two Goats, and a Terrible Decision

You are on a game show. There are three doors. Behind one is a car. Behind the other two are goats. *My guess is you want the car, but I don't know why. Goats have several advantages: a) they make funny sounds when they scream, b) some faint when frightened, and c) you can use one to cast a Greek curse on the Chicago Cubs so they never win another game!*

You choose Door 1. The host, Monty, knows where the car is. He opens one of the other doors and reveals a goat. He then offers you a choice: **stay** with Door 1 or **switch** to the remaining closed door.

Should you **stay**, **switch**, or **does it make no difference** what you do? **I think you should declare that goats are better than cars and ask for a goat!**

So what should you do?

> **What Most People Do**
>
> Most people stay. Some believe a deity secretly loves them above all others and guided them to the correct door. Switching would insult the deity. Others know they would be furious if they chose the correct door and then switched away from it. Finally, some people use their giant frontal cortex to conclude that two remaining doors must mean a 50/50 chance.
>
> All three groups stay. All three groups are wrong.

Well, people are really bad at probability, and it is not 50/50. *You should switch.* Staying with your first door wins about 1/3 of the time; switching wins about 2/3 of the time.

Why? Your original choice had a 1/3 chance of being correct and a 2/3 chance of being wrong. Monty did not open a random door. He used his knowledge to remove a goat from the two doors you did not choose. If your first choice was wrong—which happens 2/3 of the time—switching fixes it.

Do not believe me. Believing me is not the point of this class. Make R play the game ten thousand times and check:

```
set.seed(343)

play_monty <- function(switch_doors) {
  car <- sample(1:3, 1)          # where the car actually is
  pick <- sample(1:3, 1)         # your first guess

  if (switch_doors) {
    # Monty removes a goat door, so switching lands on the car
    # whenever your first guess was wrong.
    pick != car
  } else {
    pick == car
  }
```

```r
}

stay   <- replicate(10000, play_monty(switch_doors = FALSE))
switch <- replicate(10000, play_monty(switch_doors = TRUE))

c(stay = mean(stay), switch = mean(switch))
```

```
  stay switch
0.3382 0.6616
```

Two-thirds. R has no intuition to override, which is exactly why it got this right and we did not.

If the simulation still feels like a trick, draw the whole game out. There are only three ways it can start, because there are only three doors the car can be behind, and each is equally likely:

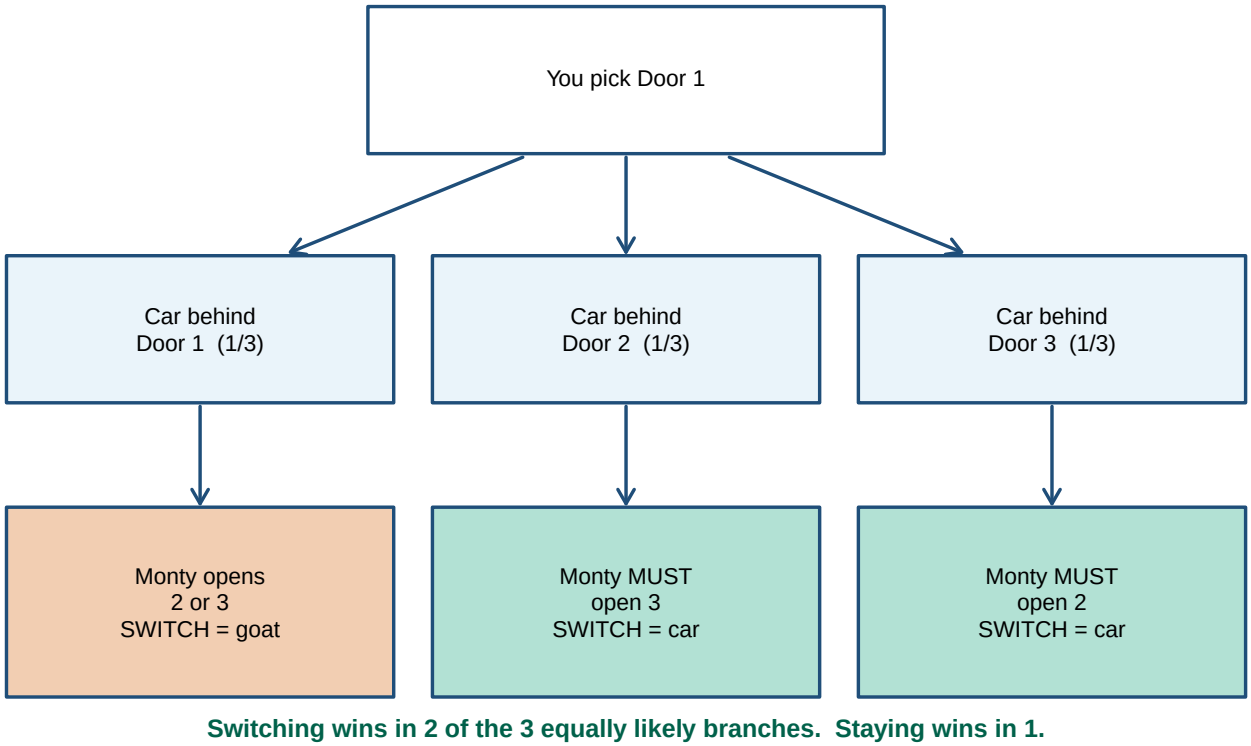


Figure 3.1: Every possible game, assuming you picked Door 1. Switching wins in two branches out of three.

The middle and right branches are where the whole thing turns. In both of them Monty has **no choice at all** about which door to open, because one door has the car and the other has your original pick. His hand is forced, and that forced move is what hands you the information. Only in the left branch, where you already had the car, does he get a free choice, and that is the one branch where switching costs you.

This answer depends on the rules: Monty knows where the car is, always opens a goat door, and always offers the switch. If the host sometimes opens the car, chooses doors randomly, or offers a switch only when he is bored, the probability changes. A probability depends on the rules that produce the outcomes. Change the rules, and you may change the probability. Keeping the old probability after quietly changing the rules is how nonsense enters wearing a lab coat.

The Monty Hall problem feels wrong because people often treat all visible options as equally likely. They are not. Two possible outcomes do not automatically mean 50/50 (Ang and Shahrill 2014). A hypothesis test can also end with two possible decisions. That does not mean each decision had a 50% probability.

3.2 What Probability Means

A probability is a number from 0 to 1 describing how likely an event is under stated assumptions.

- An impossible event has probability 0.
- A certain event has probability 1.
- $P(A) = .50$ means that across many comparable opportunities, event A should occur about half the time.

That last sentence is important. Probability does not promise that exactly 10 of your next 20 coin flips will be heads. It describes what happens over many repetitions. In a small sample, chance can behave like a raccoon inside a convenience store: energetic, destructive, and technically following the laws of nature.

The **sample space** is the set of possible outcomes. For one ordinary six-sided die:

$$S = \{1, 2, 3, 4, 5, 6\}.$$

If the die is fair, the probability of rolling a 2 is

$$P(2) = \frac{1}{6} = .167.$$

The word **fair** is an assumption. Statistics is full of assumptions. We will test some and defend others. Occasionally, we will discover that an assumption came from someone in 1974 who "felt" it was right. Everyone else cited them and put their fingers in their ears.

3.3 Three Rules That Do Most of the Work

3.3.1 The complement rule: "not A"

The probability that an event does not occur is

$$P(\text{not } A) = 1 - P(A).$$

Let event A mean a participant figures out what your study is really about. Suppose:

$$P(A) = .20$$

The probability that the participant does **not** notice them is:

$$P(\text{not } A) = 1 - .20 = .80$$

This example does not include a research assistant revving a chainsaw to increase the participant's heart rate. Everyone would notice that, so $P(\text{not } A) = 0$. More importantly, the ethics board already rejected it.

3.3.2 The addition rule: "A or B"

For events that cannot occur together,

$$P(A \text{ or } B) = P(A) + P(B).$$

On one die roll, the probability of a 1 or a 2 is $1/6 + 1/6 = 2/6$.

If events can overlap, subtract the overlap so it is not counted twice:

$$P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B).$$

3.3.3 The multiplication rule: "A and B"

If two events are **independent**, knowing one occurred tells us nothing about the probability of the other:

$$P(A \text{ and } B) = P(A)P(B).$$

The probability of getting heads twice with a fair coin is $.5 \times .5 = .25$.

Independent does not mean "different." It means one event does not change the probability of the other.

The next example uses a standard deck: 52 cards, 13 of them hearts. **With replacement** means you put the card back and reshuffle, so the deck resets and forgets you. **Without replacement** means the card stays out, so the deck remembers. That distinction returns much later, when we resample our own data with replacement and call it bootstrapping.

Drawing two hearts **with replacement** gives

$$\frac{13}{52} \times \frac{13}{52} = .0625.$$

Without replacement, the first card changes what remains:

$$\frac{13}{52} \times \frac{12}{51} = .0588.$$

The second probability is **conditional** on what happened first.

> ⚠ Mutually Exclusive Is Not the Same as Independent
>
> **Mutually exclusive** events cannot happen together. If one occurs, the other becomes impossible.
>
> **Independent** events can happen together, but one does not change the probability of the other.
>
> For one die roll, rolling a 2 and rolling a 5 are mutually exclusive. They are not independent. Once you roll a 2, the probability that the same roll was a 5 collapses to zero.
>
> These terms sound vaguely similar and are waiting for an opportunity to ruin your exam.

3.4 Conditional Probability: The Vertical Bar Is Not Decoration

$P(B \mid A)$ means "the probability of B given that A occurred."

The general multiplication rule is

$$P(A \text{ and } B) = P(A)P(B \mid A).$$

Conditional probabilities answer questions about a restricted group. For example:

- $P(\text{passes})$ asks about everyone.
- $P(\text{passes} \mid \text{attended review})$ asks only about students who attended the review.
- $P(\text{attended review} \mid \text{passes})$ asks something different.

Reversing the condition reverses the question. The probability that a person is tall given that they play basketball is not the probability that they play basketball given that they are tall. Likewise, a p-value will eventually describe how probable data this extreme—or more extreme—would be if H_0 and the model assumptions were true. It does not describe the probability that H_0 is true. Swapping those is one of psychology's favorite avoidable catastrophes.

! The Direction of the Bar Matters

$$P(B \mid A) \neq P(A \mid B)$$

The probability of playing basketball **given that someone is tall** is not the probability of being tall **given that someone plays basketball**.

Read everything after the vertical bar first. It identifies the group we are standing inside. Reversing the bar changes the group and usually changes the answer.

3.5 Chance Is Fickle, Not Obligated to Look Random

Imagine 20 coin flips. Humans usually invent sequences with roughly equal heads and tails, few long streaks, and tasteful amounts of alternation. Real coins have no taste. They produce clusters and streaks because **streaks are part of randomness**.

The chance of getting exactly 10 heads in 20 fair flips is only about 17.6%. Getting between 8 and 12 heads happens about 73.7% of the time. A result outside that range is not impossible. The Probability Gods did not violate a contract; you invented a contract they never signed.

i Three Ways Your Brain Gets This Wrong (They Have Names)

Gambler's fallacy. After five heads you feel a tail is "due." It is not. The coin has no memory, no sense of fairness, and no idea you are watching.

Representativeness. Real randomness looks streakier than the tidy sequences people invent. You just watched yourself expect the wrong thing.

Availability. Vivid events feel more probable than they are. Plane crashes make the news; car crashes make the traffic report. Guess which one kills more people.

```r
set.seed(242343)

flips <- 500
paths <- replicate(8, cumsum(rbinom(flips, 1, .5)) / seq_len(flips))

matplot(
  seq_len(flips), paths,
  type = "l", lty = 1, lwd = 1.7,
  col = grDevices::hcl.colors(8, "Dark 3"),
  ylim = c(0, 1),
  xlab = "Number of flips",
  ylab = "Running proportion of heads",
  main = "The Probability Gods eventually become less dramatic",
  las = 1
)
abline(h = .5, lty = 2, lwd = 2, col = "black")
```

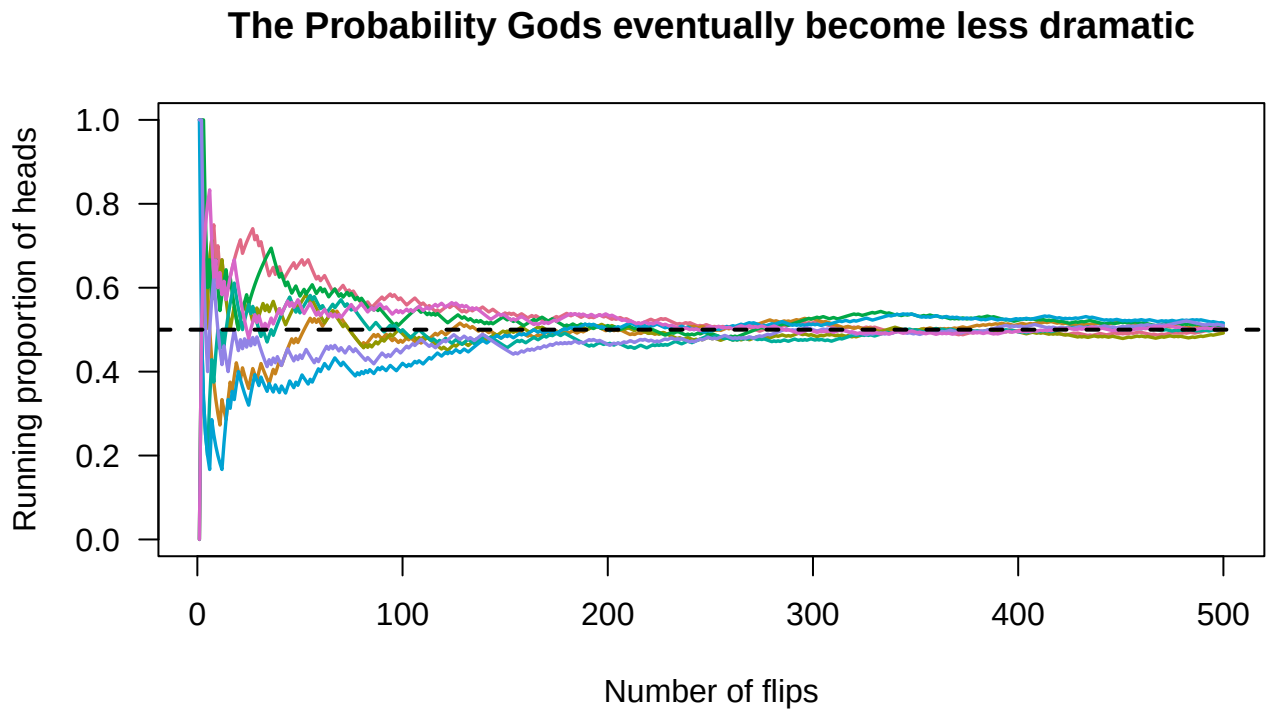


Figure 3.2: The running proportion of heads in eight fair-coin experiments. Early estimates stagger around; with more flips, they become more stable around .50.

Larger samples do not force the estimate to be correct. They make wildly inaccurate estimates less common. With 20 therapy clients, the Probability Gods might hand you 20 people who adore the treatment, or 20 people who hate your face. With 200 clients, finding enough unusually delighted or face-hating people to hijack the entire result becomes much harder.

This is the **law of large numbers**: as the number of independent observations increases, a sample average or proportion tends to settle near its expected value. It is not the law of small numbers. Twenty people do not become a population

because the grant ran out.

3.6 Expected Value Is a Long-Run Average, Not a Prophecy

The **expected value** is the probability-weighted average outcome over many repetitions:

$$E(X) = \sum xP(x).$$

Suppose a legitimate carnival game charges $2. You win $10 with probability .10 and $0 otherwise. Your expected winnings are

$$E(X) = \$10(.10) + \$0(.90) = \$1.$$

After paying $2, your expected net result is −$1 per game. You could still win $8 on the first play. Expected value does not predict one event; it describes the long-run average. The carnival can stay in business because it has patience and your money.

> **Expected Does Not Mean Most Likely**
>
> An expected value does not have to be an outcome that can actually occur.
>
> The expected number of children in a family might be 1.7. Nobody has to produce seven-tenths of a child. The value describes the long-run average across many families.
>
> Expected value is an average, not a prophecy or a declaration from the Probability Gods.

3.6.1 Numbers Bounce

Here is the picture to keep for the rest of the book, because we are going to need it constantly.

Drop a rubber ball on the sidewalk. You know roughly where it is going to end up. You are not surprised by where it lands, and you would not accept a bet that it will roll into traffic. But it bounces on the way down, and it almost never comes to rest on the exact spot you were aiming at. Sometimes it settles an inch away. Once in a while it catches an edge and ends up under a parked car.

Numbers that come out of chance behave the same way. They have a place they are aimed at, which is the expected value, and they have a bounce, which is everything else. The aim is predictable. The bounce is not.

Almost every worry in the rest of this book is one of two questions about that ball. **Where was it aimed?** That is estimation: means, proportions, effect sizes. **How far does it bounce?** That is the standard error, and it is the reason we cannot simply read a number off a sample and believe it.

So when a later chapter says a sample mean *bounces around*, this is the ball. It does not mean the number is wrong. It means the number is one landing out of many that could have happened, and that if you drop the ball again you should expect a slightly different answer.

3.7 Advanced Section: Bayes, Base Rates, and a Scary Positive Test

You test positive for a rare condition that will turn you into a goat. The condition affects 1% of people. The test correctly detects 95% of cases and has a 5% false-positive rate. What is the probability that you actually have the condition and turn into a goat?

The tempting answer is 95%. That answer has ignored almost everyone who does not have the condition.

Imagine testing 10,000 people. Assume:

- 1% actually have the condition.
- The test correctly detects 95% of people who have it.
- The test incorrectly gives a positive result to 5% of people who do not have it.

3.7.1 Step 1: Who Has the Condition?

$$10{,}000 \times .01 = 100$$

So 100 people have the condition. The other 9,900 do not.

3.7.2 Step 2: Test the 100 People Who Have It

The test correctly identifies 95% of them:

$$100 \times .95 = 95$$

Therefore:

- 95 receive a **true positive**.
- 5 receive a **false negative**.

3.7.3 Step 3: Test the 9,900 People Who Do Not Have It

The test incorrectly gives a positive result to 5% of them:

$$9{,}900 \times .05 = 495$$

Therefore:

- 495 receive a **false positive**.
- 9,405 receive a **true negative**.

3.7.4 Step 4: Count Everyone Who Tested Positive

The positive results include 95 true positives and 495 false positives:

$$95 + 495 = 590$$

A total of 590 people test positive. However, only 95 of them actually have the condition.

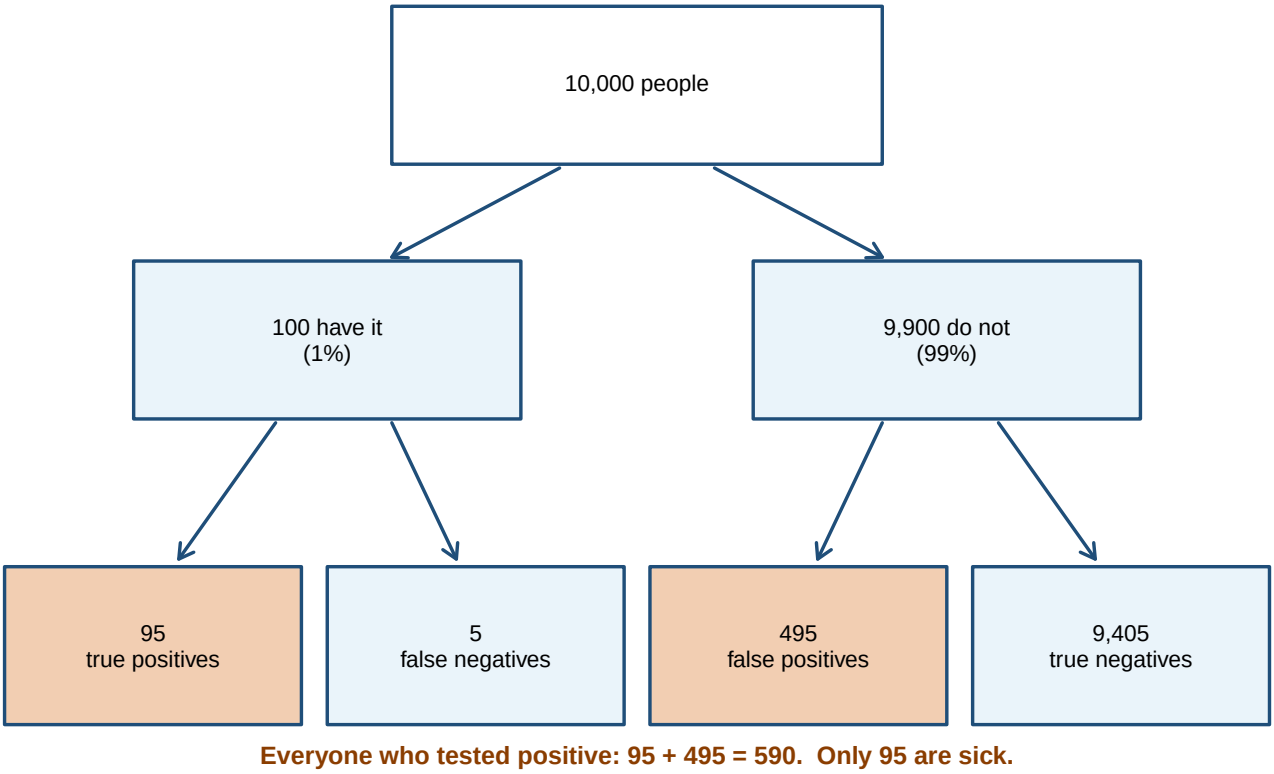


Figure 3.3: Ten thousand imaginary people, none of whom will email my department head. The two shaded boxes are everyone who tested positive. Most of them are fine.

3.7.5 Step 5: Find the Probability of Having the Condition After Testing Positive

$$P(\text{Condition} \mid \text{Positive}) = \frac{95}{590} = .161$$

Therefore, a person who tests positive has about a **16.1% probability** of actually having the condition.

The test is fairly accurate, but the condition is rare. That leaves the Probability Gods plenty of healthy people to terrify with false predictions that they will turn into goats tomorrow.

⚠ A 95% Detection Rate Is Not a 95% Diagnosis

The test detects 95% of people who have the condition:

$$P(\text{Positive} \mid \text{Condition}) = .95$$

Our actual question reverses the condition:

$$P(\text{Condition} \mid \text{Positive}) = .161$$

Those are different probabilities. Test performance, false-positive rates, and the base rate must all be considered before the test can accuse you of becoming a goat.

The **base rate** describes how common the condition is before we consider the new evidence. When the condition is rare, even a fairly accurate test can produce many false alarms. This is why probability questions are easier when we turn percentages into counts of actual imaginary people. The imaginary people are easier to organize and much less likely to contact my department head to complain about being turned into goats.

⚠ Now Let the Test Pass Sentence

Screening someone is one thing. Acting on the result is another.

Suppose we fired all the doctors and let the test decide, Judge Dredd style: the machine is judge, jury, and executioner, and there is no appeal. Of the 590 people it convicts, 495 are innocent. That is an 84% wrongful-conviction rate from a test that is 95% accurate.

The accuracy was never the problem. The base rate was.

Bayes' theorem formalizes the update:

$$P(H \mid E) = \frac{P(E \mid H)P(H)}{P(E \mid H)P(H) + P(E \mid \text{not } H)P(\text{not } H)}.$$

$P(H)$ is the **prior probability** or base rate. $P(E \mid H)$ tells us how expected the evidence is if the hypothesis is true. $P(H \mid E)$ is the **posterior probability** after combining the prior and evidence.

You do not need to become a Bayesian statistician today. You do need to remember that evidence does not arrive in an empty universe. Base rates matter, and $P(E \mid H)$ is not interchangeable with $P(H \mid E)$.

3.8 What This Has to Do with Statistics (Everything, Unfortunately)

Every study will produce a pattern. One group will have a higher mean. One correlation will be farther from zero. One brain image will contain a bright blob, possibly inside a dead salmon if you run enough tests (Bennett et al. 2010). That is a real study, and we will meet the salmon again in the multiple comparisons chapter.

Statistics asks whether the pattern is larger than we would reasonably expect from a specified chance process. To answer that question, we need to know:

- what outcomes were possible;
- what assumptions define the probability model;
- whether events are independent;

- what information we conditioned on;
- how much data we collected; and
- how often our entire analysis gave chance an opportunity to create something exciting.

3.9 The Short Story

- A probability is a number from 0 to 1, and it only means something under a stated set of rules. Change the rules and you change the answer.
- The three rules do most of the work: "not A" is $1 - P(A)$, "A or B" adds (minus the overlap), "A and B" multiplies when the events are independent.
- $P(B \mid A)$ is not $P(A \mid B)$. Read everything after the bar first—it tells you which group you are standing inside.
- Streaks are part of randomness, not evidence against it. Your brain will insist otherwise, and your brain has names for how it does this.
- Expected value is a long-run average. It does not predict your next outcome, and it does not have to be an outcome that can happen.
- When a condition is rare, even an accurate test produces mostly false alarms. Base rates decide how to read the evidence.
- Larger samples give chance fewer chances. That is the whole reason the rest of this book exists.

Probability is not a promise about what must happen next. It is a language for uncertainty. The Probability Gods remain fickle, but they are not completely lawless. Our job is to learn the laws well enough that we stop blaming divine intervention for bad design.

4 Distributions and their Moments

4.1 What is a distribution?

A **distribution** is a picture of how values spread out in a population:

- Where do most observations fall?
- How much do they vary?
- Is it symmetric, or does it have a long tail?

The average alone cannot answer these questions. Two groups can have the same mean while one forms a tidy hill and the other consists of people at both extremes glaring across an empty middle. A mean without its distribution is a witness who has omitted several important details.

4.1.1 A real distribution

Income in the US is a classic **positively-skewed** distribution: most households cluster around moderate incomes, with a few earning *very* large amounts.

This plot uses binned Census data (U.S. Census Bureau 2024), so the mean and median are approximations. Also: the \$200k+ bin is *everything* from \$200k to infinity—so if it's the tallest bar, that doesn't mean \$200k is the most common income. It just means rich people got lumped together.

That open-ended bin also means our mean below is too *low*. We estimate each bin by its midpoint, and "\$200,000 to infinity" has no midpoint, so those households get dropped. Census reports a higher mean than we compute here for exactly that reason. The richest households are the ones you cannot put a midpoint on, and they are precisely the ones dragging the real mean upward.

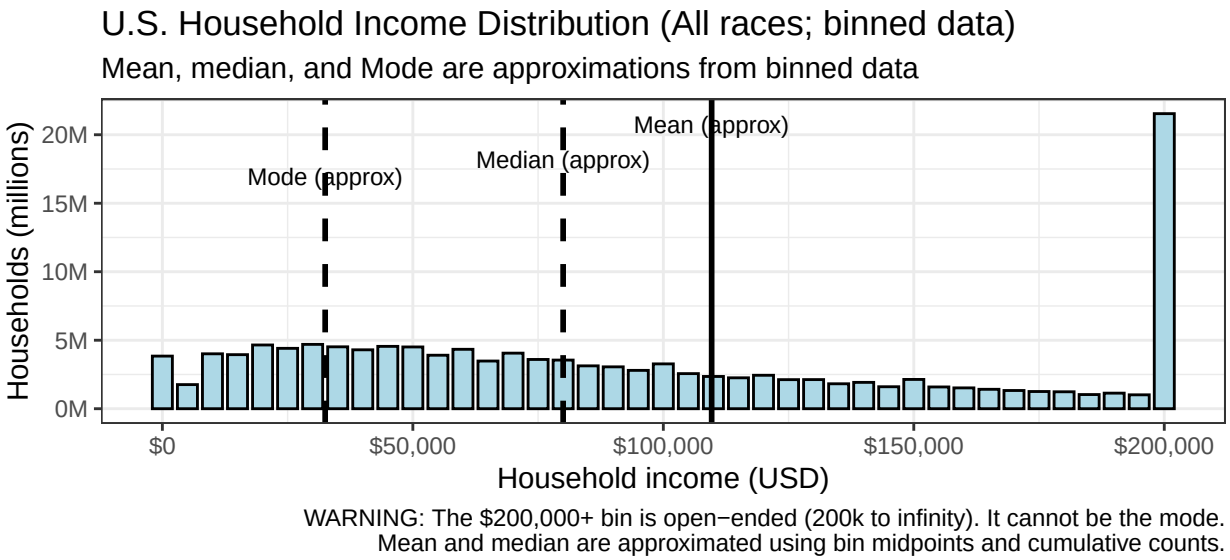


Figure 4.1: U.S. household income. The mean sits well to the right of the median, which is what skew does to an average.

Think about it:

1. Why is there a positive skew (the tail points right on the x-axis, toward the larger numbers)?
2. Which number—mean = \$110K, median = \$80K, or mode = \$32K—best represents the "average" American? Why?
3. Notice the three numbers are not close to each other. If a politician wanted to claim Americans are doing great, which one would they quote? If they wanted to claim the opposite?

4.1.2 A sidenote for the curious

Suppose a policy substantially reduced the highest incomes and moved that money into schools, housing, and infrastructure.

Predict: how would the income distribution shift? What would happen to the mean, median, and mode? Be explicit about why each one would (or would not) change. This is a distribution question, not a political one—the same reasoning works if you run the policy backward.

4.1.3 Distributions have meaning

Governments tend to use median income to decide who qualifies for financial aid, housing assistance, school lunch programs, tax credits, and healthcare subsidies.

When a distribution is highly skewed, these numbers tell very different stories. A rising *mean* can make a country look richer even when **most** households see no change—the median may stay flat. Understanding the shape helps us see who's actually benefiting from policy decisions.

The Mean Can Be Correct and Still Mislead You

In a strongly skewed distribution, a small number of enormous values can drag the mean upward.

That does not make the mean mathematically wrong. It may simply describe the US income inequality problem rather than the experience of a typical person.

Always inspect the distribution before allowing the mean to speak for everyone.

4.2 Gaussian (Normal) Distribution

Imagine a general cognitive measure influenced by many small factors: sleep, attention, schooling, mood, caffeine, random neural noise, and whether a bat-wielding researcher was standing behind the participant. When many small influences add together, scores can form a smooth, symmetric hill. That hill is the **normal distribution**.

The Gaussian distribution (named after Carl Friedrich Gauss, 1777–1855) is the foundation of classical parametric statistics because (*we pretend*) it describes a huge range of psychological phenomena.

Two parameters fully define it:

- μ (mean)—where the center is
- σ (standard deviation)—how spread out it is

A normal distribution has skewness of zero and excess kurtosis of zero. However, those two values alone do **not** prove that a distribution is normal; very strange distributions can share the same moments. The mean, variance, skewness, and kurtosis are commonly described through the distribution's **moments**. They summarize important features, but they are not a magical distribution-identification scanner.

Normality Is a Model, Not a Moral Achievement

"Normal" describes a mathematical distribution. It does not mean healthy, desirable, typical, or free from bat-wielding researchers.

A normal distribution has zero skewness and zero excess kurtosis, but matching those two values does not prove that your data are normal. Plot the distribution. Numbers are capable of lying by omission.

4.2.1 Simulating a Population in R

You Do Not Know These Words Yet, and That Is Fine

The code below asks for a mean and a standard deviation, and I have not taught you either one. On purpose.

Right now this is just a machine for manufacturing a million fake people so we have something to look at. By the end of this chapter you will know exactly what every term in it means, and you can come back and read it again with the smug satisfaction of someone who understands the joke the second time.

We'll use the normal distribution as our population. R makes this easy with `rnorm()`.

Let's create a population with:

- $\mu = 100$ points (cognitive score)
- $\sigma = 15$ points
- $N = 1,000,000$

```r
set.seed(42)  # so your million fake people are the same as my million fake people
n  <- 1e6     # 1e6 is scientific notation for 1,000,000
Mu <- 100     # population mean
S  <- 15      # population SD

# rnorm() = "random normal". Draw n scores from a normal distribution
# with the mean and SD we just specified.
Normal.Sample.1 <- rnorm(n, mean = Mu, sd = S)
```

4.2.2 Histogram (Base R)

```r
hist(Normal.Sample.1,
  breaks = 30,
  col    = "lightblue",
  border = "black",
  main   = "Raw Score Histogram",
  xlab   = "Population Distribution",
  ylab   = "Frequency")
abline(v = Mu, col = "black")
```

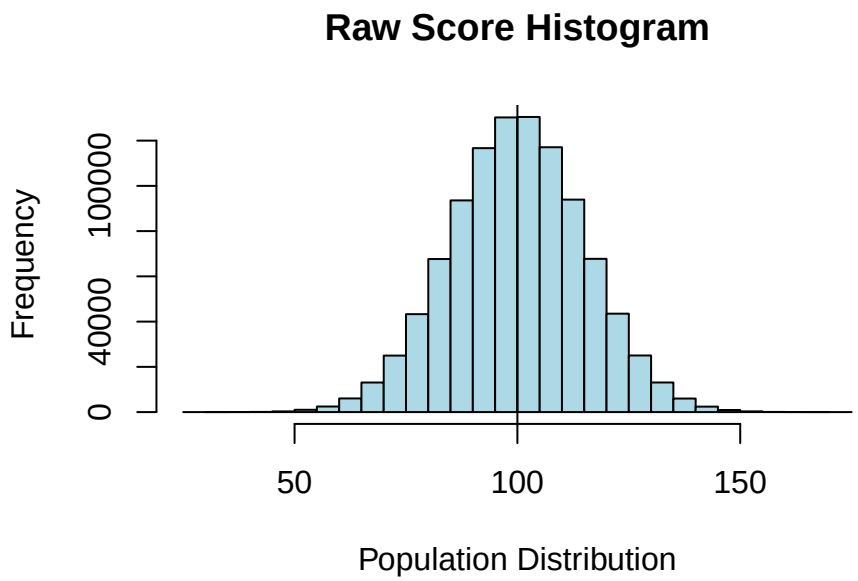


Figure 4.2: One million simulated cognitive scores. This is what we mean by a population: everybody, not a sample of them.

Looks normal. Now let's describe its shape using the 4 moments.

4.3 Moments

Four numbers capture the important features of a distribution: center, spread, symmetry, and peakiness.

> **i** Pedantic Fine Print for Graduate Students
>
> Calling all four of these "the moments" is teaching shorthand. Strictly, the mean is the first *raw* moment, the variance is the second *central* moment, and skewness and kurtosis are the third and fourth *standardized* moments. Nobody at a party cares.

4.3.1 Moment 1: Central Tendency

The center of a distribution can be described three ways:

Mean (the average):

- Standard measure of central tendency
- Heavily influenced by outliers
- Most stable estimate from sample to sample

Median (the middle number):

- Not sensitive to outliers
- Best measure when the distribution is skewed

Mode (most common number):

- Least stable from sample to sample

4.3.2 Mean

$$M = \frac{\Sigma X}{N}$$

Add up every score, divide by how many there are. You have done this since grade school.

> **!** Two Symbols for the Mean, and You Will Only Ever Have One of Them
>
> There is a mean of the **population**, written μ (the Greek letter *mu*, said "mew"), and a mean of your **sample**, written M.
>
> They are not two names for the same thing. μ is the real answer, the average of every person in the population, and **you will almost certainly never know it.** It exists, it is a fixed number, and it is none of your business. M is what you get when you grab a handful of people and average them. It is your estimate of μ, and it is the only one of the two you will ever hold in your hand.
>
> This chapter is a rare exception, because I built the population myself and can therefore cheat. I told R that $\mu = 100$. In a real study nobody hands you that number, which is exactly why the next several chapters exist.
>
> The same split is coming for spread: σ for the population, S for your sample. Same relationship, same problem, same resignation.

```
SumX <- sum(Normal.Sample.1)          # add up every score
N    <- length(Normal.Sample.1)       # count how many scores there are
M    <- SumX / N                      # divide: that is the mean
M
```

```
[1] 100.0086
```

```
# Or all at once:
M <- sum(Normal.Sample.1) / length(Normal.Sample.1)
```

The mean was 100.01—close to our target μ of 100, though notice it is not *exactly* 100 even with a million people. But just use the built-in:

```
mean(Normal.Sample.1)
```

```
[1] 100.0086
```

4.3.3 Median

```
median(Normal.Sample.1)
```

```
[1] 100.0208
```

Want to do it by hand? This is a good excuse to explain what a **function** actually is, because you have been using them since Chapter 2 without anyone defining the word.

A function is a machine you build once and then use forever. You hand it something, it does its work, it hands something back. `mean()` is a function: you hand it numbers, it hands back their average. So is `sort()`, and `length()`, and every other word in this book followed by parentheses.

You can build your own with `function()`. The thing in the parentheses is the **argument**, the placeholder for whatever gets handed in. Here is the median, built from scratch:

```
# Define a function called calculate_median that takes one input, x.
calculate_median <- function(x) {

  sorted_x <- sort(x)          # step 1: put the scores in order, smallest to largest
  n <- length(sorted_x)        # step 2: count how many there are

  if (n %% 2 == 1) {           # step 3: is n odd? (%% gives the remainder after dividing)

    sorted_x[(n + 1) / 2]      # odd, so there is one number dead in the middle. Take it.

  } else {

    # even, so there is no single middle. Take the two middle numbers
    # and average them.
    (sorted_x[n / 2] + sorted_x[(n / 2) + 1]) / 2
  }
}

# Now USE the machine we just built.
calculate_median(Normal.Sample.1)
```

```
[1] 100.0208
```

Two bits of syntax worth naming, because they show up constantly:

- `sorted_x[3]` means "the 3rd item in `sorted_x`." Square brackets pull items out by position.
- `%%` is the remainder operator. `7 %% 2` is 1, because 7 divided by 2 leaves 1 over. So `n %% 2 == 1` is R's way of asking "is n odd?"

Same answer as `median()`. Lots of code. This is why R has built-in functions!

4.3.4 Mode

No built-in mode function in R for continuous data—and it's almost never used for normal distributions. Note: `mode()` in R tells you the *type* of an object (numeric, factor, etc.), not the statistical mode.

4.3.5 Moment 2: Spread

Variance describes how spread out a distribution is. In psychology we mostly use **variance** and **standard deviation** because they work well with normal distributions and parametric statistics.

Variability comes from:

- Individual differences (genetics, mood, experience—what they ate for lunch that day)
- Research/environment factors (testing conditions, context—the researcher wearing a blood-soaked white coat)
- Measurement effects (fatigue, questions priming answers—don't think about a pink elephant)

When estimating variance from a sample, we divide by $N - 1$ instead of N to get an unbiased estimate. This is called **degrees of freedom**.

That sentence is true and completely unhelpful, so let us do better. There are two questions hiding in it, and students ask both of them every single year.

4.3.6 Why Square the Deviations?

Start with the obvious idea: to measure spread, average how far each score sits from the mean. Try it.

```
x <- c(2, 4, 4, 4, 5, 5, 7, 9)
x - mean(x)        # each score's distance from the mean
```

```
[1] -3 -1 -1 -1  0  0  2  4
```

```
sum(x - mean(x))   # add them all up
```

```
[1] 0
```

Zero. Not close to zero—zero. It will be zero for every data set you will ever collect, because that is what a mean *is*: the balance point where the negatives and positives cancel exactly.

So the honest average distance is useless as a measure of spread. Squaring fixes it: every squared deviation is positive, nothing cancels, and bigger deviations count for disproportionately more.

4.3.7 Why Divide by $N-1$ and Not N?

Here is the intuition. Once you know the mean and all but one of the deviations, the last deviation is not free—it is forced to be whatever makes the sum come out to zero. Eight numbers, but only seven of them can vary independently once the mean is fixed. You have $N-1$ independent pieces of information, so you divide by $N-1$.

That is the explanation. Here is the proof, because code is an awesome way to settle an argument.

This uses a function you will see constantly from here on: **`replicate()`**. It does exactly what the name says. You give it a number and a block of code, and it runs that code that many times, collecting every answer. `replicate(10000, ...)` means "do this ten thousand times and keep all ten thousand results." It is how we run a study over and over without recruiting a single human.

```
set.seed(343)

# The truth: a population with variance of exactly 25 (because SD = 5, and 5^2 = 25).
truth <- 25

# Run the following block 10,000 times, keeping every answer.
estimates <- replicate(10000, {

  s <- rnorm(8, mean = 100, sd = 5)   # grab a tiny sample of 8 people
  deviations <- (s - mean(s))^2       # squared distance of each score from ITS OWN mean

  # Compute the variance two ways and return both.
  c(divide_by_N   = sum(deviations) / 8,   # the tempting way
    divide_by_N_1 = sum(deviations) / 7)   # the N-1 way
})

# estimates now holds 10,000 answers of each kind. Average them.
round(rowMeans(estimates), 2)
```

```
  divide_by_N divide_by_N_1
        21.80         24.91
```

Dividing by N gives an answer that is systematically too small. Dividing by $N-1$ lands closer to the truth. Ten thousand samples do not lie, and now you never have to wonder whether that -1 is a typo. I will have lots of typos in other places for you to find.

4.3.8 Variance and SD

Population variance (when we have the entire population and know μ):

$$\sigma^2 = \frac{\Sigma(X-\mu)^2}{N}$$

Sample variance, using $N-1$ to estimate population variance:

$$S^2 = \frac{\Sigma(X-M)^2}{N-1}$$

! Greek Letters Are the Truth; Roman Letters Are Your Guess

You just met the second pair, so here is the rule that runs through the entire rest of this book.

	Population (the truth)	Sample (your estimate)
Mean	μ (mu)	M
Variance	σ^2 (sigma squared)	S^2
Standard deviation	σ (sigma)	S

Greek letters describe the population. They are fixed, real, and almost always unknowable.

Roman letters describe your sample. They are computed from data you actually collected, they change every time you collect new data, and they are your best available guess at the Greek letter sitting behind them.

So M is your guess at μ, and S is your guess at σ. When you see a Greek letter in a formula, ask yourself where that number is supposed to come from, because in real research the answer is usually "nowhere, and that is the problem we are about to solve."

R vectorizes this for you (turns into a string of numbers), but watch your parentheses. The built-in `var()` uses M and $N - 1$:

```
Variance.M <- var(Normal.Sample.1)
```

Sample variance = 225.47

$$S = \sqrt{S^2}$$

```
SD.calc <- sd(Normal.Sample.1)
```

Sample SD = 15.02

i Variance Has Weird Units; SD Comes Back Home

Variance is measured in **squared units**.

- If reaction time is measured in seconds, variance is measured in seconds squared.
- If exam scores are measured in points, variance is measured in points squared.
- If cookie-dough devotion is measured in questionable rating units, variance uses questionable rating units squared.

Standard deviation takes the square root of variance, returning to the original units. That is one reason SD is easier to interpret.

4.3.9 A Dot Plot: Where Does One Standard Deviation Live?

The standard deviation is a measure of the **typical distance of scores from the mean**. It is calculated from every score's distance from the mean. It is not defined by capturing a particular percentage of people. But what does **typical distance** mean? *What do truth and god mean, for that matter? Well, to quote the great teacher G'Kar, truth is a river and god is the mouth of that river. Note: if you got that joke, you are a super nerd and we can be friends.*

The **typical distance** concept applies really only when the distribution is approximately normal and the standard deviations divide the distribution in a very useful and predictable way:

- About 34% of scores fall between the mean and one SD **below** the mean.

- About 34% fall between the mean and one SD **above** the mean.
- Put those pieces together, and about 68% fall within one SD of the mean.
- About 95% fall within two SDs of the mean.

Let us make that visible with an extremely central psychological construct: love of cookie-dough ice cream.

We ask 240 people:

> **How much do you like cookie-dough ice cream?**

They answer on a 1–10 unipolar rating scale:

- **1 = Not at all**
- **10 = I would trade my mother for cookie-dough ice cream**

The scale is **unipolar** because it begins with none of the feeling and moves in one direction toward more of it. The opposite endpoint is not "I hate cookie-dough ice cream with the fire of a thousand suns." That would be a bipolar scale and clearly a cry for professional help.

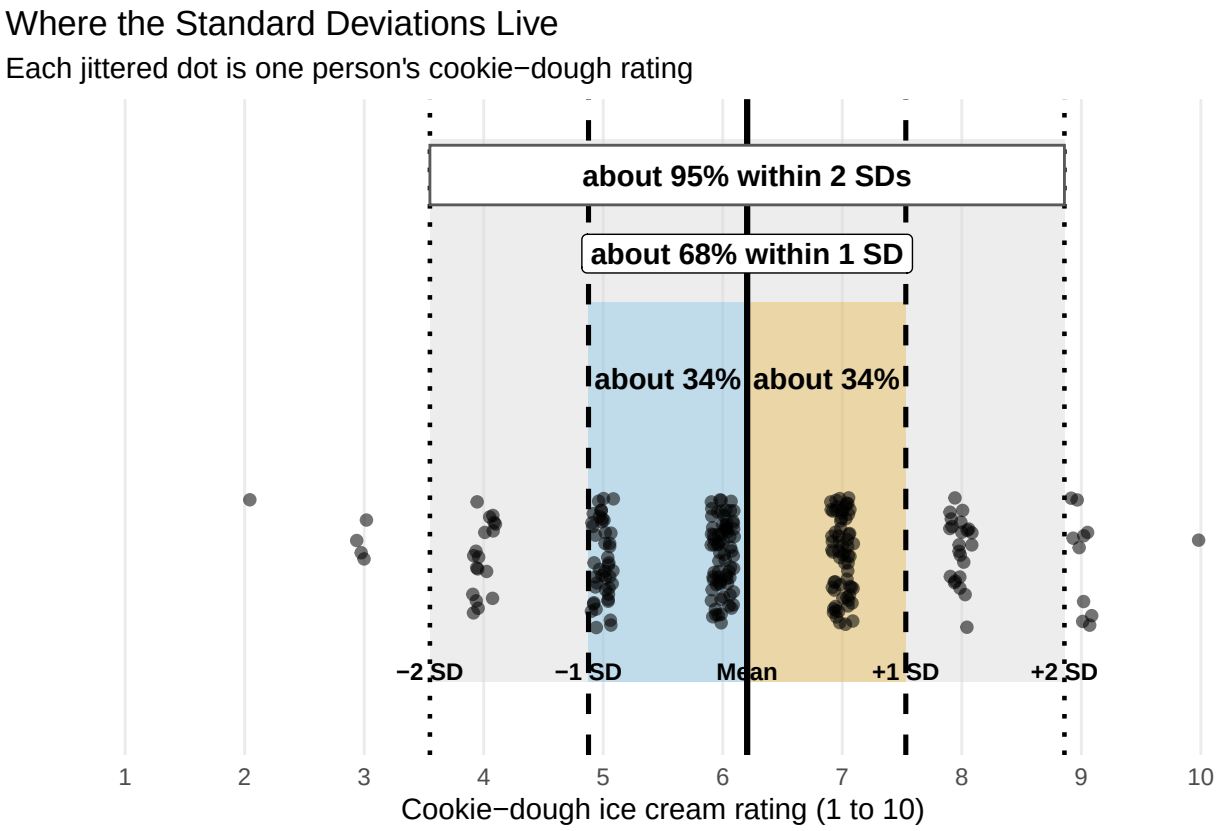


Figure 4.3: Simulated cookie-dough ratings. The dots are people; the lines mark the mean and standard-deviation boundaries. The 34%, 68%, and 95% values describe an approximately normal distribution.

In this particular simulated sample, 76.2% of ratings fall within one SD of the mean and 93.3% fall within two SDs. Those values will not equal exactly 68% and 95%. These are discrete ratings from one sample, not obedient dots hired to impersonate a perfect normal curve. Chance is fickle, and a 1–10 scale also has walls at both ends.

The important visual intuition is this: **one SD takes us from the mean through the thick central part of a normal distribution. Two SDs cover nearly everybody.** A score farther than two SDs from the mean is unusual, although "unusual" does not automatically mean impossible, important, fraudulent, or possessed by evil spirits.

4.3.10 Relationship between M and S

The standard deviation is expressed in the same units as the scores, and it does not automatically grow just because the mean grows—although annoyingly, it tends to. For positive **ratio-scale** variables, the **coefficient of variation** can compare spread relative to the mean:

$$CV = \frac{S}{|M|}$$

```
CV <- SD.calc / abs(M)
```

$CV = 0.15$.

> **The CV Needs a Mean Worth Dividing By**
>
> A problem occurs when the mean is close to zero. Dividing by a tiny mean can produce an enormous and unstable CV. If the mean is zero, the CV is undefined.
>
> Therefore, interpret the CV most confidently when zero has a meaningful interpretation, ratios are sensible, and the mean is not hovering near zero while laughing at you.

4.3.11 Moment 3: Skewness

Skewness is how asymmetrical the hill is. Generally, the farther the value is from zero, the more skewed the distribution. Human data will almost always be somewhat asymmetrical, like most of our faces. When people's ears or eyes are slightly asymmetrical, we generally choose to ignore the problem for reasons—assumptions, really—that we hope are true. More on that later.

The skewness formula is below for completeness. Almost nobody reports the metric.

$$Skewness = \frac{\Sigma(X - M)^3/N}{S^3}$$

```
library(moments)
SK.Calc <- skewness(Normal.Sample.1)
```

Skewness = 0

4.3.12 Moment 4: Kurtosis

This is peakiness! You can have flattish, longish hills (**platykurtic**) or skinny, tall hills (**leptokurtic**). I would be remiss to my mother if I did not point out that these are Greek words. *I would also like to point out that nobody has ever confused my body shape with leptokurtotic.*

The kurtosis formula is below for completeness. Not only does almost nobody report it, I do not think many people know how to make sense of the numbers it spews out.

$$Kurtosis = \frac{\Sigma(X - M)^4/N}{S^4} - 3$$

```r
K.Calc <- kurtosis(Normal.Sample.1) - 3
```

Kurtosis = 0

4.3.13 All Four Moments via Tidyverse

The tidyverse is a collection of R packages that makes data work readable and consistent. Think of it as data operations in near-plain English: take this data, group it, summarize it, done.

We need three verbs for now:

- `group_by()`—split data into groups
- `summarise()`—compute statistics per group
- `filter()`—keep only rows we want

First, put our vector into a **data frame** (R's version of a spreadsheet—rows are observations, columns are variables):

```r
DF <- data.frame(X = Normal.Sample.1)
```

Now compute all four moments at once:

```r
library(tidyverse)   # loads dplyr, ggplot2, and friends
library(moments)     # gives us skewness() and kurtosis()

Desc.table <-
  DF |>                          # take the data frame, then...
  summarise(N    = n(),                  # count the rows
            M    = mean(X),              # moment 1: center
            S    = sd(X),                # moment 2: spread
            CV   = S / abs(M),           # spread relative to the mean
            Skew = skewness(X),          # moment 3: lopsidedness
            Kurt = kurtosis(X) - 3)      # moment 4: tails (minus 3 = "excess")

round(Desc.table, 2)
```

```
      N      M     S   CV Skew Kurt
1 1e+06 100.01 15.02 0.15    0    0
```

That `|>` is a **pipe**. Read it as the word "then." It takes whatever is on its left and feeds it into the function on its right, so `DF |> summarise(...)` means "take DF, *then* summarise it." It saves you from writing `summarise(DF, ...)` with everything crammed inside one set of parentheses, and it gets very handy once you are stacking four or five operations.

> **About All Those Plots I Keep Making Without Explaining Them**
>
> You have now seen several figures built with `ggplot()`, and I have never once told you how it works. That is deliberate. Explaining the grammar of graphics in the middle of a chapter about variance would help nobody.
>
> There is a whole chapter on `ggplot()` and the tidyverse at the end of the book. When you want to make your own plots rather than read mine, go there. For now, feel free to copy any plotting code you see, change the variable names, and see what happens. That is how everyone learns it anyway.

4.4 Distributions and Probability

A distribution isn't just a picture of data—it lets us make **probability statements**.

The height of the curve at any point tells us how likely values near that point are. This is the **Probability Density Function** (PDF).

4.4.1 Probability Density Function (PDF)

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Looks scary. The concept is simple:

- Values near the mean → highest density
- Values far from the mean → lower density
- Shape is fully determined by μ and σ

In practice, we convert to **z-scores** so that mean $= 0$ and $SD = 1$, which makes everything easier.

4.4.2 Z-scores

A **z-score** tells you how far a value is from the mean in standard deviation units:

- z $= 0$ → exactly at the mean
- z $= 1$ → one SD above the mean
- z $= -1$ → one SD below the mean

Converting to z-scores does **not change the shape** of the distribution—it just re-labels the axis.

$$Z = \frac{X - M}{S}$$

> **!** A z-Score Is Not a Probability
>
> A z-score tells us how far a score is from the mean in standard-deviation units.
>
> It does **not** directly tell us the percentage of people below that score. To obtain a percentile, we must use the appropriate distribution—usually with `pnorm()` when normality is a reasonable model.
>
> A z-score of 1 means one SD above the mean. It does not mean 1%, 10%, or that the participant has won statistics.

4.4.3 Making Z-scores in R

```r
Normal.Sample.Z <- scale(Normal.Sample.1, center = TRUE, scale = TRUE)
```

```r
DF.Z <- data.frame(X = Normal.Sample.Z)
Desc.table.Z <-
  DF.Z |>
  summarise(N = n(), M = mean(X), S = sd(X))
round(Desc.table.Z, 2)
```

```
      N M S
1 1e+06 0 1
```

The shape doesn't change:

```r
hist(Normal.Sample.Z,
  breaks = 30,
  col    = "lightblue",
  border = "black",
  main   = "Z Score Histogram",
  xlab   = "Population Distribution",
  ylab   = "Frequency")
abline(v = 0, col = "black")
```

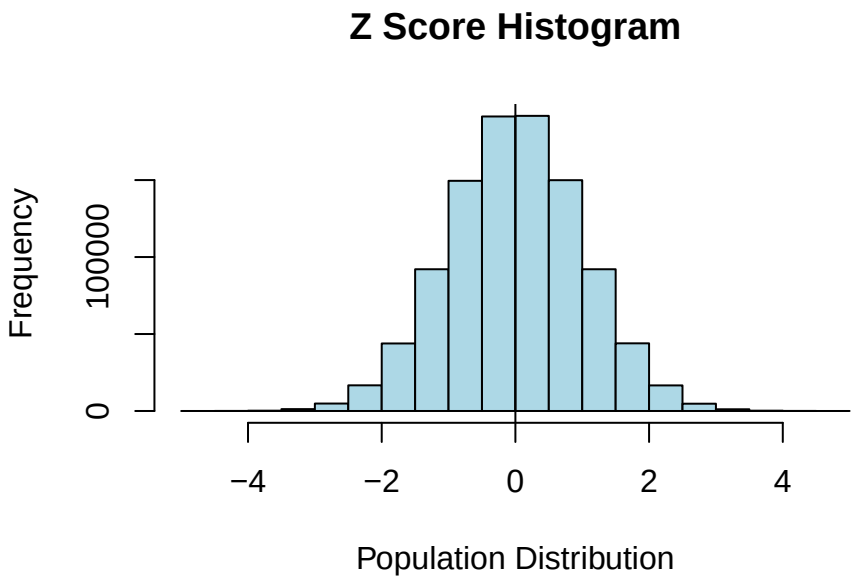


Figure 4.4: The same one million people after z-scoring. Identical shape, new axis labels. Standardizing moves the ruler, not the data.

4.4.4 Finding Probability from Z-scores

Two functions to know:

- `dnorm()` → height of the curve (density) at a value
- `pnorm()` → area under the curve to the left of a value (probability)

The PDF `dnorm()` implements:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

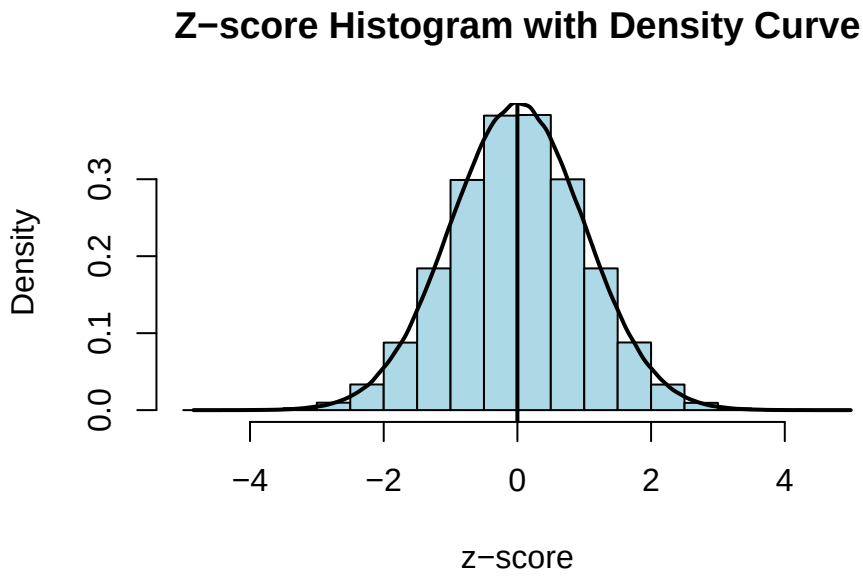


Figure 4.5: The same histogram with the density curve laid over it. `dnorm()` draws that curve; `pnorm()` measures the area underneath it.

4.4.5 How much of the population is below the mean?

Exactly 50%—the normal distribution is symmetric. Confirm it:

```
pnorm(0)
```

```
[1] 0.5
```

What proportion falls between 1 and 2 SDs above the mean?

```
pnorm(2) - pnorm(1)
```

```
[1] 0.1359051
```

No calculus. Just area under the curve.

4.4.6 Going the Other Way

`pnorm()` takes a score and hands back a proportion. Half the questions you will actually be asked run the other direction: *what score do you need to be in the top 10%?* That is `qnorm()`.

```
qnorm(.90)   # the z-score that cuts off the top 10%
```

```
[1] 1.281552
```

A z of 1.28. Convert back to raw cognitive-score units with $M + zS$:

```
100 + qnorm(.90) * 15
```

```
[1] 119.2233
```

So you need about 119 points to crack the top 10%. Cutoffs for gifted programs, clinical screening thresholds, and "we only interview the top 5% of applicants" are all this calculation wearing different toupees.

You can also ask the data directly instead of the theoretical curve. `quantile()` finds the actual score at a given percentile in your sample:

```
quantile(Normal.Sample.1, .90)
```

```
     90%
119.2508
```

Nearly identical, because we built this population to be normal. On real data the two can disagree, and when they do, the data is right and the curve was an assumption.

4.5 Real-World Example: Mental Rotation Test

We now know how to:

- Describe distributions
- Standardize scores with z-scores
- Convert z-scores to probabilities

Let's apply all of it to a real psychological measure.

The **Mental Rotations Test (MRT)** measures spatial visualization ability. Peters et al. developed a redrawn version and examined factors affecting performance (Peters et al. 1995). Here's a summary by field and gender:

	Sciences			Social Sciences & Humanities		
	n	M	SD	n	M	SD
M	817	15.23	4.42	517	12.74	4.87
F	392	11.18	4.37	1358	8.75	4.18

> ⚠ Advanced Code Has Arrived Early
>
> The code below is more advanced; you do not need to understand it yet. Why am I showing you crazy code now? To make you feel bad about yourself and the terrible life choices you made when you took a statistics class!
>
> Actually, I want you to see what R can do. I can go from a table to simulation, graphs, and analysis without a lot of code. Simulating data is my favorite way to make graduate students cry when they propose dissertations because it is the fastest way to discover whether the theory and hypotheses they proposed make any sense. When they do not, the students cry and Dorkaos, our God of Statistics, gains more strength.

4.5.1 Step 1: Simulate the population distributions

We only have summary stats, but because MRT scores are approximately normal we can **simulate each group** using `rnorm()`.

```
library(tidyverse)
set.seed(123)

params <- tribble(
  ~Field,            ~Group, ~n,    ~mean,  ~sd,
  "Sciences",        "M",    817,   15.23,  4.42,
  "Sciences",        "F",    392,   11.18,  4.37,
  "Social Sciences", "M",    517,   12.74,  4.87,
  "Social Sciences", "F",    1358,   8.75,  4.18
)

MRT.2 <- params |>
  mutate(data = pmap(list(n, mean, sd), rnorm)) |>
  unnest(data) |>
  select(Field, Group, Value = data)
```

Check that our simulation matches published values:

```
MRT.table <-
  MRT.2 |>
  group_by(Field, Group) |>
  summarise(N = n(), M = mean(Value), S = sd(Value))
MRT.table
```

```
# A tibble: 4 x 5
# Groups:   Field [2]
  Field           Group     N     M     S
  <chr>           <chr> <int> <dbl> <dbl>
```

```
1 Sciences         F       392 11.4   4.47
2 Sciences         M       817 15.3   4.34
3 Social Sciences  F      1358  8.74  4.16
4 Social Sciences  M       517 12.9   4.88
```

4.5.2 Step 2: Visualize the distributions

```
ggplot(MRT.2, aes(x = Value, fill = Group, color = Group)) +
  geom_histogram(aes(y = after_stat(density)), position = "identity", alpha = 0.3, bins = 30) +
  geom_density(alpha = 0.1) +
  facet_wrap(~ Field) +
  labs(title = "Overlaid Histograms & Density Plots", x = "Score", y = "Density") +
  theme_bw()
```

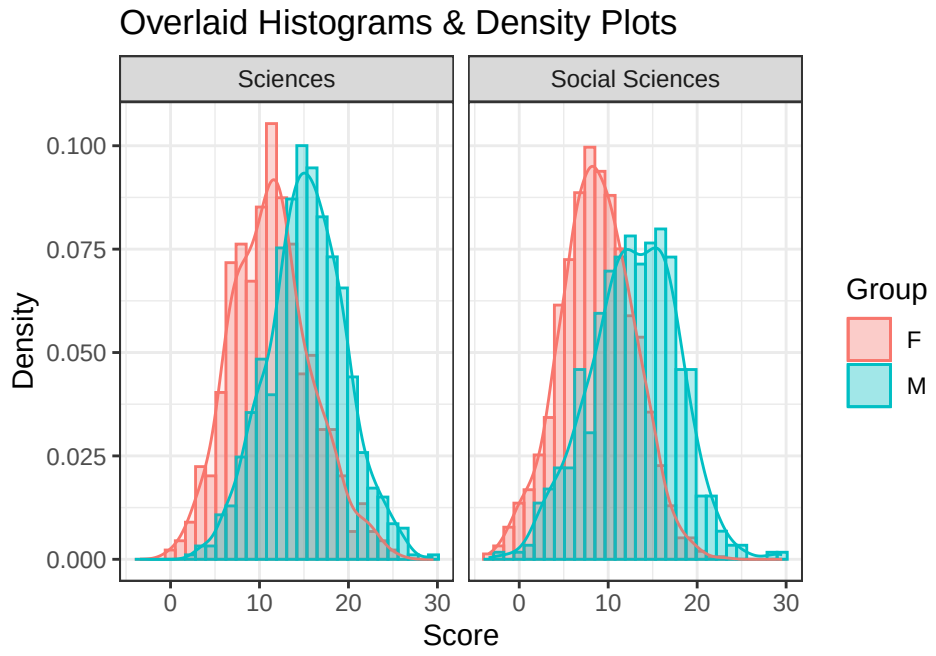


Figure 4.6: Raw MRT scores by field and group. The two groups sit in different places on the score axis, and the gap is bigger in the sciences.

4.5.3 Step 3: Convert to z-scores (within groups)

Standardize within each group—critical because each group has a different mean and SD.

```
MRT.2 <- MRT.2 |>
  group_by(Field, Group) |>
  mutate(Z = scale(Value))
MRT.2 |> head()
```

```
# A tibble: 6 x 4
# Groups:   Field, Group [1]
```

```
  Field    Group Value   Z[,1]
  <chr>    <chr> <dbl>   <dbl>
1 Sciences M      12.8 -0.583
2 Sciences M      14.2 -0.247
3 Sciences M      22.1  1.58
4 Sciences M      15.5  0.0594
5 Sciences M      15.8  0.119
6 Sciences M      22.8  1.73
```

```
ggplot(MRT.2, aes(x = Z, fill = Group, color = Group)) +
  geom_histogram(aes(y = after_stat(density)), position = "identity", alpha = 0.3, bins = 30) +
  geom_density(alpha = 0.1) +
  facet_wrap(~ Field) +
  labs(title = "Overlaid Histograms & Density Plots", x = "Z-Score", y = "Density") +
  theme_bw()
```

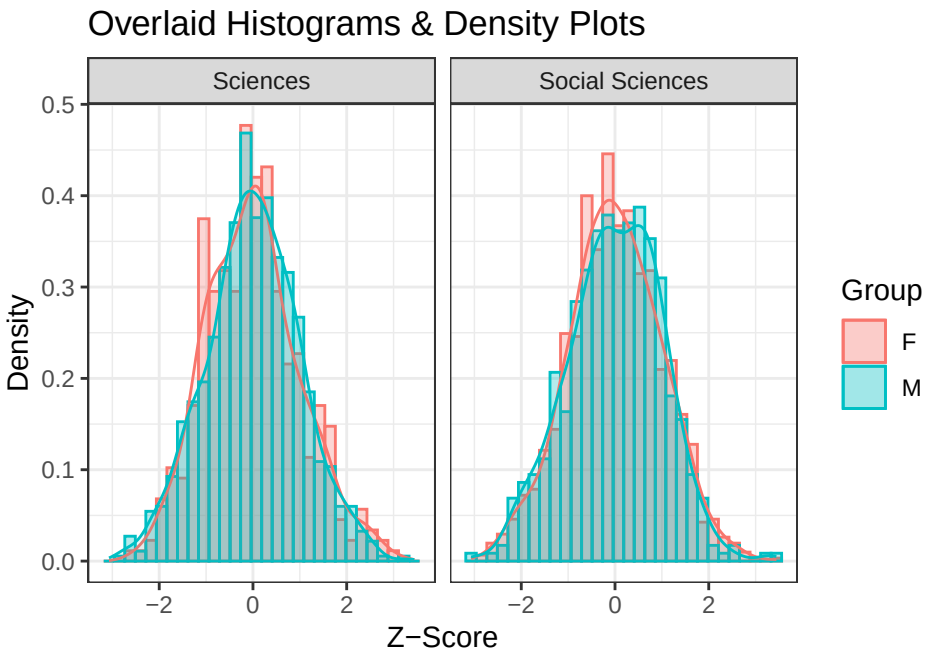


Figure 4.7: The same scores after z-scoring *within* each group. The distributions now sit on top of each other, because every group is now measured against its own mean.

Why do the curves overlap now? Because standardizing compares *relative position within each group*, not raw scores.

4.5.4 Step 4: Z-score to probability

Alex (male, social sciences) got a score of 10. How did he do relative to his peers? (Males, Social Sciences: $M = 12.74$, $S = 4.87$)

```
Z.Alex <- (10 - 12.74) / 4.87
Z.Alex
```

```
[1] -0.5626283
```

A little over half an SD below the mean. Visually:

```
MRT.2.Males.Social.Sciences <- MRT.2 |> filter(Group == "M" & Field == "Social Sciences")

ggplot(MRT.2.Males.Social.Sciences, aes(x = Z)) +
  geom_histogram(aes(y = after_stat(density)), position = "identity", alpha = 0.3, bins = 30) +
  geom_density(alpha = 0.1) +
  labs(title = "Males in the Social Sciences", x = "Z-Score", y = "Density") +
  theme_bw() +
  geom_vline(xintercept = Z.Alex, color = 'red')
```

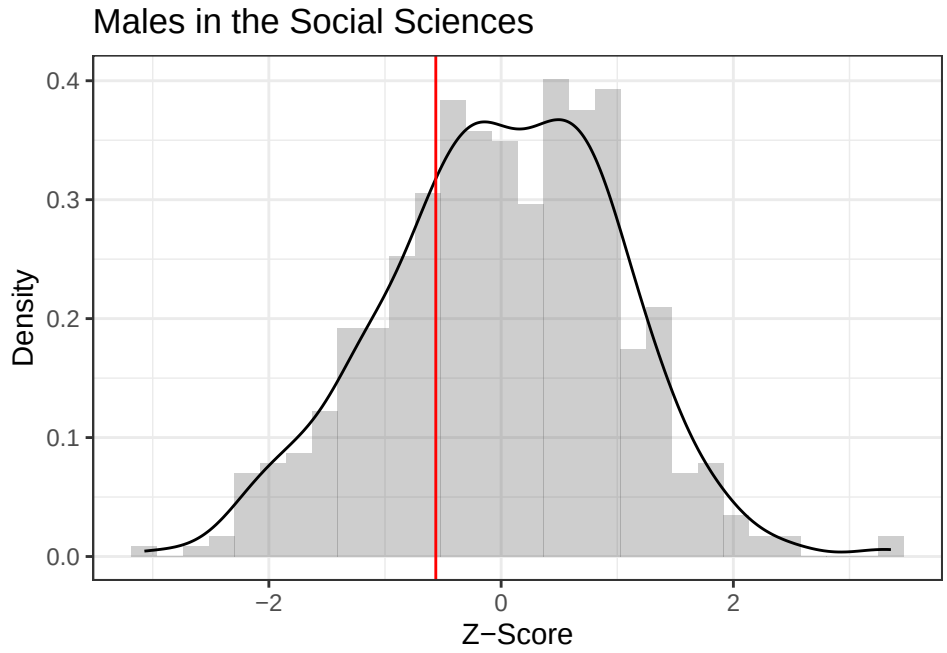


Figure 4.8: Alex's place among male social scientists. The red line is him. Most of the distribution is to the right of it.

```
pnorm(Z.Alex)
```

```
[1] 0.286844
```

Only better than 28.7% of peers. That's also called a **percentile**.

4.5.5 Step 5: Same raw score, different meaning

Emma also scored a 10, but she's in a different group (Female, Social Sciences: $M = 8.75$, $S = 4.18$).

```
Z.Emma <- (10 - 8.75) / 4.18
Z.Emma
```

```
[1] 0.2990431
```

```r
pnorm(Z.Emma)
```

```
[1] 0.6175464
```

Emma scored better than 61.8% of her peers.

```r
curve(pnorm, from = -10, to = 10,
  main = "Cumulative Density Function",
  xlab = "z-score", ylab = "Probability",
  xlim = c(-3.25, 3.25))
abline(v = Z.Emma, col = "red")
abline(v = Z.Alex, col = "blue")
```

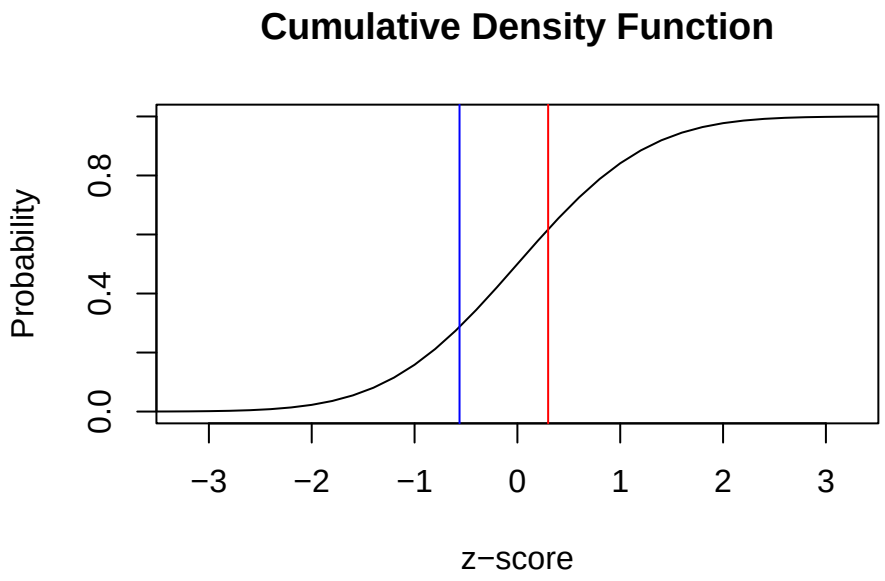


Figure 4.9: The same raw score of 10, read against two different reference groups. Emma (red) lands above the middle; Alex (blue) lands below it. The score never moved. The comparison group did.

Same raw score. Emma is above average for her group; Alex is below average for his. Raw scores mean nothing without context.

4.5.6 Big idea

- Raw scores mean nothing without context.
- Distributions give scores meaning.

4.6 The Short Story

- A distribution shows which values occur and how often.

- The mean locates the center; variance and standard deviation describe spread.
- Skewness describes asymmetry; kurtosis describes tail weight and peak shape relative to a reference distribution.
- Zero skewness and zero excess kurtosis do not prove normality. Strange distributions are capable of wearing fake mustaches.
- z-scores translate raw scores into standard-deviation units.
- The same raw score can mean very different things in different reference groups.
- Always inspect the distribution before asking one summary number to represent several hundred people without supervision.

5 Standard Error

5.1 Sampling Error and the Limits of Samples

Samples are guesses. Your sample mean will almost never equal the true population mean—and that is fine, as long as you know *how unstable the guess might be.*

Probability is fickle. With a tiny sample, the Probability Gods may hand you ten unusually delighted commuters or ten people whose train just vanished from the tracker. A larger random sample gives chance fewer opportunities to build your entire conclusion out of one weird subway car.

Suppose we randomly select 10 riders from a CTA subway car and ask:

How much do you typically like traveling on the CTA?

- 1 = Strongly dislike
- 2 = Dislike
- 3 = Neither dislike nor like
- 4 = Like
- 5 = Strongly like

Who we grab matters—daily commuters, tourists, and people who just missed their train all give different answers. The gap between our sample estimate and the true population value is called **sampling error**. It's not a mistake, it's just an unavoidable consequence of observing only part of the population.

The question is: *how bad is the error, and how can we estimate it from a single sample?*

5.1.1 Traveling on the CTA in the Lap of Luxury

The CTA hires me to do a satisfaction survey on the subway. So I take my question, and I want to find out what the population attitude toward the CTA is.

Well, the CTA handles about 1 million riders a day, but (a) they did not pay me enough to hire all the people I would need to ask all those people, and (b) I do not want to talk to random strangers on the Chicago subway about the CTA. They might punch me in the face as an answer to my question.

So I am hoping that 10 people is enough to answer their question and also lets me run this study cheaply so I can pocket all the money. *Have you seen how much groceries cost nowadays?*

5.1.2 Randomly Sample from the Population, Which Is Harder Than It Sounds

First I need R to build me a city cause I want to simulate this process, not actually interact with the "commoners". Here our simulation generates all one million riders and their opinions, rounded onto the 1–5 scale. In a real study you never get to see this, which is why the rest of the chapter exists.

```
set.seed(42)

Npop <- 1e6

# Draw a million opinions from a normal distribution centered on 3.
CTA.Population <- rnorm(Npop, mean = 3, sd = 1)

# rnorm gives decimals like 3.47, but nobody circles 3.47 on a survey.
# round() snaps every score to the nearest whole number.
CTA.Population <- round(CTA.Population)

# The scale only goes 1 to 5, but rnorm does not know that and will hand
# us the occasional 0 or 6. These two lines find every score below 1 and
# set it to 1, then every score above 5 and set it to 5.
# Read it as: "of all the scores, the ones that are too small become 1."
CTA.Population[CTA.Population < 1] <- 1
CTA.Population[CTA.Population > 5] <- 5
```

I enter a random subway car, at a random time, and randomly select 10 people and ask them to answer the question.

```
set.seed(1)          # makes the "random" draw repeatable, so you get my numbers
SampleSize <- 10

# sample() reaches into the population and pulls out 10 riders at random.
# replace = FALSE means we cannot survey the same person twice.
sample.1 <- sample(CTA.Population, SampleSize, replace = FALSE)

M.1  <- mean(sample.1)   # this sample's mean
SD.1 <- sd(sample.1)     # this sample's standard deviation
```

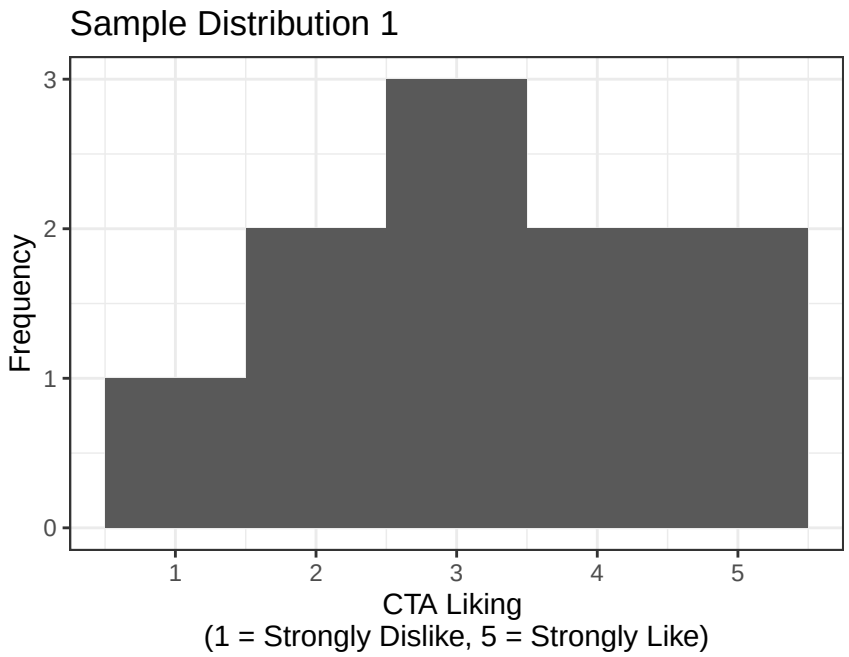


Figure 5.1: Ten riders, one afternoon. This is the entire evidence base for our first estimate.

The mean of this sample is $M = 3.2$.

Okay, so is that the **true** population mean? *Have I sacrificed enough undergrads to the Probability Gods to get it perfectly right?* Probably not, since they always demand more. Ugh, I should probably double check. So I head out again to a new random subway car, at a random time, and randomly select 10 new people and ask them to answer the question. Let's hope no one punches me...

Same code. New seed, new subway car.

```
set.seed(26)          # a different seed, so a different 10 riders
SampleSize <- 10
sample.2 <- sample(CTA.Population, SampleSize, replace = FALSE)

M.2  <- mean(sample.2)
SD.2 <- sd(sample.2)
```

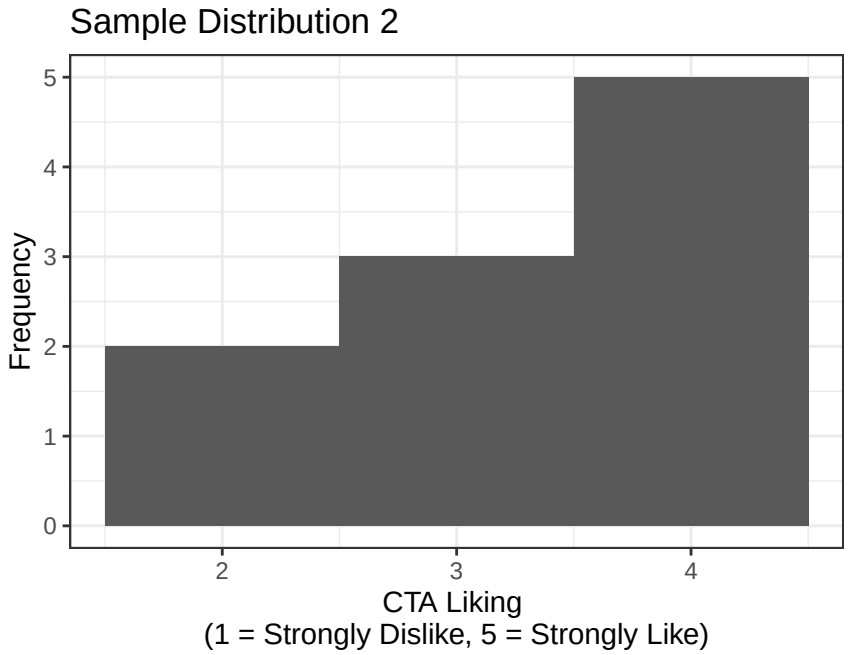


Figure 5.2: Ten different riders, a different afternoon, a different answer.

The new mean of this sample is $M = 3.3$.

What the heck?! The mean is higher now. Which sample (1 or 2) is correct? Grrrr...

5.1.3 Brute force solution?

Hmm, I am not sure. So to solve my problem, I will do what all **good** professors do: use coerced undergraduate labor! So I send all undergraduates in my class out to each randomly sample 10 people on different days, at different times, and on different subway lines, and collect the data. We will call it homework...yeah homework...

Three lines of R replace 80 undergraduates.

```
set.seed(616)
NumberofSamples <- 80
SampleSize <- 10

# replicate() runs the same line over and over. Here it runs the sampling
# 80 times, once per undergraduate, and stacks the results into a table
# with one column per sample.
Sample.Results <- replicate(NumberofSamples,
                            sample(CTA.Population,
                                   size = SampleSize,
                                   replace = FALSE))

# apply() runs a function down each row (1) or each column (2) of that table.
# We want the mean of each sample, and samples are columns, so we use 2.
Sample.Means <- apply(Sample.Results, 2, mean)
```

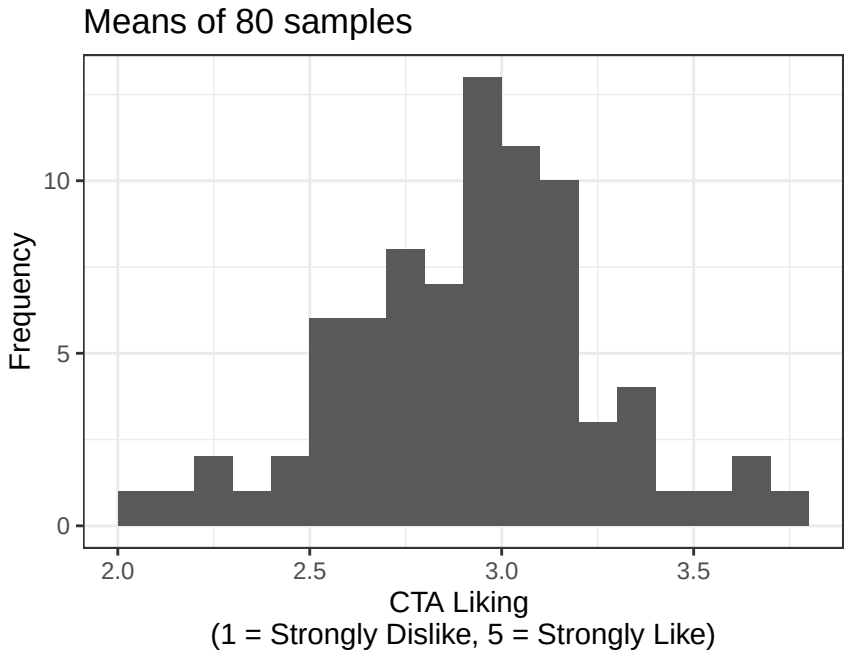


Figure 5.3: Eighty samples, eighty means. No single one of them is the truth; together they show how far off any one of them can be.

You will notice the distribution of means is normal-ish and centered around 3, and it was precisely: Mean of means = 2.97. *Remember, you would never know the true population mean. Good luck getting every rider of the CTA to answer a survey, and the Probability Gods only love me anyway, cause you have not spent your career tricking undergrads into a volcano.*

5.1.4 SD of Distribution of Means

But you can also see that there is a wide range of sample means in that figure. *So what does that mean?*

Can I *trust* any **individual sample** to tell me the true population value? *Like a stranger with candy, we need to trust but verify, so ask your mother about the van before you get in it, or about any strange sample you meet in the park.*

Recall what a standard deviation (SD) represents: **the typical distance that observations fall from their mean.**

For a single sample, SD tells us how much individual observations vary around the sample mean.

But now we're doing something slightly different.

Just like we had a standard deviation (S) around a sample mean, we can also calculate the standard deviation of the sample means themselves, that is, how much the *means vary from sample to sample.*

For a single sample, SD is:

$$S = \sqrt{\frac{\Sigma(X - M)^2}{N - 1}}$$

Here N is the number of people in the sample: 10.

For the distribution of means, we look at how each sample mean differs from the **grand mean** (mean of means):

$$S_M = \sqrt{\frac{\Sigma(M - M_{Grand})^2}{N_{samples} - 1}}$$

Here $N_{samples}$ is the number of *samples*: 80 of them, one per undergraduate. It is not the 10 people inside each sample.

> **Two Different Counts Are Hiding in This Example**
>
> Our simulation contains:
>
> - **10 people per sample**, which determines how stable each sample mean is.
> - **80 repeated samples**, which lets us see the sampling distribution.
>
> In a real study, we usually collect only one sample. We cannot directly observe its entire sampling distribution, so we estimate the standard error with $S/\sqrt{N}$—and that N is back to being the 10 people.

This quantity is the standard error (SE). Conceptually, it tells us: **how much we expect a sample mean to vary simply due to random sampling**.

> **SD and SE Describe Different Crowds**
>
> The **standard deviation** describes how individual scores vary around their sample mean.
>
> The **standard error** describes how sample means would vary across repeated samples.
>
> - SD asks: *How different are the people?*
> - SE asks: *How unstable is our estimate of the population mean?*
>
> Same units. Different crowd. Confusing them will anger the God Dorkaos.

We already have the 80 means sitting in `Sample.Means`, so the standard error is one function call:

```
# Note what these are running on: Sample.Means, the 80 means themselves.
M.S  <- mean(Sample.Means)   # the grand mean, a mean OF means
SD.S <- sd(Sample.Means)     # the standard error, an SD OF means
```

The grand mean $M_G = 2.97$

Standard error (spread of the sample means) = 0.34

Just like ±2 SD captures about 95% of individual observations in a roughly normal distribution, ±2 SE captures about 95% of the sample means in the sampling distribution.

In other words, if we repeatedly sample from this population, about 95% of the sample means should fall within two standard errors of the grand mean.

Thus, the population mean is hopefully between: 2.3 and 3.64. *Note I am being loose here and we will come back to this.*

(*Assuming random sampling and a roughly normal sampling distribution—both of which we're getting for free here.*)

⚠ Two Is a Shortcut for Now; We Return to It Next Chapter

Using $M \pm 2SE$ gives a rough 95% interval when the sampling distribution is approximately normal.

For small samples, the proper confidence interval uses a value from the **t-distribution**:

$$M \pm t^* SE$$

For example, with $N = 10$, the 95% multiplier is about 2.26—not exactly 2. We will learn where that number comes from in the t-test chapter. For now, $\pm 2SE$ is a useful approximation. But **hold your Guinness Beers**. *That is dramatic foreshadowing, and yes, English teachers, I did learn something.*

5.1.5 The Standard Error of the Mean (SEM), the One Number to Rule Them All

The Dean found out about my use of forced student labor for my personal consulting gig. Apparently it's unethical, and I owe the university 60% for "overhead" *(yes, seriously, they take 60% of grant money)*. The Dean told me to solve my problem another way or fork it over.

Since I still have student loans to pay (sadly), what if we just make some *assumptions* about the world and estimate our sampling error from one sample? Because, again, I am lazy, my mother said the van checked out, and of course I would like to keep all the money for myself.

The strange thing was that the distribution of the means from all those samples looked normal-ish. After a little digging in some old, dusty textbooks, I learned that what I was seeing was not a fluke. In fact, it's a well-known property of repeated sampling called the *Central Limit Theorem*.

Central Limit Theorem (CLT): *If you repeatedly take random samples from a population, the distribution of the sample means will be approximately normal, centered on the population mean, even if the original population is not normal, as long as the sample size is reasonably large.*

The Raw Data Do Not Become Normal

The Central Limit Theorem applies to the **distribution of sample means**. It does not promise that the individual scores—or the population they came from—will become normal.

The individual commuters remain weird. Their means become better behaved.

Because of this, we can assume that if I took repeated samples, the means would be normally distributed around the population mean, and that the amount of error depends on the sample size.

That makes intuitive sense. If I sampled 100 people per sample, there would be less random error per sample. If I only sampled 10 people, there's a much better chance my estimate would be way off.

According to this dusty old textbook, we can *approximate* the **standard error of the mean** using:

$$S_M = \frac{S}{\sqrt{N}}$$

So, if our first sample has a standard deviation of $S = 1.32$ then our estimated standard error of the mean is:

$S_M = 0.42$

This value is not too far off from the value I estimated earlier by sending out 80 undergraduates to repeatedly run the study: SE from the "actual" sampling distribution = 0.34.

In other words: the math mostly agrees with the coerced labor. *Now I can pay my loans and buy a convertible Porsche like other middle-aged men and look ridiculous.*

5.1.6 Effect of Sample Size

Okay, so what would have happened if I ran my coerced labor study with different sample sizes?

Instead of sending out more undergrads (ugh, mean Dean), I can just use math.

Recall:

$$S_M = \frac{S}{\sqrt{N}}$$

So as N increases, the SEM must decrease and it does so nonlinearly.

Let's compute the SEM (S_M) for sample sizes from 10 to 400 people per sample and visualize it.

```
# Population SD
pop_sd <- sd(CTA.Population)

# Every sample size from 10 to 400. The colon makes a sequence: 10, 11, 12, ... 400.
N_vals <- 10:400

# A data frame is R's spreadsheet: each argument becomes a column, and
# the columns line up row by row. Here row 1 holds N = 10 and its SEM,
# row 2 holds N = 11 and its SEM, and so on down to 400.
sem_df <- data.frame(
  N   = N_vals,
  SEM = pop_sd / sqrt(N_vals)   # the SEM formula, applied to all 391 values at once
)

# Plot
ggplot(sem_df, aes(x = N, y = SEM)) +
  geom_line(linewidth = 1) +
  labs(
    title = "Standard Error of the \nMean as a Function of Sample Size",
    x = "Sample Size (N)",
    y = "Standard Error of the Mean (SEM)"
  ) +
  theme_bw()
```

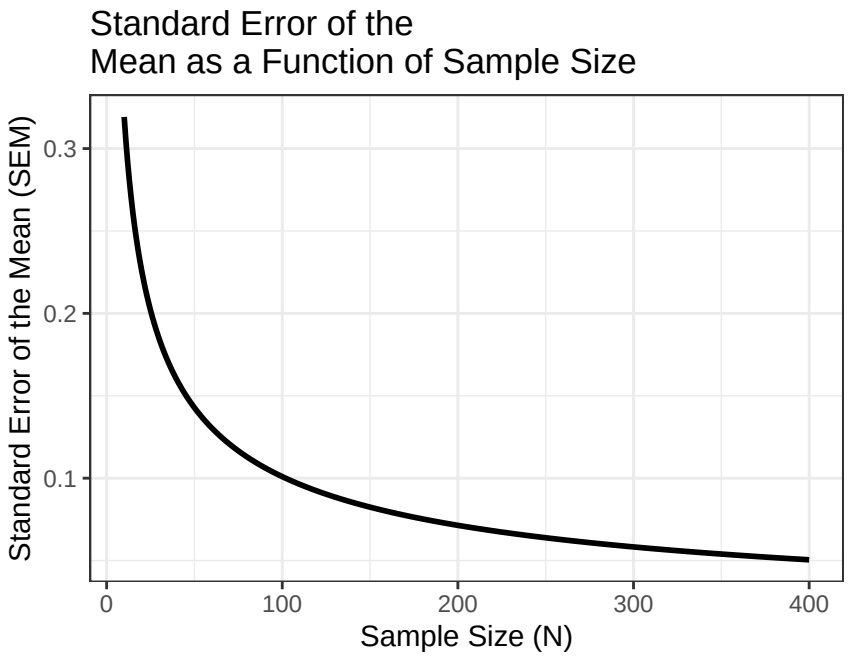


Figure 5.4: The standard error as a function of sample size. The expensive part of the curve is on the left.

So look at that, my error drops really close to zero pretty quickly. Past about 400 the curve is basically crawling: going all the way out to 10,000 riders buys me a standard error of 0.01 instead of 0.05.

So how many people would I need for this study?

The returns diminish.

- When the sample size is small, the error is large and unstable.
- As the sample size gets very large, the error continues to shrink—but only a little at a time.

> ⚠ A Larger Sample Can Produce Very Precise Nonsense
>
> Increasing N reduces random sampling error. It does **not** repair:
>
> - biased sampling;
> - bad measurements;
> - confounds;
> - dishonest responses; or
> - collecting every participant from the same suspicious subway car.
>
> A large biased sample can estimate the wrong value with extraordinary precision.

A sample is just a guess about the population. How good that guess is depends on how variable the population is to begin with. If the distribution is tight, you can get away with a smaller sample. If it's spread out, you need a lot more people to pin it down.

You never actually know if your guess is right, which is why replication exists and why we all lose sleep. If you rerun the study and the first two moments (the mean and the variance) keep bouncing around, that's your data politely telling you: get more people. This is the ball from the probability chapter, and the standard error is our estimate of how far it bounces.

Statistics doesn't give you truth, but you can make increasingly expensive guesses.

5.1.7 What Was the Truth?

I told you in the last chapter that *truth* is a river. In reality that means you will **never** actually know the true population value unless you are either

- (a) an all-knowing God, or
- (b) good friends with said God.

Luckily, if you properly appease *Dorkaos*, the Ancient Greek God of Statistics, knower of all true distributions, with sufficient sacrifices, you may briefly glimpse the truth. Below, all is revealed to you, the future acolyte:

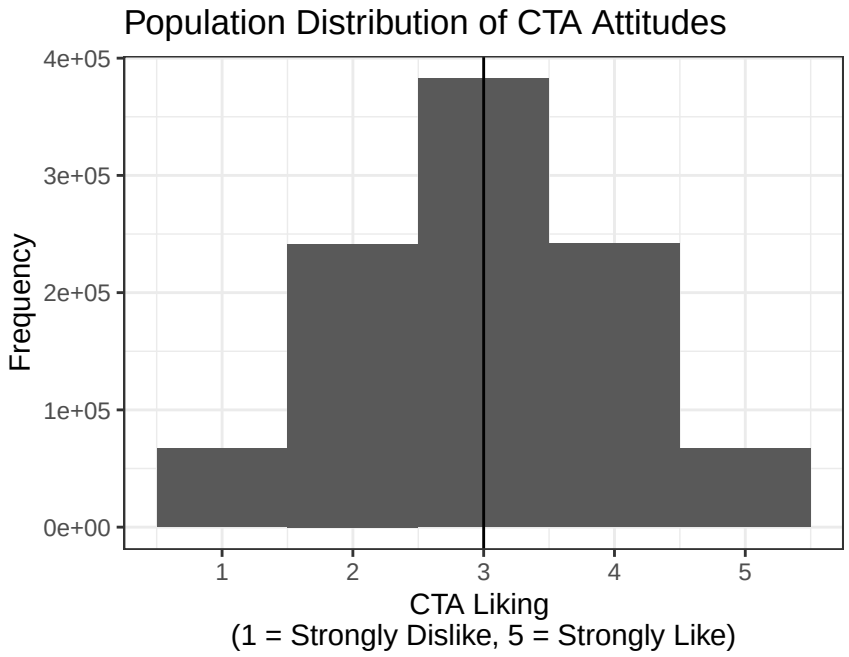


Figure 5.5: The truth, briefly visible. All one million riders, with the population mean marked.

The true mean of this population distribution is $\mu = 3$.

Note: Most people are not on speaking terms with Dorkaos, so in real life you would never truly know this value.

So the real question is: **How close did we come with our original samples?**

From sample 1

- $N = 10$,
- $M_{Sample1} = 3.2$,
- $S_{Sample1} = 1.32$,

- $S_M = 0.42$,
- So our 95% confidence guess (i.e., ± 2 SE) would be 2.37 to 4.03

From sample 2

- $N = 10$,
- $M_{Sample2} = 3.3$,
- $S_{Sample2} = 0.82$,

- $S_M = 0.26$,
- So our 95% confidence guess (i.e., ± 2 SE) would be 2.78 to 3.82

Importantly, both intervals include the true population mean. That doesn't mean we were brilliant; it means the intervals were wide. How do we shrink them?

5.1.8 What If We Sampled More People?

What if we ran a third sample with 100 people?

```
set.seed(26)
SampleSize <- 100
sample.3 <- sample(CTA.Population, SampleSize, replace = FALSE)

M3 <- mean(sample.3)
SD3 <- sd(sample.3)
SEM3 <- SD3 / sqrt(SampleSize)
```

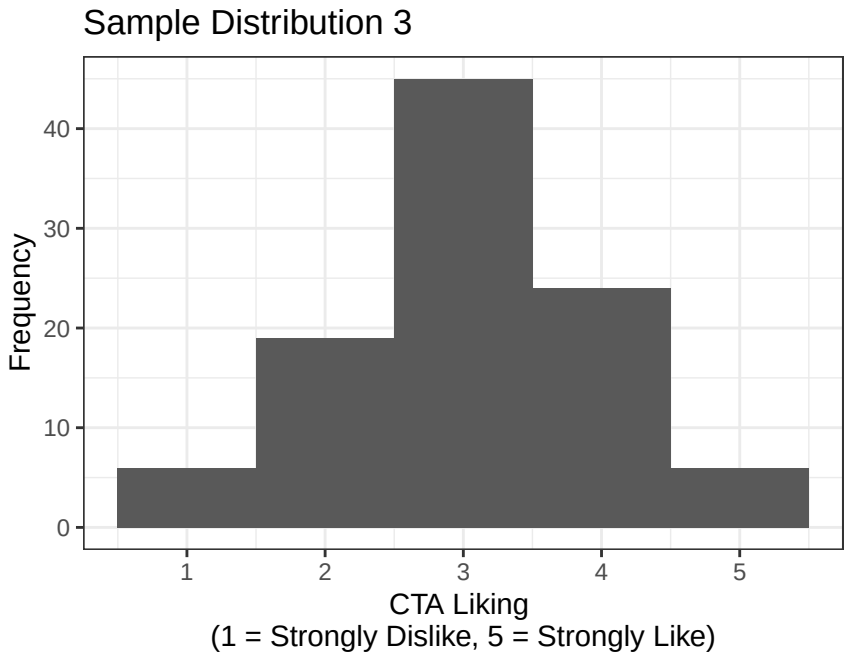


Figure 5.6: One hundred riders. Same population, same question, a much smoother picture.

Remember: $S_M = \frac{S}{\sqrt{N}}$

Sample 3:

- $N = 100$,
- $M_{Sample3} = 3.05$,
- $S_{Sample3} = 0.96$,
- $S_M = \frac{0.96}{\sqrt{100}} = 0.1$.

Our 95% confidence guess is now: 2.86 and 3.24

And now you can see the key difference: the interval is much narrower.

5.1.9 Final Report (to the CTA)

So, at last, I can report that the average Chicagoan is roughly neutral toward the CTA, with a true rating that likely falls somewhere between: 2.86 and 3.24.

Still just a guess, but now a much better one as you can see below:

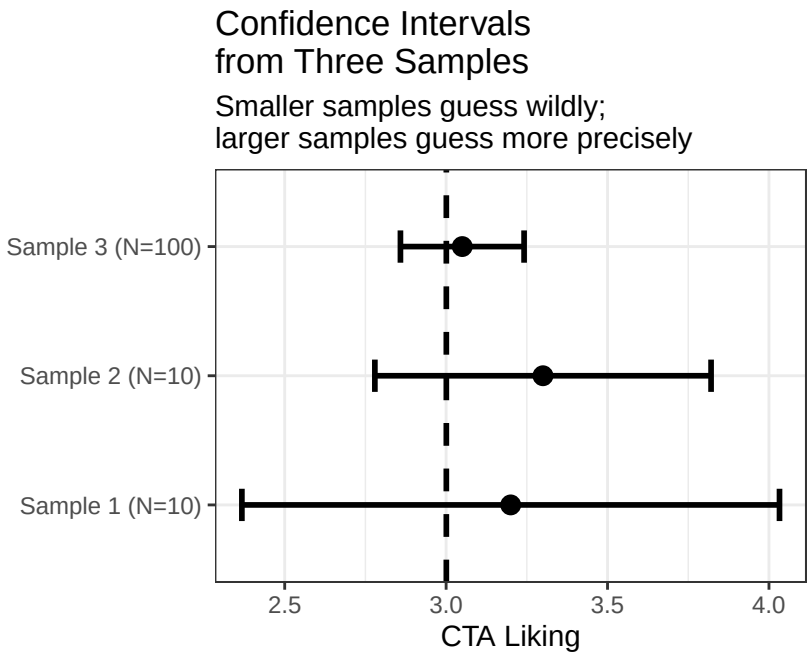


Figure 5.7: Three guesses at the same number. The dashed line is the truth, which we are not normally allowed to see.

5.1.10 What do we do with this?

Pay loans, buy groceries, get the Porsche if there is money left over?

Yes, but also all of this stuff (SDs, SEs, sampling distributions, confidence intervals) exists for one reason: *to let us draw inferences about a population using imperfect data.*

We rarely get to see the truth. Instead, we:

- take a sample,
- quantify how uncertain that sample is, and
- ask whether what we observed is plausibly just random noise or something more interesting.

! What a 95% Confidence Interval Means

Across many repeated samples, a correctly constructed 95% confidence-interval procedure captures the true population value about 95% of the time, assuming the model is appropriate.

After we calculate one interval, we do not say there is a 95% probability that the fixed population value is inside it. The procedure has 95% long-run coverage. The parameter is not wandering around trying to escape.



This Misinterpretation Is Extremely Common

Hoekstra and colleagues gave six incorrect statements about confidence intervals to 442 first-year students, 34 master's students, and 118 researchers (Hoekstra et al. 2014). Only 2% of the first-year students, none of the master's students, and 3% of the researchers correctly rejected all six statements.

Training and research experience did not protect people very well. Therefore, stop and read the definition above again. A confidence interval describes the long-run performance of a procedure. It is not the probability that this one completed interval contains the fixed parameter, the probability that the null hypothesis is true, or the range that contains 95% of individual scores.

But sometimes we want to ask a sharper question: *Is this sample consistent with a specific claim about the population?*

That question is exactly what t-tests are designed to answer.

5.1.11 The Short Story

- The standard deviation describes variation among observations.
- The standard error describes variation in an estimate across hypothetical samples.
- Increasing sample size usually makes the standard error smaller at a rate of $1/\sqrt{n}$.
- Larger samples make estimates more stable; they do not purify biased samples with statistical holy water.
- Confidence intervals show uncertainty in the estimate, not a force field around truth.

6 One Sample t-Test

6.1 Experimental Logic

In psychology, we test claims by making comparisons:

> Sometimes we compare a group to a known value: *Do more frequent train rides increase CTA liking above the neutral baseline of 3?* → one-sample t-test.
>
> Sometimes we compare two groups: *Does CTA liking differ between people in quiet vs. regular subway cars?* → independent samples t-test.
>
> Finally, we also compare people to themselves groups: *Do you like to ride the CTA more in the summer or winter?* → paired samples t-test.

Either way, the logic is the same: **are the differences we observe larger than we'd expect from random sampling noise alone?**

6.1.1 Logic behind Hypothesis testing: The Blind Men and the Elephant

A group of blind men were traveling together when they encountered a strange object blocking their path. Each man touched a different part of the object and described what he thought it was, based entirely on his limited experience.

One man, feeling the elephant's trunk, exclaimed, *"This must be a thick branch of a tree!"* Another, grasping the elephant's leg, disagreed: *"No, it's clearly a pillar!"* A third, holding the elephant's ear, insisted, *"You're both wrong—it's obviously a fan!"*

Each man was convinced he was right, believing his own perspective represented the full truth. Their disagreement quickly escalated into an argument, with each trying to prove the others wrong. Eventually, the elephant, thoroughly unimpressed, grew irritated by their foolishness and tossed them all into a nearby river for disturbing its peace.

— **A paraphrased ancient Indian parable**

This story is a useful reminder when trying to understand the world, and especially when building theories from data. Our perspectives are almost always limited, shaped by where we stand, what we've experienced, and what we want to believe. Just as each blind man grasped only one part of the elephant, our samples and hypotheses capture only small pieces of a much larger reality.

Statistical testing exists precisely because of this limitation: *it gives us a disciplined way to ask whether our small, noisy glimpses are consistent with the bigger picture, or whether we might just be arguing over trunks, legs, and ears.*

6.1.2 Experimental Example

- *Step 1*: Begin with a theory.
- *Step 2*: Develop a hypothesis derived logically from your theory.
- *Step 3*: Design a method to test your hypothesis, including measurements, sample recruitment, and analytical tools.
- *Step 4*: Collect data and compare it with your prediction.

When Step 4 fails, meaning the data don't match the prediction, we face a classic scientific dilemma. It is often difficult to determine why things went wrong:

- Was the theory flawed?
- Was the hypothesis poorly specified?
- Or were the methods inadequate?

This problem is well known in psychology and has been discussed extensively (Meehl 1978).

6.1.2.1 (Bad) Theory Example

Reinforcing good behavior in children makes them happy because it signals that they have pleased the adults around them. However, rewarding children with something they *perceive* as a forbidden treat, such as cookies, may have unintended consequences. Children know that cookies are usually off-limits and therefore associate them with rule-breaking. Receiving such a reward may overexcite them, pushing their self-control beyond its limits and leading to hyperactive behavior as they struggle to regulate their impulses.

6.1.2.2 Hypothesis

Based on this theory, we hypothesize that giving children cookies will cause them to lose control of their impulses and become monsters climbing the walls and terrorizing the cat.

To test this hypothesis, we design a simple study in which children are rewarded with chocolate chip cookies and their behavior is observed.

> **!** Science Proves Nothing!
>
> Regardless of the study's outcome, it would not rule out other plausible explanations. For example, one alternative account *(and totally wrong argument I think I heard some one say in the supermarket)* is that the refined sugar in cookies is rapidly metabolized, producing a temporary spike in blood glucose that leads to increased activity levels.
>
> This is the main difficulty of experimental inference: a single result rarely tells us which explanation is correct. It only tells us which explanations are still plausible. Thus we have theories with evidence for or against. Never Proof! Only in mathematics can you have "proof" and remember math is not science.

6.1.3 Logic of Hypothesis Tests

If X is true, then Y should also be true.

- We assume X is true.
- Therefore, we *expect* to observe Y.

However:

- If we observe Y, this does not necessarily prove that X is true, because Y could result from other factors.

- If we do *not* observe Y this only *suggests* that our theory may be flawed or incomplete.

So back to our example!

Theory: Reinforcing good behavior in children with treats leads to increased hyperactivity because children perceive the treat as a forbidden reward, triggering an excited and poorly controlled response

Hypothesis: Giving children chocolate chip cookies as a reward will result in increased hyperactivity compared to children who do not receive such rewards.

Testing the Hypothesis: If children who receive cookies show increased hyperactivity (Y is observed), this outcome is consistent with the theory but does not conclusively prove it, since alternative explanations (e.g., sugar content) could also account for the behavior.

However, if children **do not** show increased hyperactivity (Y is not observed), this challenges the theory, suggesting it may be incorrect or incomplete. At that point, we must re-examine the theory, the methods, and consider alternative explanations.

In hypothesis testing, **falsification** plays a central role. Supporting evidence increases our confidence in a theory, but evidence that contradicts our predictions undermines it. Scientific progress depends on continuously testing and attempting to falsify our theories, refining them through scrutiny and new evidence.

6.1.4 4 Steps for Hypothesis Testing

- State the hypothesis
- Set the Criteria Decision
- Collect Data and Compute Sample Statistics
- Make a Decision

6.1.5 The Null Hypothesis (H_0)

- The null hypothesis (H_0) asserts that in the general population, there is no change, no difference, or no relationship.
  - In an experiment, H_0 predicts that the independent variable (e.g., treatment—cookies) will have no effect on the dependent variable (e.g., hyperactivity) for the population.
- H_0 is the hypothesis that is subject to a final decision: to either *reject* or *fail to reject*.

6.1.6 The Scientific or Alternative Hypothesis (H_1)

- The alternative hypothesis (H_1) suggests that there is a change or a relationship in the general population.
  - In an experiment, H_1 predicts that the independent variable (e.g., treatment—cookies) will influence the dependent variable (e.g., hyperactivity) for the population.
- We are not actually directly testing H_1 which is why we either *reject* or *fail to reject* H_0.

⚠ Accepting the Null or Proving the Alternative?

Why do we use that bizarre lingo: reject or fail to reject the null? Is it cause statisticians have the personalities of androids? Well, partially. Why can't we say *accept null* or *accept/prove the alternative*, or just say in plain English, "Yo dude, this study shows we got nothing!"

To explain, let's use those very scientifically informed ghost hunting TV shows which are so much fun to watch *(don't judge me)*. Two hosts creep around an old house. The EMF meter spikes, one of them feels a cold spot, and they decide the place is haunted by Ronald Fisher, the statistician who invented ANOVA, who presumably cannot rest because people keep trashing his equations. Then the spirit box, which scans radio stations at random, coughs up ...static... "Analysis"...static..."Variance"...static... They run out convinced.

What is wrong with our hosts? Damaged pre-frontal cortex? No, they are just normal people making normal people errors of logic. First, humans want a *cause* for what they just experienced (see probability chapter). Second, it is impossible to prove that nothing is there. You cannot see a ghost or touch one, so the disproof never arrives, and the EMF meter, the cold spot, and the spirit box all get promoted to evidence. There were 1001 other explanations available: wires in the wall, a draft in an old house, a psychosomatic chill, and pareidolia, which is humans finding patterns out of randomness. But they theorized Fisher, they heard Fisher's favorite words, so it must be Fisher. They **accepted the alternative**.

Now have I proven there are no ghosts? Absolutely not! There could indeed be ghosts. Seriously, not a joke. No scientist can ever prove there are no ghosts. We cannot prove anything. If a scientist says they can, they are liars who forgot the basis of the scientific method. Remember, we can only provide evidence toward our theory.

6.1.7 Directional Hypothesis

- H_0: Chocolate chip cookies do not make children hyper.
- H_1: Chocolate chip cookies do make children hyper.

Written symbolically, where μ is the population mean hyperactivity score and 50 is the value we expect without cookies:

- $H_0 : \mu \leq 50$
- $H_1 : \mu > 50$

6.1.8 Non-directional Hypothesis

- H_0: Chocolate chip cookies do not change children's level of hyperactivity.
- H_1: Chocolate chip cookies do change children's level of hyperactivity.

Written symbolically:

- $H_0 : \mu = 50$
- $H_1 : \mu \neq 50$

Note: The default in psychology is to choose the non-directional test, for reasons we need to come back to later, so we will only talk about that version for the rest of below.

6.1.9 Set the Decision Criteria

- The decision criteria depend on the type of test, the number of tests conducted, the sample size you have, and whether the hypothesis is directional or non-directional.
- A key decision is determining how far sample means need to be from the population mean to reject the null hypothesis (H_0).

- The challenge is identifying what constitutes *abnormal* behavior in the population.

> **Abnormal Is Not a Moral Judgment**
>
> "Abnormal" is a technical term used by statisticians to describe when a score occurs outside of the "norm". Go back to the distribution chapter to find out what the technical term *normal* means.
>
> Typical rule is anything outside of 2 SD (for a person) or 2 SE for sample mean is often described as "abnormal", meaning a score that would occur outside of the 95% range that most of the people or sample means tend to fall in by chance.

Before I show you with a picture what I mean, I need to explain another small problem. Can I assume in small samples that the mean I get will be accurate? We already know the answer from the standard error chapter, no. I know in small samples I can be very uncertain about the mean. But also, I will be uncertain about its variance (spread). *Think about this example:*

> You collect 10 people in Giordano's eating deep dish pizza. By chance 9 of the 10 people are from Chicago, love deep dish and give high scores, 8 to 9.5 out of 10 with an average around 9 of 10. But that last person is a tourist from New York City. They rate it 0 out of 10 and ask if they can actually give it a -1000 out of 10. *In fact, my visiting college friend from NYC, who is a Medical Doctor, told the waitress that pizza was not only an affront to pizza but a serious health hazard and suggested they close their doors and save people's taste buds and arteries.*

What happens to the mean with that rating of 0 out of 10? Let's take a look at the scores with the tourist and without:

```
set.seed(242)
Chicagoan <- rnorm(9, mean = 9, sd = .5)
NewYorker <- 0

Full.Data <- c(Chicagoan, NewYorker)
```

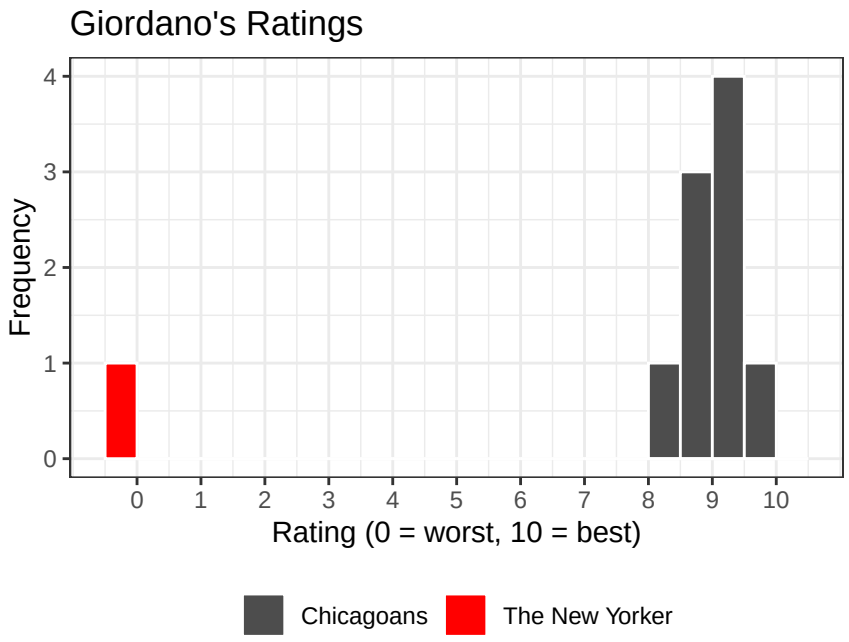


Figure 6.1: Ten pizza ratings at Giordano's. Nine Chicagoans, and one visitor in red.

The mean with just Chicagoan is 9.03 and if you add my friend, 8.12. So it drops

But look what happens to the SD:

The SD with just Chicagoan is 0.49 and if you add my friend, 2.89.

Well that is very different.

Recall our estimate of the standard error of the mean is estimated as follows: $S_M = \frac{S}{\sqrt{N}}$

SO with all 10 people, the $S_M = 0.91$

If that one tourist had never walked in and only the 9 Chicagoans were measured, we would have found a very different and much *smaller* standard error, $S_M = 0.16$

We are back to which is right? Take for example now you want to compare how people rate Giordano's to the national average for pizza, which is 7.5 out of 10. In the case of only sampling Chicagoans you would ask: is Chicago pizza "better" (i.e., abnormal) than the average American pizza? Here is the same question asked twice, once with each standard error.

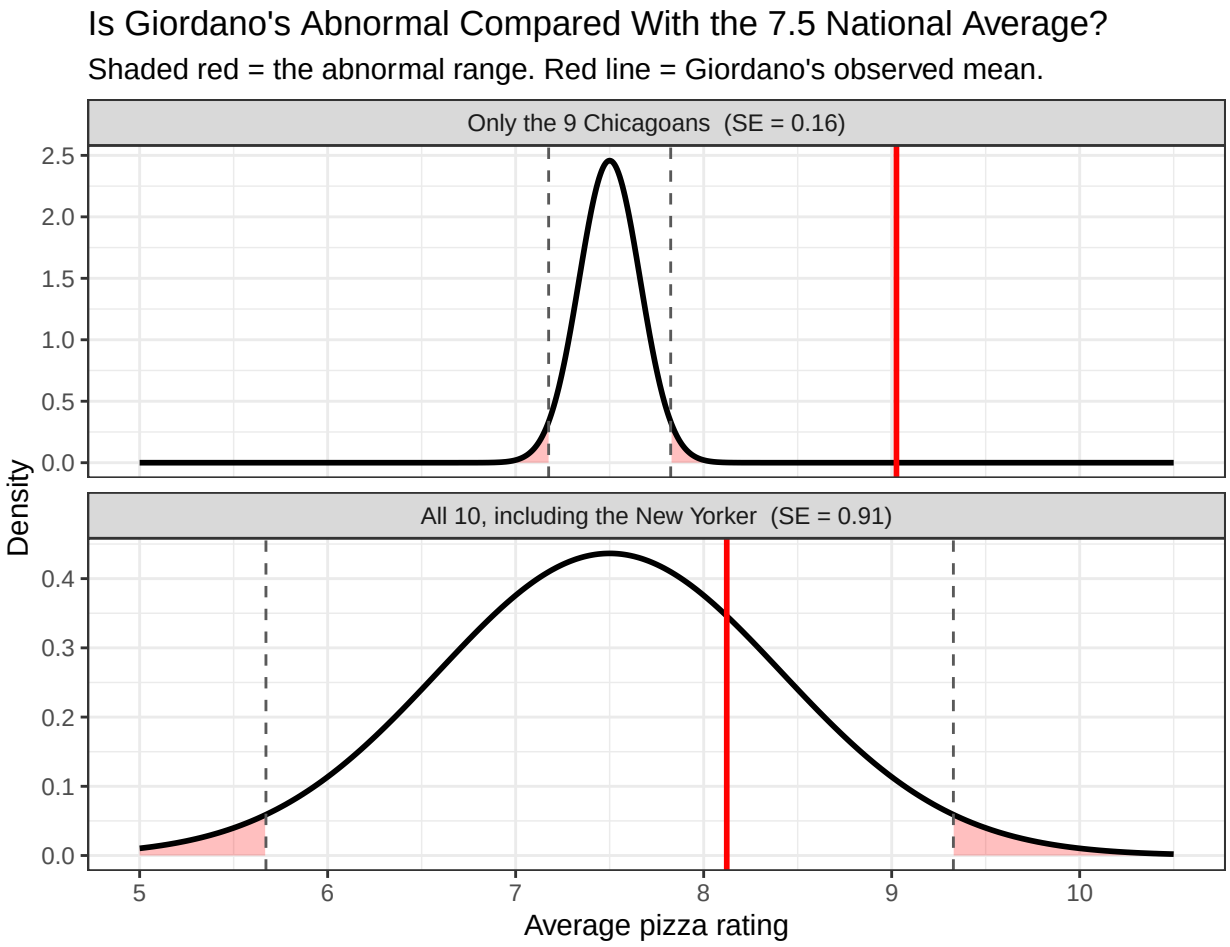


Figure 6.2: The same question asked twice. Both curves are centered on the national average of 7.5; only the standard error differs. The red line is what Giordano's actually scored.

As we can see, in one case Chicago pizza is in the abnormal range, outside of 95% of our estimates of the average American pizza score. In the other case, that one New Yorker makes Chicago pizza seem average. We are back to which is true?

Well, this problem was solved in 1908 to test Guinness Beer!

6.1.10 Accounting for Sample Size: The Transition to the t-Distribution

We also have to account for the sample size of our study, since in small samples, we know we will not get the best estimate of the population numbers (as we saw above). So, we apply a correction by moving from the standard true Normal (z) distribution to the Student's t-distribution.

> **Student's t-Test Was Created Because of Beer**
>
> The word *Student* does not refer to an undergraduate who invented a test between classes. It was the pen name of William Sealy Gosset, a chemist and brewer at Guinness. Guinness used small samples to study ingredients and production, and company policy restricted employees from publishing their work. Gosset therefore published *The Probable Error of a Mean* as "Student" in 1908 cause he was not allowed to release trade secrets (Student 1908; Lehmann 1999).
>
> In other words, one of the most famous tools in statistics exists because a brewer wanted to make better beer without pretending a tiny sample was a giant one. Statistics has rarely produced a more convincing origin story.

6.1.11 Why the t-Distribution?

We will never know the population standard deviation (σ), so we must estimate it using the sample standard deviation (S). We know that small samples are more prone to extreme fluctuations (as you saw). Because our estimate of the population variance is less reliable when n is small, the t-distribution is "heavier" in the tails than the normal distribution. So the shape of the t-distribution is determined by the degrees of freedom, calculated as $df = n - 1$. As n (and thus df) increases, the t-distribution approaches the shape of the standard normal distribution. As n decreases, the tails become "fatter", so the range holding 95% of the distribution is now much wider than 2 standard errors (actually it's 1.96, but who needs to be exact!).

6.1.12 Adjusting the Critical Region (the regions we call abnormal/weird)

Because the t-distribution has more area in its tails, the "cut-off" points (called alpha (α) level) move further away from the mean. This means:

- It is harder to reject the null hypothesis with a small sample.
- We are being conservative to account for the increased risk of error when we don't have a large amount of data.

The t-distribution acts as a "safety buffer." It ensures that we only label a result as "statistically significant" if the effect is strong enough to overcome the inherent noise of a small sample size.

6.1.13 Determining the Cut-Off Points for Two-Tailed Tests

For a two-tailed test with $\alpha = 0.05$ (i.e., 1 - .05 = .95, or 95% of the population), the cut-offs depend on sample size. For our study with 10 people that would look like this:

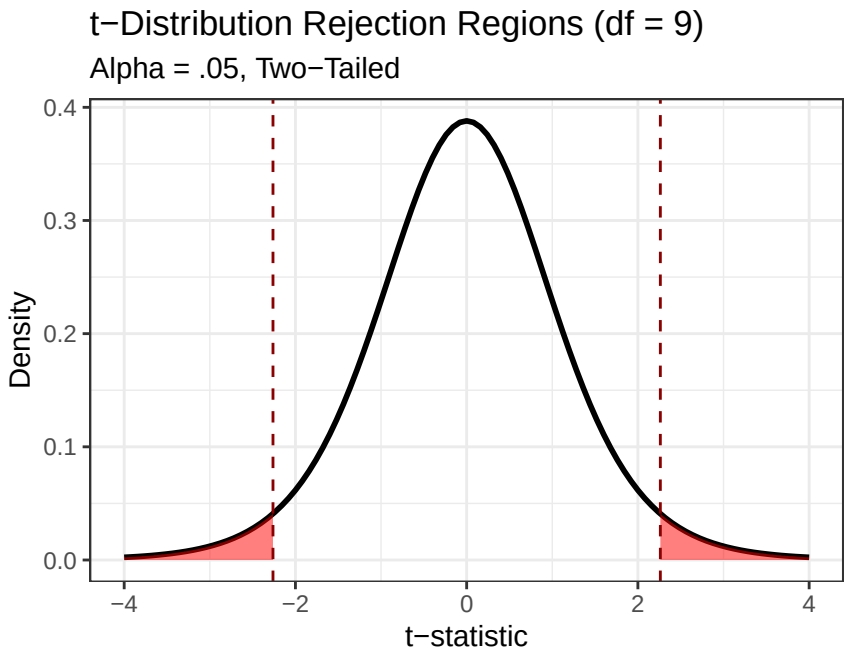


Figure 6.3: The t-distribution for df = 9 (n = 10). The shaded red areas represent the critical regions where we reject the null hypothesis.

Our cut-off ($t_{critical}$) values for $df = 9$ would be:

- t_{lower} < -2.262
- t_{upper} > 2.262

6.1.14 The Impact of Sample Size on Critical Regions

If we were to increase our sample size from $n = 10$ to $n = 100$, our degrees of freedom would increase to $df = 99$.

As n increases, our confidence in our estimate of the population standard deviation also increases. Mathematically, the t-distribution begins to pull its "fat tails" inward, looking more and more like the standard Normal (z) distribution.

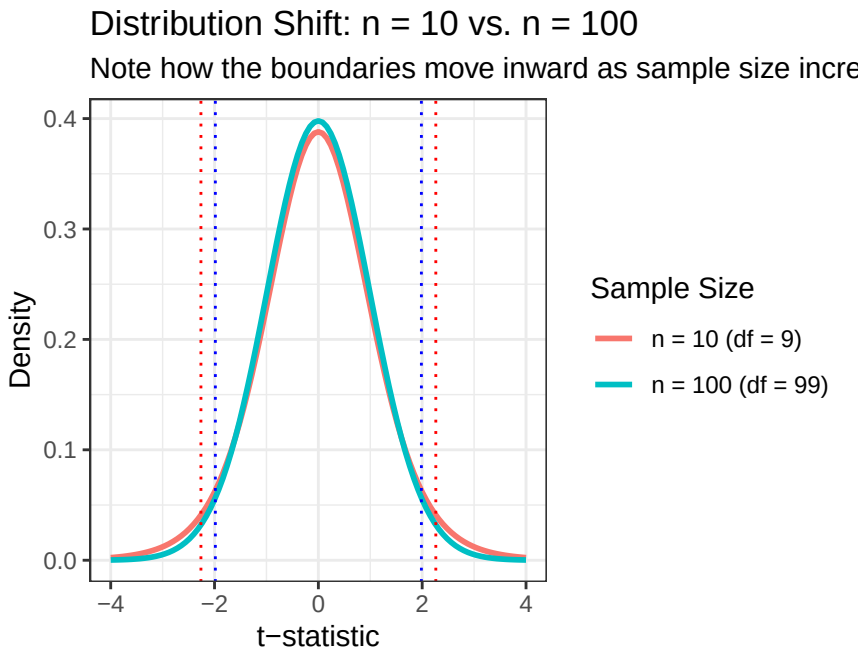


Figure 6.4: Two t-distributions overlaid. The larger sample has thinner tails, so its cut-offs sit closer to the middle.

New Cut-Off ($t_{critical}$) Points ($n = 100$):

- t_{lower} < -1.984
- t_{upper} > 1.984

Why the boundaries are smaller: with 100 people instead of 10, our sample mean is a much more reliable "anchor" for the population mean. We don't need as much of a "safety buffer."

Here is how the boundaries shift with different sample sizes:

Table 6.1: How Sample Size Changes the Threshold for Significance

Sample Size (n)	Degrees of Freedom (df)	Absolute Critical Value
10	9	2.262
100	99	1.984
Infinity (z)	Inf-1	1.960

6.1.15 Collect Data and Compute Sample Statistics

Back to our study on the kids. Remember we wanted to feed them cookies to test our theory that giving them a forbidden treat will make them hyperactive? To test this, we get a sample of children, $n = 10$, and feed them a bunch of cookies! We collected that data and in R stored their hyperactivity scores in an object called `kids.1`

Dependent Variable (DV): We will measure the children's hyperactivity, which we expect the cookies to raise.

Here is a plot of kids.1:

```r
set.seed(123)
N <- 1e6
Population <- rnorm(N, mean = 50, sd = 5)
n <- 10

set.seed(42)
kids.sample <- sample(Population, n, replace = TRUE)

# every kid gets a 2-point cookie boost (the true effect)
kids.1 <- kids.sample + 2
```

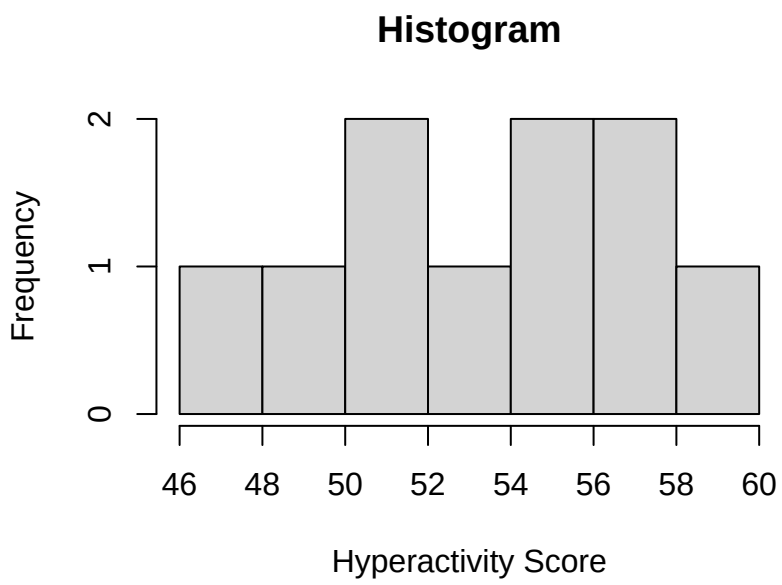


Figure 6.5: Hyperactivity scores for our 10 cookie-fed children.

6.1.16 Sample Results

The children who received cookies showed a mean of $M = 53.049$ with a standard deviation of $SD = 3.995$. Based on prior theory and past research, we hypothesize that the population mean is $\mu = 50$

We can then ask whether the data we observed are consistent with that claim, or whether the difference between the sample mean and the hypothesized population mean is larger than we would expect from random sampling error alone.

6.1.17 Compute Sample Statistics

The test statistic is computed using the following formula:

$$t_{test} = \frac{M - \mu}{S_M}$$

where the denominator is the standard error of the mean:

$$S_M = \frac{s}{\sqrt{n}}$$

We can do the math by hand, or use the *t*-test function in R. *t*-test by hand:

```
M = mean(kids.1)
s = sd(kids.1)

n = 10
Mu = 50 # given to you from the population
t.kids <- (M - Mu) / (s/sqrt(n))
t.kids
```

```
[1] 2.413795
```

one sample *t*-test in R:

```
Kids.t.test <- t.test(x = kids.1, #we put the vector of scores in
                      alternative = "two.sided", #the abnormal value can be on the left or right side
                      mu = 50, #hypothesized population mean
                      conf.level = 0.95)  # .95 is the default; shown here so you can see it

Kids.t.test
```

```
    One Sample t-test

data:  kids.1
t = 2.4138, df = 9, p-value = 0.03901
alternative hypothesis: true mean is not equal to 50
95 percent confidence interval:
 50.19156 55.90708
sample estimates:
mean of x
 53.04932
```

6.1.18 Reading What R Just Handed You

That is the first real block of R output in this book, and it is a wall. Here is every line of it, in order, in English.

```
    One Sample t-test          <- which test R decided to run

data:  kids.1                  <- the object you handed it

t = 2.4138, df = 9,            <- the test statistic and its degrees of freedom
p-value = 0.03901              <- the p-value, exact, not rounded to stars

alternative hypothesis: true mean is not equal to 50
```

```
                                <- your H1, in words. R is repeating your
                                   mu = 50 and your "two.sided" back to you

95 percent confidence interval:
 50.19156 55.90708              <- the interval around the SAMPLE mean

sample estimates:
mean of x
 53.04932                       <- the mean of your data
```

Five things worth noticing, because these are the five things students get wrong:

1. **`t = 2.4138` is the number you just computed by hand.** Scroll up. It matches. R did not do anything mysterious, it did the same division faster.
2. **`df = 9`, not 10.** Degrees of freedom are $n - 1$. You have ten children and nine degrees of freedom, and this is the number you take to the t-table, not your sample size.
3. **The `alternative hypothesis` line is R repeating your own instructions back to you.** It says "not equal to 50" because you typed `mu = 50` and `alternative = "two.sided"`. If that line does not describe the test you meant to run, you typed the wrong thing, and this is the line that tells you so. Read it every single time.
4. **The confidence interval is around your sample mean, not around 50.** It runs from 50.19 to 55.91, and the interesting thing is what it does *not* contain: 50, our hypothesized value, sits just outside the bottom end. That is the same conclusion as the p-value, arriving by a different road.
5. **`sample estimates: mean of x` is just the mean of your data.** R labels it `x` because that is the argument name you filled in. It is not a new quantity, it is `mean(kids.1)`.

What R does *not* give you: an effect size. There is no d anywhere in that output. That absence is the whole reason for the next chapter.

6.1.19 Test Statistic Result

We get a $t_{test} = 2.414$. Now, the question is: **does this value fall in the tail of the population distribution**?

Let's check our critical value.

Our $t_{critical}$ values for $df = 9$ would be:

- $t_{lower} <$ -2.262
- $t_{upper} >$ 2.262

This t-test value is greater than our upper value.

Further from the R t-test function, we can see our exact p-value which is below $p < .05$.

! The p-Value Defined at Last

Back in Chapter 1 I promised you a real definition of statistical significance once you had some probability theory. Here it is, at the first moment you actually need it.

The *p*-value is the probability of getting a test statistic at least this far from the null value, *if* the null hypothesis and the model's assumptions were true.

Look back at the shaded red tails in the figure above. Your observed t lands somewhere on that curve, and the p-value is the area past it in both directions. Small p means your data sit far out in a region the null model rarely produces.

Note what the definition does not say. It is a statement about the data given an assumption, not a statement about the assumption given the data. It is not the probability that the null hypothesis is true, not the probability that you are wrong, and not the probability that the result will replicate.

We can also see the 95 percent confidence interval from our R t-test function, and that range does NOT include $\mu = 50$, which is another way of seeing that our result is bigger than chance alone would produce.

6.1.20 Make a Decision

There are two possible outcomes:

- **Reject the null hypothesis:** This decision is reached when the data fall within the critical region.
  - This indicates the sample mean is farther from the hypothesized value than random sampling error can comfortably explain. Whether the *cookies* did it is a design question, not a statistics question. See the Important Note above.
  - **We reject the null** because it is generally easier to demonstrate that something is false than to prove it true. (For the other way of weighing evidence, see the Bayes section of the Probability chapter.)
- **Fail to reject the null hypothesis:** If the data do not provide strong evidence (i.e., they do not fall within the critical region), then the treatment is *assumed* to have no effect.

6.1.21 Decision Outcome

YES, our $t_{test} > t_{critical}$, so we REJECT the null hypothesis.

- We report: $t_{test} = 2.414$, $p = 0.039$ (since we know the exact p-value).
- In modern APA format, it is recommended to report the exact p-value.

6.1.22 One-sample *t*-test in APA format

Written the way it would appear in a paper, our result looks like this:

$t(9) = 2.41$, $p = 0.039$, 95% CI [50.19, 55.91], $d = 0.76$

Wait, what is the d thing at the end? The problem with significance testing is it does not tell you how "big" the effect is. It just tells you the result is probably not due to chance.

The d Is a Cliffhanger, and the Next Chapter Is the Payoff

That d is **Cohen's d**, an *effect size*. All you need for now is the one-sentence version: it reports how far apart things are in standard deviation units, so a d of 0.76 means our cookie-fed children sit about that many standard deviations above the value we tested against.

Report it beside your p-value, every time. A p-value tells the reader that *something* is there. Only the effect size tells them whether it is worth crossing the room for.

Where it comes from, why radar operators invented it, what counts as small or large, and what it costs you to detect one, all of that is the next chapter. It is the reason the next chapter exists.

6.2 APA Style Report

Children who received cookies showed higher hyperactivity scores ($M = 53.05$, $SD = 3.99$) than the hypothesized population value ($\mu = 50$), $t(9) = 2.41$, $p = 0.039$, 95% CI [50.19, 55.91], $d = 0.76$. The standardized difference describes magnitude; whether the effect is practically meaningful requires context and better evidence than our imaginary cookie operation.

6.3 Uncertainty and Error in Hypothesis Testing

Hypothesis testing is an inferential process because we do **not** know the actual population data, and our samples could be in error. When we infer, we can make at least two types of errors (though there are others).

6.3.1 Type I Error

Type I Error, also called a "false alarm" or "alpha error" occurs when a researcher rejects a null hypothesis that is actually **true**. In other words, we reject the null even though the treatment does **nothing**. Before observing the data, the α level sets the long-run Type I error rate for the testing procedure when the null hypothesis and its assumptions hold. Choosing $\alpha = .05$ does not mean there is a 5% probability that this particular rejected null hypothesis is true. The p-value cannot read the hidden state of the universe.

Example: Testing zinc or ColdEase on colds using self-report as a measure of improvement. Colds get better on their own, and people who paid for zinc lozenges want to believe they helped. So the study can happily "detect" an improvement that is really just the cold ending on schedule plus some hopeful self-report. The null was true and we rejected it anyway.

6.3.2 Type II Error

Type II Error, also called a "miss" or "beta error"occurs when a researcher fails to reject the null hypothesis that is actually **false**. In other words, a Type II error occurs when the treatment has an effect, but it seems like the treatment has had no effect because the hypothesis test fails to detect it. This can occur when the treatment has an effect but the study has too little power to detect it (i.e., too few parcipants to see the effect given how much noise/chance you have at getting a sample mean and SD from the sample). That happens when the effect size is small, the measurement is noisy, or the sample is small.

- **Example:** Taking kava tea to reduce stress. Kava may genuinely calm people down a little. But give it to ten stressed undergraduates, measure them with a mood scale they fill out between classes, and the effect is too quiet to hear over the noise. The null was false and we failed to reject it.

The Courtroom Version, Which Your TA Will Use

A defendant is presumed innocent, which is the null hypothesis. The jury either convicts or fails to convict.

- **Type I error:** convicting an innocent person. You rejected a true null.
- **Type II error:** acquitting a guilty person. You failed to reject a false null.

Notice that "not guilty" is not the same verdict as "innocent." The jury is saying the evidence was not strong enough, which is exactly what "fail to reject" means. This is also why raising the standard of proof trades one error for the other. Make conviction harder and fewer innocent people go to prison, while more guilty ones walk.

You met the extreme version of this in the Probability chapter, where we fired the doctors and let a 95% accurate test convict people Judge Dredd style. It got 84% of them wrong.

Your decision	Treatment does not work (H_0=TRUE)	Treatment does work (H_0=FALSE)
Reject H_0	Type 1	Correct Decision
Fail to reject H_0	Correct Decision	Type II

6.3.3 The Short Story

- A one-sample t-test compares a sample mean with a hypothesized population value.
- The t statistic is the observed difference divided by its standard error.
- A p-value measures compatibility with the null model; it is not the probability that the null is true.
- "Fail to reject" does not mean "prove no effect." Sometimes the study simply arrived carrying a tiny sample and no flashlight.
- Report the estimate, confidence interval, p-value, and effect size. The asterisk cannot do all the talking.

The t Statistic Is Signal Divided by Noise

The numerator is how far the sample mean sits from the null value. The denominator is the standard error of that mean. A large t statistic can come from a large difference, a precise estimate, or both, so report the effect and its uncertainty, not merely the ceremonial p-value.

What Comes Next?

Straight ahead is the power and effect size chapter, and it is not optional scenery. It explains the d we just dangled in front of you, it covers Type I and Type II error, and it answers the only question anyone actually asks before running a study: how many people do I need? Read it before the other two t-tests, because both of them assume it.

After that, the remaining two t-tests are variations on what you already know.

Use the one-sample t-test when one group is being compared with one number. If you want to ask whether **Group A differs from Group B**, continue to the independent-samples t-test chapter. For example, you might compare the running speed of one group of undergraduates chased by a goose with a completely different group chased by two geese. The ethics board will compare your application with a paper shredder.

If you measure the **same people twice**, continue to the paired-samples t-test chapter. For example, you might measure the same students' stress before and after telling them that the group presentation they forgot about has been moved to today. Same humans, two measurements, one sudden desire to transfer universities.

7 Power and Effect Size

Some of Today's Code Is Not Your Problem

A few figures in this chapter are generated with genuinely unpleasant code. Where you see a plot appear without the code attached, that is deliberate: I hid it because you are not expected to learn it, yet. Every function you *are* expected to learn is shown, explained, and used more than once.

If you find yourself reading the hidden code anyway, good for you! Dorkaos, the Ancient Greek God of Statistics, is watching, and approves.

Where the Exam Material Lives in This Chapter

This is the longest chapter in the book, so here is the map before you start.

- **First section**: *why* power exists, which comes down to the fact that small samples flatter you. Conceptually the hardest part.
- **The middle**: the vocabulary that most people put on the exams (Type I and Type II error, α, β, power) and how to compute power in R. If you are short on time, start at "Type I and Type II Error" and read forward.
- **Advanced Topics at the end**: for when someone you love runs a pilot study. Not Optional for graduate students.

7.1 Probability Is Fickle

Before we can talk about power, we have to admit something uncomfortable about small samples.

Recall that the standard error is the standard deviation of sample means. Let us build a population for children's hyperactivity scores, the same population from the one-sample t-test chapter, with $\mu = 50$ and $\sigma = 5$.

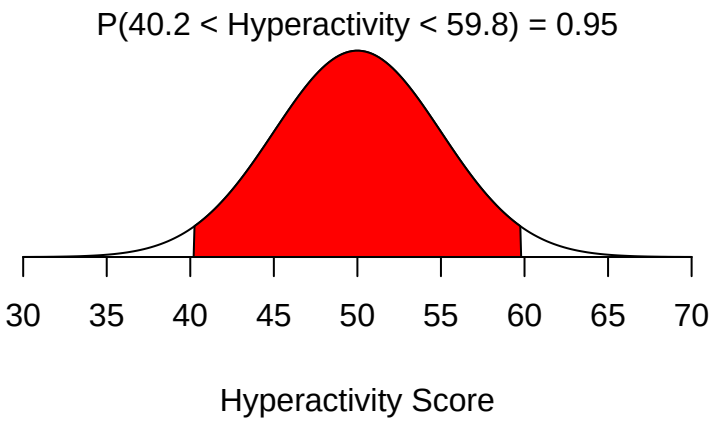


Figure 7.1: The population of hyperactivity scores, with the middle 95% shaded.

About 95% of children score between 40.2 and 59.8. Fine. Now let us collect some data.

7.1.1 Ten Small Studies

Suppose we run ten separate studies, each with $n = 13$ children. Ten data collections, thirteen children each, one hyperactivity score per child.

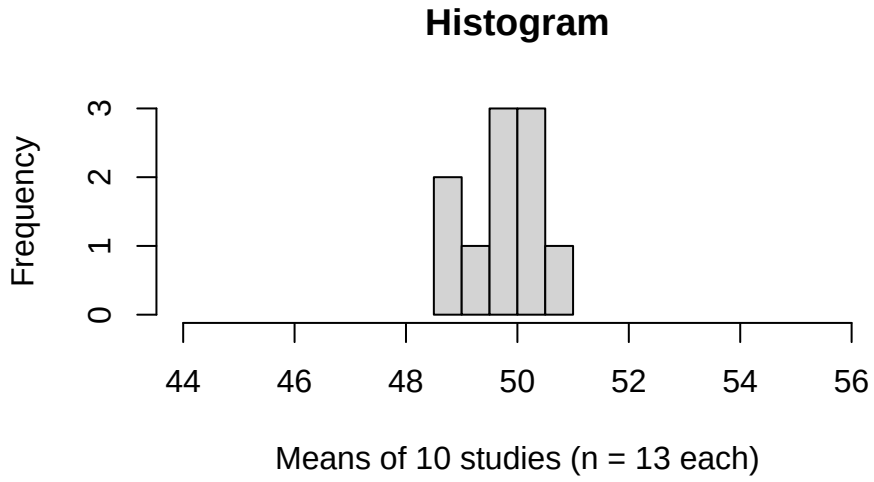


Figure 7.2: Ten studies, ten means. None of them is 50, and that is the point.

The mean of those ten sample means was 49.721, comfortably close to $\mu = 50$. Good.

Now check the *spread*. Our formula predicts a standard error of

$$\sigma_M = \frac{\sigma}{\sqrt{n}} = \frac{5}{\sqrt{13}} = 1.39.$$

In reality, those ten studies produced $SD(\text{sample means}) = 0.73$.

Those are not the same number. **Why?**

7.1.2 The Answer Is Unpleasant

Two different things are bouncing around here, and they are worth keeping apart, because this is exactly the two-counts trap the standard error chapter warned about.

The first is not very interesting. We computed a standard deviation from only **ten numbers**, the ten study means. A standard deviation built from ten values is itself a shaky estimate, and some of that gap is just that.

The second one is the reason this chapter exists. The same disease infects the S that any single study computes from its **thirteen children**. With only thirteen people, you are most likely to grab from the fat middle of the distribution, because that is where most people live. The extremes are rare, so small samples usually miss them. Miss the extremes, and your estimate of spread comes out too small.

Let us prove that second one with brute force. We will run 5,000 separate studies of thirteen children each, and look at the standard deviation each study reports back:

```
set.seed(543)

# Run 5,000 studies. Each one grabs 13 children from a population where we
# KNOW the true spread is sigma = 5, and reports the SD it happened to see.
Sample.SD <- replicate(5000, sd(rnorm(n = 13, mean = 50, sd = 5)))

hist(Sample.SD,
     main = "Histogram",
     xlab = "SD reported by each study \n(5000 studies, n = 13 each)",
     ylab = "Frequency")
abline(v = 5, col = "red", lwd = 4)   # the TRUE sigma
```

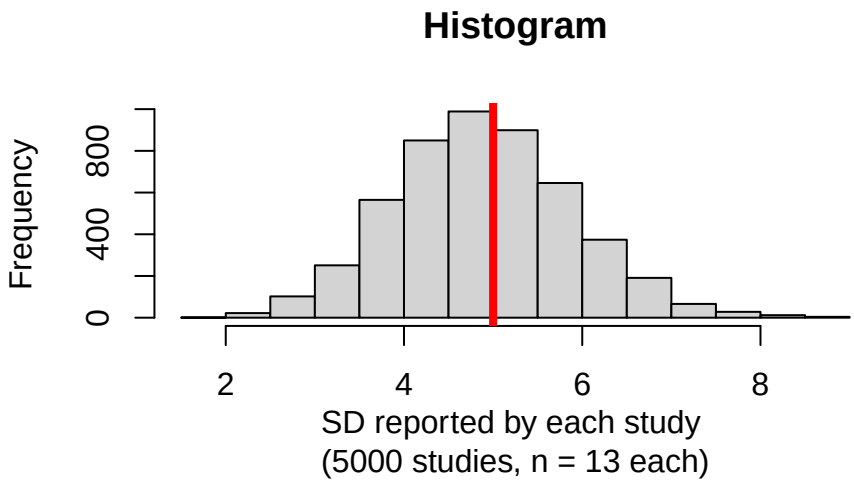


Figure 7.3: What 5,000 small studies report as their SD, against the truth in red.

The red line is the truth, $\sigma = 5$. Notice the histogram is **not centered on it**. In this simulation 55.6% of studies reported a standard deviation *below* the true value and only 44.4% landed above it. The average study reported 4.89 when the truth was 5.

Rounding did not do this. It is a known bias. The theory predicts it almost exactly. For samples of 13 from a normal population, the expected value of S works out to 4.897. Our 5,000 simulated studies averaged 4.893. The math and the brute force agree to three decimal places, which is the kind of thing that should make you trust both.

Small samples systematically underestimate variability. This is the problem Gosset walked into when he stopped using a known σ and started using a sample S as an approximation, and it is why we have a t-distribution at all.

> **!** This Is the Whole Reason Power Exists
>
> Your sample gives you one guess about the mean and one guess about the spread. Both guesses bounce around, and the smaller the sample, the harder they bounce. Worse, the guess about the spread does not bounce evenly. It lands low more often than high, in a direction that flatters you.
>
> The solution is not to stop guessing. The solution is to work out, **before you collect data**, how likely you are to detect an effect given your specific sample size and how big the effect probably is.
>
> That calculation is called power. Everything else in this chapter is machinery for doing it.

7.2 Type I and Type II Error

Every hypothesis test ends in a decision, and every decision can be wrong in exactly two ways.

	Treatment works (H_0 FALSE)	Treatment does nothing (H_0 TRUE)
Reject H_0	Correct decision (**power**)	**Type I error** (α)
Fail to reject H_0	**Type II error** (β)	Correct decision

Type I error is a false alarm. There is no effect, and you claim there is one. Its long-run rate is α, and you choose α yourself, usually .05, by simple decree.

Type II error is a miss. There *is* an effect, and you fail to find it. Its rate is β, and you do **not** get to choose it directly. You have to earn your way out of it.

Power is $1 - \beta$: the probability of detecting an effect that is genuinely there.

Notice the asymmetry. Alpha is a decision you make in about four seconds. Power is something you buy, with participants, and it is expensive.

> **⚠** What Alpha Does Not Mean
>
> Setting $\alpha = .05$ does **not** mean any of the following:
>
> - there is a 5% probability that this particular rejected null is true;
> - 5% of published findings are wrong;
> - your result has a 95% chance of replicating.
>
> It means that *if* the null hypothesis and its assumptions hold, this **procedure** falsely rejects about 5% of the time in the long run. The p-value is a statement about a procedure. It cannot read the hidden state of the universe, and it has never once tried.

7.2.1 The Errors Are Not Symmetric in Cost

A Type I error means we run around telling everyone that cookies make children hyperactive when they do not. Parents ban cookies. A generation grows up sad.

A Type II error means cookies *do* make children hyperactive, we fail to notice, and we spend the next decade blaming the children.

Which error is worse depends entirely on what happens next, which is a question statistics cannot answer for you. A false alarm on a cancer screening costs a biopsy. A miss costs considerably more.

Psychology and neuroscience have historically obsessed over Type I error and treated Type II error as somebody else's problem. Button and colleagues surveyed the neuroscience literature and estimated median statistical power at around 21% (Button et al. 2013), which means the typical study had roughly a one-in-five chance of finding an effect that was genuinely there. This is one of several reasons the replication crisis arrived carrying luggage and expecting to stay a while.

7.3 Signal Detection Theory

Given that I must estimate my standard error from an S that bounces around, I need a way to say how "big" an effect is in the first place.

The one-sample t-test chapter ended by handing you a d and refusing to explain it. Here is the explanation.

Cohen borrowed the idea from signal detection theory, developed for radar operators in World War II who had to decide whether that blip was a plane or noise. The question there was: how strong must a signal be before you can detect it over the noise? For us, the noise is the control condition and the signal is the effect of the treatment.

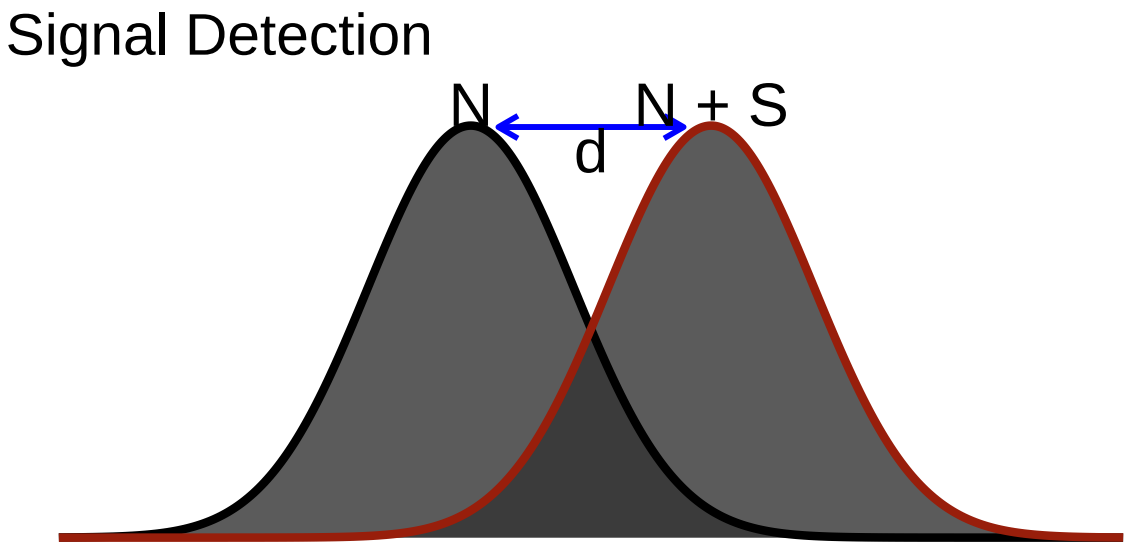


Figure 7.4: Signal detection. The gap between the two peaks, measured in standard deviations, is d.

$$d = \frac{\mu_{(Noise+Signal)} - \mu_{(Noise)}}{\sigma_{(Noise)}}$$

Signal detection theory also hands us this chapter's 2×2 for free. Here is the hearing-test version of the table above, and it is the *same table*:

	There **was** a beep	There was **no** beep
You said **YES**	Hit	False alarm
You said **NO**	Miss	Correct rejection

Hit, miss, false alarm, correct rejection. Power, Type II error, Type I error, correct decision. Same 2×2, different lab coat. Detecting a treatment effect and detecting a faint beep are the same problem, and the reason both fields drew the same table is that both are asking how far the signal has to sit from the noise before you can trust yourself.

7.3.1 Cohen's d

For our purposes:

$$d = \frac{M_{H1} - \mu_{H0}}{S_{H1}}$$

You have seen this formula before, wearing a different hat:

$$Z = \frac{M - \mu}{S}$$

So Cohen's d is in standard deviation units.

That denominator carries an assumption worth naming: we are treating the treatment as something that shifts the distribution without changing its spread. Sometimes that is true. Sometimes a treatment helps some people enormously and others not at all, which changes the spread, and then this formula is quietly describing a world that does not exist.

Size	d
Small	.2
Medium	.5
Large	.8
Huge	1.2

Those adjectives are easier to trust once you see them. Each panel below shows two distributions separated by the d in its title:

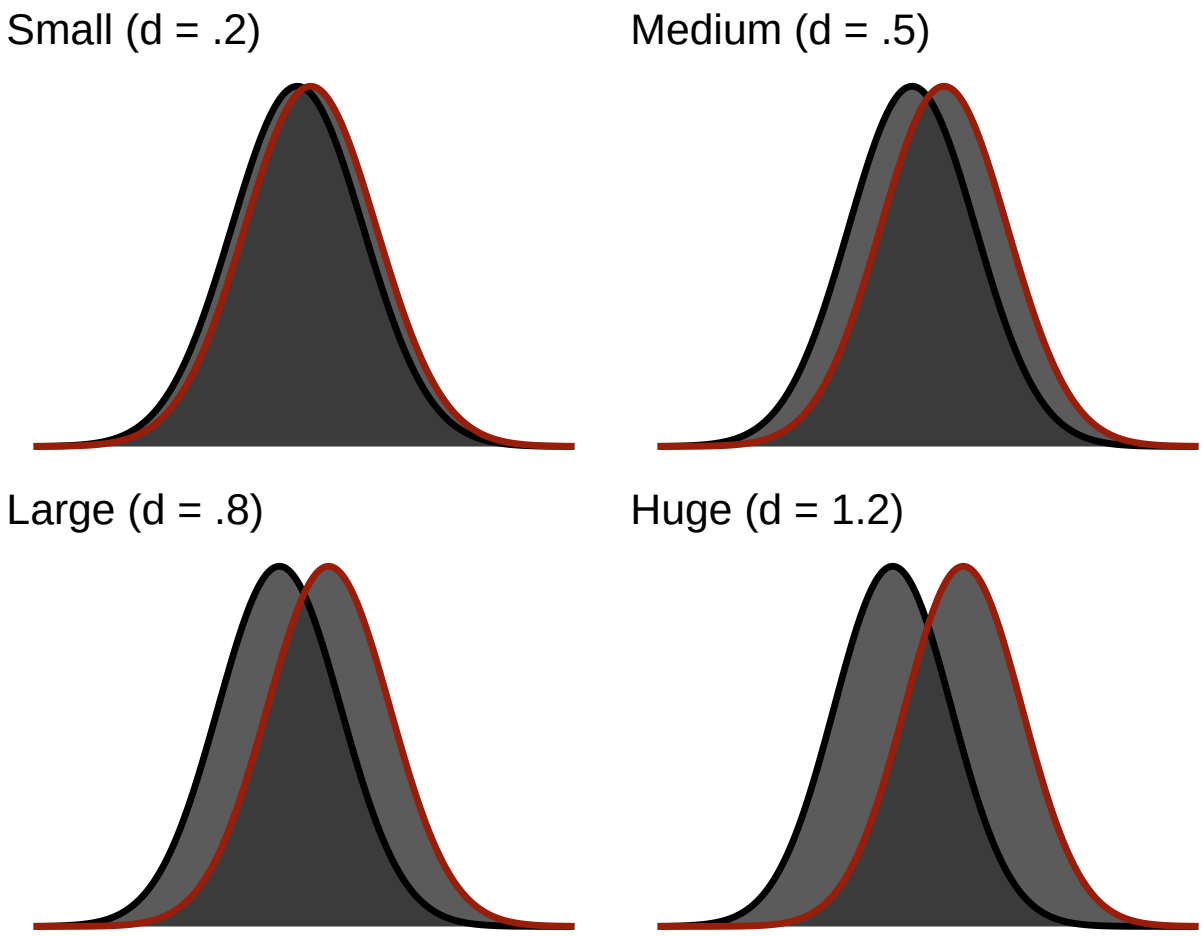


Figure 7.5: Small, medium, large, and huge. Note how much overlap survives even at the bottom right.

Even a Huge Effect Overlaps

Look at the bottom-right panel again. That is a *huge* effect, and the two distributions still share an enormous amount of ground.

That overlap is the point. Two groups can differ by a "large" amount and still contain enormous numbers of people who look exactly like each other. An effect size is a statement about distributions, not about individuals, and it has never promised you a clean separation.

These Adjectives Are Conventions, Not Laws

Cohen proposed small, medium, and large as rough guidance for a field that had none, and then spent years watching people treat them as physical constants.

A d of .2 for a cheap public health intervention delivered to forty million people may matter enormously. A d of 1.1 in a study of twelve undergraduates may matter not at all. Interpret magnitude in context, or do not interpret it.

7.4 Power

Power is the likelihood that a study will detect an effect when there genuinely is one. Power $= 1 - \beta$, where β is the Type II error rate (Cohen 1962, 1988, 1992).

Here is the whole thing in one picture.

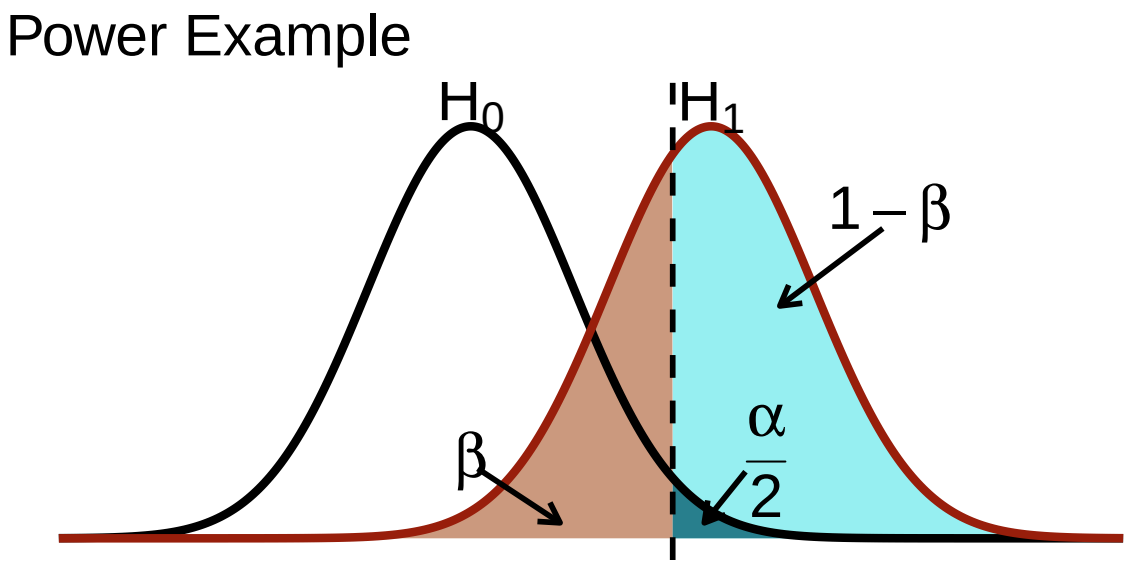


Figure 7.6: Power is the slice of the alternative distribution that clears the critical value.

Read it left to right. The black curve is the world where the null is true. The red curve is the world where the effect is real. The dashed line is your critical value, the point past which you will reject the null.

- Everything to the right of the dashed line **under the black curve** is $\alpha/2$: rejecting when you should not have.
- Everything to the left of the dashed line **under the red curve** is β: the effect was real and you missed it.
- Everything to the right of the dashed line **under the red curve** is power.

Move the dashed line left and you gain power while buying more Type I error. Move it right and the reverse. Push the red curve further right, or make both curves narrower, and power improves without that trade. Which is the entire game.

7.4.1 Uses of Power

A priori power means setting a target likelihood of success (say, power = .80) at a given α (say, .05), assuming a *true* effect size, and solving for the sample size you need (Cohen 1962, 1988, 1992). This is the useful one. It is also the hard one, because it requires knowing the true effect size, which is the thing you are running the study to find out.

We used to tell people to go find a meta-analysis. Then it turned out meta-analytic effect sizes are inflated by publication bias (Kicinski et al. 2015), because studies that found nothing were never published and therefore never averaged in. A rule of thumb: assume your effect is small-to-medium, $d = .3$ or .4, and be pleasantly surprised rather than catastrophically wrong.

Post-hoc power means estimating the power you achieved using the effect size you *observed*, your sample size, and your α.

Post-hoc power is meaningless. It is worse than meaningless, because it looks like information.

! Post-Hoc Power Is a Ghost

Observed power is a one-to-one function of your p-value. A significant result *always* yields high observed power; a non-significant result *always* yields low observed power. It cannot tell you anything your p-value did not already tell you, and it is frequently used to argue "our null result was just underpowered," which is circular reasoning with a calculator (Hoenig and Heisey 2001).

Worse, the effect size feeding it is biased. As we saw at the start of this chapter, small samples underestimate σ. A smaller denominator makes d bigger. Your observed effect is flattering you, and post-hoc power then launders the flattery into a number with a decimal point.

Compute power **before** you run the study. Afterwards, report the confidence interval and let it speak.

7.5 Power and Significance Testing

There is a subtlety here that trips up nearly everyone, so we will be slow about it.

Everything above drew the curves in **standard deviation** units. But significance testing does not happen in standard deviation units. It happens in **standard error** units. And the standard error shrinks as N grows, because $S_M = S/\sqrt{n}$.

The distance between the peaks is the effect size, and it does **not** change with sample size. What changes is how *wide* the curves are.

Homer Simpson and the Statistically Significant Hair

In "Simpson and Delilah" (The Simpsons, S2:E2), Homer takes Dimoxinil and wakes up the next morning with a full luscious head of hair. That is an enormous effect size.

In reality, Minoxidil grows back very little hair, after months of twice-daily application, in a subset of men, most of whom keep checking the mirror hopefully (*speaking from experience*). The effect is genuinely small. It is also statistically significant, because "most men experience *some* growth beyond noise," and noise here is defined in standard error units, not standard deviation units.

Statistically significant is not the same as clinically significant. It is not even the same as noticeable. Homer's hair is a story about effect size. The clinical trial is a story about standard errors.

Watch the curves narrow while the peaks stay exactly where they are. The effect below is a medium one, $d = .5$, in all four pictures.

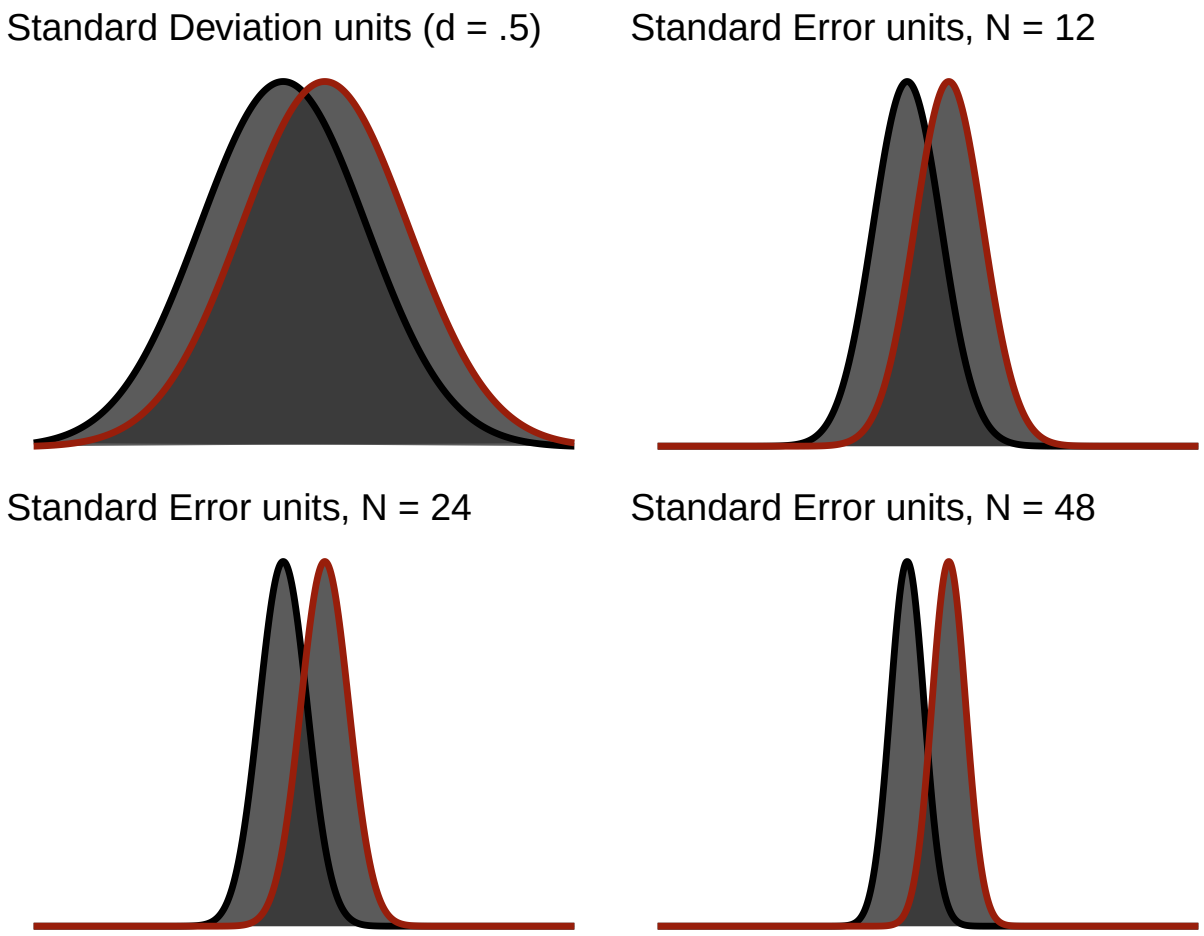


Figure 7.7: The same half-point difference, seen through progressively narrower sampling distributions.

The area representing $1 - \beta$ grows as we add participants, because the curves get thinner while the distance between them stays fixed.

This is why effect size is described as independent of sample size and significance is not. That statement is true if you know the population standard deviation σ. We never do. We approximate it with S, so our observed effect size is itself another guess, subject to the same fickleness we opened this chapter with. Which is why we hope meta-analysis, averaging over many studies, gives a steadier estimate, right up until publication bias mugs it in an alley.

⚠ Advanced: Effect Size and the Pilot Study Trap

A few lines ago I said sample size is independent of effect size. That is half true, and here is the other half.

Look at the formula again: $d = \frac{M-\mu}{S}$. We are estimating **two** things from our sample to guess at one true effect size in the population. In a small sample, both of them bounce around. But there is a hidden way they bounce, and it is not symmetric.

In a small sample you are unlikely to catch the genuinely abnormal people, because there are not many of them and you did not grab many people. Miss the extremes and your S comes out smaller than it should be. That is bias, not bad luck, and it happens on average.

So what? Look at where S sits. It is the **denominator**. When the denominator comes out too small, the effect size comes out too big.

How bad is it? Bad. Under realistic small-sample conditions the observed effect can run **roughly twice** what it should be. I am not asking you to take my word for it, because there is a simulation in the Advanced Topics section at the end of this chapter that shows exactly this happening.

Now think about what that means for a **pilot study**, where the entire point is to run a small sample to decide whether the real study is worth doing. Your pilot is systematically biased toward telling you yes. Then you power your real study on that inflated number, recruit far too few people, and find nothing.

There is no consensus in the literature beyond "interpret with caution," and I show a couple of the proposed fixes in the advanced section. My own solution is to throw lots of undergraduates into the volcano and hope it does not happen to me.

7.6 Estimating A Priori Power in R

Base R ships with a power function for t-tests. No package required.

It needs five pieces of information, and you already know all of them by other names:

Argument	What it is in the words we have been using
`n`	sample size
`delta`	the effect size, in Cohen's d units (see the tip below)
`sd`	the standard deviation (set this to 1 and `delta` becomes d)
`sig.level`	α, your Type I error rate, almost always .05
`power`	$1 - \beta$, the chance of catching the effect if it is real

You fill in four and leave one as `NULL`. R solves for the blank.

❗ Why Everyone Uses .80, and Why That Is Not Very Impressive

Convention says aim for power of **.80**. Say that out loud in the other direction: $\beta = 1 - .80 = .20$, so a study designed to the field's standard **misses a real effect one time in five.** That is the norm, not the failure case.

Plenty of people now argue for **.95**, which drops the miss rate to one in twenty. Excellent idea. Now buy it.

Sample size does not scale linearly with power. Going from .80 to .95 does not cost you a few extra participants, it costs a great many, and we will watch exactly how badly it scales later in this chapter. Get your wallet out.

How to Make power.t.test Speak Cohen's d

`power.t.test()` wants a raw mean difference (`delta`) and a standard deviation (`sd`), not a d.

The trick: **set `sd = 1` and put your d in `delta`.** Because $d = \delta/\sigma$, setting $\sigma = 1$ makes the raw difference and the standardized difference the same number.

```
power.t.test(delta = 0.4, sd = 1, ...)   # this is d = .4
```

Everything in this chapter uses that convention. If you ever want to work in raw units instead, supply both the real `delta` and the real `sd` and it will happily oblige.

The `type` argument picks which t-test you mean:

`type`	Design	What `n` means
`"one.sample"`	one group vs. a fixed value	total participants
`"two.sample"`	two independent groups	participants **per group**
`"paired"`	same people measured twice	number of **pairs**

Getting `n` wrong here is the single most common power mistake. A `two.sample` answer of 64 means **128 people**.

7.6.1 One-Sample: The Cookies Return

Recall the one-sample study: we feed children cookies and compare their hyperactivity to a hypothesized population value of $\mu = 50$. The steps were:

1. Assume a population value to test against, here $\mu = 50$.
2. Collect a sample of children and feed them cookies.
3. Compute the sample mean M and standard deviation S.
4. Ask whether M sits farther from 50 than sampling error can comfortably explain.

In a Real Study You Do Not Get to Build the Population

The first two lines below manufacture a million children out of nothing, and then the next line grabs 36 of them. That is not what research looks like.

In the real world there is no `Population` object. You go out and *find* your population, then kidnap, I mean politely ask their parents for permission after your IRB approves the study, and then feed actual children actual cookies and measure what happens.

I simulate because it is free, instant, and nobody's child ends up climbing a curtain. But everything after the sampling step is identical to what you would do with real data.

```
set.seed(123)

# Simulate our population: a million children we will never meet.
Population <- rnorm(1e6, mean = 50, sd = 5)

set.seed(42)

# Sample 36 kids (for real if you can, but I am lazy, so we simulate it)
```

```r
# and give each one a 2-point cookie boost.
kids.1 <- sample(Population, 36, replace = TRUE) + 2

M_kids <- mean(kids.1)                # our sample mean
S_kids <- sd(kids.1)                  # our sample SD
d_kids <- (M_kids - 50) / S_kids      # Cohen's d: how far from 50, in SD units

round(c(M = M_kids, SD = S_kids, n = length(kids.1), d = d_kids), 3)
```

```
     M     SD      n      d
53.035  4.356 36.000  0.697
```

Suppose we are designing this study fresh and expect a small-to-medium effect, $d = .4$. How many children do we need for 80% power?

```r
At.Power.80 <- power.t.test(n = NULL,              # <- the unknown we want solved
                            delta = .4,            # the effect size we expect
                            sd = 1,                # sd = 1 makes delta mean "Cohen's d"
                            sig.level = 0.05,      # our alpha
                            power = .80,           # the power we are demanding
                            type = "one.sample",   # which t-test
                            alternative = "two.sided")
At.Power.80
```

```
     One-sample t test power calculation

              n = 51.00957
          delta = 0.4
             sd = 1
      sig.level = 0.05
          power = 0.8
    alternative = two.sided
```

That is one function with one trick, and the trick is the `NULL`. `power.t.test()` takes five quantities that constrain each other, and it solves for **whichever one you leave as `NULL`**. Here we left `n = NULL` and asked how many people we need. Leave `power = NULL` instead and it tells you the power you have. Leave `delta = NULL` and it tells you the smallest effect you could catch. Same function, three different questions, and we will ask all three below.

The `sd = 1` is worth a second look too. It is not a claim about your data. Setting it to 1 puts `delta` in standard-deviation units, which means you can hand `delta` a Cohen's d directly and R will do the right thing.

We always **round up**: 52 children. You cannot recruit a fractional child, and rounding down leaves you below your target.

Now the same question at 95% power, which some people have started demanding.

```r
At.Power.95 <- power.t.test(n = NULL, delta = .4, sd = 1,
                            sig.level = 0.05, power = .95,
                            type = "one.sample",
                            alternative = "two.sided")
ceiling(At.Power.95$n)
```

```
[1] 84
```

84 children. Going from 80% to 95% power cost us 32 additional participants for the same effect. Cohen suggested 80% precisely because it is economical. 95% is called *high power*, and now you can see why it is unpopular with anyone who has to pay for it.

7.6.2 Solving for Power Instead of N

If the sample size is already fixed, perhaps because that is how many children were available, or how much money you had, run it the other way.

```r
# Same function. This time n is known and power is the NULL.
AP.Power.N36 <- power.t.test(n = 36, delta = .4, sd = 1,
                             sig.level = 0.05, power = NULL,
                             type = "one.sample",
                             alternative = "two.sided")
round(AP.Power.N36$power, 3)
```

```
[1] 0.646
```

With 36 children and a true d of .4, we had a 64.6% chance of detecting the effect. Which means we had a 35.4% chance of missing it entirely, publishing nothing, and concluding cookies are harmless.

7.6.3 Solving for the Effect Size You Could Detect

The third useful question: given my sample, what is the *smallest* effect I could reliably find?

```r
# Third question, same function. Now delta is the NULL.
Eff.Power.N36 <- power.t.test(n = 36, delta = NULL, sd = 1,
                              sig.level = 0.05, power = .80,
                              type = "one.sample",
                              alternative = "two.sided")
round(Eff.Power.N36$delta, 3)
```

```
[1] 0.48
```

With 36 children, the smallest effect we can detect 80% of the time is $d = 0.48$. Anything smaller than that, we will usually miss.

This is the calculation to run when a reviewer asks why your null result should be believed. "We had 80% power to detect effects of $d \geq 0.48$, so we can reasonably rule out effects of that size, but not smaller ones" is a defensible sentence. "It wasn't significant" is not.

7.7 Sample Size and Power Are Not Linear

At a fixed effect size, adding participants buys you power, but not at a constant rate.

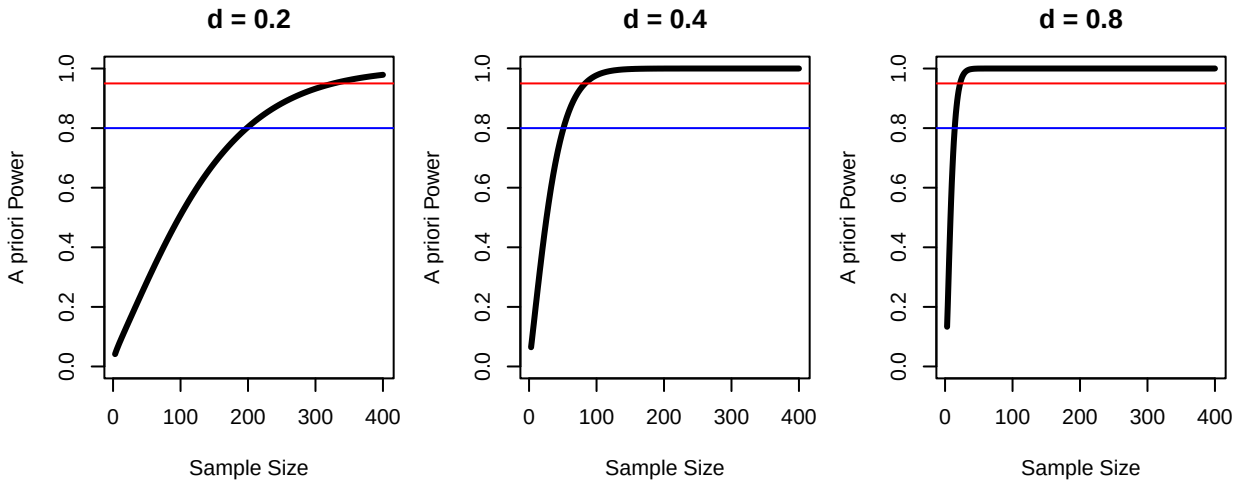


Figure 7.8: Power against sample size at three effect sizes. Each curve rises steeply and then flattens.

The blue line is 80% power; the red line is 95%. Notice how far right the curve for $d = .2$ has to travel before it crosses either one, and how quickly $d = .8$ gets there.

The larger the effect, the fewer participants you need. Which is a comforting sentence until you remember that you do not control the effect size, and that most real psychological effects are small.

7.7.1 Effect Size and Power Are Also Not Linear

Flip it around: hold the sample size fixed and vary the effect.

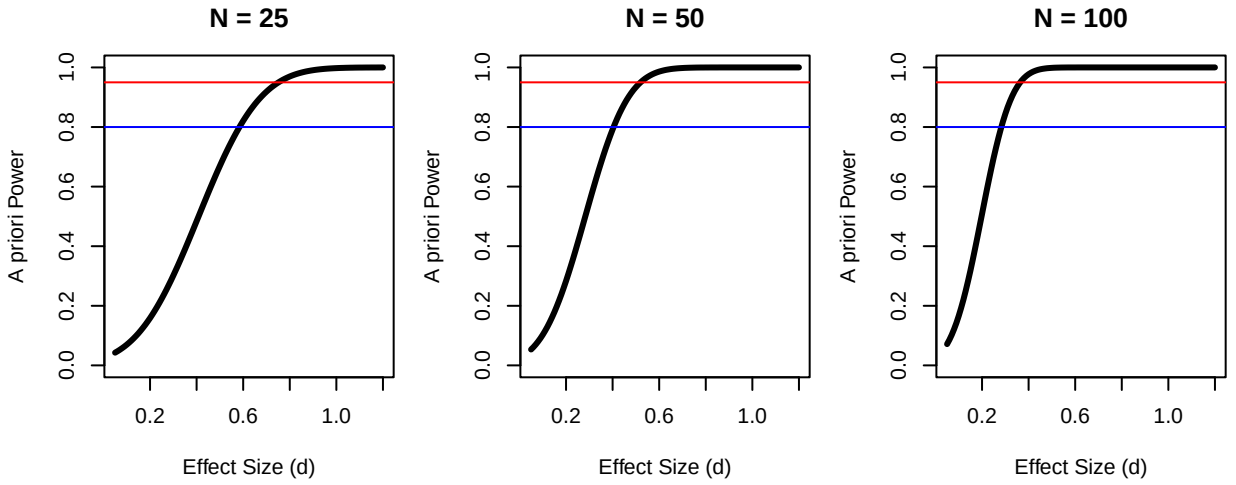


Figure 7.9: Power against effect size at three sample sizes. Same shape, different axis.

With 25 people you need a whopping effect before you are likely to find anything. With 100, medium effects come into range. The curves are S-shaped, not straight, which means there is a region where adding a few participants helps enormously and a region where it barely matters at all.

Power for the Other Two t-Tests Lives With Those Tests

Everything above used `type = "one.sample"`, because the one-sample t-test is the only one you have met so far.

The other two work identically, with one argument changed. Rather than teach them here to a reader who has not seen the tests yet, each one waits in its own chapter:

- **Two separate groups**: the independent-samples chapter closes with `type = "two.sample"`, where `n` means *per group*, plus Hedges' correction for the bias small samples put into d.
- **The same people twice**: the paired-samples chapter closes with `type = "paired"`, and shows how much cheaper a repeated-measures design is for the same effect.

Come back here for the concepts. Go there for the arithmetic.

7.8 The Short Story

- Small samples underestimate variability, in a direction that flatters you.
- Type I error is a false alarm and its rate is α, which you choose. Type II error is a miss and its rate is β, which you must earn your way out of.
- Power is $1 - \beta$: the probability of finding a real effect. It depends on the effect size, the sample size, and α.
- Effect size lives in standard deviation units. Significance lives in standard error units. Adding participants narrows the curves without moving the peaks.
- Compute power **before** you run the study. Post-hoc power is a restatement of your p-value wearing a disguise.
- `power.t.test()` solves for whatever you leave `NULL`. Set `sd = 1` and put d in `delta`.
- The other two t-tests use the same function with `type` changed, and each is powered in its own chapter. For `two.sample`, `n` is per group, so double it. For `paired`, `n` is the number of pairs and `delta` is d_z.
- Non-significant does not mean no effect. Very often it means no chance.

7.9 Advanced Power Section

Everything above is what you need to design a study and defend it. What follows is the part that explains *why the field got this so wrong for so long*. You are not required to master it.

7.10 The Observed Effect Size Is Also a Guess

Running one small pilot study to estimate an effect size used to be standard practice. It is a catastrophe, and we can show exactly how bad it is.

Consider a Mozart-effect study: compare spatial reasoning right after listening to Mozart's Sonata for Two Pianos in D Major, K. 448, against a control group who listened to Bach's Brandenburg Concerto No. 6, matched on modality, tempo, and duelling voices. If the effect is specific to Mozart, Bach should do nothing.

Simulate $\mu_{Mozart} = 109$, $\mu_{Bach} = 110$, $\sigma = 15$ in both, with $n = 30$ per group. The true effect is

$$d_{true} = \frac{110 - 109}{15} = 0.067,$$

which is to say, nothing. There is no Mozart effect in this simulated world. Now watch what a single study reports.

```
set.seed(666)
n <- 30
Mozart.Sample <- rnorm(n, 109, 15)
Bach.Sample   <- rnorm(n, 110, 15)

Sp_mb <- sqrt(((n-1)*sd(Mozart.Sample)^2 + (n-1)*sd(Bach.Sample)^2) / (2*n - 2))
Obs.d <- abs(mean(Mozart.Sample) - mean(Bach.Sample)) / Sp_mb
round(Obs.d, 3)
```

```
[1] 0.428
```

Our one study observed $d = 0.428$, against a truth of 0.067.

Now re-run that same study 5,000 times and look at the distribution of observed effect sizes.

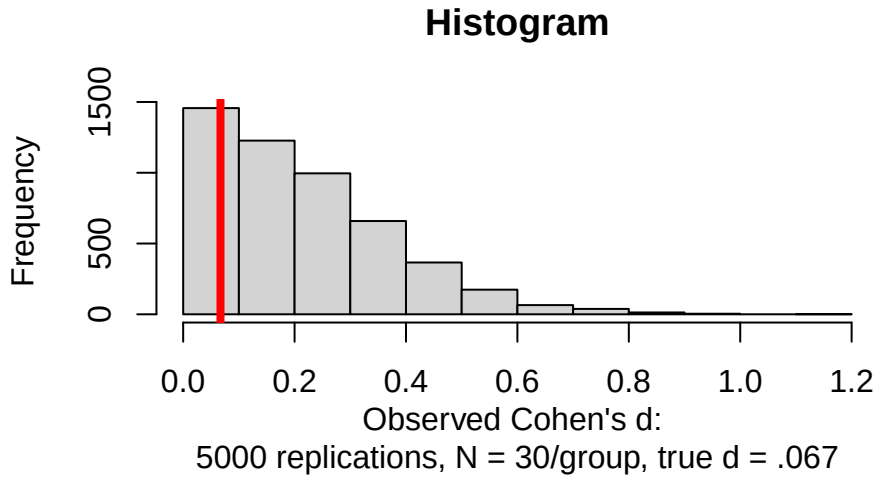


Figure 7.10: The observed effect size is a guess too, and in a small study it is a wild one.

The red line is the truth. Look at how much of the distribution sits far to the right of it.

In this simulation, an observed d at least as large as our one study's 0.428 occurred 10.62% of the time. Worse, there was a 46.32% chance of observing something above the "small effect" threshold of $d = .2$, **in a world where the true effect is essentially zero**.

> ⚠ Why Pilot Studies Cannot Estimate Effect Sizes
>
> A small pilot gives you a draw from that histogram. Not the red line. A draw.
>
> If you then feed that draw into a power analysis, you will compute the sample size needed to detect an effect that does not exist, feel rigorous about it, and run a study doomed before recruitment began.
>
> Pilot studies are for testing whether your procedure works, whether participants understand the instructions, and whether the software crashes. They are not for estimating effect sizes. They are far too small to do that, which is what makes them pilots.

7.11 Safeguard Power

If you must use an observed effect size from a previous study, do not use the point estimate. Use the **lower bound of its confidence interval**. This is called *safeguard power* (Perugini et al. 2014), and it asks the only sensible question: what if the original study was lucky?

```
# CI for d, using the standard large-sample approximation
d_ci <- function(d, n1, n2) {
  se <- sqrt((n1 + n2)/(n1 * n2) + d^2 / (2 * (n1 + n2)))
  c(lower = d - 1.96 * se, d = d, upper = d + 1.96 * se)
}

published <- d_ci(0.50, 80, 80)
round(published, 3)
```

```
lower     d upper
0.185 0.500 0.815
```

A published effect of $d = .50$ from a reasonably sized study, 80 per group, still has a confidence interval reaching down to 0.185. Powering for the point estimate and powering for that lower bound produce very different studies:

```
c(powered_for_point_estimate =
    ceiling(power.t.test(delta = 0.50, sd = 1, power = .80,
                         type = "two.sample")$n),
  powered_for_lower_bound =
    ceiling(power.t.test(delta = published["lower"], sd = 1, power = .80,
                         type = "two.sample")$n))
```

```
powered_for_point_estimate    powered_for_lower_bound
                        64                        459
```

If the original estimate was accurate, both studies find the effect. If the original was a lucky draw from the high end of its own sampling distribution, only the second one does. The first returns a null result, and everybody spends eighteen months arguing about whether the replication was competent.

⚠ Now Try It With a Small Original Study

Run the same calculation on a $d = .50$ reported from 25 participants per group:

```
round(d_ci(0.50, 25, 25), 3)
```

```
 lower      d  upper
-0.063  0.500  1.063
```

The lower bound is **negative**. The confidence interval includes zero, and no honest safeguard-power calculation is possible, because the original study is compatible with an effect in the opposite direction.

This is not a defect in the method. It is the method working correctly and telling you something you did not want to hear: **that study never established an effect size at all.** It established a range so wide that it contains "helps a lot," "does nothing," and "actively harms."

Powering a replication off a point estimate like that is not a calculation. It is a wish with decimal places.

7.12 When the Formula Will Not Do: Simulate

`power.t.test()` handles t-tests. The moment your design gets more complicated, unequal group sizes, non-normal outcomes, planned attrition, an outcome measured on a bounded scale, the closed-form solution runs out.

The general-purpose answer is to simulate. Build the world you expect, run your analysis on it thousands of times, and count how often you win.

```
# Build a machine that runs a whole study over and over and reports how
# often it found the effect. That proportion IS power.
simulate_power <- function(n_per_group, true_d, n_sims = 2000, alpha = .05) {

  hits <- replicate(n_sims, {          # run one study, n_sims times over

    # Build two groups from populations we KNOW differ by true_d.
    # Group 2 is centered on 0, group 1 on true_d, both with sd = 1,
    # so the gap between them is exactly true_d in standard deviations.
    g1 <- rnorm(n_per_group, mean = true_d, sd = 1)
    g2 <- rnorm(n_per_group, mean = 0,      sd = 1)

    # Run the test. This line is TRUE if we caught the effect, FALSE if we missed.
    t.test(g1, g2)$p.value < alpha
  })

  # hits is now 2,000 TRUEs and FALSEs. R treats TRUE as 1 and FALSE as 0,
  # so the mean of them is the proportion of studies that succeeded.
  mean(hits)
}

set.seed(343)
simulate_power(n_per_group = 30, true_d = 0.5)
```

```r
# compare to the closed-form answer
power.t.test(n = 30, delta = 0.5, sd = 1, type = "two.sample")$power
```

```
[1] 0.4735
[1] 0.477841
```

Read what that function does one more time, because the logic is the entire idea of power. We *built* a world where the effect is real and we know its exact size. Then we ran the study 2,000 times in that world and counted how often the t-test noticed. If it noticed 480 times out of 2,000, our power is .48, and the other 1,040 studies were funded, staffed, and wasted.

The last two lines are the important habit: the simulation and the formula agree. Always check a simulation against a known answer before you trust it on a problem where you do not have one.

The simulation agrees with the formula, which is how you know the simulation is trustworthy. Now you can change the generating model to anything you like, skewed outcomes, 20% dropout, a ceiling effect, and get an answer where no formula exists.

```r
simulate_power_with_dropout <- function(n_per_group, true_d,
                                        dropout = 0.20, n_sims = 2000) {
  keep <- round(n_per_group * (1 - dropout))
  replicate(n_sims, {
    g1 <- rnorm(keep, mean = true_d, sd = 1)
    g2 <- rnorm(keep, mean = 0,      sd = 1)
    t.test(g1, g2)$p.value < .05
  }) |> mean()
}

set.seed(343)
simulate_power_with_dropout(n_per_group = 30, true_d = 0.5)
```

```
[1] 0.4115
```

Twenty percent attrition costs real power. Plan for it before it happens, not in the limitations section afterwards.

7.13 Other Tools

`power.t.test()` is deliberately limited. For anything beyond t-tests:

- the **`pwr`** package covers correlations, proportions, chi-square, ANOVA, and regression with the same solve-for-`NULL` logic;
- **G*Power** is free stand-alone software and remains the standard for grant applications for people who do not code;
- **`simr`** does simulation-based power for mixed models;
- and for genuinely novel designs, the `simulate_power()` pattern above generalizes to anything you can write down.

NIH and NSF require a power analysis with every grant application. In practice it is difficult, because you must estimate an effect size you do not yet know. Assume small, plan for attrition, and remember that the most expensive study is the underpowered one that answers nothing.

! Cohen's Compromise

Cohen's recommendation was simple: do the significance test **and** report the effect size.

Two pieces of information. First, is the result distinguishable from sampling error? Second, does the effect mean anything at all?

He proposed this in 1962. APA made it standard in 2010.

Forty-eight years. Please do not make it take any longer.

i Figure Credit

The power and signal-detection illustrations in this chapter are adapted from Kristoffer Magnusson's textbook-illustration walkthrough, updated for current ggplot2 syntax.

8 Independent Samples t-Test

> **Two Groups, and Now Both of Them Bounce**
>
> Use an independent-samples *t*-test when **two different sets of people** are each measured once and you want to know whether the groups differ.
>
> This is the workhorse of experimental psychology. Two groups, one measurement each, and random assignment deciding who went where. A very large share of the experiments you will read are exactly this shape, which means you will spend far more of your career reading this test than running it.
>
> Chapter 6 compared one group against a number someone handed us. That number was fixed and given to us, something that rarely occurs in real research. Now both sides of the comparison are estimates, both of them jump around a little, and the standard error has to carry the uncertainty of two means instead of one. That is the entire structural change. Everything else you already learned still applies.

8.1 The Research Question

The CTA already paid me once to ask riders how they feel about the CTA. The answer came back somewhere around "fine, I guess," which is the most Chicago answer available. Now they want to know whether they can buy a better answer.

So: what happens if the trains actually show up every 7 minutes?

8.1.1 Design

Two groups of riders, assigned by chance, not by preference.

- **Control**: regular CTA timing, where "scheduled" is a flexible concept.
- **Experimental**: trains every 7 minutes. Like clockwork. Almost suspiciously efficient.

Nobody rides both schedules. Each rider contributes one number and goes home. That is what makes these groups *independent*, and it is the entire reason this chapter's test is the right one.

8.1.2 Dependent Variable

The same question I asked back in the standard error chapter, on the same **1-5 bipolar Likert scale**, so that the two studies can actually talk to each other:

How much do you typically like traveling on the CTA?

- 1 = Strongly dislike
- 2 = Dislike
- 3 = Neither dislike nor like

- 4 = Like
- 5 = Strongly like (you now defend the CTA on the internet)

We expect the control group to land near **3**, which is exactly where Chapter 5 found riders sitting, and the experimental group near **4**, assuming the manipulation works at all.

I know those numbers because I am about to simulate them, which is the only situation in which I ever know the truth in advance. In a real study we can only hypothesize that the experimental group will be different from the control group, and then go find out.

8.2 Step 1: State the Hypotheses

$$H_0 : \mu_1 = \mu_2$$

$$H_1 : \mu_1 \neq \mu_2$$

Where:

- μ_1 = Experimental (7-minute trains)
- μ_2 = Control (regular timing)

Under the null, $\mu_1 - \mu_2 = 0$. Why? Cause if nothing is happening, $3 - 3 = 0$.

8.3 Step 2: Collect the Data

Two groups, thirty riders each. Those two `rnorm()` lines are the entire experiment. *I mean I could go out and really get data, but again I would have to actually talk to people.*

```
set.seed(343)

n = 30
CTA.SD = 0.5

# Control group (mean ~ 3, the neutral answer Chapter 5 found)
CTA.Control <- rnorm(n, 3, CTA.SD)

# Experimental group (mean ~ 4)
CTA.Experimental <- rnorm(n, 4, CTA.SD)
```

We also stack the two groups into one data frame, because that is the shape R wants when you use formula syntax later (and the shape your lab data will arrive in).

```
CTA.df <- data.frame(
  Condition = c(rep("Control\n(regular timing)", n),
                rep("Experimental\n(every 7 min)", n)),
  Liking = c(CTA.Control, CTA.Experimental)
)
```

```r
# R subtracts groups in factor-level order, so we put Experimental first
# to match the way we wrote the hypotheses in Step 1.
CTA.df$Condition <- factor(CTA.df$Condition,
                           levels = c("Experimental\n(every 7 min)",
                                      "Control\n(regular timing)"))
```

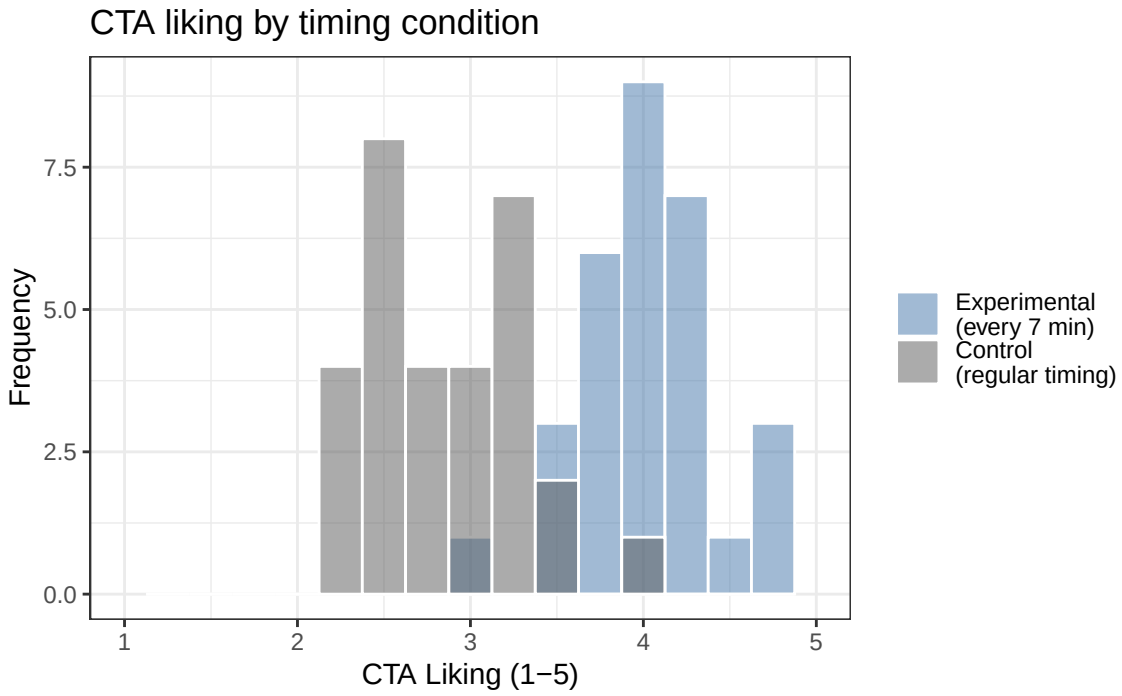


Figure 8.1: Both groups on one axis. The overlap in the middle is the part a t-test has to argue about.

Already you should see separation. But eyeballing histograms is not a statistical test (even if it seems obvious). Notice the overlap in the middle: some control riders liked the CTA more than some experimental riders did. A group difference is about where the two piles sit, not about every person in one group beating every person in the other. Group differences are what we care about here, and we do not actually concern ourselves with the feelings of individual people. What are we, psychologists?! *Oh wait...we are...okay, let me rephrase that...*

Individual commoners, I mean subway riders, all have very different experiences and attitudes toward the subway and toward the new schedule. Maybe some people like the chaos of the old system, or are freaked out by clockwork timing. What are we, Switzerland?! Each person has their reasons, and we are not clinical psychologists (well, I am not, that is for sure) who need to unpack why any one of them feels that way.

Who Are We Doing Research to Explain?

The goal of most experimental research is to understand how a treatment would affect the *average* person. Remember that average person from the distribution chapter? That is who our result is about. So who are they?

This is not easy here, cause I am randomly sampling subway riders. Does the time of day matter? What if I collected at rush hour instead of 3am on the Red Line? What if I only rode the Brown Line above the Loop, in the more affluent neighborhoods? Never forget who your sample is and who your average represents. It is what limits the scope of your results and how far they generalize.

8.4 Step 3: Compute the Standard Error

For **Welch's *t*-test**, which R uses by default, the standard error is:

$$S_{M_1-M_2} = \sqrt{\frac{S_1^2}{n_1} + \frac{S_2^2}{n_2}}$$

! The Subscript Is a Name Tag, Not an Instruction

That $M_1 - M_2$ hanging off the bottom of the S is not telling you to subtract anything. Nothing is being subtracted. It is a label.

Back in the standard error chapter we wrote S_M, meaning "the standard error of M," or in plain English, how much a sample mean bounces around from study to study. The subscript names *which* statistic we are describing. Same idea here. The statistic we care about now is no longer a single mean, it is the gap between two means, so the label is $M_1 - M_2$. Read the whole symbol out loud as "the standard error of the difference between the means" and it stops being intimidating.

Now look at what the formula actually does, because this is the part worth stopping for. The label says *difference*, but the arithmetic **adds**. Both variances go in with a plus sign.

That is not a typo. Group 1 is a guess and bounces around (varies due to chance). Group 2 is also a guess and bounces around. When you subtract two "bouncy" numbers, the bounce does not politely cancel out, it piles up like two waves at the beach. You are now uncertain about where the first mean sits *and* uncertain about where the second one sits, and both of those doubts make the gap between them harder to trust. Uncertainty accumulates.

Welch's test does not assume equal population variances. In other words, we do not have to assume that people riding the chaotic subway and people riding the clockwork subway have the same amount of variance. Why would that matter, and why would the variances be different?

Imagine the chaotic subway causes more varied feelings: some riders LOVE it and some HATE it. The clockwork one does not draw opinions that strong, so people give a narrower range of ratings. Now suppose that by pure luck you sampled too many of the HATE it crowd on the chaotic subway. It would make you think the chaotic system was more hated than it really is.

So Welch's *t*-test corrects this problem by shrinking our estimate of the degrees of freedom (*df*). Remember that our *df* sets the critical values of the *t*-distribution. By shrinking our *df*, we push those critical values further out into the tails of the distribution, so a result has to be more extreme before we are allowed to make our decision. Thus we make our test more conservative, harder to say something is there when it is not. Remember type I error, which is a bad thing to do.

Layers of Knowledge, and Why Your Brain Is Not Broken

Have you noticed that in this one section I had to call back to degrees of freedom, the *t*-distribution, type I error, and several other concepts? This is the part about statistics that freaks people out. You have to actually remember what you learned before.

So if you are lost, go back to those chapters and review. **It is normal to need to go back.** I keep the regression bible written by Cohen on my desk and have to refer to it all the time.

So how do we actually *shrink* our degrees of freedom? We use the Welch–Satterthwaite approximation, which R will calculate because we have suffered enough.

For fun, here is that formula. It is a crazy formula that you never want to calculate by hand:

$$df \approx \frac{\left(\frac{S_1^2}{n_1} + \frac{S_2^2}{n_2}\right)^2}{\frac{\left(S_1^2/n_1\right)^2}{n_1 - 1} + \frac{\left(S_2^2/n_2\right)^2}{n_2 - 1}}$$

Notice the $\approx$. This one does not even pretend to land on a whole number, which is why Welch's degrees of freedom for our study come out as 57.09 instead of a respectable integer. Nobody sits down and computes this. R does it, and we let it.

What if you do not have access to R and have to do all the calculations the old fashioned way, by hand, with a pencil and paper? *Gasp, the horror.*

What we do instead is use the classical pooled-variance version, the standard error is:

$$S_{M_1 - M_2} = \sqrt{S_p^2 \left(\frac{1}{n_1} + \frac{1}{n_2}\right)}$$

This represents how much variability we expect in the **difference between means** due to sampling error. The pooled version combines the two sample variances:

$$S_p^2 = \frac{SS_1 + SS_2}{df_1 + df_2}$$

8.5 Step 4: Compute the *t* Statistic

$$t = \frac{M_1 - M_2 - (\mu_1 - \mu_2)}{S_{M1-M2}}$$

For the pooled-variance test, the degrees of freedom are:

$$df = (n_1 - 1) + (n_2 - 1)$$

Those formulas are not decoration, so let's run them. This is the same arithmetic you would do on paper:

```
M1 <- mean(CTA.Experimental)
M2 <- mean(CTA.Control)
S1 <- sd(CTA.Experimental)
S2 <- sd(CTA.Control)

SS1 <- sum((CTA.Experimental - M1)^2)
SS2 <- sum((CTA.Control      - M2)^2)

Sp2       <- (SS1 + SS2) / ((n - 1) + (n - 1))   # pooled variance
SE_pooled <- sqrt(Sp2 * (1/n + 1/n))             # SE of the difference
t_by_hand <- (M1 - M2) / SE_pooled

c(Sp2 = Sp2, SE_pooled = SE_pooled, t_by_hand = t_by_hand)
```

```
       Sp2  SE_pooled  t_by_hand
 0.1859524  0.1113410 10.2132925
```

Now let R do the same arithmetic. Here it is in **formula syntax**, `outcome ~ group`, which is what you will do in lab (if you take more courses, that is) and what every regression in Part 2 of the book will use:

```
# The classical pooled test: matches the hand calculation above
t.test(Liking ~ Condition, data = CTA.df, var.equal = TRUE)
```

```
    Two Sample t-test

data:  Liking by Condition
t = 10.213, df = 58, p-value = 1.412e-14
alternative hypothesis: true difference in means between group Experimental
(every 7 min) and group Control
(regular timing) is not equal to 0
95 percent confidence interval:
 0.9142852 1.3600318
sample estimates:
mean in group Experimental\n(every 7 min)
                                 4.016756
  mean in group Control\n(regular timing)
                                 2.879597
```

Same number. The machine just does math faster.

R uses **Welch's *t*-test** by default because it does not require the two populations to have equal variances. That is usually the safer choice. The pooled formula above is still useful because it makes the basic machinery visible, but we should not force equal variances merely because ancient papers did it that way. Also, we have these things called computers now. We do not do this by hand anymore.

```
# Welch: drop var.equal and R defaults to the safer test
CTA.t.test <- t.test(Liking ~ Condition, data = CTA.df)

CTA.t.test
```

```
    Welch Two Sample t-test

data:  Liking by Condition
t = 10.213, df = 57.093, p-value = 1.693e-14
alternative hypothesis: true difference in means between group Experimental
(every 7 min) and group Control
(regular timing) is not equal to 0
95 percent confidence interval:
 0.9142098 1.3601072
sample estimates:
mean in group Experimental\n(every 7 min)
                                 4.016756
```

```
  mean in group Control\n(regular timing)
                                2.879597
```

Look at what changed between the two outputs, and what did not. The t statistic is *identical* here, because both groups have exactly 30 people. Only the degrees of freedom moved, from a tidy 58 down to Welch's fractional 57.09. Welch spends a fraction of a degree of freedom as insurance against unequal variances. When the groups are the same size and the spreads are similar, that insurance is nearly free, which is a good reason to just leave it on.

If trains truly arrive every 7 minutes, we should see a statistically significant increase in liking.

8.6 What This Test Assumes

Four things, and they are not equally dangerous.

Independence. Each rider answers alone. If you survey five friends traveling together, you have one opinion wearing five coats, and the model, which believes it met five strangers, becomes confident in ways it has not earned. This is the assumption that actually kills studies, and no amount of statistical fanciness resurrects them. When you genuinely cannot avoid it, because you measured the same person twice or sampled whole classrooms, you need a model that knows about the grouping, which is where mixed models come in later.

Random assignment. Nothing about the arithmetic knows how riders ended up in their groups. The reason we can talk about the 7-minute schedule *causing* anything is that chance built the groups, so the schedule is the only systematic difference between them. Let people choose their own condition and you have a quasi-experiment, the same t-test, and a much weaker sentence at the end of it.

Normality, roughly. The test assumes the *sampling distribution of each mean* is approximately normal. With 30 people per group the Central Limit Theorem does most of that work for you. Look at the histograms anyway.

Equal variances. Only the pooled test needs this one. Welch shrugs, which is precisely why it is the default.

8.7 Step 5: Visual Reporting - Means + 95% CI

```r
library(ggpubr)

ggbarplot(
  CTA.df,
  x = "Condition",
  y = "Liking",
  add = c("mean_ci", "jitter"),   # mean + 95% CI, plus the raw ratings
  fill = "Condition",
  palette = c("grey80", "grey60"),   # light enough that the dots stay visible
  ylim = c(1, 5),
  xlab = NULL,
  ylab = "Mean CTA liking (1-5)",
  title = "CTA liking by timing condition"
) +
  theme_bw(base_size = 11) +
  theme(legend.position = "none")
```

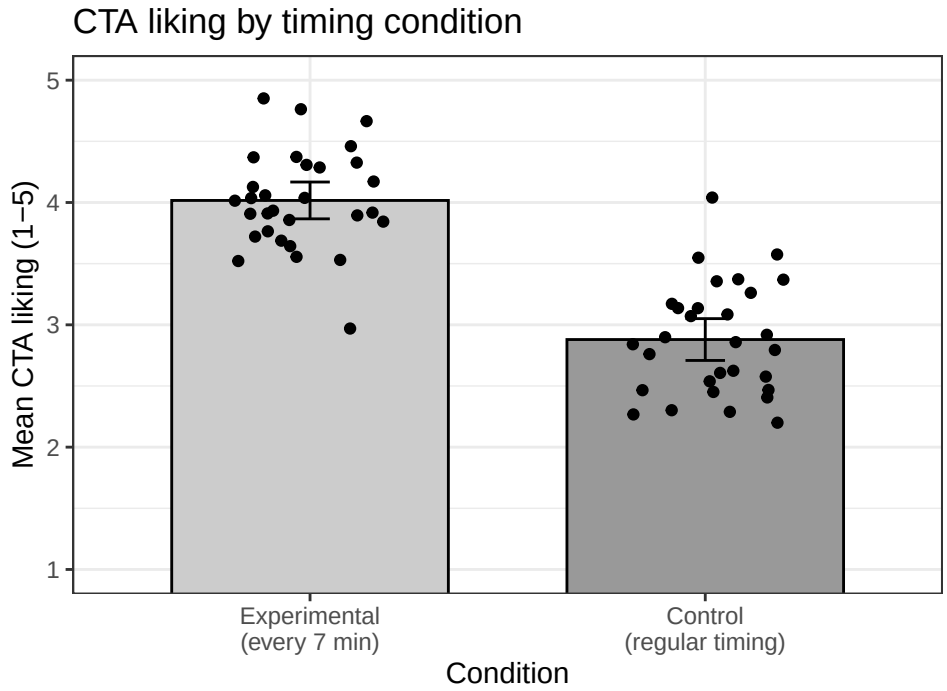


Figure 8.2: The bars are the summary; the dots are the humans.

The bars would look exactly the same if every rider in a group had given the identical score. The dots are there so you can check that they did not.

8.8 Step 6: Extract the Values for the APA Write-Up

Formatting packages can be convenient, but some do not support Welch's t-test. We can extract the values directly and avoid letting a package bully us back into an equal-variance assumption.

```
# The $ pulls one piece out of the t.test object by name.
# unname() strips the label R attaches to it: without it, the t column
# would come out headed "t.t" because the statistic arrives carrying
# its own name tag. We want a clean column, so we take the tag off.
apa_values <- data.frame(
  t       = unname(CTA.t.test$statistic),
  df      = unname(CTA.t.test$parameter),
  p       = CTA.t.test$p.value,
  CI_low  = CTA.t.test$conf.int[1],   # [1] = first value in the interval
  CI_high = CTA.t.test$conf.int[2]    # [2] = second value
)

apa_values
```

```
         t       df            p    CI_low  CI_high
1 10.21329 57.09293 1.692969e-14 0.9142098 1.360107
```

> **Never Write p = 0**
>
> Modern APA wants the exact p-value, reported to two or three decimal places: $p = .03$, $p = .008$. The one exception is when p is very small. Below .001 you stop counting zeros and write $p < .001$, which is what our output above deserves.
>
> What you must never write is $p = 0$. A p-value is the probability of data at least this extreme under the null, and that probability is never exactly zero. Your software prints `2e-16` because it ran out of decimal places, not because the universe ruled the possibility out. Rounding that to zero claims something no test can deliver.

8.9 Step 7: Cohen's d

Statistical significance tells us whether the effect is likely non-zero.

Effect size tells us **how big it is**.

For independent samples with equal variances:

$$d = \frac{M_1 - M_2}{S_p}$$

Where:

$$S_p = \sqrt{\frac{SS_1 + SS_2}{df_1 + df_2}}$$

Let's calculate it manually so you see where it comes from. We already built the pooled variance back in Step 4, so S_p is just its square root.

```
Sp <- sqrt(Sp2)              # Sp2 came from the hand calculation in Step 4
cohens_d <- (M1 - M2) / Sp

c(M_experimental = M1, M_control = M2, Sp = Sp, d = cohens_d)
```

```
M_experimental      M_control             Sp              d
     4.0167558      2.8795973      0.4312219      2.6370608
```

The value tells us how far apart the two means are in pooled standard-deviation units. Context still matters. A conventional adjective cannot tell us whether the change matters for actual riders, budgets, access, or the continued emotional survival of anyone waiting for a bus in February.

> **Yes, We Just Pooled the Variances We Refused to Pool**
>
> Earlier we ran Welch precisely so we would not have to assume equal variances, and now the effect size pools them anyway. That is standard practice, and it is not hypocrisy, because the two numbers have different jobs. The test is an inference and needs to protect itself against unequal spreads. The effect size is a description, and pooling gives a reasonable yardstick when the spreads are similar, as they are here (0.4 and 0.46).
>
> If the two spreads are very different, standardize by the control group's SD instead. That is **Glass's Δ**, and it answers a cleaner question: how far did the treatment move people, in units of how much they varied before you touched them?

Because let's be honest, trains every 7 minutes would change lives, and the data say it would be a huge effect.

8.10 Step 8: APA Results Paragraph

Participants in the every-7-minutes condition ($M = 4.02$, $SD = 0.40$) reported greater liking of the CTA than those in the regular-timing control condition ($M = 2.88$, $SD = 0.46$), Welch's $t(57.09) = 10.21$, $p < .001$, 95% CI [0.91, 1.36], $d = 2.64$.

In this simulated experiment, increasing train frequency produced a very large difference in reported attitudes. In real life, the CTA has declined our invitation to be manipulated with `set.seed(343)`.

> **!** Welch by Default; Effect Size Beside It
>
> Welch's t-test does not require the two population variances to be equal, so it is a sensible default for independent groups. Report the mean difference and its confidence interval, then add an effect size. The p-value cannot tell the reader how far apart the groups are.

8.11 How Many Riders Must I Bother?

The power chapter did all of this for the one-sample test. The function is the same, and exactly one argument changes: `type = "two.sample"`.

One thing about the answer catches almost everybody, so brace for it now.

```
# delta takes the effect size; with sd = 1, that means Cohen's d.
# Leave n = NULL and power.t.test() solves for the sample size.
ID.Power.80 <- power.t.test(n = NULL,
                            delta = cohens_d,      # our d, from Step 7
                            sd = 1,
                            sig.level = 0.05,
                            power = .80,
                            type = "two.sample",
                            alternative = "two.sided")

ceiling(ID.Power.80$n)
```

```
[1] 4
```

4 riders **per group**, so 8 people in total.

That number should make you deeply suspicious, and it should.

> **⚠** Per Group Means Per Group
>
> For `type = "two.sample"`, the `n` that comes back is the number of participants **in each group**. Double it for your recruitment target.
>
> Students report this number as their total sample roughly half the time, which halves their actual power and turns an 80% study into something closer to a coin flip with extra steps.

⚠ Advanced: Simulated Data Lies to You, Politely

Our simulated CTA effect is $d = 2.64$. That is not a large effect. That is not even a huge effect. That is two distributions in different postal codes, and it happens because I *built* the simulation with a clean one-point gap and a standard deviation of exactly 0.5.

Real psychological effects essentially never look like this. If a power analysis tells you that four people per group is sufficient, you have not discovered a cheap study. You have discovered that your assumed effect size is fictional.

Watch what a believable effect costs instead:

```
# What does each effect size cost, per group, for 80% power?
sapply(c(0.2, 0.5, 0.8, cohens_d),
       function(dd) ceiling(power.t.test(delta = dd, sd = 1, power = .80,
                                         type = "two.sample")$n))
```

```
[1] 394  64  26   4
```

Small, medium, large, and whatever we simulated. **That first number is the one to internalize.** Most of the effects you actually care about live near it.

8.11.1 Advanced: Hedges' Correction

Cohen's d is biased upward in small samples, for the reason the power chapter opened with: small samples underestimate σ, and σ is in the denominator. Smaller denominator, bigger d.

Hedges proposed a correction factor (Hedges 1981):

$$g = d \times \left(1 - \frac{3}{4(n_1 + n_2) - 9}\right)$$

```
hedges_g <- function(d, n1, n2) {
  d * (1 - 3 / (4 * (n1 + n2) - 9))
}

g_cta <- hedges_g(cohens_d, n, n)
round(c(d = cohens_d, g = g_cta), 4)
```

```
     d      g
2.6371 2.6028
```

With 30 per group the correction is small. With 8 per group it is not.

```
# The same true d of .8, "measured" at five different group sizes
round(sapply(c(5, 10, 20, 50, 200),
             function(k) hedges_g(0.8, k, k)), 4)
```

```
[1] 0.7226 0.7662 0.7841 0.7939 0.7985
```

A true d of .8 estimated from five people per group is meaningfully inflated. This is the arithmetic behind your instinct to distrust a huge effect from a tiny study, and it is a real correction, not a vibe.

8.12 The Short Story

- An independent-samples t-test compares two separate groups: the difference between the means, divided by the standard error of that difference.
- Welch's version is R's default and the better habit. The pooled version assumes equal variances and is what your lab computes by hand to show you the machinery.
- Formula syntax (`Liking ~ Condition`) is the shape everything in Part 2 will use. R subtracts groups in factor-level order, so if the sign looks backwards, check your levels before you check your arithmetic.
- Random assignment, not the t-test, is what licenses a causal sentence.
- Cohen's d puts the gap in pooled standard-deviation units. Report the interval and the effect size; a lone p-value tells the reader nothing about size.

Which Test Comes Next?

Use the independent-samples t-test when two *different* groups of people are compared on one measure.

If you measured the **same people twice**, the groups are not independent and this test is the wrong tool. Continue to the paired-samples t-test chapter, where the fact that Rider 12 appears in both columns stops being a problem and starts being the whole point.

If you have **more than two groups**, or a predictor that is a number rather than a label, the t-test runs out of road. Part 2 rebuilds all of this as regression, which handles both without complaint.

9 Paired-Samples t-Test

The Same People Have Returned

Use a paired-samples t-test when the **same people are measured twice** or when observations form meaningful pairs. The two scores from one person know each other. Pretending they are independent is statistical identity theft.

The central idea is simple: turn each pair into one difference score, then ask whether the average difference is zero. That is the whole test. Nobody needs a random effect yet. The random effects are waiting in a later chapter, and they will want receipts.

9.1 Why Pair the Scores?

An **independent-samples** design compares different people. Half the students receive chocolate during a lecture and half receive nothing except the lecture itself, which already seems cruel. Each student contributes one score.

A **repeated-measures** design measures the same students twice. Every student attends one lecture without chocolate and one with chocolate. Each student is now their own control.

A **matched-pairs** design uses different people who were deliberately matched on something important, such as age or baseline ability. This can help, but matched strangers are not magical clones. If you place two undergraduates beside each other because they have the same GPA, one may still be a morning person while the other communicates only through exhausted grunting. The analysis is identical (pair, difference, test), but matched strangers usually correlate far less than a person does with themself, so the power benefit is real and smaller. Watch for the $-2rS_1S_2$ term later in this chapter, because everything depends on that r.

Repeated scores tend to be correlated because stable characteristics follow people into both conditions. A student who generally looks at the screen for a long time may do that with or without chocolate. Another student may stare out the window in both conditions while contemplating escape. Pairing lets us remove some of those stable person-to-person differences.

9.2 A Chocolate-Fixation Study

Suppose we ask whether eating a small piece of chocolate during a lecture changes **fixation time**: the average number of seconds a student's eyes remain on the screen per glance. We measure 48 students twice:

- once with no chocolate;
- once with chocolate.

This example is simulated. Do not distribute mystery candy to human participants and announce that a textbook authorized it. I mean, I do, but no one tell the IRB (ethics review board). *Snitches get no chocolate, also maybe stitches.*

The Price of Measuring the Same Person Twice

Measuring someone twice buys you statistical power (an easier time seeing an effect if one actually exists), but it charges you for the privilege. Three things ride along with that second measurement:

- **Practice effects.** People get better at a task simply by having done it once. *We cannot wipe their memories between trials, pesky IRB again.*
- **Fatigue effects.** They get worse on the task because they are tired and would like to go home. *I tried to lock the doors, but Fire Marshal Bill paid me a visit.*
- **Carryover.** The first condition is still working on them when the second one starts. *The accumulation of experience, always causing problems.*

Here is the problem for our study. If every student sits through the no-chocolate lecture first and the chocolate lecture second, then "chocolate increased fixation time" and "students settled down in the second session" are the same sentence. No t-test can separate them, because nothing in the data distinguishes them.

The fix is **counterbalancing**. Half the students get chocolate first and half get it second. Order effects still happen, but now they land on both conditions equally and cancel out of the comparison. So assume we counterbalanced, because we are professionals.

Counterbalancing does not fix everything. Chocolate eaten in the first session does not un-eat itself ten minutes later, so with sessions that close together the sugar is still in the room during the control condition. Spacing matters, and some carryover cannot be designed away at all.

```
set.seed(343)

n <- 48

# Each student gets a personal tendency to stare, high or low, that follows
# them into BOTH conditions. This is what makes the two scores correlated,
# and it is the thing pairing will subtract back out.
participant_effect <- rnorm(n, mean = 0, sd = 1.20)

paired_wide <- data.frame(
  participant = factor(seq_len(n)),
  # each condition = a baseline + that person's tendency + fresh noise
  no_chocolate = 5.00 + participant_effect + rnorm(n, 0, 0.80),
  chocolate = 5.75 + participant_effect + rnorm(n, 0, 0.80)
)

# one difference score per person: the whole test runs on this column
paired_wide$difference <-
  paired_wide$chocolate - paired_wide$no_chocolate

# pivot_longer() stacks the two condition columns into one, turning
# 48 rows into 96. See the wide-vs-long section below for why we want both.
paired_long <- tidyr::pivot_longer(
  paired_wide,
  cols = c(no_chocolate, chocolate),   # the columns to stack
  names_to = "condition",              # new column holding the old column names
  values_to = "fixation"               # new column holding the values
)
```

```r
paired_long$condition <- factor(
  paired_long$condition,
  levels = c("no_chocolate", "chocolate"),
  labels = c("No chocolate", "Chocolate")
)
```

Those last few lines built the same data twice, in two different shapes. This confuses more students than the statistics do. Slow down here.

Wide data give each person one row, with a separate column for each condition:

```r
head(paired_wide[, c("participant", "no_chocolate", "chocolate")], 3)
```

```
  participant no_chocolate chocolate
1           1     3.216540  3.378271
2           2     6.692332  5.360605
3           3     4.404871  6.355612
```

Three students, three rows. The pairing is impossible to miss here, because each person's two scores sit side by side and you can subtract across the row with your eyes.

Long data give each *measurement* its own row, so one person now takes up two:

```r
head(paired_long[, c("participant", "condition", "fixation")], 6)
```

```
# A tibble: 6 x 3
  participant condition    fixation
  <fct>       <fct>           <dbl>
1 1           No chocolate     3.22
2 1           Chocolate        3.38
3 2           No chocolate     6.69
4 2           Chocolate        5.36
5 3           No chocolate     4.40
6 3           Chocolate        6.36
```

The same three students. The same six numbers. Six rows instead of three. Participant 1 now appears twice, and a new `condition` column keeps track of which measurement each row belongs to.

Neither shape is "more better" than the other. They are the same data wearing different pants, and different tools insist on different pants:

- `t.test(..., paired = TRUE)` wants **wide**, because you hand it two vectors and it matches them up position by position. Row 1 with row 1, row 2 with row 2, and so on.
- `ggplot()` wants **long**, because it needs one column to put on the x axis and one to put on the y axis. The plot below draws a separate line for each participant, which is only possible if there is a `condition` column to move along.
- Mixed regressions will also want long, for the same reason, when we get there.

`tidyr::pivot_longer()` turns wide into long, and `pivot_wider()` turns it back. You will do this constantly, usually at the precise moment a plot refuses to work.

> **How to Tell Which Shape You Are Holding**
>
> Count the rows and compare that to the number of humans.
>
> If the two match, you are in wide format. If you have more rows than people, you are in long format, and some column in there is telling you what the extra rows are for.

9.3 Look at the Pairs Before Testing Them

```
library(ggplot2)

ggplot(
  paired_long,
  aes(x = condition, y = fixation, group = participant)
) +
  geom_line(alpha = 0.30, color = "gray55") +
  geom_point(alpha = 0.60, color = "steelblue4") +
  stat_summary(
    aes(group = 1), fun = mean, geom = "line",
    color = "firebrick", linewidth = 1.3
  ) +
  stat_summary(
    aes(group = 1), fun = mean, geom = "point",
    color = "firebrick", size = 3
  ) +
  labs(
    x = NULL,
    y = "Mean fixation time (seconds)",
    title = "Every gray line is one exploited volunteer"
  ) +
  theme_minimal(base_size = 11)
```

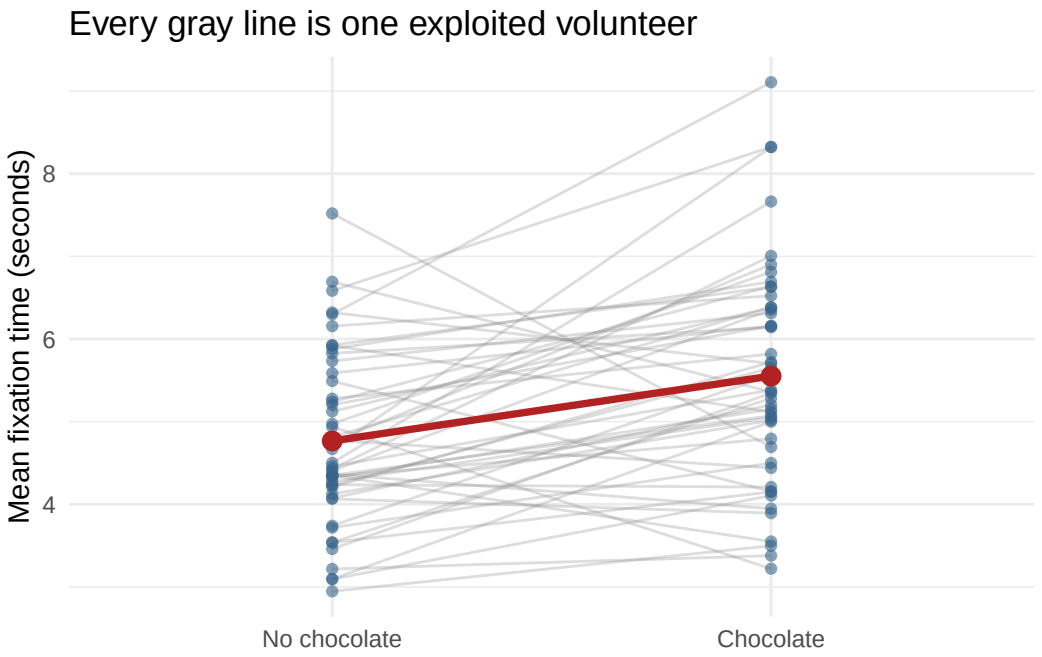


Figure 9.1: Each gray line is one student measured twice. The red line is the average change.

The red line shows the mean change. The gray lines show something the means hide: some people change more than others, and a few may move in the opposite direction. If you plot only two bars, all those people are compressed into rectangular coffins.

9.4 The Paired t-Test Is a One-Sample t-Test in Disguise

For each person, compute a difference score:

$$D_i = Y_{i,\text{chocolate}} - Y_{i,\text{no chocolate}}$$

Then ask whether the population mean difference is zero:

$$t = \frac{M_D - 0}{S_D/\sqrt{n}}$$

This is the one-sample t-test from the earlier chapter. The paired t-test does not compare two unrelated piles of scores. It reduces every pair to one difference and tests the set of differences.

```
paired_result <- t.test(
  paired_wide$chocolate,       # first vector
  paired_wide$no_chocolate,    # second vector, in the SAME person order
  paired = TRUE                # the one argument that changes everything
)

paired_result
```

```
    Paired t-test

data:  paired_wide$chocolate and paired_wide$no_chocolate
t = 4.3366, df = 47, p-value = 7.597e-05
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 0.4209697 1.1495099
sample estimates:
mean difference
      0.7852398
```

`paired = TRUE` is doing all the work in that call. Without it, R would compare the two columns as though they came from 96 unrelated students. With it, R matches them row by row, subtracts, and tests the differences. Which means **row order matters**: row 1 of the first vector must be the same human as row 1 of the second.

9.4.1 R Has Become Picky About the Formula Method

The formula method of `t.test()` does not accept `paired = TRUE`, so this will fail:

```
t.test(fixation ~ condition, data = paired_long, paired = TRUE)
```

Use the two paired vectors, as above. Even better, make the pairing visible by calculating the differences yourself:

```
# A one-sample t-test on the difference column, asking whether it differs from 0.
# No paired = TRUE needed, because we already did the subtracting ourselves.
difference_result <- t.test(paired_wide$difference, mu = 0)
difference_result
```

```
    One Sample t-test

data:  paired_wide$difference
t = 4.3366, df = 47, p-value = 7.597e-05
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 0.4209697 1.1495099
sample estimates:
mean of x
0.7852398
```

The t statistic, degrees of freedom, and p-value match. R has not discovered two separate truths. We asked the same question twice.

> **!** The Unit of Analysis Is the Pair
>
> We began with 96 scores, but the test has 48 difference scores. Therefore, $df = n - 1 = 47$. Counting every measurement as an independent person would double the apparent sample size and create confidence unsupported by the design. The spreadsheet may have 96 rows. The study still has 48 humans.

9.5 Effect Size

A common standardized effect for paired data is Cohen's d_z:

$$d_z = \frac{M_D}{S_D}$$

```
# Note the denominator: the SD of the DIFFERENCES, not of either condition.
d_z <- mean(paired_wide$difference) /
  sd(paired_wide$difference)

d_z
```

```
[1] 0.6259363
```

This standardizes the mean change by the standard deviation of the **difference scores**. Say which version of d you used because repeated-measures effect sizes have several possible denominators. Calling all of them "Cohen's d" and wandering away is how future readers begin sharpening office supplies.

9.6 Why Pairing Helps

Forget chocolate for a minute. Suppose I want to know whether wearing shoes makes you taller.

The bad design. I measure 50 people wearing shoes and 50 different people barefoot, then compare the two group averages. The shoe effect is about an inch. But people range from roughly five feet to six and a half feet, so the differences *between people* are enormous compared with the thing I actually care about. My one inch is drowning in eighteen inches of human height that has nothing to do with footwear.

The good design. I measure the same person twice, once in shoes and once barefoot. Now each person's own height cancels out of their own difference score, because their height is sitting in both measurements. The six-foot-four person gives me about an inch. The five-foot-two person also gives me about an inch. The noise disappeared and the effect did not.

That is the whole idea, and your fixation data works the same way. Some students stare at the screen longer than others no matter what is in their mouth. Comparing each student with themself strips that variation out before it ever reaches the test, and what survives is the part chocolate is responsible for.

There is a catch, and it is the reason pairing is not automatically better. This only works to the extent that a person's two scores actually travel together. Height travels together beautifully, because you are the same height in both measurements. If a person's score today told you nothing whatsoever about their score tomorrow, there would be nothing stable to subtract out, and pairing would buy you almost nothing.

If you want to see exactly how that cancelling works, and the term in the formula that controls how much you get, the next section has the details.

9.7 Advanced: Why Pairing Can Help

Here is the same argument with the algebra attached. The variance of a difference score is:

$$S_D^2 = S_1^2 + S_2^2 - 2rS_1S_2$$

That r is the correlation between a person's two scores, and it is the shoes-and-height idea in symbolic form. When the two measurements are positively correlated, the last term subtracts variability. Stable person differences cancel: the consistently attentive student is compared with themself, and the student planning an elaborate escape is also compared with themself.

Notice what happens at the extremes. If $r = 0$, the two scores know nothing about each other, that last term is zero, and the variance of the difference is just $S_1^2 + S_2^2$. You paired the data and got nothing for it. The closer r climbs toward 1, the more the last term eats, and the smaller the difference scores' variance becomes.

This often gives a smaller standard error than an independent-samples analysis. Pairing does not guarantee more power, however. If the repeated scores barely correlate, little stable variation is removed. Pairing also cannot repair a terrible measure. Measuring attention by counting how often someone blinks at the ceiling remains a terrible plan in both conditions.

9.7.1 What Does Pairing Buy You?

We have the data, so we do not have to speculate. First, how correlated are the two measurements? Then we run the analysis **incorrectly on purpose**, treating our 96 scores as if they came from 96 different students who happened to wander in:

```
# How much do a person's two scores agree with each other?
cor(paired_wide$chocolate, paired_wide$no_chocolate)

# The WRONG test, run deliberately: the same numbers with the pairing thrown away.
# Do not copy this into your homework.
unpaired_result <- t.test(paired_wide$chocolate, paired_wide$no_chocolate)

unpaired_result
```

```
[1] 0.4638286

    Welch Two Sample t-test

data:  paired_wide$chocolate and paired_wide$no_chocolate
t = 3.2144, df = 88.988, p-value = 0.001822
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 0.2998481 1.2706316
sample estimates:
mean of x mean of y
 5.551277  4.766037
```

Same numbers. Same two means. The same mean difference of 0.79 seconds. Nothing about the data changed, only what we told R about where the numbers came from.

The paired test gave $t(47) = 4.34$. Ignoring the pairing gives $t(89) = 3.21$, roughly 1.35 times smaller. The standard error tells the same story: 0.18 seconds when we keep the pairing, 0.24 seconds when we discard it.

Where did the extra error come from? From the students. Some people simply stare longer than others, in both conditions, for reasons that have nothing to do with chocolate. The paired test subtracts that stable personal tendency out and never looks at it again. The unpaired test has no idea those students are the same humans, so all of that person-to-person variability gets dumped into the error term, where it makes the difference between conditions look less trustworthy than it is.

That is the whole argument for within-subjects designs, and it is why the design chapter of every methods book keeps yelling about them. It is also, in this case, a correlation of only 0.46. A stronger correlation would have bought even more.

9.8 Assumptions You Cannot Skip

For the paired-samples t-test:

- the observations form genuine pairs;
- different pairs are independent of one another;
- the outcome is quantitative;
- the **difference scores** have no severe outliers;
- for small samples, the population distribution of difference scores is reasonably normal.

The test does **not** require the two condition variances to be equal. It operates on one set of differences. Inspect those differences, not two separate normality tests performed as ritual offerings.

```
ggplot(paired_wide, aes(x = difference)) +
  geom_histogram(binwidth = 0.35, color = "white", fill = "steelblue4") +
  geom_vline(xintercept = 0, linetype = 2, color = "firebrick") +
  labs(
    x = "Chocolate minus no-chocolate fixation time",
    y = "Number of students"
  ) +
  theme_minimal(base_size = 11)
```

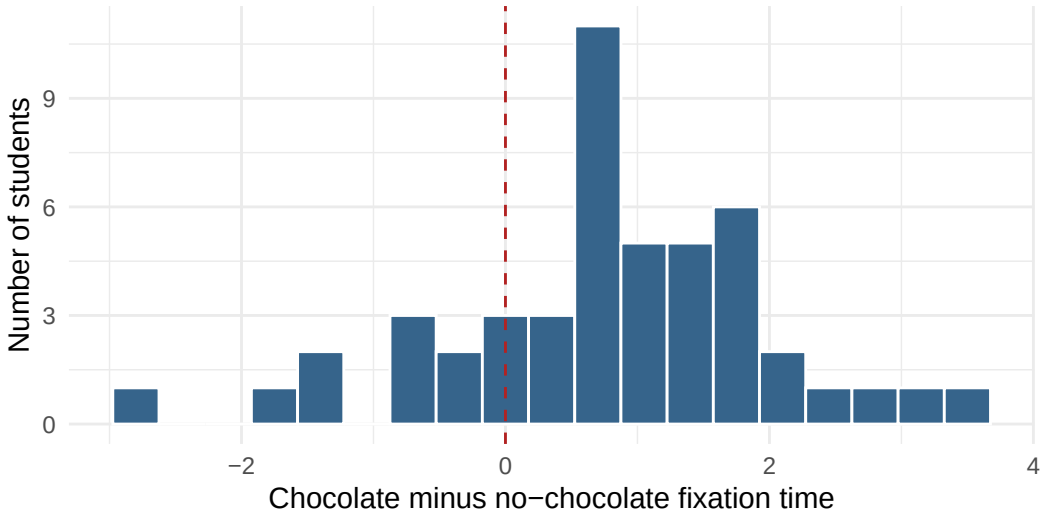


Figure 9.2: The 48 difference scores. This is the distribution the test actually looks at.

9.9 Reporting the Result

For the present study, a concise report could be:

> Students showed longer fixation times during the chocolate condition ($M = 5.55$, $SD = 1.33$) than during the no-chocolate condition ($M = 4.77$, $SD = 1.05$). The mean paired difference was 0.79 seconds, 95% CI [0.42, 1.15], $t(47) = 4.34$, $p < .001$, $d_z = 0.63$.

The statistic cannot prove that chocolate *caused* better attention unless the study was properly randomized and alternative explanations were controlled. It certainly cannot prove that replacing every lecture with a bucket of candy is sound educational policy.

9.10 When Two Measurements Are Not Enough

One difference score works beautifully when each person has two measurements and the question is about their average change. With three or more occasions, missing observations, both within-person and between-person predictors, or people who change at different rates, one difference score cannot represent the whole design.

That is where mixed regression begins. It keeps the repeated observations, tells the model which rows belong to the same human, and allows people to differ without creating one indicator variable for every person who wandered into the study.

9.11 Power: How Many People Would You Need?

Same function again, with `type = "paired"`. Two things change from the independent-samples version: the effect size you feed it is d_z, and the `n` that comes back is the number of **pairs**, which here means the number of people, since each person supplies both scores.

```r
Paired.Power.80 <- power.t.test(n = NULL,
                                delta = d_z,       # the effect size for paired data
                                sd = 1,
                                sig.level = 0.05,
                                power = .80,
                                type = "paired",
                                alternative = "two.sided")

ceiling(Paired.Power.80$n)
```

```
[1] 23
```

23 pairs, meaning 23 people, each measured twice.

9.11.1 Why Pairing Is Cheaper

Here is the payoff for repeated measures, and it is the whole reason this design is worth the counterbalancing headache. Compare what it costs to detect the *same* effect in the two designs:

```r
# Same effect size, same power, same alpha. Only the design changes.
n_independent <- ceiling(power.t.test(delta = d_z, sd = 1, power = .80,
                                      sig.level = .05,
                                      type = "two.sample")$n)

n_paired <- ceiling(power.t.test(delta = d_z, sd = 1, power = .80,
                                 sig.level = .05,
                                 type = "paired")$n)

c(independent_per_group = n_independent,
  independent_total     = 2 * n_independent,
  paired_people         = n_paired)
```

```
independent_per_group     independent_total         paired_people
                   42                    84                    23
```

84 people the hard way against 23 the smart way. The paired design needs far fewer humans, because pairing removes the stable person-to-person differences from the denominator, exactly as the shoes-and-height section promised:

$$S_D^2 = S_1^2 + S_2^2 - 2rS_1S_2$$

> **!** This Comparison Is Fair Only If You Are Careful
>
> The numbers above compare designs at the same d_z, and d_z is not on the same scale as the independent-samples d. A paired design with a strong within-person correlation can produce a large d_z from a modest raw change, because the denominator shrank.
>
> So: repeated measures usually *do* buy power, sometimes dramatically. But do not report a d_z of 1.2 and let a reader believe it means what a between-groups d of 1.2 would mean. **Say which effect size you used.** Every time. Repeated-measures effect sizes have several possible denominators, and pretending otherwise is how future readers begin sharpening office supplies.

9.12 The Short Story

- Use a paired-samples t-test when observations form genuine pairs.
- The same person measured twice is the most common paired design.
- Compute one difference score for each pair.
- A paired-samples t-test is a one-sample t-test on those differences.
- The unit of analysis is the pair, not the individual measurement.
- Inspect the distribution and outliers of the **differences**.
- Report the mean change, confidence interval, p-value, and the specific paired effect size you used.
- When the design grows beyond one clean pair of scores, continue to the mixed-regression chapter.

The grand lesson is not "memorize another command." It is "keep the two scores attached to the human who produced them." The humans already know they are the same person. R needs paperwork.

Part II

Putting Lines Through Things

10 Covariance and Correlation

10.1 Quit School, Become a Pirate, Save the Earth

Here is a graph.

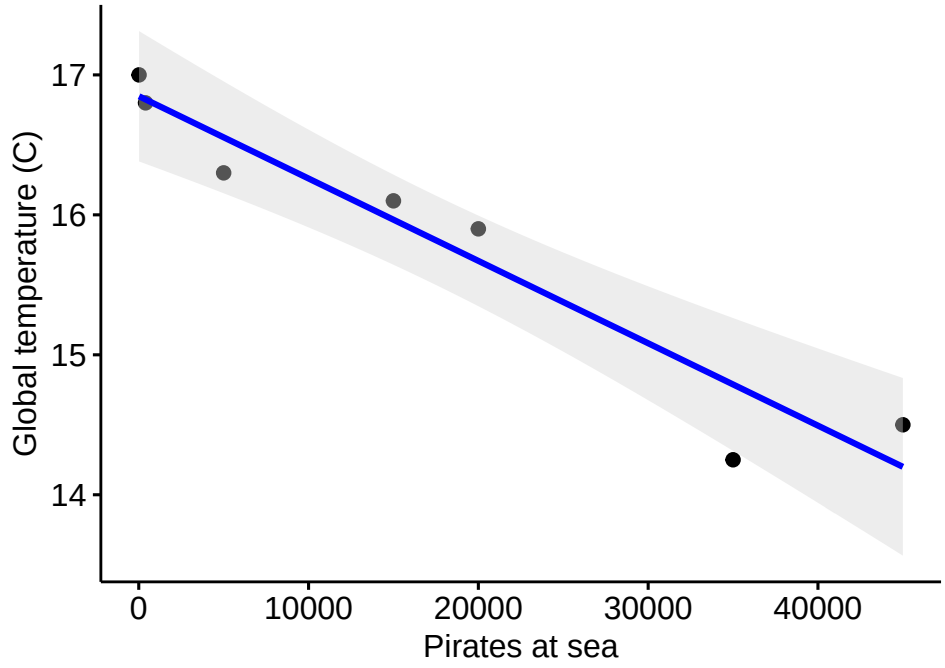


Figure 10.1: Global temperature against the number of pirates at sea. The pirate counts are made up. The correlation is not.

The pirates went away. The planet got hotter. Those two facts move together with a correlation of -.96, which is stronger than almost anything you will ever measure in psychology.

So the solution is obvious. We all quit school, buy a boat, and go be pirates until the planet cools back down.

I should probably admit that the pirate counts are invented. Nobody ran a census of pirates. The temperatures are roughly real, and that is the part worth staring at: put one honest trend next to one fabricated one, and the arithmetic cannot tell them apart. It returned -.96 with a completely straight face.

That number is what this chapter builds. It is the first tool in this book that looks at two variables at once, which matters because nearly every question you actually care about is a two-variable question. Does studying go with better grades? Does anxiety go with worse performance? Does the treatment go with the recovery?

It is also the floor that the rest of the book stands on. Regression, which takes up the next eight chapters, is this same idea in better clothes. If this one clicks, most of Part 2 is variations on it.

What it will never do is tell you *why*. It does not know what a pirate is. It has never been outside.

Two things moving together is a fact. Why they move together is an argument, and you have to make that one yourself.

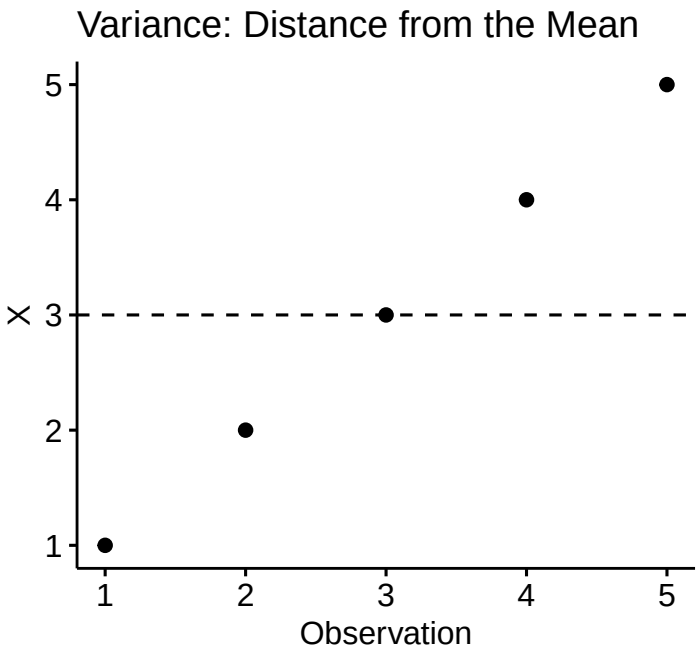
10.2 A Visual Intuition: Variance, Covariance, and Correlation

Before we can trust that number, we should know what it is made of. It gets built in three steps, and we will look at each one with a dataset small enough to check by hand.

We'll start with a very simple dataset and visualize three situations:

- One variable by itself (variance)
- Two variables moving together (positive covariance)
- Two variables moving in opposite directions (negative covariance)

10.2.1 Variance: Spread Around a Mean

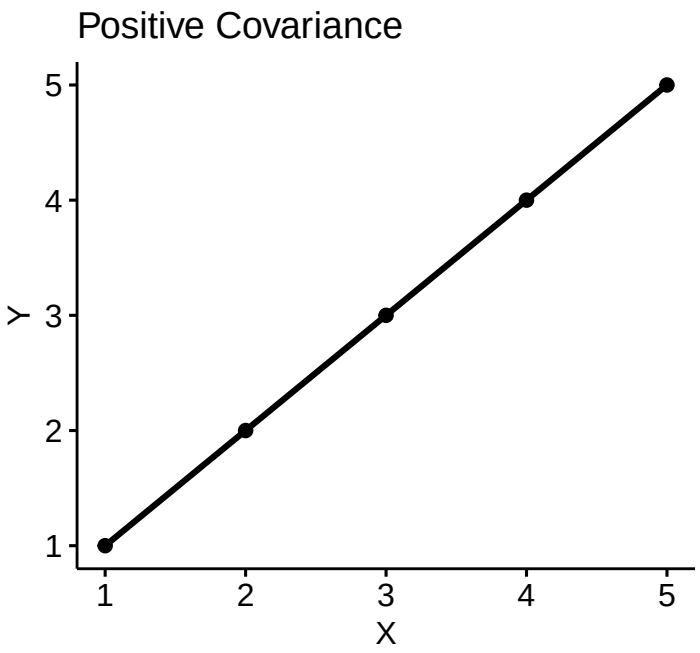


- Each point deviates from the mean of X
- Squaring those deviations and averaging them gives us variance
- Variance does not care about relationships with other variables

10.2.2 Do Two Variables Move Together?

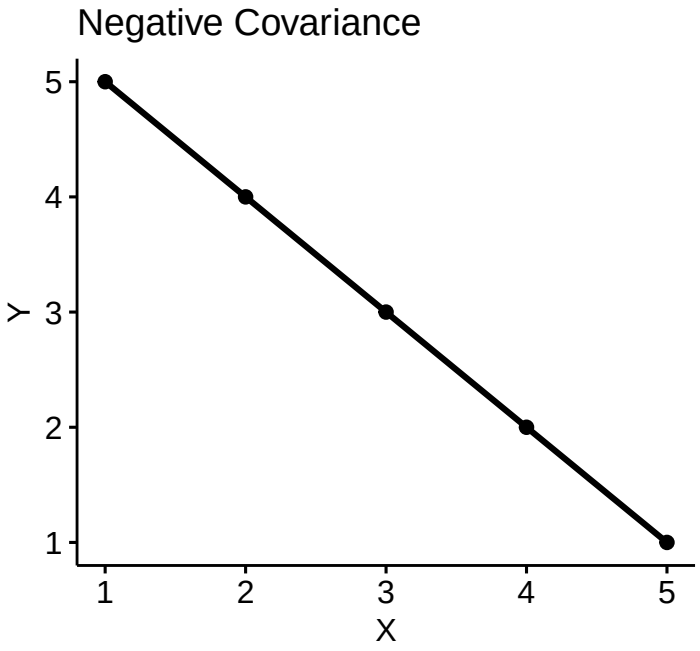
Covariance answers a two-variable question:

- When X is above its mean, is Y also above its mean (or below)?



- High X pairs with high Y
- Low X pairs with low Y
- Cross-products $(X - M_X)(Y - M_Y)$ are positive
- "co-" variance is positive

10.2.3 Negative Covariance



- High X pairs with low Y
- Low X pairs with high Y
- Cross-products $(X - M_X)(Y - M_Y)$ are negative
- "co-" variance is negative

10.2.4 What is Correlation?

Covariance has one big problem: *Its size depends on the units of measurement.*

Correlation fixes this by standardizing covariance:

- Variance → standardized = 1
- Covariance → standardized between −1 and +1

Correlation (r) tells us:

- Direction (positive vs negative)
- Strength (how tightly the points cluster)
- Scale-free comparison

You'll see shortly that:

- Perfect positive covariance yields $r = +1$
- Perfect negative covariance yields $r = -1$
- No systematic covariance yields $r = 0$

10.2.5 Big Picture (Preview)

Concept	What it measures	How many variables
Variance	Spread around a mean	1
Covariance	Shared movement	2
Correlation	Scaled shared movement	2

- Variance tells us how much *one* variable moves.
- Covariance tells us whether *two* variables move together.
- Correlation rescales covariance so we can compare relationships across studies.

Now that we have the picture, we can formalize each idea.

10.3 Variance, The Scary Math

Variance is our primary measure of spread, defined as the sum of squared variation around a mean. It is also load-bearing for most of the distributional theory in this book, which is why it keeps turning up. We'll use s for the sample standard deviation and s^2 for the sample variance.

Here are several equivalent ways to write it:

$$s^2 = Var(x) = \frac{\sum SS_X}{n-1} = \frac{\sum (X - M_X)^2}{n-1} = \frac{\sum (X - M_X)(X - M_X)}{n-1}$$

That last version is worth staring at. Written out, variance is a variable multiplied by itself.

10.4 Variance but now in 2-D: Covariance

Variance summarizes univariate distance from a mean. Covariance summarizes bivariate distance from a *pair* of means, which is how it describes the association between two variables. What was a sum of squares becomes a sum of products:

$$Cov_{xy} = \frac{\sum(X - M_X)(Y - M_Y)}{n - 1} = \frac{SP_{xy}}{n - 1}$$

> **Why You Have to Sit Through This Part**
>
> This is the most formula-heavy stretch of the chapter, and it is fair to ask why you are being made to look at it.
>
> You are not here because I want you to suffer. The sum of squares, the sum of products, and the $n - 1$ underneath them are the parts that nearly everything later in this book is assembled from. Regression slopes, R^2, model comparison, and the mixed models at the end of Part 2 are all built out of these same few pieces, usually without anyone stopping to reintroduce them.
>
> So learn them here, on five numbers you can check by hand in about a minute. Every time they come back wearing a different name, and they will, you will already know what you are looking at.

10.4.1 Perfect Positive Relationship

Why would we do this? Let's start with an illustrative example, using the smallest dataset that will sit still. Take one variable, $X = 1, 2, 3, 4, 5$, and find its variance.

```
X = c(1,2,3,4,5)
M_x<-mean(X)
SS_x <- sum((X-M_x)^2)
Var_X=SS_x/(length(X)-1) #or just do var(X)
Var_X
```

```
[1] 2.5
```

10.4.2 Identical Variables

Now imagine two variables that are exact copies of one another, as though we measured something very stable twice: $X = 1, 2, 3, 4, 5$ and $Y = 1, 2, 3, 4, 5$. Both variances should come back identical, because the two columns *are* identical.

```
X = c(1,2,3,4,5)
Y = c(1,2,3,4,5)
var(X)
```

```
[1] 2.5
```

```
var(Y)
```

```
[1] 2.5
```

So what is Cov(xy)?

```
cov(X,Y)
```

```
[1] 2.5
```

Why is it the same number? Look again at the last version of the variance formula. It is already written as $\frac{\sum(X-M_X)(X-M_X)}{n-1}$, a variable multiplied by itself. Replace one of those X's with a perfect copy called Y and not one thing about the arithmetic changes. Variance is covariance with yourself, and that is the punchline of this entire chapter.

10.4.3 Perfect Negative Relationship

Now imagine two variables that are exact opposites, like a student's liking for stats and their hatred of the same class: $X = 1, 2, 3, 4, 5$ and $Z = 5, 4, 3, 2, 1$. Notice the variances have not changed. Reversing the order of a set of numbers does nothing to how spread out they are.

```
X = c(1,2,3,4,5)
Z = c(5,4,3,2,1)
var(X)
```

```
[1] 2.5
```

```
var(Z)
```

```
[1] 2.5
```

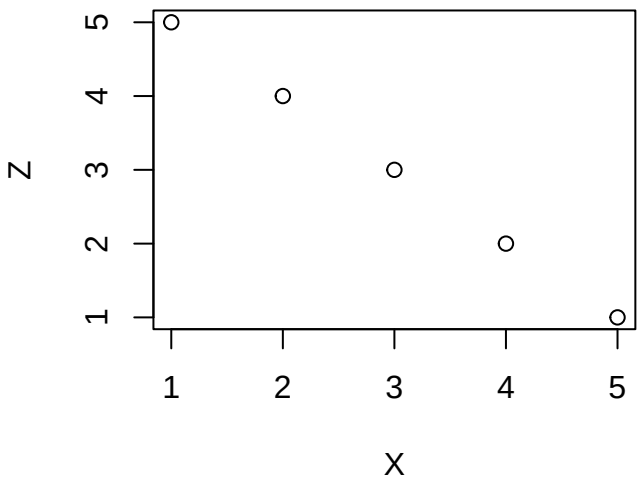
What's Cov(xz)?

```
cov(X,Z)
```

```
[1] -2.5
```

The same magnitude as the identical-variables case, with the sign flipped. The plot below shows what that sign is reporting: every step up in X is matched by a step down in Z.

```
plot(X, Z)
```



10.4.4 Why the Sign Flips

When the relationship was perfectly positive, every cross-product was positive. Now every one of them is negative:

$$(-2 * 2) + (-1 * 1) + (0 * 0) + (1 * -1) + (2 * -2)$$

Add them up and the total is negative. That negative sign is the whole signal: it says the two variables move in opposite directions.

10.4.5 No Relationship

Now two variables that aren't strongly related, say liking for stats and liking for Alex: $X = 1, 2, 3, 4, 5$ and $A = 2, 5, 3, 1, 4$. The variances still have not changed, because A is just another reshuffling of the same five numbers.

```
X = c(1,2,3,4,5)
A = c(2,5,3,1,4)
var(X)
```

```
[1] 2.5
```

```
var(A)
```

```
[1] 2.5
```

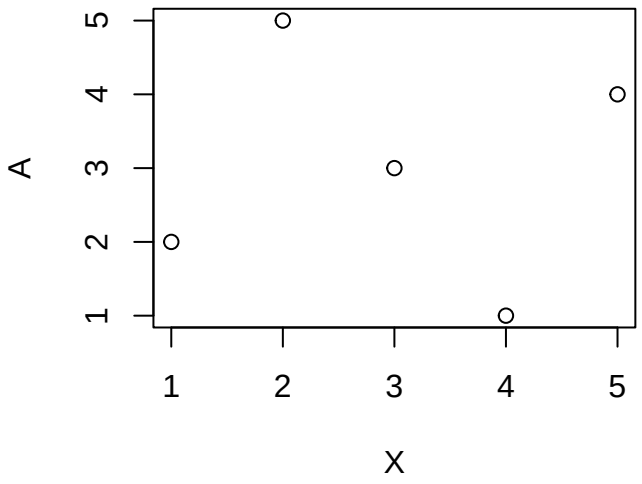
What's Cov(XA)?

```
cov(X,A)
```

```
[1] 0
```

Zero. The plot below shows why. There is no consistent uphill or downhill pattern, so the cross-products never accumulate in either direction.

```
plot(X, A)
```



10.4.6 Why Zero?

Turn both variables into deviations from their means:

- Old: $X = 1, 2, 3, 4, 5,\ A = 2, 5, 3, 1, 4$
- Now: $X = -2, -1, 0, 1, 2,\ A = -1, 2, 0, -2, 1$

Then add up the cross-products:

$$(-2 * -1) + (-1 * 2) + (0 * 0) + (1 * -2) + (2 * 1)$$

$$2 + (-2) + 0 + (-2) + 2 = 0$$

The positive products and the negative products cancel exactly, so the covariance is zero. There is no linear relationship between X and A.

10.4.7 Covariance Review

Assume for the moment that $Var(x) = Var(y)$.

- When the relationship between x and y is perfect, $Cov(xy) = Var(x) = Var(y)$.
- When the relationship is perfect but the variances differ, $Cov(xy) = \sqrt{Var(x) * Var(y)}$. That is the biggest a covariance can ever get.
- When the relationship is perfectly inverse, the same bound applies with the sign flipped: $Cov(xy) = -\sqrt{Var(x) * Var(y)}$.
- When there is no relationship, $Cov(xy) = 0$, because the positive and negative products cancel.

Covariance can never exceed $\sqrt{Var(x) * Var(y)}$ in either direction. Dividing by that maximum is exactly what a correlation does, which is why r cannot escape the range -1 to $+1$.

10.5 Correlations: Our second most common workhorse

Correlation is a broad term for any scaled measure of association, which means there is no single correlation. There is a whole family of them, and different members answer different questions about different kinds of variables.

Wait. There are multiple types of correlations?

Yes. We take joy in finding new ways to confuse you on exams.

The two you will actually use are Pearson's r and Spearman's r_s. Both are built to the same specification: they run from -1 to $+1$, and zero means no relationship. What counts as "no relationship" is where they part company. For Pearson, zero means no **linear** association. For Spearman, zero means no **monotonic** association. Either way, a strong U-shaped relationship can still produce zero, because correlation is very talented at missing questions it was never asked.

Cohen gave r the same small, medium, and large treatment he gave d (Cohen 1988), and you will be expected to know these:

Size	r
Small	.10
Medium	.30
Large	.50

> **These Adjectives Are Conventions, Not Laws**
>
> You met this warning in the power chapter attached to d. It applies at least as hard to r.
>
> An r of .10 between a cheap screening question and later dropout can be worth millions of dollars and a redesigned advising program. An r of .45 between two questionnaires that share half their items is worth nothing, because you have largely correlated a thing with itself. Interpret magnitude in the research context, or do not interpret it.

One important thing to know: Everything correlates with everything, which Paul Meehl calls the "crud factor" (also called ambient correlational noise) (Meehl 1990a, 1990b). Our goal is to determine how much, and we will deal with two variables at a time before exploring the problems created by three or more variables.

10.5.1 Correlation Types

A few popular correlations between two variables:

- Pearson's r (interval by interval) [for population = ρ, for sample = r]
- Spearman's rho (interval by ordinal) [for population = ρ_s, for sample = r_s]

Other types we will not cover:

- Kendall's tau (τ), a rank-based measure that handles pair concordance and ties differently from Spearman's correlation
- Point-biserial correlation (quantitative by genuinely dichotomous)
- Polychoric (ordinal vs ordinal) [used more in psychometrics or factor analysis of ordinal by ordinal]
- Tetrachoric correlation (dichotomous vs. dichotomous, under a latent-continuous-variable model)
- Psychopathic correlation, used only when you want to cause insanity in the reader of your paper

These measures do **not** all share one assumption set. Classical inference for Pearson's correlation commonly assumes independent pairs and, for its exact small-sample test, bivariate normality. Bivariate normality describes the joint distribution of two variables; it does not mean they become normal when "added together," and it certainly does not mean the two variables must be independent. If they were independent, their population correlation would be zero, which would make this a remarkably short chapter.

10.5.2 Pearson's Correlation

Most common type you will encounter and is a parametric method.

$$r_{xy} = \frac{\sum (X - M_X)(Y - M_Y)}{\sqrt{\sum (X - M_X)^2 \sum (Y - M_Y)^2}}$$

remember $SS = \Sigma(X - M)^2$, so thus,

$$r_{xy} = \frac{SP}{\sqrt{SS_X SS_Y}}$$

- Numerator = How much they vary together (covariance)
- Denominator = The product of how much they vary alone (variance)
- Values are bounded between -1 and 1

or more simply (see Cohen's textbook for the derivation):

$$r_{xy} = \frac{\sum xy}{\sqrt{\sum x^2 \sum y^2}}$$

10.5.3 A Study We Will Keep Using: Ice Cream and Happiness

Time for a running example, and something less alarming than pirates.

Suppose we ask 100 people two questions: how much ice cream they eat, and how happy they are. We expect the two to go together, because ice cream is delicious and happiness is what happens to people who eat delicious things. This example follows us for the rest of the chapter. It has the same structure as the pirate graph. Two variables, one span of observations, no control group, and no idea which way any arrow points.

Hold onto that. We will come back for it once we have a number.

10.5.4 Simulate Data

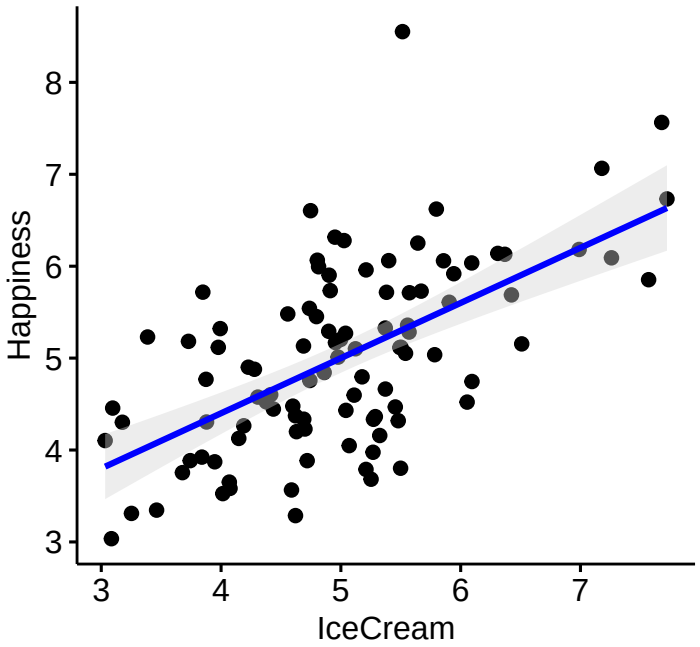
We will use the `mvrnorm` function (multivariate normal distribution) from the *MASS* package, but to do this we need to make a **covariance** matrix with a $r = .60$ and set the mean values for each variable (which I will set to 5 for each). With unit variances, this covariance matrix is also the correlation matrix, a freebie that will matter later.

```
#Set params
Means.XY<- c(5,5) #set the means of X and Y variables
r=.6 #Correlation value
CovMatrix.XY <- matrix(c(1,r,
                         r,1),2,2) # creates the covariate matrix

# Build the correlated variables using mvrnorm.
# Note: empirical=TRUE means make the correlation EXACTLY r.
# empirical=FALSE, the correlation value would be normally distributed around r
library(MASS) #create data
CorrData<-mvrnorm(n=100, mu=Means.XY,Sigma=CovMatrix.XY, empirical=TRUE)

#Convert them to a "Data.Frame", which is like SPSS data window
CorrData<-as.data.frame(CorrData)
#lets add our labels to the vectors we created
colnames(CorrData) <- c("Happiness","IceCream")
```

10.5.5 Plot the data



10.5.6 Run Pearson's r

The `cor.test` function runs Pearson's correlation. **Note:** You will notice that I have attached the data frame to each variable.

```
Corr.Result.1<-cor.test(CorrData$Happiness, CorrData$IceCream,
         method = c("pearson"))
Corr.Result.1
```

```
    Pearson's product-moment correlation

data:  CorrData$Happiness and CorrData$IceCream
t = 7.4246, df = 98, p-value = 4.193e-11
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.4574985 0.7124547
sample estimates:
cor
0.6
```

You can also call the function via a formula command.

```
Corr.Result.1<-cor.test(~Happiness + IceCream, data= CorrData,
         method = c("pearson"))
```

10.5.7 P-value for Pearson's Correlation

The classical p-value for Pearson's correlation is adapted from the t-distribution which we covered in chapter on one-sample t-test.

The statistic is built from r and the sample size, and nothing else:

$$t = \frac{r\sqrt{N-2}}{\sqrt{1-r^2}}$$

That $N-2$ is the degrees of freedom. Here is where the 2 went.

Before you can compute a single deviation score, you have to know where the middle is. So you estimate M_X from your own data, and that costs one degree of freedom. Then you estimate M_Y, and that costs another. One mean per variable, two variables, so $N-2$ are left. This is the same tax that makes each variance divide by $n-1$ rather than n: you pay once, per variable, for having to find its center first.

With our 100 people, that leaves 98 degrees of freedom.

You will meet this exact number again from the other direction. In regression, a straight line has two parameters, an intercept and a slope, and fitting each one costs a degree of freedom. Two parameters, $N-2$ left over. Same number, same reason, different story about it.

10.5.8 Report them in APA format

The `cor_apa` function in the APA package will report it in APA format for you. Here, $r(df)$ gives Pearson's r and its degrees of freedom, which is the $N-2$ we just paid for. The function has several output-format options.

$r(98) = .60$, $p < .001$

10.5.9 Pearson's correlation is scale-independent!

No matter the mean differences or range of scores, the Pearson's r will give the same results. We can also z-score the data and get the same result. However, if they are scaled non-linearly (sqrt, ^2, log,...) the correlation will change.

This is worth proving rather than saying with fancy words, so the next four sections do the same analysis four times while abusing the measurement scale in a different way each time. Watch the reported r in the corner of each plot. Three of the four will not move.

> **That Corner Label Is Not APA, and That Is Fine Here**
>
> The correlation printed on these plots comes from `ggpubr`, and it takes liberties. It writes a capital R where APA wants a lowercase italic r. It reports no degrees of freedom at all. It prints the p-value in scientific notation instead of as $p < .001$.
>
> None of that belongs in a manuscript. All of it is fine right now, because these four plots exist so we can glance at a graph and see whether the number moved. Writing mountains of code to produce a journal-ready figure, just to check something with our own eyes, is a waste of an afternoon.
>
> Keep that distinction for the rest of the book. Some code is for us: fast, slightly ugly, written to figure out what the data are doing. Other code is for the journal: slow and fussy, written so the output looks the way reviewers expect it to look. The `cor_apa` call above is the second kind. This corner label is the first kind. Use each where it belongs, and never paste the first kind into a paper.

10.5.10 Scale change: Add a constant (shift the mean)

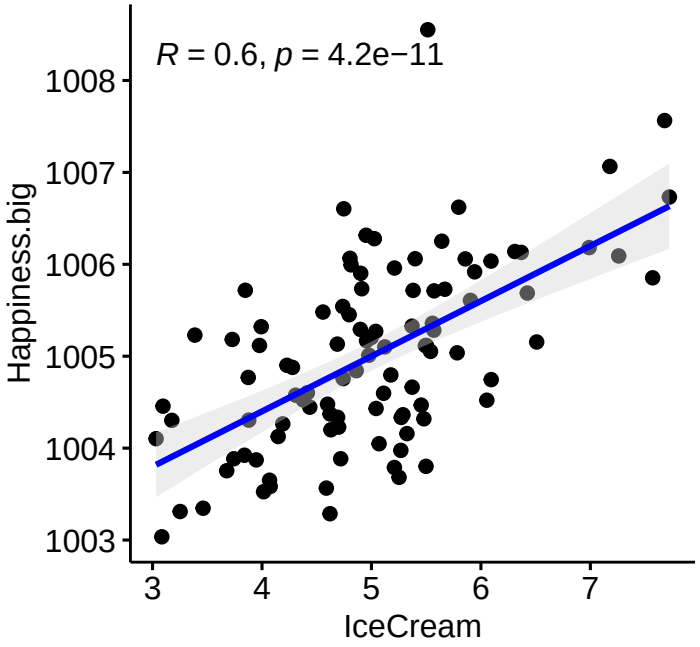


Figure 10.2: X axis: ice cream, untouched. Y axis: happiness, plus 1000. We moved the mean and nothing else.

```
r(98) = .60, p < .001
```

10.5.11 Scale change: Z-scored

Remember that, $Z = \frac{X-M}{S}$

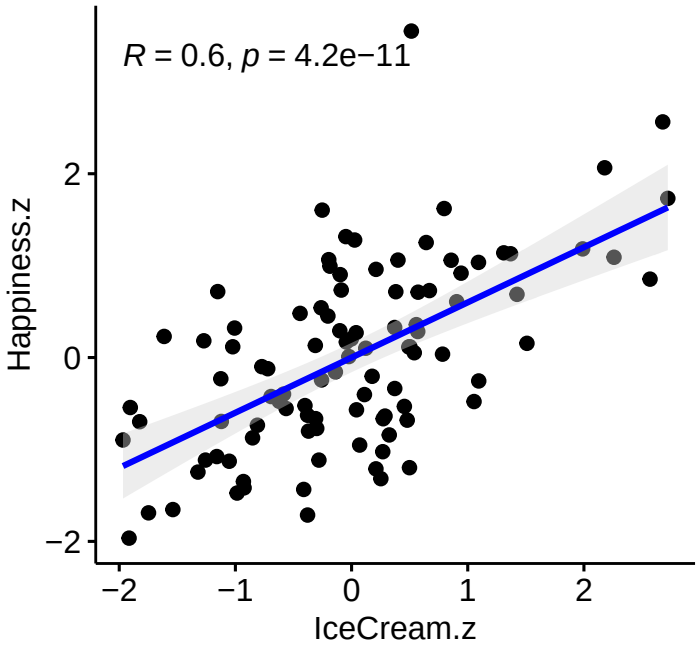


Figure 10.3: X axis: ice cream, z-scored. Y axis: happiness, z-scored. Both variables now have a mean of 0 and an SD of 1.

```
r(98) = .60, p < .001
```

10.5.12 Scale change: Linear (mixed scales)

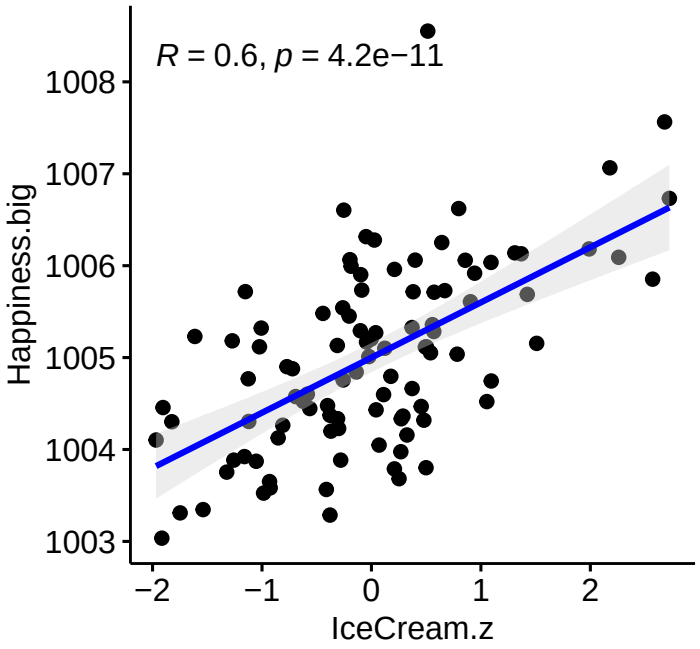


Figure 10.4: X axis: ice cream, z-scored. Y axis: happiness, plus 1000. Two different transformations at once, on two wildly different scales.

```
r(98) = .60, p < .001
```

10.5.13 Scale change: Non-linear (breaks it)

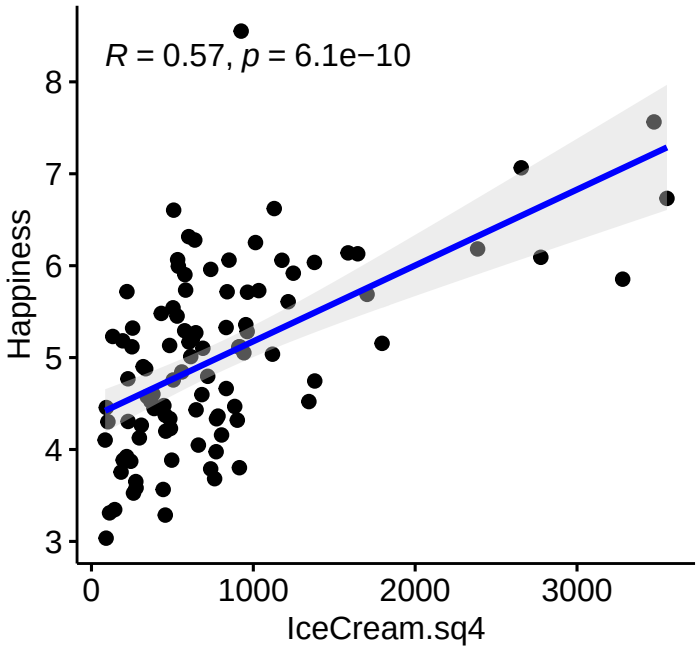


Figure 10.5: X axis: ice cream, raised to the fourth power. Y axis: happiness, untouched. This is the one that breaks, because the fourth power is not a straight line.

```
r(98) = .57, p < .001
```

10.5.14 When You Only See Part of the Picture: Restriction of Range

Rescaling a variable leaves r alone. Throwing away part of the sample does not.

Imagine we ran this study badly. Instead of surveying everyone, we set up a folding table outside the ice cream shop and surveyed only the people already standing in it. Everyone in the sample now eats a fair amount of ice cream, and the range of that variable has quietly collapsed.

```
# Keep only the top third of ice cream eaters, and throw the rest away
Cutoff   <- quantile(CorrData$IceCream, .66)
ShopOnly <- subset(CorrData, IceCream > Cutoff)

sd(CorrData$IceCream)  # spread across everyone
```

```
[1] 1
```

```
sd(ShopOnly$IceCream)  # spread among shop customers only
```

```
[1] 0.7187161
```

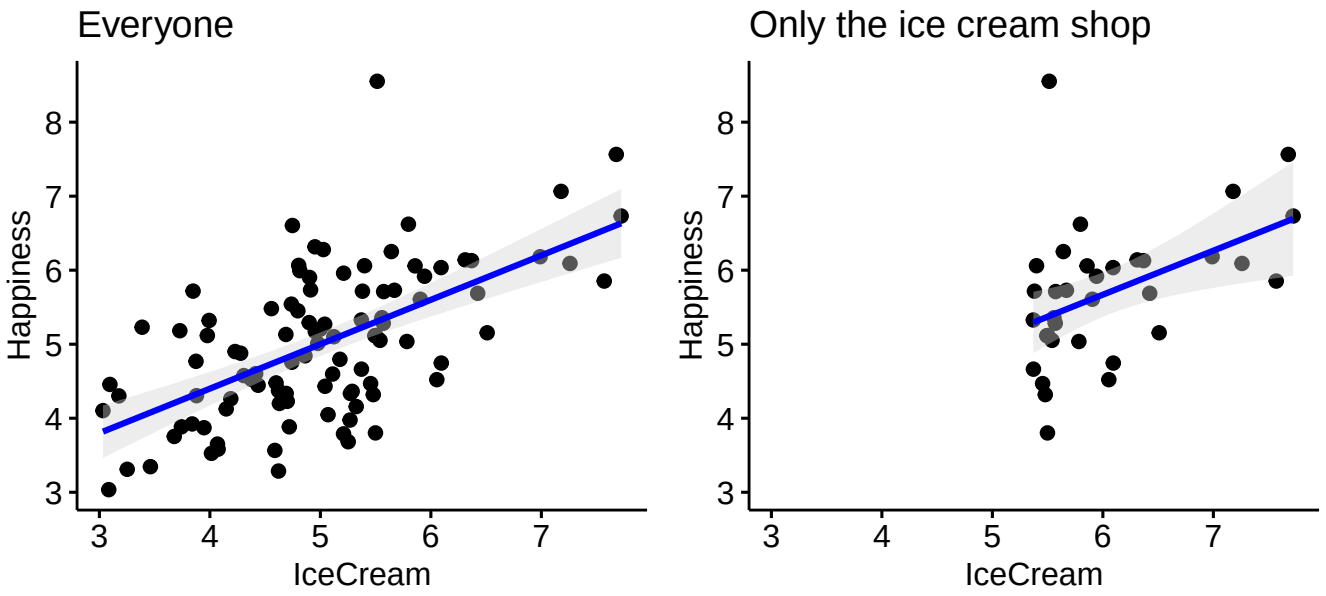


Figure 10.6: The same relationship, before and after we kept only the heaviest ice cream eaters. Both panels are drawn on the same axes.

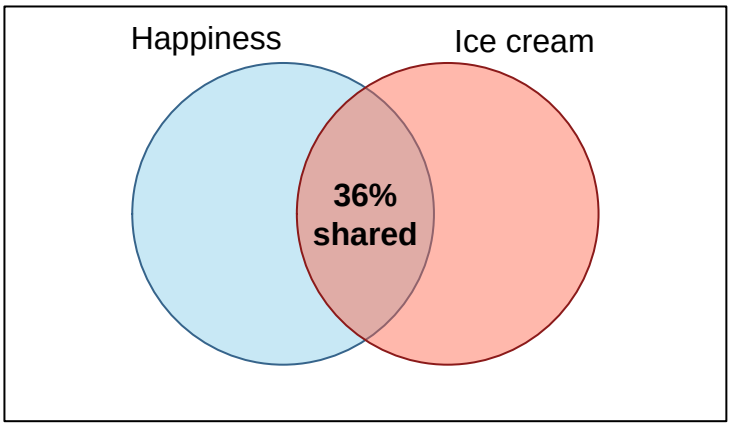
Across everyone, $r = .60$. Among shop customers only, with 34 people left, $r = .45$. Not one thing about the underlying relationship changed. We changed who we were willing to look at.

Selective samples flatten correlations, because a correlation needs variability to detect and we just deleted most of it. This is why correlations among admitted students, hired employees, and participants who survived to the end of a study so often underwhelm: somebody already removed the range you needed before you got the data.

Groups that do something stranger than flatten, and reverse a trend outright, are Simpson's paradox. That one waits for you in the mixed regression chapters.

10.5.15 Pearson's: Let's visualize our result

The overlap between the two variables is defined by r^2.



The picture is conceptual rather than drawn to exact area scale. Its job is to keep the shared piece visible, not to audition for an engineering degree.

This means that 36% of the variance of happiness overlaps with ice cream consumption. Can we conclude ice cream causes happiness? No, because we did not design a study with a control group. We can not infer causation. It seems to make sense to say "eating ice cream causes me to be happy", but the opposite could be true as well "happiness causes me to eat ice cream". How do I know which causes which? We cannot without an experiment.

There is a third possibility, and it is worse than the first two, because it does not require either variable to touch the other. Hot weather sells ice cream. Hot weather also pushes people outdoors into daylight and company, which reliably lifts mood. Heat moves both variables, and heat never once appears in our data.

Which is the pirate graph again, wearing a nicer outfit. The -.96 we opened with was never a claim that pirates cool the planet. It was two things drifting across the same two centuries for unrelated reasons, and the arithmetic could not tell. Your 36% is that same kind of number until a design rules the alternatives out.

10.5.16 Non-Parametric Correlations

Spearman and Kendall correlations are useful for ordinal data and for **monotonic** relationships: as one variable increases, the other generally keeps moving in one direction, even if the pattern bends. They do not rescue every nonlinear relationship. A U-shaped relationship can have a correlation near zero because the upward and downward pieces cancel each other while laughing at you.

10.5.17 Spearman's Correlation

Spearman is a Pearson Correlation on rank-ordered data. Which raises a fair question: what is a rank?

Ranking replaces each score with its position in the sorted order. The smallest value becomes 1, the next becomes 2, and so on up to N. The original numbers are thrown away and only their order survives.

```
Scores <- c(2, 5, 9, 40, 41)
rank(Scores)
```

```
[1] 1 2 3 4 5
```

Look at what happened to that 40 and 41. In the raw scores they sit far away from 2, 5, and 9, and almost on top of each other. As ranks they are 4 and 5, exactly one apart, the same distance as 1 and 2. Ranking flattens every gap to the same size, which is precisely why it shrugs off outliers and bent relationships.

Ties get the average of the positions they would have taken up:

```
Tied <- c(2, 5, 5, 9)
rank(Tied)
```

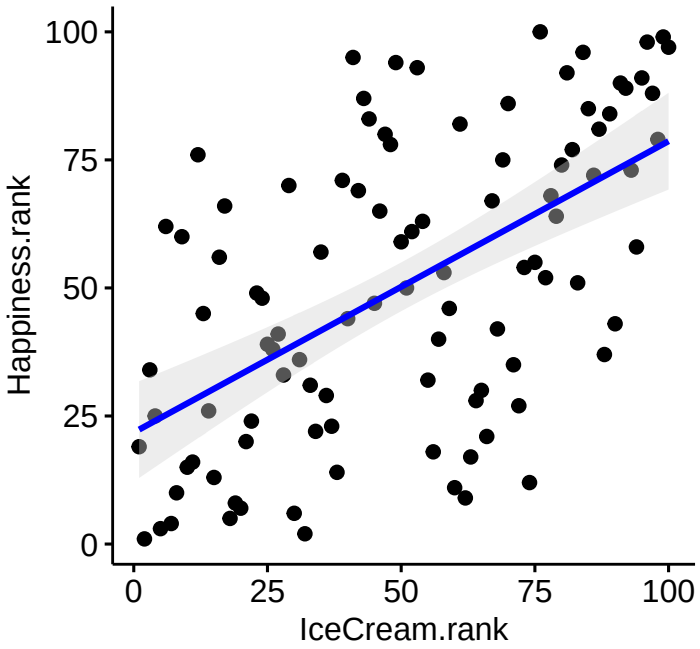
```
[1] 1.0 2.5 2.5 4.0
```

The two fives would have occupied positions 2 and 3, so they each get 2.5. This is also the tie-handling that costs us an exact p-value later.

Now let's rank order our random correlated data. You must rank each variable independently first (where ties are averaged).

```
CorrData$Happiness.rank<-rank(CorrData$Happiness)
CorrData$IceCream.rank<-rank(CorrData$IceCream)

ggscatter(CorrData, x = "IceCream.rank", y = "Happiness.rank",
   add = "reg.line",  # Add regression line
   add.params = list(color = "blue", fill = "lightgray"), # Customize reg. line
   conf.int = TRUE, # Add confidence interval
   )
```



```
cor_apa(cor.test(CorrData$IceCream.rank, CorrData$Happiness.rank, method = c("pearson")))
```

```
r(98) = .57, p < .001
```

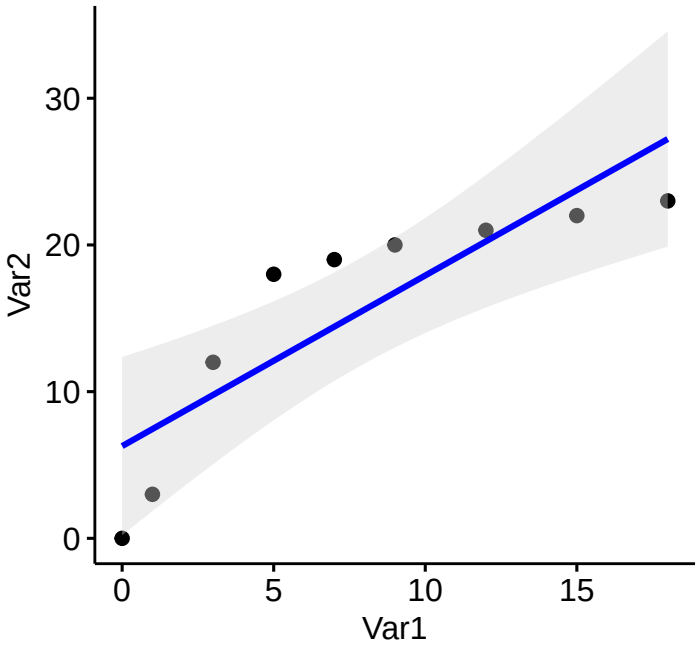
Notice that the output above labels this r and not r_s. The function is not confused. We literally ran a Pearson correlation, on ranks, which is what Spearman's correlation *is*.

Use the built-in Spearman correlation with `cor.test(..., method = "spearman")`. It ranks the raw data automatically and calculates the appropriate p-value.

```
r_s = .57, p < .001
```

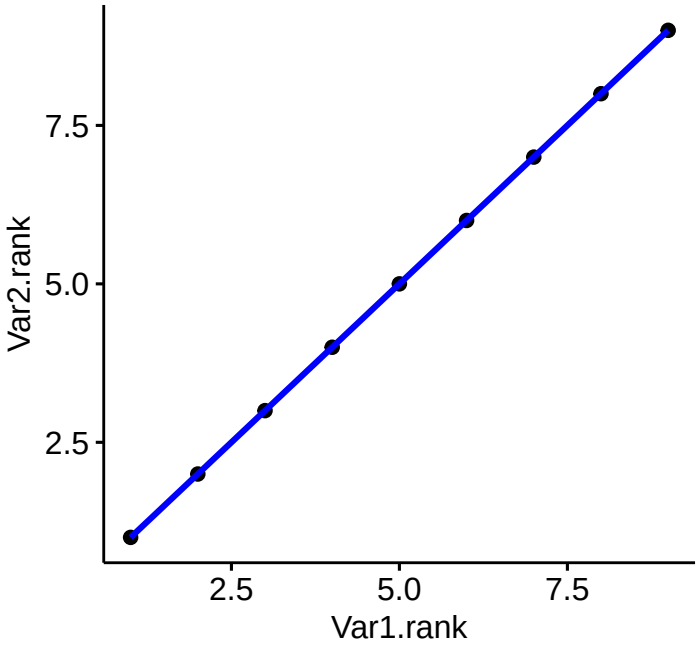
10.5.18 Pearson vs Spearman for Nonlinear Data

Let's say you get some data and clearly there is a slight nonlinearity in the relationship between the two variables. Pearson is designed for linear relationships and we can see the problem in our fitted line below.




```
r(7) = .86, p = .003
```

If we switch to a Spearman correlation the data are converted to ranks and the "bump" is now gone and our correlation gets stronger.




```
r_s = 1.00, p < .001
```

One thing to expect the first time you try this on your own data. Asking for `exact = TRUE` works here because none of these values are tied, but with tied ranks R cannot compute an exact p-value, so it warns you and quietly falls back on the t-approximation. That is fine, not a mistake. Likert-scale data ties constantly, so if you run a Spearman on questionnaire items you should count on seeing that warning every time.

10.5.19 Correlation Matrices

When we have multiple variables we can compare them all to each other at once. First we will simulate a 4 variables and all their bivariate correlations using the mvrnorm function again.

```
#Set params
Means.XY<- c(5,5,5,5) #set the means of X and Y variables
r12=.6;r13=.1;r14=.5;r23=.1;r24=.8;r34=0; #Correlation values
CovMatrix.XY <- matrix(c(1,r12,r13,r14,
                         r12,1,r23,r24,
                         r13,r23,1,r34,
                         r14,r24,r34,1),4,4) # creates the covariate matrix

# Build the correlated variables using mvrnorm.
# Note: empirical=TRUE means make the correlation EXACTLY r.
# empirical=FALSE, the correlation value would be normally distributed around r
library(MASS) #create data
CorrData2<-mvrnorm(n=100, mu=Means.XY,Sigma=CovMatrix.XY, empirical=TRUE)
#Convert them to a "Data.Frame", which is like SPSS data window
CorrData2<-as.data.frame(CorrData2)
#lets add our labels to the vectors we created
colnames(CorrData2) <- c("Happiness","IceCream", "Sprinkles","Oreos")
```

We can use the `GGally` package to plot Pearson correlations quickly in an easy to visualize format once the data are in a data frame.

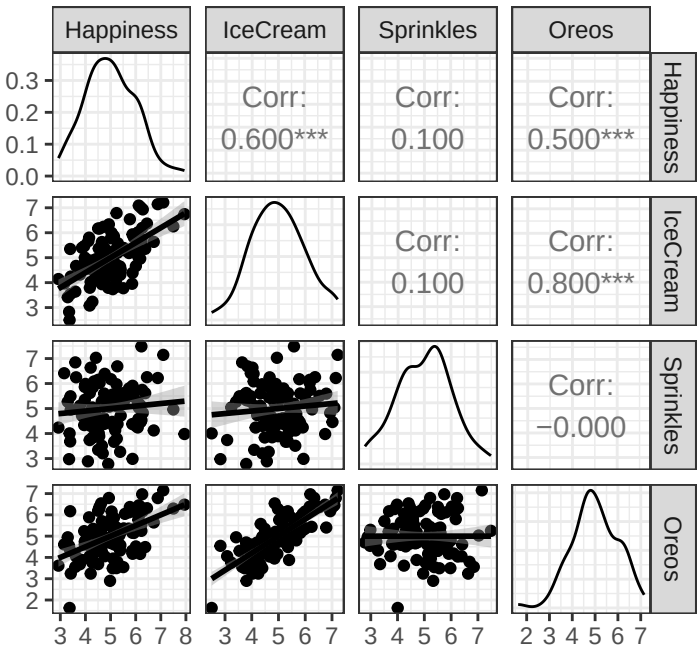


Figure 10.7: Every pair of four variables at once. Diagonal: each variable's own distribution. Below it: the scatterplot. Above it: the same pair as a number.

That is a lot of panels at once, so here is how to read it.

Find a variable name along the diagonal. That cell is the variable by itself: a smoothed histogram of its own distribution, the one-variable picture from way back at the start of this chapter. Nothing is being compared there.

Everything off the diagonal is a **pair**. To find a pair, pick a row and a column and see where they meet. Below the diagonal you get the scatterplot for that pair, with a fitted line through it. Above the diagonal you get the exact same pair reported as a number, the Pearson r.

The matrix is **symmetrical**. The panel for Happiness and Oreos below the diagonal and the number for Happiness and Oreos above it are the same relationship shown two ways, which is also why the diagonal is the fold: correlating a variable with itself would give 1.00 every time and tell you nothing.

Those asterisks next to each correlation are the significance stars from the one-sample t-test chapter, doing the same job here:

Stars	p-value
***	less than .001
**	less than .01
*	less than .05
.	less than .10
nothing	above .10

Now go back and count how few of those cells are anywhere near zero. This is where Meehl's crud factor stops being a quotation and becomes a picture. In real data these cells are rarely empty, and a matrix like this one is where most fishing expeditions begin: run enough pairs and something will eventually look publishable.

Here is what that code asks R to do:

- `ggpairs()` builds a matrix containing every pair of variables.
- `CorrData2` is the data frame whose variables enter the matrix.
- `lower = list(continuous = "smooth")` places scatterplots and fitted smooth lines in the lower triangle.
- `theme_bw()` removes some visual clutter so the relationships are easier to inspect.

10.6 The Short Story

- Covariance tells us whether two variables move together, but its size depends on their units.
- Correlation standardizes that shared movement.
- Pearson's r summarizes a linear relationship; Spearman's r_s summarizes a monotonic relationship using ranks.
- r^2 is the proportion of variance shared in a simple two-variable linear setting.
- Correlation does not establish causation. Ice cream may produce happiness, happiness may produce ice-cream consumption, or heat may produce both while quietly stealing credit.
- Plot the data. A single coefficient can hide curves, outliers, separate groups, and other tiny statistical monsters.

Correlation Is Not a Tiny Causal Arrow

An r describes the direction and strength of a relationship. It does not tell us whether X causes Y, Y causes X, or a third variable is backstage operating both puppets. Design and theory must do that work.

Credit, and Also Blame

I remember Ryne Estabrook rewriting my version of this lecture, and then I rewrote his rewrites, so give me the credit for the good parts and blame him for whatever confused you.

11 Linear Regression

11.1 The Number Correlation Refused to Give You

The last chapter ended on a complaint. Correlation tells you that two things move together and how tightly they do it, and then refuses to say anything more. It will not tell you *why*, which is a problem for theory. It also will not tell you *how much*, which is a problem for almost everything else.

You can hear the gap the moment you ask a practical question. If a student studies one more hour a week, how many exam points does that buy? An r of .45 cannot answer that, and not because it is being difficult. It threw the units away on purpose, back when we divided by both standard deviations to force the answer to sit politely between -1 and $+1$.

Regression puts the units back. It answers in points per hour, dollars per year, and scoops of ice cream per unit of happiness. Same relationship, same arithmetic underneath, but now the number means something you could act on.

That is the upgrade, and it is why the next eight chapters are built on this one.

11.2 Introduction to Regression

Many psychological studies ask: how does X *predict* Y? How does ice cream (IV) predict happiness (DV)?

Unlike the t-test (which compares group averages), regression looks at the relationship between **two continuous variables**. *Predicting* just means making an educated guess—if I know how much ice cream you ate, I can guess how happy you are. The guess won't be perfect, but it should be reasonably close.

Regression tells us **how close those guesses tend to be**.

Regression predicts. It does not automatically explain causes. If people who eat more ice cream report greater happiness, ice cream may help, happier people may seek ice cream, hot weather may increase both, or the entire pattern may be driven by one billionaire buying all the cookie dough. Design and theory decide which causal stories remain plausible.

Note: The *outcome* in ordinary regression should be interval or ratio scale. In practice, psychologists use Likert scales all the time anyway and mostly get away with it, and there is more on that fight later. Predictors are far less picky. Over the next few chapters we will feed regression categorical variables on purpose, and it will thank us for it.

11.2.1 The Experiment

I bought several tubs of vanilla ice cream, scooped out different amounts, and handed them to students. After eating, they rated their happiness on a 0–10 "feelings thermometer." Nobody got zero scoops, but some got barely a teaspoon.

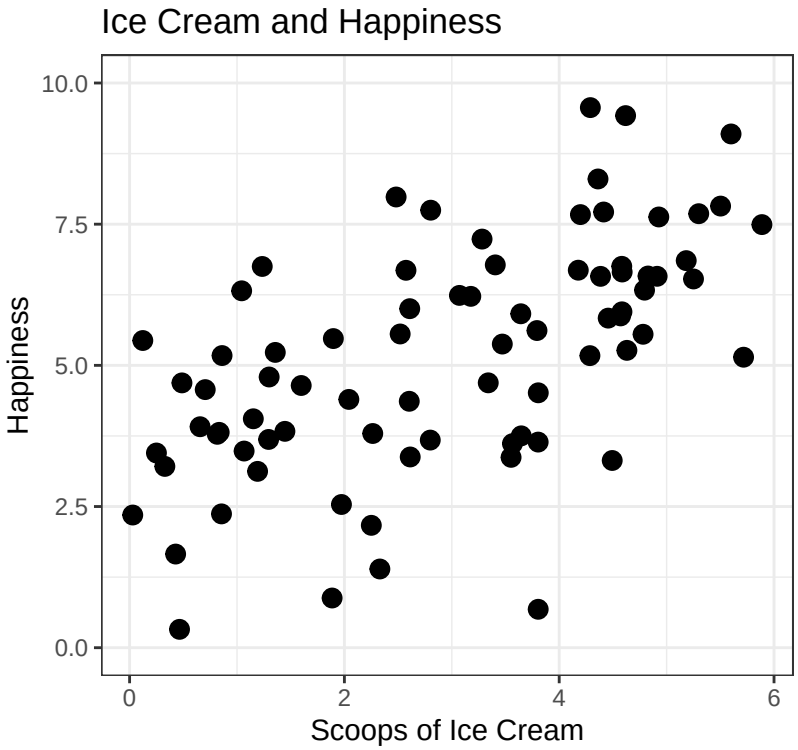


Figure 11.1: The raw data: 80 students, their scoops, and how happy they said they were afterward.

11.2.2 Why not just use correlation?

You can, and correlation is often the first step. But correlation only handles **two variables at a time**. Regression lets us add more predictors: ice cream *plus* hot fudge *plus* cookie dough. In other words, regression scales up in ways correlation can't.

Here's the correlation:

```
library(apa)
apa(cor.test(dat$Scoops, dat$Happiness))
```

```
[1] "*r*(78) = .60, *p* < .001"
```

Strong positive correlation, as expected!

11.3 Regression Theory

Correlation and regression describe the **same linear relationship**, just differently.

- **Simple linear regression** = 1 DV and 1 IV. The relationship is a straight line representing the **expected mean of Y** for each value of X.
- **Multiple regression** = 1 DV and 2+ IVs. (Coming later.)

11.3.1 The Regression Equation

You probably remember from middle school: $y = Mx + b$

The modern version comes in two lines, and the difference between them matters more than almost anything else in this chapter.

$$Y = B_0 + B_1 X + e$$

That is what the world does: model, plus miss.

$$\widehat{Y} = B_0 + B_1 X$$

That is what the model predicts. No e, because the model cannot know what it does not know.

Where:

- Y = observed value, the score the person actually gave you
- $\widehat{Y}$ = predicted value, what the line says they should have given you
- B_1 = slope (change in prediction per one-unit increase in X)
- B_0 = intercept
- e = residual, the gap between the two ($Y - \widehat{Y}$)

The hat is the whole distinction. Y is a fact about a person. $\widehat{Y}$ is a guess about a person. Mixing them up is the single most common way to get lost in regression, and it will keep mattering all the way into the mixed models at the end of Part 2.

11.4 The Ice Cream Model

We fit regression models with `lm()` (linear model). Formula structure: **DV ~ IV**.

```
Happy.Model.1 <- lm(Happiness ~ Scoops, data = dat)
summary(Happy.Model.1)
```

```
Call:
lm(formula = Happiness ~ Scoops, data = dat)

Residuals:
    Min      1Q  Median      3Q     Max
-5.1621 -0.9242  0.0332  1.0608  3.3565

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   2.9799     0.3863   7.715 3.35e-11 ***
Scoops        0.7524     0.1127   6.675 3.25e-09 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 1.681 on 78 degrees of freedom
Multiple R-squared:  0.3636,    Adjusted R-squared:  0.3554
F-statistic: 44.56 on 1 and 78 DF,  p-value: 3.246e-09
```

11.4.1 Reading the Output Without Panic

That block is the most-stared-at object in your regression course. You will meet it in every remaining chapter, it will grow extra rows, and at 2 a.m. before an assignment it will look like a ransom note. So here is what each region is, top to bottom.

Call. Your own formula, echoed back. Worth an actual glance, because this is where you find out you regressed the wrong variable.

Residuals. A five-number summary of the leftovers, the e from the equation above. You are looking for a median near zero and roughly symmetric ends. Ours run from -5.16 to 3.36, which means the model's worst miss was about 5.2 happiness points. Nobody is being lied to catastrophically.

Coefficients table. Four columns, one row per term:

- **Estimate** is B. The intercept row is B_0, the `Scoops` row is B_1.
- **Std. Error** is the standard error *of that estimate*. Same species as the SEM from the standard error chapter: run the study again with 80 different people and the slope would bounce around by roughly this much.
- **t value** is Estimate divided by Std. Error. That is the one-sample t-test logic, wearing a different kimono: how many standard errors is this estimate away from zero?
- **Pr(>|t|)** is that t's p-value, testing the null that this coefficient is really zero (no slope).

Residual standard error. The typical size of a miss, in the outcome's own units. Ours is 1.68, so a typical prediction is off by about 1.7 happiness points. The `on 78 degrees of freedom` is the $N-2$ we paid for in the correlation chapter, and for the same reason: two parameters estimated, this time an intercept and a slope.

Multiple R-squared. The proportion of outcome variance the model accounts for. There is a whole section on it below.

F-statistic. The test of the entire model at once, rather than one coefficient at a time. With a single predictor it is telling you exactly what the slope's t-test already told you, and we can prove that:

```
t.for.slope <- summary(Happy.Model.1)$coefficients["Scoops", "t value"]

t.for.slope^2                          # square the slope's t
```

```
[1] 44.55917
```

```
summary(Happy.Model.1)$fstatistic[1]   # the model's F
```

```
   value
44.55917
```

Identical. With one predictor, $F = t^2$, because "is this one slope non-zero" and "does this model do anything at all" are the same question asked twice. That stops being true the moment we add a second predictor, which is precisely when F starts earning its keep, in the hierarchical regression chapter.

The Two Numbers Everyone Skips

Students read the Estimate and the p-value and stop. The two most useful numbers in that block are the ones they scroll past: the **Std. Error**, which tells you how much to trust the estimate, and the **Residual standard error**, which tells you in plain units how wrong a typical prediction is.

A slope of 0.75 with typical misses of 1.7 happiness points is a very different finding from the same slope with typical misses of 0.2. The p-value cannot tell those apart. You can.

11.4.2 Intercept

The intercept is the predicted happiness **when ice cream = 0**.

```
range(dat$Scoops)
```

```
[1] 0.02927577 5.88743768
```

Nobody had zero scoops, so we can't actually interpret this directly. **Never predict outside the range of your data.** (We can fix this by centering X, more on that later.)

11.4.3 Slope

The slope is 0.752. For each additional scoop, happiness changes by that amount.

11.4.4 Making a prediction

The equation: $Y = B_0 + B_1 X$

What's the predicted happiness for someone who eats **5 scoops**?

- $Y = 2.98 + (0.752 \times 5)$
- $Y = 6.742$

11.5 Predicted Values and Residuals

The line gives one predicted happiness score for every amount of ice cream. Real people do not sit perfectly on that line because humans are noisy, stubborn, and occasionally having a terrible day for reasons unrelated to dessert.

A **residual** is the distance between the observed score and the model's prediction:

$$e_i = Y_i - \widehat{Y}_i.$$

- A positive residual means the person was happier than the model predicted. - A negative residual means the person was less happy than the model predicted. - A residual of zero means the prediction was exactly right, which is suspiciously cooperative behavior from a human.

R stores both pieces inside the fitted model:

```r
dat$Predicted <- fitted(Happy.Model.1)
dat$Residual <- residuals(Happy.Model.1)

head(dat[c("Scoops", "Happiness", "Predicted", "Residual")])
```

```
     Scoops Happiness Predicted   Residual
1 0.4874424  4.689709  3.346669  1.3430399
2 2.7996025  3.676425  5.086421 -1.4099958
3 4.1977287  7.669921  6.138422  1.5314991
4 4.7945917  6.331497  6.587523 -0.2560254
5 3.3393869  4.690570  5.492574 -0.8020041
6 5.2990277  7.685625  6.967079  0.7185465
```

```r
mean(dat$Residual)
```

```
[1] -1.061651e-16
```

The residuals have a mean of approximately zero because ordinary least squares (i.e., math that finds the best line) tries to balance the line through the data such that the positive and negative mistakes cancel out.

```r
ggplot(dat, aes(x = Scoops, y = Happiness)) +
  geom_smooth(method = "lm", se = FALSE, color = "gray45") +
  geom_segment(
    aes(xend = Scoops, yend = Predicted, color = Residual),
    linewidth = 0.7
  ) +
  geom_point(aes(color = Residual), size = 2.5) +
  geom_point(aes(y = Predicted), shape = 1, size = 2.5) +
  scale_color_gradient2(low = "#0072B2", mid = "white", high = "#D55E00") +
  labs(
    title = "Residuals Are the Model's Vertical Mistakes",
    x = "Scoops of Ice Cream",
    y = "Happiness"
  ) +
  theme_bw()
```

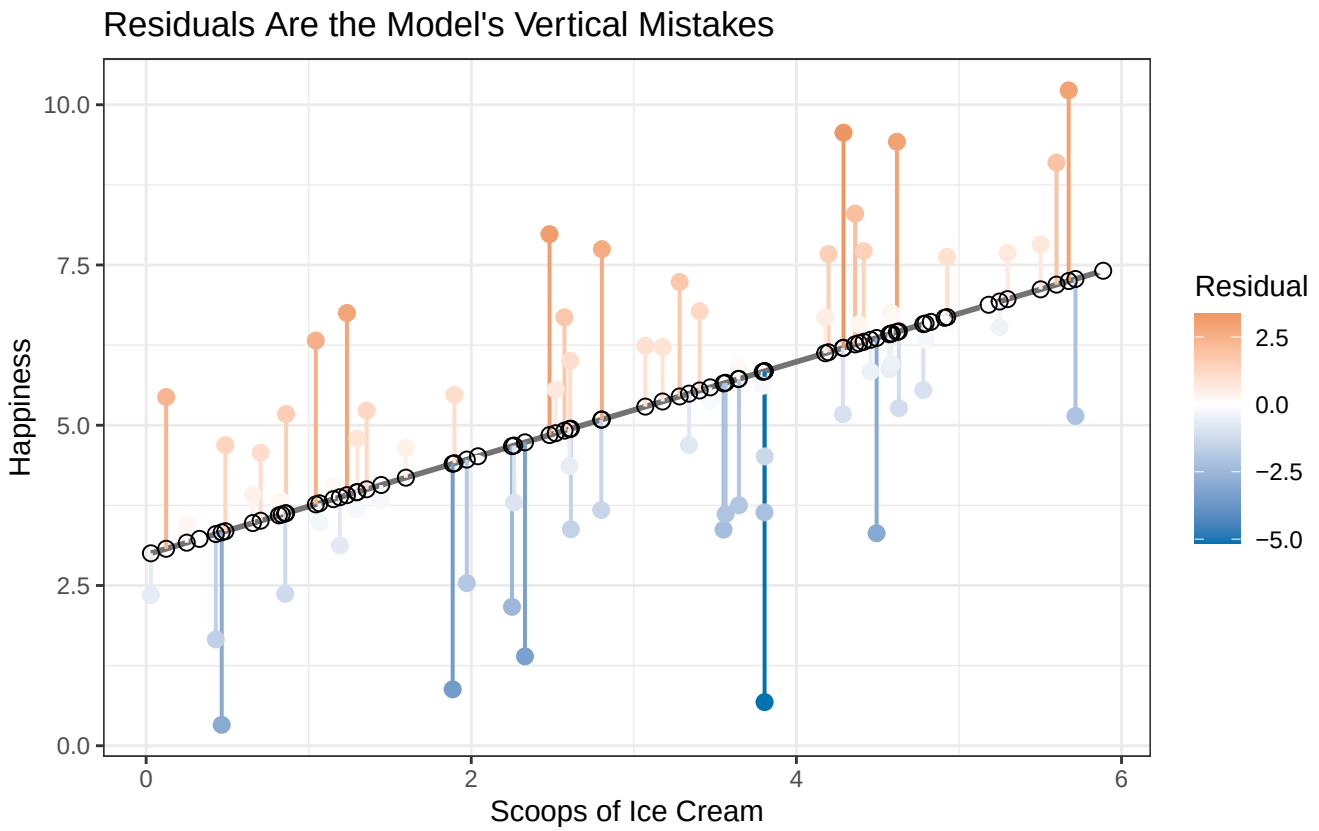


Figure 11.2: Every vertical segment is one residual: the gap between what a student reported and what the line predicted for them.

The filled points are observed happiness. The hollow points are predicted happiness. Each vertical segment is one residual. This is the error the model did not explain.

11.6 Why This Line and Not Some Other Line?

Nothing so far has explained where the line came from. R drew one, we believed it. But an infinite number of straight lines could be drawn through that cloud of points, so something had to choose.

Here is the rule. Take any line you like. Measure every vertical miss, square each one, and add them all up. That total is the line's score, and lower is better. The regression line is the line that makes that total as small as it can possibly be. That is the entire idea, and it is what **least squares** means.

We can check that R did its job. The fitted line's squared misses:

```
sum(residuals(Happy.Model.1)^2)
```

```
[1] 220.3025
```

Now some rival lines. The first is the flattest possible one, a slope of zero, which ignores ice cream entirely and predicts the mean happiness for everybody. The other two keep our intercept but guess the slope wrong:

```r
b0 <- coef(Happy.Model.1)[1]

# Slope of 0: the mean-only model, ice cream tells us nothing
sum((dat$Happiness - mean(dat$Happiness))^2)
```

```
[1] 346.155
```

```r
# Too steep
sum((dat$Happiness - (b0 + 1.5*dat$Scoops))^2)
```

```
[1] 745.2635
```

```r
# Too flat
sum((dat$Happiness - (b0 + 0.3*dat$Scoops))^2)
```

```
[1] 412.5871
```

The fitted line wins, and it is not close. Every wrong slope costs you, and the further off you guess, the more it costs.

Why squared, and not just the raw misses? Because raw residuals sum to zero by construction, as we saw a moment ago. A criterion that adds up to zero for every possible line is a criterion that cannot choose between lines. Squaring kills the signs, which is exactly the trick variance pulled back in the distributions chapter, for exactly the same reason. It also means one enormous miss hurts far more than several small ones, which is a genuine design decision on the part of those who created regression.

Hold onto that mean-only number, 346.2. It is the score of a model with no predictor in it: your best guess before ice cream entered the room. The fitted line scores 220.3. The gap between those two numbers is what ice cream bought you, and turning that gap into a proportion is the entire content of the R^2 section coming up.

11.7 The Assumptions: What Could Make the Line Lie?

Regression assumptions are claims about the errors left after fitting the line. They are not personality tests that the raw variables must pass before R permits them into the building.

For now, focus on five questions:

1. **Linearity:** Is a straight line a reasonable summary of the average relationship?
2. **Independence:** Are the residuals from different observations reasonably independent?
3. **Constant variance:** Is the vertical spread of residuals roughly similar across fitted values?
4. **Approximately normal residuals:** Are the residuals reasonably bell-shaped, especially for small-sample tests and intervals?
5. **Influential observations:** Is one strange person steering the entire line while everyone else watches?

11.7.1 Residuals Versus Fitted Values

```r
plot(
  fitted(Happy.Model.1),
  residuals(Happy.Model.1),
  xlab = "Fitted happiness",
  ylab = "Residual",
  pch = 19,
  col = grDevices::adjustcolor("#1F4E79", alpha.f = 0.55)
)
abline(h = 0, lty = 2, col = "#D55E00", lwd = 2)
```

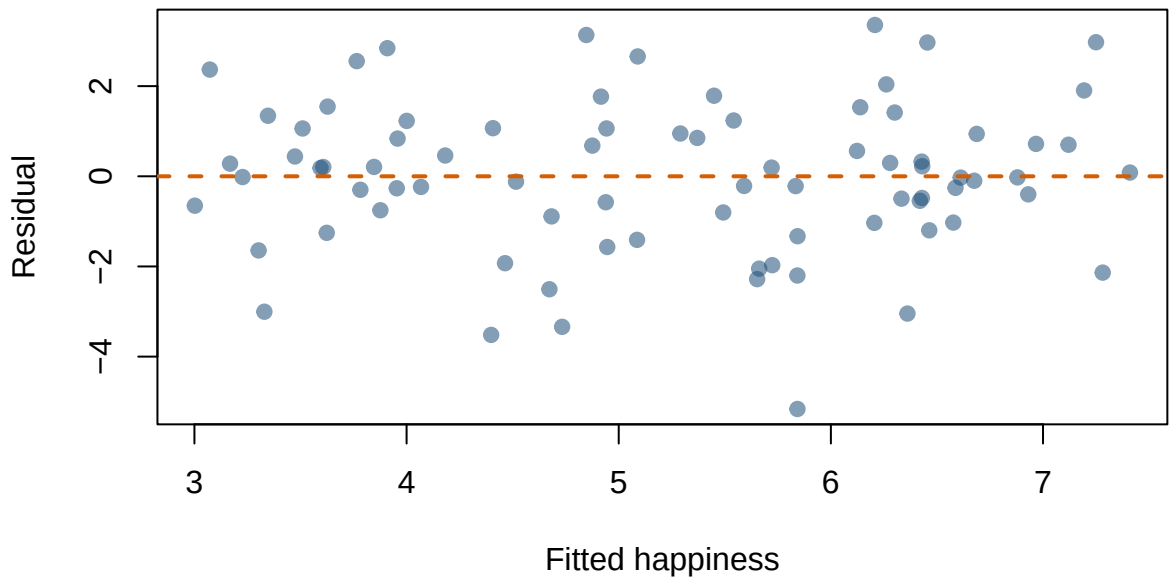


Figure 11.3: Residuals against fitted values. We want a boring cloud around the dashed line, and this one is reasonably boring.

The **residuals-versus-fitted plot** asks whether the model is making similar kinds of errors across the range of values it predicts. Each dot represents one observation.

- **X-axis, Fitted values:** what the regression model **predicted** for each observation.
- **Y-axis, Residuals:** how wrong that prediction was: **observed - predicted**. A residual of 0 means the model predicted the observation perfectly. Positive residuals mean the observed value was higher than predicted; negative residuals mean it was lower than predicted.

Ideally, we want a fairly boring cloud of points scattered around zero, with roughly the same vertical spread from left to right. **Boring is good here.** It means the model is not making errors in an obvious, systematic way.

A **curve** suggests that the straight-line model missed a nonlinear pattern. For example, if residuals tend to be positive, then negative, then positive again as fitted values increase, the relationship may have a bend that our straight regression line cannot capture.

A **funnel** suggests **heterogeneity of variance**, also called **heteroscedasticity**: the model's errors are much more variable for some predicted values than for others. *Note: This is a word I have been teaching since about 2008 and still cannot reliably say out loud.*

Why care? Ordinary regression calculates standard errors under the assumption that the residual variance is reasonably constant. Strong heteroscedasticity can therefore make standard errors and confidence intervals inaccurate.

But changing variance can also reveal something psychologically interesting. Perhaps ice cream affects people similarly at low doses but produces wildly different reactions after the fourth scoop. Some people achieve joy. Others remember lactose intolerance.

> ⚠ Uneven Residuals Can Be a Clue
>
> Changing residual spread can mean that uncertainty differs across people, conditions, or levels of the outcome. Do not immediately transform the outcome until the graph stops complaining. First ask what process could create the pattern.

11.7.2 The Q-Q Plot

```
qqnorm(residuals(Happy.Model.1), pch = 19, col = "#1F4E79")
qqline(residuals(Happy.Model.1), col = "#D55E00", lwd = 2)
```

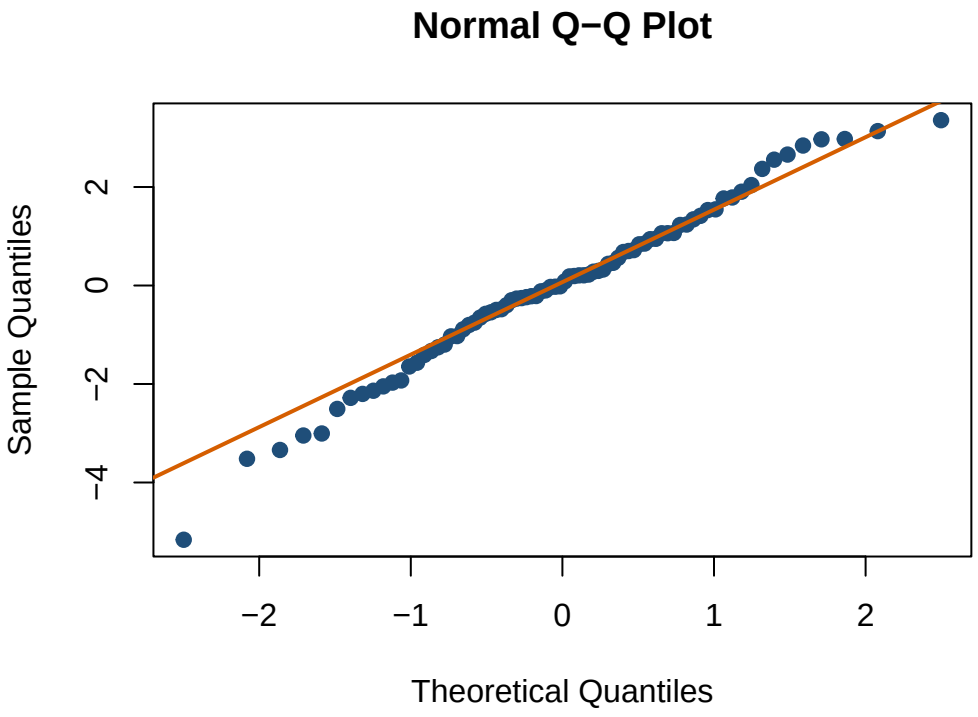


Figure 11.4: A Q-Q plot of the residuals. Points near the line mean the residuals are behaving roughly normally.

A **Q-Q plot** asks whether the residuals have roughly the shape we would expect from a normal distribution. Each dot represents one residual, after the residuals have been sorted from smallest to largest.

- **X-axis:** where a residual *should fall* if the residuals followed a perfect normal distribution. These are called **theoretical quantiles**.
- **Y-axis:** where the residual *actually falls* in our data. These are the **observed residuals** (or, depending on the software, standardized residuals).

If the residuals are approximately normal, the observed values should line up with the values expected from a normal distribution, producing dots that fall roughly along a straight line. In other words, the x-axis says, "If these residuals were perfectly normal, how extreme should this observation be?" and the y-axis says, "How extreme was it actually?"

Do not expect perfection. Small wiggles and deviations are normal, because we collected data from humans, not from androids or clones. What we care about are **systematic departures from the line**: strong curves can indicate a distribution with the wrong shape, while points breaking away at either end can indicate unusually heavy tails or potential outliers. One observation escaping toward Canada probably means something went wrong for that one person. All of them running for the border means something went wrong with the whole model.

R can produce four standard diagnostic plots at once:

```
par(mfrow = c(2, 2))
plot(Happy.Model.1)
par(mfrow = c(1, 1))
```

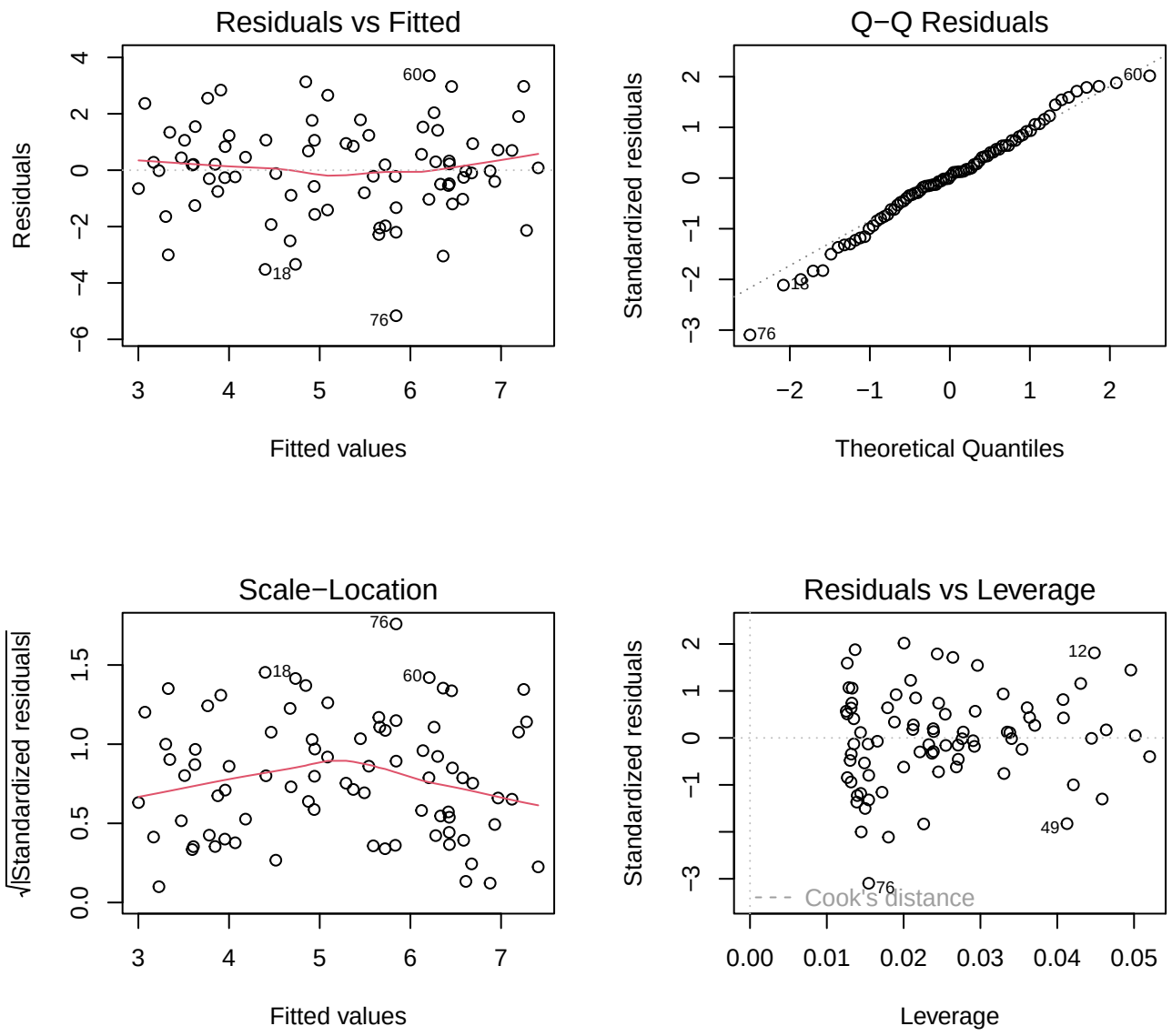


Figure 11.5: The four diagnostic plots R gives you for free. The top two are the ones we just built by hand.

The first two are the residual-versus-fitted and Q-Q plots we just learned. The other two help identify unequal spread and influential observations. You do not need to master them yet. The advanced diagnostics chapter returns with more detail and several additional ways for data to misbehave.

11.8 Residual Variance and R^2

Before using ice cream to predict happiness, our best simple guess for everyone was the mean. The total variance describes how far scores spread around that mean. After fitting the model, residual variance describes how far scores spread around the regression line.

Those are the two lines we just raced. The mean-only model was the flat one that lost; the fitted line was the one that won. R^2 is simply that race expressed as a proportion: of all the spread we started with, what fraction did the line

manage to account for?

```
TotalVariance <- var(dat$Happiness)
ResidualVariance <- var(residuals(Happy.Model.1))

c(TotalVariance = TotalVariance,
  ResidualVariance = ResidualVariance,
  R_squared_from_variance = 1 - ResidualVariance / TotalVariance,
  R_squared_from_model = summary(Happy.Model.1)$r.squared)
```

```
         TotalVariance        ResidualVariance R_squared_from_variance
             4.3817090               2.7886391               0.3635727
  R_squared_from_model
             0.3635727
```

R^2 describes the proportion of observed outcome variance explained by the fitted model. It is an effect-size measure for the model, not proof that the model is causal, correct, or ready to be tattooed onto your arm.

11.9 Relationship to Correlation

Why did we start with correlation if regression is so great? To show they're the same thing, just scaled differently. Run a regression on **standardized** (z-scored) variables:

```
dat$Happiness.z <- scale(dat$Happiness)
dat$Scoops.z    <- scale(dat$Scoops)

std.reg <- lm(Happiness.z ~ Scoops.z, data = dat)
summary(std.reg)
```

```
Call:
lm(formula = Happiness.z ~ Scoops.z, data = dat)

Residuals:
     Min       1Q   Median       3Q      Max
-2.46608 -0.44152  0.01588  0.50675  1.60349

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -7.487e-17  8.976e-02   0.000        1
Scoops.z     6.030e-01  9.033e-02   6.675 3.25e-09 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.8029 on 78 degrees of freedom
Multiple R-squared:  0.3636,    Adjusted R-squared:  0.3554
F-statistic: 44.56 on 1 and 78 DF,  p-value: 3.246e-09
```

11.9.1 What didn't change?

The **t-statistic and *p*-value** for the slope. These don't change under any *linear* rescaling: adding constants, multiplying by constants, z-scoring. Shift and stretch the axes all you like and the significance test is untouched.

Nonlinear transformations are a different animal. Logs, square roots, and powers change the model itself, along with t, p, and what the slope even means. The correlation chapter showed this the hard way with ice cream raised to the fourth power, where a perfectly good relationship fell apart the moment we bent the scale.

11.9.2 What changed?

The intercept is now **zero**. When all variables are standardized (mean = 0), the regression line must pass through [0, 0]. And the slope changed numerically:

```
print(summary(std.reg), digits = 6)
```

```
Call:
lm(formula = Happiness.z ~ Scoops.z, data = dat)

Residuals:
      Min        1Q    Median        3Q       Max
-2.466080 -0.441520  0.015876  0.506753  1.603487

Coefficients:
                Estimate   Std. Error t value   Pr(>|t|)
(Intercept) -7.48683e-17  8.97626e-02 0.00000          1
Scoops.z     6.02970e-01  9.03290e-02 6.67527 3.2461e-09 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.802862 on 78 degrees of freedom
Multiple R-squared:  0.363573,  Adjusted R-squared:  0.355413
F-statistic: 44.5592 on 1 and 78 DF,  p-value: 3.24612e-09
```

```
cor.test(dat$Happiness, dat$Scoops)
```

```
    Pearson's product-moment correlation

data:  dat$Happiness and dat$Scoops
t = 6.6753, df = 78, p-value = 3.246e-09
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.4417814 0.7264454
sample estimates:
      cor
0.6029699
```

The standardized slope equals the correlation. Not a coincidence.

When everything is standardized, one unit = one SD. So the correlation tells us: every time X increases by 1 SD, Y increases by r SDs. Standardized regression slopes are called **beta weights (β)**.

11.9.3 Converting slope back to raw units

$$B_1 = r\frac{sd_y}{sd_x} = \frac{Cov(x,y)}{Var(x)}$$

Your slope is just the correlation times the ratio of SDs. Makes sense, since slope is rise over run:

- Double Y → twice as much rise → B_1 doubles
- Double X → twice as much run → B_1 is cut in half

11.9.4 What about the intercept?

Once you know B_1, you know the angle of the line. But the line must also pass through **[mean(X), mean(Y)]**:

```
mean(dat$Scoops)
```

```
[1] 2.993878
```

```
mean(dat$Happiness)
```

```
[1] 5.232601
```

$$Mean(Y) = B_0 + B_1 \times Mean(X)$$

Solve for B_0 and you're done.

11.10 APA Style Report

Here is how the whole thing gets written up. Every number below is computed from the model rather than typed, which is why it cannot go stale when the data change.

> Ice cream consumption significantly predicted happiness, $b = 0.75$, $SE = 0.11$, $\beta = .60$, $t(78) = 6.68$, $p < .001$. The overall model explained 36.4% of the variance in happiness, $R^2 = .364$, $F(1, 78) = 44.56$, $p < .001$.

Then say what it means in words a human would use:

> Each additional scoop predicted about 0.75 more points of happiness on the 0 to 10 thermometer.

A few things to notice, because this paragraph is the template for every regression write-up in the rest of the book:

- Report b **and** SE **and** t **and** p for the predictor. The slope alone is not a result, it is half of one.
- Report the model as a whole with R^2 and F, including both degrees of freedom: $F(1, 78)$. The 1 is the number of predictors, the 78 is $N - 2$.

- That β of .60 should look familiar. It is the correlation from the top of this chapter, which is the payoff we proved a section ago. In simple regression you can report either (or both), but never present them as two separate findings.
- The plain-English sentence is not optional decoration. Being able to say what your slope means in the units your grandmother would understand is a graded skill, in this course and in every paper you will ever write.

> **Do Not Copy p < .001**
>
> The write-up above says $p < .001$ because this model's p-value genuinely is 3.25e-09. Report the exact value to two or three decimals whenever it is larger than .001, and never copy `p < .001` out of an example because it looked authoritative.

11.11 The Short Story

- Simple linear regression predicts a quantitative outcome from one predictor.
- The intercept is the predicted outcome when $X = 0$.
- The slope is the expected change in Y for a one-unit increase in X.
- The line is chosen by **least squares**: of all possible lines, it is the one with the smallest total of squared vertical misses.
- In the `summary()` output, Std. Error says how much to trust the estimate and Residual standard error says how wrong a typical prediction is. With one predictor, $F = t^2$.
- Residuals are observed outcomes minus predicted outcomes: the model's leftovers.
- R^2 is the proportion of observed outcome variance explained by the fitted linear model.
- With one predictor, the standardized slope equals Pearson's r.
- Prediction is not causation. A straight line cannot randomize participants, control confounds, or stop Reviewer 2 from asking whether heat explains the ice cream.

> **A Slope Always Has Units**
>
> The slope is the expected change in Y for a one-unit increase in X. Before deciding that it is large, small, heroic, or pathetic, say what one unit of X and one unit of Y actually mean.

> **Credit**
>
> I stole Ryne Estabrook way of explaining least squares cause it was better than mine. Also probably other things.

12 Linear Regression with Categorical Variables

12.1 The Promise From Last Chapter

The last chapter said the outcome has to be quantitative but the predictors are far less picky, and that we would start feeding regression categorical variables on purpose.

This is that chapter.

The question is genuinely strange the first time you meet it. Regression multiplies a predictor by a slope, and you cannot multiply "Texting" by 0.34. So how does a model built entirely out of arithmetic handle a variable whose values are words?

The answer is that we translate the words into numbers, in a way specific enough that the slopes still mean something afterward. Do that and something nice falls out: the t-test you learned in Part 1 turns out to have been a regression the whole time, wearing a disguise.

12.2 Distracted Driving

Driving requires constant attention. When something unexpected happens, such as a car braking suddenly, you must react quickly. Psychologists studying attention have shown that distractions slow reaction time. One of the most dangerous distractions is texting while driving.

The data in this chapter are simulated. Do not reproduce the study by handing undergraduates phones, placing them in actual traffic, and chasing the car with a bat to improve ecological validity seen in city style road rage. Use a simulator and an ethics-approved protocol. The Institutional Review Board enjoys fewer surprises than the Probability Gods.

In this example we examine **braking reaction time** (seconds) under different driving conditions.

Reaction time is measured in seconds. Larger values indicate slower reactions (which is bad).

12.2.1 Dummy Coding

When a predictor is categorical, regression cannot use the labels directly.
Regression requires numbers, so the categories must be translated into numeric variables.

This process is called **dummy coding**. *Who are you calling a dummy? The coding scheme not the coder.* It is the most common coding scheme, and the **default used by R** whenever it detects a factor.

12.2.2 Dummy Coding Details

Dummy coding compares each group to a **reference group**.

The reference group serves as the **baseline** or **zero point** for the model.
All other groups are interpreted relative to that baseline.

In other words, the model asks: *How much does each group deviate from the reference group?*

12.3 Two-Level Example

Suppose we conduct a study comparing two driving conditions: **No distraction** vs **Texting while driving**

We collect data for: $n = 30$ drivers per group (*dataframe = dat2*)

We have two driving conditions (and predict):

Condition	Reaction Time Prediction
No distraction	faster
Texting	slower

To represent this in regression, we create a dummy variable.

Variable	C1
No Distraction	0
Texting	1

Here:

- **0 represents the reference group**
- **1 represents the comparison group**

So the dummy variable indicates whether a driver is texting. Note: We usually put the 1 on our experimental group.

12.3.1 How This Connects to Regression

The regression model becomes

$$RT = b_0 + b_1 C_1$$

Interpretation:

- b_0 = the mean reaction time for the **reference group**
- b_1 = how much the **Texting** group differs from that reference group

12.3.2 Predicted Values

For **No Distraction**:

$C_1 = 0$

$$RT = b_0 + b_1 0$$

Which simplifies to

$$RT = b_0$$

- the **intercept (b_0) equals the mean reaction time for No Distraction**

For **Texting**:

$C_1 = 1$

$$RT = b_0 + b_1 1$$

Which simplifies to

$$RT = b_0 + b_1$$

- the **slope (b_1) coefficient represents the difference between Texting and No Distraction**

So the regression coefficients directly reproduce the group means.

12.3.3 R Makes the Dummies for You

In R, you usually **do not need to create dummy variables yourself**.

When a predictor is stored as a **factor**, R automatically creates the dummy-coded variables internally when fitting the model.

The **first level of the factor becomes the reference group**.

For example:

```
dat2$Condition <- factor(dat2$Condition,
                         levels = c("No Distraction", "Texting"))
```

Because `"No Distraction"` appears first, it becomes the **reference group**.

When we get our regression output from R you see it as:

- the **intercept equals the mean reaction time for No Distraction**
- the **ConditionTexting coefficient represents the difference between Texting and No Distraction**

12.3.4 Comparing the Groups with a *t*-test

We first test the hypothesis that texting increases reaction time.

First we load the packages needed for this example.

- **tidyverse**: used for data manipulation and summarizing the data
- **apa**: used to produce APA-style statistical output

```
library(apa)
library(tidyverse)
```

12.3.5 Calculate Group Means and Standard Deviations

Before running a statistical test, it is always good practice to examine the descriptive statistics.

The code below calculates the **mean (M)** and **standard deviation (SD)** of reaction time for each condition.

```
dat2 |>
  group_by(Condition) |>
  summarise(
    M = mean(RT),
    SD = sd(RT)
  )
```

```
# A tibble: 2 x 3
  Condition          M    SD
  <fct>          <dbl> <dbl>
1 No Distraction 0.664 0.137
2 Texting        1.01  0.121
```

This gives us a quick summary of how reaction time differs across the two driving conditions.

12.3.6 Run the Independent Samples *t*-test

Next we test whether the difference between groups is statistically significant.

```
texting.ttest <- t.test(RT ~ Condition,
                        data = dat2,
                        var.equal = TRUE)
```

Reminder:

- `RT ~ Condition` tells R we are predicting **reaction time** from the **driving condition**
- `data = dat2` specifies which dataset to use
- `var.equal = TRUE` assumes equal variances across groups (the standard Student *t*-test)

12.3.7 Print the Results in APA Format

Finally, we format the results in **APA style** using the `apa` package.

```
apa(texting.ttest, format="rmarkdown")
#Note: format can also be "text", "markdown", "rmarkdown", "html", "latex" or "docx"
#I have to use rmarkdown cause of how I created this chapter. You might use use "text"
```

```
[1] "*t*(58) = -10.21, *p* < .001, *d* = -2.64"
```

12.3.8 APA write up

Drivers who texted while driving had significantly slower reaction times (M = 1.01, SD = 0.12) than drivers with no distraction (M = 0.66, SD = 0.14), $t(58)$ = -10.21, $p < .001$, d = -2.64, indicating that texting substantially increased reaction time.

Small note: the negative sign on t simply reflects the order of subtraction (No Distraction minus Texting).

> **A d of 2.6 Should Be Making You Suspicious**
>
> By now you have read the power chapter, so that effect size should raise an eyebrow. Real texting-while-driving effects are large, which is exactly why it is illegal, but they are not science-fiction large. Published estimates sit closer to d of 0.5 to 1.
>
> I built this simulation to make the pictures come out clean. Simulated data lies politely, and it will happily calibrate your intuitions to a world that does not exist.

12.3.9 The Same Analysis Using Regression

Now we run the equivalent regression model.

```
model2 <- lm(RT ~ Condition, data = dat2)
summary(model2)
```

```
Call:
lm(formula = RT ~ Condition, data = dat2)

Residuals:
     Min       1Q   Median       3Q      Max
-0.31427 -0.09282 -0.00903  0.08735  0.34813

Coefficients:
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)       0.66388    0.02362   28.11  < 2e-16 ***
ConditionTexting  0.34115    0.03340   10.21 1.41e-14 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.1294 on 58 degrees of freedom
```

```
Multiple R-squared:  0.6427,    Adjusted R-squared:  0.6365
F-statistic: 104.3 on 1 and 58 DF,  p-value: 1.412e-14
```

12.3.10 Why Are the *t*-values the Same?

Stop and compare two numbers before going any further, because this is the whole point of the section.

In the `ConditionTexting` row above: $t = 10.21$, $p < .001$.

Back in the *t*-test: $t =$ -10.21, $p < .001$.

Same magnitude, opposite sign. Not similar. *Identical.*

The sign flip is bookkeeping: `t.test()` subtracted in the order No Distraction minus Texting, and the regression reports Texting relative to No Distraction. Everything else matches because the two procedures are doing the same arithmetic:

- The regression slope **is** the difference between the group means.
- Its standard error **is** the pooled standard error of that difference.
- Dividing one by the other is precisely what the *t*-test did.

That equivalence comes with one condition. It holds because we ran the *t*-test with `var.equal = TRUE`. Ordinary regression assumes one common residual variance across groups, which is the same assumption the pooled Student *t*-test makes. Ask for Welch's version instead, which does not assume that, and the match breaks. This is the other side of the Welch discussion from the independent-samples chapter.

The regression equation is

$$RT = b_0 + b_1 X$$

Interpretation:

- b_0 = mean reaction time for **No Distraction**
- b_1 = difference between **Texting** and **No Distraction**

Because the dummy variable for No Distraction is 0:

For No Distraction

$$RT = b_0$$

For Texting

$$RT = b_0 + b_1$$

So the intercept is simply the mean reaction time for the reference group.

```
coef(model2)
```

```
     (Intercept) ConditionTexting
       0.6638792        0.3411475
```

Which gives us:

Coefficient	Value
Intercept	0.664
ConditionTexting	0.341

Now we can reconstruct the means.

12.3.11 No Distraction

$$RT = b_0$$

$$RT = 0.664$$

So the predicted mean reaction time is **0.664 seconds**.

12.3.12 Texting

$$RT = b_0 + b_1$$

$$RT = 0.664 + 0.341$$

$$RT = 1.005$$

So the predicted mean reaction time is **1.005 seconds** for texting while driving.

These values match the observed group means from the data.

Regression coefficients are simply another way of expressing the group means and their differences.

12.3.13 Extracting the Means with `emmeans`

We can also extract the model-estimated means so we do not have to reconstruct them manually from the regression equation.

```
library(emmeans)

emmeans(model2, ~ Condition)
```

```
 Condition      emmean     SE df lower.CL upper.CL
 No Distraction  0.664 0.0236 58    0.617    0.711
 Texting         1.005 0.0236 58    0.958    1.052

Confidence level used: 0.95
```

12.3.14 What the Code Does

`library(emmeans)` loads the **emmeans** package. This package is designed to extract and compare means from statistical models such as regression.

`emmeans(model2, ~ Condition)` tells R:

- use the regression model **model2**
- estimate the means for each level of **Condition**

The `~ Condition` part means:
compute the means for the variable Condition.

12.3.15 What "Estimated Marginal Means" Means

The name **emmeans** stands for **Estimated Marginal Means**.

Estimated marginal means are the **means predicted by the statistical model**.

Why *marginal*? Before computers did all this instantly, people wrote the means in the margins of tables using actual pencils. They were Number 2 pencils, naturally—the same pencils you never remember to bring to my exams. iPads are new. Most of the old people who write R packages or statistics books were born in the 1900s and are older than the internet.

In this simple example, they correspond directly to the group means:

Condition	Predicted Mean
No Distraction	b_0
Texting	$b_0 + b_1$

So `emmeans()` is simply doing the same math we just performed manually:

- extracting the intercept
- adding the appropriate regression coefficients
- reporting the predicted group means

12.3.16 Why This Is Useful

For simple models like this one, calculating the means by hand is straightforward.

However, as models become more complex (for example, with multiple predictors or interactions), manually reconstructing the means becomes tedious.

`emmeans` provides a convenient way to:

- extract the model-based means
- compare groups
- obtain confidence intervals
- perform additional tests

All while keeping the results tied directly to the regression model.

12.3.17 Testing the Difference with `emmeans`

We can also test the difference between groups directly from the regression model using `emmeans`.

```
pairs(emmeans(model2, ~ Condition))
```

```
 contrast                 estimate     SE df t.ratio p.value
 No Distraction - Texting   -0.341 0.0334 58 -10.213 <0.0001
```

12.3.18 What the Code Does

The function `emmeans(model2, ~ Condition)` first computes the **estimated marginal means** for each level of the factor **Condition** based on the regression model.

The function `pairs()` then compares those means to one another.

In this case, there are only two conditions, so the function tests the difference between:

- **Texting**
- **No Distraction**

12.3.19 How This Connects to the Regression Model

Recall that the regression equation is

$$RT = b_0 + b_1 X$$

where

- b_0 = mean reaction time for **No Distraction**
- b_1 = difference between **Texting** and **No Distraction**

The comparison performed by `pairs()` is therefore testing whether:

$$b_1 = 0$$

If b_1 is significantly different from zero, the two groups have different mean reaction times.

12.3.20 Comparison to *t*-test

Even though the output may look similar to a t-test, the comparison is now being computed **within the regression framework**.

The means and the group comparison are both derived directly from the fitted regression model.

12.3.21 APA Regression Write up

Drivers who texted while driving had significantly slower reaction times than drivers with no distraction. Relative to this reference group (no distractions), the Texting condition increased reaction time by 0.34 seconds, $b = 0.34$, $SE = 0.03$, $t(58) = 10.21$, $p < .001$. The model explained a substantial proportion of variance in reaction time, $R^2 = .64$, $F(1, 58) = 104.31$, $p < .001$, indicating that differences in driving condition accounted for about 64% of the variability in reaction times across drivers.

12.3.22 Plotting the Model-Estimated Means

We can also visualize the model-estimated means returned by `emmeans`.

First we store the estimated means in an object.

```
means <- emmeans(model2, ~ Condition)

means
```

```
 Condition      emmean     SE df lower.CL upper.CL
 No Distraction  0.664 0.0236 58    0.617    0.711
 Texting         1.005 0.0236 58    0.958    1.052

Confidence level used: 0.95
```

To make plotting easier, we convert the results into a data frame.

```
means_df <- as.data.frame(means)

means_df
```

```
 Condition         emmean       SE df  lower.CL  upper.CL
 No Distraction 0.6638792 0.023619 58 0.6166006 0.7111577
 Texting        1.0050267 0.023619 58 0.9577482 1.0523053

Confidence level used: 0.95
```

The data frame contains:

- `emmean` = the estimated mean
- `lower.CL` = lower confidence interval
- `upper.CL` = upper confidence interval

Now we can quickly plot these values using **ggplot**.

```
library(ggplot2)

ggplot(means_df, aes(x = Condition, y = emmean)) +
  geom_col() +
  geom_errorbar(aes(ymin = lower.CL, ymax = upper.CL), width = .1) +
  ylab("Reaction Time (seconds)") +
  xlab("Driving Condition")+theme_bw()
```

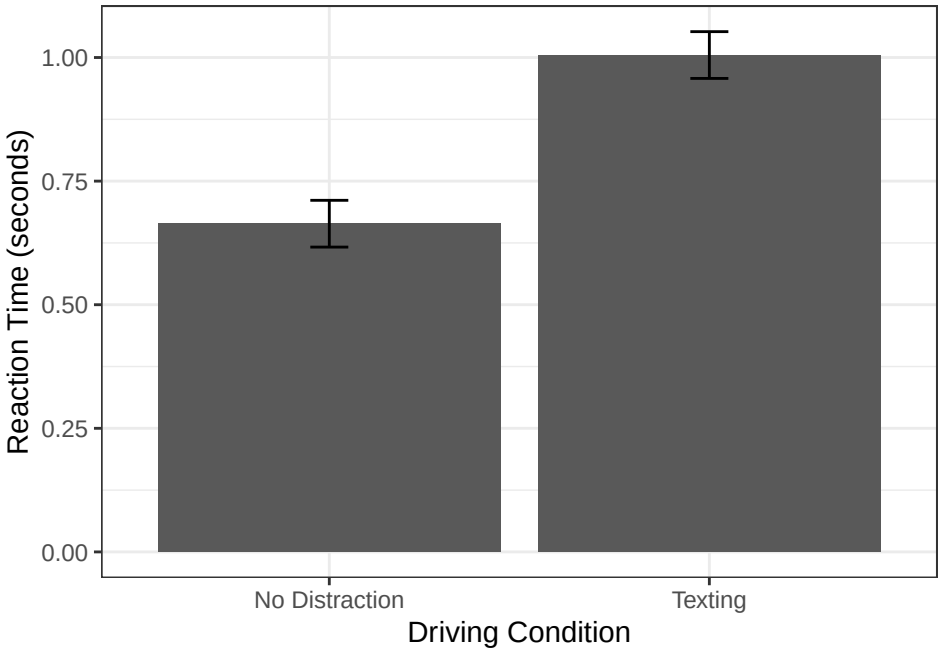


Figure 12.1: Model-estimated mean reaction time in each condition, with 95% confidence intervals. The bars come straight out of the regression.

This plot shows the **model-estimated means and their 95% confidence intervals**.

Because the means come from `emmeans`, the figure directly represents the regression model we fit earlier.

12.4 Three-Level Example

Now suppose we repeat the study but add a third condition:

Listening to a statistics lecture while driving to class.

12.4.1 Factoring the Three-Level Predictor

Again we convert the predictor to a factor.

```
dat3$Condition <- factor(dat3$Condition,
                         levels = c("No Distraction",
                                    "Texting",
                                    "Lecture"))

levels(dat3$Condition)
```

```
[1] "No Distraction" "Texting"        "Lecture"
```

The first level becomes the reference group.

So again:

```
NoDistraction = reference (0)
```

R now creates **two dummy variables** internally.

Conceptually this looks like:

Condition	Texting	Lecture
No distraction	0	0
Texting	1	0
Lecture	0	1

12.4.2 Multiple *t*-tests Problem

If we used t-tests, we would need three comparisons:

1. No distraction vs texting
2. No distraction vs lecture
3. Lecture vs texting

Running multiple tests increases the probability of a **Type I error**.

If $\alpha = .05$, then the probability of at least one false positive $(1 - (1 - \alpha)^j)$ [j = # of t-tests]:

$$1 - (1 - .05)^3$$

```
1 - (1 - .05)^3
```

```
[1] 0.142625
```

This is about **14%**, much larger than the intended 5%.

Regression helps here. It is less magic than it sounds. Fitting one model does not delete the multiple-testing problem. What it does is organize every comparison under a single model and hand you a **gatekeeper**: one overall test of whether the group means differ at all, which you check *before* going looking for which pairs differ. That gate is the omnibus F, and we are about to meet it.

12.4.3 Regression with Three Groups

Regression allows us to test all comparisons within a single model.

```
model3 <- lm(RT ~ Condition, data = dat3)
summary(model3)
```

```
Call:
lm(formula = RT ~ Condition, data = dat3)

Residuals:
     Min       1Q   Median       3Q      Max
-0.31427 -0.09670 -0.00662  0.07988  0.34813

Coefficients:
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)       0.66388    0.02380  27.898  < 2e-16 ***
ConditionTexting  0.34115    0.03365  10.137  < 2e-16 ***
ConditionLecture  0.19229    0.03365   5.714 1.52e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.1303 on 87 degrees of freedom
Multiple R-squared:  0.5429,    Adjusted R-squared:  0.5323
F-statistic: 51.66 on 2 and 87 DF,  p-value: 1.63e-15
```

12.4.4 The Gate: the Omnibus F

Look at the very last line of that output, the one everybody scrolls past:

```
F-statistic: 51.66 on 2 and 87 DF
```

That is the gatekeeper. It tests one null hypothesis: **all three group means are equal**. Not "texting differs from no distraction," not any particular pair, just whether there is any difference anywhere in the set.

Ours is $F(2, 87) = 51.66$, $p < .001$, so the gate is open. Something in there differs, and we have earned the right to go find out what.

If that F is not significant you stop. The means are not distinguishable, and hunting through pairs anyway is precisely the 14% problem we just calculated, now with extra steps.

The model is

$$RT = b_0 + b_1(\text{Texting}) + b_2(\text{Lecture})$$

Interpretation:

- b_0 = mean reaction time for **No Distraction**
- b_1 = difference between **Texting** and **No Distraction**
- b_2 = difference between **Lecture** and **No Distraction**

12.4.5 Reconstructing the Group Means from the Regression Model

Recall that the regression model is

$$RT = b_0 + b_1(\text{Texting}) + b_2(\text{Lecture})$$

From the model output we obtained the following coefficients:

Coefficient	Value
b_0 (Intercept)	0.664
b_1 (Texting)	0.341
b_2 (Lecture)	0.192

Again, the intercept represents the **mean reaction time for the reference group**, which is **No Distraction**.

```
coef(model3)
```

```
     (Intercept) ConditionTexting ConditionLecture
       0.6638792        0.3411475        0.1922929
```

12.4.6 No Distraction

For the reference group, both dummy variables equal 0.

$$RT = b_0$$

Substituting the coefficient:

$$RT = 0.664$$

So the predicted mean reaction time for **No Distraction** is **0.664 seconds**.

12.4.7 Texting

For the texting group:

Texting = 1
Lecture = 0

$$RT = b_0 + b_1 1 + b_2 0$$

Substituting the coefficients:

$$RT = 0.664 + 0.341$$

$$RT = 1.005$$

So the predicted mean reaction time for **Texting** is **1.005 seconds**.

12.4.8 Lecture

For the lecture group:

Texting = 0
Lecture = 1

$$RT = b_0 + b_1 0 + b_2 1$$

Substituting the coefficients:

$$RT = 0.664 + 0.192$$

$$RT = 0.856$$

So the predicted mean reaction time for **Lecture** is **0.856 seconds**.

12.4.9 Prediction summarized

The regression coefficients allow us to **reconstruct the group means directly**.

Condition	Model Equation	Predicted Mean
No Distraction	b_0	0.664
Texting	$b_0 + b_1$	1.005
Lecture	$b_0 + b_2$	0.856

Regression is simply expressing the group means and their differences using an equation.

12.4.10 Extracting the Model-Estimated Means

Just like before, `emmeans` will do all of that reconstruction for us.

```
means3 <- emmeans(model3, ~ Condition)
means3
```

```
 Condition      emmean     SE df lower.CL upper.CL
 No Distraction  0.664 0.0238 87    0.617    0.711
 Texting         1.005 0.0238 87    0.958    1.052
 Lecture         0.856 0.0238 87    0.809    0.903

Confidence level used: 0.95
```

These values represent the **means predicted by the regression model**, and they match the numbers we just assembled by hand.

12.4.11 Changing the Reference Group

The reference group is a choice, not a fact about the world. `factor(levels = )` sets it when you build the variable. To switch it afterwards, use `relevel()`:

```
dat3$Condition2 <- relevel(dat3$Condition, ref = "Texting")

model3b <- lm(RT ~ Condition2, data = dat3)
coef(model3b)
```

```
            (Intercept) Condition2No Distraction        Condition2Lecture
              1.0050267                -0.3411475                -0.1488547
```

Every coefficient is now expressed relative to Texting. The intercept is Texting's mean, 1.005, and the other two rows are how much *slower or faster* the remaining conditions are compared to it. No Distraction now carries a negative coefficient, -0.341, because being undistracted is faster than texting.

Here is the part worth watching, though. Nothing about the model actually moved:

```
# Same variance explained, same omnibus test, same fitted means
c(R2.original = summary(model3)$r.squared,
  R2.releveled = summary(model3b)$r.squared)

c(F.original = summary(model3)$fstatistic[1],
  F.releveled = summary(model3b)$fstatistic[1])
```

```
 R2.original R2.releveled
   0.5428557    0.5428557
 F.original.value F.releveled.value
         51.65595          51.65595
```

Identical. Releveling changed which comparisons the coefficients describe. It did not change the fit, the group means, the residuals, or the drivers.

12.4.12 APA Write-Up

A regression model was used to predict reaction time from driving condition. The overall model was significant, $R^2 = .54$, $F(2, 87) = 51.66$, $p < .001$, indicating that driving condition accounted for about 54% of the variability in reaction times.

Because **No Distraction** was entered as the first factor level, it served as the **reference group**, and the intercept represented the mean reaction time for that condition ($M = 0.66$, $SE = 0.02$, 95% CI [0.62, 0.71]).

Relative to this reference group, texting significantly increased reaction time, $b = 0.34$, $SE = 0.03$, $t(87) = 10.14$, $p < .001$. Listening to a statistics lecture also increased reaction time relative to the no-distraction condition, $b = 0.19$, $SE = 0.03$, $t(87) = 5.71$, $p < .001$.

12.4.13 Testing Differences Between Groups

We can also test the differences between all pairs of conditions.

```
pairs(means3, adjust="none")
```

```
 contrast                 estimate     SE df t.ratio p.value
 No Distraction - Texting   -0.341 0.0337 87 -10.137 <0.0001
 No Distraction - Lecture   -0.192 0.0337 87  -5.714 <0.0001
 Texting - Lecture           0.149 0.0337 87   4.423 <0.0001
```

This compares the estimated means for:

- No Distraction vs Texting
- No Distraction vs Lecture
- Lecture vs Texting

All comparisons are derived directly from the regression model. This contains the comparison we did not test earlier (texting vs. lecture). You can add that to your APA report as well.

12.4.14 About That `adjust="none"`

You should be suspicious of that argument, because we spent a whole section calculating that three unadjusted tests inflate the error rate to 14%. So why are we allowed to run three unadjusted tests?

Because we checked the gate first. The omnibus F was significant, which is what earns the right to look at individual pairs without adjusting each one. This is **Fisher's protected LSD**, and the word doing the work is *protected*: the F is the protection, and it was bought before any pair was examined.

The protection has a hard expiry date. Memorize it. It is a favorite exam question:

> **!** The Protection Expires at Three Groups
>
> With exactly **three** groups, a significant omnibus F followed by unadjusted pairwise tests keeps the familywise error rate at α. This is a genuine mathematical result, not a convention.
>
> With **four or more** groups it stops working, and the error rate creeps back up. At that point you need a real correction (Tukey, Holm, Bonferroni), which is what the Multiple Comparisons chapter is for.
>
> So `adjust = "none"` here is a defensible choice with three groups and a significant F. Pasted into a four-group analysis, it is just the 14% problem again.

12.4.15 APA for follow up test

In addition, pairwise comparisons of the estimated marginal means showed that texting produced significantly slower reaction times than listening to a statistics lecture, $b = 0.15$, $SE = 0.03$, $t(87) = 4.42$, $p < .001$. Thus, although both distractions impaired reaction time relative to the no-distraction condition, texting was associated with the greatest slowing of reactions.

12.4.16 Plotting the Model-Estimated Means

We can also visualize the results using a simple plot.

First we convert the estimated means to a data frame.

```
means3_df <- as.data.frame(means3)
means3_df
```

```
 Condition         emmean         SE df  lower.CL  upper.CL
 No Distraction 0.6638792 0.02379701 87 0.6165800 0.7111783
 Texting        1.0050267 0.02379701 87 0.9577276 1.0523259
 Lecture        0.8561720 0.02379701 87 0.8088729 0.9034712

Confidence level used: 0.95
```

Now we plot the estimated means and their 95% confidence intervals.

```
library(ggplot2)

ggplot(means3_df, aes(x = Condition, y = emmean)) +
  geom_col() +
  geom_errorbar(aes(ymin = lower.CL, ymax = upper.CL), width = .1) +
  ylab("Reaction Time (seconds)") +
  xlab("Driving Condition")+theme_bw()
```

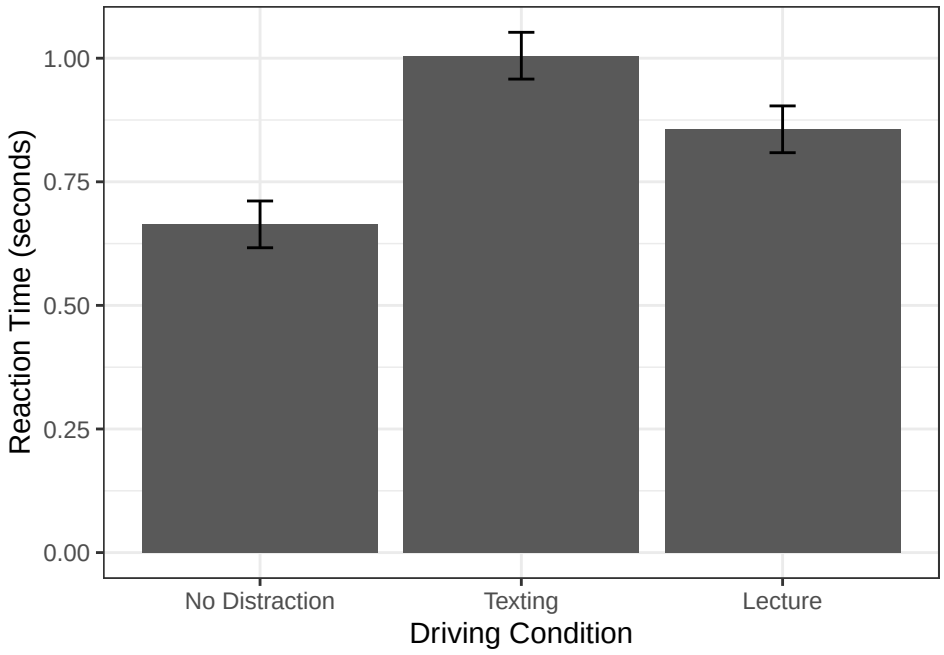


Figure 12.2: All three conditions, model-estimated means with 95% confidence intervals. Lecture sits between the other two.

This figure shows the **model-estimated mean reaction time for each driving condition** along with the 95% confidence intervals.

12.5 The Short Story

- R represents a categorical predictor with numeric contrast variables.
- Under ordinary treatment/dummy coding, the intercept is the reference-group mean.
- Each category coefficient is that group's difference from the reference group.
- With two groups, the regression test is equivalent to an independent-samples t-test under the matching variance assumptions.
- With three or more groups, the overall model asks whether the group means are all equal; follow-ups ask which contrasts differ.
- Changing the reference group changes the coefficient labels, not the fitted group means or the underlying data.
- Set factor levels deliberately. Otherwise R may select a reference group alphabetically, because the alphabet has apparently been granted authority over your scientific question.

Coding Changes the Coefficients, Not the People

Changing the reference group changes which comparison appears in each coefficient. It does not change the fitted group means, residuals, R^2, or the participants—who remain blissfully unaware that R renamed the argument.

13 Multiple Regression: Statistical Control

13.1 Why One Predictor Is Never Enough

People are complicated. When a student does well on an exam, is it because they studied? Because they weren't anxious? Because they found the material interesting? It's all of the above and more.

So far we've used **simple linear regression** (1 DV, 1 IV). But real prediction involves multiple factors at once. **Multiple regression** lets us predict an outcome from two or more predictors simultaneously.

The key concept: **statistical control**: estimating the association between X and Y while *holding other measured variables constant*. This lets us ask not just "does studying predict performance?" but "does studying predict performance **above and beyond** what anxiety alone explains?"

Without statistical control, you can't untangle those two things. With multiple regression, you can.

13.2 A Study on Exam Performance

Let's ground this in a specific example. Imagine you are a researcher interested in what predicts student exam performance. After reading the literature, you identify two strong candidates:

1. **Study Hours** (X1): how many hours per week a student spends studying for the exam. More studying should lead to better scores.
2. **Test Anxiety** (X2): how anxious a student feels about the exam (measured on a 0–100 scale, where 0 = no anxiety and 100 = extreme anxiety). More anxiety is expected to hurt performance.

You recruit 100 undergraduate students and collect these measures before a major exam. After the exam, you record their score (0–100).

13.2.1 The Complication: Our Two Predictors Are Correlated

Here is the twist. It turns out that Study Hours and Test Anxiety are **not independent of each other**. Students who study more tend to feel *less* anxious, perhaps because preparation builds confidence, or because anxious students have trouble sitting down to study in the first place.

This means:

- Students with *high* Study Hours tend to have *low* Test Anxiety.
- Students with *low* Study Hours tend to have *high* Test Anxiety.

In other words, Study Hours and Test Anxiety are **negatively correlated** with each other.

This creates a problem. If you only look at Study Hours → Exam Score, some of the apparent benefit of studying might actually be coming from lower anxiety, not from the studying itself. And if you only look at Anxiety → Exam Score, some of the apparent harm of anxiety might be coming from studying *less*, not from the anxiety per se.

Multiple regression lets us disentangle these overlapping influences.

13.2.2 Expected Correlations

Here are the correlations we expect to find in our data:

	Study Hours	Test Anxiety	Exam Score
Study Hours	—	−.35	+.45
Test Anxiety	−.35	—	−.40
Exam Score	+.45	−.40	—

- Study Hours *positively* predicts exam scores ($r = .45$): more studying, better scores.
- Test Anxiety *negatively* predicts exam scores ($r = -.40$): more anxiety, lower scores.
- Study Hours and Test Anxiety are *negatively correlated* ($r = -.35$): students who study more tend to be less anxious.

That third correlation is the key complication and the reason we need multiple regression.

13.2.3 Simulating the Data

Always look at your data before running any analysis. Let's check the first few rows and the summary statistics.

```
head(dat)
```

```
  Exam StudyHours Anxiety
1 43.1        5.3    61.3
2 72.7       10.5    40.4
3 88.8        6.3    45.9
4 73.2        7.4    24.3
5 66.8        6.7    64.5
6 65.7        8.7    57.9
```

```
summary(dat)
```

```
      Exam         StudyHours        Anxiety
 Min.   :43.10   Min.   : 1.500   Min.   :19.20
 1st Qu.:65.85   1st Qu.: 5.850   1st Qu.:44.45
 Median :74.40   Median : 7.700   Median :50.30
 Mean   :75.00   Mean   : 7.999   Mean   :50.00
 3rd Qu.:85.28   3rd Qu.:10.025   3rd Qu.:56.73
 Max.   :97.40   Max.   :17.500   Max.   :72.20
```

And here is a pairs plot showing all three pairwise relationships simultaneously. Each panel in the lower triangle shows a scatterplot, and the diagonal shows histograms for each variable.

```
library(GGally)
ggpairs(
  dat,
  lower = list(continuous = "smooth"),
  diag  = list(continuous = "barDiag")
) + theme_bw()
```

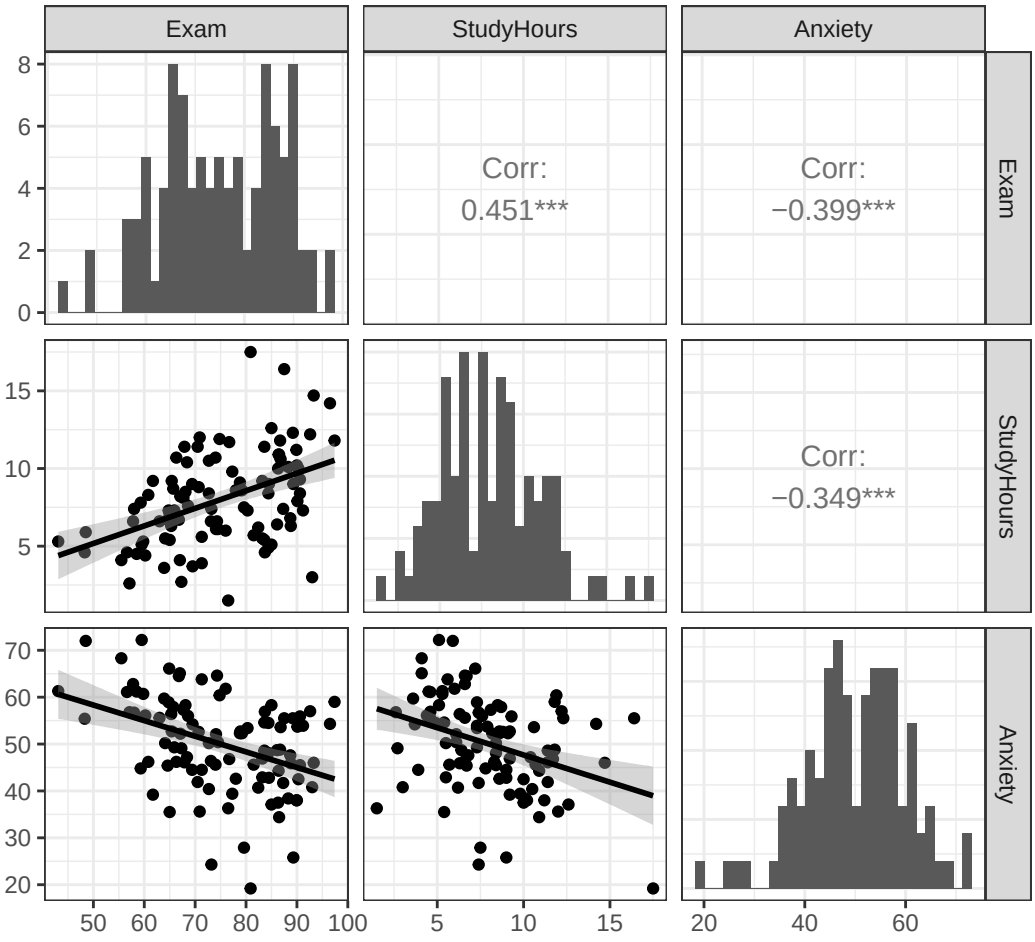


Figure 13.1: All three variables against each other. Read it the way we read the matrix in the correlation chapter: diagonal is each variable alone, below is the scatterplot, above is the number.

The plot is compact because each variable takes a turn as both an x-axis and a y-axis. Read one panel at a time. Do not stare at the whole matrix until a causal theory appears.

You can see all three correlations in the scatterplots:

- **Bottom left:** Study Hours and Test Anxiety slope downward, so students who study more tend to be less anxious.
- **Bottom middle:** Test Anxiety and Exam Score slope downward, so more anxious students tend to score lower.
- **Bottom right:** Study Hours and Exam Score slope upward, so more studying is associated with higher scores.

13.3 Understanding Overlapping Variance

Before we run the multiple regression, we need to understand *why* having two correlated predictors creates the problem we described above. The key concept is **overlapping variance**.

13.3.1 The Venn Diagram Approach

When you use simple regression with a single predictor, things are straightforward: X either predicts Y or it does not. We can visualize this as two circles, one for X and one for Y, where the *overlap* represents the shared variance between them. As you already know, that overlap, when squared, gives us R^2.

When we add a second predictor, things get more interesting. Now we have three circles (X1, X2, and Y), and some of the circles overlap with each other:

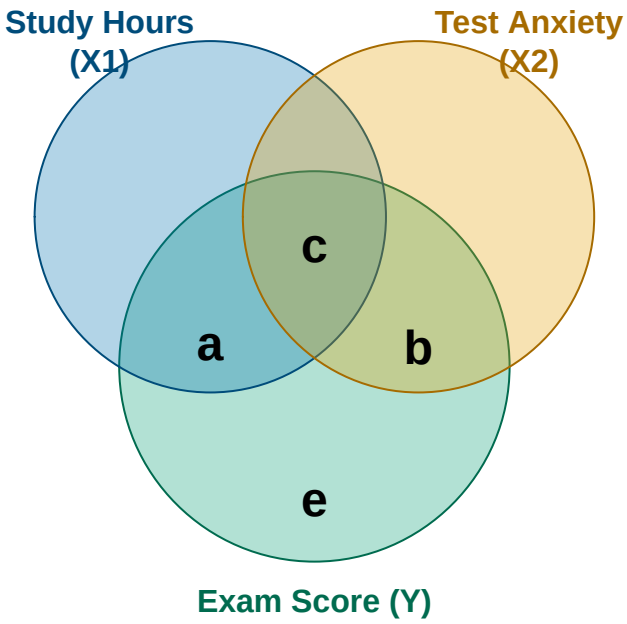


Figure 13.2: A Ballantine for Study Hours, Test Anxiety, and Exam Score. The areas are conceptual, not drawn to scale.

In this diagram, we can label four distinct regions:

- **Region a** = variance in Y that is **uniquely explained by X1** (Study Hours), above and beyond X2.
- **Region b** = variance in Y that is **uniquely explained by X2** (Test Anxiety), above and beyond X1.

- **Region c** = variance in Y that **both X1 and X2 share**. Neither predictor can claim exclusive credit for this part.
- **Region e** = variance in Y that is **unexplained** by either predictor (residual error).

The total variance in Y explained by the model is $a + b + c$.

13.3.2 The Problem with Region c

Region c is the heart of the matter. It represents variance in exam scores that both Study Hours and Anxiety are *jointly* associated with, because the two predictors are correlated with each other.

Here is the problem that region c creates:

- If you only look at **Study Hours → Exam** (simple regression), the observed relationship includes region a *and* region c. Part of what looks like a "Study Hours effect" is actually shared with Anxiety, because students who study more tend to be less anxious, and less anxious students score higher.
- If you only look at **Anxiety → Exam** (simple regression), the observed relationship includes region b *and* region c. Part of what looks like an "Anxiety effect" is actually shared with Study Hours, for the same reason.

So the bivariate slopes mix unique and shared information. In this particular example, each slope is larger in absolute value when examined alone because it also has access to variance shared with the other predictor.

Multiple regression solves this by estimating each predictor's unique contribution. In the multiple regression model, Study Hours gets credit only for region a, and Anxiety gets credit only for region b. Region c is acknowledged as shared variance and included in R^2, but it is not falsely assigned to either predictor.

In these simulated data, the multiple-regression slopes will be **smaller** than the corresponding simple-regression slopes. That is not a universal rule. Depending on the correlation pattern, adding a control can make a slope shrink, grow, or reverse. The controlled slopes answer a different question: what does each predictor contribute conditionally, given the other predictor in the model?

13.4 The Multiple Regression Equation

You already know the simple regression equation:

$$Y = B_0 + B_1X + e$$

Multiple regression extends this to two (or more) predictors:

$$Y = B_0 + B_1X_1 + B_2X_2 + e$$

Where:

- B_0 = intercept (predicted Y when all predictors = 0)
- B_1 = slope for X1 (Study Hours)
- B_2 = slope for X2 (Test Anxiety)
- e = residual error (observed − predicted)

In our example:

$$\widehat{Exam} = B_0 + B_1 \cdot StudyHours + B_2 \cdot Anxiety$$

The equation looks very similar to simple regression. But the **interpretation of the slopes has changed in an important way**.

13.4.1 What "Controlling For" Means

In simple regression, B_1 tells you: *"For each one-unit increase in X, how much does Y change?"*

In multiple regression, B_1 tells you something more precise: *"For each one-unit increase in X1, how much does Y change, **among people who have the same value of X2**?"*

This is what **statistical control** means. The slope for Study Hours in our multiple regression model describes the Study Hours–Exam association *as if we were comparing students at the same anxiety level.* We are estimating that association after holding anxiety **constant**, or equivalently, after **controlling for** anxiety.

Think of it this way: imagine sorting all 100 students into groups based on their anxiety score. Within the low-anxiety group, every student has roughly the same anxiety, so exam differences cannot be attributed to measured anxiety differences inside that group. They may be associated with study hours, or with other variables we did not measure. Now imagine doing the same thing for the medium-anxiety group and the high-anxiety group. Multiple regression performs a continuous version of this comparison for every value of Anxiety to isolate the unique predictive contribution of Study Hours.

The key insight: a slope in multiple regression is the association between that predictor and Y after accounting for the predictor's linear relationship with the other predictors in the model. It is the unique predictive contribution of X1 above and beyond what X2 already explains. It is not automatically a causal effect.

13.5 Running the Analysis in R

Let's run the analysis step by step. We will start with simple regression for each predictor alone, then combine them. This approach lets you see exactly what changes, and why, when you add the second predictor.

13.5.1 Step 1: Study Hours Alone

```
Model.1 <- lm(Exam ~ StudyHours, data = dat)
summary(Model.1)
```

```
Call:
lm(formula = Exam ~ StudyHours, data = dat)

Residuals:
    Min      1Q  Median      3Q     Max
-27.041  -8.274   1.174   8.799  26.996
```

```
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  60.6078     3.0717  19.731  < 2e-16 ***
StudyHours    1.7987     0.3597   5.001  2.5e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 10.76 on 98 degrees of freedom
Multiple R-squared:  0.2033,    Adjusted R-squared:  0.1952
F-statistic: 25.01 on 1 and 98 DF,  p-value: 2.501e-06
```

So, looking at Study Hours alone:

- For every additional hour of studying per week, exam scores increase by about 1.8 points.
- This is a statistically significant positive effect.
- Study Hours alone explains about 20.3% of the variance in exam scores ($R^2 = 0.203$).

13.5.2 Step 2: Test Anxiety Alone

```r
Model.2 <- lm(Exam ~ Anxiety, data = dat)
summary(Model.2)
```

```
Call:
lm(formula = Exam ~ Anxiety, data = dat)

Residuals:
    Min      1Q  Median      3Q     Max
-26.482  -8.321  -1.005   8.369  26.716

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  98.9535     5.6637  17.472  < 2e-16 ***
Anxiety      -0.4792     0.1111  -4.313 3.84e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 11.06 on 98 degrees of freedom
Multiple R-squared:  0.1595,    Adjusted R-squared:  0.151
F-statistic:  18.6 on 1 and 98 DF,  p-value: 3.844e-05
```

Looking at Test Anxiety alone:

- For every 1-point increase in test anxiety, exam scores decrease by about 0.48 points.
- This is also statistically significant.
- Anxiety alone explains about 16% of the variance ($R^2 = 0.16$).

13.5.3 Step 3: Both Predictors Together

Now for the key step of entering both predictors simultaneously. In R, we add a second predictor using the + sign in the formula:

```
Model.3 <- lm(Exam ~ StudyHours + Anxiety, data = dat)
summary(Model.3)
```

```
Call:
lm(formula = Exam ~ StudyHours + Anxiety, data = dat)

Residuals:
     Min       1Q   Median       3Q      Max
-24.3420  -7.8649  -0.3942   7.4378  22.0359

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  80.2059     7.2176  11.112  < 2e-16 ***
StudyHours    1.4149     0.3693   3.831 0.000226 ***
Anxiety      -0.3306     0.1111  -2.976 0.003684 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 10.36 on 97 degrees of freedom
Multiple R-squared:   0.27, Adjusted R-squared:  0.2549
F-statistic: 17.94 on 2 and 97 DF,  p-value: 2.351e-07
```

13.6 Interpreting the Multiple Regression Output

Let's work through what changed when we added both predictors.

13.6.1 The Slopes Have Shrunk. Why?

Compare the slope estimates across all three models:

Predictor	Simple Regression (Alone)	Multiple Regression (Controlling for the Other)
Study Hours	1.799	1.415
Test Anxiety	-0.479	-0.331

Both slopes are smaller in absolute value when the two predictors are in the model together. This is exactly what the Venn diagram predicts. Here is why:

When Study Hours was the *only* predictor (Model 1), its prediction included region a *and* region c: its unique association with exam performance plus variance shared with Anxiety. Once both predictors enter Model 3, the Study Hours coefficient is based on Study Hours variation that Anxiety does not linearly predict. Region c still contributes to the model's overall R^2, but neither coefficient can claim it as uniquely its own.

The same logic applies in reverse: Anxiety's slope shrinks from -0.479 to -0.331 because some of its bivariate association with Exam Score overlaps with lower Study Hours.

This shrinkage is **not a problem**. Model 3 is answering the conditional question we asked. Its slopes estimate each predictor's unique association with Exam Score given the other measured predictor. They are not automatically "truer," more causal, or less biased; that depends on the research design, measurement quality, and whether controlling for that variable makes theoretical sense.

13.6.2 Interpreting Each Controlled Slope

Here is how to interpret each slope in Model 3 in plain language:

Study Hours ($B_1 = 1.41$):

> "For students with the *same* level of test anxiety, each additional hour of weekly study is associated with a 1.41-point increase in exam score."

Test Anxiety ($B_2 = -0.33$):

> "For students who study the *same* number of hours per week, each one-point increase in test anxiety is associated with a 0.33-point decrease in exam score."

The phrase **"for students with the same [other variable]"** is the key to interpreting any slope in multiple regression. When you see a slope reported in a multiple regression model, it always carries that "holding everything else constant" qualifier even when it is not stated explicitly.

13.6.3 The Intercept

The intercept ($B_0 = 80.21$) is the predicted exam score when **both** Study Hours $= 0$ **and** Anxiety $= 0$. Since no student in our sample studied for zero hours *and* had zero anxiety simultaneously, this is not a prediction we would ever actually make. Like in simple regression, the intercept just anchors the equation mathematically. It becomes more meaningful when predictors are mean-centered (but we will save that for another time).

13.6.4 Making a Prediction

Now that we have the full equation, we can make predictions. For a student who studies 10 hours per week and has an anxiety score of 45:

```
# Predicted exam score for a student with 10 study hours and anxiety = 45
B0 <- Model.3$coefficients[1]
B1 <- Model.3$coefficients[2]
B2 <- Model.3$coefficients[3]

predicted <- B0 + B1 * 10 + B2 * 45
round(predicted, 1)
```

```
(Intercept)
       79.5
```

This student would be predicted to score about 79.5 on the exam.

13.7 How Well Does the Model Fit? R^2

Let's look at how the model's explanatory power (R^2) changes across our three models.

```
r2_model1 <- summary(Model.1)$r.squared
r2_model2 <- summary(Model.2)$r.squared
r2_model3 <- summary(Model.3)$r.squared
```

```
Model 1 (Study Hours only): R-squared = 0.203
```

```
Model 2 (Anxiety only):     R-squared = 0.16
```

```
Model 3 (Both predictors):  R-squared = 0.27
```

The full model ($R^2 = 0.27$) explains more variance than either predictor alone. But notice something: the combined model does **not** explain as much as the simple sum of the two individual R^2 values ($0.203 + 0.16 = 0.363$). Why not?

Go back to the Venn diagram. When Study Hours was the only predictor, it explained regions $a + c$. When Anxiety was the only predictor, it explained regions $b + c$. Together, they explain regions $a + b + c$. But notice that region c was already counted in Model 1's R^2. Adding Anxiety on top of Study Hours can only add region b to what was already explained. It cannot double-count region c.

This is why R^2 does not add up neatly when predictors are correlated. The combined model explains more than either predictor alone, but less than the naive sum of their individual R^2 values.

13.7.1 Breaking Down the Venn Diagram Numerically

We can actually estimate the size of each region in the Venn diagram using simple subtraction:

```
# Unique variance of Study Hours (region a)
region_a <- r2_model3 - r2_model2

# Unique variance of Anxiety (region b)
region_b <- r2_model3 - r2_model1

# Shared variance (region c)
region_c <- r2_model1 + r2_model2 - r2_model3
```

```
Region a - unique to Study Hours: 0.11
```

```
Region b - unique to Anxiety:     0.067
```

```
Region c - shared variance:       0.093
```

```
Total R-squared:                  0.27
```

In our data:

- Study Hours has about 11% unique variance in exam scores, its unique predictive contribution after accounting for anxiety (region a).
- Anxiety has about 6.7% unique variance, its unique predictive contribution after accounting for study hours (region b).
- About 9.3% of the variance in exam scores is shared between the two predictors: variance that both contribute to together, but neither can claim exclusively (region c).

This numerical breakdown puts the Venn diagram concept in concrete terms. The **squared semi-partial correlations** correspond directly to regions a and b. The regression slopes are based on the same unique predictor variation, but express the expected outcome change in Exam Score units.

> ⚠ If Region c Comes Out Negative, You Found Something
>
> Run this subtraction on your own data and region c can come back **negative**, which is impossible for an overlapping area and tends to make students assume they broke the arithmetic.
>
> You did not. A negative c means the two predictors are doing something called **suppression**, where adding one predictor makes the other's contribution *larger* rather than smaller. It is real, it is not rare, and it is the point at which the Venn diagram metaphor quietly resigns and goes home with its ball. The next chapter takes suppression apart properly. Grad readers who want the full explaination should see Cohen, Cohen, West and Aiken, chapter 3.

13.7.2 Adding Predictors Always Increases R^2

There is an important quirk about R^2: it will **always increase** when you add another predictor to the model, even if that predictor is completely useless random noise. This is simply because any variable, no matter how irrelevant, will share at least a tiny sliver of variance with Y by chance in any given sample.

This means if you kept piling more and more predictors into a model, R^2 would keep creeping upward from sampling variability alone.

13.7.3 Adjusted R^2

Adjusted R^2 corrects for this problem by applying a penalty for each additional predictor:

$$\text{Adjusted } R^2 = 1 - (1 - R^2)\,\frac{n-1}{n-k-1}$$

Where n = sample size and k = number of predictors in the model. With a small sample and many predictors, the penalty is larger. With a large sample and few predictors, the penalty is small.

Raw R^2 describes how well the model fits **this sample**. Adjusted R^2 asks a slightly less flattering question: after penalizing us for the number of predictors we used, how much variance does the model appear to explain? Because raw R^2 capitalizes on accidental relationships in the current sample, adjusted R^2 is usually closer to what we might expect in the population or a new sample. It is not a prophecy, but it is less easily impressed.

Adjusted R^2 can increase when a new predictor improves the model enough to overcome the penalty. It can decrease when the added predictor contributes too little. This makes it a fairer comparison tool when models contain different numbers of predictors.

```
adj_r2_model1 <- summary(Model.1)$adj.r.squared
adj_r2_model3 <- summary(Model.3)$adj.r.squared
```

```
Model 1 Adjusted R-squared: 0.195
```

```
Model 3 Adjusted R-squared: 0.255
```

Our adjusted R^2 increased from 0.195 to 0.255 when we added Anxiety. This confirms that Anxiety is genuinely useful after the penalty for adding a predictor, the model still explains meaningfully more variance.

People commonly report **both** R^2 and adjusted R^2. Raw R^2 describes the variance explained in the observed sample. Adjusted R^2 provides the more conservative estimate after accounting for model size and is often the better guess about how much explanatory power will survive beyond this particular group of exploited undergraduates.

13.8 Visualizing Multiple Regression

Tables of numbers are useful, but plots help build intuition. Let's visualize what is happening.

13.8.1 The Study Hours Slope Before and After Controlling

This first pair of plots shows the predicted relationship between Study Hours and Exam Score in the two models. The key question is: how does the slope change when we control for Anxiety?

We draw these with `plot_model()` from **sjPlot**. Your homework uses it too. Read the code and not just the picture. It takes predictions straight out of a fitted model: `type = "pred"` means "show me predicted values," and `terms` picks which predictor goes on the x-axis. Both panels are locked to the same y-axis with `coord_cartesian()`, because a slope comparison across two differently scaled plots is not a comparison at all.

```
library(patchwork)

p.study.before <- plot_model(
  Model.1,
  type       = "pred",
  terms      = "StudyHours",
  colors     = "blue",
  title      = "Study Hours and Exam\n(Without controlling for Anxiety)",
  axis.title = c("Study Hours per Week", "Predicted Exam Score")
) +
  coord_cartesian(ylim = c(55, 95)) +
  theme_bw()

p.study.after <- plot_model(
  Model.3,
  type       = "pred",
  terms      = "StudyHours",
  colors     = "blue",
  title      = "Study Hours and Exam\n(Controlling for Anxiety)",
  axis.title = c("Study Hours per Week", "Predicted Exam Score")
) +
  coord_cartesian(ylim = c(55, 95)) +
  theme_bw()
```

```
p.study.before + p.study.after
```

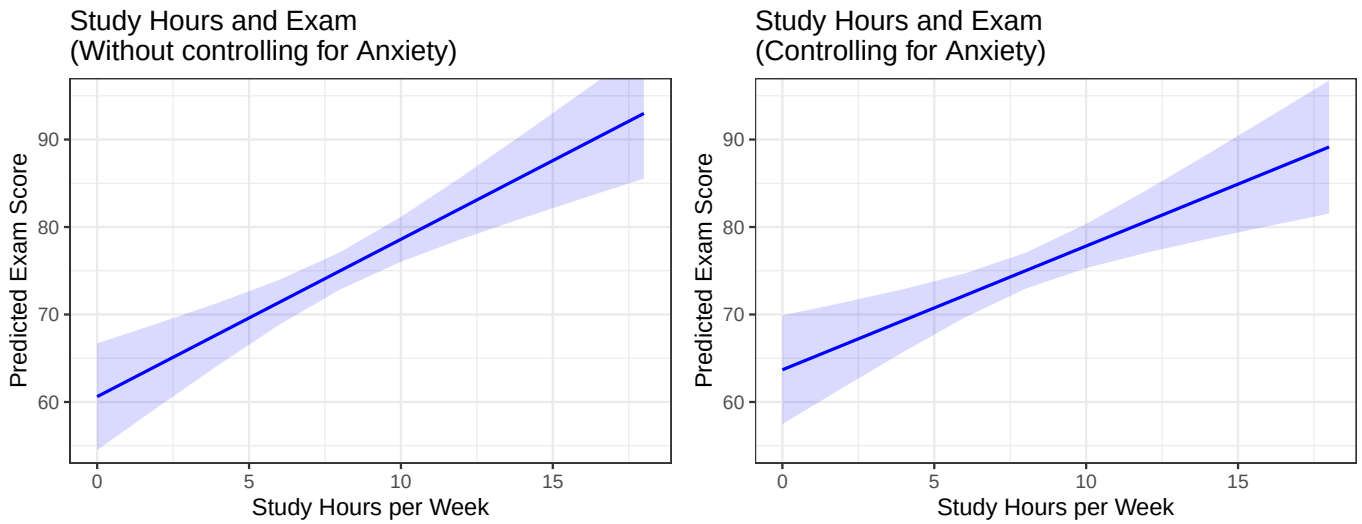


Figure 13.3: The Study Hours slope, before and after controlling for Anxiety. Same y-axis on both panels, so the difference you see is real.

The slope becomes slightly shallower after controlling for Anxiety. This is statistical control in action: some of the original Study Hours effect was shared with (and partially driven by) lower anxiety levels in high-studying students.

Be careful about what the controlled slope is, though. It answers the **conditional** question: what studying is associated with among students at the same anxiety level.

What it is *not* is more **precise**. Precision means the standard error, and here the controlled slope's standard error is slightly *larger* than the uncontrolled one. That is the normal pattern: correlated predictors share information, and the more of it they share, the less each one is separately pinned down. A later chapter measures exactly this with something called *variance inflation*.

13.8.2 The Anxiety Slope Before and After Controlling

```
p.anx.before <- plot_model(
  Model.2,
  type       = "pred",
  terms      = "Anxiety",
  colors     = "red",
  title      = "Anxiety and Exam\n(Without controlling for Study Hours)",
  axis.title = c("Test Anxiety (0-100)", "Predicted Exam Score")
) +
  coord_cartesian(ylim = c(55, 95)) +
  theme_bw()

p.anx.after <- plot_model(
  Model.3,
  type       = "pred",
```

```
  terms      = "Anxiety",
  colors     = "red",
  title      = "Anxiety and Exam\n(Controlling for Study Hours)",
  axis.title = c("Test Anxiety (0-100)", "Predicted Exam Score")
) +
  coord_cartesian(ylim = c(55, 95)) +
  theme_bw()

p.anx.before + p.anx.after
```

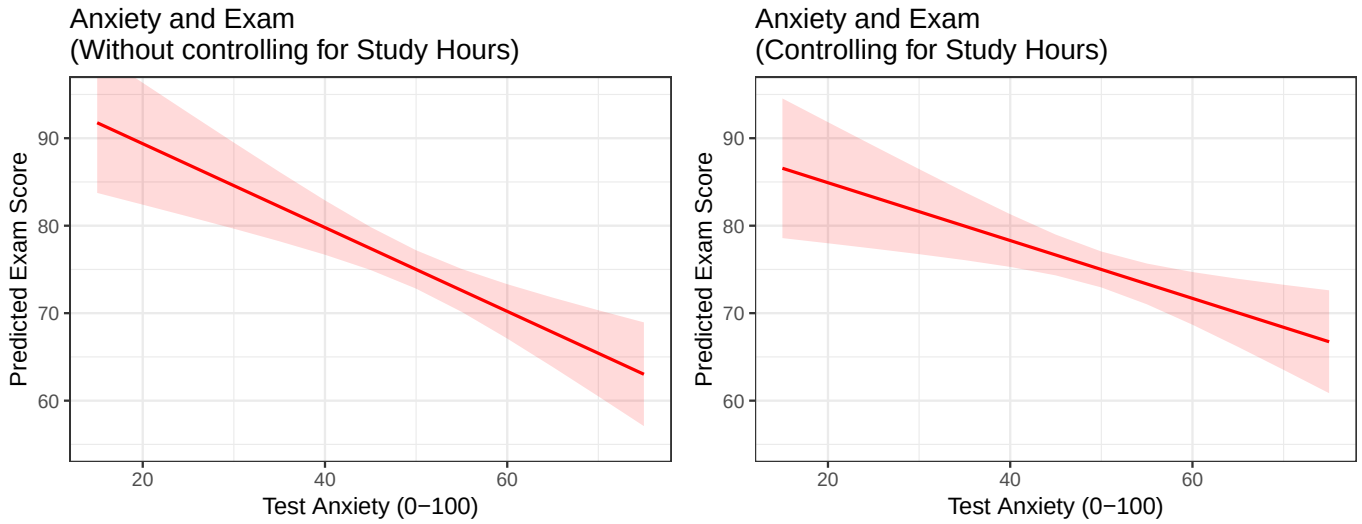


Figure 13.4: The Anxiety slope, before and after controlling for Study Hours. Same y-axis again.

The same pattern appears in reverse: the negative slope for Anxiety becomes slightly smaller after we account for the fact that highly anxious students also tend to study less. The controlled slope describes Anxiety's conditional association with performance while Study Hours is held constant.

13.8.3 The Full Model: Both Predictors Together

We can also visualize both predictors simultaneously. The plot below shows how predicted exam scores change with Study Hours, for students at three different anxiety levels (low, average, and high).

```
plot_model(
  Model.3,
  type         = "pred",
  terms        = c("StudyHours", "Anxiety [40, 50, 60]"),
  title        = "Predicted Exam Score by Study Hours and Anxiety Level",
  legend.title = "Test Anxiety",
  axis.title   = c("Study Hours per Week", "Predicted Exam Score")
) +
  coord_cartesian(ylim = c(55, 95)) +
  theme_bw()
```

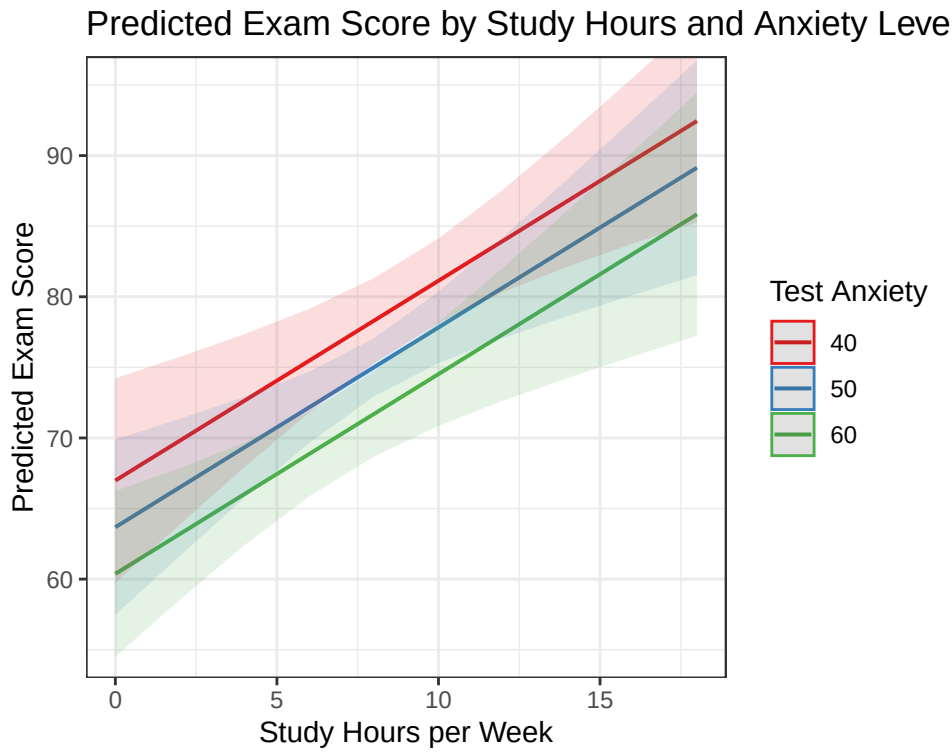


Figure 13.5: Predicted exam score by study hours, drawn separately for low, average, and high anxiety. The lines are parallel, and that is the point.

Each line represents students at a different anxiety level: low anxiety (40), average anxiety (50), and high anxiety (60). Notice:

- **More studying always helps**: all three lines slope upward, regardless of anxiety level.
- **Higher anxiety always hurts**: the lines are shifted downward for higher-anxiety students.
- **The lines are parallel**: the predicted difference associated with each additional study hour is the same at every anxiety level. This is what it means for the terms to be **additive** in our model: the model contains separate Study Hours and Anxiety slopes but no interaction between them.

This plot gives you a visual intuition for what "controlling for" means: the slope of each line within one anxiety group represents the Study Hours association holding Anxiety constant, exactly what the multiple regression coefficient estimates.

13.9 A Note on What Did Not Change

In Model 3, **both predictors remain statistically significant** after controlling for each other. The fact that both slopes shrank does not mean either predictor is unimportant. Both retain unique predictive associations with Exam Score: Study Hours contributes above and beyond Anxiety (region a), and Anxiety contributes above and beyond Study Hours (region b).

The meaning: **a slope that shrinks when you add a control variable is not automatically a sign of a weak finding.** It means the controlled and uncontrolled models answer different questions. In this example, the controlled slope is based on predictor variance not linearly explained by the other predictor.

Adding a control variable can also make a slope grow, disappear, reverse, or change statistical significance. Those patterns can reflect confounding, suppression, collinearity, measurement problems, or a poorly chosen control. The Statistical Control chapter takes apart what statistical control is actually doing.

13.10 APA Write-Up

Here is how you would write up these results in APA format:

Prior to the main analysis, zero-order correlations were examined. Study hours positively correlated with exam performance ($r = .45$, $p < .001$), and test anxiety negatively correlated with exam performance ($r = -.40$, $p < .001$). The two predictors were also significantly correlated with each other ($r = -.35$, $p < .001$), indicating that students who studied more tended to report lower anxiety.

A multiple linear regression was conducted with exam score as the outcome and study hours and test anxiety as predictors. The overall model was statistically significant, $F(2, 97) = 17.94$, $p < .001$, $R^2 = 0.27$, adjusted $R^2 = 0.255$.

After controlling for test anxiety, study hours significantly predicted exam performance, $b = 1.41$, $SE = 0.37$, $t(97) = 3.83$, $p < .001$. After controlling for study hours, test anxiety significantly predicted exam performance, $b = -0.33$, $SE = 0.11$, $t(97) = -2.98$, $p < 0.004$.

Together, these results indicate that Study Hours and Test Anxiety each uniquely predict Exam Score in this model: students who study more and experience less anxiety tend to perform better, and both associations remain after accounting for the correlation between the predictors.

A few things to notice in this write-up:

- We tend to report the zero-order correlations first, so readers understand the bivariate relationships before seeing the controlled effects.
- We use the phrase **"after controlling for..."** (or equivalently, "above and beyond" or "holding constant") to signal that a multiple regression slope is being reported.
- We report F, R^2, and adjusted R^2 for the overall model, then report b, SE, t, and p for each individual predictor.
- We do not report Cohen's d here. In regression, b (the unstandardized slope) is the primary effect size. The overall model's R^2 tells us the total proportion of variance explained.

13.11 The Short Story

- One predictor is never enough, because predictors are correlated with each other. That is the normal condition of psychological data.
- A slope in multiple regression is a **unique** association: what this predictor is worth while holding the other predictors constant. Every such slope carries an invisible "holding the others constant" clause, and you should read it aloud every time.
- Slopes can shrink, grow, disappear, or flip sign when a control enters. None of those outcomes is the "true" one. Each model answers a different question, and you have to know which question you asked.
- R^2 values do not add up across models, because the variance the predictors share (region c) only gets counted once.
- Adjusted R^2 charges rent for every predictor you add, which is why it can go down when you add a useless one.
- Statistical control is not experimental control. Holding a variable constant in an equation is not the same as holding it constant in the world, and only design decides which causal stories survive.

Next: Did the Second Predictor Earn Its Keep?

The next chapter, Hierarchical Regression, picks up Models 1 and 3 where we just left them and asks whether adding Test Anxiety improved the model by more than chance. After that, Statistical Control: What Are We Removing? uses these same students to show exactly how residuals, semi-partial correlation, partial correlation, standardized beta, and change in R^2 fit together. No new imaginary undergraduates will be recruited until the current ones have finished the exam.

14 Hierarchical Regression: Testing Whether Your Model Improves

Everything in this chapter runs on one simulated dataset: 100 students, with exam scores, study hours, and test anxiety wired together at correlations I picked. Here it is, so you can run the rest of the chapter yourself.

```
library(MASS)

set.seed(42)

n   <- 100
rY1 <-  0.45   # Study Hours → Exam Score
rY2 <- -0.40   # Test Anxiety → Exam Score
r12 <- -0.35   # Study Hours <-> Test Anxiety

Sigma <- matrix(c(
   1,   rY1, rY2,
  rY1,   1,  r12,
  rY2,  r12,  1
), ncol = 3)

raw <- as.data.frame(mvrnorm(n, mu = c(0, 0, 0), Sigma = Sigma, empirical = TRUE))
colnames(raw) <- c("Exam", "StudyHours", "Anxiety")

dat <- data.frame(
  Exam       = round(raw$Exam       * 12 + 75, 1),
  StudyHours = round(raw$StudyHours *  3 +  8, 1),
  Anxiety    = round(raw$Anxiety    * 10 + 50, 1)
)
```

14.1 I Made This Model Better by Adding Absolutely Nothing

Here is a model getting better. We start with our theory model and just keep adding meaningless random variables to it. So `Model.0 <- lm(Exam ~ StudyHours)`, then `Model.1 <- lm(Exam ~ StudyHours + RandomX1)`, then `Model.2 <- lm(Exam ~ StudyHours + RandomX1 + RandomX2)`, and so forth. Only the first model is real. The rest are garbage models.

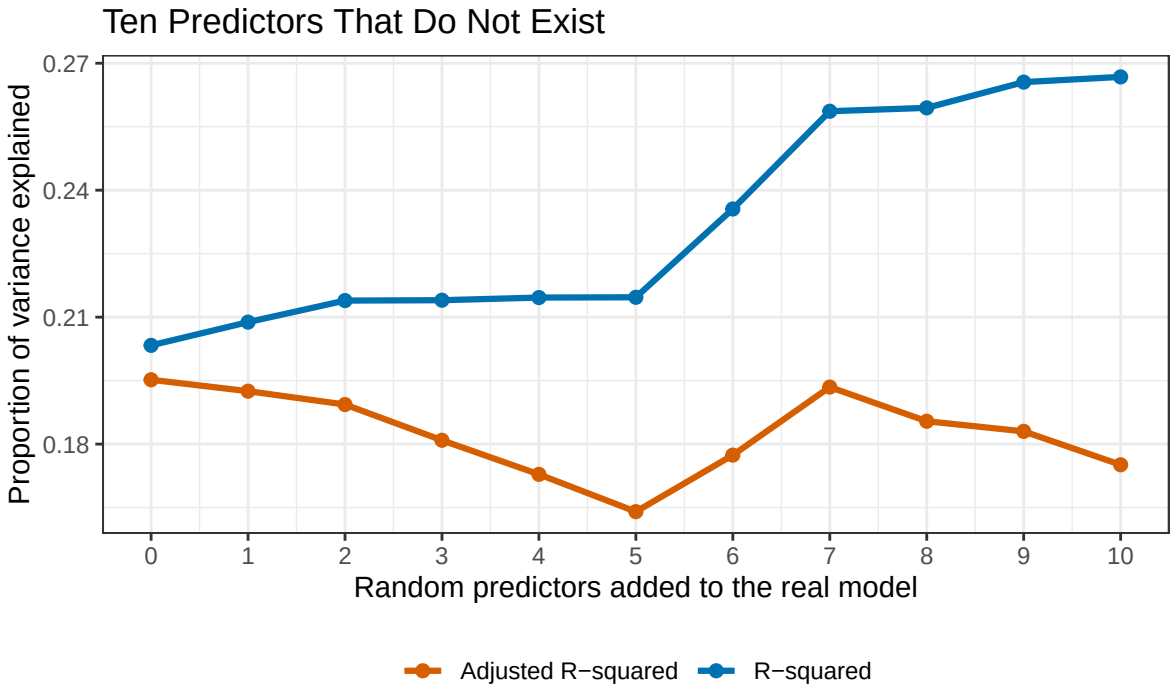


Figure 14.1: Ten columns of random numbers, added to the exam model one at a time. The blue line is R^2 and it never once goes down. The orange line is adjusted R^2, which charges rent for each new predictor and ends lower than it started.

The rising line is R^2, the proportion of exam-score variance the model explains. It starts with one real predictor, Study Hours, at 0.203. Then I add random variables one at a time. Ten in all, and every single one leaves the model fitting better than it was before.

I did not measure anything as the variables were just made by `rnorm()`—pure Gaussian noise. They do not know what an exam is, they have never met a student, and each one existed for roughly a millisecond before being hired as a predictor for this model. R^2 climbed to 0.267 anyway, because R^2 is not permitted to fall below that of the initial real model. Every predictor you add hands least squares one more knob, and least squares will always find some small turn of that knob that fits *this particular sample* a little "more better". *Does this worry you? It should.*

Here is the part that should worry you even more. A real predictor with real theory behind it raises R^2 by 0.067. Ten columns of pure noise raised it by 0.063. Measured in raw R^2, garbage and theory bought roughly the same thing.

The orange line is the first hint of a defense against this problem. **Adjusted** R^2 charges more rent for every predictor you take on, and by the tenth noise column it has sagged from 0.195 to 0.175, below where it started. That is useful. It is also only a rule of thumb. It can tell you something smells wrong. It cannot tell you whether to believe a result.

What you need is a way to ask the harder question: **when a model improves, did it improve by more than nothing would have?** That question has a distribution, a test, and a one-line R function, and it is what the rest of this chapter is for.

Adding a predictor always helps. That is precisely why "it helped" is not evidence of anything.

14.2 Where We Left Off, and the Question We Skipped

In the previous chapter, we used **multiple regression** to predict exam performance from two predictors simultaneously: Study Hours and Test Anxiety. We learned that each predictor has a *unique* effect on exam scores, an effect that persists even after statistically controlling for the other predictor.

By the end of that chapter, we had fit three models:

- **Model 1:** Exam ~ Study Hours (simple regression, one predictor)
- **Model 2:** Exam ~ Test Anxiety (simple regression, one predictor)
- **Model 3:** Exam ~ Study Hours + Test Anxiety (multiple regression, both predictors)

We showed that Model 3 explained more variance in exam scores (R^2) than either predictor alone, and we compared the slopes before and after controlling. But we never formally asked: **does adding Test Anxiety to the model produce a statistically significant improvement?** Or put differently, is the improvement in R^2 when we go from Model 1 to Model 3 bigger than we would expect from chance alone?

That question is the heart of **hierarchical regression**, and it is the focus of this chapter.

14.3 What Is Hierarchical Regression?

Hierarchical regression is a strategy for building and comparing regression models in a deliberate, theory-driven order. Instead of putting all your predictors in at once (**Simultaneous regression**), you add them in meaningful *steps*, and at each step you ask whether the new addition genuinely improved your ability to predict the outcome.

The word "hierarchical" refers to the hierarchy of models, not a hierarchy of people or importance. Each model in the sequence is more complex than the one before it, and you evaluate whether that added complexity is worth it.

> **i** Two Things Are Called Hierarchical, and This Is Only One of Them
>
> What we are doing here is adding predictors in theory-driven steps. It has nothing to do with **hierarchical linear models**, which are multilevel or mixed models for clustered data: students inside classrooms, trials inside participants, patients inside hospitals. Those get their own chapter, Mixed Regression.
>
> This matters because searching "hierarchical regression in R" will drop you into `lme4` and random effects, and you will conclude you need machinery you do not need. Same adjective, unrelated ideas. Blame history.

This approach is useful any time you have a specific question about the *incremental value* of a predictor. In our example, the question is precise and testable:

> **Does Test Anxiety explain variance in exam performance above and beyond what Study Hours already explains?**

Notice the phrase "above and beyond." That is the language of hierarchical regression. You are not just asking whether Test Anxiety predicts exam scores (it does; we know this from the simple regression). You are asking whether it adds *something new* to a model that already includes Study Hours.

14.4 Nested Models: Which Comparisons Are Even Legal

To compare two models formally, they need to be **nested**. Two models are nested when the predictors in the simpler model are a *subset* of the predictors in the more complex model. In other words, the simpler model is contained inside the more complex one.

Here is how nesting works in our example:

Model	Predictors	Is it nested in...?
Model 1	Study Hours	n/a
Model 3	Study Hours + Test Anxiety	Yes, contains Model 1

Model 1 only has Study Hours. Model 3 has Study Hours **plus** Test Anxiety. Because Model 1's predictor (Study Hours) is fully contained within Model 3, these two models are nested. The difference between them is exactly one predictor: Test Anxiety.

This is what makes the comparison meaningful. When we compare these two models, we are directly asking: **what does Test Anxiety add, given that Study Hours is already in the model?** We are not comparing apples to oranges. We are asking a precise, surgical question about one specific addition.

If the models were *not* nested (say Model A had Study Hours and Model B had Test Anxiety), we could not make a direct statistical comparison, because the difference between them involves both removing one predictor *and* adding a different one. The formal test we will use below only works for nested models.

14.5 Setting Up the Hierarchical Comparison

In our example, the research question drives the ordering of the models:

Step 1 (Block 1): Enter Study Hours. This is our primary predictor: the one we already know matters, and the one that came first in our theoretical thinking about what drives exam performance.

Step 2 (Block 2): Add Test Anxiety. This is the incremental question: does anxiety explain variance in exam scores *above and beyond* what study behavior already tells us?

We are deliberately **not** starting from scratch with each model. We are building on what came before, which is why this is called hierarchical entry.

Let's run both models:

```
# Block 1: Study Hours only
Model.1 <- lm(Exam ~ StudyHours, data = dat)

# Block 2: Add Test Anxiety
Model.3 <- lm(Exam ~ StudyHours + Anxiety, data = dat)
```

And let's recall what R^2 looks like for each:

```
r2_m1 <- summary(Model.1)$r.squared
r2_m3 <- summary(Model.3)$r.squared

cat("Model 1 (Study Hours only): R-squared =", round(r2_m1, 3), "\n")
```

```
Model 1 (Study Hours only): R-squared = 0.203
```

```
cat("Model 3 (Both predictors):  R-squared =", round(r2_m3, 3), "\n")
```

```
Model 3 (Both predictors):  R-squared = 0.27
```

```
cat("Change in R-squared: Delta R-squared =", round(r2_m3 - r2_m1, 3), "\n")
```

```
Change in R-squared: Delta R-squared = 0.067
```

The R^2 went up when we added Test Anxiety. But R^2 **always** goes up when you add a predictor. Even a useless one will explain a tiny sliver of variance just by chance. So we need a formal test to decide whether this improvement is more than chance.

14.6 The Change in R^2: What Are We Measuring?

Before we run the test, let's understand what ΔR^2 (read: "delta R-squared," or "change in R-squared") actually represents.

Recall the Venn diagram from the previous chapter. When Study Hours is the only predictor, it explains regions $a + c$: its unique share of the variance in exam scores, plus the variance it shares with Test Anxiety. When we add Test Anxiety to the model, we pick up region b, the variance in exam scores that Test Anxiety explains *uniquely*, after Study Hours has already claimed its portion.

$$\Delta R^2 = R^2_{\text{Model 3}} - R^2_{\text{Model 1}} = \text{Region } b$$

So ΔR^2 is the proportion of variance in exam scores that Test Anxiety explains *uniquely*: the additional predictive power it brings after Study Hours has already been accounted for. This is precisely what we want to know.

This quantity has a second name, and you are going to meet it in the next chapter. Predict Test Anxiety from Study Hours, keep the leftovers, correlate those leftovers with Exam Score, and square the result. You get 0.067, which is the same ΔR^2 printed above. That number is the **squared semi-partial correlation**, sr^2, and Statistical Control takes it apart properly. The identity $sr^2 = \Delta R^2$ is not a coincidence. It is region b measured twice, in two vocabularies.

The question is: is this ΔR^2 of 0.067 (6.7% of variance) large enough to conclude that Test Anxiety is a genuinely useful addition? Or could a difference this big arise just from sampling variability, even if Test Anxiety had no real predictive power?

14.7 The F-Test for Model Comparison

To judge our gain we need to know what a *useless* predictor buys, and that is not a question we have to be clever about. We can hire five thousand useless predictors and watch what happens.

14.7.1 First, Look at What Nothing Buys

```r
set.seed(7)

null_delta <- replicate(5000, {
  d <- dat
  d$Noise <- rnorm(nrow(d))                    # a predictor made of nothing
  summary(lm(Exam ~ StudyHours + Noise, data = d))$r.squared - r2_m1
})
```

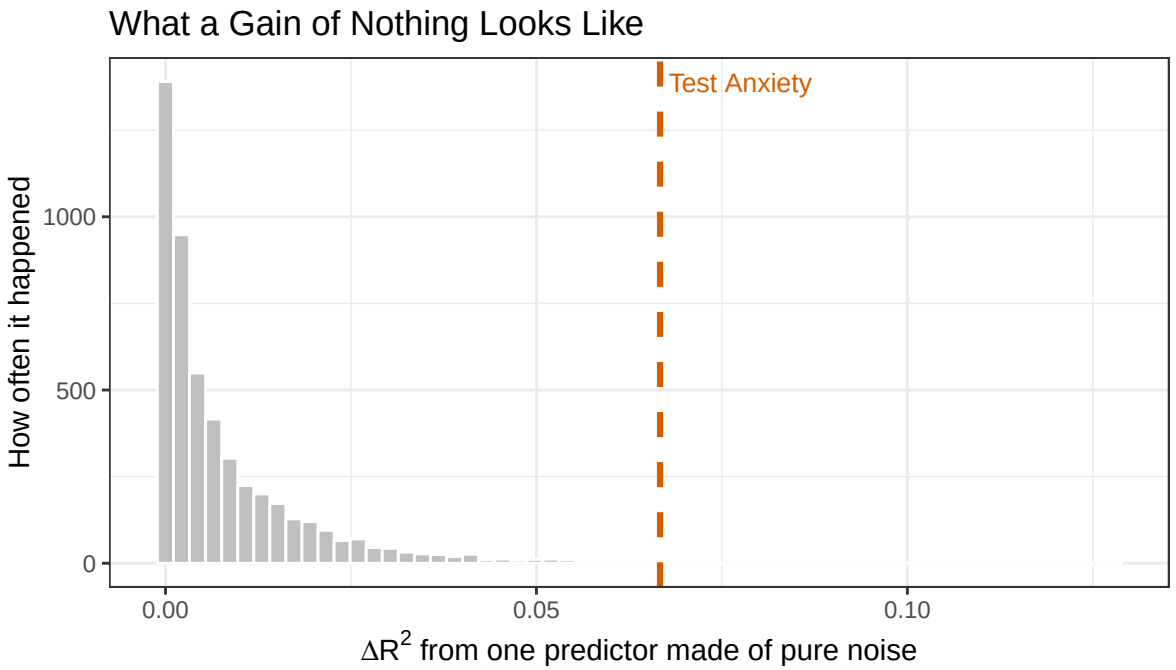


Figure 14.2: Five thousand ΔR^2 values, each from adding one column of random numbers to the Study Hours model. This is what a gain of nothing actually looks like. The dashed line is the gain Test Anxiety produced.

Not one of those five thousand variables knows anything about exam scores. The average one still bought 0.8% of the variance, and the luckiest of the five thousand bought 12.8%. Nothing is never worth exactly nothing.

This is why "my R^2 went up" is not a finding. The gain is always positive. The question is whether your gain lands further right than noise can comfortably reach, and the histogram tells you where that is: 3.0% of variance is the point only one noise predictor in twenty ever beats.

Our observed ΔR^2 of 0.067 is the dashed line. Out of five thousand noise runs, 18 of them beat it, which comes to $p = 0.0036$.

Hold on to that number. We come back for it shortly.

14.7.2 The Test That Skips the Five Thousand Models

Simulating five thousand models every time you want to check one predictor would make for long afternoons. The **F-test for model comparison** gets you the same answer from arithmetic. It tests the null hypothesis that the added predictor(s) explain *zero additional variance in the population*, meaning the improvement in R^2 is nothing more than noise.

$$H_0 : \Delta R^2 = 0 \text{ (Test Anxiety adds nothing beyond Study Hours)}$$

$$H_1 : \Delta R^2 > 0 \text{ (Test Anxiety adds predictive power)}$$

The F-statistic for this test is computed as:

$$F = \frac{\Delta R^2 / \Delta df_{\text{model}}}{(1 - R^2_{\text{full}}) / df_{\text{residual, full}}}$$

Where:

- ΔR^2 = the improvement in R^2 from the simpler to the more complex model
- Δdf_{model} = the number of new predictors added (how many degrees of freedom were "used up" by the new predictors)
- R^2_{full} = the R^2 of the more complex (full) model
- $df_{\text{residual, full}}$ = residual degrees of freedom in the full model ($n - k - 1$, where k is the total number of predictors)

The numerator captures how much new variance the added predictor explains *per additional parameter*. The denominator captures the average unexplained variance per residual degree of freedom, essentially the baseline "noise level" of the full model. If the ratio is large, the new predictor is explaining more variance than you would expect from noise. If the ratio is close to 1, the new predictor is not doing much better than chance.

You do not need to compute this by hand. In R, the `anova()` function computes it for you when you feed it two nested models. This is one of the rare occasions where statistics asks for less than you were braced for.

14.7.3 Running the Comparison in R

```
anova(Model.1, Model.3)
```

```
Analysis of Variance Table

Model 1: Exam ~ StudyHours
Model 2: Exam ~ StudyHours + Anxiety
  Res.Df   RSS Df Sum of Sq      F   Pr(>F)
1     98 11354
2     97 10404  1     950.1 8.8581 0.003684 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Let's read this output carefully. The `anova()` output when comparing two models shows:

- **RSS** (Residual Sum of Squares): the total unexplained variance in each model. A lower RSS means a better-fitting model.
- **Df**: the residual degrees of freedom for each model, and the change in degrees of freedom between models.
- **F**: the F-statistic for the comparison.
- **Pr(>F)**: the p-value for that F-statistic.

The key row to focus on is the second one. That is where the comparison lives. The F-statistic tells you whether the drop in residual sum of squares (i.e., the gain in explained variance) between Model 1 and Model 3 is statistically significant.

Now collect the number you were holding. The F-test reports $p = 0.0037$. The five thousand noise models reported 0.0036. Those are not two methods that happen to agree. They are the same question, and the histogram is what the formula is a shortcut *for*. The formula just does not make you wait.

14.7.4 Remember: This *F* Is a *t* You Have Already Met

As we reviewed before, when you add exactly one predictor, the model-comparison F is the square of that predictor's t in the full model. Not approximately. The same number, to every decimal R will print.

```
t_anxiety <- summary(Model.3)$coefficients["Anxiety", "t value"]

c(t_squared    = t_anxiety^2,
  F_from_anova = anova(Model.1, Model.3)$F[2])
```

```
   t_squared F_from_anova
    8.858143     8.858143
```

Two tests, one fact. The t-test asks whether Test Anxiety's slope differs from zero once Study Hours is in the model. The F-test asks whether adding Test Anxiety improves fit once Study Hours is in the model. Those turn out to be the same question in two accents.

This stops being true the moment you add two or more predictors in a single step, because then the F is testing the whole block at once and there is no single t to square.

14.7.5 What to Look For

When you run a hierarchical model comparison, here is what you check:

1. Is the *F*-test significant? If $p < .05$, the added predictor(s) explain a statistically significant amount of additional variance. The improvement in model fit is unlikely to be due to chance. In our case:

```
F( 1 , 97 ) = 8.86
```

```
p = 0.0037
```

```
Delta R-squared = 0.067
```

The F-test is statistically significant ($F(1, 97) = 8.86$, $p = 0.0037$). This tells us that adding Test Anxiety to a model that already contains Study Hours produces a statistically significant improvement in fit. The 6.7% additional variance explained is more than we would expect from chance.

2. How large is ΔR^2? Statistical significance tells you the improvement is real, but it does not tell you how *big* it is. Always look at ΔR^2 to judge practical importance:

Our $\Delta R^2 = 0.067$, which means Test Anxiety explains an additional 6.7% of the observed variance in Exam Score above and beyond Study Hours. There are no universal small, medium, and large labels for raw ΔR^2. Whether this improvement matters depends on the outcome, measurement quality, theory, cost, and consequences. A tiny improvement

in predicting a rare medical crisis may matter enormously; the same improvement in predicting which student will choose the blue pen may not deserve a parade. Report the number and interpret it in context instead of sending it to the Small/Medium/Large Sorting Hat.

3. What do the individual coefficients look like in the final model? After confirming that the model improved, you examine the full model to interpret the individual slopes. The conclusion is only complete when you can say both *that* the model improved and *how* each predictor contributes.

14.8 Interpreting the Full Model

Now that we have confirmed the model improved significantly, let's look at the final model in full:

```
summary(Model.3)
```

```
Call:
lm(formula = Exam ~ StudyHours + Anxiety, data = dat)

Residuals:
     Min       1Q   Median       3Q      Max
-24.3420  -7.8649  -0.3942   7.4378  22.0359

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  80.2059     7.2176  11.112  < 2e-16 ***
StudyHours    1.4149     0.3693   3.831 0.000226 ***
Anxiety      -0.3306     0.1111  -2.976 0.003684 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 10.36 on 97 degrees of freedom
Multiple R-squared:   0.27, Adjusted R-squared:  0.2549
F-statistic: 17.94 on 2 and 97 DF,  p-value: 2.351e-07
```

Reading this together with the hierarchical comparison gives us a complete picture:

- **Study Hours** (b = 1.41): Each additional hour of studying per week is associated with a 1.41-point increase in exam score, controlling for anxiety. This is statistically significant.
- **Test Anxiety** (b = -0.33): Each one-point increase in anxiety is associated with a 0.33-point *decrease* in exam score, controlling for study hours. This is also statistically significant, and it is the predictor whose incremental value we just confirmed with the F-test.

The overall model $R^2 = 0.27$: together, Study Hours and Test Anxiety explain 27% of the variance in exam performance.

14.9 Visualizing the Model Comparison

It helps to see the R^2 values side by side. Here is a simple summary of where we are:

```
cat("Model 1 (Study Hours only):       R-squared =", round(r2_m1,  3),
    " | Adjusted R-squared =", round(summary(Model.1)$adj.r.squared, 3), "\n")
```

```
Model 1 (Study Hours only):       R-squared = 0.203  | Adjusted R-squared = 0.195
```

```
cat("Model 3 (Study Hours + Anxiety):  R-squared =", round(r2_m3,  3),
    " | Adjusted R-squared =", round(summary(Model.3)$adj.r.squared, 3), "\n")
```

```
Model 3 (Study Hours + Anxiety):  R-squared = 0.27  | Adjusted R-squared = 0.255
```

```
cat("Change: Delta R-squared =", round(r2_m3 - r2_m1, 3), "\n")
```

```
Change: Delta R-squared = 0.067
```

Notice that the **adjusted** R^2 also increased from 0.195 to 0.255. Recall that adjusted R^2 penalizes models for adding predictors. When adjusted R^2 goes up, it means the new predictor is earning its keep: the improvement in fit outweighs the cost of the extra parameter. This is consistent with the significant F-test.

We can also visualize this as a stacked bar showing how the explained variance is distributed:

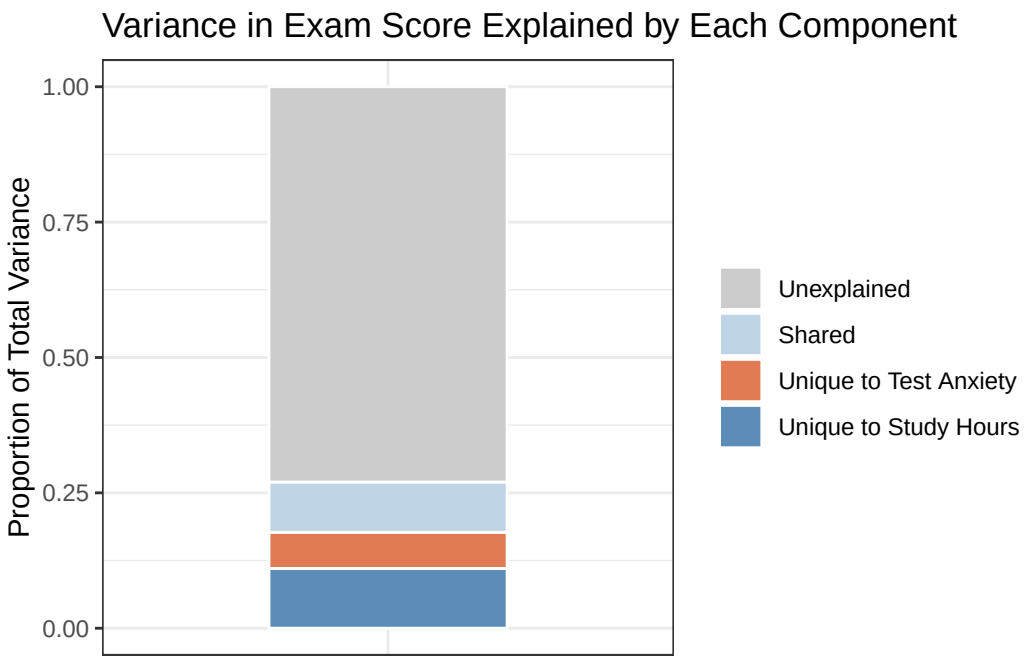


Figure 14.3: Total variance in Exam Score, split into four pieces: the part only Study Hours explains, the part only Test Anxiety explains (the orange band, which is the ΔR^2 we just tested), the part they share, and the part the model never reaches.

The orange segment is Test Anxiety's unique contribution ($\Delta R^2 = 0.067$), and it is what the F-test just confirmed is statistically significant. It represents the additional explanatory power Test Anxiety brings to a model that already contains Study Hours. Adding it was worth it.

14.10 The Logic for Grandma

Hierarchical regression is built on a simple idea: **if a predictor matters, it should explain variance in the outcome that no other predictor in the model can already account for.** If it does not, it is just explaining the same territory as something already in the model, and we should not clutter up our model with redundant predictors.

Here is the sequence of reasoning:

1. **Start with what you already know.** We know Study Hours predicts exam scores. Build a model with it first.
2. **Ask: does the new predictor add something?** We want to know if Test Anxiety explains variance in exam scores *above and beyond* Study Hours. The ΔR^2 tells us the size of the gain.
3. **Test whether the gain is real.** The F-test evaluates whether that ΔR^2 could plausibly have arisen by chance. A significant F tells us: no, the improvement is larger than chance fluctuations in R^2 would predict.
4. **Interpret the final model.** If the gain is significant, include the new predictor and interpret its slope. If the gain is not significant, you have evidence that the new predictor adds nothing to the model (at least, not above and beyond what is already there).

This four-step logic applies no matter how many predictors you are adding. You could be testing whether a single predictor improves fit, or whether a whole *block* of predictors (say, all demographic variables, or all anxiety-related measures) improves fit over a baseline model. The mechanics are the same.

14.11 Blocks of Predictors

One of the most powerful applications of hierarchical regression is adding **blocks** of predictors, groups of theoretically related variables, all at once. For example, imagine we had three anxiety-related measures instead of one. We could build:

- **Block 1:** Study Hours
- **Block 2:** Test Anxiety + Generalized Anxiety + Worry (three anxiety measures added simultaneously)

The F-test for Block 2 would then ask: do these three anxiety measures *together* explain additional variance in exam scores, above and beyond Study Hours? The degrees of freedom for the F-test would be 3 (one per added predictor), and the test is still a single, omnibus comparison. No need to test each anxiety measure separately at this stage.

This is particularly useful when you want to:

- **Control for covariates** (e.g., age, gender) by entering them in Block 1, then test your predictors of interest in Block 2
- **Test a theoretical construct** measured by multiple items, rather than testing each item individually
- **Evaluate competing theories** by building them into successive models and checking which block produces the largest gain in R^2

It is also the tool that would have rescued me from the graph at the top of this chapter. Those ten columns of random numbers are a block. Enter them as one and test them properly:

```
Model.Junk <- lm(Exam ~ ., data = junk_dat)   # Study Hours + ten columns of noise

anova(Model.1, Model.Junk)
```

```
Analysis of Variance Table

Model 1: Exam ~ StudyHours
Model 2: Exam ~ StudyHours + Noise1 + Noise2 + Noise3 + Noise4 + Noise5 +
    Noise6 + Noise7 + Noise8 + Noise9 + Noise10
  Res.Df   RSS Df Sum of Sq      F Pr(>F)
1     98 11354
2     88 10450 10    904.08 0.7613 0.6651
```

Ten predictors, 0.063 of extra R^2, and $p = 0.665$. Compare that with Test Anxiety: one predictor, a gain of almost exactly the same size, $p = 0.0037$. The staircase went up either way. Only one of them survived being asked to justify itself.

14.12 What Not to Do: The Danger of "Fishing"

Hierarchical regression is a tool for **theory-driven** model building. The order in which you enter predictors should be decided *before* you look at the results, based on your understanding of the literature and your hypotheses.

It is tempting to try many different orderings and report the one that looks best. This is a form of "fishing," running multiple comparisons in the hope of finding a significant result, and it inflates the Type I error rate. If you run 20 model comparisons, you should expect one of them to be "significant" by chance at the $p < .05$ threshold, even if nothing is real.

You have already seen the machinery that makes this work. That histogram of noise gains had a right tail, and the luckiest of the five thousand junk predictors bought 12.8% of the variance, comfortably more than our real predictor did. Reordering your blocks until something turns significant is not analysis. It is drawing from that tail until you like what you get.

The safe approach: decide on your model sequence based on theory, and report all models you tested, not just the one that "worked."

14.13 APA Write-Up

Here is how to report a hierarchical regression analysis in APA format:

Prior to the main analysis, zero-order correlations were examined. Study hours positively correlated with exam performance ($r = .45$, $p < .001$), and test anxiety negatively correlated with exam performance ($r = -.40$, $p < .001$). The two predictors were also significantly correlated with each other ($r = -.35$, $p < .001$).

A hierarchical multiple regression was conducted to examine whether test anxiety explained variance in exam performance above and beyond study hours. Study hours was entered in Step 1, and test anxiety was added in Step 2.

In Step 1, study hours significantly predicted exam performance, $b = 1.8$, $SE = 0.36$, $t(98) = 5$, $p < .001$, $R^2 = 0.203$, adjusted $R^2 = 0.195$.

In Step 2, the addition of test anxiety produced a statistically significant improvement in model fit, $\Delta R^2 = 0.067$, $F(1, 97) = 8.86$, $p = 0.0037$. In the final model, both study hours ($b = 1.41$, $SE = 0.37$, $t(97) = 3.83$, $p < .001$) and test anxiety ($b = -0.33$, $SE = 0.11$, $t(97) = -2.98$, $p = 0.0037$) significantly predicted exam performance. The final model explained 27% of the variance in exam scores, $R^2 = 0.27$, adjusted $R^2 = 0.255$.

A few things to notice in this write-up:

- Report each *step* as a block, not just the final model. Readers need to see what each step added.
- Report ΔR^2 and the F-test for the change. These are the key hierarchical results.
- After reporting the model comparison, also report the individual slopes from the final model.
- Use the phrase **"above and beyond"** or **"produced a statistically significant improvement in model fit"** to signal that a hierarchical comparison is being reported.

> **!** The Order Must Come from the Question
>
> ΔR^2 belongs to a particular sequence of models. Enter predictors in an order justified by theory or design, not the order that made the Probability Gods finally hand you an asterisk.

14.14 The Short Story

- **Raw R^2 always goes up.** Ten columns of random numbers raised it about as much as the real predictor did. Every test in this chapter exists because of that one fact.
- **Nested** means the simpler model's predictors are a subset of the bigger model's. Only nested models can be compared this way.
- ΔR^2 is the variance the new predictor explains that nothing already in the model could reach. Region b. The next chapter calls the same quantity sr^2.
- **The *F*-test** asks whether that gain beats sampling noise. Add exactly one predictor and it equals the square of that predictor's t in the full model.
- **Adjusted R^2 is the sanity check.** Raw R^2 always goes up. Adjusted R^2 goes up only when the new predictor earned its seat.
- **The order comes from theory,** decided before you look at the output. Reordering until something turns significant is fishing, and it inflates your Type I error rate.
- **Report each step,** then ΔR^2, then the F for the change, then the slopes from the final model. In that order.

15 Statistical Control: What Are We Removing?

⚠ Advanced Topic: Your Brain May File a Complaint

This is difficult material. Graduate students routinely stare at partial and semi-partial correlations as if the correlations have personally betrayed them. If this chapter requires two readings, that does not mean you are bad at statistics. It means we are removing different pieces of relationships and then giving the leftovers nearly identical names.

Keep asking three questions:

1. What variable are we controlling?
2. Which variable or variables are we removing it from?
3. What does the resulting number mean?

15.1 Statistical Control Is Not a Statistics Jail

In Multiple Regression: Statistical Control, we predicted **Exam Score** from **Study Hours** and **Test Anxiety**. Study Hours and Anxiety were related to one another, which meant their relationships with Exam Score overlapped.

Multiple regression gave us a Study Hours slope while *controlling for* Anxiety. That phrase sounds reassuringly official. It also hides nearly all the interesting work.

We did not put Anxiety in a tiny statistical jail. We did not force every student to experience precisely 50 units of Anxiety. We used a model to remove the **linear association** Anxiety had with another variable and then studied what remained.

This chapter takes that process apart. We will use the exact same example and simulated data as the multiple-regression chapter, because changing the variables while explaining an already difficult idea would be rude.

Our main question will remain:

> Do Study Hours predict Exam Score after accounting for Test Anxiety?

15.2 The Same Exam Study

The expected correlations are:

Relationship	Correlation	Translation
Study Hours with Exam Score	+.45	Students who study more tend to score higher.
Test Anxiety with Exam Score	−.40	Students who are more anxious tend to score lower.
Study Hours with Test Anxiety	−.35	Students who study more tend to be less anxious.

That last relationship creates the control problem. When Study Hours predicts Exam Score, part of that association may travel through the fact that students who study more also tend to be less anxious.

```
library(MASS)

set.seed(42)

n   <- 100
rY1 <-  0.45  # Study Hours with Exam Score
rY2 <- -0.40  # Test Anxiety with Exam Score
r12 <- -0.35  # Study Hours with Test Anxiety

Sigma <- matrix(c(
   1,   rY1, rY2,
  rY1,   1,  r12,
  rY2,  r12,  1
), ncol = 3)

raw <- as.data.frame(
  mvrnorm(
    n,
    mu = c(0, 0, 0),
    Sigma = Sigma,
    empirical = TRUE
  )
)

colnames(raw) <- c("Exam", "StudyHours", "Anxiety")

dat <- data.frame(
  Exam       = round(raw$Exam * 12 + 75, 1),
  StudyHours = round(raw$StudyHours * 3 + 8, 1),
  Anxiety    = round(raw$Anxiety * 10 + 50, 1)
)
```

```
round(cor(dat[c("StudyHours", "Anxiety", "Exam")]), 2)
```

```
           StudyHours Anxiety  Exam
StudyHours       1.00   -0.35  0.45
Anxiety         -0.35    1.00 -0.40
Exam             0.45   -0.40  1.00
```

15.3 The Ballantine, Now Made Fresh

The circles below represent variance. The overlapping portions represent shared variance. This diagram is called a **Ballantine** because apparently three overlapping circles needed a name, and statisticians had already used "Venn diagram" for something fun at parties.

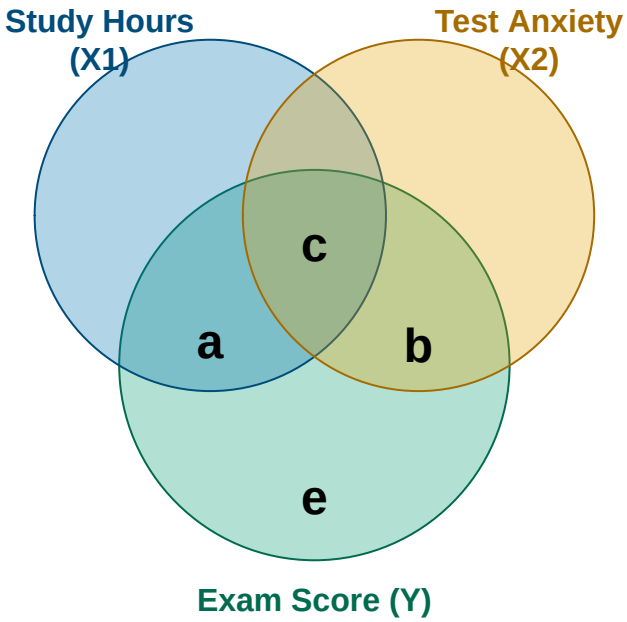


Figure 15.1: A Ballantine for Study Hours, Test Anxiety, and Exam Score. The areas are conceptual, not drawn to scale.

- **Region** a is Exam Score variance uniquely associated with Study Hours.
- **Region** b is Exam Score variance uniquely associated with Test Anxiety.
- **Region** c is Exam Score variance shared by both predictors. Neither predictor gets exclusive custody.
- **Region** e is Exam Score variance the full model does not explain.

The full regression model explains $a + b + c$. Our main target in this chapter is region a: what Study Hours contributes above and beyond Anxiety.

> **i** Circles Are a Metaphor
>
> The areas are not drawn to scale, variables are not actually circular, and residuals do not fall onto the floor when we remove them. The diagram helps us track which variance belongs in each calculation. Do not demand anatomical accuracy from it.

15.4 Residuals: The Leftovers

Control begins with a familiar regression. First, predict Study Hours from Anxiety:

```
study_from_anxiety <- lm(StudyHours ~ Anxiety, data = dat)
summary(study_from_anxiety)
```

```
Call:
lm(formula = StudyHours ~ Anxiety, data = dat)

Residuals:
    Min      1Q  Median      3Q     Max
-7.9379 -1.8870  0.1195  1.4007  8.9786

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 13.25031    1.45120   9.131 9.33e-15 ***
Anxiety     -0.10503    0.02847  -3.690 0.000369 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.833 on 98 degrees of freedom
Multiple R-squared:  0.122, Adjusted R-squared:  0.113
F-statistic: 13.61 on 1 and 98 DF,  p-value: 0.0003691
```

```
dat$ExpectedStudy <- fitted(study_from_anxiety)
dat$StudyResidual <- residuals(study_from_anxiety)
```

For every student:

$$\text{Study residual} = \text{Actual Study Hours} - \text{Expected Study Hours}$$

Suppose an entirely hypothetical student named Alex has an Anxiety score of 60, studies 10 hours, and earns a 78 on the exam. There is no connection between this fictional Alex and the author except the name, profession, and desperate wish that ten hours of studying always happened.

```
alex_anxiety <- 60
alex_actual_study <- 10
alex_actual_exam <- 78

alex_expected_study <- unname(
  predict(study_from_anxiety, newdata = data.frame(Anxiety = alex_anxiety))
)

alex_study_residual <- alex_actual_study - alex_expected_study

alex_table <- data.frame(
```

```
  Anxiety = alex_anxiety,
  Expected_Study = round(alex_expected_study, 1),
  Actual_Study = alex_actual_study,
  Study_Residual = round(alex_study_residual, 1)
)

knitr::kable(alex_table)
```

Anxiety	Expected_Study	Actual_Study	Study_Residual
60	6.9	10	3.1

Given an Anxiety score of 60, the model expected Alex to study about 6.9 hours. Alex studied 10 hours, so the Study Hours residual is about +3.1. Alex studied that many hours **more than expected given Anxiety**.

A positive residual means "more than expected." A negative residual means "less than expected." A residual of zero means the model finally got one exactly right and will be insufferable about it.

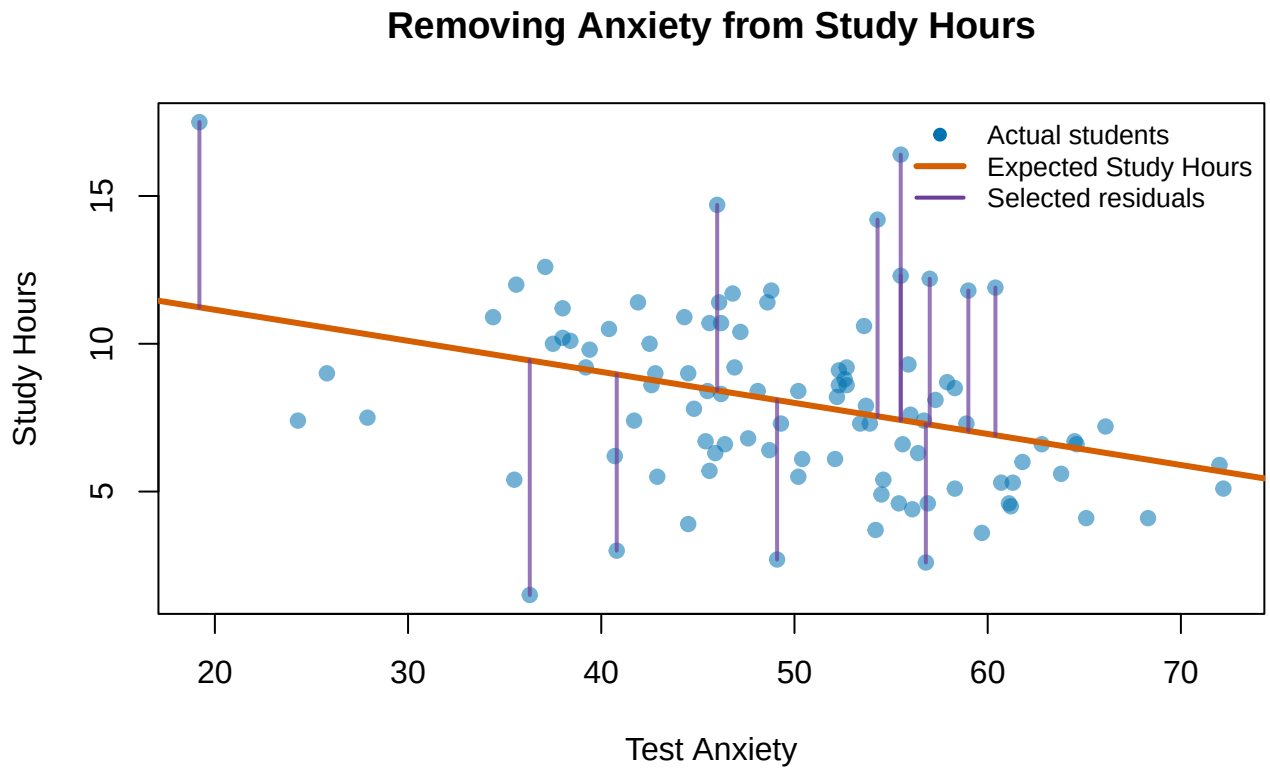


Figure 15.2: Study Hours predicted from Test Anxiety. Selected vertical lines show residuals: the distance between each student's actual and expected Study Hours.

Anxiety has now explained whatever linear association it can explain in Study Hours. The Study Hours residuals are the leftovers. Notice what happens if we correlate those leftovers with Anxiety:

```r
cor(dat$Anxiety, dat$StudyResidual)
```

```
[1] 3.217625e-17
```

It is essentially zero. That is not luck. Least-squares regression creates residuals that are uncorrelated with the predictors used to create them.

15.5 Semi-Partial Correlation: Remove Anxiety from One Side

We will learn **semi-partial correlation first** because it connects directly to multiple regression.

To find the semi-partial relationship between Study Hours and Exam Score while controlling for Anxiety:

1. Predict Study Hours from Anxiety.
2. Save the Study Hours residuals.
3. Leave Exam Score completely alone.
4. Correlate residualized Study Hours with the original Exam Score.

```r
sr_study <- cor(dat$Exam, dat$StudyResidual)
sr_study
```

```
[1] 0.3323602
```

The semi-partial correlation is $sr = 0.332$.

Its plain-language question is:

> Do students who study more than we would expect from their Anxiety tend to earn higher Exam Scores?

The answer is yes. More of the Study Hours that Anxiety cannot explain is associated with better Exam performance.

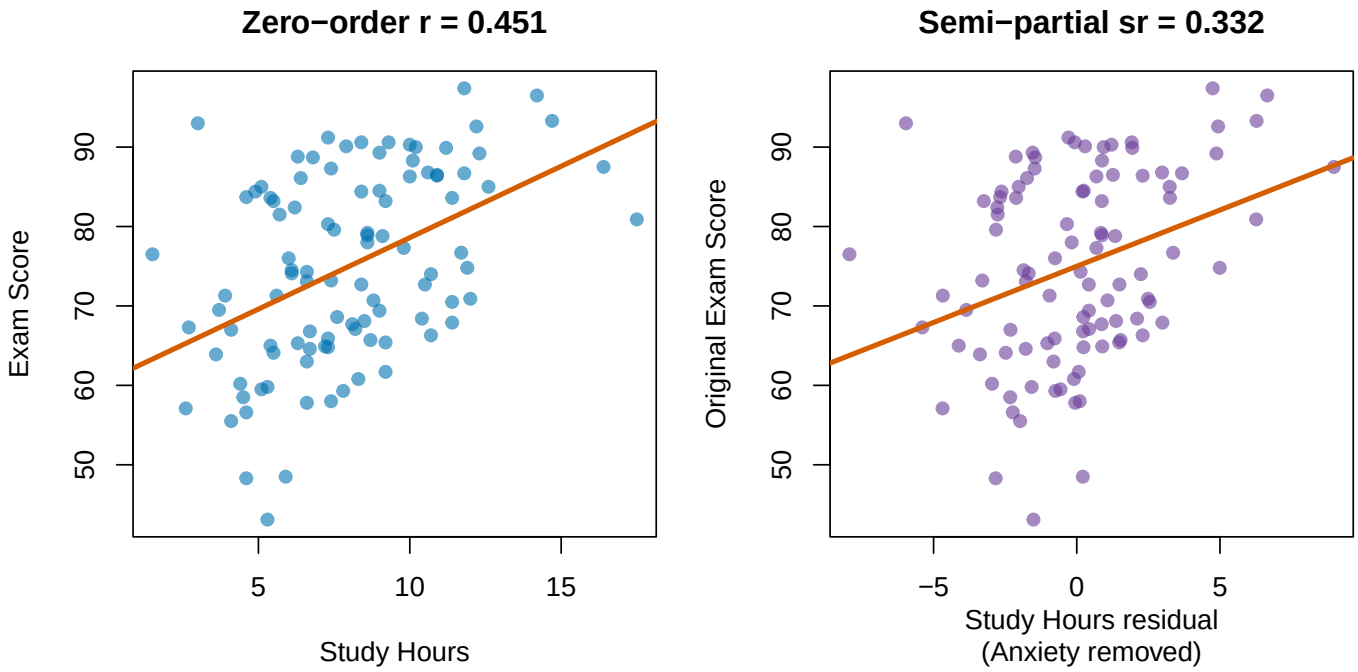


Figure 15.3: Left: the zero-order Study Hours–Exam Score relationship. Right: the semi-partial relationship after removing Anxiety from Study Hours only.

15.5.1 Why Squaring sr Matters

Squaring a semi-partial correlation gives the unique proportion of **total outcome variance** associated with that predictor:

```
sr2_study <- sr_study^2
sr2_study
```

```
[1] 0.1104633
```

Therefore, Study Hours uniquely accounts for about 11% of the total variance in Exam Score after the Anxiety-only model has explained everything it can, including the shared region c. In the Ballantine, the additional contribution is region a.

This also equals the change in R^2 when Study Hours is added to a model that already contains Anxiety.

```
anxiety_model <- lm(Exam ~ Anxiety, data = dat)
full_model <- lm(Exam ~ Anxiety + StudyHours, data = dat)

r2_anxiety <- summary(anxiety_model)$r.squared
r2_full <- summary(full_model)$r.squared
delta_r2_study <- r2_full - r2_anxiety

data.frame(
  Quantity = c(
    "R-squared: Anxiety only",
```

```
    "R-squared: Anxiety + Study Hours",
    "Change in R-squared",
    "Squared semi-partial correlation"
  ),
  Value = round(c(r2_anxiety, r2_full, delta_r2_study, sr2_study), 3)
)
```

```
                          Quantity Value
1          R-squared: Anxiety only  0.16
2 R-squared: Anxiety + Study Hours  0.27
3              Change in R-squared  0.11
4 Squared semi-partial correlation  0.11
```

The last two values match:

$$sr^2 = \Delta R^2$$

That equation is the main reason semi-partial correlation matters in multiple regression. It tells us exactly how much total explanatory power Study Hours adds **above and beyond** Anxiety.

We can formally compare the nested models with `anova()`:

```
anova(anxiety_model, full_model)
```

```
Analysis of Variance Table

Model 1: Exam ~ Anxiety
Model 2: Exam ~ Anxiety + StudyHours
  Res.Df   RSS Df Sum of Sq      F    Pr(>F)
1     98 11978
2     97 10404  1    1574.3 14.678 0.0002263 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The test asks whether adding Study Hours improves prediction enough that we should take the improvement seriously, rather than blaming the Probability Gods for a tiny accidental increase.

15.6 Partial Correlation: Remove Anxiety from Both Sides

Partial correlation takes one additional step. We remove Anxiety from **both** Study Hours and Exam Score:

1. Predict Study Hours from Anxiety and save the residuals.
2. Predict Exam Score from Anxiety and save those residuals too.
3. Correlate the two sets of residuals.

```r
exam_from_anxiety <- lm(Exam ~ Anxiety, data = dat)
dat$ExpectedExam <- fitted(exam_from_anxiety)
dat$ExamResidual <- residuals(exam_from_anxiety)

pr_study <- cor(dat$StudyResidual, dat$ExamResidual)
pr_study
```

```
[1] 0.362534
```

The partial correlation is $pr = 0.363$.

Its plain-language question is:

> Among the portions of Study Hours and Exam Score that Anxiety cannot explain, how strongly are the remaining portions related?

Return to Hypothetical Alex. Based on Anxiety alone, the model predicts an Exam Score of about 70.2. Alex actually earned a 78. Therefore, Alex scored about 7.8 points **higher than expected given Anxiety**.

Partial correlation asks whether students who study more than Anxiety predicts also score higher than Anxiety predicts.

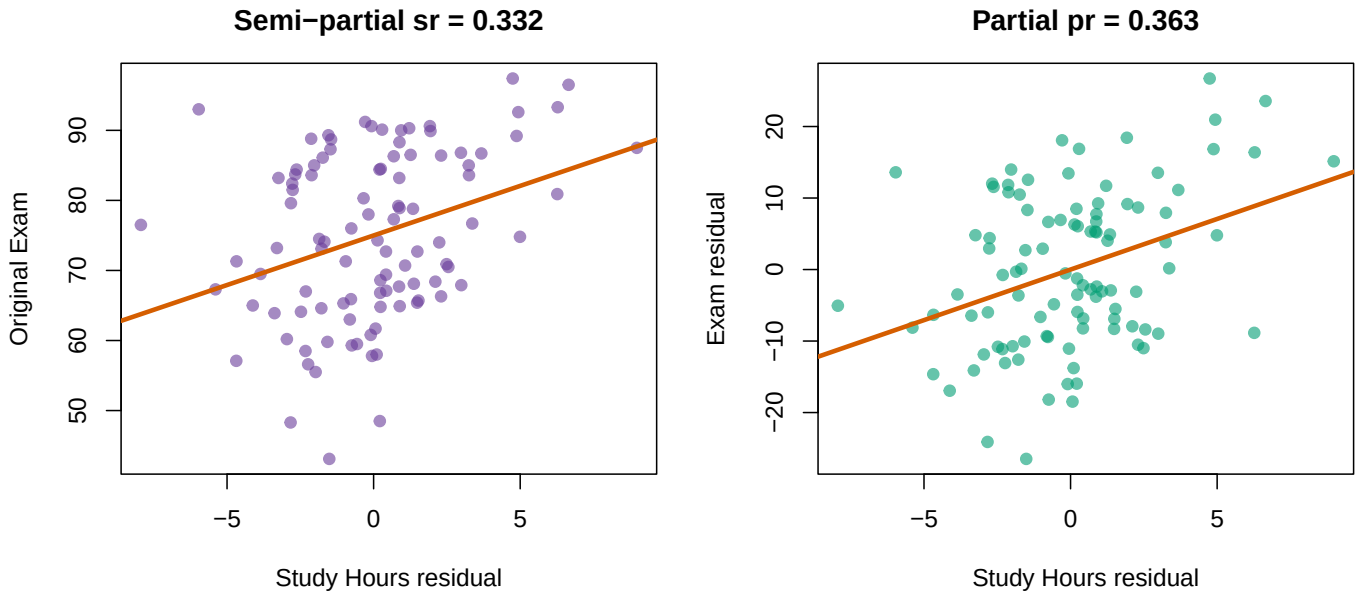


Figure 15.4: Semi-partial correlation leaves Exam Score untouched. Partial correlation removes Anxiety from both Study Hours and Exam Score.

15.6.1 Why Partial Correlation Is Larger

For these data:

```r
pr2_study <- pr_study^2
pr2_study
```

```
[1] 0.1314309
```

The squared partial correlation is about 13.1%. This does **not** mean Study Hours suddenly explains 13.1% of total Exam Score variance. Semi-partial correlation already told us that the unique contribution to total variance was 11%.

Partial correlation uses a smaller denominator. Anxiety has already removed the Exam Score variance it can explain. Of the Exam Score variance **remaining after Anxiety**, Study Hours is associated with about 13.1%.

In Ballantine language:

$$sr_1^2 = a$$

$$pr_1^2 = \frac{a}{a + e}$$

The numerator is the same unique Study Hours region. Partial correlation looks larger because it compares region a with only the Exam variance left after Anxiety is removed. It did not discover a stronger effect. It changed the denominator.

15.7 Semi-Partial Versus Partial

Quantity	Remove Anxiety from Study Hours?	Remove Anxiety from Exam Score?	What does the square mean?
Zero-order r	No	No	Total variance shared by Study Hours and Exam Score
Semi-partial sr	Yes	No	Unique Study Hours contribution to **total** Exam Score variance; ΔR^2
Partial pr	Yes	Yes	Study Hours association with Exam Score variance **remaining after Anxiety**

If you remember only one distinction, remember this:

> **Semi-partial leaves Y alone. Partial residualizes both sides.**

"Semi" does not mean the analysis tried less hard. It means the control variable was removed from only one member of the target relationship.

15.8 The Moderately Mathy Version

The residual method is the method I want you to understand. The formulas show why the two correlations differ.

Let:

- r_{Y1} be the Exam Score–Study Hours correlation,
- r_{Y2} be the Exam Score–Anxiety correlation, and
- r_{12} be the Study Hours–Anxiety correlation.

The Study Hours semi-partial correlation controlling for Anxiety is:

$$sr_{Y(1.2)} = \frac{r_{Y1} - r_{Y2}r_{12}}{\sqrt{1 - r_{12}^2}}$$

The partial correlation is:

$$pr_{Y1.2} = \frac{r_{Y1} - r_{Y2}r_{12}}{\sqrt{(1 - r_{Y2}^2)(1 - r_{12}^2)}}$$

Notice that the numerators are identical. Both calculations isolate the same unique Study Hours–Exam Score relationship. The partial-correlation denominator also removes the outcome variance associated with Anxiety.

```
r_y1 <- cor(dat$Exam, dat$StudyHours)
r_y2 <- cor(dat$Exam, dat$Anxiety)
r_12 <- cor(dat$StudyHours, dat$Anxiety)

sr_formula <- (r_y1 - r_y2 * r_12) / sqrt(1 - r_12^2)

pr_formula <- (r_y1 - r_y2 * r_12) /
  sqrt((1 - r_y2^2) * (1 - r_12^2))

round(c(
  Residual_sr = sr_study,
  Formula_sr = sr_formula,
  Residual_pr = pr_study,
  Formula_pr = pr_formula
), 3)
```

```
Residual_sr  Formula_sr Residual_pr  Formula_pr
      0.332       0.332       0.363       0.363
```

The residual and formula routes agree. Statistics has briefly behaved itself.

15.9 How This Becomes the Multiple-Regression Slope

Here is the important connection. Compare three slopes:

1. The Study Hours slope from the full multiple regression.
2. The slope predicting original Exam Score from residualized Study Hours.
3. The slope predicting residualized Exam Score from residualized Study Hours.

```
semipartial_lm <- lm(Exam ~ StudyResidual, data = dat)
partial_lm <- lm(ExamResidual ~ StudyResidual, data = dat)

round(c(
  Full_multiple_regression = coef(full_model)["StudyHours"],
  Original_Y_on_residualized_X = coef(semipartial_lm)["StudyResidual"],
```

```
  Residualized_Y_on_residualized_X = coef(partial_lm)["StudyResidual"]
), 6)
```

```
           Full_multiple_regression.StudyHours
                                      1.414884
    Original_Y_on_residualized_X.StudyResidual
                                      1.414884
Residualized_Y_on_residualized_X.StudyResidual
                                      1.414884
```

All three slopes equal about 1.41 Exam Score points per Study Hour.

This is not a coincidence. The Study Hours coefficient in multiple regression is built from the Study Hours variation that Anxiety cannot linearly explain. This result is sometimes called the **Frisch–Waugh–Lovell theorem**, which is an impressive name for "residualize the relevant pieces and you recover the same slope."

One caution before someone reports the wrong t-value. The **slopes** match. The standard errors and t-tests from these shortcut regressions do not, because those regressions do not know that Anxiety already spent a degree of freedom. Report your inference from the full model. Use the residualized regressions to *understand* where that slope came from.

The practical interpretation is:

> Among students at the same Anxiety level, one additional Study Hour is associated with about 1.41 additional Exam Score points.

That is a conditional association. It is not automatically a causal effect.

15.10 Where Standardized Beta Fits

Now we have several numbers describing the Study Hours relationship. This is where students reasonably begin asking whether statistics could have used fewer symbols.

```
dat_z <- as.data.frame(scale(dat[c("Exam", "StudyHours", "Anxiety")]))
standardized_model <- lm(Exam ~ StudyHours + Anxiety, data = dat_z)
beta_study <- coef(standardized_model)["StudyHours"]

comparison <- data.frame(
  Quantity = c(
    "Zero-order correlation (r)",
    "Standardized regression slope (beta)",
    "Semi-partial correlation (sr)",
    "Partial correlation (pr)",
    "Squared semi-partial (sr-squared)",
    "Squared partial (pr-squared)"
  ),
  Value = round(c(
    r_y1,
    beta_study,
    sr_study,
    pr_study,
```

```
    sr2_study,
    pr2_study
  ), 3),
  Meaning = c(
    "Uncontrolled Study Hours-Exam association",
    "Conditional slope in standard-deviation units",
    "Unique Study association scaled by total Exam variance",
    "Unique Study association scaled by remaining Exam variance",
    "Unique contribution to total Exam variance; change in R-squared",
    "Proportion of residual Exam variance associated with Study"
  )
)

knitr::kable(comparison)
```

Quantity	Value	Meaning
Zero-order correlation (r)	0.451	Uncontrolled Study Hours-Exam association
Standardized regression slope (beta)	0.355	Conditional slope in standard-deviation units
Semi-partial correlation (sr)	0.332	Unique Study association scaled by total Exam variance
Partial correlation (pr)	0.363	Unique Study association scaled by remaining Exam variance
Squared semi-partial (sr-squared)	0.110	Unique contribution to total Exam variance; change in R-squared
Squared partial (pr-squared)	0.131	Proportion of residual Exam variance associated with Study

The standardized beta is $\beta = 0.355$. It tells us how many standard deviations Exam Score changes for a one-standard-deviation increase in Study Hours while Anxiety is held constant.

Beta, *sr*, and *pr* have the same sign here and test the same unique predictor contribution, but they are **not interchangeable**. They standardize the relationship using different variance. Never report one while calling it another because the decimals looked lonely.

15.11 `ppcor`: A Shortcut

For a strict correlation question involving one outcome, one target predictor, and a control variable, the `ppcor` package can verify our work:

```
library(ppcor)
# Semi-partial: remove Anxiety from the SECOND variable, StudyHours
spcor.test(
  x = dat$Exam,
  y = dat$StudyHours,
  z = dat$Anxiety
)

# Partial: remove Anxiety from both Exam and StudyHours
pcor.test(
  x = dat$Exam,
  y = dat$StudyHours,
```

```
  z = dat$Anxiety
)
```

The order matters for `spcor.test()` because the control variable is removed from the **second** variable. Order matters less conceptually for partial correlation because the control is removed from both target variables.

`ppcor` can accept more than one control variable, but it remains a specialized correlation shortcut. Real analyses quickly acquire several predictors, categorical variables, interactions, missing observations, model comparisons, diagnostics, and Reviewer 2. Use `lm()` as your default. It scales to those problems and shows exactly which model you estimated. Use `ppcor` as a quick check when the question is genuinely simple.

15.12 Statistical Control Is Not Experimental Control

> **!** Control Does Not Manufacture Causality
>
> We removed linear associations from numbers. We did **not** randomly assign Anxiety, repair measurement error, eliminate variables we forgot to measure, or reach backward through time.
>
> After controlling for Anxiety, the Study Hours coefficient describes the association between Study Hours and Exam Score conditional on Anxiety. Calling it "the true causal effect of studying" would require a research design and assumptions much stronger than this regression alone provides.

Adding a control variable does not guarantee that a slope will shrink. Depending on the correlation pattern, a slope can:

- shrink,
- grow,
- reverse direction,
- become more precise,
- become less precise, or
- stare into the abyss and become a suppression effect.

Control variables must be selected using theory and research design. Throwing every available column into a regression is not rigor. It is making the software eat the entire refrigerator because you could not decide what to cook.

15.13 Four Questions Before You Escape

15.13.1 1. Caffeine, Sleep, and Exam Scores

You want to know how much **total Exam Score variance** Caffeine uniquely explains after Sleep Hours are already in the model. Do you want a partial or semi-partial correlation?

15.13.2 2. The Entirely Ethical Chainsaw Study

You want the relationship between Fear and Heart Rate after removing Running Speed from **both** variables. Participants are not actually chased with chainsaws because the IRB has once again ruined science. Partial or semi-partial?

15.13.3 3. Reviewer 2 Predicts Rejection

You want to know how much R^2 increases when Formatting Errors are added after Manuscript Length. Which squared correlation should equal that change in R^2?

15.13.4 4. The Causal Caffeine Disaster

You observe that Caffeine predicts Exam Score after controlling for Sleep. May you conclude that forcing students to drink six espressos will improve their grades?

Answers, If You Have Suffered Enough

1. **Semi-partial.** You want Caffeine's unique contribution to total Exam Score variance.
2. **Partial.** Running Speed is removed from both Fear and Heart Rate.
3. **Squared semi-partial correlation.** $sr^2 = \Delta R^2$ when Formatting Errors are added last.
4. **Absolutely not.** Statistical control does not establish causality, and six espressos may merely establish a new relationship with campus medical services.

15.14 The Short Story

- **Residual**: what remains after subtracting a model's predicted value from the observed value.
- **Semi-partial correlation**: remove the control variable from one target variable; sr^2 is the unique addition to total R^2.
- **Partial correlation**: remove the control variable from both target variables; pr^2 describes the relationship within the outcome variance that remains.
- **Multiple-regression slope**: can be recovered by predicting Y from the residualized target predictor.
- **Standardized beta**: the controlled regression slope in standard-deviation units.
- **Statistical control**: adjustment for measured associations, not a tiny machine that produces causal truth.

If you can identify what was removed, what it was removed from, and what denominator remains, you understand statistical control. That places you ahead of many graduate students and at least several published papers.

16 Continuous-by-Categorical Interactions: When the Effect of One Variable Depends on Another

16.1 Two Slopes, and Nothing That Compares Them

Here is a study you could run tomorrow. Closely follow 200 first-semester students wherever they go and video record all their social interactions for an entire semester, while you yell "*IGNORE ME*" every 30 seconds to ensure they know not to have their behaviors affected by you observing them 24/7 with a camera. Next, have a trained panel of clinical psychologists review all their interactions and rate how warm and agreeable each one is on a 0 to 10 scale. Since we know what they are doing at all times, we would also note whether they actually went out to parties and social events, or stayed home all semester. At the end of the semester we ask each victim, I mean each student who agreed to participate and signed all the IRB forms and waived all rights to their videos, how close a friendship they formed with one particular person they met in week one of the study, on a 0 to 100 "feelings" thermometer.

Below is the simulation of that study.

```
set.seed(42)
n <- 200
X <- runif(n, 0, 10)
Z <- rbinom(n, 1, 0.5)
Y <- 0.05 * X + 0 * Z + 5 * X * Z + 15 + rnorm(n, sd = 7)
Friend.Data <- data.frame(BeingNice = X, GoingOut = Z, Closeness = Y)
Friend.Data$GoingOut_ <- factor(Friend.Data$GoingOut,
                                levels = c(0, 1),
                                labels = c("Stay in", "Go out"))
```

Next, you then split the students into the ones who went out and the ones who stayed in, and run an ordinary regression in each half predicting closeness from niceness. Two groups, two regressions, nothing you have not already done a dozen times.

Stay in People:

```
Stay.In <- subset(Friend.Data, GoingOut_ == "Stay in")
coef(summary(lm(Closeness ~ BeingNice, data = Stay.In)))
```

```
             Estimate Std. Error    t value     Pr(>|t|)
(Intercept) 14.6930979  1.2346357 11.9007558 1.017885e-21
BeingNice    0.1997669  0.2107005  0.9481084 3.450798e-01
```

Go out People:

```r
Go.Out  <- subset(Friend.Data, GoingOut_ == "Go out")
coef(summary(lm(Closeness ~ BeingNice, data = Go.Out)))
```

```
             Estimate Std. Error   t value     Pr(>|t|)
(Intercept) 15.354310   1.550136  9.905138 1.149587e-15
BeingNice    4.904171   0.252405 19.429767 1.983002e-32
```

Read the two slopes. Among students who stayed in, each point of niceness buys 0.2 points of closeness, which is nothing, and the p-value of 0.35 agrees that it is nothing. Among students who went out, each point of niceness buys 4.9 points, and its p-value has run off the bottom of the printout. Same trait, same outcome, same semester, same campus. Being nice appears to be worth something to one group and nothing at all to the other.

Great. Being nice only pays off if you leave the house. Write it up, submit it, wait.

Now defend it. A reviewer asks the one question anybody would ask: are those two slopes *actually* different, or did two numbers just bounce apart the way numbers sometimes do by chance? Now, go find that test in the output above. I will wait...

What, it is not there? Weird. You have two intercepts, two slopes, four standard errors and four p-values, and every single one of them is a statement about one group considered entirely on its own. You drew an inference based on two separate tables that have nothing to do with each other. You inferred, because the stay-in slope was smaller than the go-out slope, that being nice clearly does more for the people who go out. The comparison the whole study was built to make was never performed, and no amount of staring at `p = .35` and `p < .001` will conjure it, because a small p-value next to a large one is not a test of the difference between them. *You cannot test a difference with two analyses that have never met.*

That missing test is what this chapter is for. The tool is called an **interaction**, and by the end of it you will have these exact two slopes back, 0.2 and 4.9, living inside one model that also hands you a significance test for the gap between them.

16.2 So Far Everything Was Additive

So far in this book, you have learned to build regression models where each predictor has its own independent, additive effect on the outcome. Study hours helps exam scores. Test anxiety hurts them. Each predictor does its job, and we can interpret the effects one at a time.

But human behavior is rarely that tidy. In the real world, the effect of one variable often *changes* depending on the value of another variable. A new medication might be effective for women but not men. Praise for effort might motivate some students while leaving others cold. Practice might matter enormously for novices but very little for experts.

These situations, where the effect of one predictor **depends on** another predictor, are called **interactions**, or equivalently, **moderation effects**. In this chapter, you will learn to detect, estimate, visualize, and interpret an interaction between a continuous variable and a categorical one.

Before we touch any data, we need to build the concept carefully. That is where we will start.

16.3 The Friendship Study

You have probably heard the expression "right place, right time." It turns out that this is not just a saying; it is backed by social psychological research. A classic study by Back, Schmukle, and Egloff (2008) found that the friendships people form in university are driven, in large part, by seating proximity. Literally, the people you end up sitting next to on the first day of class are the people most likely to become your close friends by the end of the semester. Chance determines who you sit near. *Geography becomes destiny.*

You decide to replicate and extend this work. You follow 200 first-semester students throughout their first semester, recording how close of a friendship they formed with one particular person (measured on a 0–100 feelings thermometer, where 0 = complete strangers and 100 = best friends). You also measure two things that might predict the quality of that friendship:

- **Being Nice:** each person is rated on their general agreeableness and warmth by a trained panel, on a scale from 0 to 10. This is a continuous variable.
- **Going Out:** whether the student regularly attended social events and parties throughout the semester. This is a binary variable: *Stay in* (0) vs. *Go out* (1).

The intuition behind your study is this: being a warm, friendly person should help you make a close friend. But being nice in your apartment, alone, does not do much for your social life. To turn niceness into friendship, you have to actually be around people. **The effect of being nice might depend on whether you go out.**

That is your hypothesis. And testing it is the focus of this chapter.

> **i** Two IVs, One DV, and Nobody Measured Twice
>
> Worth naming the parts before we model them, because from here on there is more than one of some of them.
>
> The **DV** (dependent variable, the outcome) is friendship closeness, 0 to 100. There is exactly one.
>
> The **IVs** (independent variables, the predictors) are Being Nice and Going Out. Two of them. That is what makes an interaction possible in the first place: you need two IVs before you can ask whether the effect of one depends on the other.
>
> Both IVs are **between subjects**. Each student contributes one row, sits in one Going Out group, and is measured once. Nobody appears twice. That is worth saying out loud because it is not always true, and when the same person is measured repeatedly the model has to change to account for it. That is a later chapter.
>
> One more thing, and it matters more than it looks. **Regression does not care whether an IV was randomly assigned or merely observed.** The arithmetic is identical either way. We did not assign anybody to be nice, and we certainly did not assign anybody to go to parties; we watched and wrote it down. So this is a quasi-experiment, the math will run exactly as if it were not, and every causal word you are tempted to write in the discussion section is on you, not on the model.

Back, M. D., Schmukle, S. C., & Egloff, B. (2008). Becoming friends by chance. *Psychological Science, 19*(5), 439–440.

16.4 What Is an Interaction? The Concept Before the Math

16.4.1 "It Depends" as a Scientific Statement

In everyday conversation, "it depends" can sound like an evasion, a way of refusing to commit to an answer. In statistics, it is a precise and testable claim. When we say "the effect of X on Y depends on Z," we mean something specific: **the slope of X on Y is not the same across different values of Z.** That is an interaction.

Let's make this concrete. There are two fundamentally different ways the world could work in our friendship study.

Possibility 1: Being nice always helps (no interaction). Whether you stay in or go out, being a nicer person leads to a closer friendship. The social opportunities provided by going out matter too: going out raises closeness overall, but the *benefit of being nice* is the same either way.

Possibility 2: Being nice only pays off if you go out (interaction). For students who stay in all semester, niceness makes little difference; they simply do not have enough social contact for their personality to matter. For students who go out, however, being nice is powerful: it turns those social opportunities into genuine close friendships.

These two possibilities predict completely different patterns in the data. Before we analyze a single number, we can sketch out what each would look like.

16.4.2 Visual Hypotheses: Sketching What We Expect to See

One of the most useful habits in data analysis is to draw a picture of what you *expect* your results to look like before you run a single model. This keeps your hypotheses honest: you cannot accidentally convince yourself you predicted something after the fact. It also gives you an intuition for what you are looking for.

In our study, we want to predict *Friendship Closeness* (Y) from *Being Nice* (X) separately for people who *Stay in* and *Go out* (Z). This naturally produces two lines on a plot: one for each group. The question is what those two lines look like relative to each other.

Here are our two competing hypotheses, visualized:

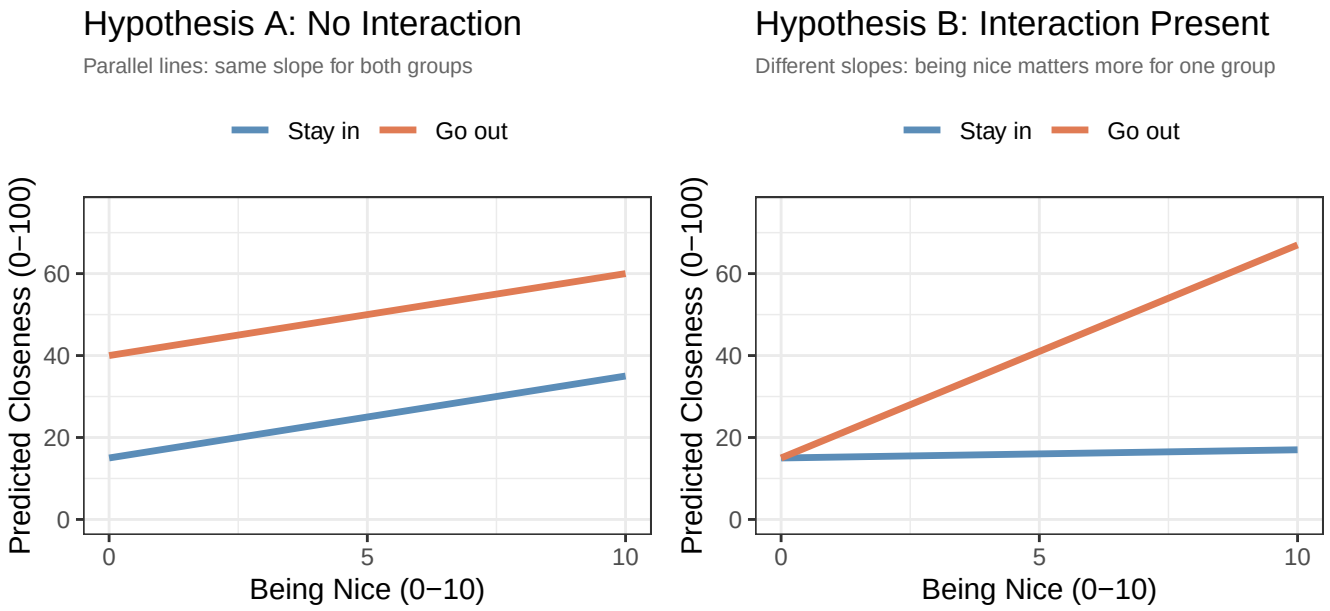


Figure 16.1: The two hypotheses drawn before any data are analysed. Left: the additive world, where both groups gain the same amount from being nice. Right: the interaction world, where only the Go out group gains anything.

Look at the difference between these two pictures:

- In **Hypothesis A**, the two lines are **parallel**. The "Go out" line is higher than the "Stay in" line (going out raises closeness overall), but the *slope* of both lines is the same. Being nice helps by about the same amount whether you go out or not. This is what a **main effects only** world looks like: each variable has an independent, additive effect.

- In **Hypothesis B**, the two lines are **not parallel**. For people who *stay in*, the line is almost flat: being nice barely moves the needle. For people who *go out*, the line is steep: being nice strongly predicts closeness. The slope for "Go out" is much steeper than the slope for "Stay in." This is an **interaction**: the slope of Being Nice *depends on* whether you go out.

The key visual signature of an interaction is **non-parallel lines**. If you run an interaction model and your lines look parallel, the interaction is not doing anything. If the lines fan out, cross, or differ in steepness, you have an interaction.

We will use this visual intuition throughout this chapter to interpret the output.

16.4.3 The Two-Regressions Intuition

You have already met this idea, in the first thing we did in this chapter. Splitting the data by group and running a separate regression in each half gives you a slope per group:

- **Regression for Stay-in students:** a slope for Being Nice. Call it b_{stay}.
- **Regression for Go-out students:** a slope for Being Nice. Call it b_{go}.

If there is no interaction, $b_{stay} \approx b_{go}$: the two slopes are about the same. If there is an interaction, $b_{stay} \neq b_{go}$: the slopes differ.

That approach *shows* you an interaction and cannot *test* one, for the reason the cold open ran into: two estimates from two disconnected analyses, and no standard error anywhere for the quantity you care about, which is the gap.

An interaction model fits both slopes simultaneously in a single model and attaches a significance test to the **difference** between them. That test is the interaction term. When it is significant, the slopes differ by more than sampling variability would predict. When it is not, the data do not support the claim that the slopes differ at all.

So an interaction model is a formalized, simultaneous, significance-tested version of the two regressions you already ran. Hold onto that image; when we get to B_3 it stops being an analogy and starts being literally the same numbers.

16.5 Before We Run the Models: Two Concepts You Need First

16.5.1 Centering: Making Zero Meaningful

You have seen the intercept in every regression model we have run in this book, and you know it means "the predicted value of Y when X = 0." That definition is simple enough. The problem is that zero is not always a meaningful value.

In our friendship study, *Being Nice* is measured on a 0–10 scale. Zero means "not nice at all," a real value technically, but one that describes an essentially sociopathic person who presumably does not exist in our sample. When our regression model reports an intercept, it is making a prediction for a hypothetical person with zero niceness. That is not a very useful prediction.

Now consider what happens when we build an interaction model. The intercept will be the predicted closeness score for a person with zero niceness *who also stays in*. The coefficient for Going Out will be the gap between the two groups *at zero niceness*. These values are anchored to a strange and artificial point that nobody in our data actually occupies.

Centering solves this by shifting the scale of a continuous variable so that zero has a meaningful interpretation: **the average person in the sample.**

The process is simple: subtract the mean.

$$X_c = X - M$$

After centering:

- A score of $X_c = 0$ means this person is exactly average on Being Nice (relative to this sample).
- A score of $X_c = 2$ means this person is 2 points *above* the average.
- A score of $X_c = -3$ means this person is 3 points *below* the average.

The scale is shifted, but nothing else changes. The **distances between people are identical**. The **slope of the regression line is identical**. The R^2 **is identical**. The only thing that changes is the meaning of the intercept (and the meaning of some of the other coefficients in the interaction model, as we will see).

Important distinction: centering is not the same as standardizing.

You may recall from earlier in the book that we can also *standardize* (or *z-score*) a variable by subtracting the mean *and* dividing by the standard deviation:

$$Z = \frac{X - M}{S}$$

Standardizing changes the units of the variable: a standardized coefficient represents the change in Y for a one-standard-deviation increase in X. Centering does **not** change the units: a centered coefficient still represents the change in Y for a one-unit increase in X. In this chapter, we will only center, not standardize. We want to preserve the original units of our scale (a one-point increase in niceness = one point) while simply making zero mean "the average person."

When we center *Being Nice*, the intercept in our interaction model will represent the predicted closeness score for a person of **average niceness who stays in**. That is a real, interpretable, meaningful prediction; a much better anchor than "a person with zero niceness." We will do it to our data in a moment, as soon as the data exist.

16.5.2 Dummy Coding: A Quick Review

You already know how to dummy code a categorical variable for regression. When we use a binary categorical predictor like Going Out (Stay in vs. Go out), R automatically converts it into a dummy variable: one group is coded as 0 (the **reference group**) and the other as 1 (the **comparison group**). In our study, *Stay in* will be the reference group (coded 0), and *Go out* will be the comparison group (coded 1). We tell R which group comes first by setting the factor levels in the desired order.

With dummy coding in place:

- The **intercept** represents the predicted outcome for the reference group (Stay in) at whatever value the continuous predictor takes.
- The **coefficient for GoingOut** represents the gap between the two groups, holding the continuous predictor constant.

We will see exactly how this plays out, and how centering changes the interpretation, as we work through the models.

16.6 The Data

We collected data from 200 first-semester university students. Each student's friendship closeness with one person they met at the start of the semester was recorded at the end of the semester using a 0–100 feelings thermometer. Their agreeableness and warmth was rated by a trained panel on a 0–10 scale, and we noted whether they regularly attended social events and parties throughout the semester.

Here is a summary of the sample broken down by social engagement:

```
library(tidyverse)

Friend.Data |>
  group_by(GoingOut_) |>
  summarise(
    n         = n(),
    M_Nice    = round(mean(BeingNice), 2),
    M_Close   = round(mean(Closeness), 2),
    SD_Close  = round(sd(Closeness), 2)
  )
```

```
# A tibble: 2 x 5
  GoingOut_     n M_Nice M_Close SD_Close
  <fct>     <int>  <dbl>   <dbl>    <dbl>
1 Stay in     116   5.07    15.7     6.65
2 Go out       84   5.43    42.0    15.6
```

The "Go out" group sits far higher on the closeness thermometer on average. This makes sense: going out creates more social contact. But the standard deviations are large and similar across groups, meaning there is substantial individual variability within each group. Something beyond just whether you go out is driving that spread. That is what we will investigate.

The mean of *Being Nice* in our sample is:

```
round(mean(Friend.Data$BeingNice), 2)
```

```
[1] 5.22
```

16.6.1 How to Center in R

Now that the data exist, we can center. We use the `scale()` function with `center = TRUE` and `scale = FALSE`. Setting `scale = FALSE` tells R to subtract the mean *without* dividing by the standard deviation, so we get the shift without the rescaling.

```
# as.numeric() matters here; see the warning below
Friend.Data$BeingNice.C <- as.numeric(scale(Friend.Data$BeingNice,
                                            center = TRUE, scale = FALSE))

# Always check it worked: a centered variable has a mean of (essentially) zero
mean(Friend.Data$BeingNice.C)
```

```
[1] -7.105427e-17
```

That is zero with a rounding error's worth of dust on it, which is exactly what centering is supposed to produce. R prints it in scientific notation to be annoying; `1.78e-16` is a decimal point followed by fifteen zeros and then a rounding error. A centered value of 0 now corresponds to a person who scored approximately 5.22 on the original 0–10 scale.

Two things in that one line are waiting to bite you.

Watch the `scale = FALSE`. Leave it out and `scale()` helpfully divides by the standard deviation as well, so you have silently z-scored instead of centered. Nothing errors. Your model runs, your plot draws, and every slope you report afterwards is in units nobody asked for.

Watch the `as.numeric()`. `scale()` does not return a vector; it returns a one-column *matrix* with the mean bolted on as an attribute. `lm()` does not care, which is the problem, because it means you will not find out until three sections from now when some plotting package refuses to draw your model and says something incomprehensible about `nmatrix.1`. Wrap it in `as.numeric()` and the column is an ordinary number like everything else in your data frame.

16.7 A First Look at the Data

Before running any models, let's visualize the relationships in the data. This is always step one.

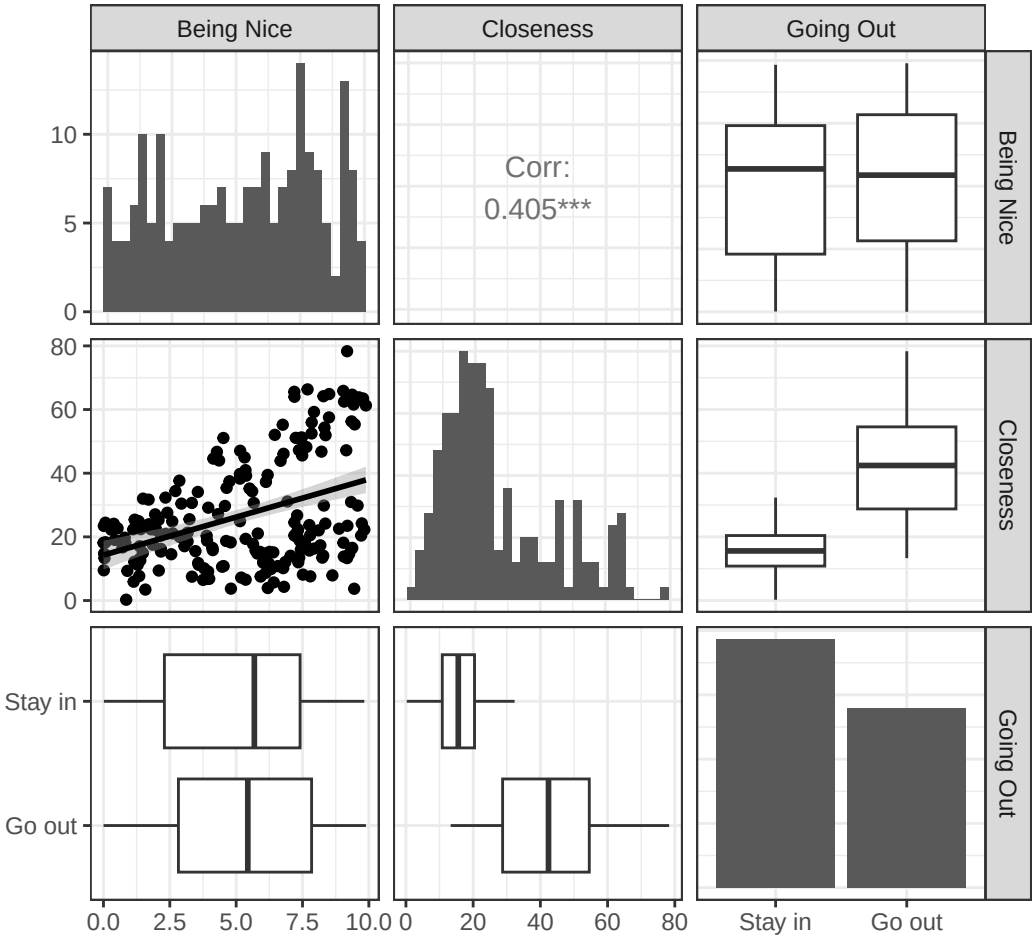


Figure 16.2: The standard first look: every variable against every other variable. The Being Nice by Closeness scatterplot in particular has far more scatter than a correlation of .41 alone would suggest.

A few things stand out immediately:

- **Being Nice and Closeness** have a moderate positive correlation overall. More agreeable people tend to form closer friendships.
- **Going Out and Closeness** show a clear difference: students who go out score much higher on the thermometer on average.
- The **overall correlation between Being Nice and Closeness** is moderate, but look at the scatterplot: there is a lot of spread around the trend line. Something else might be going on.

The key question is whether the relationship between Being Nice and Closeness looks *the same* for both groups. The pairs plot makes this hard to see directly. That is what the interaction model is for.

16.8 Building the Models

16.8.1 Model 1: The Additive Model

We start with the simpler model: the one that says each variable has its own independent effect, with no interaction. This is our **main effects model** or **additive model**.

$$\widehat{Closeness} = B_0 + B_1 \cdot BeingNice_C + B_2 \cdot GoingOut + \varepsilon$$

Note two things about how we set this up:

1. We use the **centered** version of Being Nice (`BeingNice.C`), so the intercept will be the predicted closeness score for an average-niceness person.
2. We use the **factored** GoingOut variable (`GoingOut_`), with *Stay in* as the reference group, so R handles the dummy coding automatically.

```
Model.1 <- lm(Closeness ~ BeingNice.C + GoingOut_, data = Friend.Data)
summary(Model.1)
```

```
Call:
lm(formula = Closeness ~ BeingNice.C + GoingOut_, data = Friend.Data)

Residuals:
    Min      1Q  Median      3Q     Max
-21.320  -7.209  -1.063   5.883  28.356

Coefficients:
                Estimate Std. Error t value Pr(>|t|)
(Intercept)      16.0242     0.8813  18.182   <2e-16 ***
BeingNice.C       2.1281     0.2307   9.226   <2e-16 ***
GoingOut_Go out  25.5128     1.3613  18.741   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 9.485 on 197 degrees of freedom
Multiple R-squared:  0.6996,    Adjusted R-squared:  0.6966
```

```
F-statistic: 229.4 on 2 and 197 DF,  p-value: < 2.2e-16
```

16.8.2 What do the coefficients mean?

- **Intercept:** The predicted closeness score for a person of *average niceness* (BeingNice.C = 0, i.e., roughly a 5.2 on the original scale) who *stays in* (GoingOut = reference group). This is a real, interpretable prediction: someone who is neither particularly nice nor antisocial, and who mostly stays home.
- **BeingNice.C:** Controlling for whether the student goes out or not, each additional point of niceness (above the average) is associated with about 2.13 more points of closeness. This is averaged across both groups.
- **GoingOut_Go out:** Controlling for how nice the student is, going out is associated with about 25.5 more points of closeness than staying in. This is the overall main effect of social engagement.

16.8.3 Visualizing the additive model

The critical test of the additive model is whether it produces parallel lines when we plot the predicted values.

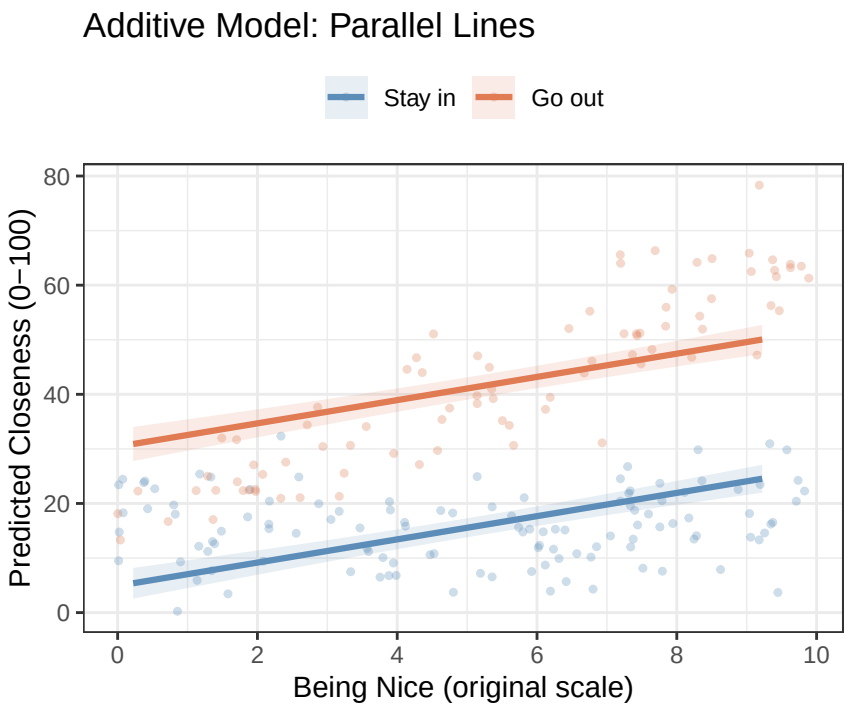


Figure 16.3: The additive model's predictions, with the raw data underneath. The lines are parallel because the model has no way to make them anything else.

The additive model forces the two lines to be parallel: that is built into the model's assumptions when there is no interaction term. Notice how the lines run at the same angle, just shifted up for the "Go out" group. This model says: being nice helps equally, no matter what.

But does this model actually fit the data? Look at the raw data points (the faint dots). The "Go out" points (orange) seem to fan upward much more steeply as niceness increases, while the "Stay in" points (blue) look relatively flat. The parallel lines do not seem to capture that pattern. We need a model that lets the slopes differ.

And there is one more diagnostic concern with the additive model. Let's look at the residuals:

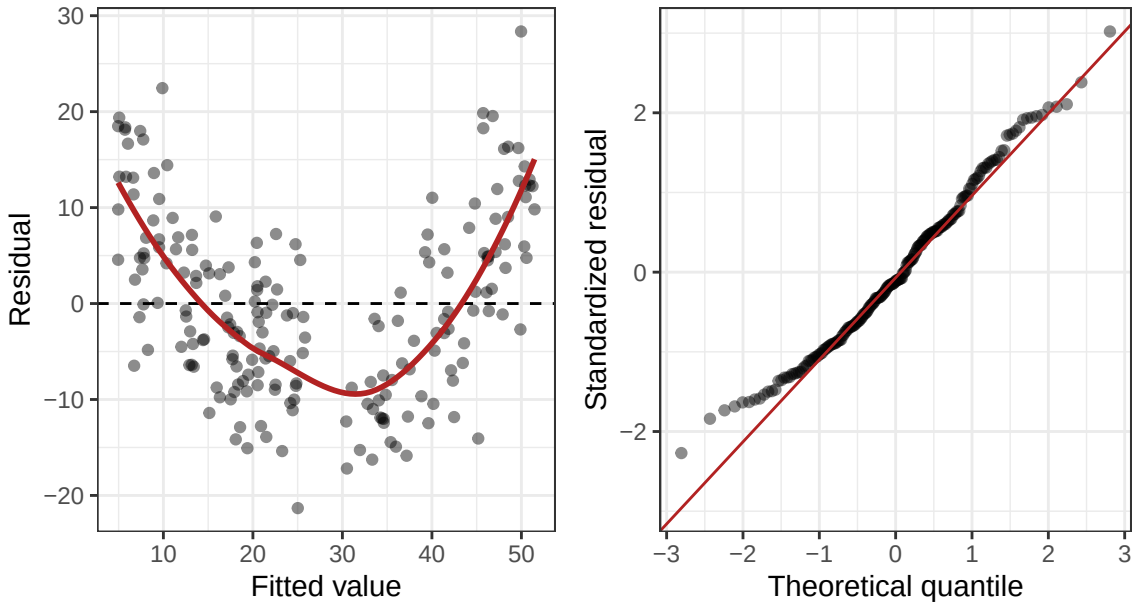


Figure 16.4: Diagnostics for the additive model. Left: residuals against fitted values, with a loess curve. Right: a normal quantile-quantile plot of the standardized residuals.

The residual plot on the left shows a clear curved pattern, a classic sign that the model is missing something systematic. The data have a structure that a straight-line additive model cannot capture. This is exactly what you would expect when a real interaction exists and you have not modeled it. Time to add the interaction term.

16.8.4 Model 2: The Interaction Model

Now we add the interaction term. The equation becomes:

$$\widehat{Closeness} = B_0 + B_1 \cdot BeingNice_C + B_2 \cdot GoingOut + B_3 \cdot BeingNice_C \times GoingOut + \varepsilon$$

The new term, $B_3 \cdot BeingNice_C \times GoingOut$, is the product of our two predictors. This is what allows the slope of Being Nice to be *different* for each group. In R, we use the `*` operator, which automatically includes both main effects and the interaction:

```
Model.2 <- lm(Closeness ~ BeingNice.C * GoingOut_, data = Friend.Data)
summary(Model.2)
```

```
Call:
lm(formula = Closeness ~ BeingNice.C * GoingOut_, data = Friend.Data)

Residuals:
     Min       1Q   Median       3Q      Max
-18.2429  -4.4894  -0.2029   4.3964  17.9396

Coefficients:
                          Estimate Std. Error t value Pr(>|t|)
```

```
(Intercept)                   15.7363     0.6183  25.449   <2e-16 ***
BeingNice.C                    0.1998     0.2106   0.949    0.344
GoingOut_Go out               25.2284     0.9548  26.422   <2e-16 ***
BeingNice.C:GoingOut_Go out    4.7044     0.3289  14.304   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 6.651 on 196 degrees of freedom
Multiple R-squared:  0.853, Adjusted R-squared:  0.8508
F-statistic: 379.2 on 3 and 196 DF,  p-value: < 2.2e-16
```

Before we interpret these coefficients in detail, let's ask the most important question first: **does this more complex model actually fit the data better?**

16.8.5 Does the Interaction Model Explain More Variance?

Adding the interaction term uses up one additional degree of freedom. We need to check whether the improvement in model fit justifies this cost. We do this with an **F-test for model comparison**, the same logic as hierarchical regression. The null hypothesis is that the interaction explains zero additional variance:

$$H_0 : \Delta R^2 = 0 \quad \text{(the interaction adds nothing)}$$

```
Additive model R-squared:     0.7

Interaction model R-squared:  0.853

Change in R-squared:          0.153
```

```
knitr::kable(anova(Model.1, Model.2), digits = 3,
             caption = "Model Comparison: Additive vs. Interaction")
```

Table 16.1: Model Comparison: Additive vs. Interaction

Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
197	17722.135	NA	NA	NA	NA
196	8670.551	1	9051.584	204.613	0

The interaction model explains substantially more variance than the additive model. The F-test is significant, meaning that the improvement in R^2 ($\Delta R^2 = 0.153$) is far larger than we would expect by chance. Adding the interaction term was worth it. The two lines are not parallel in the population.

Let's also check the residuals from the interaction model:

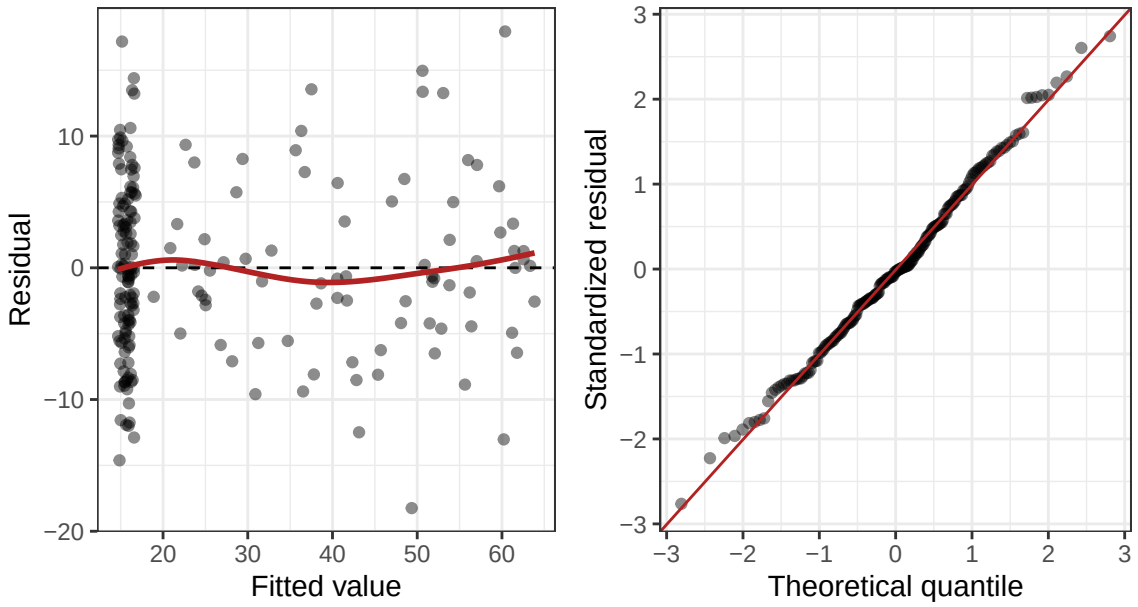


Figure 16.5: The same two diagnostics for the interaction model. Compare the loess curve on the left with the bowed one from the additive model.

The residual pattern is now much healthier: the curved structure is gone, and the points are scattered more evenly around zero. This confirms that the interaction model is capturing what was actually going on in the data.

16.9 Interpreting the Interaction Model

16.9.1 The Plot First

Before reading the numbers, look at the picture. The plot below shows the predicted values from the interaction model:

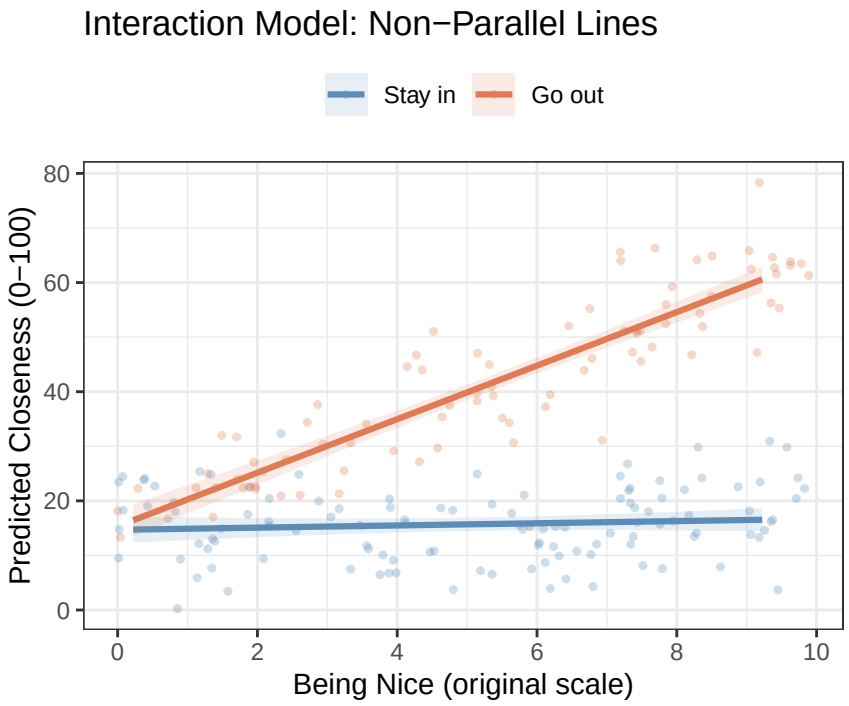


Figure 16.6: The interaction model's predictions on the same data. The model is now allowed to give each group its own slope, and it uses that freedom immediately.

This is **Hypothesis B** from our earlier sketch come to life in real data. The "Stay in" line (blue) is essentially flat: being nice does not seem to predict friendship closeness for students who spend the semester at home. The "Go out" line (orange) has a strong positive slope: among students who regularly attend social events, being a warmer and more agreeable person leads to substantially closer friendships. The two lines start at roughly the same point on the left and diverge dramatically as niceness increases.

Compare this to the parallel-lines plot from the additive model. The two models tell completely different stories. The additive model said: being nice helps equally for everyone. The interaction model says: being nice only really pays off if you are actually around people to be nice *to*.

Now let's connect this picture to the numbers.

16.9.2 Walking Through Each Coefficient

Let's go back to the model output and interpret each term carefully, using the plot as our guide.

```
Call:
lm(formula = Closeness ~ BeingNice.C * GoingOut_, data = Friend.Data)

Residuals:
     Min       1Q   Median       3Q      Max
-18.2429  -4.4894  -0.2029   4.3964  17.9396

Coefficients:
                          Estimate Std. Error t value Pr(>|t|)
```

```
(Intercept)                  15.7363     0.6183  25.449   <2e-16 ***
BeingNice.C                   0.1998     0.2106   0.949    0.344
GoingOut_Go out              25.2284     0.9548  26.422   <2e-16 ***
BeingNice.C:GoingOut_Go out   4.7044     0.3289  14.304   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 6.651 on 196 degrees of freedom
Multiple R-squared:  0.853, Adjusted R-squared:  0.8508
F-statistic: 379.2 on 3 and 196 DF,  p-value: < 2.2e-16
```

16.9.3 The Intercept

The intercept (B_0) is the predicted closeness score when *both* predictors equal zero.

With our setup:

- `BeingNice.C = 0` means a person of **average niceness** (the mean of 5.22 on the original scale)
- `GoingOut = "Stay in"` is the reference group (value = 0)

So the intercept is the predicted closeness score for **an average-niceness person who stays in**. Looking at our model output, the intercept is approximately 15.7.

On the plot, this is the point on the blue "Stay in" line directly above the center of the x-axis (average niceness). It sits around 16 on the 0–100 thermometer: not strangers exactly, but nowhere near best friends. About what proximity alone buys you when you never leave the building.

The intercept also comes with a p-value, testing whether that predicted value differs from zero. It does. But "average students have a nonzero opinion of the person they sat next to" is not a finding anybody will publish. Intercept p-values are usually ignorable; the slope tests are where the questions live.

16.9.4 B_1: The Slope of Being Nice for the Stay-In Group

`BeingNice.C` ($B_1 = 0.2$) tells you the slope of Being Nice *for the reference group*, that is, the slope of the blue line for students who *stay in*.

This is called a **simple effect** rather than a main effect, because it is not the effect of Being Nice *on average across groups*: it is the effect of Being Nice *within one specific group*. Specifically, it is the rate of change along the blue line.

The p-value for `BeingNice.C` tests: **is the slope of the "Stay in" line significantly different from zero?**

Looking at the output, this slope is small and non-significant ($p \approx 0.344$). This is exactly what the flat blue line in our plot tells you: for students who stay in, being nicer is not significantly predictive of closer friendships. There is essentially no relationship there.

16.9.5 B_2: The Group Difference at Average Niceness

`GoingOut_Go out` ($B_2 = 25.2$) tells you the predicted difference between the "Go out" and "Stay in" groups **at the centering point of Being Nice**, that is, at average niceness (BeingNice.C = 0).

This is a **main effect** in the sense that it is evaluated at the mean of the continuous variable (thanks to centering). It tells you: for a person of average niceness, how much more closeness do they get by going out versus staying in?

On the plot, this is the vertical gap between the two lines at the center of the x-axis (average niceness). Students who go out score about 25 points higher on the thermometer than students who stay in, when both groups are at average niceness. This gap is highly significant.

16.9.6 B_3: The Interaction Term

`BeingNice.C:GoingOut_Go out` ($B_3 = 4.7$) is the heart of the analysis. This coefficient represents **how much steeper the "Go out" slope is compared to the "Stay in" slope.**

Think of it as the answer to: *by how many extra points of closeness, per niceness point, does the "Go out" group outpace the "Stay in" group?*

- The slope for the "Stay in" group is $B_1 = 0.2$ (essentially flat)
- The slope for the "Go out" group is $B_1 + B_3 = 0.2 + 4.7 = 4.9$

Look at those two numbers, then scroll back to the two regressions we ran at the top of the chapter before we knew what an interaction was. They are the same numbers. 0.2 and 4.9, recovered to the last decimal place, because splitting the data by group and letting an interaction term do the splitting for you are the same arithmetic wearing different clothes.

What is new is B_3 itself. Two separate regressions could hand you both slopes and then shrug. This model hands you the *distance* between them, 4.7 points of closeness per point of niceness, with a standard error attached and a p-value underneath. That is the test the reviewer asked for, and it is emphatically significant.

Here is a summary table to tie everything together:

Coefficient	Value	What it represents	What it tests
Intercept	15.74	Predicted closeness for average-niceness, Stay-in student	Is this > 0?
BeingNice.C	0.2	Slope of Being Nice *for Stay-in group* (simple effect)	Is the Stay-in slope $\neq 0$?
GoingOut (Go out)	25.23	Group gap at average niceness (main effect)	Is there a group difference at the mean?
BeingNice.C × GoingOut	4.7	Difference in slopes between groups	Are the two slopes significantly different?

16.9.7 The One-Line Version

The plot above was built by hand, and we will build a fully polished one at the end of the chapter. That is the right amount of effort when a figure is going into a manuscript. It is far too much effort when you are sitting in front of a model at 11pm wanting to know what shape it is.

For that, `sjPlot` does the whole thing in one line:

```
library(sjPlot)
plot_model(Model.2, type = "int")+theme_bw()
```



Figure 16.7: The same interaction, drawn by `plot_model()` with no formatting work at all. Non-parallel lines, same story, one line of code.

This is the exploratory version, and exploratory code is allowed to be fast and slightly ugly. Notice what it gives you and what it does not. The x-axis is *centered* niceness, running from about −6.6 to +6.6 rather than the original 0 to 10, so the reader has to know what centering is before the axis means anything. It labels the variable "Being Nice C", because it is reading your column name and has no idea what you meant.

The hand-built version buys back the original axis and control over everything a reviewer will complain about.

Learn both. Use the fast one until the figure actually matters.

16.10 Simple Slopes: Testing Each Group's Slope Separately

The interaction term tells us that the two slopes *differ significantly*. But we also want to know, for each group separately: **is the relationship between Being Nice and Closeness significantly different from zero?**

We already have the answer for the "Stay in" group: the `BeingNice.C` coefficient is the "Stay in" slope, and it is non-significant. Being nice does not predict friendship closeness for students who stay in.

But what about the "Go out" group? We know the "Go out" slope ($B_1 + B_3 \approx 4.9$) is large and positive, but we need a formal significance test.

We use the `emtrends()` function from the `emmeans` package, which extracts and tests the slope of Being Nice *separately within each level* of Going Out:

```r
library(emmeans)
simple.slopes <- emtrends(Model.2, "GoingOut_", var = "BeingNice.C")
# infer = TRUE shows both confidence intervals and significance tests in one table
summary(simple.slopes, infer = TRUE)
```

```
 GoingOut_ BeingNice.C.trend    SE  df lower.CL upper.CL t.ratio p.value
 Stay in                 0.2 0.211 196   -0.215    0.615   0.949  0.3439
 Go out                  4.9 0.253 196    4.406    5.402  19.412 <0.0001

Confidence level used: 0.95
```

This output gives us what we need:

- **Stay in group:** The slope of Being Nice is approximately 0.2, which is *not* significantly different from zero ($p = 0.344$). For students who stay in, being nicer does not predict how close a friendship they form.
- **Go out group:** The slope of Being Nice is approximately 4.9, which is *significantly* positive ($p < .001$). For students who go out, every additional point of niceness is associated with about 5 more points of closeness.

Notice that `emtrends()` has handed back the same two slopes for the third time: once from the separate regressions at the top of the chapter, once as B_1 and $B_1 + B_3$, and now here. The estimates never move. The interaction model buys you two things. It pools one error term across all 200 students instead of two from 116 and 84. And it puts a standard error on the *gap*. That gap is the only quantity in this chapter that the separate regressions could not produce at all.

This completes the picture. The effect of Being Nice on friendship closeness is **conditional**: it only emerges among students who are out there meeting people. For students who stay home, personality differences do not seem to matter much for friendship formation. Going out is what activates the effect of being a warm and agreeable person.

16.11 APA Write-Up

Here is how you would write up these results following APA style. Notice the structure: descriptive statistics first, overall model fit, the interaction test, then the simple slopes to unpack what the interaction means.

Prior to the main analysis, descriptive statistics were examined by group. Students who stayed in reported lower friendship closeness on average ($M = 15.7$, $SD = 6.7$) than students who went out regularly ($M = 42$, $SD = 15.6$).

A hierarchical multiple regression was conducted to examine whether going out moderated the relationship between agreeableness and friendship closeness. Being Nice (centered at the sample mean) and Going Out (dummy coded, *Stay in* as reference group) were entered in Step 1 as main effects; their product (the interaction term) was entered in Step 2. The additive model (Step 1) was statistically significant, $F(2, 197) = 229.42$, $p < .001$, $R^2 = 0.7$. Adding the interaction term (Step 2) produced a statistically significant improvement in model fit, $\Delta R^2 = 0.153$, $F(1, 196) = 204.61$, $p < .001$, indicating that the effect of agreeableness on friendship closeness differed significantly between students who stayed in and those who went out.

In the full interaction model ($R^2 = 0.853$, adjusted $R^2 = 0.851$), the interaction between agreeableness and going out was statistically significant, $b = 4.7$, $SE = 0.33$, $t(196) = 14.3$, $p < .001$. To interpret the interaction, simple slopes for agreeableness were examined separately for each group. Among students who stayed in, agreeableness did not significantly predict friendship closeness, $b = 0.2$, $SE = 0.21$, $t(196) = 0.95$, $p = 0.344$. Among students who went out, agreeableness was a strong positive predictor of friendship closeness, $b = 4.9$, $SE = 0.25$, $t(196) = 19.41$, $p < .001$. These results indicate that the benefit of agreeableness for friendship formation is conditional on social engagement: being a warm and agreeable person leads to closer friendships only among students who are regularly out meeting others.

A few things to notice about this write-up:

- We report the **model comparison** (the ΔR^2 and its F-test) as the primary test of whether an interaction exists.
- We then report the **interaction coefficient** (b, SE, t, p) from the full model.
- We follow up with **simple slopes** (the slope of the continuous variable separately for each group) to explain *what* the interaction looks like. The interaction tells us the slopes differ; the simple slopes tell us how.
- We use the phrase **"the effect of X was conditional on Z"** to describe an interaction in plain language.
- We **do not** emphasize or interpret the main effects from the interaction model in isolation, because in the presence of a significant interaction, the main effects represent the effect at one specific value of the other variable (in this case, the reference group or the mean) and can be misleading to report without that context.

16.12 APA-Style Figure

A well-constructed figure is often the clearest way to communicate an interaction. Below is a publication-ready interaction plot that you can adapt for your own write-ups. This figure shows the predicted values from the interaction model with 95% confidence bands, and nothing else.

Figure 1. Predicted friendship closeness as a function of agreeableness and social engagement. Lines represent predicted values from the interaction model; shaded bands represent 95% confidence intervals. Among students who regularly went out (*Go out*; orange), agreeableness was a strong positive predictor of friendship closeness. Among students who stayed in (*Stay in*; blue), agreeableness was not significantly associated with friendship closeness.

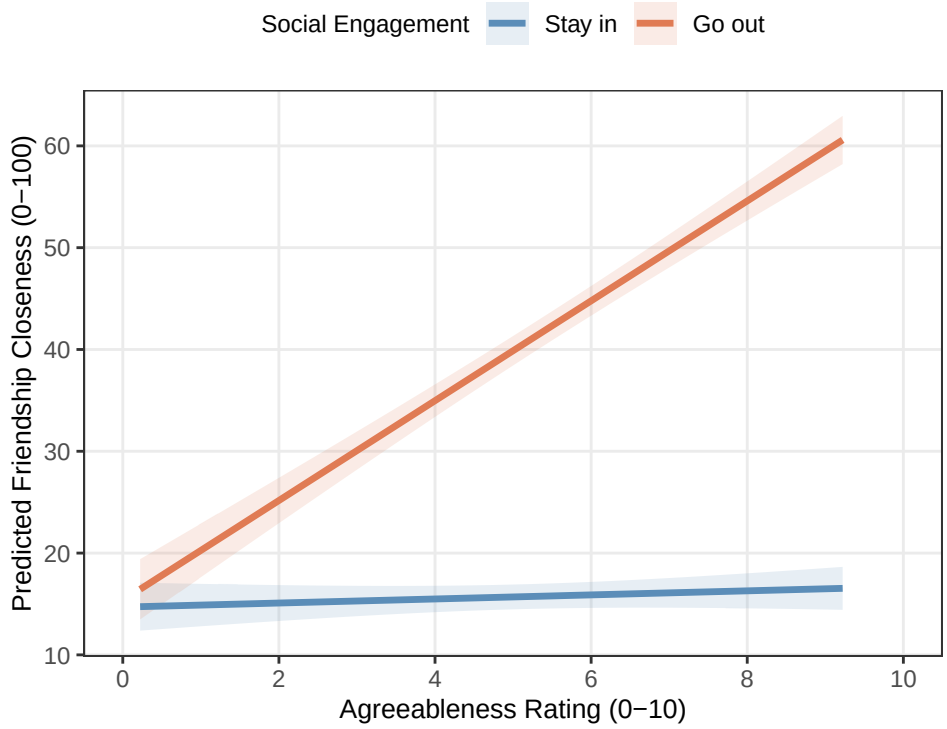
```
# --- APA-Style Interaction Plot ---
library(ggplot2)

# Step 1: Get predicted values from the model using emmeans
Inter.Em.Fig <- emmeans(Model.2,
                        ~ BeingNice.C | GoingOut_,
                        at = list(BeingNice.C = seq(Min.BN, Max.BN, 0.25)))
Fig.Data <- as.data.frame(Inter.Em.Fig)

# Step 2: Transform centered x-axis back to original scale (0-10)
mean.BN <- mean(Friend.Data$BeingNice)
Fig.Data$BeingNice.orig <- Fig.Data$BeingNice.C + mean.BN

# Step 3: Build the plot
ggplot(Fig.Data, aes(x    = BeingNice.orig,
                     y    = emmean,
                     color = GoingOut_,
                     fill  = GoingOut_,
                     group = GoingOut_)) +
  # Confidence ribbons
  geom_ribbon(aes(ymin = lower.CL, ymax = upper.CL),
              alpha = 0.15, color = NA) +
  # Regression lines
  geom_line(linewidth = 1.2) +
  # Formatting
  scale_color_manual(values = c("Stay in" = "#5b8db8",
                                "Go out"  = "#e07b54"),
```

```
                    labels = c("Stay in", "Go out")) +
scale_fill_manual(values  = c("Stay in" = "#5b8db8",
                              "Go out"  = "#e07b54"),
                  labels = c("Stay in", "Go out")) +
scale_x_continuous(breaks = seq(0, 10, 2), limits = c(0, 10)) +
labs(x     = "Agreeableness Rating (0-10)",
     y     = "Predicted Friendship Closeness (0-100)",
     color = "Social Engagement",
     fill  = "Social Engagement") +
theme_bw() +
theme(legend.position   = "top",
      legend.title      = element_text(size = 9),
      legend.text       = element_text(size = 9),
      axis.title        = element_text(size = 10),
      panel.grid.minor  = element_blank())
```



Note: *I removed the raw data points here. In larger, more complex models where you have lots of controls, the raw points can conflict with the plotted slopes and confuse the reader.*

> ⚠ **Do Not Diagnose an Interaction by Staring at Two P-Values**
>
> One simple slope being significant while the other is not does **not** prove that the slopes differ. That was the whole problem the chapter opened with: two separate regressions handed us a significant slope and a non-significant one, and between them there was no test at all. The interaction coefficient is the direct test of that difference. The plot and simple slopes explain the interaction after it has been tested.

16.13 The Short Story

If you remember six things from this chapter, make it these.

- **An interaction means the slopes differ.** "The effect of X depends on Z" sounds like a hedge. It is a claim: X's slope changes across levels of Z. Non-parallel lines are the visual signature.
- **Center the continuous predictor so that zero is a person who exists.** Centering does not change the slope, the fit, or the R^2. It changes what the intercept and B_2 are *about*, from a hypothetical zero-niceness person to the average person in your sample.
- B_1 **is the reference group's slope, not an average.** It is a simple effect. In a model with an interaction, it describes exactly one group, the one coded 0.
- B_2 **is the group gap at the mean of the centered predictor**, not the group gap in general. Move the centering point and this number moves with it.
- B_3 **is the difference between the slopes, and it is the test.** It is the one quantity that running two separate regressions cannot give you.
- **Test first, explain second.** The ΔR^2 F-test asks whether an interaction exists. Simple slopes and the plot describe what it looks like. Doing this in the other order is how people talk themselves into interactions that are not there.

And the reporting order, which is the same every time: **model comparison test, then the interaction coefficient, then the simple slopes, then the figure.** The test establishes that there is something to explain. Everything after it is the explanation.

17 Categorical-by-Categorical Interactions: The 2-by-2 Design

17.1 The Design Psychology Cannot Stop Running

Open a psychology journal to a random page. There is a very good chance the analysis you land on is a 2 × 2: two variables, two levels each, four groups, and one figure with two lines that either are or are not parallel. Drug versus placebo crossed with young versus old. Threat crossed with reassurance. Word lists crossed with sleep. It is the most-run design in the field, and if you go to graduate school in this discipline you will run one within the year.

So let us run one. Same friendship study as the prior chapter, same hypothesis, same 200 students.

There is one obstacle. A 2 × 2 needs two categorical variables, and Being Nice is not categorical. It is a perfectly good continuous rating from 0 to 10. So we are going to keep the least agreeable students, keep the most agreeable students, throw the 119 people in the middle of the scale directly into the bin, and call what survives **Low Nice** and **High Nice**.

Yes, that is roughly as bad as it sounds. We will come back to exactly how bad, and to why this book is allowed to do it and you are not, at the moment we actually do it.

17.2 Now Both Predictors Are Categories

In the prior chapter you learned how to detect, estimate, and interpret an interaction between a *continuous* predictor and a *categorical* one. In the friendship study, Being Nice (0–10 scale) was continuous and Going Out (Stay in vs. Go out) was categorical. The interaction told you that the **slope** of Being Nice on friendship closeness was different for students who went out compared to those who stayed in.

That kind of interaction is very common in psychology research. But so is a second kind: interactions where **both predictors are categorical**. This is the foundation of the **two-way factorial ANOVA**, and the 2 × 2 is its simplest and most common version.

This chapter assumes you have already read the prior chapter material. You know what an interaction is, you know how to interpret the interaction model equation, and you know how to use `anova()` to compare a main effects model to an interaction model. What is new here is what changes, and what stays the same, when both predictors are categorical.

17.2.1 What Changes from prior chapter

The biggest conceptual change is this: **there is no continuous variable to center.** In the prior chapter centering was necessary because the intercept needed a meaningful zero point on the continuous scale. When both predictors are categorical, every predictor already takes on a small set of discrete values, and the "zero point" is simply the reference category of each variable, a specific cell in the design, not an abstract mean.

Everything else follows from that:

- No centering is needed. Both variables just need dummy coding.
- The interaction coefficient is no longer a "difference in slopes." There are no slopes here. Instead, B_3 is the **difference in group differences**, sometimes called the "difference of differences." We will unpack this carefully.
- The follow-up to the interaction is **simple effects analysis**: testing, within each level of one variable, whether the levels of the other variable differ significantly. We use `pairs()` from the `emmeans` package instead of `emtrends()`.
- Effect sizes for pairwise comparisons are expressed as **Cohen's *d***, not unstandardized slopes.

What **stays the same**: the interaction is still about "it depends." The visual signature is still non-parallel lines (or unequal gaps in a bar chart). The model comparison is still done with `anova()` and ΔR^2. The APA write-up still follows the same structure.

17.2.2 What the 2×2 Looks Like as a Table

Before we touch any data, it helps to lay out the full design as a 2 × 2 table. Each cell represents a unique combination of the two categorical predictors, and we expect a separate (estimated) mean outcome in each cell.

	Stay in	**Go out**
Low Nice	μ_{11}: Low Nice × Stay in	μ_{12}: Low Nice × Go out
High Nice	μ_{21}: High Nice × Stay in	μ_{22}: High Nice × Go out

The interaction question is: **is the difference between High Nice and Low Nice the same in both columns (Going Out groups)?** Or equivalently: **is the difference between Go out and Stay in the same in both rows (Niceness groups)?**

Both questions ask the same thing. The answer is the interaction.

> **i** Same Two IVs, Same One DV, Still Nobody Measured Twice
>
> A quick inventory, since the parts have been renamed but not replaced.
>
> The **DV** (the outcome) is still friendship closeness, 0 to 100. The **IVs** (the predictors) are still two: Niceness Group and Going Out. Two IVs is what makes an interaction possible; that has not changed. What changed is only that both IVs are now categorical.
>
> Both are still **between subjects**: one row per student, one cell per student, nobody measured twice. A 2 × 2 in which the same people turn up in more than one cell is a repeated-measures design, which is a different model and a later chapter.
>
> And it is still true that **regression does not care whether an IV was randomly assigned or merely observed.** Neither of ours was assigned. We did not randomly allocate students to be agreeable, and we did not send half of them out to parties. The 2 × 2 table you are about to see will look exactly like a table from a real experiment, the F values are computed exactly the same way, and none of that buys you a causal claim. The design decides what you may conclude. The arithmetic never does.

17.3 A Difference of Differences

17.3.1 What It Means When Both Predictors Are Categorical

In the prior chapter, the interaction coefficient B_3 told you how much steeper the slope of Being Nice was for the "Go out" group compared to the "Stay in" group. Slopes are a continuous concept: the rate at which friendship closeness increases as niceness increases by one unit.

When Being Nice is categorical (Low vs. High), there are no slopes. Instead, there is a **gap**: the difference in predicted friendship closeness between High Nice and Low Nice students. The interaction now asks whether that gap is the same size in both Going Out groups.

Here is the idea in formula form. Define:

- **Effect of Niceness for Stay-in students** $= \mu_{21} - \mu_{11}$ (High Nice – Low Nice, among those who stay in)
- **Effect of Niceness for Go-out students** $= \mu_{22} - \mu_{12}$ (High Nice – Low Nice, among those who go out)

The **interaction** is the difference between these two effects:

$$B_3 = (\mu_{22} - \mu_{12}) - (\mu_{21} - \mu_{11})$$

This is the "difference of differences." If there is no interaction, the effect of Niceness is the same size in both Going Out groups, and $B_3 = 0$. If there is an interaction, the effect of Niceness is larger in one group than the other, and $B_3 \neq 0$.

Equivalently (and this is worth pausing on), you can write it the other way:

$$B_3 = (\mu_{22} - \mu_{21}) - (\mu_{12} - \mu_{11})$$

This version asks: is the effect of Going Out (Go out – Stay in) the same for High Nice and Low Nice students? The two formulations are mathematically identical and represent the same interaction. Which framing you use depends on which story you want to tell. In our study, we are primarily interested in whether the **benefit of being nice depends on whether you go out**, so we will usually use the first framing.

17.3.2 Visual Hypotheses: What We Expect to See

As in the prior chapter, we can sketch out our two competing possibilities before running a single model.

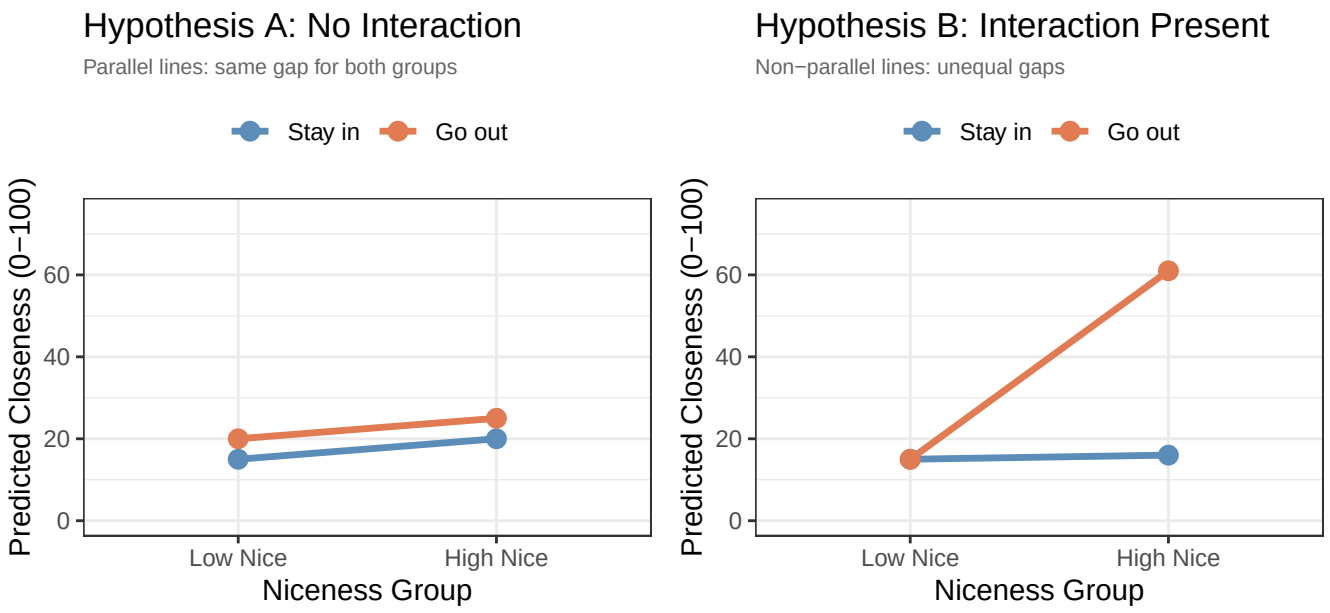


Figure 17.1: The 2 × 2 version of the two hypotheses. With only two levels on the x axis, non-parallel no longer means different slopes; it means differently sized gaps.

Notice the visualization has changed from the prior chapter. Instead of regression lines across a continuous X-axis, we now have two points connected by a line for each Going Out group. The key visual signature of an interaction is still **non-parallel lines**. But "non-parallel" now means the **gaps between the two groups are different sizes** across the niceness levels, rather than different slopes.

- In **Hypothesis A**, the two lines are parallel: the gap between "Go out" and "Stay in" is the same for Low Nice and High Nice students. Being nice adds a fixed amount regardless of going out.
- In **Hypothesis B**, the lines fan out: the gap between "Go out" and "Stay in" is tiny for Low Nice students but enormous for High Nice students. This is our interaction hypothesis: being nice only pays off substantially if you are also out there meeting people.

17.4 Before We Run the Models: Dummy Coding Both Variables

17.4.1 No Centering Required

In the prior chapter, you had to center the continuous predictor (Being Nice) before building the interaction model. Centering shifted the scale so that zero represented the average person, making the intercept interpretable.

When both predictors are categorical, there is nothing to center. Each predictor already takes on discrete values. The "zero point" for each predictor is simply its **reference category**, whichever group we designate as the baseline.

This is actually simpler than the prior chapter. The only setup step is making sure both variables are properly factored in R, with the reference groups ordered first.

17.4.2 Dummy Coding Both Variables

We have two categorical predictors, each with two levels:

Variable	Levels	Reference group (coded 0)	Comparison group (coded 1)
NiceGroup	Low Nice, High Nice	Low Nice	High Nice
GoingOut	Stay in, Go out	Stay in	Go out

With this coding, the **four coefficients** in the interaction model have the following meanings:

Coefficient	What it represents
B_0 (Intercept)	Predicted closeness for **Low Nice × Stay in** students, the baseline cell
B_1 (NiceGroup: High Nice)	High Nice – Low Nice **among Stay-in students** (simple effect of niceness for stay-in group)
B_2 (GoingOut: Go out)	Go out – Stay in **among Low Nice students** (simple effect of going out for low-nice group)
B_3 (NiceGroup × GoingOut)	The "difference of differences": does the effect of niceness differ by going-out group?

Notice that B_1 and B_2 are no longer main effects in the traditional sense: they are **simple effects**, each representing the effect of one variable *holding the other variable at its reference level*. This is the same logic as in the prior chapter, where B_1 was the slope for the reference group only. Here, B_1 is the niceness gap for the reference going-out group only.

17.4.3 Recovering All Four Cell Means

One of the useful features of the 2×2 interaction model is that you can reconstruct **every cell mean** directly from the coefficients. Here is the formula for each cell:

Cell	Formula	Plain meaning
Low Nice × Stay in	B_0	Baseline
High Nice × Stay in	$B_0 + B_1$	Baseline + niceness effect (Stay in)
Low Nice × Go out	$B_0 + B_2$	Baseline + going-out effect (Low Nice)
High Nice × Go out	$B_0 + B_1 + B_2 + B_3$	All four terms

The final cell is the most important: for High Nice students who also go out, the prediction is the baseline plus the simple effect of niceness, plus the simple effect of going out, **plus the interaction term**. The interaction term B_3 adjusts the prediction for the cell where both variables are at their non-reference level. If there is no interaction ($B_3 = 0$), the effects of the two variables add up independently and the model is purely additive.

17.5 The Data

We return to our 200 first-semester students. Recall that we measured friendship closeness (0–100 feelings thermometer), agreeableness (0–10), and whether they went out regularly.

For this analysis, we focus on the **extremes of the niceness distribution**: students who scored below 2 on the agreeableness scale (near 0 = **Low Nice**) and students who scored above 8 (**High Nice**). Everyone in the middle of the scale is dropped, which is most of the sample, and we keep only the contrast between the most and least agreeable people in it.

> ⚠ Why We Are Allowed to Do This, and You Are Not
>
> I am about to butcher a perfectly good continuous variable, on purpose, in front of you. I am doing it because the 2×2 is how a large share of psychology structures its experiments, and you need to be able to read, run, and interpret one.
>
> Doing this to *real* continuous data is a different matter. Median splits and extreme-groups designs throw away information you paid to collect, shrink your statistical power, and can manufacture effects that are not there (MacCallum et al. 2002). Never do this or I will chase you with my Nerf bat. Notice what happens below: 200 participants become 81. We are discarding well over half the study to make a picture that is easier to draw.
>
> The rule is simple. **If your variable is continuous, use the previous chapter's model.** Categorize when the variable is genuinely categorical, or when someone else already categorized it before you arrived, not because a 2×2 is easier to explain in a talk.

Here is the simulation, including the filter that does the damage. This is the same 200 students and the same seed as the prior chapter, so `Friend.Data.Full` below is identical to the dataset we used there:

```
library(tidyverse)

set.seed(42)
n <- 200
X <- runif(n, 0, 10)
Z <- rbinom(n, 1, 0.5)
Y <- 0.05 * X + 0 * Z + 5 * X * Z + 15 + rnorm(n, sd = 7)
Friend.Data.Full <- data.frame(BeingNice = X, GoingOut = Z, Closeness = Y)
```

```
Friend.Data.Full$GoingOut_ <- factor(Friend.Data.Full$GoingOut,
                                     levels = c(0, 1),
                                     labels = c("Stay in", "Go out"))

# Create niceness group for extremes
Friend.Data.Full$NiceGroup_ <- ifelse(Friend.Data.Full$BeingNice < 2,  "Low Nice",
                                ifelse(Friend.Data.Full$BeingNice > 8, "High Nice", NA))

# Keep only extreme groups
Friend.Data <- Friend.Data.Full |>
  filter(!is.na(NiceGroup_)) |>
  mutate(NiceGroup_ = factor(NiceGroup_, levels = c("Low Nice", "High Nice")))

n_low_stayin  <- sum(Friend.Data$NiceGroup_ == "Low Nice"  & Friend.Data$GoingOut_ == "Stay in")
n_low_goout   <- sum(Friend.Data$NiceGroup_ == "Low Nice"  & Friend.Data$GoingOut_ == "Go out")
n_high_stayin <- sum(Friend.Data$NiceGroup_ == "High Nice" & Friend.Data$GoingOut_ == "Stay in")
n_high_goout  <- sum(Friend.Data$NiceGroup_ == "High Nice" & Friend.Data$GoingOut_ == "Go out")
```

The resulting dataset has 81 participants spread across four cells:

```
Friend.Data |>
  group_by(NiceGroup_, GoingOut_) |>
  summarise(n = n(),
            M_Close  = round(mean(Closeness), 2),
            SD_Close = round(sd(Closeness), 2),
            .groups = "drop")
```

```
# A tibble: 4 x 5
  NiceGroup_ GoingOut_     n M_Close SD_Close
  <fct>      <fct>     <int>   <dbl>    <dbl>
1 Low Nice   Stay in      25    16.0     7.27
2 Low Nice   Go out       16    22.6     4.97
3 High Nice  Stay in      21    19       7.05
4 High Nice  Go out       19    60.3     7.32
```

The cell counts are not equal, which is typical when you extract extremes from a continuous distribution rather than randomly assigning participants to conditions. Unequal cells are no problem for the regression estimates. (Classic ANOVA arithmetic assumes balanced cells; with unequal n the ANOVA world descends into a fight about "Types of sums of squares" that regression coefficients let you skip. If you ever meet Type III SS in the wild, that is what the fight was about, and we will see it happen later in this chapter.)

The cell means already tell an interesting story. Among Low Nice students, going out is worth something but not much: about 16 on the thermometer if they stay in, about 23 if they go out. Among High Nice students the same comparison is enormous, roughly 19 against 60. That is the difference of differences we just defined, sitting in the raw means before we have fit anything, and it matches Hypothesis B.

17.6 A First Look at the Data

Before running any models, let's visualize the four cells directly.

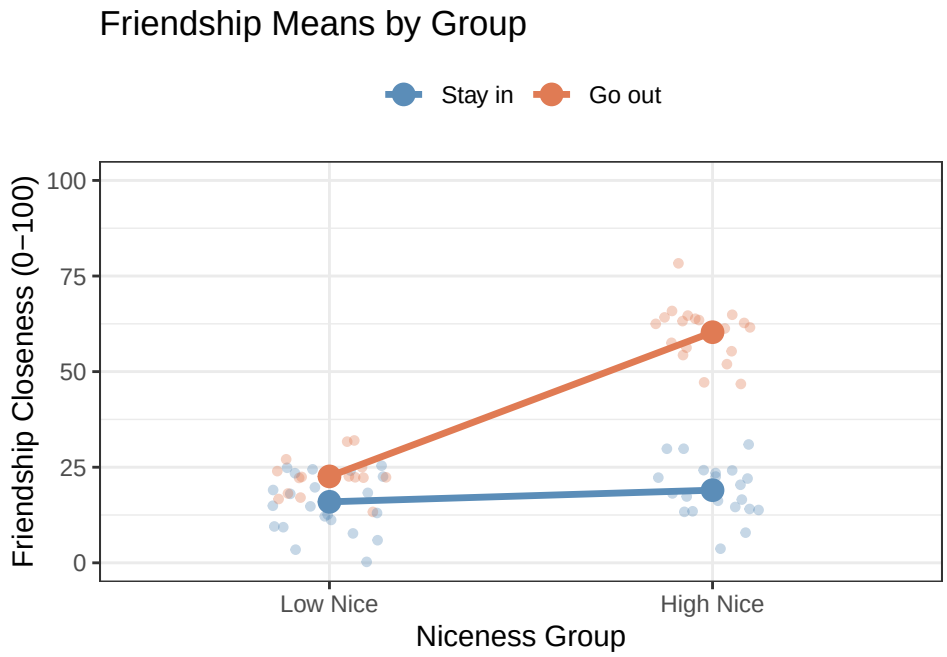


Figure 17.2: The four cells before any model is fit. Jittered raw scores with the observed cell means connected by group.

The pattern is striking. The "Stay in" line (blue) is almost flat: Low Nice and High Nice students who stay in have close to the same closeness scores. The "Go out" line (orange) rises sharply: High Nice students who go out form far closer friendships than Low Nice students who go out. The two lines are clearly **non-parallel**, which is the visual signature of an interaction. This picture already suggests Hypothesis B.

17.7 Building the Models

17.7.1 Model 1: The Additive Model

We begin with the main effects (additive) model. This model says the two variables each have an independent effect on friendship closeness, and those effects add up without interacting.

$$\widehat{Closeness} = B_0 + B_1 \cdot NiceGroup + B_2 \cdot GoingOut + \varepsilon$$

```
Model.1 <- lm(Closeness ~ NiceGroup_ + GoingOut_, data = Friend.Data)
summary(Model.1)
```

```
Call:
lm(formula = Closeness ~ NiceGroup_ + GoingOut_, data = Friend.Data)

Residuals:
     Min       1Q   Median       3Q      Max
-23.4417  -9.7999   0.2044   9.9769  27.0074

Coefficients:
```

```
                    Estimate Std. Error t value Pr(>|t|)
(Intercept)            9.112      1.980   4.602 1.59e-05 ***
NiceGroup_High Nice   18.025      2.465   7.314 1.98e-10 ***
GoingOut_Go out       24.173      2.487   9.718 4.40e-15 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 11.05 on 78 degrees of freedom
Multiple R-squared:  0.674, Adjusted R-squared:  0.6657
F-statistic: 80.64 on 2 and 78 DF,  p-value: < 2.2e-16
```

The additive model estimates a single effect of NiceGroup (averaged across both Going Out groups) and a single effect of GoingOut (averaged across both Niceness groups). These are conventional main effects.

Notice that the R^2 here (0.674) is already reasonably high, driven primarily by the Going Out main effect. But as we will see, forcing the effects to be additive is a poor fit for the actual pattern in the data.

17.7.2 Visualizing the Additive Model

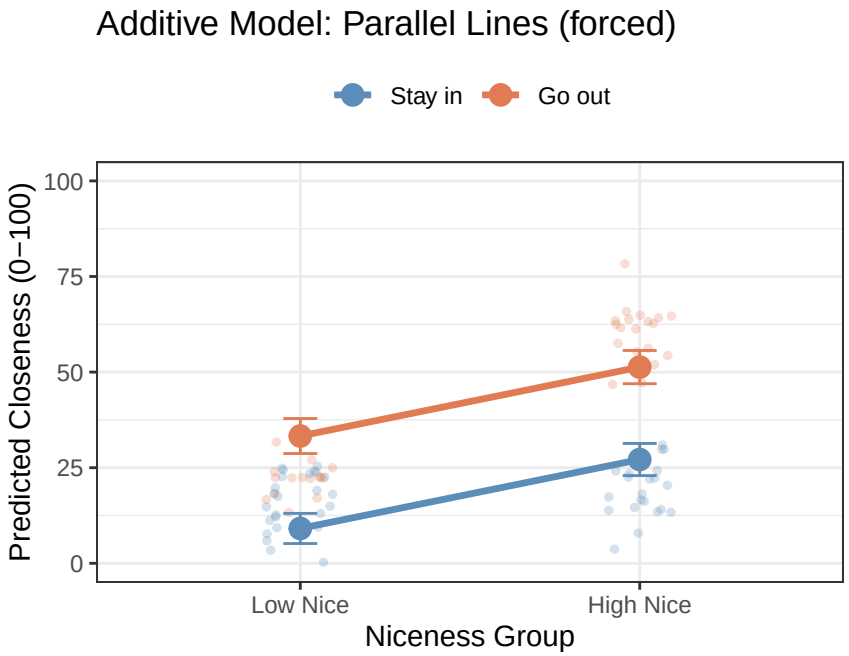


Figure 17.3: The additive model's estimated cell means, with error bars and the raw data behind them. The lines are parallel because the model is not permitted to draw anything else.

The additive model forces the two lines to be parallel. The gap between "Go out" and "Stay in" is identical for Low Nice and High Nice students. But look at the raw data: the "Go out" students in the High Nice group (orange, right side) are far higher than what the parallel-line model predicts for them, and the "Stay in" students in the same group are not as high as the model suggests. The parallel assumption is clearly wrong.

17.7.3 Model 2: The Interaction Model

Now we add the interaction term:

$$\widehat{Closeness} = B_0 + B_1 \cdot NiceGroup + B_2 \cdot GoingOut + B_3 \cdot NiceGroup \times GoingOut + \varepsilon$$

```
Model.2 <- lm(Closeness ~ NiceGroup_ * GoingOut_, data = Friend.Data)
summary(Model.2)
```

```
Call:
lm(formula = Closeness ~ NiceGroup_ * GoingOut_, data = Friend.Data)

Residuals:
     Min       1Q   Median       3Q      Max
-15.7001  -4.7327  -0.0055   4.3511  18.0093

Coefficients:
                                  Estimate Std. Error t value Pr(>|t|)
(Intercept)                         15.950      1.367  11.670  < 2e-16 ***
NiceGroup_High Nice                  3.045      2.023   1.505  0.13632
GoingOut_Go out                      6.649      2.188   3.039  0.00324 **
NiceGroup_High Nice:GoingOut_Go out 34.663      3.077  11.265  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 6.834 on 77 degrees of freedom
Multiple R-squared:  0.8769,    Adjusted R-squared:  0.8721
F-statistic: 182.8 on 3 and 77 DF,  p-value: < 2.2e-16
```

17.7.4 Does the Interaction Model Explain More Variance?

```
knitr::kable(anova(Model.1, Model.2), digits = 3,
             caption = "Model Comparison: Additive vs. Interaction")
```

Table 17.1: Model Comparison: Additive vs. Interaction

Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
78	9522.081	NA	NA	NA	NA
77	3595.951	1	5926.13	126.896	0

```
Additive model R-squared:     0.674

Interaction model R-squared:  0.877

Change in R-squared:          0.203
```

The interaction model explains substantially more variance. The F-test ($\Delta R^2 = 0.203$, $F(1, 77) = 126.9$, $p < .001$) confirms that this improvement is far larger than would be expected by chance. Adding the interaction term was worth it.

17.8 Interpreting the Interaction Model

17.8.1 The Plot First

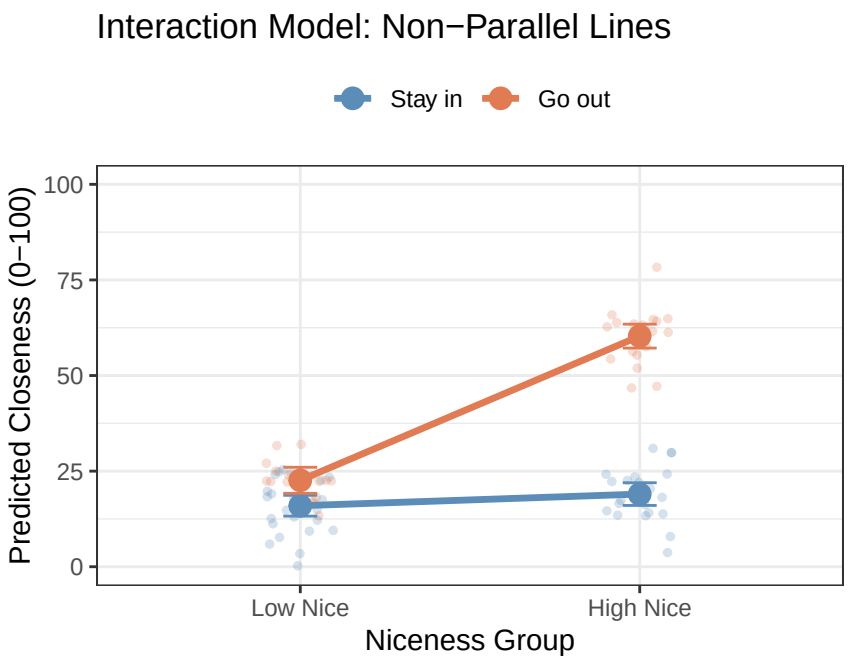


Figure 17.4: The interaction model's estimated cell means. Freed to give each row its own gap, the model reproduces the four observed cell means exactly.

This is the interaction model's predicted means for each cell. Notice the non-parallel lines: the "Stay in" line is nearly flat (very small gap between Low Nice and High Nice), while the "Go out" line rises steeply (a large gap between Low Nice and High Nice). Compare this to the additive model's forced parallel lines. The story told by these two models is completely different.

17.8.2 Walking Through Each Coefficient

```
Call:
lm(formula = Closeness ~ NiceGroup_ * GoingOut_, data = Friend.Data)

Residuals:
     Min       1Q   Median       3Q      Max
-15.7001  -4.7327  -0.0055   4.3511  18.0093

Coefficients:
                                  Estimate Std. Error t value Pr(>|t|)
```

```
(Intercept)                             15.950      1.367  11.670  < 2e-16 ***
NiceGroup_High Nice                      3.045      2.023   1.505  0.13632
GoingOut_Go out                          6.649      2.188   3.039  0.00324 **
NiceGroup_High Nice:GoingOut_Go out     34.663      3.077  11.265  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 6.834 on 77 degrees of freedom
Multiple R-squared:  0.8769,    Adjusted R-squared:  0.8721
F-statistic: 182.8 on 3 and 77 DF,  p-value: < 2.2e-16
```

17.8.3 The Intercept ($B_0 = 15.95$)

The intercept is the predicted closeness score for the **baseline cell**: Low Nice students who Stay in (both predictors at 0). This is a real, specific group of people in the dataset. A thoroughly disagreeable person who never leaves home still lands at about 15.95 points on the friendship thermometer, presumably because they live near someone and physical proximity does the work even for the unsociable.

17.8.4 B_1: Effect of High Niceness Among Stay-in Students ($B_1 = 3.05$)

B_1 is the difference in predicted closeness between High Nice and Low Nice students **within the Stay-in group**. Looking at the "Stay in" line in the plot, this gap is small: $B_1 = 3.05$ points on a 100-point scale, and non-significant.

For students who stay home, being highly agreeable provides virtually no additional benefit for friendship formation compared to being low in agreeableness. Without social exposure, personality differences do not matter much.

This is a **simple effect**: it is the effect of NiceGroup *within one specific level* of GoingOut (the reference level, Stay in). It is not a main effect.

17.8.5 B_2: Effect of Going Out Among Low Nice Students ($B_2 = 6.65$)

B_2 is the difference in predicted closeness between Go-out and Stay-in students **within the Low Nice group**. Students low in agreeableness who go out score about 6.65 points higher on the thermometer than their equally disagreeable counterparts who stay in. Small, but real: this one is significant.

This is also a **simple effect**: the effect of GoingOut *within one specific level* of NiceGroup (the reference level, Low Nice).

17.8.6 B_3: The Interaction Term ($B_3 = 34.66$)

This is the heart of the analysis. B_3 tells you how much **larger** the effect of niceness is for the Go-out group compared to the Stay-in group.

We can reconstruct the simple effect of Niceness for each Going Out group:

Group	Simple effect of Niceness	Formula
Stay in	3.05	B_1
Go out	37.71	$B_1 + B_3 = 3.05 + 34.66$

The difference between these two simple effects is $B_3 = 34.66$. For Go-out students, being highly agreeable is worth about 37.71 more points of closeness than being low in agreeableness. For Stay-in students, the same comparison yields only 3.05 points. The interaction coefficient captures this dramatic difference.

Here is the full cell mean summary:

Cell	Predicted mean	Derivation
Low Nice × Stay in	15.95	B_0
High Nice × Stay in	19	$B_0 + B_1$
Low Nice × Go out	22.6	$B_0 + B_2$
High Nice × Go out	60.31	$B_0 + B_1 + B_2 + B_3$

17.9 Following Up the Interaction: Simple Effects

The interaction term tells us the two niceness gaps are significantly different between Going Out groups. But we also want to know: **within each Going Out group, is the difference between High Nice and Low Nice statistically significant?**

We use `pairs()` from the `emmeans` package to test each pairwise comparison within each level of the other variable.

17.9.1 Simple Effects of Niceness within Each Going-Out Group

```
library(emmeans)

Fit.2 <- emmeans(Model.2, ~ NiceGroup_ | GoingOut_)
pairs(Fit.2, adjust = "none") |> summary(infer = TRUE)
```

```
GoingOut_ = Stay in:
 contrast             estimate   SE df lower.CL upper.CL t.ratio p.value
 Low Nice - High Nice    -3.05 2.02 77    -7.07    0.983  -1.505  0.1363

GoingOut_ = Go out:
 contrast             estimate   SE df lower.CL upper.CL t.ratio p.value
 Low Nice - High Nice   -37.71 2.32 77   -42.33  -33.091 -16.262 <0.0001

Confidence level used: 0.95
```

- **Stay-in group:** The difference between High Nice and Low Nice students is -3.05 points, which is **not** statistically significant ($p = 0.136$). For students who stay in, agreeableness does not significantly predict friendship closeness.
- **Go-out group:** The difference between High Nice and Low Nice students is -37.71 points, which is **highly** significant ($p < .001$). Among students who go out, being highly agreeable is associated with substantially closer friendships than being low in agreeableness.

17.9.2 Effect Sizes: Cohen's *d*

Pairwise comparisons in this framework can be accompanied by Cohen's d as an effect size measure. We use the model's residual standard deviation as the pooled standard deviation:

```
eff_size(Fit.2,
         sigma = sigma(Model.2),
         edf   = df.residual(Model.2),
         method = "pairwise")
```

```
GoingOut_ = Stay in:
 contrast             effect.size    SE df lower.CL upper.CL
 Low Nice - High Nice      -0.446 0.298 77    -1.04    0.148

GoingOut_ = Go out:
 contrast             effect.size    SE df lower.CL upper.CL
 Low Nice - High Nice      -5.518 0.559 77    -6.63   -4.404

sigma used for effect sizes: 6.834
Confidence level used: 0.95
```

The Cohen's d values confirm the pattern:

- **Stay in:** $d = 0.45$, which by Cohen's rough conventions is a small-to-medium effect, and which our test could not distinguish from zero at this sample size. Note that these two statements are not the same statement, and the gap between them is the whole reason we threw away 119 people. With 46 stay-in students, an effect this size is exactly the kind that goes undetected.
- **Go out:** $d = 5.52$. That is not a typo, and "large" does not really cover it. Cohen's benchmark for large is 0.8; this is over five standard deviations. Effects like this do not happen in real personality research, they happen in simulated data where the author picked the parameters, and you should be suspicious of any real paper reporting one.

17.10 APA Write-Up

Here is how you would report these results following APA style. The structure parallels the prior chapter: model comparison, interaction coefficient, then simple effects.

Cell means and standard deviations are presented in Table 1. Among students who stayed in, friendship closeness was similar regardless of agreeableness group (Low Nice: $M = 16$, $SD = 7.3$; High Nice: $M = 19$, $SD = 7.1$). Among students who went out, High Nice students formed substantially closer friendships than Low Nice students (Low Nice: $M = 22.6$, $SD = 5$; High Nice: $M = 60.3$, $SD = 7.3$).

A hierarchical multiple regression was conducted to examine whether social engagement (Going Out) moderated the relationship between agreeableness group (Low vs. High Nice) and friendship closeness. Niceness group and Going Out (both dummy coded with Low Nice and Stay in as reference groups) were entered in Step 1 as main effects; their product was entered in Step 2. The additive model (Step 1) was statistically significant, $F(2, 78) = 80.64$, $p < .001$, $R^2 = 0.674$. Adding the interaction term (Step 2) produced a statistically significant improvement in model fit, $\Delta R^2 = 0.203$, $F(1, 77) = 126.9$, $p < .001$, indicating that the effect of agreeableness on friendship closeness differed significantly depending on whether students went out.

In the full interaction model (R^2 = 0.877, adjusted R^2 = 0.872), the interaction between Niceness Group and Going Out was statistically significant, b = 34.66, SE = 3.08, $t(77)$ = 11.26, $p < .001$. To interpret the interaction, simple effects of Niceness Group were examined separately within each level of Going Out. Among students who stayed in, agreeableness group did not significantly predict friendship closeness, b = -3.05, SE = 2.02, $t(77)$ = -1.51, p = 0.136, d = 0.45. Among students who went out, High Nice students formed significantly closer friendships than Low Nice students, b = -37.71, SE = 2.32, $t(77)$ = -16.26, $p < .001$, d = 5.52. These results indicate that the benefit of high agreeableness for friendship formation is conditional on social engagement: being a warm and agreeable person leads to substantially closer friendships only among students who are regularly out meeting others.

Things to notice about this write-up:

- We report **Cohen's *d*** alongside the simple effects, since the comparisons are between two categorical groups. This is the appropriate effect size for a two-group mean difference.
- The **model comparison** (ΔR^2, F-test) is still the primary test of the interaction, exactly as in the prior chapter.
- We describe the interaction in plain language: *"the benefit of high agreeableness ... is conditional on social engagement."*
- The **main effects** from the interaction model are not heavily emphasized. In the presence of a significant interaction, they represent effects at the reference level of the other variable (simple effects), not population-level averages.

17.11 APA-Style Figure

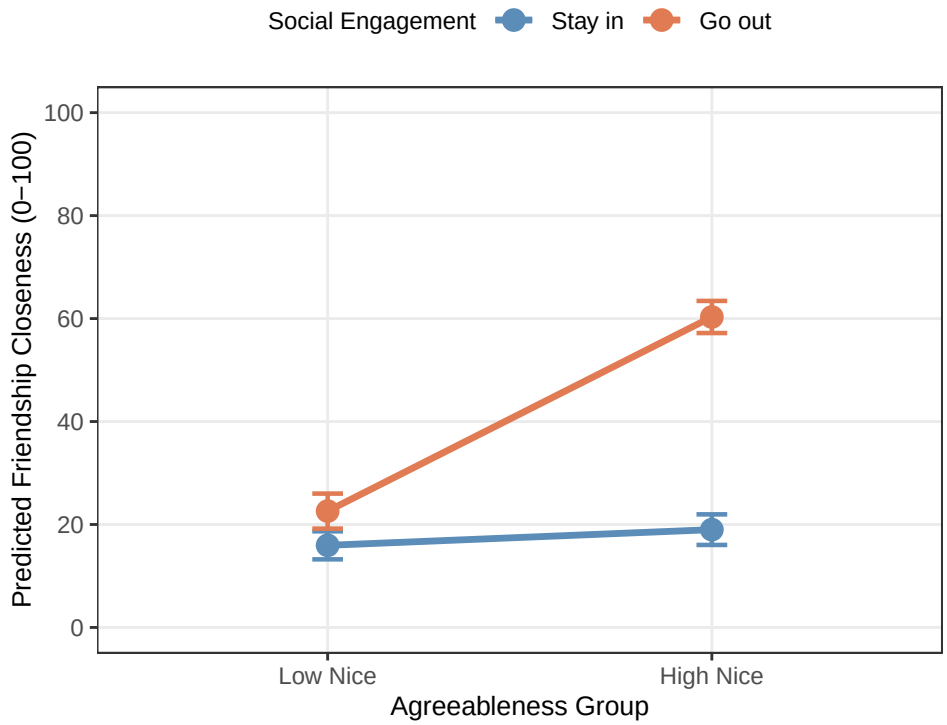
Figure 1. Predicted friendship closeness as a function of agreeableness group and social engagement. Points represent model-estimated cell means; error bars represent 95% confidence intervals. Among students who went out (*Go out*; orange), highly agreeable students formed substantially closer friendships than less agreeable students (d = 5.52). Among students who stayed in (*Stay in*; blue), agreeableness group was not significantly associated with friendship closeness (d = 0.45).

```
# APA interaction plot: categorical × categorical
library(ggplot2)

Fig.Data <- as.data.frame(emmeans(Model.2, ~ NiceGroup_ | GoingOut_))

ggplot(Fig.Data, aes(x = NiceGroup_, y = emmean,
                     color = GoingOut_, group = GoingOut_)) +
  # Confidence intervals
  geom_errorbar(aes(ymin = lower.CL, ymax = upper.CL),
                width = 0.08, linewidth = 0.8) +
  # Connecting lines
  geom_line(linewidth = 1.2) +
  # Cell mean points
  geom_point(size = 3.5) +
  scale_color_manual(values = c("Stay in" = "#5b8db8",
                                "Go out"  = "#e07b54"),
                     labels = c("Stay in", "Go out")) +
  scale_y_continuous(limits = c(0, 100),
                     breaks = seq(0, 100, 20)) +
  labs(x     = "Agreeableness Group",
       y     = "Predicted Friendship Closeness (0-100)",
       color = "Social Engagement") +
```

```
  theme_bw() +
  theme(legend.position   = "top",
        legend.title      = element_text(size = 9),
        legend.text       = element_text(size = 9),
        axis.title        = element_text(size = 10),
        panel.grid.minor  = element_blank())
```



17.12 The ANOVA Costume: A Sheep in Wolf's Clothing

I have been rude about ANOVA (Analysis of Variance) since the preface, so it is only fair to introduce you properly, once, so you know what it is when someone inflicts it on you.

Everything we just did has a second name. Run the exact same model through `anova()` and it comes back dressed as a two-way factorial ANOVA:

```
anova(Model.2)
```

```
Analysis of Variance Table

Response: Closeness
                     Df  Sum Sq Mean Sq F value    Pr(>F)
NiceGroup_            1  8158.5  8158.5  174.70 < 2.2e-16 ***
GoingOut_             1 11529.4 11529.4  246.88 < 2.2e-16 ***
NiceGroup_:GoingOut_  1  5926.1  5926.1  126.90 < 2.2e-16 ***
Residuals            77  3596.0    46.7
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

That is the table a paper means when it says "a 2 × 2 ANOVA revealed a significant interaction." Same data, same model, same `Model.2` object we have had this whole time. Nothing was refit.

Look at the interaction row. $F = 126.9$. Now square the t for B_3 from the regression output: $t = 11.265$, and $t^2 = 126.9$. These are the same number, because they are the same test. The ANOVA table is not giving you a second opinion. It is giving you the first opinion in a different font.

17.12.1 The Sums of Squares Fight, and Why Everyone Says "Type III"

Earlier I promised you would meet Type III sums of squares. Here it is, and our unequal cells are what make it visible.

When cells are unbalanced, the predictors share variance, and "how much variance does Niceness Group explain?" stops having one answer. It has three, depending on what you insist the other terms get credit for first.

- **Type I** (sequential) is what `anova()` gave you above. Terms are tested in the order you typed them, and each one is adjusted only for the terms above it. The first term listed gets first claim on any shared variance. Type the predictors in a different order and you get a different table, which should tell you everything you need to know about relying on it.
- **Type II** adjusts each main effect for the other main effect, but ignores the interaction when testing main effects.
- **Type III** adjusts every effect for *every* other effect in the model, interaction included. Each term is asked what it contributes last, after everybody else has already taken what they wanted.

Type III is the one psychology uses. Here is why. For about thirty years the discipline ran its analyses in SPSS, whose `GLM` procedure defaults to Type III and does not make a fuss about it. Most researchers never chose Type III; SPSS chose it for them, and nobody was told there had been a decision. That default is now baked into the literature: when you read "$F(1, 76) = 4.31$" in a psychology paper from roughly 1990 onward, it is almost certainly Type III, and the paper will almost certainly not say so. R makes you ask for it out loud, which feels like R being difficult but is really R declining to make a methodological choice on your behalf.

> ⚠ Type III Without Effect Coding Is Nonsense, and R Will Not Warn You
>
> This is the trap that catches nearly everyone, including me, earlier in this very chapter.
>
> Type III main-effect tests are only meaningful if your categorical predictors use **effect coding** (`contr.sum`), where the levels are coded −1 and +1 around a midpoint. R's default is **dummy coding** (`contr.treatment`), coded 0 and 1, which is what makes B_1 and B_2 interpretable as simple effects, and which is exactly what we have wanted all chapter.
>
> Run `Anova(model, type = 3)` on a dummy-coded model and R returns a table without complaint. The numbers are simply not main effects. On our data it reports Niceness Group at $F = 2.27$ and Going Out at $F = 9.24$, which look like a null result and a small effect. They are neither. They are the *simple effects* B_1 and B_2 in disguise: square the t values from the regression output and you get 2.27 and 9.24, the same two numbers. Type III asked "what does niceness do at zero on the other variable", and with dummy coding zero means the Stay-in group, not the average.
>
> So refit with `contr.sum` before asking for Type III. The interaction row is unaffected either way, because the interaction is already the last term in and has nothing left to be adjusted for.

```
library(car)

# Type III needs effect coding (-1/+1), not R's default dummy coding (0/1)
Model.2.sum <- lm(Closeness ~ NiceGroup_ * GoingOut_, data = Friend.Data,
                  contrasts = list(NiceGroup_ = contr.sum,
                                   GoingOut_  = contr.sum))
```

```r
Anova(Model.2.sum, type = 3)
```

```
Anova Table (Type III tests)

Response: Closeness
                     Sum Sq Df F value    Pr(>F)
(Intercept)           68505  1 1466.89 < 2.2e-16 ***
NiceGroup_             8191  1  175.40 < 2.2e-16 ***
GoingOut_             11345  1  242.94 < 2.2e-16 ***
NiceGroup_:GoingOut_   5926  1  126.90 < 2.2e-16 ***
Residuals              3596 77
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

All three types, side by side:

Term	Type I (`anova`)	Type II	Type III (psychology's default)
NiceGroup	174.7	139.83	175.4
GoingOut	246.88	246.88	242.94
NiceGroup × GoingOut	126.9	126.9	126.9

Two things to take from that table.

The interaction never moves. 126.9 in all three columns, and it would be 126.9 under any other scheme somebody invents next year. The highest-order term is always adjusted for everything below it, so there is nothing left to argue about. The entire sums-of-squares fight is a fight about main effects, and it has never once touched the test you actually came here for.

The main effects move. Niceness Group is 174.7 under Type I, 139.83 under Type II, and 175.4 under Type III. Same data, same model, same software, three answers. Notice that they do not fan out neatly: Type I and Type III land close together here while Type II sits well below both, and that ordering is a fact about this particular unbalanced dataset rather than a rule you can carry to the next one. All three are arithmetically correct. They are answers to three different questions about who gets credit for shared variance, and which question you asked is invisible in a published table that says only "$F =$" and a number.

The regression coefficients never had this problem. B_1 was always the niceness gap within the Stay-in group, unambiguously, and nobody had to pick a Roman numeral or set a contrast option to find that out.

This is what I mean when I say regression scales and ANOVA does not. It is the same model either way; one of them just makes you argue about it.

Two Doors We Are Walking Past, On Purpose

That `contrasts = list(NiceGroup_ = contr.sum, ...)` argument was doing real work, and I handed it to you without explaining it. Two things sit behind it, and both are graduate material rather than 343 material.

Coding schemes. Dummy coding (0/1) and effect coding (−1/+1) are two members of a family. You can hand a regression *any* set of contrast weights, which lets you test specific theoretical comparisons directly as coefficients instead of running an omnibus test and then fishing through pairwise follow-ups. That is how ANOVA's "planned contrasts" work, and it is all just regression with a different design matrix.

Anything bigger than a 2 × 2. Everything in this chapter leaned on both variables having exactly two levels, which is what let a single coefficient B_3 *be* the interaction. Go to a 3 × 2 and that stops being true. A three-level factor enters as two columns, not one; the interaction becomes two coefficients, not one, with $(a-1)(b-1)$ degrees of freedom; there is no single "difference of differences" number to report; and your follow-up comparisons multiply fast enough that you have to start controlling for how many you ran.

Both live in the advanced contrasts chapter. If you are an undergraduate, you are done here and you may leave happy.

17.13 Comparison: Continuous × Categorical (the prior chapter) vs. Categorical × Categorical (This chapter)

Here are the two approaches side by side.

Feature	Prior chapter (Continuous × Categorical)	This chapter (Categorical × Categorical)
Being Nice	Continuous (0–10)	Categorical (Low vs. High)
Centering	Required	Not needed
Interaction coefficient	Difference in slopes	Difference of differences
Follow-up	`emtrends()`, simple slopes	`pairs()`, simple effects
Effect size	Unstandardized slope (b)	Cohen's d
Sample used	All 200 participants	~80 (extremes only)

The categorical version uses fewer participants (only the extremes of the niceness distribution), which means less statistical power. The continuous version squeezes more information out of the data because it uses every participant and models the full gradient of niceness effects.

However, the categorical × categorical design is conceptually simpler and maps directly onto the two-way factorial ANOVA framework that many researchers in psychology use. Understanding both is essential.

An Interaction Is a Difference of Differences

First compare the groups inside one condition. Then compare them inside the other condition. The interaction asks whether those two differences are different. If you skip that last comparison, you have described two results but have not tested an interaction.

17.14 The Short Story

- **A 2 × 2 is four cells and four coefficients**, and the four coefficients rebuild the four cell means exactly. B_0, $B_0 + B_1$, $B_0 + B_2$, $B_0 + B_1 + B_2 + B_3$.

- B_0 **is the reference cell**, not a grand mean. It is one specific corner of the design: both variables at their reference level.
- B_1 **and** B_2 **are simple effects, not main effects.** Each is the effect of one variable *at the reference level of the other*. Move the reference level and these numbers change.
- B_3 **is the difference of differences, and it is the test.** How much bigger is the niceness gap for the go-out group than for the stay-in group? Nothing else in the table answers that.
- **Follow up with `pairs()` within levels, and report Cohen's *d*.** Simple effects say what the interaction looks like; they do not establish that it exists. The ΔR^2 F-test does that, first.
- **It is all the same model.** `anova()` redraws the identical fit as a factorial ANOVA, and the interaction F is just the B_3 t squared.

And the punchline from the comparison table, because it is the thing most worth carrying out of these two chapters: **the continuous version uses all of your data.** We got a cleaner-looking 2×2 here by throwing away 119 of 200 students. If your predictor is continuous, keep it continuous and use the previous chapter's model. The 2×2 is a design you should be able to read fluently, not a shape you should force your data into.

18 Mixed Regression: Between People and Within People

18.1 Your Paired *t*-Test Was a Mixed Model All Along

The paired-samples *t*-test chapter gave 48 students one lecture with chocolate and one without, then measured how long their eyes stayed on the screen. Here is that study again, same seed and same 48 students, so this chapter runs from the top without sending you back a chapter to fetch the data.

```
set.seed(343)

n <- 48

# Each student has a personal tendency to stare that follows them into BOTH
# conditions. That shared tendency is the entire reason pairing works.
participant_effect <- rnorm(n, mean = 0, sd = 1.20)

paired_wide <- data.frame(
  participant  = factor(seq_len(n)),
  no_chocolate = 5.00 + participant_effect + rnorm(n, 0, 0.80),
  chocolate    = 5.75 + participant_effect + rnorm(n, 0, 0.80)
)

paired_wide$difference <-
  paired_wide$chocolate - paired_wide$no_chocolate

# 48 rows become 96: one row per measurement instead of one row per person.
paired_long <- tidyr::pivot_longer(
  paired_wide,
  cols      = c(no_chocolate, chocolate),
  names_to  = "condition",
  values_to = "fixation"
)

paired_long$condition <- factor(
  paired_long$condition,
  levels = c("no_chocolate", "chocolate"),
  labels = c("No chocolate", "Chocolate")
)
```

There are two ways to analyze this. The first is the one you already know. Collapse each student down to a single difference score and ask whether those differences average to zero:

```r
paired_result <- t.test(
  paired_wide$chocolate,
  paired_wide$no_chocolate,
  paired = TRUE
)

paired_result
```

```
    Paired t-test

data:  paired_wide$chocolate and paired_wide$no_chocolate
t = 4.3366, df = 47, p-value = 7.597e-05
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 0.4209697 1.1495099
sample estimates:
mean difference
      0.7852398
```

The second way never computes a difference at all. It hands R all 96 measurements in long format, plus one piece of information the *t*-test was only ever told indirectly, through the row order: which measurements came from the same human.

```r
library(lme4)      # fits the mixed model
library(lmerTest)  # adds degrees of freedom and p-values to the fixed effects

chocolate_model <- lmer(
  fixation ~ condition + (1 | participant),
  data = paired_long
)

summary(chocolate_model)
```

```
Linear mixed model fit by REML. t-tests use Satterthwaite's method [
lmerModLmerTest]
Formula: fixation ~ condition + (1 | participant)
   Data: paired_long

REML criterion at convergence: 297.6

Scaled residuals:
     Min       1Q   Median       3Q      Max
-1.87123 -0.50482 -0.02554  0.43379  2.44023

Random effects:
 Groups      Name        Variance Std.Dev.
 participant (Intercept) 0.6453   0.8033
 Residual                0.7869   0.8871
```

```
Number of obs: 96, groups:  participant, 48

Fixed effects:
                   Estimate Std. Error      df t value Pr(>|t|)
(Intercept)          4.7660     0.1727 78.1367  27.591  < 2e-16 ***
conditionChocolate   0.7852     0.1811 47.0000   4.337  7.6e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Correlation of Fixed Effects:
            (Intr)
condtnChclt -0.524
```

Find the `conditionChocolate` row in that output. Then look back at the *t*-test.

```r
choc_fixed <- summary(chocolate_model)$coefficients

comparison <- data.frame(
  method   = c("paired t-test", "lmer"),
  t        = c(unname(paired_result$statistic),
               choc_fixed["conditionChocolate", "t value"]),
  df       = c(unname(paired_result$parameter),
               choc_fixed["conditionChocolate", "df"]),
  p        = c(paired_result$p.value,
               choc_fixed["conditionChocolate", "Pr(>|t|)"]),
  estimate = c(unname(paired_result$estimate),
               choc_fixed["conditionChocolate", "Estimate"])
)

print(comparison, digits = 6, row.names = FALSE)
```

```
        method       t df           p estimate
 paired t-test 4.33661 47 7.59724e-05  0.78524
          lmer 4.33661 47 7.59724e-05  0.78524
```

Those are not two answers that happen to agree. They are one answer, printed twice.

The degrees of freedom are worth a second look, because the two rows got to 47 by completely different routes. The *t*-test knows it has 48 differences and subtracts one. `lmer` has no idea what a difference score is; `lmerTest` works out how much independent information the 96 rows actually contain, using an approximation called **Satterthwaite's method**, and arrives at 47 anyway.

18.1.1 What `(1 | participant)` Did

That term is the whole difference between the two calls. It tells R that rows sharing a participant number came from the same person, and that people arrive at a study with their own resting level of staring. Some students watch the screen. Some students watch the door.

It handles this without estimating 48 separate student coefficients. It estimates the *spread* of the students, a single number, and treats each person's own level as a draw from it:

```
print(VarCorr(chocolate_model), comp = c("Variance", "Std.Dev."))
```

```
 Groups      Name        Variance Std.Dev.
 participant (Intercept) 0.64532  0.80332
 Residual                0.78689  0.88707
```

The `participant` line is how much students differ from each other. The `Residual` line is how much one student bounces around their own level from one measurement to the next. Those are the two piles the 96 numbers get sorted into, and we can ask what fraction of the variance sits in the first pile:

```
choc_var <- as.data.frame(VarCorr(chocolate_model))
icc <- choc_var$vcov[1] / sum(choc_var$vcov)

icc
```

```
[1] 0.4505776
```

So roughly 45 percent of the variation in fixation time is students being different students, before chocolate enters the discussion. The paired t-test dealt with that variation by subtracting it away and never mentioning it again. The mixed model estimates it and prints it.

That proportion is also the model's answer to a question the paired t-test chapter asked directly: how much do a person's two measurements agree? That chapter measured the agreement head-on, as a plain correlation of 0.46, and called it the r in the $-2rS_1S_2$ term that everything depended on. The two numbers are close without being identical, because they are two different estimates of the same thing: one computed directly from the pairs, one implied by the variances the model fit. The point is that the quantity has stopped being a term in a formula and become a parameter with a variance attached.

"Each person is their own control" is a sentence from a methods lecture. `(1 | participant)` is that sentence written as an equation.

18.1.2 The Fixed Effect Is the Chocolate Effect

The **fixed effects** are the part of the model that answers the research question, and there are only two:

- `(Intercept)`, 4.77 seconds, is average fixation time with no chocolate.
- `conditionChocolate`, 0.79 seconds, is what chocolate adds on top of that.

The second one is not merely close to the t-test's mean difference. It is the mean difference, 0.79 seconds, reached from the opposite direction: the t-test computed 48 differences and averaged them, while the model never computed a single difference and got there anyway.

Fixed effects answer the research question. Random effects are how the model earns the right to answer it using 96 rows that came from 48 people.

18.1.3 So What Was the Point of All That?

Nothing yet, honestly. With two measurements and no other predictors, the paired t-test is shorter to type and gives the same answer, so keep using it. The mixed model has not earned anything.

It starts paying off the moment the design acquires something a difference score cannot hold:

- **A between-person predictor.** Treatment group, age, baseline severity. Subtracting within a person throws the person away, and group is a property of the person.
- **A missing row.** A student who skipped a session has no difference score at all, so the t-test deletes the entire student.
- **Three or more occasions.** There stops being such a thing as *the* difference.
- **People who change at different *rates*,** not merely by different amounts. A difference score has nowhere to put a slope.

The rest of this chapter does the first two. The last two need a bigger model than this chapter builds, and they get their own chapter later in the book.

> **i** Where This Chapter Goes, and Where It Stops
>
> Two stops, and you just finished the first. Next is the **mixed 2-by-2**: one within-subjects factor, one between-subjects factor, four cells, one interaction. That is the design most repeated-measures studies in psychology actually run, and it is where this chapter stops.
>
> A mixed design combines a **between-subjects** variable with a **within-subjects** variable, which summons fixed effects, an interaction, repeated observations, and random effects all at once. It is not impossible. It is merely statistics arriving with all its relatives and expecting you to provide chairs.

18.2 The Mixed 2-by-2: Baseline versus Week 6, by Group

Suppose researchers compare cognitive behavioral therapy (**CBT**) with a control condition for reducing depression symptoms. Participants are randomly assigned to CBT or control and complete a depression inventory twice: at baseline and again after six weeks.

- **Occasion** is within subjects: every person appears at baseline and at Week 6.
- **Group** is between subjects: each person is in CBT *or* control, never both.
- **Depression score** is the quantitative outcome.

You have seen this shape before. The categorical-by-categorical interaction chapter had two categorical predictors and four cells. So does this:

	Control	**CBT**
Baseline	μ_{11}: Control at baseline	μ_{12}: CBT at baseline
Week 6	μ_{21}: Control at Week 6	μ_{22}: CBT at Week 6

The research question is not whether the groups differ, and not whether scores change. It is whether they change **differently**:

$$B_3 = (\mu_{22} - \mu_{12}) - (\mu_{21} - \mu_{11})$$

Find the change in CBT. Find the change in Control. Compare the two changes. That is the same **difference of differences** the earlier chapter spent itself on, and it has not changed jobs here.

18.2.1 What Stays the Same, and the One Thing That Changes

Feature	The earlier categorical 2-by-2	This mixed 2-by-2
Categorical predictors	Two	Two
Cells	Four	Four
The interaction	Difference of differences	Difference of differences
Who fills the cells	Different people in every cell	The same person fills two of them
The model	`lm(Y ~ A * B)`	the same, plus a participant random intercept

The fixed part of the formula does not change at all. We add the random intercept because the two rows from one person are related, and pretending they came from strangers would count one human twice and let the model become confident in ways it has not earned.

18.2.2 The Data

This is simulated data. Real treatment studies require clinical expertise, ethical oversight, careful monitoring, informed consent, and substantially more planning than "Alex found `rnorm()`."

```
set.seed(3436)

n_per_group <- 50

people <- data.frame(
  participant = factor(seq_len(2 * n_per_group)),
  group = factor(
    rep(c("Control", "CBT"), each = n_per_group),
    levels = c("Control", "CBT")
  ),
  # each person's own baseline level, and their own rate of change
  person_intercept = rnorm(2 * n_per_group, 0, 5.0),
  person_slope     = rnorm(2 * n_per_group, 0, 0.45)
)

# every person gets a baseline row and a Week 6 row
mixed_data <- merge(people, data.frame(week = c(0, 6)), by = NULL)
mixed_data <- mixed_data[order(mixed_data$participant,
                               mixed_data$week), ]

mixed_data$depression <-
  35 +
  mixed_data$person_intercept +
  (-0.20 * mixed_data$week) +                    # control drifts down slowly
  ifelse(mixed_data$group == "CBT",
         -1.20 * mixed_data$week, 0) +           # CBT drops faster: the effect
```

```
  mixed_data$person_slope * mixed_data$week +
  rnorm(nrow(mixed_data), 0, 2.0)

# the same variable twice: week as a number, occasion as a readable label
mixed_data$occasion <- factor(
  mixed_data$week,
  levels = c(0, 6),
  labels = c("Baseline", "Week 6")
)
```

The data are **long**: one row per person per occasion, 200 rows from 100 people. The participant identifier repeats because the person repeats. If your participant column disappears, stop. Do not proceed directly to the ritual sacrifice.

Notice that the simulation gave every person their own `person_slope`, so the invented humans genuinely do improve at different rates, not merely by different amounts. Measured twice, nobody can tell the difference. The last section of this chapter is about designs where you would want to.

18.2.3 Look at the Four Cells First

```
aggregate(
  depression ~ occasion + group,
  data = mixed_data,
  FUN = function(x) c(M = mean(x), SD = sd(x))
)
```

```
  occasion   group depression.M depression.SD
1 Baseline Control    35.793859      5.832925
2   Week 6 Control    33.862191      6.672687
3 Baseline     CBT    34.224262      5.804333
4   Week 6     CBT    26.540161      6.450846
```

```
library(ggplot2)

group_means <- aggregate(
  depression ~ occasion + group,
  data = mixed_data,
  FUN = mean
)

ggplot(
  mixed_data,
  aes(x = occasion, y = depression, group = participant, color = group)
) +
  geom_line(alpha = 0.15) +
  geom_line(
    data = group_means,
    aes(group = group), linewidth = 1.4
  ) +
```

```
  geom_point(
    data = group_means,
    aes(group = group), size = 2.8
  ) +
  labs(
    x = NULL,
    y = "Depression score",
    color = "Group",
    title = "Group averages sitting on top of actual humans"
  ) +
  theme_minimal(base_size = 11)
```

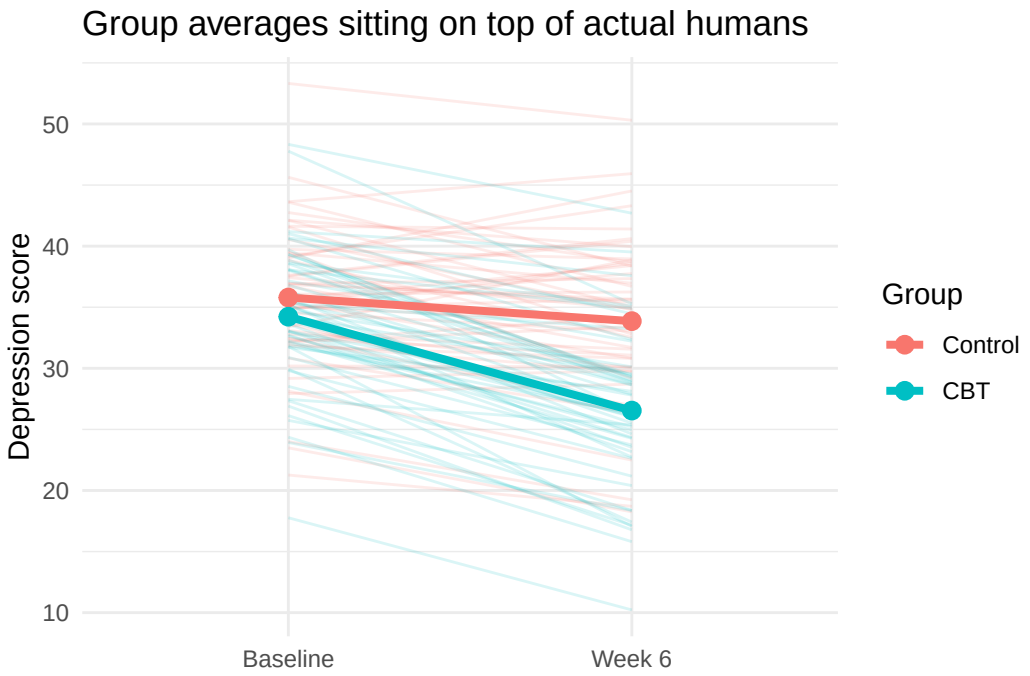


Figure 18.1: The four cells before any model. Each faint line is one participant's baseline and Week 6 score. The heavy lines connect the group means. Non-parallel lines are the visual signature of an interaction, and here that means unequal change.

The two groups start at nearly the same place, which is what randomization is supposed to buy. By Week 6 the CBT mean has dropped much further. But people do not all begin at the same score or change by the same amount. Some improve a lot, some barely, and some appear to have received therapy from a motivational poster in a dentist's office.

Never decide whether an effect is statistically significant by staring at whether confidence intervals overlap. CI overlap is not the direct test of a difference, and interaction tests are about **differences between differences**. Fit the model that asks the question.

18.3 Why Ordinary Regression Is Not Enough

We could hand those four cells to `lm()` and it would answer without complaint:

```r
ordinary_model <- lm(depression ~ occasion * group, data = mixed_data)
```

That model treats all 200 rows as independent. They are not. Two rows belong to each participant, and two scores from the same person know each other, which is the identity theft the paired t-test chapter warned about.

This structure is called **nesting**:

occasions nested within participants

The levels are:

- Level 1: repeated occasions (j);
- Level 2: participants (i).

Group belongs to Level 2 because it does not change within a person. Occasion belongs to Level 1 because it does.

We will come back to `ordinary_model` once the mixed model is fit, because the comparison is more interesting than the warning.

18.4 The Fixed Effects

The fixed part is ordinary regression on the four cells:

$$Y_{ij} = b_0 + b_1(\text{Week 6}_{ij}) + b_2(\text{CBT}_i) + b_3(\text{Week 6}_{ij} \times \text{CBT}_i) + \text{random part} + e_{ij}$$

Because Baseline and Control are the reference levels, each coefficient is one step around the 2-by-2:

Coefficient	What it estimates
b_0	Mean depression for Control at baseline
b_1	Week 6 minus baseline, within Control
b_2	CBT minus Control, at baseline
b_3	How much *more* CBT changed than Control

The CBT change is $b_1 + b_3$. The interaction b_3 is the treatment question, and if it is negative then CBT scores fell further than Control scores.

Notice why coding matters. A "main effect of Group" is not a universal average group difference here. It is the group difference *at baseline*, before anybody has been treated, which in a randomized trial we hope is nothing at all. Choosing baseline as the reference is what makes that coefficient answer a question worth asking instead of describing the difference during Week Zero Hundred and Twelve, a period known only to the Probability Gods.

18.5 The Random Intercept

The repeated observations require us to say how people differ. Each participant gets their own starting level:

$$Y_{ij} = b_0 + b_1 \text{Week 6}_{ij} + b_2 \text{CBT}_i + b_3(\text{Week 6}_{ij} \times \text{CBT}_i) + u_{0i} + e_{ij}$$

That u_{0i} is the same `(1 | participant)` that reproduced the paired t-test at the top of the chapter, doing the same job on a larger design: give participants different intercepts, by estimating the variance of those intercepts rather than 100 unrelated participant coefficients.

```
mixed_model <- lmer(
  depression ~ occasion * group + (1 | participant),
  data = mixed_data,
  REML=TRUE
)

summary(mixed_model)
```

```
Linear mixed model fit by REML. t-tests use Satterthwaite's method [
lmerModLmerTest]
Formula: depression ~ occasion * group + (1 | participant)
   Data: mixed_data

REML criterion at convergence: 1164

Scaled residuals:
     Min       1Q   Median       3Q      Max
-1.74720 -0.54546 -0.03384  0.48026  1.76200

Random effects:
 Groups      Name        Variance Std.Dev.
 participant (Intercept) 32.536   5.704
 Residual                 5.927   2.434
Number of obs: 200, groups:  participant, 100

Fixed effects:
                        Estimate Std. Error       df t value Pr(>|t|)
(Intercept)              35.7939     0.8771 114.2475  40.811  < 2e-16 ***
occasionWeek 6           -1.9317     0.4869  98.0000  -3.967 0.000138 ***
groupCBT                 -1.5696     1.2404 114.2475  -1.265 0.208293
occasionWeek 6:groupCBT  -5.7524     0.6886  98.0000  -8.354 4.43e-13 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Correlation of Fixed Effects:
            (Intr) occsW6 grpCBT
occasionWk6 -0.278
groupCBT    -0.707  0.196
occsnW6:CBT  0.196 -0.707 -0.278
```

REML: The Deliberately Oversimplified Version

By default, `lmer()` uses **REML** (*restricted maximum likelihood*) to estimate the model. I am going to hand-wave aggressively here because understanding exactly how REML works requires considerably more statistical machinery than we need.

You already know **OLS**, which chooses regression coefficients to make the residual errors as small as possible. Mixed-effects models have an additional problem: they also have to estimate **variances**, such as how much participant intercepts vary from person to person.

REML is a method designed to get better estimates of those variance components. Very loosely, it tries to estimate the variance **after accounting for the fact that we already used the data to estimate the fixed effects**. That correction matters especially when samples are not enormous.

For now, the practical rule is simple:

When we care about estimating the random-effects structure, we generally use REML. When we later compare models that differ in their fixed effects, we will switch REML off.

That is intentionally an oversimplification. Future Us will deal with the details when Future Us actually needs them.

18.6 Read the Model Without Panic

```
fixed_effects <- summary(mixed_model)$coefficients
round(fixed_effects, 3)
```

```
                       Estimate Std. Error      df t value Pr(>|t|)
(Intercept)              35.794      0.877 114.248  40.811    0.000
occasionWeek 6           -1.932      0.487  98.000  -3.967    0.000
groupCBT                 -1.570      1.240 114.248  -1.265    0.208
occasionWeek 6:groupCBT  -5.752      0.689  98.000  -8.354    0.000
```

Read the rows in order, and check each one against the table of four coefficients above:

1. `(Intercept)`, 35.79, is the Control group's baseline mean.
2. `occasionWeek 6`, -1.93, is how much Control changed over six weeks.
3. `groupCBT`, -1.57, is the baseline difference between groups, and it is not significant, $p = 0.208$. Randomization did its job.
4. `occasionWeek 6:groupCBT`, -5.75, is the interaction.

So CBT scores fell about 5.75 points further than Control scores over the same six weeks. That is the treatment effect. It does not mean CBT lowered every participant's score by that amount. Fixed effects describe an average pattern while the random effects document the inconvenient existence of individuals.

Notice that Control changed too, and significantly, $p < .001$. People in the control condition got better over six weeks. If we had run a one-group study, measured twice, and found that, we would have congratulated ourselves and possibly written a press release. This is the entire reason the control group exists, and the reason the interaction, not the change, is the research question.

> ⚠ The p-Values Are Approximations, and This Is Not Your Engine's Fault
>
> Every column in that table except the estimate itself rests on an approximation. There is no exact sampling distribution waiting for a model like this one, so the numbers in the `df` column were not counted, they were **estimated**, using Satterthwaite's method, like you saw with Welch's t-test. They often come out fractional.
>
> This is a property of mixed models, not of `lmer`. Change engines and the approximation changes costume rather than disappearing: `glmmTMB`, which you will meet in graduate school, fits the same model and reports Wald z statistics with no degrees of freedom at all. Same model. Different plumbing. Same uncertainty.

18.7 What Did the Random Intercept Buy?

The paired t-test chapter ran its analysis wrong on purpose to show what pairing was worth. Do the same thing here. `ordinary_model` is still sitting there, fit by `lm()`, believing it met 200 unrelated people.

```
lm_fixed <- summary(ordinary_model)$coefficients

data.frame(
  term        = rownames(fixed_effects),
  estimate_lm = round(lm_fixed[, "Estimate"], 3),
  estimate_mixed = round(fixed_effects[, "Estimate"], 3),
  SE_lm       = round(lm_fixed[, "Std. Error"], 3),
  SE_mixed    = round(fixed_effects[, "Std. Error"], 3),
  row.names   = NULL
)
```

```
                    term estimate_lm estimate_mixed SE_lm SE_mixed
1            (Intercept)      35.794         35.794 0.877    0.877
2          occasionWeek 6      -1.932         -1.932 1.240    0.487
3                groupCBT      -1.570         -1.570 1.240    1.240
4 occasionWeek 6:groupCBT      -5.752         -5.752 1.754    0.689
```

The estimates are **identical**, to every decimal place shown. Ignoring the pairing did not bias a single coefficient. That is worth saying plainly, because it is the part people get wrong: this is not a story about wrong answers.

The standard errors are a different matter. The interaction's standard error is 1.75 when we pretend the rows are strangers and 0.69 when we admit they are not, roughly 2.5 times larger. And look at what that does to the Control change, `occasionWeek 6`:

```
c(
  p_lm    = round(lm_fixed["occasionWeek 6", "Pr(>|t|)"], 3),
  p_mixed = round(fixed_effects["occasionWeek 6", "Pr(>|t|)"], 5)
)
```

```
   p_lm p_mixed
0.12100 0.00014
```

Same estimate. One analysis finds nothing, the other finds a clear effect. The difference is entirely whether the model was told that the two rows belong to one human.

Now look at the row the random intercept did *not* help: `groupCBT`, where the standard error is the same in both models, out to nine decimal places. That is not a coincidence. It tells you what the random intercept is for. `groupCBT` compares two different sets of people at one occasion. There is no repeated measurement inside that comparison, so there is nothing to subtract out, and it has to pay full price for person-to-person variation either way. The random intercept helps *within*-person comparisons. It has nothing to offer a between-person one.

Where did the extra error come from, then? From the people. 85 percent of the variance here is participants differing from one another, and `lm()` has no mechanism for setting that aside, so it lands in the error term and makes every within-person comparison look less trustworthy than it is. The random intercept is how you get it back.

18.8 The Interaction Is Also a *t*-Test on Change Scores

With exactly two occasions we can go back to difference scores after all. Subtract baseline from Week 6 for every person, then compare those change scores between the two groups with an ordinary independent-samples *t*-test, which is a test from Part 1 of this book.

```
change_data <- tidyr::pivot_wider(
  mixed_data[, c("participant", "group", "week", "depression")],
  names_from   = week,
  values_from  = depression,
  names_prefix = "week"
)

change_data$change <- change_data$week6 - change_data$week0

# ref = "CBT" so the contrast runs CBT minus Control, matching the model
change_result <- t.test(
  change ~ relevel(group, ref = "CBT"),
  data = change_data,
  var.equal = TRUE
)

change_result
```

```
    Two Sample t-test

data:  change by relevel(group, ref = "CBT")
t = -8.3542, df = 98, p-value = 4.425e-13
alternative hypothesis: true difference in means between group CBT and group Control is not equal to 0
95 percent confidence interval:
 -7.118870 -4.385998
sample estimates:
    mean in group CBT mean in group Control
            -7.684102             -1.931668
```

```r
b3 <- fixed_effects["occasionWeek 6:groupCBT", ]

change_comparison <- data.frame(
  method = c("t-test on change scores", "mixed-model interaction"),
  t      = c(unname(change_result$statistic), b3["t value"]),
  df     = c(unname(change_result$parameter), b3["df"]),
  p      = c(change_result$p.value,           b3["Pr(>|t|)"]),
  estimate = c(
    mean(change_data$change[change_data$group == "CBT"]) -
      mean(change_data$change[change_data$group == "Control"]),
    b3["Estimate"]
  )
)

print(change_comparison, digits = 6, row.names = FALSE)
```

```
                  method        t df           p estimate
 t-test on change scores -8.35423 98 4.42529e-13 -5.75243
 mixed-model interaction -8.35423 98 4.42529e-13 -5.75243
```

One answer, printed twice, for the second time in this chapter.

So this chapter has now caught the mixed model impersonating two different t-tests. A paired t-test is a mixed model with a random intercept. A mixed 2-by-2 interaction is an independent-samples t-test on change scores. Neither is a coincidence, and neither is a reason to abandon the t-tests: in a complete, balanced 2-by-2 like this one they are easier to run and they are right.

The mixed model is what keeps working when the design stops being complete and balanced, which is the next thing that happens to it.

18.9 Test the Interaction Directly

Coefficient tests are not the only way to ask about the interaction. We can also compare two models: one that lets the groups change differently, and one that does not.

When comparing **fixed effects**, refit both with maximum likelihood. This is the switch the REML callout promised, and this is where we throw it, in both models.

```r
reduced_ml <- lmer(
  depression ~ occasion + group + (1 | participant),
  data = mixed_data,
  REML = FALSE
)

full_ml <- lmer(
  depression ~ occasion * group + (1 | participant),
  data = mixed_data,
  REML = FALSE
)
```

```r
anova(reduced_ml, full_ml)
```

```
Data: mixed_data
Models:
reduced_ml: depression ~ occasion + group + (1 | participant)
full_ml: depression ~ occasion * group + (1 | participant)
           npar    AIC    BIC  logLik -2*log(L)  Chisq Df Pr(>Chisq)
reduced_ml    5 1231.5 1248.0 -610.76    1221.5
full_ml       6 1179.8 1199.5 -583.87    1167.8 53.776  1  2.247e-13 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

This likelihood-ratio test asks whether allowing the groups to change by different amounts improves model fit. It is not the only defensible test; coefficient tests and other approximations are also used. The important point is to match the test to the question and identify the method, not to collect p-values like cursed trading cards.

> **anova() Would Have Caught Us, but Only anova()**
>
> Hand two REML fits to `anova()` and it refits them with maximum likelihood itself, printing `refitting model(s) with ML (instead of REML)` on its way past. It makes the decision visible in your code instead of leaving it to a message you might scroll past. `anova()` is being the adult in the room.

After model comparison, report estimates from the final model fit with REML when appropriate.

18.10 Follow Up the Interaction

The interaction says the two changes differ. It does not say what either change was. For that, estimate Week 6 minus baseline separately within each group:

```r
library(emmeans)

cell_means <- emmeans(mixed_model, ~ occasion | group,
                      lmer.df = "satterthwaite")

contrast(
  cell_means,
  method = list("Week 6 - Baseline" = c(-1, 1)),
  adjust = "none"
)
```

```
group = Control:
 contrast          estimate    SE df t.ratio p.value
 Week 6 - Baseline    -1.93 0.487 98  -3.967  0.0001

group = CBT:
 contrast          estimate    SE df t.ratio p.value
 Week 6 - Baseline    -7.68 0.487 98 -15.782 <0.0001
```

```
Degrees-of-freedom method: satterthwaite
```

These are **simple effects**: how each group changed on its own. Control improved a little, CBT improved a lot, and the interaction is the statement that those two amounts are not the same. Simple effects describe the pattern. They do not replace the interaction test, and reporting only the simple effects is how people accidentally claim an interaction they never tested.

We can also ask for the four cell means the model implies:

```
cell_means
```

```
group = Control:
 occasion emmean    SE  df lower.CL upper.CL
 Baseline   35.8 0.877 114     34.1     37.5
 Week 6     33.9 0.877 114     32.1     35.6

group = CBT:
 occasion emmean    SE  df lower.CL upper.CL
 Baseline   34.2 0.877 114     32.5     36.0
 Week 6     26.5 0.877 114     24.8     28.3

Degrees-of-freedom method: satterthwaite
Confidence level used: 0.95
```

Do not automatically test every possible pair because the software makes it easy. That is how a sensible research question becomes 28 tests and a dead salmon (Bennett et al. 2010). Choose comparisons that answer the theory, and adjust for multiplicity when you conduct a family of tests.

18.11 Reporting the Result

A compact results paragraph could read:

> Depression scores were analyzed with a linear mixed-effects model containing fixed effects of Occasion, Group, and their interaction, plus a participant random intercept. The model was fit by restricted maximum likelihood using `lmer()` from the lme4 package, and p-values were obtained from lmerTest using Satterthwaite's approximation to the denominator degrees of freedom. The groups did not differ at baseline, b = -1.57, p = 0.208. The Occasion by Group interaction was negative, b = -5.75, SE = 0.69, $t(98)$ = -8.35, $p < .001$, indicating that depression declined more from baseline to Week 6 in the CBT condition than in the control condition.

A real report should also provide confidence intervals, the cell means, the exact random-effects structure, software, missing-data handling, and theoretically planned follow-ups. The paragraph above is the skeleton. Please give it organs before submitting it to Reviewer 2.

18.12 Advanced Topics

What follows is the part that matters once the data are real rather than simulated: people miss appointments, models need checking, and designs eventually outgrow the one this chapter can handle. The Short Story at the end covers the whole chapter, both halves.

18.13 Missing Repeated Observations

Mixed models can use participants who are missing some occasions; they do not require deleting every person with one missing row. This is a major advantage over older complete-case repeated-measures procedures, and with two occasions it is the difference between losing a person and losing half of one.

```
set.seed(347)

mixed_missing <- mixed_data
# 10% of the Week 6 visits simply do not happen
gone <- mixed_missing$week == 6 & runif(nrow(mixed_missing)) < 0.10
mixed_missing$depression[gone] <- NA

missing_model <- lmer(
  depression ~ occasion * group + (1 | participant),
  data = mixed_missing
)

c(
  rows_used = sum(!is.na(mixed_missing$depression)),
  people_used = length(unique(
    mixed_missing$participant[!is.na(mixed_missing$depression)]
  ))
)
```

```
  rows_used people_used
        189         100
```

Every one of the 100 people is still in the model, contributing whatever they turned up for. A change-score analysis would have deleted all 11 of the people who missed the second visit, because a difference needs both halves, and with it the perfectly good baseline score each of them did provide.

However, "the model ran" does not mean missing data ceased to matter. Standard likelihood-based mixed-model inference commonly relies on a **missing at random** assumption conditional on variables in the model. If people with worsening symptoms leave the study for reasons the model does not capture, the missingness can bias results. Investigate dropout, include relevant predictors when justified, and conduct sensitivity analyses in serious work.

18.14 Check the Model

Every model in this book has arrived with a diagnostic ritual attached. This one does too, except the ritual is harder, and I am going to be honest with you about how much of it we are skipping.

Here is the problem. A mixed model has more places for an assumption to hide than a regression does: the residuals, the random effects, and the relationship between them. Worse, the ordinary residuals a mixed model hands you are not the clean, independent, interchangeable things `lm()` produces, so the plots you learned to read in the diagnostics chapter do not mean quite the same thing here. The assumptions **rhyme** with the ones you already know. Checking them does not.

So we use a package that does the hard part for us. **DHARMa** takes your fitted model, simulates a few hundred fresh datasets from it, and then asks, for each of your real observations, where it falls inside its own simulated crowd. If the

model is telling the truth, those positions should be spread evenly between 0 and 1. That is the trick: no matter how strange the raw residuals look, the *simulated* residuals should always be uniform when the model is right, so there is one picture to learn instead of one per situation.

```
library(DHARMa)

sim_res <- simulateResiduals(mixed_model)

plotQQunif(sim_res)
```

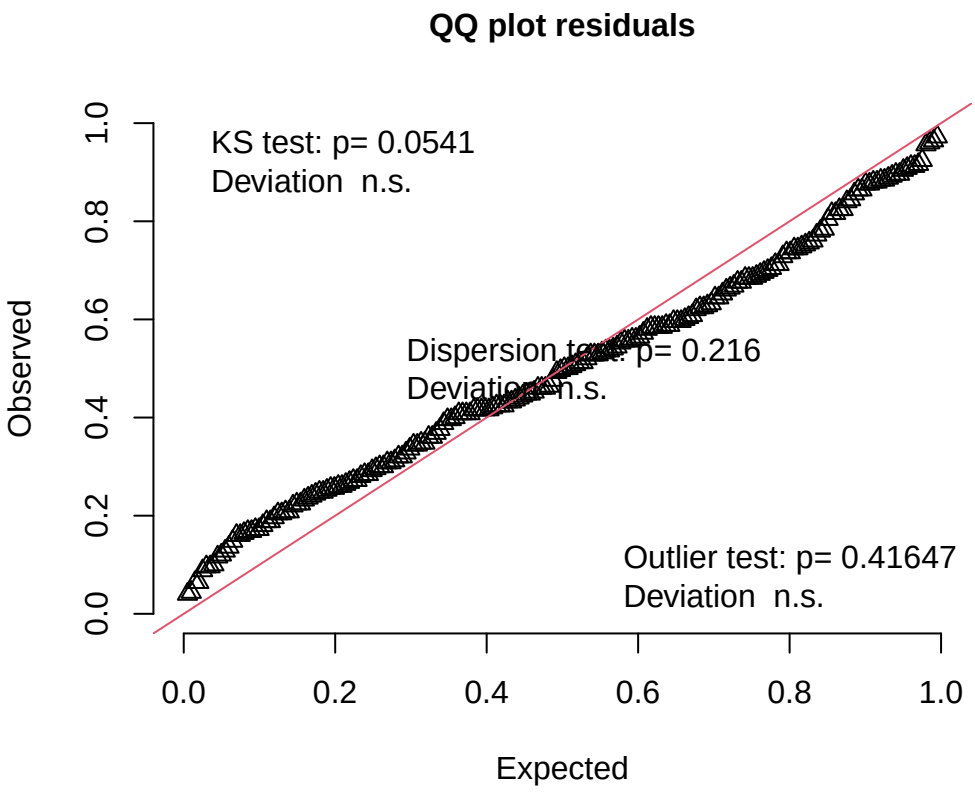


Figure 18.2: DHARMa's QQ plot. If the model is right, the points sit on the red diagonal. The three tests printed on the plot check the overall shape, the spread, and whether there are more extreme points than the model expects.

The points follow the diagonal with a gentle bow, and the three tests printed on the plot are ones you can ignore for now.

The second plot asks a different question: are the residuals equally well behaved everywhere, or does the model fit one part of the design better than another?

```
plotResiduals(sim_res, form = mixed_data$occasion)
```

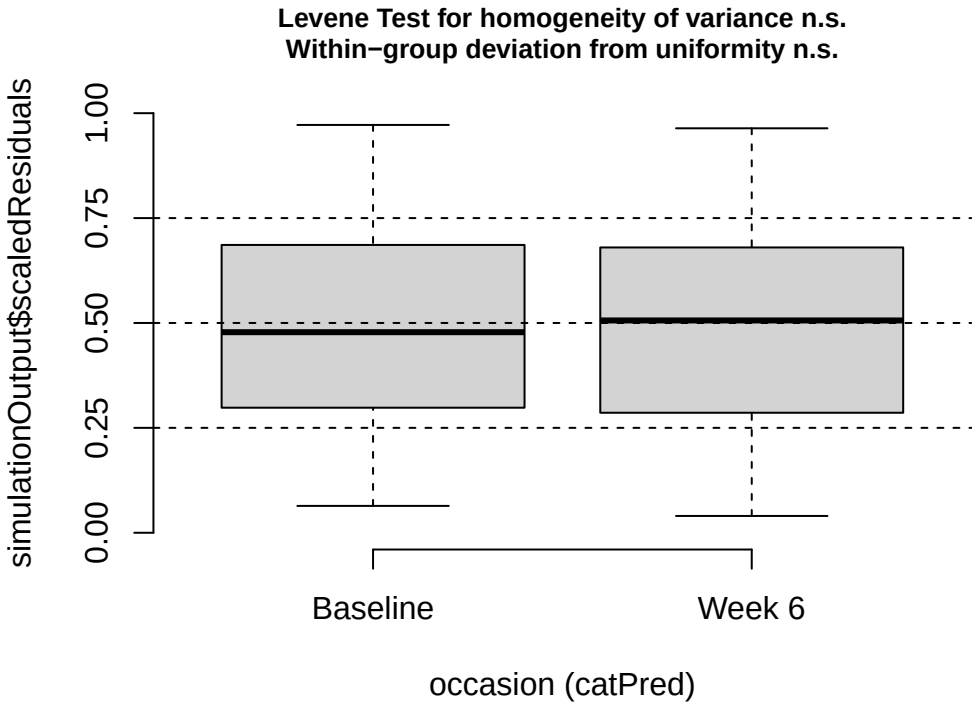


Figure 18.3: DHARMa residuals split by occasion. Both boxes should straddle 0.5 with similar spread, which they do. This is where unequal variance across conditions would show up.

Two boxes, both centered near 0.5, both about the same height, and both tests `n.s.` The model is not doing noticeably worse at Week 6 than at baseline. Swap `occasion` for `group` and you can ask the same question about CBT versus Control.

That is where we stop, and stopping here is a real choice rather than a complete answer.

> ⚠ This Is the Shallow End, and It Is a Very Deep Pool
>
> Mixed models are an enormous area of statistics, and their diagnostics are an active research area rather than a settled checklist. Two DHARMa plots are the minimum, not the standard.
>
> If your design is bigger than the one in this chapter, you need more than this chapter. My advice is a graduate mixed-models course, and the advanced sections of this book, not a longer checklist from me.
>
> This is also, genuinely, hard material. First-time graduate students meet it and struggle. If it feels like a lot, that is the material, not you.

18.15 When Your Design Outgrows This Chapter

Everything here rested on the simplest possible repeated-measures design: each person measured twice, on one within-subjects factor, crossed with one between-subjects factor. For that design, what this chapter showed you works, and it works well.

Push past it and the difficulty does not increase gently. Two within-subjects factors, or measurements at four or six occasions over time, and you are immediately making decisions this chapter never had to make. Which random effects

should the model include? What happens when the maximal model refuses to converge, or converges to a variance of exactly zero? Should occasions close together be allowed to resemble each other more than occasions far apart? Those are real questions with contested answers, and getting them wrong changes your results rather than merely annoying you.

Here is the part I am obliged to admit. For a **balanced, complete** repeated-measures design with several within-subjects factors, old-school repeated-measures ANOVA is often *easier* than the mixed model. It was built for exactly that shape of data, it will fit without an argument, and it will not ask you to choose a random-effects structure. You know how I feel about ANOVA. It is still true.

The catch is what it demands in return. Repeated-measures ANOVA wants every person to have every observation, so one missed session deletes the whole participant, and it leans on **sphericity**: the assumption that the variances of all the pairwise differences between occasions are equal. With two occasions sphericity cannot fail. There is only one difference and nothing for it to disagree with. From three occasions up it becomes a real constraint, and a violated one costs you.

So neither tool is the grown-up choice in all situations. The mixed model handles missing rows and unbalanced designs that ANOVA cannot touch. ANOVA handles complicated balanced within-subjects designs without the fight. We did not escape assumptions by learning mixed models. We traded a rigid set for a flexible set that has to be chosen thoughtfully.

If a repeated-measures ANOVA is the right tool for your design, there is a review of it in the advanced sections of this book. Use whichever tool the design actually calls for, and say in your write-up which one you used and why.

18.16 The Short Story

- A paired t-test is a mixed model with one two-level within-subjects predictor and a random intercept. Same estimate, same t, same df, same p.
- A mixed design contains at least one between-subjects predictor and one within-subjects predictor.
- Repeated occasions are nested within people, so ordinary independent-error regression is incomplete.
- Ignoring that nesting does not bias the coefficients. It ruins the standard errors, which is enough to change the answer.
- A mixed 2-by-2 has four cells and a difference-of-differences interaction, exactly like the all-between 2-by-2. The one new thing is the participant random intercept.
- With Baseline as the reference, the Group coefficient is the baseline group difference, which randomization should have made boring.
- The Occasion by Group coefficient is the difference between the two groups' changes.
- With exactly two occasions, that interaction is also an independent-samples t-test on change scores.
- Random intercepts let people start at different levels, which is all a two-occasion design can support.
- Compare fixed-effects structures using models fit with maximum likelihood, not REML.
- Check a mixed model with **simulated** residuals, from DHARMa, rather than by reading raw residual plots the way you would for an ordinary regression. Two plots is the minimum, not the standard.
- This chapter covers the simplest repeated-measures design there is. More within-subjects factors, or more occasions, gets hard quickly, and for balanced complete designs old-school repeated-measures ANOVA is often the easier tool.
- Mixed models can retain partially observed participants, but missingness can still bias the answer.
- Never count rows and forget to count people. Two hundred observations from 100 people are not 200 independent humans, no matter how urgently the grant deadline approaches.

The model is "mixed" because it combines **fixed** and **random** effects, not because we placed several statistical procedures into a blender and hoped the lid stayed on.

Part III

Okay, But What If It's Complicated?

19 Advanced: Regression Diagnostics and Influential Weirdos

19.1 An Unusually Responsible Therapy Study

We predict symptom improvement from therapy attendance and baseline symptom severity. Sixty people, two predictors, nothing exotic.

One of those sixty is a problem, and we are going to build the problem on purpose so you can watch what it does. Participant 60 attends nearly every session and reports a suspiciously enormous improvement, possibly because they misunderstood the scale or wanted the research assistant to stop calling. Here is the whole simulation, including the three lines at the bottom that do the damage.

```
set.seed(343)
n <- 60
TherapyData <- data.frame(
  Sessions = runif(n, 0, 12),
  Baseline = rnorm(n, 60, 10)
)

TherapyData$Improvement <-
  1.8 * TherapyData$Sessions +
  0.18 * TherapyData$Baseline +
  rnorm(n, 0, 5)

TherapyData$Sessions[60] <- 12
TherapyData$Baseline[60] <- 45
TherapyData$Improvement[60] <- 60

TherapyModel <- lm(Improvement ~ Sessions + Baseline, data = TherapyData)
summary(TherapyModel)
```

```
Call:
lm(formula = Improvement ~ Sessions + Baseline, data = TherapyData)

Residuals:
     Min       1Q   Median       3Q      Max
-12.4882  -2.9666  -0.4928   2.5910  26.0396

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
```

```
(Intercept) 11.77323    5.68723   2.070    0.043 *
Sessions     1.93638    0.21937   8.827 2.96e-12 ***
Baseline    -0.02332    0.09016  -0.259    0.797
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.865 on 57 degrees of freedom
Multiple R-squared:  0.5827,    Adjusted R-squared:  0.568
F-statistic: 39.79 on 2 and 57 DF,  p-value: 1.525e-11
```

That is a good-looking model. Attendance predicts improvement, the coefficient is nowhere near zero, the p-value turned up in scientific notation, and the model accounts for 58% of the variance in improvement. Write it up, thank the undergraduates, go home.

Now fit the same model to the same study with participant 60 removed. Not a suspicious subgroup, not a condition, not a wave of attrition. One row out of sixty.

```r
TherapyModel.60 <- update(TherapyModel, data = TherapyData[-60, ])

round(rbind(
  Everyone  = c(coef(TherapyModel),    R2 = summary(TherapyModel)$r.squared),
  Without60 = c(coef(TherapyModel.60), R2 = summary(TherapyModel.60)$r.squared)
), 4)
```

```
          (Intercept) Sessions Baseline     R2
Everyone      11.7732   1.9364  -0.0233 0.5827
Without60      7.4684   1.7110   0.0631 0.6245
```

Three things moved, and none of them is the thing you were watching.

The baseline coefficient **changed sign**. With everyone in, higher baseline severity predicts slightly less improvement. With one person out, it predicts slightly more. Neither version is significant, which is its own kind of embarrassing: the direction of that effect was never being decided by the other fifty-nine people.

The intercept stopped being significant, going from $p = 0.043$ to $p = 0.106$. If you had a hypothesis about the intercept, you no longer have a result.

And R^2 went **up**, from 0.583 to 0.625. This surprises people. Deleting participant 60 took proportionally more out of the unexplained pile than out of the total. The residual standard error fell from 5.86 to 4.63. The model did not get better. It got easier.

Notice what did *not* warn you. Nothing errored. Nothing printed a message. The first model is not missing a step, and no amount of rereading its output would have told you that one row was steering it. R fit exactly what you asked for, with the serene indifference of a vending machine.

A model cannot tell you that one person wrote its conclusion. That is the job you are about to learn.

19.2 Advanced Topic: Who Is This For?

In the Intro to Regression chapter we asked five questions about the line: linearity, independence, constant variance, approximately normal residuals, and whether one strange person is steering everything while the others watch. This chapter is those five questions with a magnifying glass. You have just seen question five win.

This chapter is for graduate students and advanced undergraduates who already know how to fit and interpret a regression model. We now ask a more alarming question: **Should we trust the model we just fit?**

Regression will always give you coefficients if the mathematics permits it. It does not pause to ask whether one participant entered a reaction time of 94 years or whether the residuals resemble a startled squid.

> **A Significant Model Can Still Be a Bad Model**
>
> A small p-value does not certify linearity, equal residual variance, independence, honest measurement, or sensible causal reasoning. It only answers a narrower question under the model's assumptions.

19.3 Residuals: What the Model Failed to Explain

For person i,

$$e_i = Y_i - \widehat{Y}_i.$$

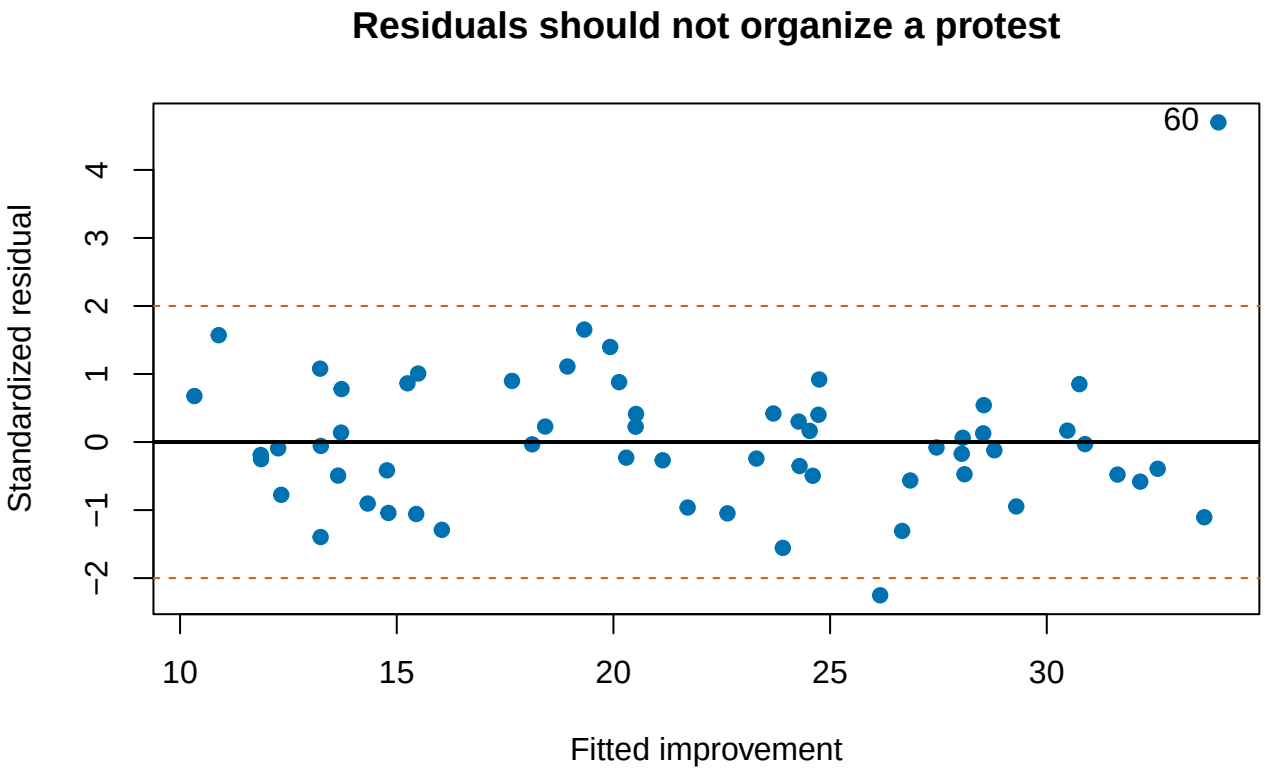
A residual is the observed outcome minus the model's prediction. Positive residuals mean the person scored above the prediction; negative residuals mean they scored below it.

The ordinary linear model assumes:

$$Y_i = \beta_0 + \beta_1 X_{1i} + \cdots + \beta_k X_{ki} + \epsilon_i,$$

where the errors have an expected value of zero, are independent, and have approximately constant variance. Normality matters primarily for conventional small-sample inference, not because every raw variable must form a perfect bell.

```
plot(
  fitted(TherapyModel), rstandard(TherapyModel),
  pch = 19, col = "#0072B2",
  xlab = "Fitted improvement",
  ylab = "Standardized residual",
  main = "Residuals should not organize a protest"
)
abline(h = 0, lwd = 2)
abline(h = c(-2, 2), lty = 2, col = "#D55E00")
# pos = 2 puts the label to the LEFT: participant 60 sits against the right
# edge, so labelling to the right runs the text off the plot.
text(fitted(TherapyModel)[60], rstandard(TherapyModel)[60], "60", pos = 2)
```



Look for curves, funnels, clusters, and isolated points. A random cloud around zero is comforting. A visible pattern means the model has left information lying on the floor.

> **i** Raw Residuals Versus Standardized Residuals
>
> Raw residuals remain in the outcome's units. Standardized or studentized residuals divide by an estimated residual standard deviation, making unusually large residuals easier to identify. Values beyond roughly ± 2 deserve inspection. That is an investigation threshold, not an automatic deletion command.

19.4 Linearity

Linear regression assumes the mean outcome is a linear combination of the predictors. If the true relationship bends, a straight line may average over the bend and lie everywhere with great consistency.

Possible responses include transformations, polynomial terms, splines, or a model with a more appropriate link function. The graph should guide the question; repeatedly adding powers until an asterisk appears is polynomial fishing.

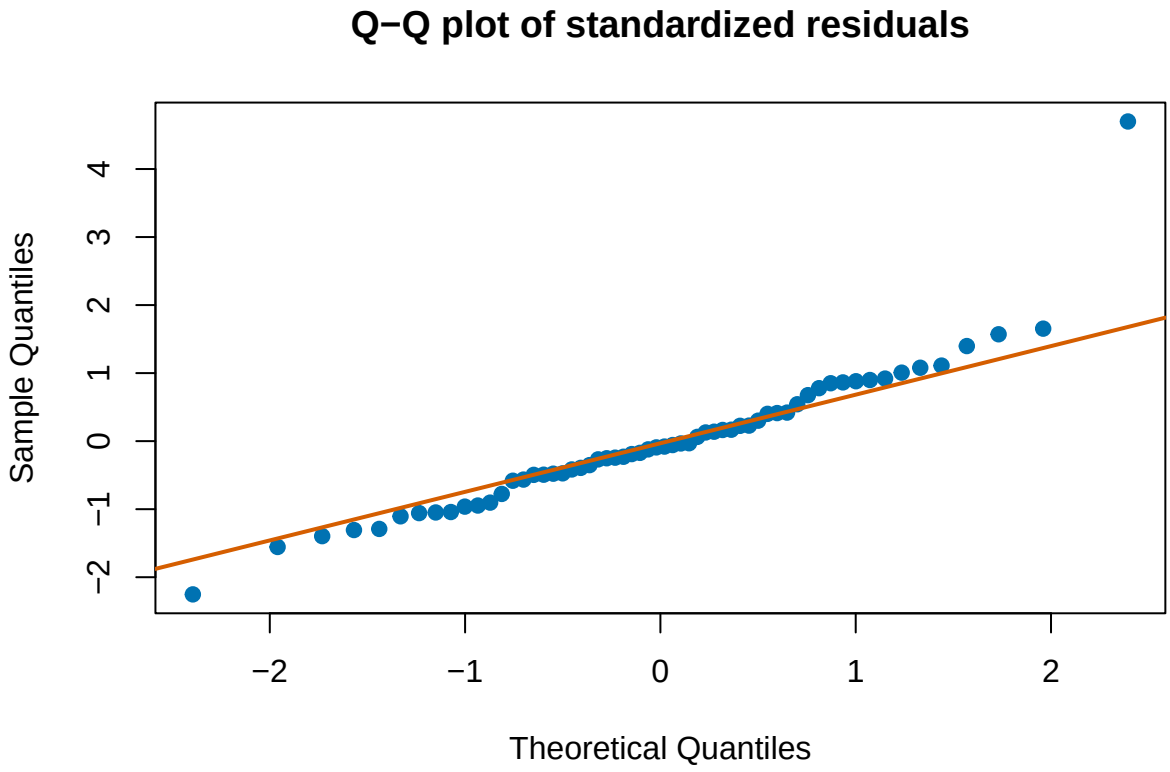
19.5 Homoscedasticity

Homoscedasticity means that residual spread is roughly constant across fitted values. A funnel shape indicates **heteroscedasticity**: the model predicts some portions of the outcome more precisely than others.

Heteroscedasticity does not automatically bias ordinary least-squares slopes, but it can make conventional standard errors and confidence intervals inaccurate. We can reconsider the outcome scale, use heteroscedasticity-consistent standard errors, or fit a model that explicitly represents changing variance.

19.6 Normality of the Errors

```
qqnorm(rstandard(TherapyModel), pch = 19, col = "#0072B2",
       main = "Q-Q plot of standardized residuals")
qqline(rstandard(TherapyModel), col = "#D55E00", lwd = 2)
```



A Q–Q plot compares residual quantiles with those expected under a normal distribution. Mild deviations are ordinary. Severe tail departures may signal outliers, a wrong outcome distribution, or a model that has ignored important structure.

> **Do Not Test Normality and Surrender**
>
> With a large sample, a formal normality test can reject harmless deviations. With a small sample, it may miss serious ones. Inspect the residuals, consider the design, and ask whether the inferential procedure is robust enough for the observed problem.

19.7 Leverage, Influence, and Cook's Distance

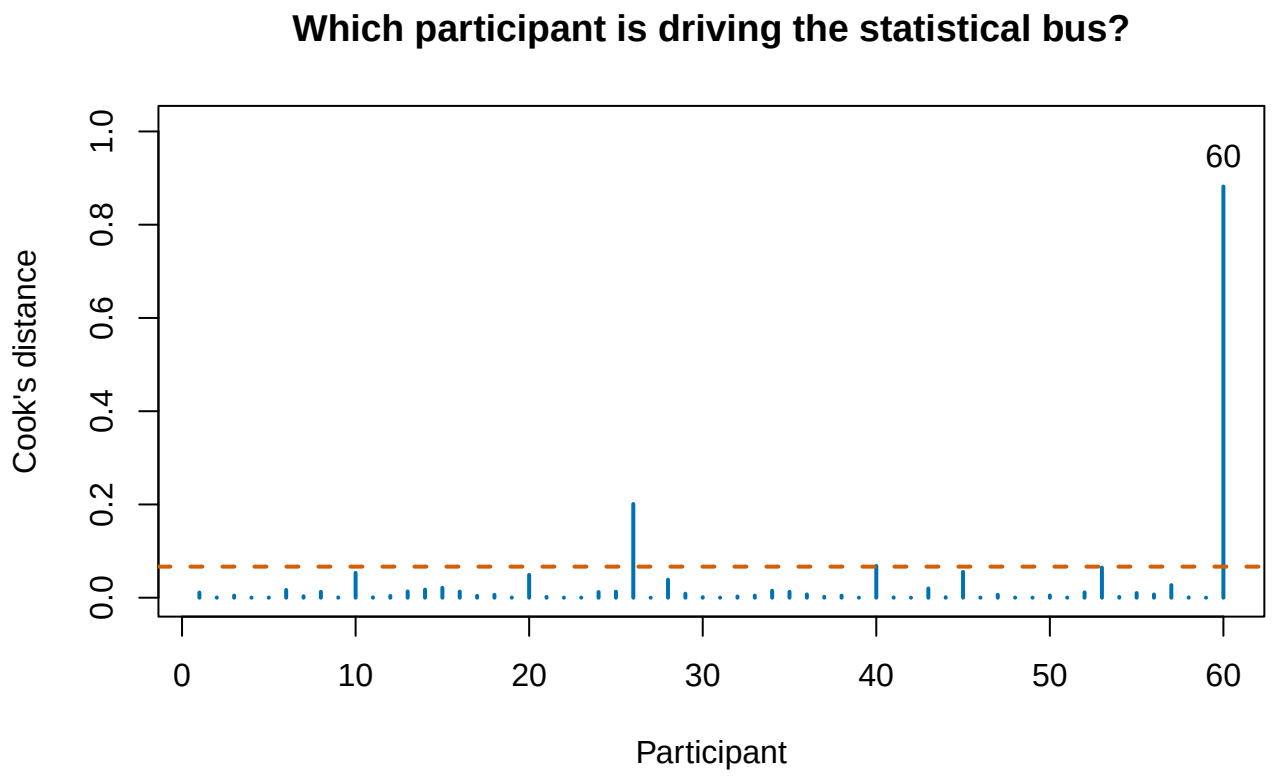
An observation has high **leverage** when its predictor values are unusual. It is **influential** when removing it meaningfully changes the fitted model. A person can have a large residual without much leverage, or high leverage while sitting obediently on the fitted line.

Cook's distance combines residual size and leverage. One common expression is:

$$D_i = \frac{e_i^2}{p\hat{\sigma}^2}\frac{h_{ii}}{(1-h_{ii})^2},$$

where p is the number of fitted coefficients and h_{ii} is leverage.

```
CookD <- cooks.distance(TherapyModel)
plot(
  CookD, type = "h", lwd = 2, col = "#0072B2",
  xlab = "Participant", ylab = "Cook's distance",
  # headroom so the "60" label above the tallest spike is not clipped
  ylim = c(0, max(CookD) * 1.15),
  main = "Which participant is driving the statistical bus?"
)
abline(h = 4 / n, lty = 2, col = "#D55E00", lwd = 2)
text(60, CookD[60], "60", pos = 3)
```



Rules such as $D_i > 4/N$ are screening devices. They tell us where to look, not whom to throw into the volcano.

Notice that the $4/N$ line flags 3 people here, not one. Participants 26 and 40 are over the line too, and they are fine. Participant 60's Cook's distance is 0.88, which is roughly 4 times the next largest. A screening rule hands you a short list of people to think about. It does not rank them, and it does not tell you which ones changed anything.

19.7.1 The Sensitivity Refit, Done Properly

The opening of this chapter refit the model without participant 60 as a stunt, to show you that it mattered. This is the same comparison written the way you would actually report it, with the standard errors and p-values attached, because "the coefficient moved" is not a finding until the reader can see how much of that movement is noise.

```
SideBySide <- cbind(
  b.all    = coef(TherapyModel),
  SE.all   = coef(summary(TherapyModel))[, "Std. Error"],
  p.all    = coef(summary(TherapyModel))[, "Pr(>|t|)"],
  b.drop   = coef(TherapyModel.60),
  SE.drop  = coef(summary(TherapyModel.60))[, "Std. Error"],
  p.drop   = coef(summary(TherapyModel.60))[, "Pr(>|t|)"]
)

round(SideBySide, 4)
```

```
             b.all SE.all  p.all b.drop SE.drop p.drop
(Intercept) 11.7732 5.6872 0.0430 7.4684  4.5491 0.1062
Sessions     1.9364 0.2194 0.0000 1.7110  0.1773 0.0000
Baseline    -0.0233 0.0902 0.7968 0.0631  0.0727 0.3888
```

Read it one row at a time.

Sessions is the conclusion of the study, and it survives. The slope drops from 1.94 to 1.71, about a 12% reduction, and stays overwhelmingly significant either way. That is what a robust result looks like: the number moves, because every number moves, and the claim does not.

Baseline does not survive, and it never had anything to survive with. Its sign depends entirely on participant 60, and it is nowhere near significant in either model. This is how to state a null result: not "baseline severity does not predict improvement," but "this study could not tell you, and one person was setting the direction."

The rule that follows is short, and it is the whole reason the comparison exists. **Report both.** Put the sensitivity refit in the paper, or in the supplement, and say what changed. Do not run this comparison, notice that one version is friendlier to your hypothesis, and report only that one. That is not a sensitivity analysis. That is shopping with extra steps.

> **What to Say When the Refit Disagrees With You**
>
> "Excluding one influential case (Cook's $D = 0.88$), the attendance slope fell from 1.94 to 1.71 and remained significant, while the baseline coefficient reversed sign and was nonsignificant in both models."
>
> One sentence. It costs you nothing, it is more informative than the result it qualifies, and it is much cheaper than having a reader discover it for you.

19.8 Multicollinearity

When predictors are strongly related, the model struggles to separate their unique slopes. Coefficients may become unstable, standard errors inflate, and signs may change when another predictor enters.

For predictor j, the variance inflation factor is:

$$VIF_j = \frac{1}{1 - R_j^2},$$

where R_j^2 comes from predicting X_j with the other predictors. A large VIF means the other predictors nearly reconstruct that predictor.

That definition tells you how VIF is computed but not why anyone cares. For that, look at where it actually lives, which is inside the standard error of the coefficient:

$$SE(b_j) = \underbrace{\frac{\hat{\sigma}}{\sqrt{\sum_i \left(X_{ij} - \overline{X}_j\right)^2}}}_{\text{what you would get if } X_j \text{ stood alone}} \times \underbrace{\sqrt{\frac{1}{1 - R_j^2}}}_{\sqrt{VIF_j}}$$

Read the two halves separately, because they answer different questions.

The **left factor** is the standard error you would earn in a world where X_j were unrelated to every other predictor. It improves in the three ways you would expect: a smaller $\hat{\sigma}$ (less unexplained noise), a larger sample, or more spread in X_j itself. That last one is worth pausing on, since it is the algebraic reason a restricted range hurts you. If everyone in your study scored between 4 and 6 on the predictor, the sum in that denominator is small and nothing can rescue the standard error.

The **right factor** is the tax. It is exactly $\sqrt{VIF_j}$, it is never less than 1, and it is the entire contribution of multicollinearity to your uncertainty. When $R_j^2 = 0$ the tax is $\sqrt{1} = 1$ and you pay nothing. When $R_j^2 = .75$ the tax is $\sqrt{4} = 2$ and your standard error doubles.

So VIF is not a diagnostic that sits beside the model commenting on it. It is a multiplier already inside every standard error the model printed.

In R the whole thing is one line, and it is the line you will actually use:

```
# car (Companion to Applied Regression) supplies vif(), plus most of the
# regression diagnostics psychologists reach for.
library(car)

vif(TherapyModel)
```

```
Sessions Baseline
1.013679 1.013679
```

Both come back at 1.01, which is a polite way of saying nothing is wrong here. Sessions and Baseline correlate at r = -0.12. This is what fine looks like. Trouble is easier to spot once you have seen fine.

19.8.1 Doing It By Hand, Because It Is Two Regressions

`vif()` is not consulting an oracle. It is running the auxiliary regression named in the definition, taking the R^2, and dividing. Here is the whole thing, plus a check that the standard-error formula above is not decoration.

```
# Step 1: predict the predictor from the other predictors.
AuxModel <- lm(Sessions ~ Baseline, data = TherapyData)
R2j <- summary(AuxModel)$r.squared

# Step 2: that is it. That is the whole formula.
c(R2j = R2j, VIF.by.hand = 1 / (1 - R2j), VIF.from.car = unname(vif(TherapyModel)[1]))
```

```
         R2j  VIF.by.hand VIF.from.car
  0.01349464   1.01367923   1.01367923
```

```
SigmaHat <- summary(TherapyModel)$sigma
SSj <- sum((TherapyData$Sessions - mean(TherapyData$Sessions))^2)

StandingAlone <- SigmaHat / sqrt(SSj)

c(
  if.Sessions.stood.alone = StandingAlone,
  times.sqrt.VIF          = StandingAlone * sqrt(1 / (1 - R2j)),
  actual.SE.from.lm       = unname(coef(summary(TherapyModel))["Sessions", "Std. Error"])
)
```

```
if.Sessions.stood.alone          times.sqrt.VIF       actual.SE.from.lm
              0.2178867               0.2193719               0.2193719
```

The second and third numbers agree to every digit R will print, because they are the same quantity computed two ways. The standard error `lm()` reported is the standing-alone standard error multiplied by $\sqrt{VIF}$, and here that multiplier is 1.007, which rounds to nothing at all.

19.8.2 The Extreme Case: Two Predictors That Are Secretly One

Before the interesting failures, look at the failure that cannot hide. Suppose we predict school achievement from the proportion of students on free lunch, and, because a paper told us they are distinct indicators of school poverty, we also enter the proportion on paid lunch.

```
set.seed(343)
n_schools <- 120

SchoolData <- data.frame(FreeLunch = runif(n_schools, .05, .95))
SchoolData$PaidLunch <- 1 - SchoolData$FreeLunch          # the whole problem

SchoolData$Achievement <- 70 - 25 * SchoolData$FreeLunch + rnorm(n_schools, 0, 6)

cor(SchoolData$FreeLunch, SchoolData$PaidLunch)
```

```
[1] -1
```

A correlation of exactly -1. Every school that is 30% free lunch is 70% paid lunch, necessarily, because the two proportions are the same fact written twice. Now put both in the model.

```
BothModel <- lm(Achievement ~ FreeLunch + PaidLunch, data = SchoolData)

summary(BothModel)
```

```
Call:
lm(formula = Achievement ~ FreeLunch + PaidLunch, data = SchoolData)

Residuals:
     Min       1Q   Median       3Q      Max
-18.8069  -3.3393   0.3135   3.4791  14.1907

Coefficients: (1 not defined because of singularities)
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   69.976      1.134   61.69   <2e-16 ***
FreeLunch    -24.385      2.008  -12.14   <2e-16 ***
PaidLunch         NA         NA      NA       NA
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.526 on 118 degrees of freedom
Multiple R-squared:  0.5556,    Adjusted R-squared:  0.5518
F-statistic: 147.5 on 1 and 118 DF,  p-value: < 2.2e-16
```

Read the line above the coefficient table: **`Coefficients: (1 not defined because of singularities)`**. `PaidLunch` came back `NA`. R did not average the two predictors, or split the credit, or pick the better one. It silently deleted a predictor from your model and carried on.

That is the correct behavior, and the reason is worth stating precisely. The regression is asked to estimate the effect of raising free lunch *while holding paid lunch constant*, and there is no such thing. There is not one school in the data where free lunch moved and paid lunch did not, so the question has no answer, and no amount of sample size will produce one.

R will even tell you the exact dependency it found:

```
alias(BothModel)
```

```
Model :
Achievement ~ FreeLunch + PaidLunch

Complete :
           (Intercept) FreeLunch
PaidLunch  1           -1
```

Read that as an equation: `PaidLunch` equals 1 minus `FreeLunch`. It has reconstructed the definition we used to build the variable.

VIF cannot help here either. It tells you why:

```
vif(BothModel)
```

```
Error in `vif.default()`:
! there are aliased coefficients in the model
```

$VIF_j = 1/(1 - R_j^2)$ and R_j^2 is exactly 1, so the formula is dividing by zero. There is nothing to inflate; the predictor has no unique variance at all.

19.8.3 Now Enter Them One at a Time

Here is the part that makes this worth showing rather than just warning about. Fit each predictor on its own and put the three models side by side.

```
FreeOnly <- lm(Achievement ~ FreeLunch, data = SchoolData)
PaidOnly <- lm(Achievement ~ PaidLunch, data = SchoolData)

Summarize <- function(Model, term) {
  c(b  = unname(coef(Model)[term]),
    SE = coef(summary(Model))[term, "Std. Error"],
    t  = coef(summary(Model))[term, "t value"],
    R2 = summary(Model)$r.squared)
}

round(rbind(
  Both.in.FreeLunch = Summarize(BothModel, "FreeLunch"),
  FreeLunch.alone   = Summarize(FreeOnly,  "FreeLunch"),
  PaidLunch.alone   = Summarize(PaidOnly,  "PaidLunch")
), 4)
```

```
                        b     SE        t     R2
Both.in.FreeLunch -24.3847 2.0078 -12.1449 0.5556
FreeLunch.alone   -24.3847 2.0078 -12.1449 0.5556
PaidLunch.alone    24.3847 2.0078  12.1449 0.5556
```

Separately, each one is a perfectly respectable predictor. Free lunch predicts achievement going down; paid lunch predicts it going up. Either would sail through review on its own.

But look at what the three rows share. The standard errors are **identical**. The t values are identical in size and differ only in sign. R^2 is 0.5556 in all three models, including the one where R threw a predictor away. And the two slopes are the same number with the sign flipped, -24.38 against 24.38.

They are not two findings. They are one finding, and the direction is a decision you made when you chose which proportion to name.

19.8.4 The Version That Does Not Warn You

Perfect collinearity is the harmless case, because R stops. Now add a rounding error's worth of noise, so the two variables are merely *almost* the same fact.

```
set.seed(343)

# Same variable, plus measurement error with a standard deviation of 0.01.
SchoolData$PaidLunch.Measured <- 1 - SchoolData$FreeLunch + rnorm(n_schools, 0, .01)

NearlyBoth <- lm(Achievement ~ FreeLunch + PaidLunch.Measured, data = SchoolData)

round(coef(summary(NearlyBoth)), 4)
```

```
                  Estimate Std. Error t value Pr(>|t|)
(Intercept)        140.8473    55.2712  2.5483   0.0121
FreeLunch          -95.2407    55.2836 -1.7228   0.0876
PaidLunch.Measured -71.0554    55.4028 -1.2825   0.2022
```

```
vif(NearlyBoth)
```

```
         FreeLunch PaidLunch.Measured
           762.284            762.284
```

Nothing errored. Nothing was set to `NA`. No message appeared above the table. And the model is now a disaster:

- The free-lunch slope was -24.4 on its own. It is now -95.2.
- **Neither predictor is significant.** Free lunch sits at $p = 0.088$ and paid lunch at $p = 0.202$, in data where the effect is real, enormous, and was significant a moment ago.
- The VIFs are about 762, so the standard errors are inflated by a factor of roughly 28.
- R^2 went from 0.5556 to 0.5617. The second predictor bought essentially nothing and cost you the first one.

This is what "they will eat each other" looks like in a coefficient table. Two predictors carrying the same information cannot both be credited for it, so each is left holding whatever the other did not explain, which is nearly nothing. The variance is still there. The model simply cannot say whose it is.

A student who ran only this model would conclude that school poverty does not predict achievement. The real effect they missed is about 24 achievement points between a school where nobody qualifies for free lunch and one where everybody does.

19.8.5 Collinearity From Fine to Hair-on-Fire

The two cases above are the ends of a continuum. Here is the middle, using the same true effect measured four times. `X1` and `Y` are identical in every row; the only thing that changes is how strongly the *other* predictor correlates with `X1`.

```r
CollinearDemo <- function(r) {
  set.seed(343)
  X1 <- rnorm(200)
  X2 <- r * X1 + sqrt(1 - r^2) * rnorm(200)
  Y  <- 2 * X1 + rnorm(200, 0, 4)

  Model <- lm(Y ~ X1 + X2)
  c(
    r     = r,
    VIF   = unname(vif(Model)[1]),
    SE.X1 = coef(summary(Model))["X1", "Std. Error"],
    p.X1  = coef(summary(Model))["X1", "Pr(>|t|)"]
  )
}

CollinearTable <- t(sapply(c(0, .5, .9, .99), CollinearDemo))

round(CollinearTable, 4)
```

```
        r     VIF  SE.X1   p.X1
[1,] 0.00  1.0021 0.3344 0.0000
[2,] 0.50  1.2863 0.3788 0.0001
[3,] 0.90  5.1322 0.7567 0.0391
[4,] 0.99 50.2847 2.3684 0.4255
```

Nothing about the relationship between `X1` and `Y` changed down that table. The effect is genuinely there in all four rows, built by the same line of code from the same numbers. What changed is our ability to see it: at $r = .9$ the effect scrapes past significance at $p = 0.039$, and at $r = .99$ it is gone entirely, at $p = 0.43$.

Nothing mysterious is happening, and you already have the formula that predicts it. The $\sqrt{VIF_j}$ tax is doing all of the work, and the table lets you check it.

At $r = .9$ the VIF is 5.13, so the formula says the standard error should be $\sqrt{5.13} = 2.27$ times the standing-alone version. The table says `SE.X1` climbed from 0.334 to 0.757, a ratio of 2.26. Prediction and observation, agreeing to two decimals, on a table generated before anyone mentioned the formula.

The last row is the useful warning. A VIF of 50 is a $\sqrt{}$ tax of about 7.1, which means that predictor is being measured with roughly 7.1 times less precision than the design nominally bought. The effect is still exactly 2. You just paid your entire sample size to the other predictor.

This is also the cost that the Multiple Regression: Statistical Control chapter promised you would meet later, where the controlled slope came with a slightly larger standard error than the uncontrolled one. This is later, and this is the meter.

> ⚠ The 5-and-10 Rule Is Folklore, Not Divine Law
>
> You will read that VIF above 5 is concerning and above 10 is serious. Nobody derived those numbers. They are round, they are memorable, and they have no distributional justification.
>
> The question that has an answer is narrower: **is the standard error on the coefficient I care about small enough to answer my question?** A VIF of 12 on a nuisance covariate you will never interpret is not a problem. A VIF of 3 on your primary predictor, in a study that was already underpowered, might be fatal. Look at the interval, not the threshold.

Multicollinearity is also not a data-cleaning problem, but it can be a theory problem.

First, dropping one of two correlated predictors does make the VIF fall, but it changes the model into a different model that answers a different question, and the slope you are left with is no longer controlled for the thing you dropped. You have not fixed the collinearity. You have stopped asking the question that produced it.

Second, and this is the one that actually happens: suppose you have two predictors, one is "free student lunch" and the second is whether the student is part of a school voucher program. You might enter both into the model cause some paper you read said they are independent and different predictors of SES. Well, the data say otherwise, cause of who the sample you collected happens to be. The theory may say X, but the data say Y. You cannot keep them both if they are the focal predictors, since they will eat each other, exactly as free lunch and paid lunch did above. If they were just covariates you needed to control for, that is okay.

Multicollinearity is not repaired by staring harder at the output. Possible solutions include improving the design, collecting more informative cases, combining redundant measures when theory supports it, or narrowing the question.

19.9 Independence and Clustering

Ordinary regression assumes residuals are independent. Repeated observations from one person, students within classrooms, siblings within families, and responses to the same stimulus are not independent simply because the spreadsheet gave them separate rows.

If residuals share a source, use a design-aware method such as a mixed-effects model. The mixed regression chapter does exactly that: it tells the model which rows belong to the same person, instead of letting it believe it met a room full of strangers.

This is the one assumption on the list that a diagnostic plot will not rescue you from. Non-independence is a fact about how the data were collected, and you know it before you fit anything. No residual plot reliably reveals clustering you have not told the model about, and no robust standard error repairs it.

19.10 A Diagnostic Workflow

1. Plot the raw data.
2. Fit the theoretically justified model.
3. Plot residuals against fitted values.
4. Inspect Q–Q behavior and residual tails.
5. Examine leverage and influence.
6. Check predictor redundancy.
7. Ask whether observations are independent.
8. Investigate suspicious cases against the original records.
9. Report consequential problems and sensitivity analyses.

! Deleting the Problem Is Not Diagnosing the Problem

Never remove an observation merely because it weakens the hypothesis. Correct impossible values, explain defensible exclusions, and compare conclusions with and without influential cases. If one person reverses the entire result, that instability is part of the result.

19.11 The Mathematics Under the Diagnostic Pictures

Ordinary least squares produces fitted values through the **hat matrix**:

$$\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}, \qquad \mathbf{H} = \mathbf{X}(\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'.$$

The diagonal value h_{ii} is observation i's leverage. Leverage comes from an unusual predictor pattern, not an unusual outcome by itself. A point can have high leverage and still sit obediently on the regression line. It becomes influential when its leverage combines with a meaningful residual.

Studentized residuals scale each residual by an estimate of its own uncertainty. Cook's distance then asks how much the fitted model would change if observation i vanished:

$$D_i \propto \frac{r_i^2}{p}\frac{h_{ii}}{1-h_{ii}}.$$

The exact cutoffs are screening rules, not automatic deletion warrants. Inspect the case, verify the data, refit with and without it, and explain what changed.

19.12 When the Variance Changes

Heteroscedasticity does not usually bias the OLS slope when the mean model is correct, but it can damage the ordinary standard errors. Possible responses include modeling the variance, transforming the outcome when scientifically sensible, using a model family that matches the outcome, or reporting heteroscedasticity-consistent standard errors. The picture tells you a problem exists; theory tells you which response makes sense.

That last option you will see some papers use has multiple names: "robust standard errors" and "the sandwich estimator". However, most people using them could not say what is being made robust to what, so here is the machinery.

19.12.1 Where the Ordinary Standard Errors Come From

Every standard error `lm()` printed came out of one matrix:

$$\widehat{Var}(\mathbf{b}) = \hat{\sigma}^2 \left(\mathbf{X}'\mathbf{X}\right)^{-1}.$$

Take the square roots of its diagonal and you have the `Std. Error` column. Notice the single $\hat{\sigma}^2$ standing outside the matrix. That is the whole homoscedasticity assumption, written down: **one** error variance, the same for every observation, so it factors out and one number can be estimated by pooling all the residuals.

The general expression, which does not assume that, is:

$$Var(\mathbf{b}) = \underbrace{\left(\mathbf{X}'\mathbf{X}\right)^{-1}}_{\text{bread}} \underbrace{\left(\mathbf{X}'\Omega\mathbf{X}\right)}_{\text{filling}} \underbrace{\left(\mathbf{X}'\mathbf{X}\right)^{-1}}_{\text{bread}}$$

where Ω is the diagonal matrix of individual error variances σ_i^2. **That is the sandwich, and the name is descriptive rather than whimsical:** the same bread on both sides, something else in the middle.

The two formulas are not rivals. Set every σ_i^2 equal to a common σ^2, so $\Omega = \sigma^2\mathbf{I}$, and the filling collapses to $\sigma^2\mathbf{X}'\mathbf{X}$, which cancels one of the breads and leaves $\sigma^2(\mathbf{X}'\mathbf{X})^{-1}$ exactly. **The ordinary standard errors are the sandwich, under the assumption that every observation is equally noisy.** Robust standard errors are not a different theory. They are the same expression with the assumption removed.

19.12.2 Estimating the Filling

The problem is that Ω contains one variance per person and we have one observation per person, so it cannot be estimated the usual way. White's insight was that we never need Ω itself, only the sandwiched product $\mathbf{X}'\Omega\mathbf{X}$, and that squared residuals are good enough for that job even though e_i^2 is a hopeless estimate of any individual σ_i^2 (White 1980). The errors average out inside the product.

The `HC` family is a set of choices about what exactly to put on that diagonal:

Type	Diagonal entry	What it is doing
`HC0`	e_i^2	White's original. Biased low in small samples.
`HC1`	$\frac{n}{n-p}\, e_i^2$	HC0 with the degrees-of-freedom correction `lm()` already applies.
`HC2`	$\dfrac{e_i^2}{1 - h_{ii}}$	Corrects for the fact that high-leverage points have artificially small residuals.
`HC3`	$\dfrac{e_i^2}{(1 - h_{ii})^2}$	Approximates leaving each case out and refitting. The conservative default.

Long and Ervin tested these on thousands of simulated small samples and recommend **HC3 as the default whenever n is below about 250**, which is most of psychology (Long and Ervin 2000). Use HC3 unless you have a specific reason not to.

19.12.3 Building It Yourself

Three lines of matrix algebra, and no package:

```
X <- model.matrix(TherapyModel)   # the design matrix, intercept column included
e <- residuals(TherapyModel)
h <- hatvalues(TherapyModel)      # the same leverages as before

Bread <- solve(t(X) %*% X)
Filling <- t(X) %*% diag(e^2 / (1 - h)^2) %*% X   # HC3

HC3.by.hand <- Bread %*% Filling %*% Bread

sqrt(diag(HC3.by.hand))
```

```
(Intercept)    Sessions    Baseline
  7.5973832   0.2895539   0.1306439
```

Now the package version:

```
# sandwich: robust and clustered covariance matrices.
# lmtest: re-tests a fitted model against any covariance matrix you hand it.
library(sandwich)
library(lmtest)

coeftest(TherapyModel, vcov = vcovHC(TherapyModel, type = "HC3"))
```

```
t test of coefficients:

            Estimate Std. Error t value  Pr(>|t|)
(Intercept) 11.77323    7.59738  1.5496    0.1268
Sessions     1.93638    0.28955  6.6875 1.054e-08 ***
Baseline    -0.02332    0.13064 -0.1785    0.8590
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Identical, which is the point of showing both. `vcovHC()` returns the covariance matrix, and `coeftest()` takes your model plus that matrix and re-runs the t-tests with it. They are two separate jobs from two separate packages, which is why the call looks the way it does.

The estimates are untouched, to the last decimal. Nothing was refit. The slope of `Sessions` is the same number it always was, because the sandwich changes only the second formula, the one about uncertainty. The standard error rises from 0.22 to 0.29, so t falls from 8.83 to about 6.69. The conclusion does not change, which is the usual outcome and is not a reason to skip it.

You can watch the ladder climb, too:

```
sapply(c("const", "HC0", "HC1", "HC2", "HC3"), function(type) {
  sqrt(diag(vcovHC(TherapyModel, type = type)))
}) |> round(4)
```

```
             const    HC0    HC1    HC2    HC3
(Intercept) 5.6872 6.8635 7.0418 7.2201 7.5974
Sessions    0.2194 0.2627 0.2695 0.2757 0.2896
Baseline    0.0902 0.1177 0.1207 0.1240 0.1306
```

`const` is the classical assumption, and it is the smallest in every row. The four HC variants increase in the order the table above predicts, because each one leans a little harder on the leverage correction.

Know What the Sandwich Is Reacting To

This dataset is not actually heteroscedastic. So why did the standard errors grow? Participant 60, and now you have the formula to see exactly how.

The HC3 filling weights each observation by $e_i^2/(1-h_{ii})^2$. Participant 60 has a residual of about 26 and a leverage of 0.107, against an average leverage of 0.05. Squaring a residual that large is most of it; the leverage correction then multiplies that by a further 1.25.

```
HC3weights <- residuals(TherapyModel)^2 / (1 - hatvalues(TherapyModel))^2

round(c(
  share.participant.60 = unname(HC3weights[60] / sum(HC3weights)),
  share.next.largest   = unname(sort(HC3weights, decreasing = TRUE)[2] / sum(HC3weights)),
  share.if.all.equal   = 1 / length(HC3weights)
), 3)
```

```
share.participant.60   share.next.largest   share.if.all.equal
               0.370                 0.085                0.017
```

One row out of sixty supplies 37% of the filling. If everyone contributed equally each would supply about 1.7%. Refit without participant 60 and the effect vanishes: the HC3 and ordinary attendance standard errors then land within 1% of each other.

The lesson is not that the sandwich lied. It is that "my robust standard errors are much larger than my ordinary ones" is itself a diagnostic finding, and the thing it most often points at is influence rather than spread. Go look at the case before you congratulate yourself for being rigorous.

Finally, a shortcut worth knowing about now that you have built these plots by hand. `performance::check_model(TherapyModel)` produces most of this chapter's diagnostics as one annotated panel, with reference bands showing what the plots should look like when nothing is wrong. Use it. It is genuinely good, and it is much faster than what we just did.

The hand-built versions still teach better, for the reason this whole chapter exists: a bundled panel tells you which plot looks unusual, and you still have to be the person who decides whether that matters, checks the case against the original records, and reports what changed. The software is obedient. You still have to be the adult in the room.

19.13 The Four Packages, and Where the Real Derivations Live

This chapter used four packages. They are not interchangeable. Here is what each one is for.

Package	What it does here	Where it came from
car	`vif()`, and most of the regression diagnostics psychologists use	Companion to Fox and Weisberg's regression textbook (Fox and Weisberg 2019)
sandwich	`vcovHC()`, the robust covariance matrix itself	Zeileis's HC and HAC implementation (Zeileis 2004)
lmtest	`coeftest()`, which re-tests a fitted model against any covariance matrix you hand it	Zeileis and Hothorn's diagnostic-checking toolkit (Zeileis and Hothorn 2002)
performance	`check_model()`, the all-in-one diagnostic panel	Part of the easystats project (Lüdecke et al. 2021)

Note the division of labor between `sandwich` and `lmtest`, since it explains a call that otherwise looks fussy. `sandwich` *computes covariance matrices* and knows nothing about hypothesis tests. `lmtest` *runs tests* and does not care where the covariance matrix came from. That is why the robust t-test needs one function from each, and it is also why the same `vcovHC()` result can be handed to `confint()`, to `car::linearHypothesis()`, and to most other things that accept a `vcov` argument.

Every Package Is Carrying Its Own Bibliography

Everything in this chapter has a proper derivation somewhere, and R will tell you where. Run `citation("sandwich")` and you get the three papers the package implements, formatted and ready to paste. This works for **any** installed package.

```
citation("sandwich")
vignette(package = "sandwich")          # list what is bundled
vignette("sandwich")                    # open the HC and HAC paper
```

The `sandwich` package ships the full Zeileis (2004) paper as a vignette, so the actual derivation of every `HC` type in that table is sitting on your hard drive right now. `lmtest` ships its own. This is not documentation in the "list of arguments" sense; these are peer-reviewed journal articles that happen to be installed alongside the code.

Two habits follow. **Cite the packages you analyze with**, since somebody wrote and maintains them for free and citation is the only currency involved. And when you want the argument behind a method rather than the syntax, look in the package before you look on the internet. The derivation for this chapter's robust standard errors is `vignette("sandwich")`, and it is better than anything a search will hand you.

19.14 The Short Story

- Diagnostics ask two questions: do the model's leftovers behave as assumed, and is a small number of observations writing the conclusion?
- **Leverage is not influence.** Leverage is an unusual predictor pattern; influence is when removing the case moves the fit. A point needs both to matter.
- **Screening thresholds are for finding cases, not deleting them.** $D_i > 4/N$ flagged three people here and only one of them changed anything.
- **Refit without the influential case and report both models.** If one person reverses the result, that instability *is* the result. Reporting only the friendlier version is shopping with extra steps.
- **A VIF is a precision meter, not a pass/fail test.** $\sqrt{VIF}$ is how much your standard error was inflated; the 5-and-10 rule is folklore.
- **Independence is a design fact, not a residual pattern you can patch.** Clustered data needs a design-aware model, not a robust standard error and some optimism.
- A significant coefficient is not a certificate. It is one number from a model that has not yet been checked.

20 P-Values and Multiple Comparisons

> ⚠ Advanced topic
>
> This chapter assumes that you already understand null hypotheses, Type I and Type II errors, power, effect sizes, and confidence intervals. Undergraduates may continue, but should bring snacks and a responsible graduate student. Graduate students should bring two snacks because they are expected to pretend this is easy.

20.1 First, Let Us Put a Dead Fish in an MRI

Bennett and colleagues placed a mature Atlantic salmon in an fMRI scanner and showed it photographs of people in social situations. The salmon was asked to decide what emotion each person was experiencing. There was a slight methodological complication: **the salmon was dead** (Bennett et al. 2010).

The researchers ran tens of thousands of voxel-by-voxel tests. With an uncorrected threshold of $p < .001$, they found apparently active voxels in the salmon's brain cavity and spinal column. After proper familywise-error and false-discovery-rate corrections, the salmon's social-cognitive abilities vanished. Even relaxed corrected thresholds found nothing, which is generally what we hope to discover about the thoughts of a dead fish.

If Atlantic salmon become difficult to recruit, Alex can substitute for the fish because he is approximately as brain-dead. The major design problem is that Alex may twitch, complain, demand cookies, and ask whether the scan counts toward tenure.

The study was deliberately absurd, but the statistical problem was real. If you conduct enough tests, fickle chance receives enough opportunities to manufacture something exciting. A tiny p-value attached to one selected result does not tell us how many other tests were run before that result crawled out of the computer wearing a crown.

Before we control many p-values, however, we should establish what one p-value means. This seems obvious. It is not. Researchers have spent nearly a century proving that they can misunderstand a definition with remarkable consistency.

20.2 What a P-Value Is

A p-value answers this conditional question:

> **Assuming the null hypothesis and the statistical model are correct, how probable are results at least as incompatible with the null hypothesis as the result we observed?**

In shorthand:

$$p = P(\text{data this extreme or more extreme} \mid H_0, \text{model})$$

Small p-values say that the observed result does not sit comfortably inside the null model. They do not identify which assumption is wrong. Perhaps the null hypothesis is a poor description. Perhaps independence failed, the model was

misspecified, the measurement was terrible, or one participant completed the experiment while being chased by a goat Statistics can tell you that the whole package looks strained. It cannot interrogate the goat.

Statistical significance is a decision rule. If the p-value is at or below a threshold chosen in advance, usually $\alpha = .05$, we call the result statistically significant. The threshold is a convention, not a cliff in nature. A result with $p = .049$ and one with $p = .051$ are not members of different species.

20.2.1 The Conditional-Probability Trap

The most famous mistake is reversing the condition:

$$P(D \mid H_0) \neq P(H_0 \mid D)$$

The probability of screaming **given that** a foam-bat-wielding research assistant is chasing you is probably high. The probability that a foam-bat-wielding research assistant is chasing you **given that** you are screaming is not necessarily high. You may have seen your tuition bill.

Cohen emphasized that a p-value gives $P(D \mid H_0)$, not the probability that the null is true after seeing the data (Cohen 1994). To calculate $P(H_0 \mid D)$, we would need additional information, including prior probabilities and an explicit alternative model. Turning one conditional probability around does not make it Bayesian. It makes it wrong.

20.2.2 A P-Value Is Not Any of These Things

A p-value is **not**:

- The probability that the null hypothesis is true.
- The probability that the research hypothesis is false.
- The probability that the result happened "because of chance."
- The Type I error rate for this particular result.
- The probability that the study will replicate.
- The size or practical importance of the effect.
- A tiny certificate declaring the study free of confounds, bias, fraud, or nonsense.

Lane-Getaz documented thirteen recurring p-value misconceptions in statistics students and experienced researchers (Lane-Getaz 2005). Even graduate students who had survived multiple statistics courses often confused p-values with the probability of a hypothesis, replicability, effect size, or Type I error. Taking another statistics course helps, but apparently does not exorcise the demon automatically.

20.3 Cohen's Complaint: The Ritual Ate the Science

Cohen's title, *The Earth Is Round ($p < .05$)*, mocked the habit of testing a nearly certain or scientifically uninteresting null and then acting impressed when it was rejected (Cohen 1994). In much of psychology, an exact **nil hypothesis**, a population difference or correlation of precisely zero, is implausible. With a large enough sample, a trivial departure from zero can become statistically significant.

That produces two failures:

1. Researchers mistake "detectable" for "important."

2. Researchers treat rejection of one null model as confirmation of their favorite theory, even though confounds, measurement problems, alternative theories, and several Probability Gods remain available.

Cohen did not offer a new sacred test to solve this problem. He recommended exploratory data analysis, graphs, effect sizes, confidence intervals, better measurement, power planning, and replication. In other words: look at what happened, estimate how much happened, admit the uncertainty, and see whether it happens again. Annoyingly, the solution is scientific judgment rather than purchasing a newer computer that can do a more powerful fancy test.

20.4 Significant Here, Not Significant There

Gelman and Stern identified another extremely common error: concluding that two effects differ because one is statistically significant and the other is not (Gelman and Stern 2006).

Suppose two independent studies estimate effects of:

$$25 \pm 10 \qquad \text{and} \qquad 10 \pm 10$$

The first estimate is statistically significant and the second is not. But their estimated **difference** is 15, with a standard error of:

$$SE_{difference} = \sqrt{10^2 + 10^2} = 14.14$$

The two effects are only about 1.06 standard errors apart. That difference is not statistically significant. "Treatment A worked but Treatment B did not" is not evidence that Treatment A worked **better** than Treatment B. Test the comparison you want to claim.

This matters for interactions, subgroups, replications, and breathless headlines. If an effect is significant for women but not men, younger adults but not older adults, or Tuesday but not Wednesday, you have not established a group difference until you test that difference directly.

20.5 One Test at a Time Creates a Family Problem

Set $\alpha = .05$ for one true null hypothesis. In repeated use, that test falsely rejects the null about 5% of the time. Now run j independent tests, all with true null hypotheses. The probability of **at least one** false positive is:

$$FWER = 1 - (1 - \alpha)^j$$

With 20 independent tests:

$$1 - .95^{20} = .642$$

There is about a 64% chance of at least one false positive. You did not change α for any individual test. You simply gave fickle chance twenty auditions and published whichever one could sing.

```r
set.seed(343)

alpha <- .05
m_grid <- 1:100
theoretical_fwer <- 1 - (1 - alpha)^m_grid

simulated_at <- c(1, 5, 10, 20, 50, 100)
simulated_fwer <- vapply(simulated_at, function(m) {
  p_values <- matrix(runif(20000 * m), ncol = m)
  mean(rowSums(p_values <= alpha) > 0)
}, numeric(1))

plot(
  m_grid, theoretical_fwer,
  type = "l", lwd = 3, col = "#1F4E79",
  ylim = c(0, 1),
  xlab = "Number of tests in the family",
  ylab = "Probability of one or more false positives"
)
points(simulated_at, simulated_fwer, pch = 19, col = "#D55E00")
abline(h = .05, lty = 2, col = "gray50")
legend(
  "bottomright",
  inset = c(.01, .10),
  legend = c("Theoretical FWER", "Simulation"),
  col = c("#1F4E79", "#D55E00"),
  lty = c(1, NA), pch = c(NA, 19), bty = "n"
)
text(78, .075, "5% reference", col = "gray40", cex = .82)
```

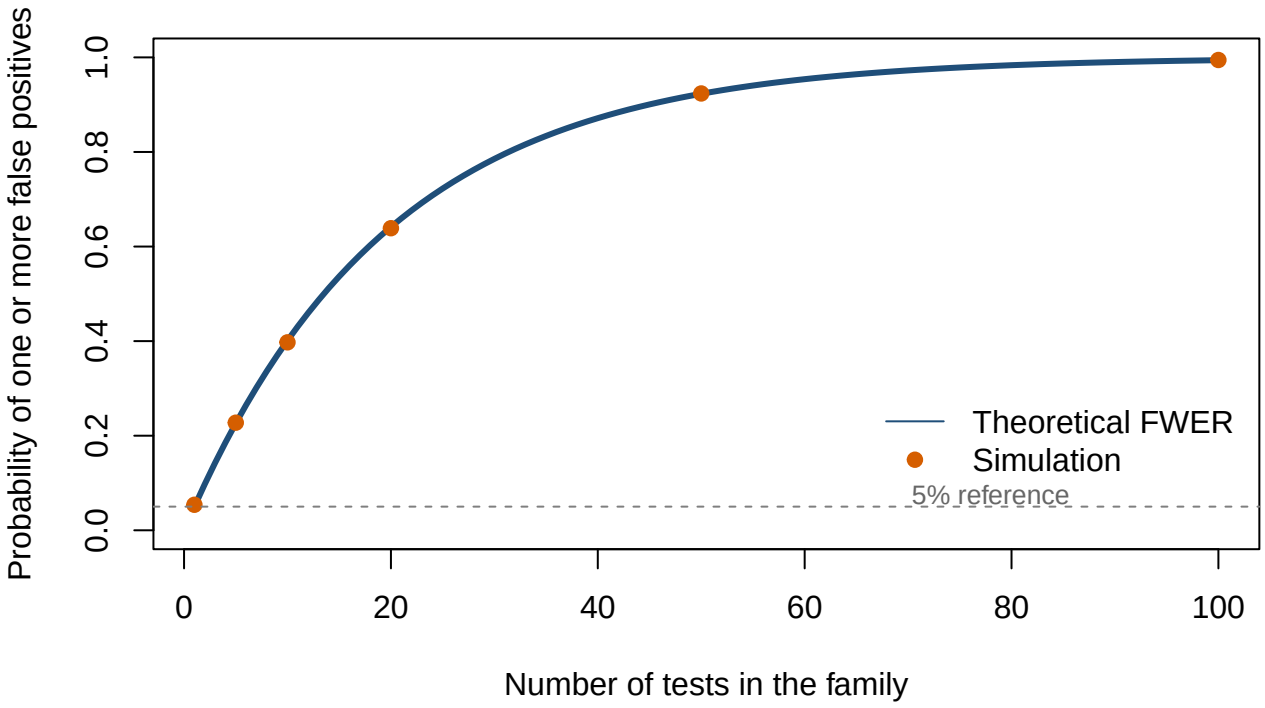


Figure 20.1: The probability of at least one false positive grows rapidly as more independent true null hypotheses are tested at alpha = .05. Simulated points sit on the theoretical curve.

The independence formula is not exact for correlated tests, such as neighboring fMRI voxels. Dependence changes the arithmetic but does not make the problem disappear. Specialized methods can exploit that dependence; pretending the other tests never happened cannot.

20.5.1 What Counts as a Family?

A **family** is the set of tests over which you want a joint error guarantee. Defining it requires scientific judgment.

- All pairwise comparisons following one omnibus test often form a family.
- All outcomes used to support one broad claim may form a family.
- Thousands of voxels in one brain contrast form a family.
- Every analysis ever conducted by everyone named Alex does not form one useful family, although it may form a cry for help.

The family should be defined by the claim and analysis plan, preferably before looking at results. Dividing twenty tests into twenty "families of one" after seeing the p-values is not a clever solution. It is p-hacking with office supplies.

20.6 Familywise Error Rate: Protect the Whole Family

The **familywise error rate** (FWER) is the probability of making **one or more Type I errors anywhere in the family**:

$$FWER = P(V \geq 1)$$

Here, V is the number of false rejections. FWER control is strict. It is appropriate when even one false positive would be costly or when a small, confirmatory set of claims is being tested.

20.6.1 Bonferroni

For j tests, Bonferroni compares each p-value with α/j, or multiplies each p-value by j and caps the result at 1:

$$p_{Bonferroni} = \min(jp, 1)$$

Bonferroni works under arbitrary dependence, which is useful, but it can sacrifice substantial power when there are many tests. Reviewer 2 always asks for this correction cause they want to kill your power and thus your paper. Don't do it unless you powered for it in advance.

20.6.2 Holm's Sequential Method

Holm's procedure orders the p-values and tests them sequentially (Holm 1979). It controls FWER under the same broad conditions as Bonferroni and is never less powerful. If your entire justification for plain Bonferroni is "that is the one I remember," R remembers Holm for you.

```
raw_p <- c(.001, .012, .021, .046, .210, .730)

data.frame(
  test = paste("Hypothesis", 1:length(raw_p)),
  raw = raw_p,
  bonferroni = p.adjust(raw_p, method = "bonferroni"),
  holm = p.adjust(raw_p, method = "holm")
)
```

```
          test   raw bonferroni  holm
1 Hypothesis 1 0.001      0.006 0.006
2 Hypothesis 2 0.012      0.072 0.060
3 Hypothesis 3 0.021      0.126 0.084
4 Hypothesis 4 0.046      0.276 0.138
5 Hypothesis 5 0.210      1.000 0.420
6 Hypothesis 6 0.730      1.000 0.730
```

20.7 False Discovery Rate Allows a Few Duds

In large exploratory analyses, strict FWER control may erase too many real signals. The **false discovery rate** (FDR) instead controls the expected proportion of false positives among the rejected hypotheses:

$$FDR = E\left(\frac{V}{\max(R, 1)}\right)$$

V is the number of false rejections and R is the total number of rejections. The `max(R,1)` convention sets the proportion to zero when nothing is rejected.

Benjamini and Hochberg's procedure controls FDR under independence and certain forms of positive dependence (Benjamini and Hochberg 1995). It is commonly available in R as `method = "BH"`.

FDR of .05 does **not** guarantee that exactly 5% of one study's discoveries are false. Some studies will have none, some will have more, and some will reject nothing. The control applies to the expected false-discovery proportion over repeated uses of the procedure.

```
data.frame(
  test = paste("Hypothesis", 1:length(raw_p)),
  raw = raw_p,
  holm_fwer = p.adjust(raw_p, method = "holm"),
  bh_fdr = p.adjust(raw_p, method = "BH")
)
```

```
          test   raw holm_fwer bh_fdr
1 Hypothesis 1 0.001     0.006  0.006
2 Hypothesis 2 0.012     0.060  0.036
3 Hypothesis 3 0.021     0.084  0.042
4 Hypothesis 4 0.046     0.138  0.069
5 Hypothesis 5 0.210     0.420  0.252
6 Hypothesis 6 0.730     0.730  0.730
```

20.7.1 What FDR Is For

FDR gets described as the lenient option, which makes it sound like the correction you reach for when the real ones were inconvenient. That is not what it is. FDR is the right tool for a particular question, and the question is not "is this specific comparison real?" It is:

> **Is anything happening here at all, and if so, where should I look next?**

That is a screening question. It is what you ask of ten thousand voxels, twenty thousand genes, or a battery of forty outcomes you collected because the grant said you could. You are not making forty claims. You are trying to find the handful worth a second study, and you can tolerate a few dead ends in that handful, because the second study is what settles it.

The obvious worry is that a method willing to tolerate false discoveries will manufacture them out of nothing. So let us hand it nothing.

Here is a brain with no activity in it whatsoever: ten thousand voxels, every null true, every p-value pure noise. This is the salmon.

```
set.seed(343)

n_voxels <- 10000
dead_brain <- runif(n_voxels)   # nothing is happening anywhere, by construction

DeadCounts <- c(
  uncorrected.p05  = sum(dead_brain < .05),
  uncorrected.p001 = sum(dead_brain < .001),
```

```
  BH_FDR           = sum(p.adjust(dead_brain, "BH") < .05),
  Holm             = sum(p.adjust(dead_brain, "holm") < .05)
)

DeadCounts
```

```
 uncorrected.p05 uncorrected.p001           BH_FDR             Holm
             542               10                0                0
```

Read that top row and then remember it is a corpse. An uncorrected threshold of .05 finds 542 active voxels in a dead brain. At the salmon paper's own uncorrected threshold of $p < .001$, it still finds 10, which is the same order as what Bennett and colleagues reported, because it is the same arithmetic doing the same thing.

BH finds zero. So does Holm. The correction that supposedly waves things through waved nothing through, because there was nothing to wave.

That is not a lucky seed either. Run two thousand dead brains and count how often BH declares even one discovery:

```
set.seed(343)

any_discovery <- replicate(2000, {
  any(p.adjust(runif(n_voxels), "BH") < .05)
})

CryWolfRate <- mean(any_discovery)

CryWolfRate
```

```
[1] 0.0445
```

About 4.4%, which is the guarantee working. When every null is true there is nothing to discover, so the false-discovery proportion is either 0 or 1, and controlling its expectation at .05 means declaring anything at all no more than 5% of the time. **Under a global null, FDR control gives you familywise control for free.**

Now give it a brain with something in it: the same ten thousand voxels, two hundred of them genuinely active.

```
set.seed(343)

z <- rnorm(n_voxels)
z[1:200] <- z[1:200] + 4          # 200 real effects, 9,800 true nulls
live_brain <- 2 * pnorm(-abs(z))

Real  <- 1:200
Noise <- 201:n_voxels

CountUp <- function(adjusted) {
  c(found = sum(adjusted < .05),
    real  = sum(adjusted[Real] < .05),
    false = sum(adjusted[Noise] < .05))
}
```

```r
BrainTable <- rbind(
  Uncorrected = CountUp(live_brain),
  BH_FDR      = CountUp(p.adjust(live_brain, "BH")),
  Holm        = CountUp(p.adjust(live_brain, "holm"))
)

BrainTable
```

```
            found real false
Uncorrected   690  198   492
BH_FDR        166  154    12
Holm           58   58     0
```

This is the entire argument in three rows.

Uncorrected reports 690 findings, and 492 of them are noise. Most of that table is garbage and it looks exactly like the real part.

Holm makes no mistakes at all: 0 false discoveries out of 58. It also misses 142 of the 200 real effects, which in a screening study means 142 things you will never look at again.

BH recovers 154 of the 200 and pays 12 false ones for them. That is the trade, stated plainly: you accept that roughly one in twenty of the things on your follow-up list is a dead end, in exchange for a follow-up list that actually contains the findings.

Note that the observed false-discovery proportion here is 0.072, slightly above the .05 you asked for. That is not a bug, and it is the point made two paragraphs above the first table in this section: FDR controls the *expected* proportion across repeated uses, so any single study can land above or below it.

> **Why the Salmon Is the Right Test of a Correction**
>
> The dead salmon is usually told as a joke about fMRI. It is better understood as a **calibration check**, and it is one every method in this chapter passes.
>
> A correction is a promise about how often you will be fooled. The cheapest way to test such a promise is to hand the procedure data with nothing in it and confirm that it says nothing. The salmon is a physical version of the simulation above, and both give the same answer: the uncorrected threshold hallucinates, and every corrected one, FDR included, does not.
>
> So when someone objects that FDR is too permissive for serious work, the reply is that it is permissive about *which* discoveries it lets through, not about *whether* there is anything there. On a dead fish it reports a dead fish.

20.8 What the Corrections Trade Away

No method is perfect. Tight false-positive control usually reduces the ability to detect real effects. The useful question is not "Which correction is best?" It is "Which error promise matches this research goal?"

The simulation below contains 20 tests per study. Five hypotheses have real standardized signals and fifteen null hypotheses are true. We compare no adjustment, Bonferroni, Holm, and Benjamini–Hochberg over 5,000 simulated studies.

```r
set.seed(343)

n_families <- 5000
n_tests <- 20
n_real <- 5

z_values <- matrix(rnorm(n_families * n_tests), nrow = n_families)
z_values[, 1:n_real] <- z_values[, 1:n_real] + 2.5
p_matrix <- 2 * pnorm(-abs(z_values))

adjust_rows <- function(method) {
  t(apply(p_matrix, 1, function(p) p.adjust(p, method = method) <= .05))
}

rejections <- list(
  Unadjusted = p_matrix <= .05,
  Bonferroni = adjust_rows("bonferroni"),
  Holm = adjust_rows("holm"),
  BH_FDR = adjust_rows("BH")
)

summarize_method <- function(reject) {
  true_discoveries <- rowSums(reject[, 1:n_real, drop = FALSE])
  false_discoveries <- rowSums(reject[, (n_real + 1):n_tests, drop = FALSE])
  total_discoveries <- true_discoveries + false_discoveries
  false_discovery_proportion <- ifelse(
    total_discoveries == 0,
    0,
    false_discoveries / total_discoveries
  )

  c(
    average_true_discoveries = mean(true_discoveries),
    probability_any_false_positive = mean(false_discoveries > 0),
    average_false_discovery_proportion = mean(false_discovery_proportion)
  )
}

round(t(vapply(rejections, summarize_method, numeric(3))), 3)
```

```
           average_true_discoveries probability_any_false_positive
Unadjusted                    3.501                          0.526
Bonferroni                    1.503                          0.037
Holm                          1.539                          0.039
BH_FDR                        2.076                          0.122
           average_false_discovery_proportion
Unadjusted                              0.152
Bonferroni                              0.018
Holm                                    0.019
```

```
BH_FDR                                0.039
```

The unadjusted method finds more real signals but also produces at least one false positive far too often. Bonferroni and Holm protect strongly against any false positive, with Holm usually retaining a little more power. BH accepts a higher chance that a study contains some false positives in exchange for finding more real signals, while controlling their expected proportion among the discoveries.

Goal	Sensible default
A few planned, confirmatory tests where any false claim is costly	Holm FWER control
A simple method that remains valid under broad dependence	Bonferroni, accepting lower power
Many exploratory tests where some false leads can be tolerated and checked later	Benjamini–Hochberg FDR control
A small number of clearly preregistered primary tests	Test those primary hypotheses as planned; correct the separate exploratory family

20.9 Multiple Comparisons in Regression

A regression with many coefficients is also a collection of tests. The correct family depends on the claim.

If one coefficient was the preregistered primary hypothesis and the others were prespecified covariates, automatically correcting every coefficient together may not represent the design. If you added 84 predictors, 31 interactions, six subgroups, and the participant's astrological sign until something became significant, you have a multiple-testing problem plus a confession.

Store the p-values you intend to treat as one family and adjust them directly. Here is the kitchen-sink model described above, built small enough to inspect: 100 people, twelve predictors, and only the first two doing anything at all. The other ten are `rnorm()` with a job title.

```
set.seed(343)

n_people <- 100
kitchen_sink <- as.data.frame(matrix(rnorm(n_people * 12), nrow = n_people))
names(kitchen_sink) <- paste0("V", 1:12)

# Only V1 and V2 are real. V3 through V12 are noise.
kitchen_sink$Focus <-
  0.45 * kitchen_sink$V1 -
  0.38 * kitchen_sink$V2 +
  rnorm(n_people)

sink_model <- lm(Focus ~ ., data = kitchen_sink)

coefficient_p <- coef(summary(sink_model))[-1, "Pr(>|t|)"]

round(data.frame(
  raw  = coefficient_p,
  holm = p.adjust(coefficient_p, method = "holm"),
  BH   = p.adjust(coefficient_p, method = "BH")
), 4)
```

```
       raw   holm     BH
V1  0.0000 0.0004 0.0004
V2  0.0034 0.0379 0.0207
V3  0.7522 1.0000 0.8242
V4  0.8672 1.0000 0.8672
V5  0.3091 1.0000 0.6183
V6  0.4809 1.0000 0.7888
V7  0.2578 1.0000 0.6183
V8  0.5569 1.0000 0.7888
V9  0.1922 1.0000 0.5766
V10 0.5916 1.0000 0.7888
V11 0.0411 0.4105 0.1642
V12 0.7555 1.0000 0.8242
```

Read the `raw` column first and pretend you do not know how the data were made. Three predictors clear .05: V1, V2, V11. Two of them are real. The third, V11, is a column of random numbers that happened to line up, at $p = 0.041$.

This was not arranged. Ten true nulls tested at $\alpha = .05$ give about a 40% chance of at least one false positive, so meeting one here is the ordinary outcome rather than bad luck. Run this chunk with a different seed and you will sometimes get none and occasionally get two.

Both corrections do their job. Holm and Benjamini-Hochberg each keep V1 and V2, the two real effects, and both kill the impostor. Notice also that BH is gentler than Holm on the survivors, which is the trade the previous section measured: it is buying power by promising less.

For planned contrasts, use software that can adjust the contrast family explicitly. Always report the method, the family of tests, and whether the analyses were confirmatory or exploratory.

20.10 You Will Meet This in emmeans

Everything above is theory you will use once a year. This section is the thing you will type next month.

In this book's pipeline, multiplicity arrives in exactly one place: `pairs(emmeans(...))` after a model with a categorical predictor. The categorical predictors chapter left a promise here. It ran three unadjusted comparisons behind a significant omnibus F, which is Fisher's protected LSD, and it warned that the protection expires at exactly three groups. So let us arrive with four.

The study is the same distracted-driving design, with one more condition added and a deliberately modest sample, because a correction that never changes anything teaches nothing.

```
library(emmeans)

set.seed(343)
n <- 20

drive_data <- data.frame(
  Condition = factor(
    rep(c("No Distraction", "Podcast", "Lecture", "Texting"), each = n),
    levels = c("No Distraction", "Podcast", "Lecture", "Texting")
  ),
  RT = c(
```

```
    rnorm(n, 0.70, 0.25),
    rnorm(n, 0.86, 0.25),
    rnorm(n, 0.92, 0.25),
    rnorm(n, 1.08, 0.25)
  )
)

drive_model <- lm(RT ~ Condition, data = drive_data)
anova(drive_model)
```

```
Analysis of Variance Table

Response: RT
          Df Sum Sq Mean Sq F value    Pr(>F)
Condition  3 2.2645 0.75484  17.744 7.952e-09 ***
Residuals 76 3.2330 0.04254
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The omnibus test is significant, so under the old three-group rule we would now be entitled to look at pairs. With four groups there are six pairs, and the entitlement has expired.

```
drive_means <- emmeans(drive_model, ~ Condition)

pairs(drive_means)
```

```
 contrast                  estimate     SE df t.ratio p.value
 No Distraction - Podcast   -0.2816 0.0652 76  -4.318  0.0003
 No Distraction - Lecture   -0.3269 0.0652 76  -5.011 <0.0001
 No Distraction - Texting   -0.4623 0.0652 76  -7.088 <0.0001
 Podcast - Lecture          -0.0452 0.0652 76  -0.693  0.8993
 Podcast - Texting          -0.1807 0.0652 76  -2.770  0.0348
 Lecture - Texting          -0.1354 0.0652 76  -2.077  0.1701

P value adjustment: tukey method for comparing a family of 4 estimates
```

Read the last line of that output. It says `P value adjustment: tukey method for comparing a family of 4 estimates`. You did not ask for that. `emmeans` decided that six all-pairwise comparisons constitute a family, applied Tukey, and mentioned it in a footnote below the table you were reading.

This is the single most important fact in this section, and it cuts both ways. The default is defensible, because Tukey was built for precisely this situation. But a reader who does not know it is there will report Tukey-adjusted p-values while describing them as ordinary ones, and a reader who wanted something else will not get it by hoping.

20.10.1 Tukey Is More Than a Button

Tukey's HSD is the correction psychologists meet most and explain least. Most people treat it as the button labelled "post hoc." Here is what it does.

Every other method in this chapter takes a pile of p-values and adjusts them, without caring where they came from. Tukey does not work that way. It asks a specific question about a specific design: **if all k group means came from the same population, how far apart would the largest and smallest of them drift by chance alone?**

That quantity has its own distribution, the **studentized range**, usually written q. Tukey compares every pair against it. The logic is worth sitting with for a moment, because it is genuinely clever: if you protect the *biggest* gap in the set, every smaller gap is protected automatically. You do not have to buy six comparisons separately, because six comparisons among four means are not six independent questions. They are six views of four numbers.

That is why Tukey is not just another cautious correction. With equal group sizes and roughly equal variances it controls the familywise error rate **exactly**, rather than by being conservative and hoping. Bonferroni and Holm are valid under *any* dependence structure, which sounds like a stronger property and is really the concession: they are generic because they know nothing about your design, so they cannot take advantage of the fact that your comparisons share means.

The assumption doing all of that work is that the groups are independent. The studentized range is derived for k separate samples with one common error variance. That is a between-subjects design, and it is why the right answer to "when should I use Tukey?" contains a design rather than a number of tests.

Here is the same comparison with the correction turned off, and with Holm instead.

```
pairs(drive_means, adjust = "none")
```

```
 contrast                estimate     SE df t.ratio p.value
 No Distraction - Podcast  -0.2816 0.0652 76  -4.318 <0.0001
 No Distraction - Lecture  -0.3269 0.0652 76  -5.011 <0.0001
 No Distraction - Texting  -0.4623 0.0652 76  -7.088 <0.0001
 Podcast - Lecture         -0.0452 0.0652 76  -0.693  0.4901
 Podcast - Texting         -0.1807 0.0652 76  -2.770  0.0070
 Lecture - Texting         -0.1354 0.0652 76  -2.077  0.0412
```

```
pairs(drive_means, adjust = "holm")
```

```
 contrast                estimate     SE df t.ratio p.value
 No Distraction - Podcast  -0.2816 0.0652 76  -4.318  0.0002
 No Distraction - Lecture  -0.3269 0.0652 76  -5.011 <0.0001
 No Distraction - Texting  -0.4623 0.0652 76  -7.088 <0.0001
 Podcast - Lecture         -0.0452 0.0652 76  -0.693  0.4901
 Podcast - Texting         -0.1807 0.0652 76  -2.770  0.0211
 Lecture - Texting         -0.1354 0.0652 76  -2.077  0.0824

P value adjustment: holm method for 6 tests
```

Put the three side by side.

```
adjust_column <- function(method) {
  as.data.frame(pairs(drive_means, adjust = method))$p.value
}

data.frame(
  contrast = as.data.frame(pairs(drive_means))$contrast,
  none     = round(adjust_column("none"), 4),
```

```
  tukey    = round(adjust_column("tukey"), 4),
  holm     = round(adjust_column("holm"), 4)
)
```

```
                 contrast   none  tukey   holm
1 No Distraction - Podcast 0.0000 0.0003 0.0002
2 No Distraction - Lecture 0.0000 0.0000 0.0000
3 No Distraction - Texting 0.0000 0.0000 0.0000
4        Podcast - Lecture 0.4901 0.8993 0.4901
5        Podcast - Texting 0.0070 0.0348 0.0211
6        Lecture - Texting 0.0412 0.1701 0.0824
```

Five of the six comparisons say the same thing under all three methods, which is the usual and slightly boring result. The sixth does not. **Lecture versus Texting** is significant unadjusted at $p = 0.0412$, and survives neither correction: Tukey puts it at 0.17 and Holm at 0.082.

That single row is the entire chapter in miniature. "Texting is worse than listening to a lecture" is a claim you may publish if you looked at that pair alone, and may not publish if you looked at all six and picked this one. The data did not change. The number of chances you gave yourself did.

20.10.2 Where Holm Helps and Where It Does Not

Look down the Tukey and Holm columns and something awkward appears: Holm gives the smaller number in five of the six rows. If Tukey is the tailored tool for this design, why is the generic one winning?

Because "which is more powerful" is the wrong question, and the answer depends on *where in the list* you are standing.

Holm is a **step-down** procedure. Sort the p-values smallest to largest, then multiply the smallest by m, the next by $m - 1$, the next by $m - 2$, and so on. That first step deserves your attention:

> **Holm's adjustment for the smallest p-value in the family is exactly Bonferroni.**

Not similar. Identical. Both multiply that one p-value by the full family size, and R will confirm it on the strongest comparison in the table above:

```
SmallestOf <- function(method) {
  min(as.data.frame(pairs(drive_means, adjust = method))$p.value)
}

c(
  raw          = SmallestOf("none"),
  raw.times.6  = 6 * SmallestOf("none"),
  holm         = SmallestOf("holm"),
  bonferroni   = SmallestOf("bonferroni")
)
```

```
         raw  raw.times.6         holm   bonferroni
5.991865e-10 3.595119e-09 3.595119e-09 3.595119e-09
```

Holm is never *worse* than Bonferroni, which is why this chapter recommends it over Bonferroni, but the improvement is entirely in the comparisons after the first. **If your headline finding is the single strongest contrast in the family, Holm has done nothing for you that Bonferroni would not have done.**

That is the real cost, and it is why Holm hurts most in the design where you can least afford it. Between-subjects studies spend their sample on telling people apart, so the standard errors are large, and the one comparison the paper is about is usually the one at the top of the sorted list, taking the full $\times m$ penalty.

Within-subjects designs are the opposite situation. Each person serves as their own control, between-person variance drops out of the error term, and the same number of humans buys much smaller standard errors. There you can usually afford Holm's conservatism, and you often need something like it, because Tukey's studentized range assumes independent groups and repeated measures are not independent. Holm's indifference to dependence stops being a concession and becomes the reason to use it.

So the two facts point in the same direction:

- **Between-subjects, exploring which means differ:** Tukey. It is built for exactly this and it is exact, not conservative.
- **Within-subjects, or a handful of theory-chosen contrasts:** Holm. It stays valid when observations are correlated, and the design has usually bought you the power to pay for it.

One footnote on magnitude. Tukey's edge over Bonferroni at the top of the list is real but modest here, about 1.03 times, because four groups do not offer much dependence to exploit. That advantage grows as groups are added, since the number of pairs grows much faster than the number of means. With eight groups you have 28 comparisons carrying only 7 degrees of freedom between them, and Tukey's willingness to notice that starts to matter a great deal.

Which `adjust=` Do I Want? Answer With Your Design

Pick by what the study is and what you are trying to learn, not by how many tests happen to be on the screen.

- **Between-subjects, all pairwise, working out which conditions differ?** Leave the default. `adjust = "tukey"` is built for exactly this and `emmeans` is already applying it. Say so in the write-up.
- **Within-subjects or repeated measures?** `adjust = "holm"`. Tukey's studentized range assumes independent groups, which you do not have, and Holm stays valid under any dependence. The design has usually bought you the power to afford it.
- **A handful of contrasts your theory named in advance?** `adjust = "holm"` again, and the family is those planned contrasts, not every pair the software is willing to generate.
- **Exactly three groups behind a significant omnibus *F*?** `adjust = "none"` is defensible, and only there. That is the protected LSD case from the categorical predictors chapter. At four groups, as above, it is the inflation problem wearing a lab coat.
- **Screening a large pile of outcomes to decide what to study next?** `adjust = "fdr"`, and call the results exploratory, because that is what you have promised. The dead-brain demonstration earlier in this chapter is what that promise buys you.

The trap to avoid: reaching for Holm in a between-subjects pairwise design because it sounds more rigorous than the default. On your strongest comparison Holm *is* Bonferroni, so that instinct costs you power at the exact place your paper needs it, and buys nothing Tukey was not already giving you.

Whatever you pick, name it and name the family. "Pairwise comparisons were Tukey-adjusted across all six condition pairs" is one clause, and it is the difference between a reader trusting your table and a reader rebuilding it.

20.11 P-Values Are Not Evil

P-values are not evil. They are also not tiny judges who determine whether a theory is innocent or guilty. They describe how incompatible the data are with a specified null model, under a pile of assumptions that must remain attached.

A defensible statistical argument uses:

- The design and quality of the measurement.
- The estimated effect and its confidence interval.
- The p-value as one piece of evidence rather than a personality test for the hypothesis.
- A correction that matches the family and scientific goal.
- Transparency about how many analyses were attempted.
- Replication, because one clever correction cannot make a single study carry an entire theory on its back.

The dead salmon did not teach us that fMRI is fake. It taught us that if we give chance enough opportunities, eventually even dinner appears to think. Correct the tests, show the effects, admit the uncertainty, and please stop asking deceased seafood to perform social cognition without preregistration.

20.12 The Short Story

- $p = P(\text{data this weird or weirder} \mid H_0 \text{ and the model})$. Never the reverse, and never the probability that your hypothesis is wrong.
- **"Significant here, not significant there" is not a comparison.** If you want to claim two effects differ, test the difference.
- **A family is defined by the claim**, and defined before you peek. Splitting twenty tests into twenty families of one after seeing the p-values is p-hacking with office supplies.
- **FWER when any single false claim would be costly**, and prefer Holm to Bonferroni, since it is never less powerful. **BH-FDR when you are exploring** and false leads are survivable.
- **Choose the pairwise correction by design, not by test count.** Between-subjects, working out which conditions differ: Tukey, which is exact for that job because the studentized range is built for independent groups. Within-subjects: Holm, which stays valid when observations are correlated, in a design that has usually bought you the power to pay for it.
- **On the strongest comparison in a family, Holm *is* Bonferroni.** The step-down only helps the comparisons after the first, so reaching for Holm in a between-subjects pairwise design costs power exactly where the paper needs it.
- **FDR answers "is anything here, and where do I look next?"** It is a screening tool, not a lenient one. On a dead brain it reports a dead brain.
- FDR of .05 is a promise about the long run, not a guarantee that 5% of *this* study's discoveries are false.
- **`emmeans` already adjusted your pairwise comparisons.** It defaults to Tukey and says so in a footnote. Know what your software chose before you describe it in a sentence.
- **Report the method and the family**, every time. One clause does it.
- No correction rescues a study from bad measurement, and no correction is a substitute for replication.
- The salmon was dead the whole time.

21 Advanced: Continuous and Higher-Order Interactions

21.1 Two Models, One Dataset, Opposite Headlines

We asked 240 students how much they slept, how much caffeine they drank, and how well they could concentrate. Here is the whole study, invented on the spot, with the interaction built in on purpose so you can see it sitting in the generating model.

```
set.seed(343)
n <- 240
InterData <- data.frame(
  Sleep = pmin(pmax(rnorm(n, 6.5, 1.3), 2.5), 10),
  Caffeine = pmin(pmax(rnorm(n, 220, 100), 0), 500)
)

InterData$Concentration <-
  40 +
  3.5 * InterData$Sleep +
  0.055 * InterData$Caffeine -
  0.012 * InterData$Sleep * InterData$Caffeine +
  rnorm(n, 0, 5)

InterData$Sleep.C <- InterData$Sleep - mean(InterData$Sleep)
InterData$Caffeine.C <- InterData$Caffeine - mean(InterData$Caffeine)
```

Now fit the obvious model. Concentration from sleep, caffeine, and their interaction, using the variables exactly as they were measured.

```
RawModel <- lm(Concentration ~ Sleep * Caffeine, data = InterData)

round(coef(summary(RawModel)), 5)
```

```
              Estimate Std. Error  t value Pr(>|t|)
(Intercept)    38.02109    4.71938  8.05638  0.00000
Sleep           3.55281    0.69413  5.11835  0.00000
Caffeine        0.06179    0.02001  3.08855  0.00225
Sleep:Caffeine -0.01226    0.00293 -4.19148  0.00004
```

Caffeine helps. The slope is 0.0618, it is significantly greater than zero at $p = 0.002$, and you can write the sentence yourself: more caffeine, better concentration.

Now subtract the mean from each predictor and fit the same three terms again.

```r
InteractionModel <- lm(Concentration ~ Sleep.C * Caffeine.C, data = InterData)

round(coef(summary(InteractionModel)), 5)
```

```
                  Estimate Std. Error   t value Pr(>|t|)
(Intercept)       57.31365    0.33775 169.69342  0.00000
Sleep.C            0.90777    0.28659   3.16756  0.00174
Caffeine.C        -0.01876    0.00356  -5.27079  0.00000
Sleep.C:Caffeine.C -0.01226    0.00293  -4.19148  0.00004
```

Caffeine hurts. The slope is -0.0188, and it is significantly *less* than zero at $p < .001$.

Both of those sentences describe the same 240 people. Neither model is wrong, neither is a typo, and nothing was dropped or added between them. Watch what did not move.

```r
FittedGap <- max(abs(fitted(RawModel) - fitted(InteractionModel)))

c(
  Interaction.raw      = unname(coef(RawModel)[4]),
  Interaction.centered = unname(coef(InteractionModel)[4]),
  R2.raw               = summary(RawModel)$r.squared,
  R2.centered          = summary(InteractionModel)$r.squared,
  Largest.fitted.diff  = FittedGap
)
```

```
    Interaction.raw Interaction.centered              R2.raw
      -1.226242e-02        -1.226242e-02        2.070618e-01
       R2.centered  Largest.fitted.diff
       2.070618e-01         2.415845e-13
```

The interaction coefficient is identical. R^2 is identical. Across all 240 predicted values, the largest disagreement between the two models shows up in the 12th decimal place, which is not a small difference but a zero that has been through floating-point arithmetic. These are not two models. This is one model, wearing two sets of labels.

So what changed? Only the location of zero. In the first model, the caffeine coefficient is the effect of caffeine **on a student who slept zero hours**. In the second, it is the effect of caffeine **on a student who slept the average amount**, about 6.6 hours. Those are different students, so they get different answers, and the model was never claiming otherwise. We were.

A coefficient in an interaction model is not an effect. It is an effect at a place.

The rest of this chapter is about choosing that place on purpose, saying which one you chose, and noticing when software has quietly chosen for you.

21.2 It Depends, Again

The earlier interaction chapters asked whether a slope differs across categories. Here both predictors can be continuous. This produces a model that is compact to write and remarkably easy to explain badly (Aiken and West 1991).

Caffeine predicts concentration differently depending on sleep. Caffeine may help a rested student focus while merely allowing an exhausted student to hear colors at a higher frame rate.

21.2.1 Fast Review: Categorical-by-Continuous Interactions

The previous interaction chapters used a categorical moderator. With a dummy variable G coded 0 and 1,

$$Y = \beta_0 + \beta_1 X + \beta_2 G + \beta_3 XG + \epsilon.$$

For the reference group, $G = 0$, so its regression line is

$$\widehat{Y} = \beta_0 + \beta_1 X.$$

For the comparison group, $G = 1$, so its line is

$$\widehat{Y} = (\beta_0 + \beta_2) + (\beta_1 + \beta_3)X.$$

Therefore, β_2 is the group difference in intercepts when $X = 0$, and β_3 is the group difference in slopes. A continuous-by-continuous interaction uses the same logic. The only difference is that the moderator can take many values instead of two. We therefore choose meaningful values at which to examine the conditional slope.

21.3 The Continuous-by-Continuous Model

$$Y = \beta_0 + \beta_1 X + \beta_2 Z + \beta_3 XZ + \epsilon.$$

The slope of X is:

$$\frac{\partial Y}{\partial X} = \beta_1 + \beta_3 Z.$$

That derivative is the entire interaction in one line: the slope of X changes as Z changes.

> **!** There Is No Single Main Effect of X
>
> When XZ is in the model, β_1 is the slope of X specifically when $Z = 0$. If zero is meaningless or outside the data, the coefficient answers a question nobody asked.

21.4 Is the Interaction Worth It?

Before interpreting a product term, check that it bought you anything. Fit the model without it and compare.

```
AdditiveModel <- lm(Concentration ~ Sleep.C + Caffeine.C, data = InterData)

anova(AdditiveModel, InteractionModel)
```

```
Analysis of Variance Table

Model 1: Concentration ~ Sleep.C + Caffeine.C
Model 2: Concentration ~ Sleep.C * Caffeine.C
  Res.Df    RSS Df Sum of Sq      F   Pr(>F)
1    237 6940.9
2    236 6460.0  1     480.9 17.569 3.92e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

It did. Adding the product term improves fit at $p < .001$, so the claim that caffeine's slope depends on sleep is doing real work rather than decorating the model. If this comparison had come back nonsignificant you would report the additive model. You would not hunt for a subgroup where the interaction reappears.

Centering places zero at the sample mean. It changes the interpretation of the intercept and lower-order slopes. It does **not** change fitted values, residuals, R^2, or the interaction coefficient.

Centering is usually helpful for three reasons:

1. β_1 becomes the slope of X when the moderator is at its mean instead of at an arbitrary zero.
2. It can reduce **nonessential multicollinearity** created only by multiplying variables together.
3. It keeps products from growing into unnecessarily enormous numbers.

Centering does not solve **essential multicollinearity** caused by predictors containing genuinely overlapping information. If sleep and exhaustion measure nearly the same construct, subtracting their means will not make them strangers.

Whatever coding you choose, use it consistently. Do not put centered variables in the product while leaving uncentered versions as the lower-order terms. You would be fitting a mathematical casserole that no longer represents the intended model.

21.4.1 The Standardized-Beta Trap

Sooner or later a reviewer will ask for standardized coefficients on your moderation model. The request is reasonable. The obvious fix is wrong. Better to meet it here than in a letter from Reviewer 2.

The correct procedure is to z-score X and Z first, and *then* multiply them. The tempting shortcut is to build the product first and z-score that. These are not the same variable, because the z-score of a product is not the product of z-scores.

```r
InterData$Sleep.Z    <- as.numeric(scale(InterData$Sleep))
InterData$Caffeine.Z <- as.numeric(scale(InterData$Caffeine))
InterData$Concen.Z   <- as.numeric(scale(InterData$Concentration))

# Right: standardize the components, then let R form the product.
ProperBetas <- lm(Concen.Z ~ Sleep.Z * Caffeine.Z, data = InterData)

# Wrong: standardize the product term itself.
InterData$Product.Z <- as.numeric(scale(InterData$Sleep * InterData$Caffeine))
WrongBetas <- lm(Concen.Z ~ Sleep.Z + Caffeine.Z + Product.Z, data = InterData)

BetaCompare <- cbind(
  Proper = coef(ProperBetas),
  Wrong  = coef(WrongBetas)
```

```
)

# Relabel: the last row is Sleep.Z:Caffeine.Z in one model and Product.Z in the
# other. They occupy the same position, and that is exactly the point.
rownames(BetaCompare) <- c("(Intercept)", "Sleep", "Caffeine", "Sleep x Caffeine")

round(BetaCompare, 4)
```

```
                 Proper   Wrong
(Intercept)      0.0032  0.0000
Sleep            0.1836  0.7187
Caffeine        -0.3083  1.0156
Sleep x Caffeine -0.2380 -1.4854
```

The interaction "beta" is -0.24 one way and -1.49 the other. Both lower-order betas are wrong in the second column too, and one of them is 1.02, which should be your first clue: a standardized coefficient larger than 1 in a model this simple is usually a confession rather than a finding.

Here is the part that makes this genuinely dangerous. The two models fit **identically**. R^2 is 0.2071 in both, and the interaction's t is -4.19 in both, so the significance test is untouched. Nothing looks broken. The only thing that changed is the number you would put in the table and describe in words, and it is off by a factor of about 6.2.

> **Do Not Trust a Standardized Column You Did Not Build**
>
> If you want standardized coefficients for an interaction model, **z-score the components first and refit**. Do not read them off a "standardized" or "beta" column produced by software that standardized the product term along with everything else, because several packages do exactly that and none of them will warn you.
>
> The check takes ten seconds: refit with z-scored predictors and confirm the numbers match the column you were about to report. If they do not, the column is describing a variable nobody constructed on purpose.
>
> Friedrich made this argument in 1982 and it has needed remaking roughly once per graduate cohort ever since (Friedrich 1982). Cohen, Cohen, West and Aiken work through it at length in their chapter on interactions (Cohen et al. 2003).

Leave a predictor uncentered when zero is scientifically meaningful and you want the lower-order coefficient evaluated there. For example, zero milligrams of caffeine is understandable. Zero hours of sleep is also understandable, but anyone reporting it should probably not be completing your study.

> **Centering Is Translation, Not Exorcism**
>
> Centering can make lower-order terms interpretable and reduce nonessential correlation between a product term and its components. It does not repair poor measurement, restricted range, or two predictors that are fundamentally redundant.

21.5 Plot the Model

We will draw the caffeine slope at low sleep, average sleep, and high sleep. The common $\pm 1SD$ convention is useful because it gives three recognizable values. It is not a law delivered by the Moderation Gods.

```r
sleep_values <- c(
  Low = -sd(InterData$Sleep.C),
  Average = 0,
  High = sd(InterData$Sleep.C)
)

caffeine_grid <- seq(min(InterData$Caffeine.C), max(InterData$Caffeine.C), length.out = 100)
plot(
  InterData$Caffeine, InterData$Concentration,
  pch = 19, col = rgb(0.3, 0.3, 0.3, 0.22),
  # y label is "Concentration", not "Predicted concentration": the dots are raw
  # observed scores. Only the three overlaid lines are predictions.
  xlab = "Caffeine (mg)", ylab = "Concentration",
  main = "Caffeine does not have one universal slope"
)

line_colors <- c("#D55E00", "#0072B2", "#009E73")
for (i in seq_along(sleep_values)) {
  newdata <- data.frame(
    Sleep.C = sleep_values[i],
    Caffeine.C = caffeine_grid
  )
  lines(
    caffeine_grid + mean(InterData$Caffeine),
    predict(InteractionModel, newdata = newdata),
    col = line_colors[i], lwd = 3
  )
}
legend("topright", legend = paste(names(sleep_values), "sleep"),
       col = line_colors, lwd = 3, bty = "n")
```

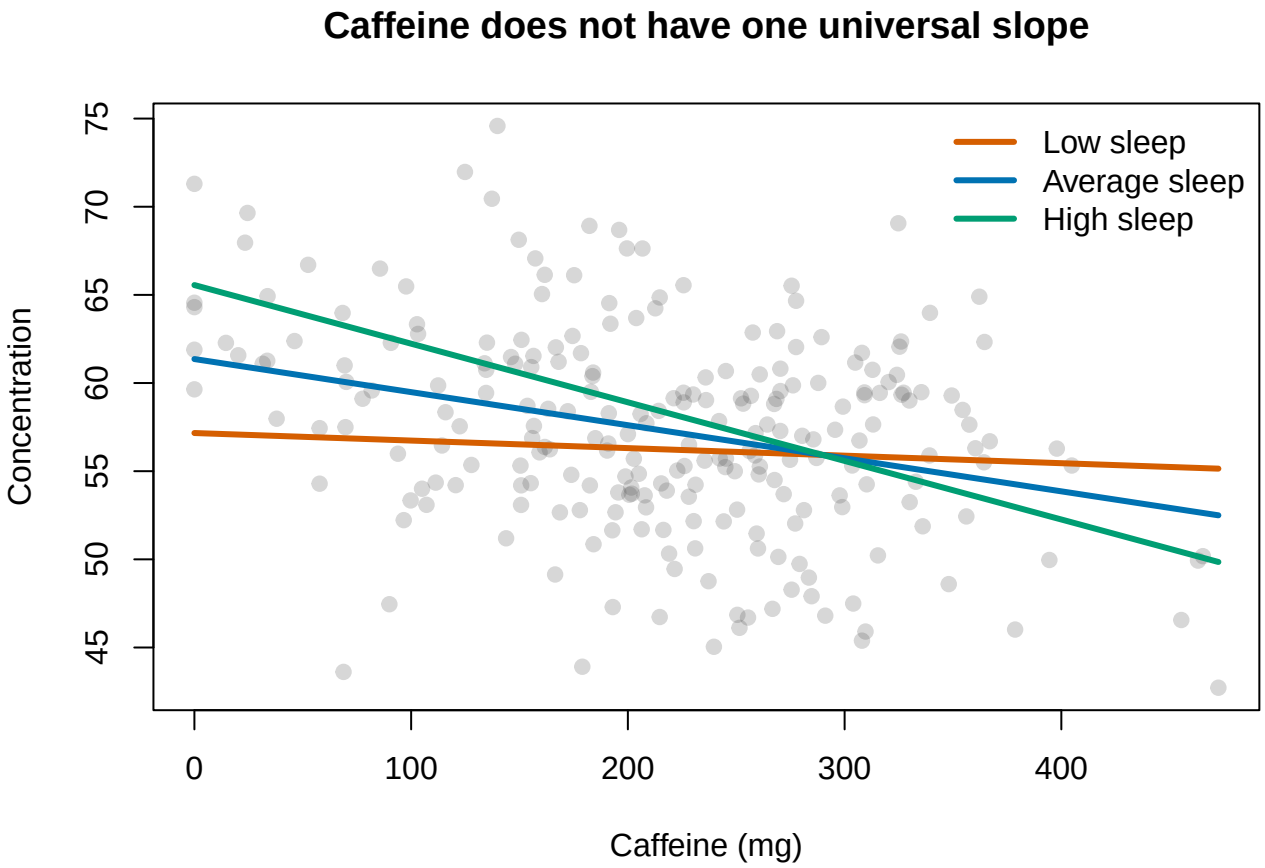


Figure 21.1: Points are observed concentration scores. The three lines are model predictions at low, average, and high sleep, defined as one standard deviation below the mean, the mean, and one standard deviation above it.

21.6 Simple Slopes

For a chosen value z^* of the moderator, the simple slope of caffeine is:

$$b_{Caffeine|z^*} = b_1 + b_3 z^*.$$

Its standard error also depends on the covariance between b_1 and b_3:

$$SE(b_1 + b_3 z^*) = \sqrt{Var(b_1) + z^{*2} Var(b_3) + 2 z^* Cov(b_1, b_3)}.$$

This is why we let software test simple slopes. Doing it by hand is possible, but so is churning butter.

```
emmeans::emtrends(
  InteractionModel,
  specs = "Sleep.C",
  var = "Caffeine.C",
  at = list(Sleep.C = unname(sleep_values))
)
```

```
 Sleep.C Caffeine.C.trend      SE  df lower.CL upper.CL
   -1.18         -0.00428 0.00528 236  -0.0147  0.00613
    0.00         -0.01876 0.00356 236  -0.0258 -0.01175
    1.18         -0.03324 0.00462 236  -0.0423 -0.02415

Confidence level used: 0.95
```

Use theoretically meaningful moderator values when possible. Quantiles may be better than $\pm 1SD$ for a skewed moderator. Never probe values outside the observed range merely because the resulting line looks exciting.

21.6.1 What Software Is Doing When It Tests a Simple Slope

`emtrends()` is not creating a new kind of regression. It is combining coefficients from the fitted model. We can prove this by changing the value represented by zero.

```
SleepSD <- sd(InterData$Sleep.C)

InterData$Sleep.AtLow <- InterData$Sleep.C - (-SleepSD)
InterData$Sleep.AtMean <- InterData$Sleep.C
InterData$Sleep.AtHigh <- InterData$Sleep.C - SleepSD

SlopeAtLow <- lm(
  Concentration ~ Caffeine.C * Sleep.AtLow,
  data = InterData
)
SlopeAtMean <- lm(
  Concentration ~ Caffeine.C * Sleep.AtMean,
  data = InterData
)
SlopeAtHigh <- lm(
  Concentration ~ Caffeine.C * Sleep.AtHigh,
  data = InterData
)

rbind(
  LowSleep = coef(summary(SlopeAtLow))["Caffeine.C", ],
  MeanSleep = coef(summary(SlopeAtMean))["Caffeine.C", ],
  HighSleep = coef(summary(SlopeAtHigh))["Caffeine.C", ]
)
```

```
              Estimate  Std. Error    t value     Pr(>|t|)
LowSleep  -0.004277577 0.005281348 -0.8099405 4.187903e-01
MeanSleep -0.018759542 0.003559155 -5.2707860 3.064031e-07
HighSleep -0.033241507 0.004617131 -7.1996017 8.019402e-12
```

In each model, the coefficient for caffeine is evaluated where the newly coded sleep variable equals zero. The fitted values and interaction remain the same. We only rotated the coefficient labels until the question we wanted was sitting in the convenient chair.

You do not need to test slopes at every possible moderator value. That converts one interaction into a large family of tests and invites the multiple-comparisons demons. Test values required by theory. Plot enough of the observed range to show the shape.

21.7 Moderation Is a Theoretical Label

When we say that Z **moderates** the effect of X on Y, we mean that the relationship between X and Y changes across values of Z. Statistically, this is an interaction. Calling one variable the moderator does not change the equation.

Which predictor is the moderator depends on the theory. In the caffeine-and-sleep example, we can say sleep moderates the caffeine slope or caffeine moderates the sleep slope. The fitted surface is identical. The story determines which conditional relationship we emphasize.

Moderation and mediation both involve three-variable systems, but they are not the same procedure. Moderation asks **when or for whom** a relationship changes. Mediation asks whether a relationship may operate **through** another variable. Sharing the letter M does not make them statistical cousins.

21.8 Common Theoretical Patterns

Words such as *synergistic*, *antagonistic*, and *buffering* describe patterns in predicted values. They do not create different estimators. We still fit a product term, inspect the coding, calculate conditional slopes, and plot the model.

21.8.1 Synergistic Interaction

A synergistic interaction occurs when the combined effect of two predictors is larger than their additive effects would suggest. Imagine that better sleep improves concentration, moderate caffeine improves concentration, and caffeine is especially helpful when a student is rested enough to remember where their mouth is.

When both predictors have positive lower-order slopes, a positive product term often creates this pattern. However, the sign depends on coding. Inspect predictions before naming it.

21.8.2 Antagonistic or Interference Interaction

An antagonistic interaction occurs when the combined effect is smaller than simple addition predicts. Perhaps using one study app improves performance and using a second app also improves performance, but using both produces 47 notifications and an emergency need to redesign the color theme. The second tool has diminishing returns as use of the first increases.

When both lower-order slopes are positive, a negative product term often produces antagonism.

21.8.3 Buffering Interaction

A buffering interaction usually combines a risk factor with a protective factor. Suppose academic stress predicts worse sleep, but social support weakens that relationship. As support rises, the negative stress slope approaches zero.

The term *buffering* depends strongly on which variable is defined as the risk and which outcome direction is good. Reverse-score stress into serenity and several coefficient signs reverse even though every fitted value remains unchanged.

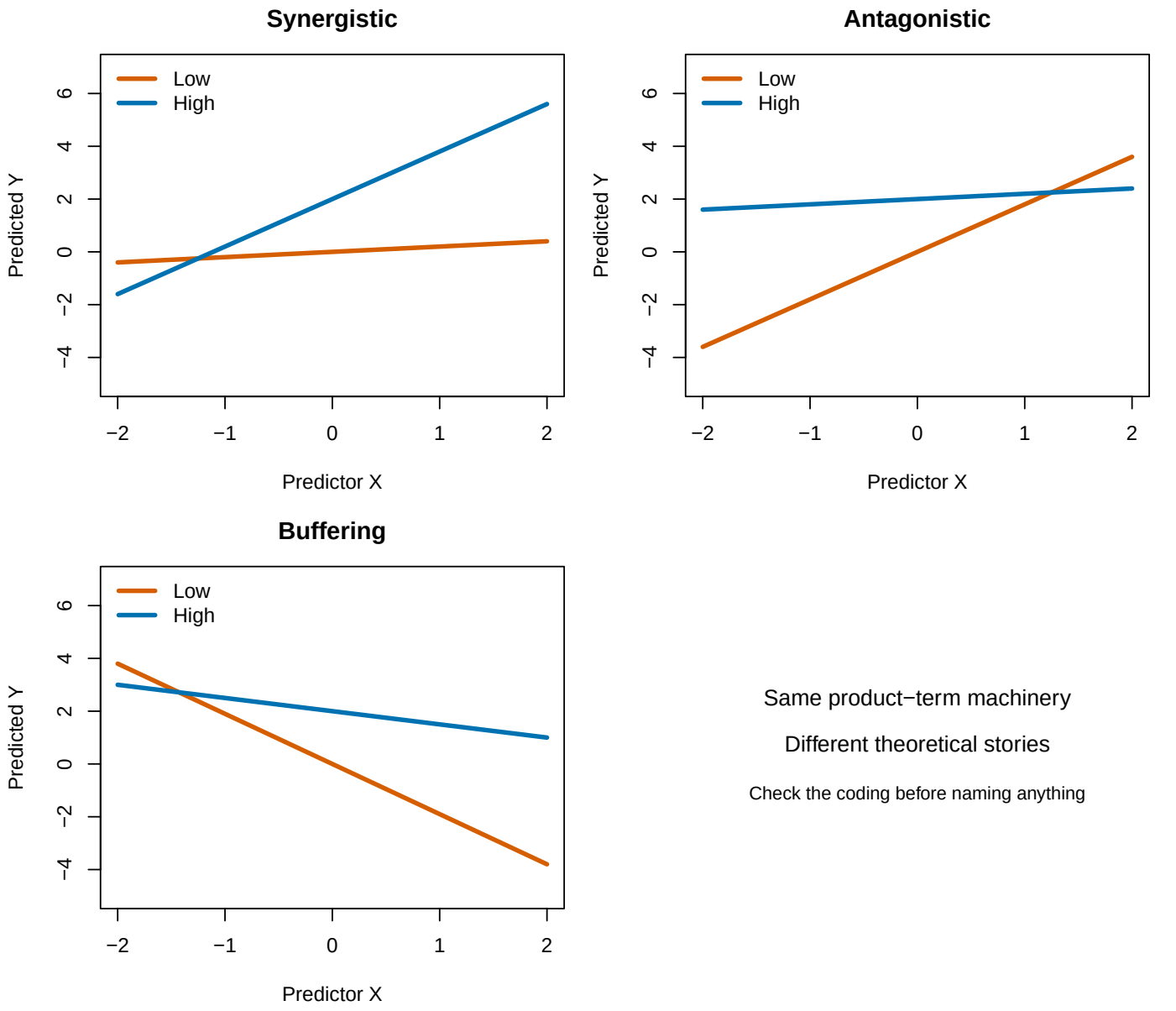
```
PatternX <- seq(-2, 2, length.out = 80)
ModeratorValues <- c(Low = -1, High = 1)
PatternColors <- c("#D55E00", "#0072B2")

old_par <- par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))

for (PatternName in c("Synergistic", "Antagonistic", "Buffering")) {
  plot(
    NA, xlim = range(PatternX), ylim = c(-5, 7),
    xlab = "Predictor X", ylab = "Predicted Y",
    main = PatternName
  )

  for (j in seq_along(ModeratorValues)) {
    Z <- ModeratorValues[j]
    if (PatternName == "Synergistic") {
      Y <- 1 + PatternX + Z + .8 * PatternX * Z
    } else if (PatternName == "Antagonistic") {
      Y <- 1 + PatternX + Z - .8 * PatternX * Z
    } else {
      Y <- 1 - 1.2 * PatternX + 1.0 * Z + .7 * PatternX * Z
    }
    lines(PatternX, Y, lwd = 3, col = PatternColors[j])
  }
  legend("topleft", names(ModeratorValues), col = PatternColors, lwd = 3, bty = "n")
}

plot.new()
text(.5, .62, "Same product-term machinery", cex = 1.15)
text(.5, .48, "Different theoretical stories", cex = 1.15)
text(.5, .32, "Check the coding before naming anything", cex = .95)
```




```
par(old_par)
```

21.9 Reverse-Scoring Flips the Signs

If $Z^* = -Z$, the signs of its main effect and interactions reverse. Predictions remain identical after the coding is translated correctly. A negative interaction does not inherently mean antagonism, harm, or Reviewer 2. Its meaning depends on variable coding.

> **Interaction Labels Are Stories, Not New Statistics**
>
> Terms such as *synergistic*, *antagonistic*, and *buffering* describe theoretical patterns. The model still estimates slopes and products. Decide which story applies only after inspecting coding, predictions, and the scientific context.

21.10 Three-Way Interactions

A three-way interaction asks whether a two-way interaction changes across a third variable:

$$Y = \beta_0 + \beta_1 X + \beta_2 Z + \beta_3 W + \beta_4 XZ + \beta_5 XW + \beta_6 ZW + \beta_7 XZW + \epsilon.$$

If $\beta_7 \neq 0$, the X-by-Z interaction depends on W.

```
set.seed(345)
ThreeData <- InterData
ThreeData$Deadline <- factor(
  sample(c("Tomorrow", "Next week"), nrow(ThreeData), replace = TRUE),
  levels = c("Next week", "Tomorrow")
)
Deadline01 <- as.numeric(ThreeData$Deadline == "Tomorrow")

ThreeData$Concentration3 <-
  60 +
  2.0 * ThreeData$Sleep.C +
  .035 * ThreeData$Caffeine.C -
  3.0 * Deadline01 -
  .008 * ThreeData$Sleep.C * ThreeData$Caffeine.C +
  .010 * ThreeData$Sleep.C * ThreeData$Caffeine.C * Deadline01 +
  rnorm(nrow(ThreeData), 0, 5)

ThreeWayModel <- lm(
  Concentration3 ~ Sleep.C * Caffeine.C * Deadline,
  data = ThreeData
)

summary(ThreeWayModel)
```

```
Call:
lm(formula = Concentration3 ~ Sleep.C * Caffeine.C * Deadline,
    data = ThreeData)

Residuals:
     Min       1Q   Median       3Q      Max
-10.7898  -3.2580  -0.3208   3.2044  14.2709

Coefficients:
                                 Estimate Std. Error t value Pr(>|t|)
(Intercept)                     59.798560   0.465421 128.483  < 2e-16 ***
Sleep.C                          2.653541   0.383840   6.913 4.55e-11 ***
Caffeine.C                       0.035025   0.004856   7.212 7.72e-12 ***
DeadlineTomorrow                -3.066370   0.652950  -4.696 4.54e-06 ***
Sleep.C:Caffeine.C              -0.009451   0.003852  -2.454   0.0149 *
Sleep.C:DeadlineTomorrow        -0.336507   0.555119  -0.606   0.5450
```

```
Caffeine.C:DeadlineTomorrow         -0.004959   0.006891  -0.720   0.4725
Sleep.C:Caffeine.C:DeadlineTomorrow  0.008943   0.005685   1.573   0.1171
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.054 on 232 degrees of freedom
Multiple R-squared:  0.4607,    Adjusted R-squared:  0.4444
F-statistic: 28.31 on 7 and 232 DF,  p-value: < 2.2e-16
```

The * operator includes all main effects, all two-way interactions, and the three-way interaction. Keep those lower-order terms. Removing them because they are nonsignificant changes the model's meaning and usually creates interpretive debris.

To interpret a three-way interaction:

1. Plot the predicted two-way relationship at meaningful values of the third variable.
2. Test the two-way interaction within those values or groups.
3. Probe simple slopes only where theory requires them.
4. Report enough numbers that the reader does not have to reconstruct the model with candles and a Ouija board.

21.10.1 Unpack the Three-Way Interaction

First test the sleep-by-caffeine interaction separately within each deadline condition. Then examine caffeine slopes at meaningful sleep values within each condition.

```
emmeans::joint_tests(
  ThreeWayModel,
  by = "Deadline"
)
```

```
Deadline = Next week:
 model term         df1 df2 F.ratio p.value
 Sleep.C              1 232  47.792 <0.0001
 Caffeine.C           1 232  52.020 <0.0001
 Sleep.C:Caffeine.C   1 232   6.020  0.0149

Deadline = Tomorrow:
 model term         df1 df2 F.ratio p.value
 Sleep.C              1 232  33.382 <0.0001
 Caffeine.C           1 232  37.822 <0.0001
 Sleep.C:Caffeine.C   1 232   0.015  0.9034
```

```
ThreeWaySlopes <- emmeans::emtrends(
  ThreeWayModel,
  specs = c("Sleep.C", "Deadline"),
  var = "Caffeine.C",
  at = list(Sleep.C = unname(sleep_values))
)

ThreeWaySlopes
```

```
 Sleep.C Deadline  Caffeine.C.trend      SE  df lower.CL upper.CL
   -1.18 Next week           0.0462 0.00694 232   0.0325   0.0599
    0.00 Next week           0.0350 0.00486 232   0.0255   0.0446
    1.18 Next week           0.0239 0.00636 232   0.0113   0.0364
   -1.18 Tomorrow            0.0307 0.00757 232   0.0158   0.0456
    0.00 Tomorrow            0.0301 0.00489 232   0.0204   0.0397
    1.18 Tomorrow            0.0295 0.00627 232   0.0171   0.0418

Confidence level used: 0.95
```

The three-way coefficient tells us whether the two-way interaction differs between deadline conditions. It does not tell us which simple slope is important, whether the pattern matches the theory, or whether the predicted values are remotely plausible. That requires unpacking.

> **A Three-Way Interaction Is Not a Personality Trait**
>
> Do not predict one merely because you are an ambitious graduate student and your committee has not yet confiscated your keyboard. Three-way interactions demand more power, more careful measurement, and a theory precise enough to explain a changing two-way interaction.

21.11 The Johnson-Neyman Question

Simple slopes at the mean and one standard deviation above and below the mean are convenient. They are not sacred locations handed down by Dorkaos. The **Johnson-Neyman** approach solves for the values of moderator Z where the conditional slope of X changes from statistically distinguishable from zero to not distinguishable from zero.

The conditional slope is

$$b_{X|Z} = b_1 + b_3 Z.$$

Its standard error depends on the variances and covariance of b_1 and b_3:

$$SE(b_{X|Z}) = \sqrt{Var(b_1) + Z^2 Var(b_3) + 2Z Cov(b_1, b_3)}.$$

21.11.1 Solving It, Which Is Just a Quadratic

The boundary we want is where the conditional slope is exactly t_{crit} standard errors from zero:

$$\frac{b_1 + b_3 Z}{SE(b_{X|Z})} = \pm t_{crit}.$$

Square both sides, substitute the standard error, and move everything to one side. The Z terms collect into an ordinary quadratic $AZ^2 + BZ + C = 0$ with

$$A = t_{crit}^2 Var(b_3) - b_3^2, \quad B = 2\left(t_{crit}^2 Cov(b_1, b_3) - b_1 b_3\right), \quad C = t_{crit}^2 Var(b_1) - b_1^2.$$

That is the entire Johnson-Neyman method. There is no new estimator here, no new distribution, and nothing that could not be done with the coefficients and covariance matrix you already have. Packages exist that will do this for you, but the machinery is small enough to look at once.

```
CoefVar <- vcov(InteractionModel)

# Pull these out BY NAME. The rows of vcov() are in model order, so Caffeine.C
# is the third row here, not the second, and positional indexing silently
# hands you the variance of the wrong coefficient. Nothing errors. The roots
# just come out wrong.
b1  <- coef(InteractionModel)["Caffeine.C"]
b3  <- coef(InteractionModel)["Sleep.C:Caffeine.C"]
v11 <- CoefVar["Caffeine.C", "Caffeine.C"]
v33 <- CoefVar["Sleep.C:Caffeine.C", "Sleep.C:Caffeine.C"]
v13 <- CoefVar["Caffeine.C", "Sleep.C:Caffeine.C"]

TCrit <- qt(.975, df.residual(InteractionModel))

A <- TCrit^2 * v33 - b3^2
B <- 2 * (TCrit^2 * v13 - b1 * b3)
C <- TCrit^2 * v11 - b1^2

JNRoots <- sort(c(
  (-B - sqrt(B^2 - 4 * A * C)) / (2 * A),
  (-B + sqrt(B^2 - 4 * A * C)) / (2 * A)
))

# Roots are on the centered scale; add the mean back to read them as hours.
JNHours <- JNRoots + mean(InterData$Sleep)

rbind(Centered = JNRoots, Hours = JNHours)
```

```
         Caffeine.C Caffeine.C
Centered  -3.215866 -0.8036323
Hours      3.353304  5.7655371
```

The two boundaries land at 3.35 and 5.77 hours of sleep, and they split the sleep axis into three regions:

- Below about 3.4 hours, caffeine's slope is significantly **positive**.
- Between 3.4 and 5.8 hours, the slope is not distinguishable from zero.
- Above about 5.8 hours, the slope is significantly **negative**. For rested students, more caffeine predicts worse concentration.

Before repeating any of that in a paper, count how many people are actually in each region. This is the step everyone skips.

```
table(cut(
  InterData$Sleep,
  breaks = c(-Inf, JNHours, Inf),
  labels = c("Slope positive", "Not distinguishable", "Slope negative")
))
```

```
     Slope positive Not distinguishable      Slope negative
                  1                   53                 186
```

That changes the write-up considerably. Both boundaries are technically inside the observed range of 2.54 to 9.64 hours, so nothing here is extrapolation in the strict sense. But the significantly-positive region contains 1 participant out of 240. The model is confident about a region where the study essentially has no data, because a confidence band is narrow near the middle of the moderator and the boundary happens to fall where the band is still narrow enough to exclude zero.

So report it this way: the upper boundary at 5.77 hours is well supported, with 186 participants above it, and it is the finding. The lower boundary is where the data ran out, not a discovery about sleep-deprived students. **"Inside the observed range" is a weaker guarantee than it sounds, and the fix is to report the counts alongside the region.**

The $\pm 1SD$ convention would have found none of this, and its problem here is more specific than being arbitrary. One standard deviation below the mean is 5.39 hours, which the convention would label "low sleep" in a table and which sits *inside* the not-distinguishable region. So the conventional probe reports a nonsignificant slope at "low sleep" and significant ones at average and high, in a sample whose sleep actually runs down to 2.5 hours. The label is doing work the number cannot support.

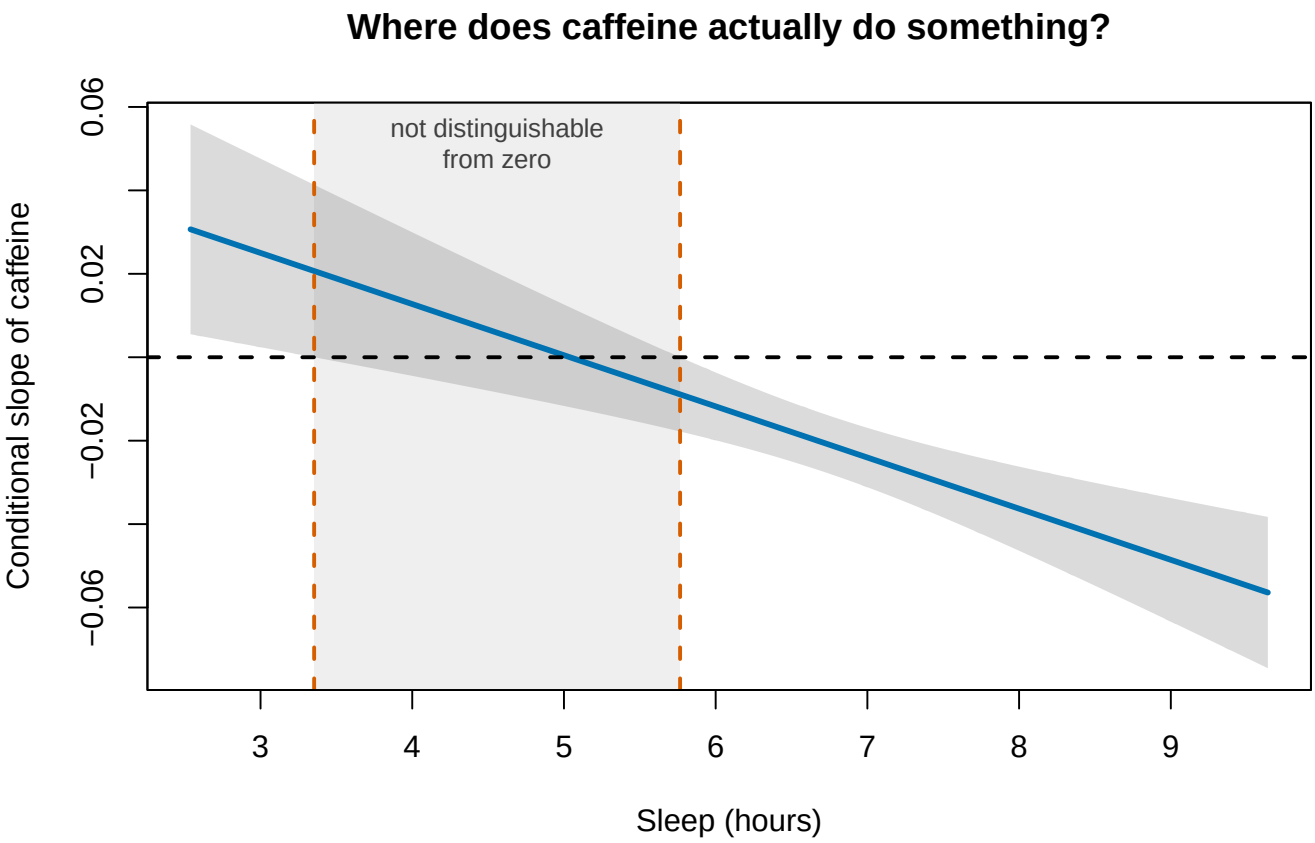
```r
SleepGrid   <- seq(min(InterData$Sleep.C), max(InterData$Sleep.C), length.out = 200)
SlopeGrid   <- b1 + b3 * SleepGrid
SlopeSE     <- sqrt(v11 + SleepGrid^2 * v33 + 2 * SleepGrid * v13)
HoursGrid   <- SleepGrid + mean(InterData$Sleep)

plot(
  HoursGrid, SlopeGrid, type = "n",
  ylim = range(SlopeGrid - TCrit * SlopeSE, SlopeGrid + TCrit * SlopeSE),
  xlab = "Sleep (hours)",
  ylab = "Conditional slope of caffeine",
  main = "Where does caffeine actually do something?"
)

# The region between the two roots is where zero stays inside the interval.
rect(JNHours[1], par("usr")[3], JNHours[2], par("usr")[4],
     col = "#EFEFEF", border = NA)

polygon(
  c(HoursGrid, rev(HoursGrid)),
  c(SlopeGrid - TCrit * SlopeSE, rev(SlopeGrid + TCrit * SlopeSE)),
  col = rgb(0.3, 0.3, 0.3, 0.20), border = NA
)

lines(HoursGrid, SlopeGrid, lwd = 3, col = "#0072B2")
abline(h = 0, lty = 2, lwd = 2)
abline(v = JNHours, lty = 2, lwd = 2, col = "#D55E00")
text(mean(JNHours), max(SlopeGrid + TCrit * SlopeSE) * .92,
     "not distinguishable\nfrom zero", cex = .85, col = "gray25")
box()
```



Notice that the confidence band is narrowest near the middle of the sleep distribution and flares at both ends. That is not a quirk of this dataset. A conditional slope is estimated most precisely where you have the most data, and the $\pm 1SD$ points are convenient partly because they are still in the crowded part of the moderator.

Johnson-Neyman regions can be useful when the moderator is truly continuous. Do not report a region far outside the observed values of Z. The equation is willing to extrapolate into space; your data did not go there. A boundary at 5.8 hours of sleep, with 186 people beyond it, is a finding. A boundary at 22 hours of sleep is a rounding error with ambitions.

21.11.2 The Package That Does All of That in One Call

Here is an R package that will also do this with less code.

```
library(interactions)

# Calculate the J-N intervals and generate the plot
jn_result <- johnson_neyman(
  model = InteractionModel,
  pred = Caffeine.C,
  modx = Sleep.C,
  alpha = 0.05,          # Significance threshold (default is 0.05)
  control.fdr = FALSE    # Set to TRUE to control False Discovery Rate if desired
```

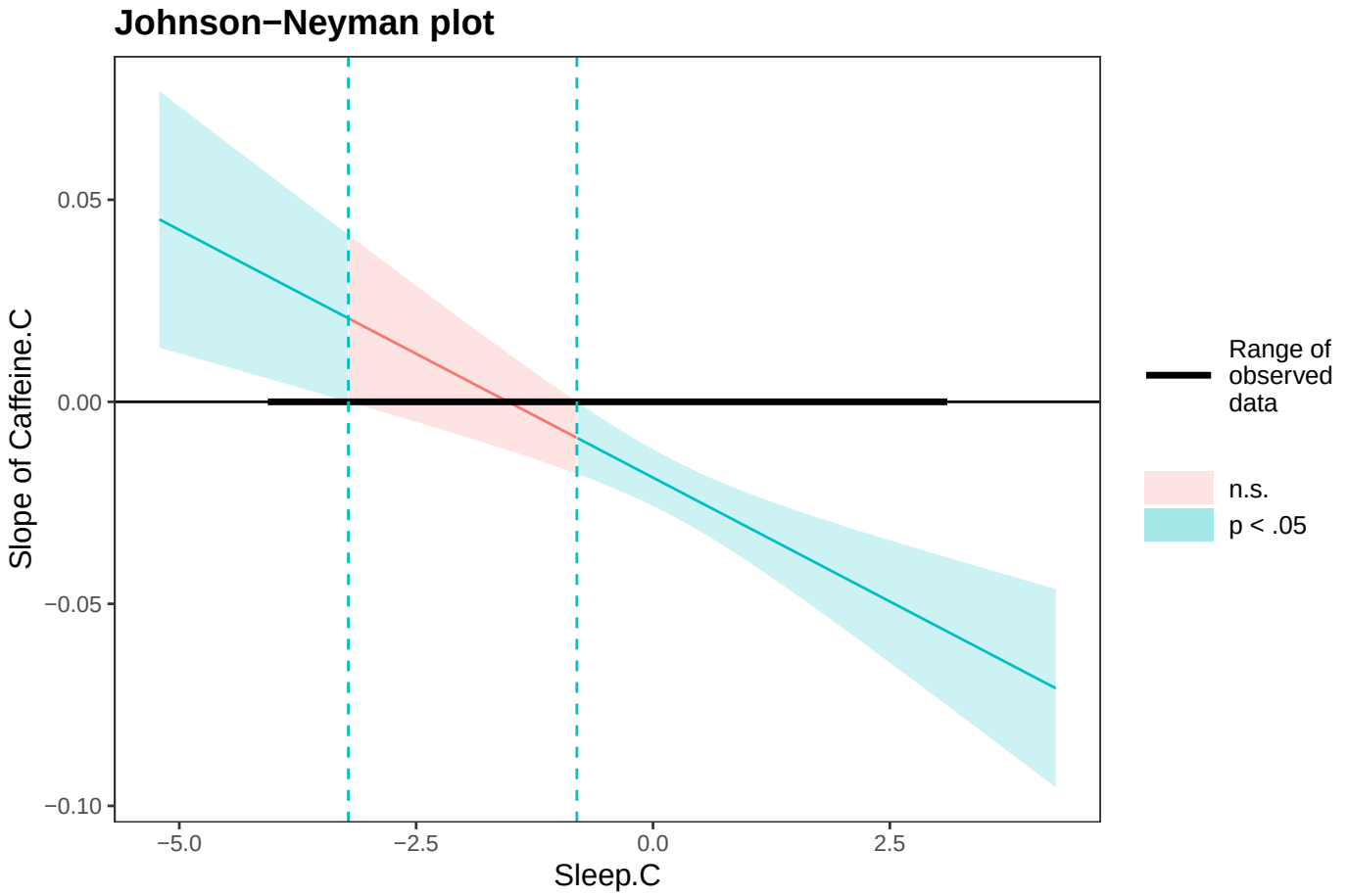
```
)

# Print the text output containing the precise cut-off points
print(jn_result)
```

```
JOHNSON-NEYMAN INTERVAL

When Sleep.C is OUTSIDE the interval [-3.22, -0.80], the slope of
Caffeine.C is p < .05.

Note: The range of observed values of Sleep.C is [-4.03, 3.07]
```



Check its boundaries against the ones we solved for by hand. The package reports the interval on the centered scale, because `Sleep.C` is what is actually in the model, so it prints -3.22 and -0.80 where we reported 3.35 and 5.77 hours. Add the mean of 6.57 back and they are the same two numbers. Not close. The same:

```
c(
  manual.lower  = unname(JNRoots[1]),
  package.lower = unname(jn_result$bounds[1]),
  manual.upper  = unname(JNRoots[2]),
  package.upper = unname(jn_result$bounds[2])
)
```

```
manual.lower package.lower  manual.upper package.upper
  -3.2158656    -3.2158656    -0.8036323    -0.8036323
```

That agreement is the reason for doing it by hand first. The package is not performing a technique you have not already performed with `vcov()` and the quadratic formula. It is doing the same arithmetic faster and drawing a better plot. Use it, now that you could check it.

The plot is worth reading closely, because it makes the shading do the work our hand-built version did with a grey rectangle: pink where the caffeine slope is not distinguishable from zero, blue-green where it is. Note also the thick black bar lying along zero. That marks the range of sleep values **actually observed**, and the plot deliberately draws the line and band past both ends of it. The package is showing you exactly the extrapolation this section just warned about, and trusting you to notice the bar.

Two of its arguments are worth knowing about. `alpha` moves the significance threshold that defines the boundaries, so a stricter alpha widens the middle region where the slope cannot be distinguished from zero. And `control.fdr` switches to an interval that accounts for the fact that a Johnson-Neyman region is secretly an enormous family of simple-slope tests, one at every value of the moderator. Turning it on here pushes the boundaries outward, which is the multiple-comparisons problem from the multiple comparisons chapter turning up somewhere almost nobody looks for it.

21.12 Centering Cannot Remove the Interaction

Mean-centering changes the value represented by zero and therefore changes the lower-order coefficient labels. It does not change fitted values, residuals, R^2, or the interaction coefficient in an ordinary two-way product model. If an interaction disappears merely because you centered correctly, something else in the analysis changed too.

21.13 The Short Story

- The slope of X is $b_1 + b_3 Z$. That derivative **is** the interaction; everything else in this chapter is bookkeeping around it.
- b_1 is the slope of X where $Z = 0$, so make zero mean something. A coefficient in an interaction model is not an effect, it is an effect at a place.
- **Center for interpretation, not for exorcism.** Centering moves the labels. It does not change fitted values, R^2, or the interaction coefficient, and it cannot make a real interaction go away.
- Probe at values theory chose, or use Johnson-Neyman, and stay inside the observed data. $\pm 1SD$ is a convention, not a law, and it can miss where the sign actually changes.
- **Standardize the components before forming the product**, never the product itself. The fit is identical and the reported beta is not.
- Words like *synergistic*, *antagonistic*, and *buffering* are stories about coding. Reverse-score a variable and the label flips while every prediction stays put, so inspect predictions before naming anything.
- A three-way interaction is a two-way interaction that changes. It is not a coefficient with extra punctuation, and it costs power, measurement, and a theory precise enough to justify it.

22 Generalized Linear Models

> ⚠ Historical Lecture
>
> This chapter has not yet been adapted to match the format and style of the rest of the book. It is included in its original lecture structure for readers who need to use generalized linear models.

22.1 Generalized Linear Models (GLMs)

- **This is an entire area of regression, and we could spend a full semester on it. Today's goal is a crash course on the basics of the most common GLM: logistic regression.**
- So far, you have been using a special case of the GLM in which we assume that the underlying distribution is Gaussian.
- Now we will expand our approach to examine the larger family.
- When you run a GLM, you need to state the family and link function you will use.

22.1.1 Family and Link

- Linear regression assumes that the DV is Gaussian, with mean μ, standard deviation σ, and a possible response range from $-\infty$ to ∞.
- GLMs allow us to examine categorical and count responses that cannot satisfy the same requirements as linear regression.
- Other distributions, such as the binomial and Poisson, require a different relationship between the mean and variance.
- We use a link function to connect the mean of the DV to the linear predictor.
- Here are common families used in psychology.

Family	Variance	Link
Gaussian	Gaussian	identity
Binomial	Binomial	logit/probit
Poisson	Poisson	log

22.2 Linear Regression on a Binomial DV

- Let's simulate what happens when we analyze a binomial DV as if it were a plain old regression.
- My grandmother had a very hard time predicting gender in the 90s because clothing became big and baggy, men wore earrings and also started sporting long hair
- Let's simulate her prediction process: gender (DV; 0 = male, 1 = female) based on the hair length (IV; -3 to 3) and bagginess of clothing (IV; -3 to 3) of the person she was looking at.

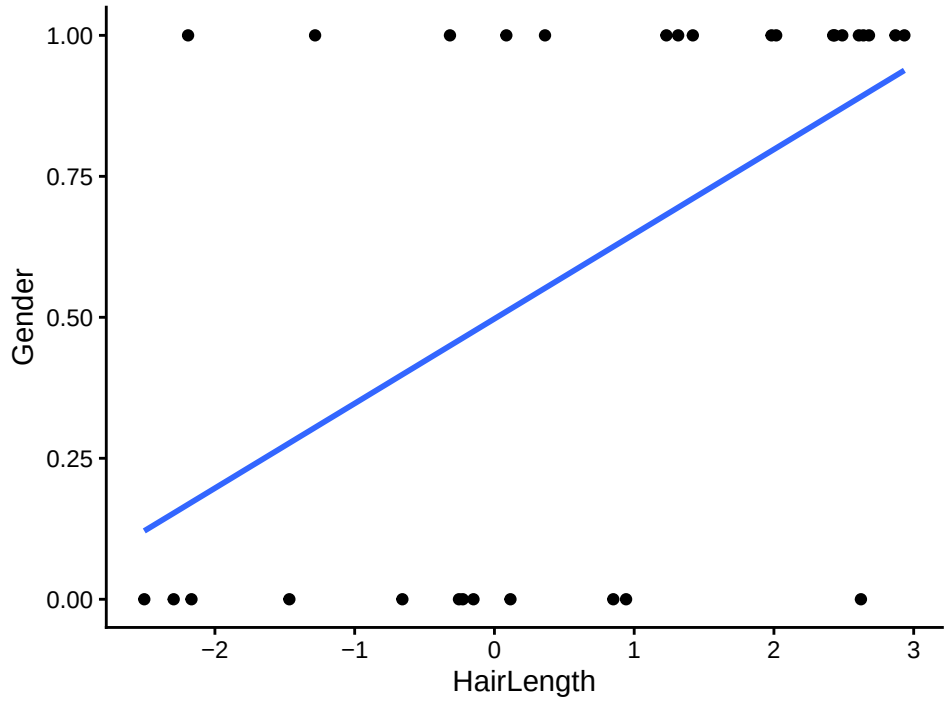
- The goal is to predict gender, not the hair length of each gender, so t-tests are out. We can also add other predictors later.

```
library(ggplot2)

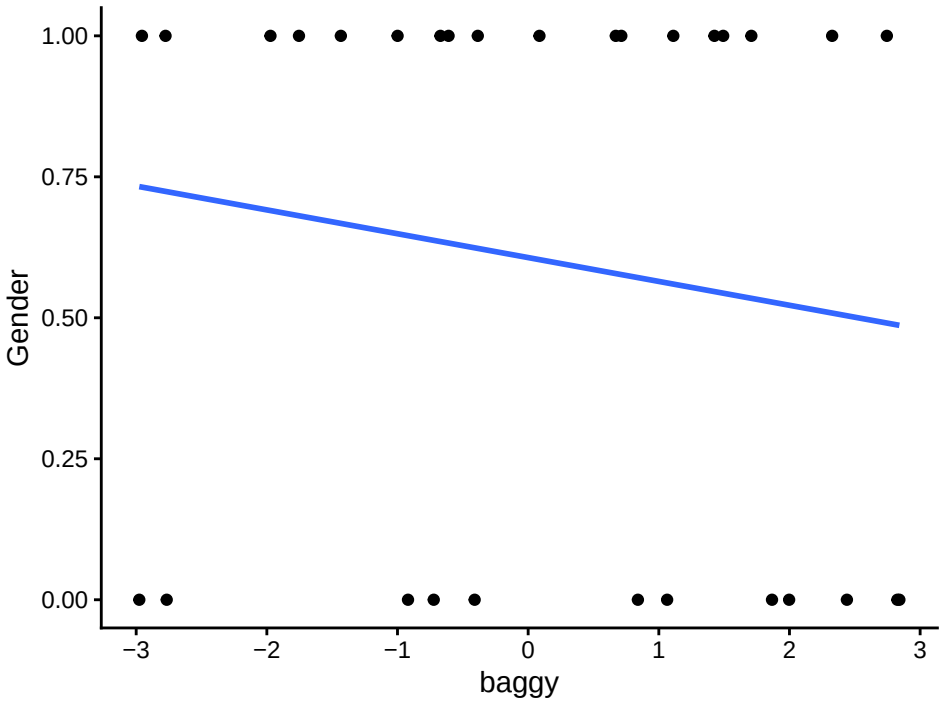
set.seed(42)
n=30
x = runif(n,-3,3) #  Hair length centered [-3 short, 3 = long]
j = runif(n,-3,3) #  Baggy clothing centered [-3 not baggy, 3 = really baggy]
z =  .8*x - .2*j
pr = 1/(1+exp(-z))  # pass through an inv-logit function
y = rbinom(n,1,pr)  # response variable

 #now feed it to glm:
LogisticStudy1= data.frame(Gender=y,HairLength=x, baggy=j)

ggplot(LogisticStudy1, aes(x=HairLength, y=Gender)) + geom_point() +
  stat_smooth(method="lm", formula=y~x, se=FALSE)+
  theme_classic()
```




```
ggplot(LogisticStudy1, aes(x=baggy, y=Gender)) + geom_point() +
  stat_smooth(method="lm", formula=y~x, se=FALSE)+
  theme_classic()
```


1.00
0.75
0.50
0.25
0.00
Gender
−3
−2
−1
0
1
2
3
baggy

```r
LM.1 <- lm(Gender ~ HairLength + baggy, data = LogisticStudy1)
summary(LM.1)
```

```
Call:
lm(formula = Gender ~ HairLength + baggy, data = LogisticStudy1)

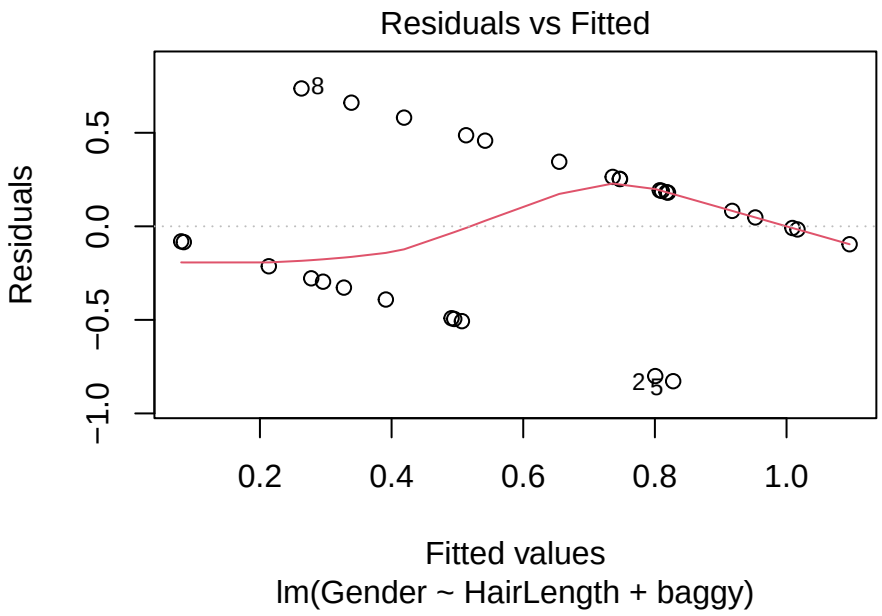
Residuals:
     Min       1Q   Median       3Q      Max
-0.82779 -0.29141  0.01924  0.25321  0.73687

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.50104    0.08278   6.053 1.84e-06 ***
HairLength   0.15984    0.04508   3.546  0.00145 **
baggy       -0.06411    0.04292  -1.494  0.14688
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.4213 on 27 degrees of freedom
Multiple R-squared:  0.3344,    Adjusted R-squared:  0.2851
F-statistic: 6.782 on 2 and 27 DF,  p-value: 0.004106
```

- The intercept reflects the predicted value of gender at the mean hair length and mean clothing bagginess.
- The slope for hair says that as hair gets longer, the model predicts a greater probability of female.
- The slope for bagginess says that as clothes get baggier, the model predicts a greater probability of male, but the slope is not significant.
- How do the residuals look?

```r
plot(LM.1, which =1)
```



- That looks odd because the fitted values are continuous, but the response is binomial.
- Let's see how well we predicted the result for each individual.

```
LogisticStudy1$Predicted.Value<-predict(LM.1)
summary(LogisticStudy1$Predicted.Value)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
0.08036 0.35206 0.59839 0.60000 0.81592 1.09583
```

- Yikes. The model predicted values outside the possible range.
- Let's say any prediction above .5 is female and anything at or below .5 is male.
- Then we will examine a contingency table comparing predicted and observed results.

```
LogisticStudy1$Predicted.Gender<-ifelse(LogisticStudy1$Predicted.Value > .5,1,0)
C.Table<-with(LogisticStudy1,
   table(Predicted.Gender, Gender))
C.Table
```

```
                Gender
Predicted.Gender  0  1
              0   9  3
              1   3 15
```

- Correct predictions are 0-0 and 1-1; incorrect predictions are mismatches.

```
PercentPredicted<-(C.Table[1,1]+C.Table[2,2])/sum(C.Table)*100
PercentPredicted
```

```
[1] 80
```

- The model did a good job making predictions in this simple case, but the ordinary R^2 is not appropriate for a binary outcome.

22.2.1 Summary

- Using linear regression on these data produced predictions outside the bounded range, a violation of homoscedasticity, and an R^2 that is not appropriate for this outcome.
- Instead, let's try a GLM. First, we need to understand the binomial distribution and logit link function.

22.3 Logistic Regression

- Logistic regression is the regression model we use here to analyze a binomial DV.

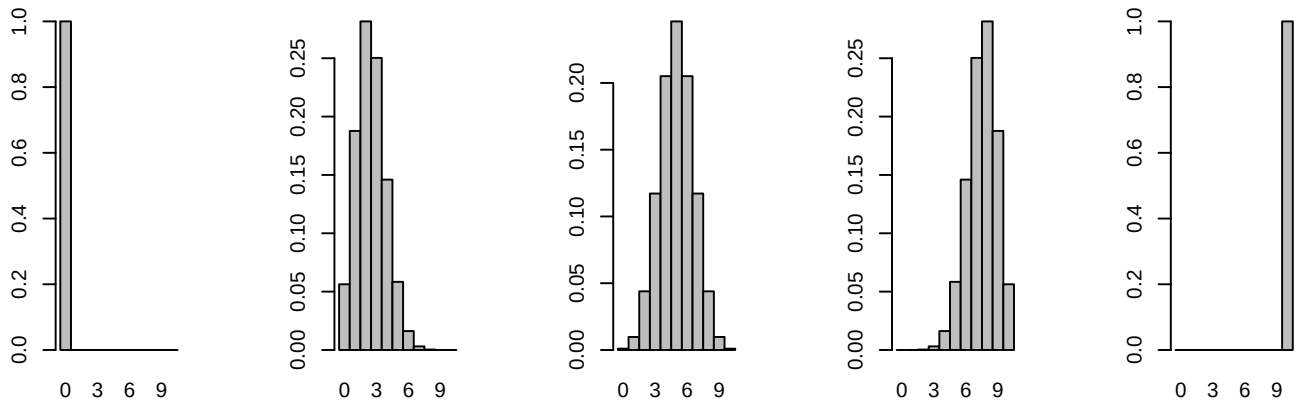
22.3.1 Binomial Distribution

- The binomial probability mass function can be expressed as follows:

$$p(n \mid N) = \binom{N}{n} p^n (1-p)^{N-n}$$

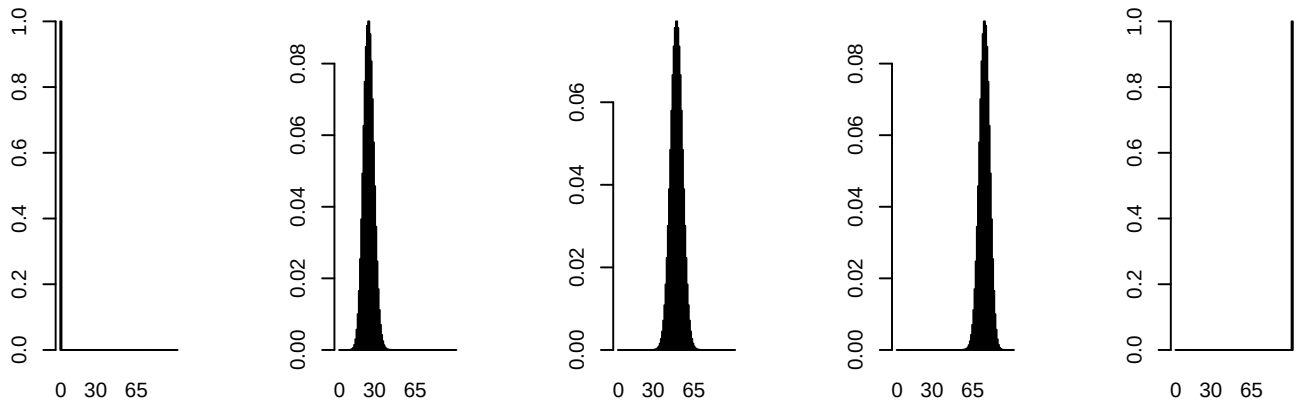
- Here, n is the number of successes in N trials at a specified probability p.
- The distribution changes as a function of the underlying probability of obtaining a 0 or 1.
- The plot below shows $N = 10$ trials, with probability changing from 0 to 1 in increments of .25.

```
par(mfrow=c(1, 5))
for(p in seq(0, 1, len=5))
{
    x <- dbinom(0:10, size=10, p=p)
    barplot(x, names.arg=0:10, space=0)
}
```



- As the number of trials increases, the distribution looks more normal except near the boundaries. Here are 100 trials.

```r
par(mfrow=c(1, 5))
for(p in seq(0, 1, len=5))
{
    x <- dbinom(0:100, size=100, p=p)
    barplot(x, names.arg=0:100, space=0)
}
```



22.3.2 Logit Link Function

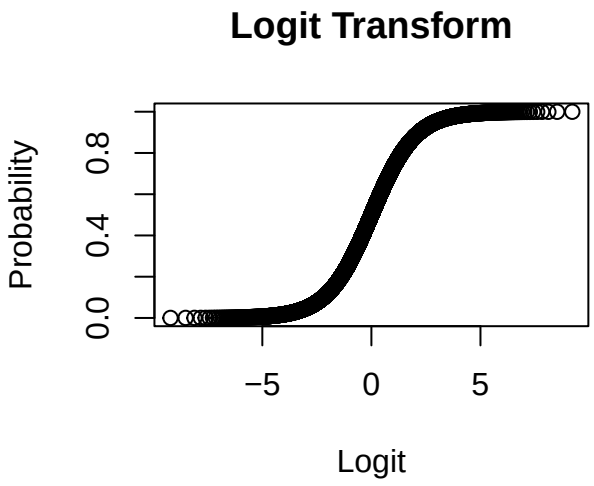
- We can bound our results by making the best-fitting curve asymptotic to the response boundaries.
- To make this work, we need to switch from a straight line to a sigmoid curve.
- The linear predictor can range from $-\infty$ to ∞, so we transform the probability with the logit link.

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right)$$

- I will use log for the natural logarithm to be consistent with R. Some texts write the natural logarithm as ln.

```r
logit.Transform<-function(p) {log(p/(1-p)) }

plot(logit.Transform(seq(0,1,.0001)),seq(0,1,.0001),
     main="Logit Transform",ylim = c(0,1),
  xlab="Logit",ylab="Probability")
```



22.3.3 Fit the Logistic Regression

- $\text{logit}(Gender) = B_1(HairLength) + B_2(baggy) + B_0$
- We can accomplish this by changing the function in R from `lm()` to `glm()` and specifying the family as `binomial(link = "logit")`.
- First, let's fit the model and then make sense of the parameters.

```
LR.1 <- glm(
  Gender ~ HairLength + baggy,
  data = LogisticStudy1,
  family = binomial(link = "logit")
)
summary(LR.1)
```

```
Call:
glm(formula = Gender ~ HairLength + baggy, family = binomial(link = "logit"),
    data = LogisticStudy1)

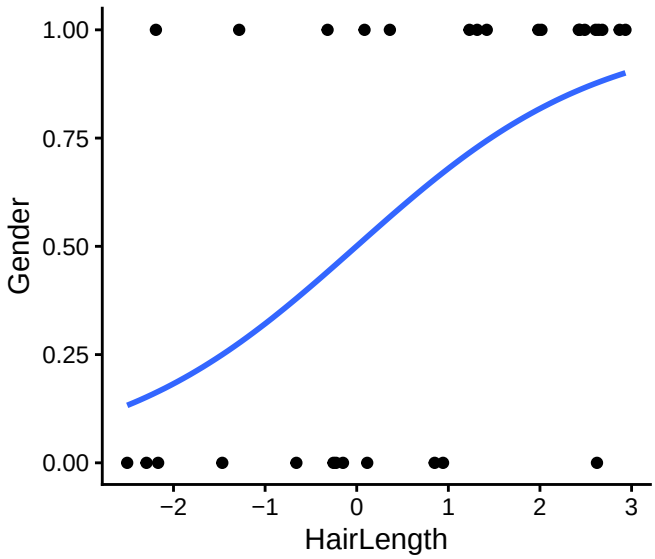
Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)   0.0849     0.4767   0.178  0.85864
HairLength    0.8695     0.3213   2.706  0.00681 **
baggy        -0.4288     0.3069  -1.397  0.16238
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

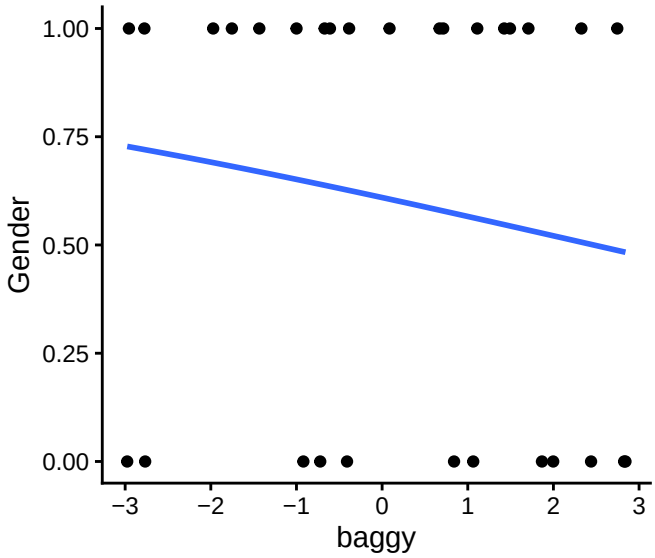
    Null deviance: 40.381  on 29  degrees of freedom
Residual deviance: 28.955  on 27  degrees of freedom
AIC: 34.955
```

```
Number of Fisher Scoring iterations: 5
```

```
library(ggplot2)
ggplot(LogisticStudy1, aes(x=HairLength, y=Gender)) + geom_point() +
  stat_smooth(method="glm", method.args=list(family="binomial"), se=FALSE)+
  theme_classic()
```



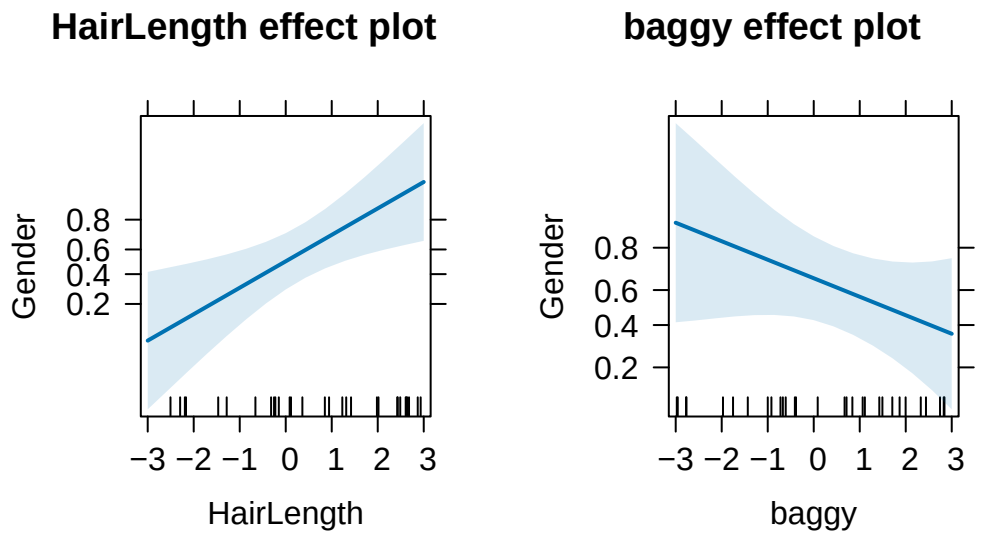

```
ggplot(LogisticStudy1, aes(x=baggy, y=Gender)) + geom_point() +
  stat_smooth(method="glm", method.args=list(family="binomial"), se=FALSE)+
  theme_classic()
```



22.3.4 Plot in Probabilities

- The y-axis shows the predicted probability of gender as a function of hair length.

```
library(effects)
PredictedData<-allEffects(LR.1)
plot(PredictedData)
```



- How did we get the probability from the logit?

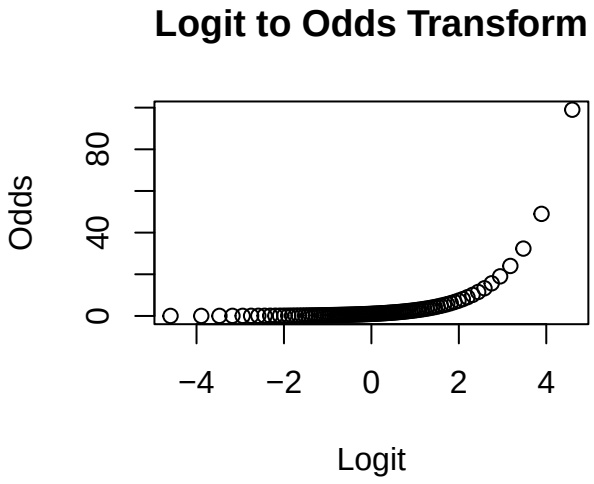
22.3.5 Interpret Coefficients

- Raw coefficients are in log-odds, which are difficult to interpret directly.
- We can transform them into odds or predicted probabilities.
- The odds of success are the ratio of the probability of success to the probability of failure.
- A 50% probability corresponds to odds of 1 to 1.

$$Odds = e^{logit(x)}$$

```
L.to.O.Transform<-function(p) {exp(logit.Transform(p))}

plot((logit.Transform(seq(0,.99,.01))),L.to.O.Transform(seq(0,.99,.01)),
     main="Logit to Odds Transform",
  xlab="Logit",ylab="Odds")
```



- Here are our regression estimates expressed as odds ratios.

```
LR.1.Odds.Ratios <- exp(coef(LR.1))
LR.1.Odds.Ratios
```

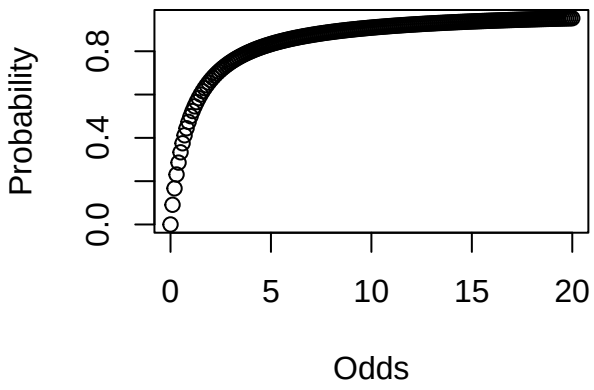
```
(Intercept)  HairLength       baggy
   1.088608    2.385786    0.651300
```

- Physicians often discuss odds, but psychologists usually prefer probabilities.

$$P = \frac{Odds}{1 + Odds}$$

```
O.to.P.Transform <- function(odds) odds / (1 + odds)
odds <- seq(0, 20, .1)

plot(odds, O.to.P.Transform(odds),
     main="Odds to Probability Transform",
  xlab="Odds",ylab="Probability")
```



- We can convert the complete regression equation to obtain predicted probabilities directly.

$$p_{(Gender)} = \frac{e^{(B_1(HairLength)+B_2(baggy)+B_0)}}{1+e^{(B_1(HairLength)+B_2(baggy)+B_0)}}$$

- Or, more simply:

$$p_{(Gender)} = \frac{1}{1+e^{-(B_1(HairLength)+B_2(baggy)+B_0)}}$$

```
prediction_grid <- data.frame(
  HairLength = c(-2, 0, 2),
  baggy = 0
)
prediction_grid$Predicted.Probability <- predict(
  LR.1,
  newdata = prediction_grid,
  type = "response"
)
prediction_grid
```

```
  HairLength baggy Predicted.Probability
1         -2     0             0.1605479
2          0     0             0.5212122
3          2     0             0.8610402
```

- Report the coefficients from the logistic regression, and translate important results into odds ratios or predicted probabilities.
- A coefficient cannot be converted into a probability by itself because the probability depends on the complete linear predictor, including the intercept and values of the other predictors.
- As in linear regression, add the relevant predictor terms first and then apply the inverse-logit transformation shown above.

22.3.6 Wait: What About My R-Squared?

- Ordinary least-squares R^2 does not carry over unchanged to these models.
- A binary outcome's conditional variance depends on its mean, so homoscedasticity does not apply in the same way.
- Researchers have developed several pseudo-R^2 measures to describe fit or improvement over a null model.
- Common versions include Cox and Snell, Nagelkerke, McFadden, and Efron.
- An accessible description of these measures is available from UCLA Statistical Consulting.
- Because many of these measures compare a fitted model with a restricted or null model, we first need to examine model fitting for GLMs.

22.3.6.1 Deviance Testing

- Ordinary least squares provides the fitting criterion for linear regression. GLMs commonly use maximum likelihood instead.
- In OLS regression, we used this decomposition:

$$SS_{Resid} = SS_Y - SS_{Regression}$$

- In other words, residual = observed value - fitted value.
- For most GLMs, the fitting algorithm iterates until it finds the parameter values that maximize the likelihood.
- The null model contains only an intercept. A more complex model contains the intercept plus predictors.

$$\text{Likelihood ratio} = \frac{L_{Simple}}{L_{Complex}}$$

- The likelihood-ratio statistic is:

$$G^2 = -2\log\left(\frac{L_{Simple}}{L_{Complex}}\right) = 2(LL_{Complex} - LL_{Simple})$$

First, I will show you pseudo-R^2. Then we will examine how to test differences in model fit.

- Null deviance:

$$D_{Null} = -2[\log(L_{Null}) - \log(L_{Saturated})]$$

- Model deviance:

$$D_K = -2[\log(L_K) - \log(L_{Saturated})]$$

- This plays a role similar to residual sums of squares in OLS.

22.3.6.2 Pseudo-R-Squared

- We need deviance scores for some pseudo-R^2 measures. For example:

$$R^2_L = \frac{D_{Null} - D_k}{D_{Null}}$$

- SPSS reports Cox and Snell, so people use it often.
- Nagelkerke rescales Cox and Snell so that its maximum can reach 1.
- McFadden's version is also common.

$$R^2_{McFadden} = 1 - \frac{LL_k}{LL_{Null}}$$

- We can obtain several measures from the `pscl` package.
- `llh` is the log-likelihood from the fitted model.
- `llhNull` is the log-likelihood from the intercept-only restricted model.
- $G^2 = 2(LL_K - LL_{NULL})$ is the likelihood-ratio statistic comparing the fitted and null models.
- McFadden = McFadden pseudo-R^2
- r2ML = Cox & Snell pseudo-R^2
- r2CU = Nagelkerke pseudo-R^2

```
library(pscl)
pR2(LR.1)
```

```
fitting null model for pseudo-r2
```

```
        llh     llhNull          G2    McFadden        r2ML        r2CU
-14.4774392 -20.1903500  11.4258216   0.2829525   0.3167270   0.4281675
```

22.3.6.3 Wait, Why Do I See *z* and Not *t*-Values?

- Tests of individual predictors use Wald z-statistics rather than t-tests.
- The Wald z-statistic for a coefficient is:

$$z = \frac{B_j}{SE_{B_j}}$$

- Squaring this statistic gives a one-degree-of-freedom Wald chi-square statistic:

$$\chi^2_{Wald} = \left(\frac{B_j}{SE_{B_j}}\right)^2$$

- You can also request sequential likelihood-ratio tests. The order of predictors matters for this table.

```
anova(LR.1, test="Chisq")
```

```
Analysis of Deviance Table

Model: binomial, link: logit

Response: Gender

Terms added sequentially (first to last)

           Df Deviance Resid. Df Resid. Dev Pr(>Chi)
NULL                         29     40.381
HairLength  1   9.0638        28     31.317 0.002607 **
baggy       1   2.3620        27     28.955 0.124322
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

22.3.7 Hierarchical Testing

- Sequential testing can be difficult if you have many predictors.
- Instead of testing a change in ordinary R^2, we compare the deviance of nested models.
- A likelihood-ratio test compares the models using a chi-square reference distribution.

```
LR.Model.1<-glm(Gender~baggy,data=LogisticStudy1, family=binomial(link = "logit"))
LR.Model.2<-glm(Gender~baggy+HairLength,data=LogisticStudy1, family=binomial(link = "logit"))
anova(LR.Model.1,LR.Model.2,test = "Chisq")
```

```
Analysis of Deviance Table

Model 1: Gender ~ baggy
Model 2: Gender ~ baggy + HairLength
  Resid. Df Resid. Dev Df Deviance Pr(>Chi)
1        28     39.639
2        27     28.955  1   10.684 0.001081 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

- Here, Model 2 has lower residual deviance and fits better, so hair length improved prediction.

22.3.8 How Well Am I Predicting?

- Model 1

```
fitted.results <- predict(LR.Model.1,newdata=LogisticStudy1,type='response')
fitted.results <- ifelse(fitted.results > 0.5,1,0)
misClasificError <- mean(fitted.results != LogisticStudy1$Gender)
print(paste('Accuracy = ',round(1-misClasificError,3)))
```

```
[1] "Accuracy =  0.633"
```

- Model 2

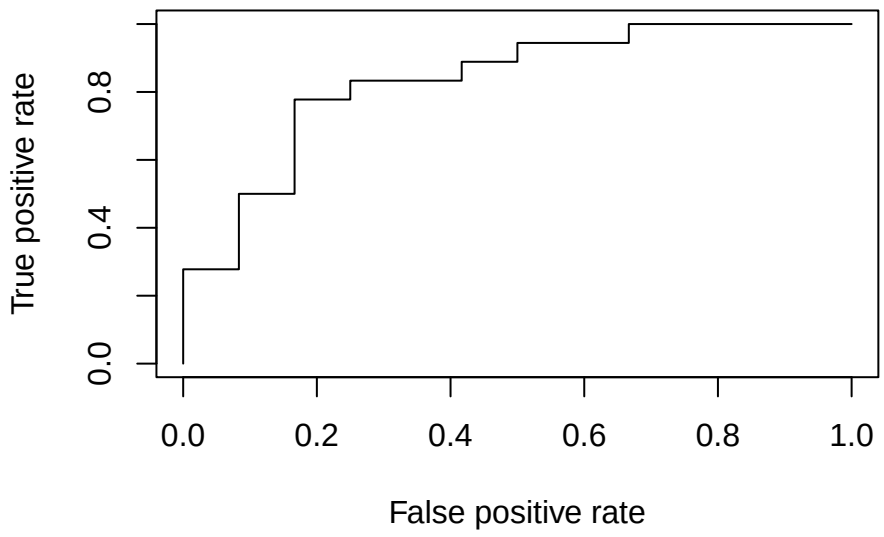
```
fitted.results <- predict(LR.Model.2,newdata=LogisticStudy1,type='response')
fitted.results <- ifelse(fitted.results > 0.5,1,0)
misClasificError <- mean(fitted.results != LogisticStudy1$Gender)
print(paste('Accuracy = ',round(1-misClasificError,3)))
```

```
[1] "Accuracy =  0.767"
```

- This is not the only way to examine accuracy.
- We have correct responses, misses, and false alarms. Remember our Type I and Type II error boxes.
- To visualize this relationship, we will examine a receiver operating characteristic (ROC) curve.
- We will calculate the area under the curve as a measure of discrimination. Values closer to 1 indicate better discrimination, whereas .5 indicates chance-level discrimination.

```
library(ROCR)
fitted.results <- predict(LR.Model.2,newdata=LogisticStudy1,type='response')
pr <- prediction(fitted.results, LogisticStudy1$Gender)

prf <- performance(pr, measure = "tpr", x.measure = "fpr")
plot(prf)
```




```
auc <- performance(pr, measure = "auc")
auc <- auc@y.values[[1]]
print(paste('Area under the Curve = ',round(auc,3)))
```

```
[1] "Area under the Curve =  0.833"
```

23 Mediation

> ⚠ Historical Lecture
>
> This chapter has not yet been adapted to match the format and style of the rest of the book. It is included in its original lecture structure for readers who need to conduct and interpret a mediation analysis.

23.1 Mediation

- Baron and Kenny (1986) had 66,271 citations as of March 27, 2017.
- See Kenny's mediation page for useful information.
- Mediation is extremely common in social psychology and is now used throughout psychology.
- Baron and Kenny define moderator and mediator as follows:

  "The moderator function of third variables, which partitions a focal independent variable into subgroups that establish its domains of maximal effectiveness regarding a given dependent variable"

  "The mediator function of a third variable, which represents the generative mechanism through which the focal independent variable can influence the dependent variable of interest"

- In short, the mediator describes a pathway through which X may be related to Y.
- Having one mediator is called simple mediation.

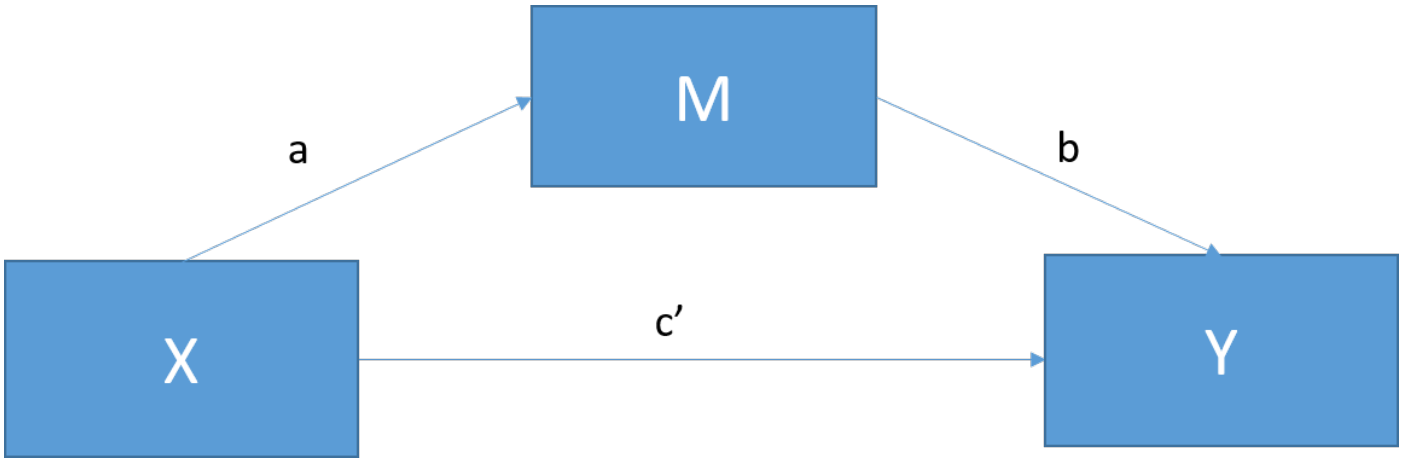


Figure 23.1: Simple mediation

Preacher and Hayes (2008) explain that the relationship between X and Y can be separated into the direct effect c' and the indirect effect through M. Path a represents the relationship between X and M, path b represents the relationship between M and Y while controlling for X, and the indirect effect is the product ab.

- Mediation attempts to explain part of the total relationship through an indirect path.

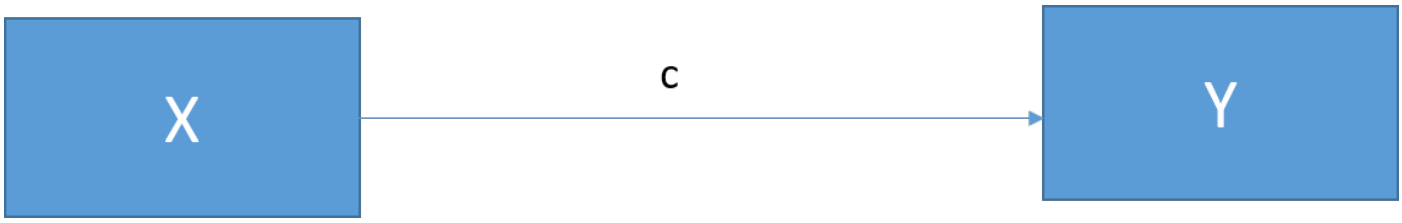


Figure 23.2: Total effect

- We can connect the total effect to simple mediation:

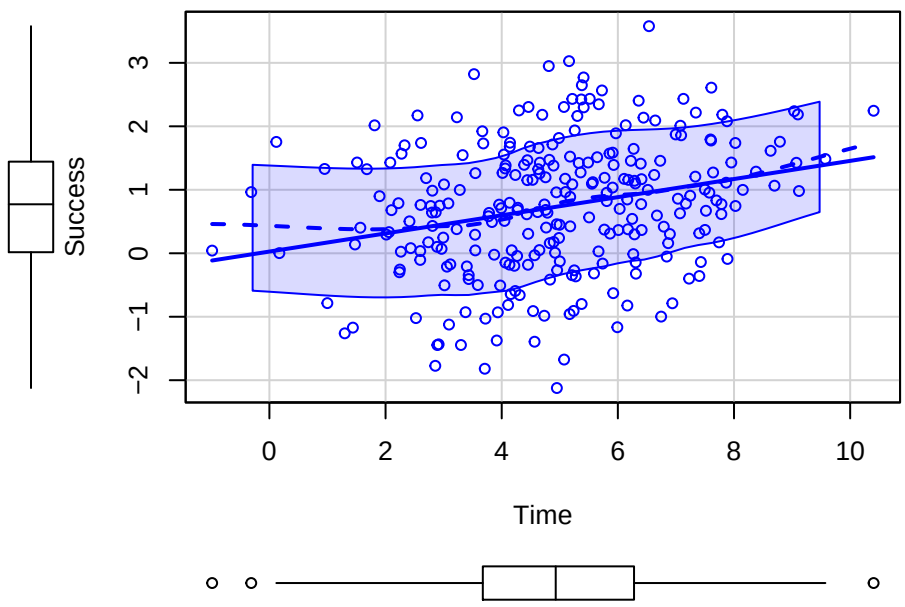
$$c = c' + ab$$

$$total = direct + indirect$$

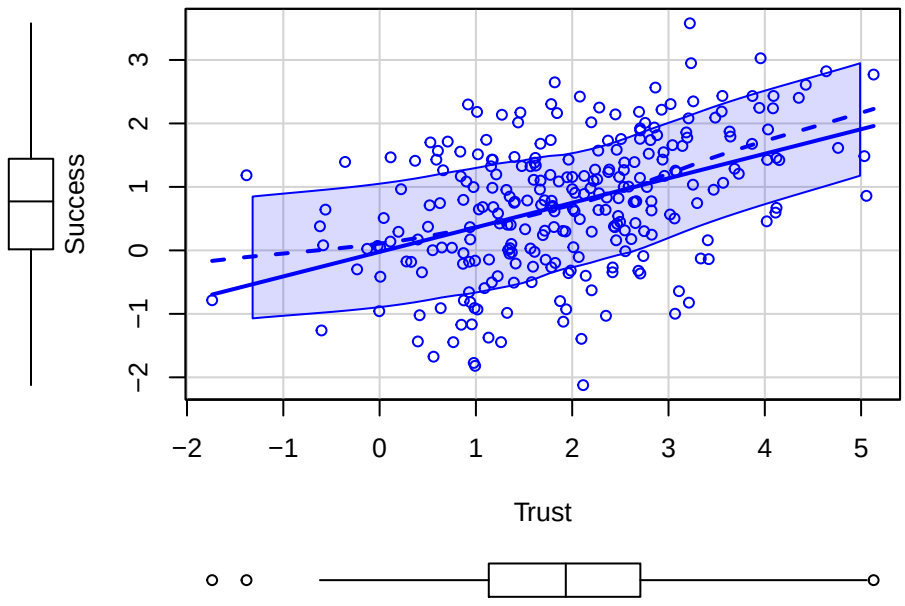
23.1.1 A Hypothetical Example of Mediation

- Children with the ability to delay gratification tend to be more successful later in life, as illustrated by the marshmallow test.
- You hypothesize that their trust in authority figures to keep promises is an intermediate step in the pathway to later success.
- You collect data from 268 four-year-olds and give them the marshmallow test, measuring how long they wait before eating the marshmallow. Rather than waiting 20 years, you operationalize success as performance on a kindergarten entrance exam at the end of the year. You also measure how much they trust authority figures on a 1-to-10 scale using an established battery for children.

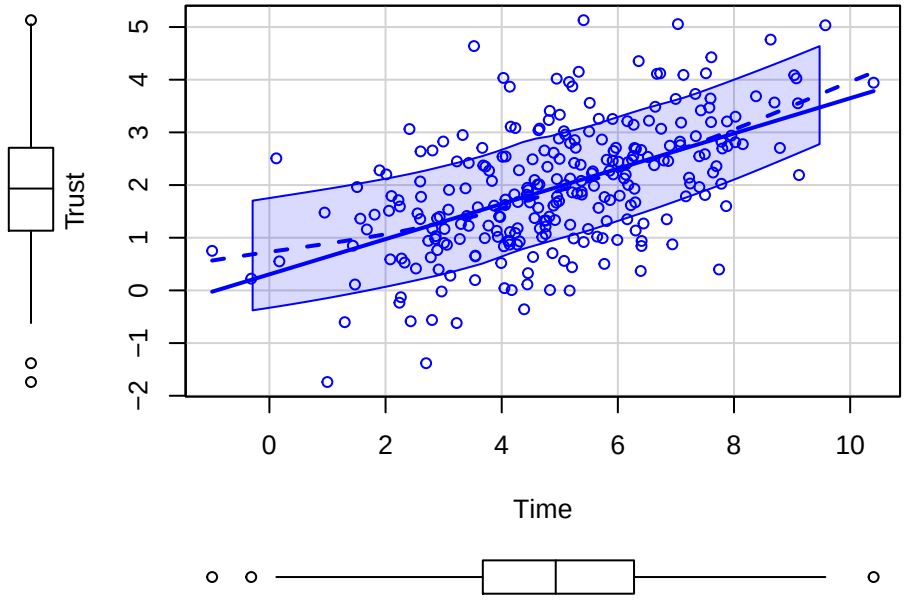
```
library(car)
set.seed(42)
# For simulation of mediation steps see Hallgren, 2013
# path a strength
a=.4
# path b strength
b=.4
# path c' strength
cp=.01
# people
n <- 268
# Normal distribution of time (mins)
X <- rnorm(n, 5, 2)
# Mediator
M <- a*X+rnorm(n, 0, 1)
# Our equation to  create Y
Y <- cp*X + b*M + rnorm(n, sd=1)
#Built our data frame
Marshmallow.Data<-data.frame(Success=Y,Time=X,Trust=M)
# plots
scatterplot(Success~Time, data= Marshmallow.Data, smoother=FALSE)
```




```
scatterplot(Success~Trust, data= Marshmallow.Data, smoother=FALSE)
```




```
scatterplot(Trust~Time, data= Marshmallow.Data,smoother=FALSE)
```



23.1.1.1 Baron and Kenny Steps for Testing Mediation

- Step 1: Test $Y \sim X$.
- Is there a relationship? In the traditional procedure, you proceeded only if the total effect was significant. Modern mediation analysis does not require a significant total effect before testing the indirect effect.

```
Model.1<-lm(Success~Time, data= Marshmallow.Data)
summary(Model.1)
```

```
Call:
lm(formula = Success ~ Time, data = Marshmallow.Data)

Residuals:
     Min       1Q   Median       3Q      Max
-2.85692 -0.65355  0.04275  0.71568  2.61690

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.02687    0.16932   0.159    0.874
Time         0.14286    0.03187   4.483  1.1e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.01 on 266 degrees of freedom
Multiple R-squared:  0.07024,   Adjusted R-squared:  0.06675
F-statistic:  20.1 on 1 and 266 DF,  p-value: 1.096e-05
```

- Step 2: Test $M \sim X$.

- This model estimates path a.

```
Model.2<-lm(Trust~Time, data= Marshmallow.Data)
summary(Model.2)
```

```
Call:
lm(formula = Trust ~ Time, data = Marshmallow.Data)

Residuals:
     Min       1Q   Median       3Q      Max
-2.58912 -0.62407 -0.01152  0.64980  3.15542

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.30489    0.16781   1.817   0.0704 .
Time         0.33436    0.03158  10.586   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.001 on 266 degrees of freedom
Multiple R-squared:  0.2964,    Adjusted R-squared:  0.2938
F-statistic: 112.1 on 1 and 266 DF,  p-value: < 2.2e-16
```

- Step 3: Test $Y \sim M + X$.
- This model estimates path b while controlling for X and estimates the direct effect c' while controlling for M.

```
Model.3<-lm(Success~Trust+Time, data= Marshmallow.Data)
summary(Model.3)
```

```
Call:
lm(formula = Success ~ Trust + Time, data = Marshmallow.Data)

Residuals:
     Min       1Q   Median       3Q      Max
-2.91375 -0.57108  0.06568  0.68179  2.34683

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.08574    0.15884  -0.540    0.590
Trust        0.36934    0.05768   6.403 6.89e-10 ***
Time         0.01937    0.03542   0.547    0.585
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.9417 on 265 degrees of freedom
Multiple R-squared:  0.1948,    Adjusted R-squared:  0.1887
F-statistic: 32.06 on 2 and 265 DF,  p-value: 3.395e-13
```

- Step 4: Inspect c' in Model 3 and test the indirect effect ab directly.
- Historically, researchers used the labels *complete mediation* when c' was no longer significant and *partial mediation* when c' remained significant but became smaller. These labels depend too heavily on sample size and are no longer recommended as the main conclusion.

23.1.1.2 Interpretation

- In this simulation, the direct path c' is close to zero.
- Time is not significant in Model 3, but it could be significant in a larger sample.
- The data are consistent with part of the relationship between time and success operating through trust.
- How do we test the indirect effect, path ab?

23.1.1.3 Significance of the Indirect Path

- In our example, paths a and b are both strong and significant. This is called joint significance.
- Joint significance provides evidence consistent with an indirect effect, but we should test and report the indirect effect directly.
- The Sobel test estimates the standard error of ab in the equation $c = c' + ab$ and tests the product against zero.
- The Sobel test relies on a normal approximation for the product ab. That sampling distribution is often asymmetric, especially in smaller samples, so the test can have low power.
- Other tests exist, but the Sobel test was the most common in its day.

```
library(bda)
mediation.test(Marshmallow.Data$Trust,Marshmallow.Data$Time,Marshmallow.Data$Success)
```

```
                Sobel       Aroian      Goodman
z.value 5.478920e+00 5.461111e+00 5.496905e+00
p.value 4.279292e-08 4.731637e-08 3.865154e-08
```

- Bootstrapping has largely replaced the Sobel test, so we will focus on bootstrap intervals in the examples.
- Two common bootstrap intervals are the bias-corrected and accelerated (BCa) interval and the percentile interval.
- Their performance differs across situations; the important point is to name the interval you use and report the number of bootstrap draws.

23.1.1.3.1 The `mediation` Package in R

- You need Model 2 ($M \sim X$) and Model 3 ($Y \sim M + X$) from above.
- The terminology changes because this package can handle larger designs with control variables and several types of regression models.

```
library(mediation)
Med.Boot.BCa <- mediate(Model.2, Model.3, boot = TRUE,
                        boot.ci.type = "bca", sims=200, treat="Time", mediator="Trust")
summary(Med.Boot.BCa)
```

```
Causal Mediation Analysis

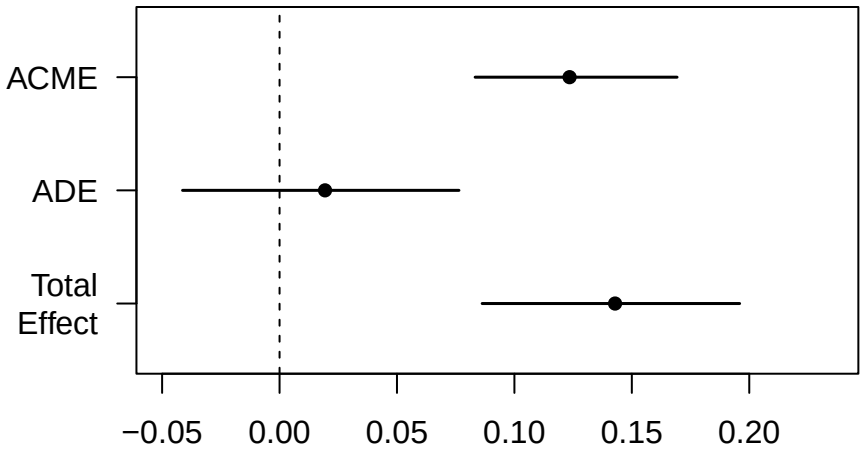
Nonparametric Bootstrap Confidence Intervals with the BCa Method

                Estimate 95% CI Lower 95% CI Upper p-value
ACME            0.123493     0.083273     0.169255  <2e-16 ***
ADE             0.019368    -0.041295     0.076380    0.53
Total Effect    0.142860     0.086269     0.195890  <2e-16 ***
Prop. Mediated  0.864428     0.598614     1.814922  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Sample Size Used: 268


Simulations: 200
```

```
plot(Med.Boot.BCa)
```



- ACME: average causal mediation effect [total effect - direct effect]
- ADE: average direct effect [total effect - indirect effect]
- Total Effect = Direct (ADE) + Indirect (ACME)
- These are unstandardized effects
- Proportion mediated: conceptually, ACME / total effect
- Note: I used 200 simulations to keep this lecture example quick, but use substantially more for a real analysis.
- By changing the code, we can request a percentile interval instead of BCa.

```
Med.Boot.perc <- mediate(Model.2, Model.3, boot = TRUE,
                         boot.ci.type = "perc", sims=200, treat="Time", mediator="Trust")
summary(Med.Boot.perc)
```

```
Causal Mediation Analysis

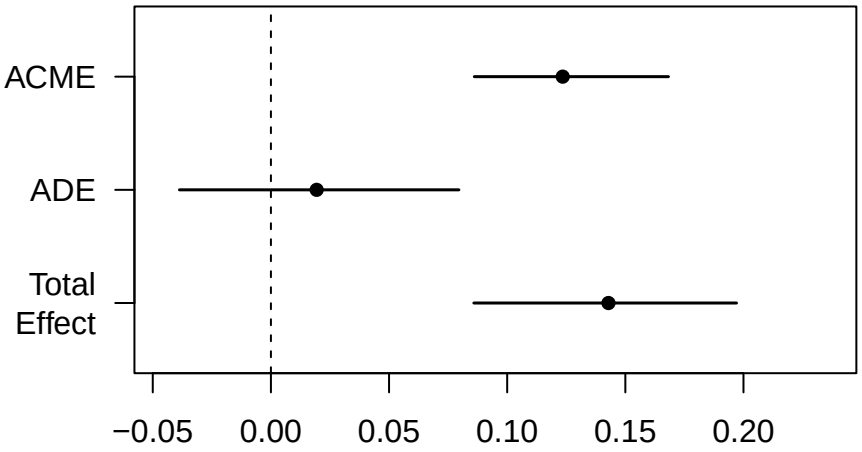
Nonparametric Bootstrap Confidence Intervals with the Percentile Method

                Estimate 95% CI Lower 95% CI Upper p-value
ACME            0.123493     0.086087     0.168238  <2e-16 ***
ADE             0.019368    -0.038699     0.079556    0.54
Total Effect    0.142860     0.085883     0.197044  <2e-16 ***
Prop. Mediated  0.864428     0.570264     1.399996  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Sample Size Used: 268


Simulations: 200
```

```
plot(Med.Boot.perc)
```



- These results align with the pattern from the regression steps.
- One benefit is that the package can accommodate several kinds of mediator and outcome models.
- More complicated mediation models may require additional packages or custom code.

23.1.1.4 Testing the Mediator as a Moderator?

- Could it be that our mediator is really just a moderator?

23.1.2 Power

- When you plan a mediation analysis, think carefully about the power you will need.
- Indirect effects are products of coefficients and often require larger samples than expected.
- Kenny developed an application for estimating power in simple designs.
- Larger designs may require custom simulations. The `powerMediation` package can help with some designs.

23.1.3 Partial Mediation

- Historically, *partial mediation* referred to a pattern in which the direct and indirect effects were both present.
- Low power can make the labels *complete* and *partial* unstable.
- Following Hayes (2013), avoid making the main conclusion that mediation is complete or partial. Report the direct and indirect effects with their uncertainty.
- Below, I created a simulation with power near .80 to examine both effects.

```
library(car)
set.seed(42)
# For simulation of mediation steps see Hallgren, 2013
# path a strength
```

```r
a=.4
# path b strength
b=.4
# path c' strength
cp=.2
# people
n <- 177
# Normal distribution of time (mins)
X <- rnorm(n, 5, 2)
# Mediator
M <- a*X+rnorm(n, 0, 1)
# Our equation to  create Y
Y <- cp*X + b*M + rnorm(n, sd=1)
#Built our data frame
Marshmallow.Data.Part<-data.frame(Success=Y,Time=X,Trust=M)

Part.Model.2<-lm(Trust~Time, data= Marshmallow.Data.Part)
summary(Part.Model.2)
```

```
Call:
lm(formula = Trust ~ Time, data = Marshmallow.Data.Part)

Residuals:
    Min      1Q  Median      3Q     Max
-2.6575 -0.6674  0.0206  0.6008  2.4748

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.13523    0.18694  -0.723     0.47
Time         0.42292    0.03516  12.028   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.9276 on 175 degrees of freedom
Multiple R-squared:  0.4526,    Adjusted R-squared:  0.4494
F-statistic: 144.7 on 1 and 175 DF,  p-value: < 2.2e-16
```

```r
Part.Model.3<-lm(Success~Trust+Time, data= Marshmallow.Data.Part)
summary(Part.Model.3)
```

```
Call:
lm(formula = Success ~ Trust + Time, data = Marshmallow.Data.Part)

Residuals:
    Min      1Q  Median      3Q     Max
-2.4400 -0.6523 -0.0782  0.6563  3.2448
```

```
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.10836    0.21285   0.509  0.61135
Trust        0.40810    0.08594   4.749 4.26e-06 ***
Time         0.17242    0.05403   3.191  0.00168 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.055 on 174 degrees of freedom
Multiple R-squared:  0.358, Adjusted R-squared:  0.3506
F-statistic: 48.52 on 2 and 174 DF,  p-value: < 2.2e-16
```

```
Part.Med.Boot.BCa <- mediate(Part.Model.2, Part.Model.3, boot = TRUE,
                             boot.ci.type = "bca", sims=200, treat="Time", mediator="Trust")
summary(Part.Med.Boot.BCa)
```

```
Causal Mediation Analysis

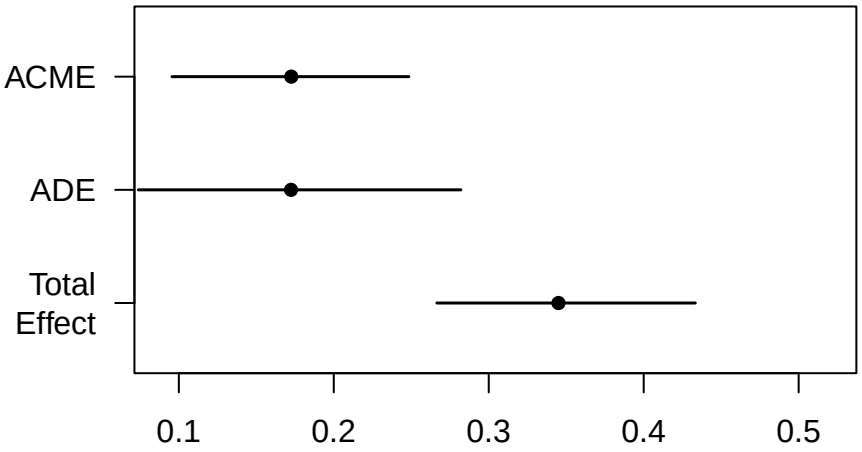
Nonparametric Bootstrap Confidence Intervals with the BCa Method

               Estimate 95% CI Lower 95% CI Upper   p-value
ACME           0.172593     0.095605     0.248438 < 2.2e-16 ***
ADE            0.172423     0.073841     0.282019 < 2.2e-16 ***
Total Effect   0.345016     0.266533     0.433325 < 2.2e-16 ***
Prop. Mediated 0.500247     0.264405     0.751547 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Sample Size Used: 177


Simulations: 200
```

```
plot(Part.Med.Boot.BCa)
```


ACME
ADE
Total
Effect
0.1
0.2
0.3
0.4
0.5

24 Moderated Mediation

> ⚠ Historical Lecture
>
> This chapter has not yet been adapted to match the format and style of the rest of the book. It is included in its original lecture structure for readers who need to conduct and interpret a moderated mediation analysis.

24.1 Moderated Mediation

- Moderation examines factors that influence the strength of a relationship between X and Y.
- Mediation examines an intermediate pathway connecting X and Y.
- Moderated mediation asks whether an indirect effect depends on a moderator.
- In practical terms, what if parenting style moderated the mediation through trust in authority from the previous chapter?
- Strict parents might foster respect for authority, whereas permissive parents might foster opposition to authority.
- We will measure parenting style from -3 to 3, ranging from permissive to authoritative.
- Our hypothesis is that the indirect effect through trust is stronger for children with authoritative parents than for children with permissive parents.
- Children with authoritative parents might also be more well-behaved in general, so we will examine moderation of both direct and indirect paths.

```
library(mediation)

set.seed(42)

# Path a strength
a <- .4

# Path b strength
b <- .4

# Path c' strength
cp <- .01

# People
n <- 268

# Normal distribution of time (minutes)
X <- rnorm(n, 5, 2)

# Moderator
Mod <- runif(n, -3, 3)
```

```r
# Mediator
M <- a * X * Mod + rnorm(n, 0, 1)

# Equation used to create Y
Y <- cp * X * Mod + b * M + M * Mod + rnorm(n, sd = 1)

# Build the data frame
Marshmallow.Mod <- data.frame(
  Success = Y,
  Time = X,
  Trust = M,
  Parents = Mod
)
```

24.1.1 Moderation on Paths a, b, and c'

- In this example, we think that parenting style moderates the direct path (c') and both parts of the indirect path (a and b).
- We therefore include interactions with the moderator in both models.
- The mediator model becomes $M \sim X * Mod$.
- The outcome model becomes $Y \sim M * Mod + X * Mod$.
- This corresponds to Hayes's PROCESS Model 59.

```r
Mod.Med.Model.2 <- lm(
  Trust ~ Time * Parents,
  data = Marshmallow.Mod
)
summary(Mod.Med.Model.2)
```

```
Call:
lm(formula = Trust ~ Time * Parents, data = Marshmallow.Mod)

Residuals:
    Min      1Q  Median      3Q     Max
-2.8374 -0.6565  0.0026  0.6875  3.2869

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.064912   0.173395  -0.374  0.70844
Time         0.001584   0.032561   0.049  0.96125
Parents     -0.281470   0.100098  -2.812  0.00529 **
Time:Parents 0.455080   0.019547  23.281  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.022 on 264 degrees of freedom
Multiple R-squared:  0.9225,    Adjusted R-squared:  0.9216
```

```
F-statistic:  1047 on 3 and 264 DF,  p-value: < 2.2e-16
```

```
Mod.Med.Model.3 <- lm(
  Success ~ Trust * Parents + Time * Parents,
  data = Marshmallow.Mod
)
summary(Mod.Med.Model.3)
```

```
Call:
lm(formula = Success ~ Trust * Parents + Time * Parents, data = Marshmallow.Mod)

Residuals:
    Min      1Q  Median      3Q     Max
-2.9024 -0.6513  0.0677  0.5975  3.2446

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)   -0.05491    0.16485  -0.333    0.739
Trust          0.38262    0.05842   6.549 3.04e-10 ***
Parents        0.02400    0.09669   0.248    0.804
Time          -0.01450    0.03244  -0.447    0.655
Trust:Parents  1.00946    0.01052  95.955  < 2e-16 ***
Parents:Time   0.01251    0.03245   0.385    0.700
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.9705 on 262 degrees of freedom
Multiple R-squared:  0.9768,    Adjusted R-squared:  0.9763
F-statistic:  2203 on 5 and 262 DF,  p-value: < 2.2e-16
```

- In the `mediation` package, we list the moderator as a covariate and specify the values at which we want to estimate the effects.
- We will use one standard deviation below and above the mean. We could also use zero to examine the effect at the average parenting-style score.
- This allows us to examine the conditional direct and indirect effects.
- Let's look at permissive parents first.

```
Permissive <- mean(Marshmallow.Mod$Parents) - sd(Marshmallow.Mod$Parents)
Permissive
```

```
[1] -1.680635
```

```
Mod.Med.Boot.BCa.1 <- mediate(
  Mod.Med.Model.2,
  Mod.Med.Model.3,
  covariates = list(Parents = Permissive),
  boot = TRUE,
  boot.ci.type = "bca",
```

```
  sims = 200,
  treat = "Time",
  mediator = "Trust"
)
summary(Mod.Med.Boot.BCa.1)
```

```
Causal Mediation Analysis

Nonparametric Bootstrap Confidence Intervals with the BCa Method

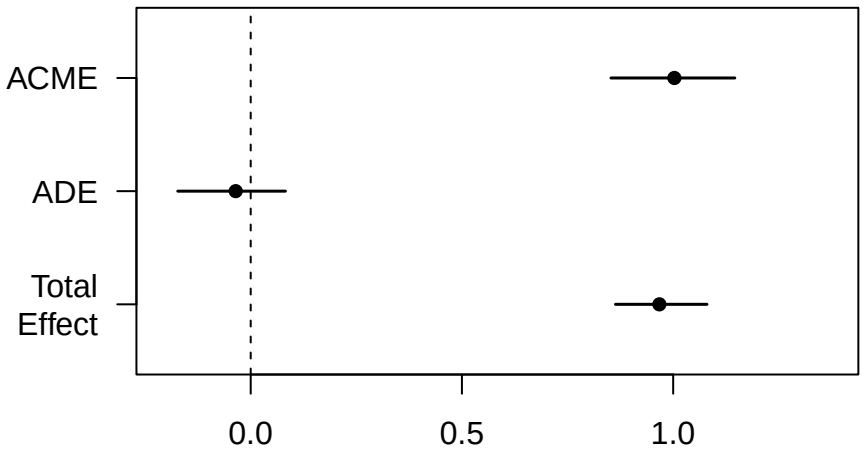
(Inference Conditional on the Covariate Values Specified in `covariates')

                Estimate 95% CI Lower 95% CI Upper p-value
ACME            1.002825     0.852665     1.145835  <2e-16 ***
ADE            -0.035519    -0.172622     0.082280    0.62
Total Effect    0.967307     0.863385     1.079992  <2e-16 ***
Prop. Mediated  1.036719     0.920462     1.179271  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Sample Size Used: 268


Simulations: 200
```

```
plot(Mod.Med.Boot.BCa.1)
```



- Now, let's look at authoritative parents.

```r
Authoritative <- mean(Marshmallow.Mod$Parents) + sd(Marshmallow.Mod$Parents)
Authoritative
```

```
[1] 1.71852
```

```r
Mod.Med.Boot.BCa.2 <- mediate(
  Mod.Med.Model.2,
  Mod.Med.Model.3,
  covariates = list(Parents = Authoritative),
  boot = TRUE,
  boot.ci.type = "bca",
  sims = 200,
  treat = "Time",
  mediator = "Trust"
)
summary(Mod.Med.Boot.BCa.2)
```

```
Causal Mediation Analysis

Nonparametric Bootstrap Confidence Intervals with the BCa Method

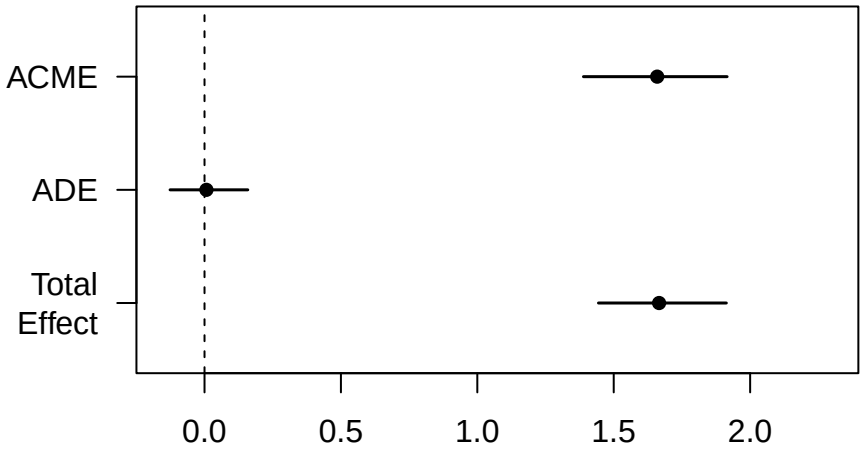
(Inference Conditional on the Covariate Values Specified in `covariates')

                 Estimate 95% CI Lower 95% CI Upper p-value
ACME            1.6592943    1.3890530    1.9155855  <2e-16 ***
ADE             0.0069974   -0.1262825    0.1586092    0.83
Total Effect    1.6662917    1.4440059    1.9127474  <2e-16 ***
Prop. Mediated  0.9958006    0.9168537    1.0735322  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Sample Size Used: 268


Simulations: 200
```

```r
plot(Mod.Med.Boot.BCa.2)
```



- To test whether the conditional direct or indirect effects differ across moderator values, we can use `test.modmed()`.

```
Mod.Med.Boot.BCa.3 <- mediate(
  Mod.Med.Model.2,
  Mod.Med.Model.3,
  boot = TRUE,
  boot.ci.type = "bca",
  sims = 200,
  treat = "Time",
  mediator = "Trust"
)

test.modmed(
  Mod.Med.Boot.BCa.3,
  covariates.1 = list(Parents = Permissive),
  covariates.2 = list(Parents = Authoritative),
  sims = 200
)
```

```
    Test of ACME(covariates.1) - ACME(covariates.2) = 0

data:  estimates from Mod.Med.Boot.BCa.3
ACME(covariates.1) - ACME(covariates.2) = -0.65647, p-value < 2.2e-16
alternative hypothesis: true ACME(covariates.1) - ACME(covariates.2) is not equal to 0
95 percent confidence interval:
 -0.8919021 -0.4049856
```

```
    Test of ADE(covariates.1) - ADE(covariates.2) = 0

data:  estimates from Mod.Med.Boot.BCa.3
ADE(covariates.1) - ADE(covariates.2) = -0.042516, p-value = 0.7
alternative hypothesis: true ADE(covariates.1) - ADE(covariates.2) is not equal to 0
95 percent confidence interval:
 -0.1982374  0.1478085
```

- In this simulation, the conditional indirect effects differ, but the conditional direct effects do not.
- The data are therefore consistent with the indirect effect being moderated by parenting style in the predicted direction.
- The 200 bootstrap draws keep this lecture example quick. Use substantially more draws for a real analysis.

Part IV

ANOVA is dead, Long Live ANOVA

25 Historical ANOVA Lectures

25.1 About These Chapters

This section contains historical lectures on how to conduct, interpret, and write up analyses of variance (ANOVAs) in R. The lectures cover:

- one-way and two-way between-subjects ANOVAs;
- one-way and two-way within-subjects ANOVAs;
- mixed-design ANOVAs; and
- common follow-up analyses.

These lectures have not yet been adapted to read like the other chapters in this book. They are included in their original lecture style for readers who need to conduct an ANOVA now. They will eventually be revised and integrated more fully with the rest of the book.

26 Introduction to Analysis of Variance (ANOVA)

Note: Some complex R code was used to generate today's lecture. I have hidden it in the rendered chapter. If you see `echo = FALSE` inside the `.qmd` file, that code is not something you are expected to understand or learn. I will explain the functions you need to learn.

26.1 Design Example

We are interested in how much people trust a partner after engaging in different levels of joint action. Two people enter the lab, one of whom is a confederate. They are asked to either (a) move chairs independently, (b) move a couch together while the confederate helps, or (c) move a couch together while the confederate hinders. Moving a couch requires the two people to coordinate their actions. After the study ends, participants rate how trustworthy their partner seems from 1 to 7, with 7 indicating high trust.

No Joint Action	Helpful Joint Action	Disrupted Joint Action
2	5	2
3	4	3
5	7	3
3	4	1
2	5	1
M = 3	M = 5	M = 2
SS = 6	SS = 6	SS = 4

26.1.1 Comparing Three Groups

To use t-tests, I would have to compare G1 with G2, G2 with G3, and G1 with G3. This becomes a problem when I have four or more groups. Each additional test increases the risk of committing a Type I error. A dead salmon will teach us this lesson in a week. We need a way to test all three groups at once to determine whether at least one group differs from another. How do I compare three or more groups at once? I know, let's just fix the t-test formula!

$$t = \frac{M_1 - M_2 - M_3}{\sqrt{\frac{S_p^2}{n_1} + \frac{S_p^2}{n_2} + \frac{S_p^2}{n_3}}}$$

What is wrong with my fix?

26.2 Analysis of Variance

- Analysis of variance (ANOVA) is a procedure for testing hypotheses with two or more treatments.
- ANOVA can handle multiple samples, while t-tests can only compare two samples.
- ANOVA is used to decide whether:

  - The treatments have no effect
  - Treatments do have differences

26.2.1 ANOVAs vs. *t*-Tests

ANOVAs are closely related to t-tests: $F = t^2$. This is why the F-distribution has no negative tail. Hypothesis testing with ANOVAs works much like hypothesis testing with t-tests. A t-test is essentially a ratio of the mean difference to noise (error or chance), while ANOVA uses an F ratio: variance caused by the treatment divided by variance due to noise.

26.2.1.1 Types of One-Way ANOVAs

- Fixed-factor ANOVAs, which will be the focus of the rest of the semester
- Random-factor ANOVAs, which have been replaced by mixed models in many applications and which I teach a whole course on

26.2.2 Terms

- FACTOR: Independent or quasi-independent variable (the manipulated variable)
- Factors can have *LEVELS* that make up the factor

  For example, you are testing how temperature (factor) affects concentration. You test four groups at four different temperatures (levels) to see which temperature has the strongest effect.

26.2.3 Types of studies

- Like t-tests, ANOVAs can use repeated-measures or independent-measures designs.
  - A repeated-measures design tests the same participant multiple times, often over time.
  - An independent-measures design uses a separate group for each level of a factor. We will focus on these designs for a few weeks.
- A mixed design combines repeated and independent measures. We will review this design after learning about repeated-measures and independent-measures ANOVAs.

26.2.4 Logic of ANOVA

ANOVA asks, "Is one of these things not like the others?"



H_0: All groups have equal means.

H_1: At least one group differs from the others.

ANOVA does not tell us which group is different, and the F-distribution has no negative tail for reasons you will see below. We will learn how to identify the different group next class.

26.2.4.1 Logic of F-testing

- First, we need to analyze the variance. It is in the name of ANOVA, right?
  - There are two components we need to calculate:

$$F = \frac{S^2_{Treatment}}{S^2_{Error}}$$

    * Between-treatments variance: we will calculate the variance between groups! *This is the variance we* **caused**.
    * Within-treatment variance: the variance within each cell is the error term. *That is the variance due to* **chance**. As with a t-test, we assume that the variance in group 1 equals the variance in group 2 and group 3. This is the homogeneity of variance (HOV) assumption.
- We can rewrite our F-test as $F = \frac{S^2_B}{S^2_W}$.

  Note: ANOVA uses a more technical name for a variance estimate: mean square, or MS for short. Thus, when you see MS, it is **conceptually equivalent** to S^2. The square root of MS is conceptually equivalent to S.

- In the notation most people will recognize:

$$F = \frac{MS_B}{MS_W}$$

26.2.4.2 Between-Subjects Variance

- Two explanations for between-treatment variance:
  - Treatment effect
  - Chance
    - Individual differences: what if one person likes it cold and others function better at very warm temps
    - Experimental error: just by taking the measurement you can cause an error

26.2.4.3 Within-Treatment Variance

- There is one explanation for within-treatment variance if HOV is met and there are no confounding or nuisance variables:
  - Noise/chance differences (individual differences) > However, imagine that different RAs ran each condition and each gave the instructions differently. Condition would be confounded with RA, which could affect the mean differences or add variance that should have been controlled.

26.2.4.4 F-Ratio Logic

$$F = \frac{MS_B}{MS_W}$$

$$F = \frac{Treatment(with\ Noise)}{Noise}$$

If the treatment effect is 0 but there is noise:

$$F = \frac{(0\ with\ Noise)}{Noise} = 1$$

Thus, when $F = 1$, you have no treatment effect.

But if the treatment effect is 2 and the noise is 1:

$$F = \frac{Treatment(with\ Noise)}{Noise} = \frac{2+1}{1} = 3$$

26.3 Analysis Procedures

We return to our study data. We will run our analysis first by hand and examine each stage of the calculation. We will next do it in R.

No Joint Action	Helpful Joint Action	Disrupted Joint Action
2	5	2
3	4	3
5	7	3
3	4	1
2	5	1
M = 3	M = 5	M = 2
SS = 6	SS = 6	SS = 4

26.3.1 By Hand Calculation

These are simplified formulas for equal sample sizes.

26.3.1.1 Between-Subject Variance

Remember that $S^2 = \frac{SS}{n-1} = \frac{\Sigma(X-M)^2}{n-1}$. We can transform this into $MS_B = \frac{SS_B}{k-1} = \frac{n\Sigma(M-G)^2}{df_{BT}}$, where $G = \frac{\Sigma M}{K}$ is the grand mean, K is the number of factor levels, and n is the number of people per cell. This formula only works with equal sample sizes.

26.3.1.1.1 Calculation

1. $K = 3$
2. $G = \frac{3+5+2}{3} = 3.33$
3. $df_{BT} = 3 - 1 = 2$
4. $n = 5$
5. $SS_{BT} = 5 * [(3 - 3.33)^2 + (5 - 3.33)^2 + (2 - 3.33)^2] = 23.33$
6. $MS_{BT} = \frac{SS_B}{df_B} = \frac{23.33}{2} = 11.67$

26.3.1.2 Within-Subject Variance

Here we calculate $MS_{WT} = \frac{SS_W}{N-K} = \frac{\Sigma(SS_{cell})}{df_W}$. Calculate $SS_W = \Sigma(X - M)^2$ for each group, then sum the values.

26.3.1.2.1 Calculation

1. $N = n * k = 5 * 3 = 15$
2. $df_W = 15 - 3 = 12$
3. $SS_W = 6 + 6 + 4 = 16$
4. $MS_W = \frac{16}{12} = 1.33$

26.3.1.3 *F*-Value

1. $F = \frac{MS_B}{MS_W} = \frac{11.67}{1.33} = 8.75$

26.3.1.4 Significance

You will need to look up two things in the F table: df_{Num} and df_{den}. These are the degrees of freedom that go into the numerator and denominator. See the handout for tables and this Shiny app: https://shiny.rit.albany.edu/stat/fdist/

The F_{crit} changes with df, as it does in a t-test. When $K = 2$, $F_{crit} = t^2_{crit}$.

26.3.2 Helper Table

That is hard to track, so we build a source table to show what we are doing.

Source	SS	DF	MS	F
Between	SS_B	df_B	MS_B	F
Within	SS_W	df_W	MS_W	
Total	SS_T	df_T		

With the formulas (formal equations)

Source	SS	DF	MS	F
Between	$n\sum_{i=1}^{k}(M_i - G)^2$	$K - 1$	$\frac{SS_B}{df_B}$	$\frac{MS_B}{MS_W}$
Within	$\sum_{i=1}^{K}\sum_{j=1}^{n}(X_{i_j} - M_i)^2$	$N - K$	$\frac{SS_W}{df_W}$	
Total	$\sum_{j=1}^{N}(X_j - G)^2$	$N - 1$		

With the formulas (conceptually simplified)

Source	SS	DF	MS	F
Between	$nSS_{treatment}$	$K - 1$	$\frac{SS_B}{df_B}$	$\frac{MS_B}{MS_W}$
Within	$\sum SS_{within}$	$N - K$	$\frac{SS_W}{df_W}$	
Total	SS_{scores}	$N - 1$		

Note: $SS_B + SS_W = SS_T$ & $df_B + df_W = df_T$

26.3.3 Effect Size

We cannot use Cohen's d. Instead, we will use eta-squared, the percentage of variance accounted for by the treatment. Remember that SS_B is the variation caused by the treatment, while SS_T is the variation due to treatment plus noise.

$$\eta^2 = \frac{SS_B}{SS_B + SS_W} = \frac{SS_B}{SS_T}$$

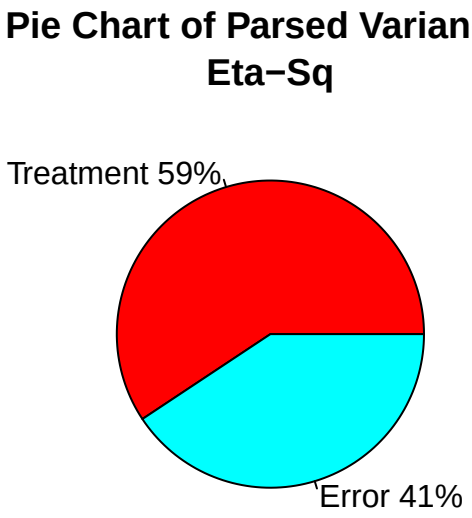
This eta-squared formula for a one-way ANOVA equals the more modern generalized eta-squared, η^2_g. R will therefore report η^2_g (Bakeman, 2005). We will return to these formulas in the future.

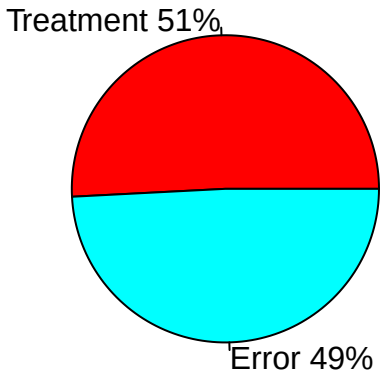
Note: η_g^2 tends to yield values larger than the corresponding population value. Corrections exist, although they are not often reported: ω^2 and ω_g^2.

$$\omega^2 = \frac{SS_B - (K-1)MS_W}{SS_T + MS_W}$$

26.3.3.1 Understanding η^2 (or ω^2)

η^2 is useful for understanding how your ANOVA "parsed" the variance. In fact, η^2 equals R^2 in regression, which tells us how much of the variation in the dots is explained by the line. We can calculate the percentage of variance accounted for because ANOVA is a special case of regression.





26.3.3.2 Effect Size for Power

For power analysis, we will need f, which Cohen developed. As with d for t-tests, a simple *conceptual* conversion is $f = \frac{d}{2}$.

$$f = \sqrt{\frac{K-1}{K}\frac{F}{n}}$$

ES	Value	Size
f	.1	Small effect
f	.25	Medium effect
f	.4	Large effect

$f_{observed} = 1.08$, Large effect!

26.3.4 Using R Functions

The formulas I provided work when there are equal sample sizes and no missing data. With unbalanced cells, we have to decide how to parse the variance because ANOVA is a special case of regression. We will use Type III sums of squares. In future lectures, I will explain the different methods for parsing the variance.

No Joint Action	Helpful Joint Action	Disrupted Joint Action
2	5	2
3	4	3
5	7	3
3	4	1
2	5	1
M = 3	M = 5	M = 2
SS = 6	SS = 6	SS = 4

26.3.4.1 Build Our Data Frame

We can read the data into R from a CSV file, or we can type the data frame by hand. I typed it below.

```
n=5; k=3;
JA.Study<-
  data.frame(SubNum=seq(1:(k*n)),
             Condition=ordered(
                       c(rep("No",n),
                        rep("Helpful",n),
                        rep("Distrupted",n))
                       ),
             DV=c(2,3,5,3,2,5,4,7,4,5,2,3,3,1,1))
```

You can also use `View(JA.Study)` to see it.

26.3.4.2 Check the Means and SS Calculations with dplyr

dplyr grammar: start with the dataset, *THEN* split the data by groups, *THEN* summarize the variables you specify.

```
library(dplyr)
Means.Table<-JA.Study %>%
  group_by(Condition) %>%
  summarise(N=n(),
            Means=mean(DV),
            SS=sum((DV-Means)^2),
            SD=sd(DV),
            SEM=SD/N^.5)
knitr::kable(Means.Table, digits = 2)
```

Condition	N	Means	SS	SD	SEM
Distrupted	5	2	4	1.00	0.45
Helpful	5	5	6	1.22	0.55
No	5	3	6	1.22	0.55

```
# Note remember knitr::kable makes it pretty, but you can just call `Means.Table`
```

Good, our means and SS values match, so I entered the data correctly.

26.3.4.3 Run the ANOVA

`afex` is a package developed to run SPSS-like ANOVAs in R using Type III sums of squares. R defaults to Type I, which we will discuss later. `afex` has several output formats, but the most useful for us are `anova` and `nice`. These formats show different information and let us use the resulting objects in different ways.

26.3.4.3.1 ANOVA-Style Output

- This style is close to our ANOVA source table, but it is formatted like a regression F-source table.
- You can ignore the intercept line (we will cover that next semester).
- Condition = SS_B and the name will always match your IV name.
- Residual = Error. In regression, residual means leftover. This is the error term: chance.
- There is no total (you can do that with addition)

```
library(afex)

ANOVA.JA.Table<-aov_car(DV~Condition + Error(SubNum),
                  data=JA.Study, return='Anova')
ANOVA.JA.Table
```

```
Anova Table (Type III tests)

Response: dv
             Sum Sq Df F value    Pr(>F)
(Intercept) 166.667  1  125.00 1.059e-07 ***
Condition    23.333  2    8.75  0.004531 **
Residuals    16.000 12
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

26.3.4.3.2 Nice-Style Output

- This style reports everything you need in APA format! That is why it is called `nice`, cause it is nice to you!

```
Nice.JA.Table<-aov_car(DV~Condition + Error(SubNum),
                  data=JA.Study, return='nice')
Nice.JA.Table
```

```
Anova Table (Type 3 tests)

Response: DV
     Effect    df  MSE       F  ges p.value
1 Condition 2, 12 1.33 8.75 ** .593    .005
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1
```

26.4 Assumptions

We assume HOV and normality. We can test HOV using the Brown-Forsythe formula, which subtracts the median within each cell and recalculates the ANOVA. If there is no variance imbalance, we should see a result similar to the middle plot below, with a nonsignificant p-value greater than .05. If we violate this assumption, there is a special test called the Brown-Forsythe ANOVA, but almost no one reports it.

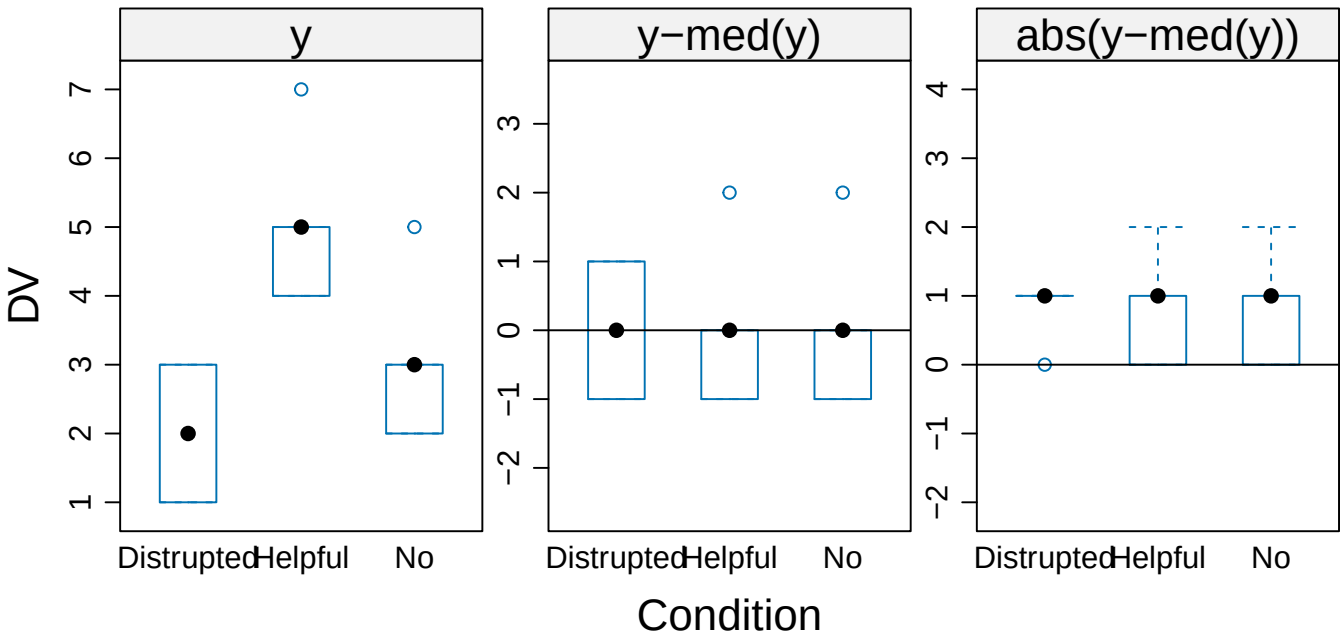
Using the `HH` package, we can visualize violations. As you can see, the middle plot shows no deviation.

```
library(HH)
hov(DV~Condition, data=JA.Study)
```

```
    hov: Brown-Forsyth

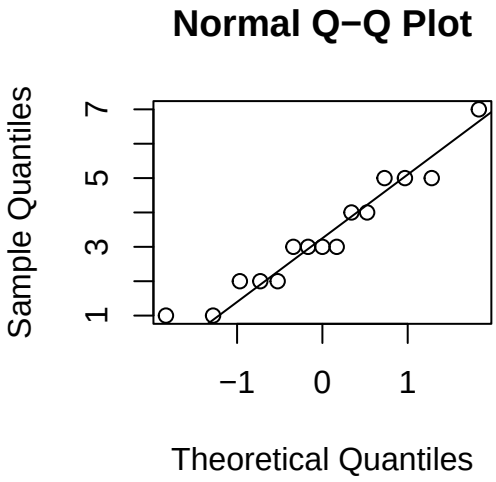
data:  DV
F = 5.7778e-32, df:Condition = 2, df:Residuals = 12, p-value = 1
alternative hypothesis: variances are not identical
```

```
hovPlot(DV~Condition, data=JA.Study)
```



Testing for normality with such a small sample does not make sense. In general, normality tests such as the Shapiro-Wilk test tend to be overly sensitive. We often examine Q-Q plots of the DV. If the data points deviate substantially from the line, that suggests a violation of normality.

```
qqnorm(JA.Study$DV)
qqline(JA.Study$DV)
```



26.5 APA-Style Report

$F(df_n, df_d) = x.xx, p = 0.xx, \eta_g^2 = 0.xx$

A one-way between-subjects ANOVA was conducted to compare the effect of joint action on feelings of trust across the no, helpful, and disrupted joint-action conditions. There was a significant effect of joint action on feelings of trust, $F(2, 12) = 8.75, p < 0.01, \eta_g^2 = 0.59$.

26.5.1 Plot ANOVA

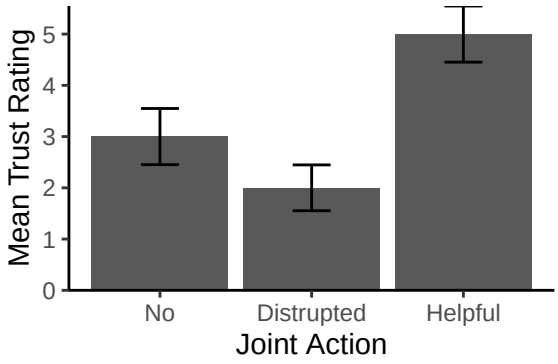
Normally, we plot bar graphs with one SEM unit. I use the `Means.Table` object, which contains the means. I also reset the order with the `factor` command. We will apply some fancy code to make our plot more APA!

```
library(ggplot2)

Means.Table$Condition<-factor(Means.Table$Condition,
                              levels = c("No","Distrupted","Helpful"),
                              labels = c("No","Distrupted","Helpful"))

Plot.1<-ggplot(Means.Table, aes(x = Condition, y = Means))+
  geom_col()+
  scale_y_continuous(expand = c(0, 0))+ # Forces plot to start at zero
  geom_errorbar(aes(ymax = Means + SEM, ymin= Means - SEM),
                position=position_dodge(width=0.9), width=0.25)+
  xlab('Joint Action')+
  ylab('Mean Trust Rating')+
  theme_bw()+
  theme(panel.grid.major=element_blank(),
```

```
        panel.grid.minor=element_blank(),
        panel.border=element_blank(),
        axis.line=element_line(),
        legend.title=element_blank())
Plot.1
```



26.6 References

Bakeman, R. (2005). Recommended effect size statistics for repeated measures designs. *Behavior Research Methods*, 37(3), 379-384.

27 Follow-Ups to One-Way ANOVAs

27.1 Experimentwise Error Rate

The experimentwise error rate is the likelihood that you will make at least one Type I error in your experiment as you run more and more t-tests. Each test has a *per-comparison* error rate (α_{pc}).

$$\alpha_{EW} = 1 - (1 - \alpha_{pc})^j$$

where j is the number of comparisons.

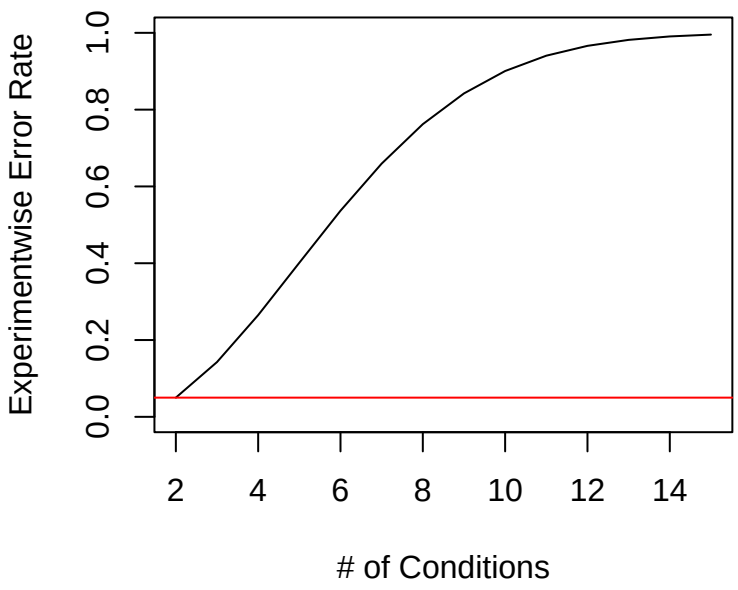
$$j = \frac{K(K-1)}{2}$$

[This calculation assumes that each comparison is independent and that sampling is conducted with replacement.]

We call this the experimentwise error rate (α_{EW}). If we set $\alpha_{pc} = .05$ and increase K from 2 to 15, we can see what happens as the number of conditions grows and we make all possible pairwise comparisons.

```
Aew <-function(alpha,K){1-(1-alpha)^(K*(K-1)/2)}
K=seq(2,15)
Aew.Report<-mapply(Aew, alpha=.05, K)

plot(K,Aew.Report, type='l',
     xlab = "# of Conditions", ylab="Experimentwise Error Rate", ylim=c(0,1))
abline(h=.05, col='red')
```



As we conduct more tests, we become increasingly likely to commit a Type I error.

27.1.1 Familywise Error

This is similar to experimentwise error, but it focuses on the risk of a Type I error within a "family" of tests. A family of tests will make more sense when we get to two-way ANOVA follow-ups. In the example at the end, I treat the planned contrasts as one family and the unplanned contrasts as another. The logic will become clearer later this semester.

[Note: Experimentwise error is also called familywise error.]

27.1.2 Design Example 1 for Last Class

We are interested in how much people trust a partner after engaging in different levels of joint action. Two people enter the lab, one of whom is a confederate. They are asked to either (a) move chairs independently, (b) move a couch together while the confederate helps, or (c) move a couch together while the confederate hinders. Moving a couch requires the two people to coordinate their actions. After the study ends, participants rate how trustworthy their partner seems from 1 to 7, with 7 indicating high trust.

No Joint Action	Helpful Joint Action	Disrupted Joint Action
2	5	2
3	4	3
5	7	3
3	4	1
2	5	1
M = 3	M = 5	M = 2

27.1.2.1 Run the ANOVA

```
library(afex) #

ANOVA.Table<-aov_car(DV~Condition + Error(SubNum),
                  data=JA.Study)
ANOVA.Table
```

```
Anova Table (Type 3 tests)

Response: DV
     Effect    df  MSE       F  ges p.value
1 Condition 2, 12 1.33 8.75 ** .593    .005
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1
```

So, we need to track down which groups are different.

27.1.3 Which Groups Are Different?

If we run three t-tests with $\alpha_{pc} = .05$, then with $j = 3$ our experimentwise error rate is $\alpha_{EW} = 0.143$.

27.1.4 Philosophy I (Familywise) Correction for Error Rate

This is the dominant philosophy in experimental psychology. Neuroscience is a different story, which we will cover next week.

Declared	Null is True (no difference)	Null is False (difference)	Total
Sig	Type 1 (V)	True Positives (S)	R
Non-Sig	True Negative (U)	Type II (T)	m-R
Total	Mo	M-Mo	

V / Mo = Probability of getting at least one wrong: we want to keep this at 5%

Different correction methods have been proposed over the years, ranging from the most anti-conservative to the most conservative. They come in and out of favor based on the fashion of the times within each subfield.

27.1.5 Approaches to Follow-Up Testing

27.1.5.1 Confirmatory: Hypothesis-Driven Follow-Up

Preferred approach: Examine only the conditions you predicted would differ. That means you must formulate your predictions carefully. This is hard and takes time to learn. It causes people the most frustration. It is required for preregistered reports and is increasingly required by reviewers. We are starting to leave the dark ages when reviewers asked for everything to be compared with everything because Type I error problems are better understood.

- Pros: You will conduct fewer tests and commit fewer Type I errors. Because you run fewer tests, your follow-ups can be less conservative, which also produces fewer Type II errors, as you will see below.
- Cons: You might miss something interesting. However, many people argue that chasing those interesting findings helped cause the replication crisis.

Summary: Low Type I and low Type II error rates, but a risk of missing something novel, assuming it is not a Type I error.

27.1.5.2 Exploratory: Test-Everything Approach

Out of favor: Test all comparisons and anything else you find interesting. This is the easiest approach to understand and was once common, but it is becoming difficult to publish. It requires very conservative follow-up methods because the risk of Type I error is high.

- Pros: You see everything
- Cons: If you do not do it correctly, much of what you find will be a Type I error. If you do it correctly, you will commit many Type II errors.

Summary: If you do it correctly, you have low Type I and very high Type II error rates. If you do it incorrectly, you have very high Type I and low Type II error rates.

27.1.5.3 Hybrid Approach

Happy medium? Use the confirmatory approach for planned comparisons. Test anything exploratory as conservatively as you can with unplanned comparisons. Carefully distinguish the two approaches for your reader. Otherwise, it may look as though you selected different corrections for no clear reason.

27.2 Contrasts

A contrast lets you customize what you want to compare. It is the best way to conduct confirmatory testing. You can use pairwise comparisons or more complex tests that merge and compare groups. This is a huge topic, so we will cover only the basic types today and return to advanced forms, such as orthogonal and polynomial contrasts, later in the semester.

27.2.1 Linear Contrast

This is basically an F-test in which you compare only the groups of interest. It also lets you merge groups for a comparison.

$$F_{contrast} = \frac{MS_{contrast}}{MS_W}$$

Note: your book says:

$$F_{contrast} = \frac{SS_{contrast}}{MS_W}$$

That is correct because $df_{contrast}$ always equals 1 when we compare two groups at once $(2-1=1)$. Thus, $MS_{contrast} = \frac{SS_{contrast}}{1}$. We can skip the $MS_{contrast}$ step because $MS_{contrast} = SS_{contrast}$.

27.2.2 Pairwise (linear) Contrast

27.2.2.1 Step 1

Decide how you want to weight the cells. We call each weight C, and **the weights must add to zero**. I want to compare No versus Helpful (H1), No versus Disrupted (H2), and Helpful versus Disrupted (H3). The specific numbers you choose are arbitrary.

If we conduct three hypothesis tests, they form a family. We can review the code above to see our error rate for three tests: $\alpha_{EW} = 0.143$.

Cell	No Joint Action	Helpful Joint Action	Disrupted Joint Action	Sum
M	3	5	2	
C_{H1}	-1	1	0	0
C_{H2}	-1	0	1	0
C_{H3}	0	-1	1	0

27.2.2.2 Step 2

Linear Sum (L_{H1})

$$L = \sum_{i=1}^{k} C_i M_i$$

$$L = -1 * 3 + 1 * 5 + 0 * 2 = 2$$

27.2.2.3 Step 3

Calculate the $MS/SS_{contrast}$

$$MS_{contrast} = \frac{nL^2}{\sum C^2}$$

$$MS_{contrast} = \frac{nL^2}{\sum C^2} = \frac{5 * 2^2}{(-1)^2 + (1)^2 + (0)^2} = \frac{20}{2} = 10$$

27.2.2.4 Step 4

Calculate $F_{contrast}$

$$F_{contrast} = \frac{MS_{contrast}}{MS_W} = \frac{10}{1.33} = 7.5$$

Note: You can convert it to t as follows:

$$\sqrt{F_{contrast}} = t_{contrast}$$

$$\sqrt{7.5} = 2.739$$

27.2.2.5 Step 5

Look up F_{crit} using $df = (1, df_w)$ or t_{crit} using $df = df_w$.

27.2.3 Doing Contrasts in R

The `emmeans` package brings the ANOVA error term into the contrast. `emmeans` stands for estimated marginal means. When we get to two-way ANOVAs, the meaning of marginal means will become clearer. First, we run the ANOVA and pass the resulting `afex` object to `emmeans`. Estimated marginal means are based on the model's fitted values rather than the raw data. ANOVA is a special case of regression, so this method also works for regression analysis.

```
library(emmeans)
Fitted.Model<-emmeans(ANOVA.Table, ~Condition)
Fitted.Model
```

```
 Condition  emmean    SE df lower.CL upper.CL
 Distrupted      2 0.516 12    0.875     3.13
 Helpful         5 0.516 12    3.875     6.13
 No              3 0.516 12    1.875     4.13

Confidence level used: 0.95
```

Notice that we created a new object containing the estimated marginal mean for each cell, the SE, the df_w term, and the confidence intervals. The SE is the same for all cells because it is created from MS_W when sample sizes are equal: $SE = \frac{MS_w}{\sqrt{n}}$.

Pay attention to the order of the object. It does not match our table, so be careful.

Set up the hypotheses (make sure the order matches your fitted emmeans object!)

```
set1 <- list(
  H1_No.VS.H = c(0,1,-1),
  H2_No.VS.D = c(1,0,-1),
  H3_D.VS.H = c(-1,1,0))
```

Check: Do they each sum to zero?

```
c(sum(set1$H1_No.VS.H),sum(set1$H2_No.VS.D),sum(set1$H3_D.VS.H))
```

```
[1] 0 0 0
```

Run the contrast. `adjust = "none"` means that the p-value is not corrected for multiple comparisons.

```
contrast(Fitted.Model, set1, adjust = "none") %>% summary(infer = TRUE)
```

```
 contrast   estimate   SE df lower.CL upper.CL t.ratio p.value
 H1_No.VS.H        2 0.73 12    0.409    3.591   2.739  0.0180
 H2_No.VS.D       -1 0.73 12   -2.591    0.591  -1.369  0.1960
 H3_D.VS.H         3 0.73 12    1.409    4.591   4.108  0.0015

Confidence level used: 0.95
```

It matches the by-hand calculation!

27.2.3.1 Pairwise Contrast = Fisher's Protected *t*-Test (aka LSD)

LSD stands for Least Significant Difference. It is the most anti-conservative method because it provides *almost* no protection against Type I error, although it is used as the base pairwise method for other approaches. It modifies the *t*-test formula directly:

$$t = \frac{M_1 - M_2}{\sqrt{\frac{S_p^2}{n_1} + \frac{S_p^2}{n_2}}}$$

$$LSD = \frac{M_1 - M_2}{\sqrt{\frac{MS_W}{n_1} + \frac{MS_W}{n_2}}}$$

This works because MS_W is basically S_p^2: MS_W is the pooled variance across all groups, while S_p^2 uses only the groups of interest. The error term therefore contains the pooled variance of all groups, making this a safer version of the *t*-test formula for our follow-up tests.

LSD is only appropriate for confirmatory testing with three or fewer comparisons.

$$LSD_{SE} = \sqrt{\frac{MS_W}{n_1} + \frac{MS_W}{n_2}}$$

$$LSD_{SE} = \sqrt{\frac{1.33}{5} + \frac{1.33}{5}}$$

$LSD_{SE} = .73$, which matches our contrast SE.

27.2.3.1.1 Doing LSD in R

LSD can be calculated automatically and is a little easier than writing the linear contrast.

```
Fitted.Model<-emmeans(ANOVA.Table, ~Condition)
summary(pairs(Fitted.Model, adjust='none'))
```

```
 contrast             estimate   SE df t.ratio p.value
 Distrupted - Helpful       -3 0.73 12  -4.108  0.0015
 Distrupted - No            -1 0.73 12  -1.369  0.1960
 Helpful - No                2 0.73 12   2.739  0.0180
```

The estimate is now the mean difference between groups. The SE comes from the LSD equation, and the test uses df_W to calculate p-values. This is identical to our linear contrast.

27.2.4 "Custom" Contrast

A custom contrast compares merged groups.

27.2.4.1 Step 1

Decide how you want to weight the cells. We call each weight C, and the weights must add to zero. I want to compare No + Disrupted versus Helpful.

Cell	No Joint Action	Helpful Joint Action	Disrupted Joint Action
M	3	5	2
C_{H4}	-.5	1	-.5

27.2.4.2 Step 2

Linear Sum (L)

$$L = \sum_{i=1}^{k} C_i M_i$$

$$L = -.5 * 3 + 1 * 5 + -.5 * 2 = 2.5$$

27.2.4.3 Step 3

Calculate the $MS/SS_{contrast}$

$$MS_{contrast} = \frac{nL^2}{\sum C^2}$$

$$MS_{contrast} = \frac{nL^2}{\sum C^2} = \frac{5 * 2.5^2}{(-.5)^2 + (1)^2 + (-.5)^2} = \frac{31.25}{1.5} = 20.833$$

27.2.4.4 Step 4

Calculate $F_{contrast}$

$$F_{contrast} = \frac{MS_{contrast}}{MS_W} = \frac{20.833}{1.33} = 15.625$$

Note: You can convert it to a t-test as follows:

$$\sqrt{F_{contrast}} = t_{contrast}$$

$$\sqrt{15.625} = 3.953$$

27.2.4.5 Step 5

Look up F_{crit} using $df = (1, df_w)$ or t_{crit} using $df = df_w$.

27.2.5 Doing Custom Contrasts in R

Check the order of my fitted values.

```
# Fitted.Model<-emmeans(ANOVA.Table, ~Condition) #we already created this
Fitted.Model
```

```
 Condition  emmean    SE df lower.CL upper.CL
 Distrupted      2 0.516 12    0.875     3.13
 Helpful         5 0.516 12    3.875     6.13
 No              3 0.516 12    1.875     4.13

Confidence level used: 0.95
```

- Set up the hypothesis (make sure the order matches your fitted emmeans object!)
- *Make sure that the numbers add to 1 within each group you merge.* This does not affect the contrast, but it matters for the estimate used in the effect-size calculation.

```
set2 <- list(
  H4 = c(-.5,1,-.5))
```

- Check: Do they each sum to zero?

```
sum(set2$H4)
```

```
[1] 0
```

- Run the contrast

```
contrast(Fitted.Model, set2, adjust = "none")   %>% summary(infer = TRUE)
```

```
 contrast estimate    SE df lower.CL upper.CL t.ratio p.value
 H4            2.5 0.632 12     1.12     3.88   3.953  0.0019

Confidence level used: 0.95
```

It matches the by-hand calculation!

27.3 Correcting p-Values for Multiple Comparisons

There are several ways to correct p-values for multiple comparisons, ranging from anti-conservative to ultra-conservative. By default, `emmeans` corrects within a set of comparisons. Our pairwise contrast has $j = 3$, while our custom contrast has $j = 1$, because we ran them as separate families of tests. `emmeans` therefore will not correct for four tests, nor will it correct the custom contrast, no matter what you tell it to do. *It will ignore you. Look for the note below the contrast to see whether a correction was applied.*

27.3.1 Tukey's HSD

The Honestly Significant Difference test, also called Tukey's test, protects against Type I error better than LSD. It is popular for *pairwise* comparisons because it is more conservative than LSD, but it does not destroy power and is easy to understand. I think people like it because they do not "lose" all their effects, although it is still not very conservative overall. The formulas are similar to LSD, but Tukey uses the studentized-range distribution and its q-table to produce more conservative critical values. You compare q_{exp} with q_{crit} as you would in a t-test.

$$q = \frac{M_1 - M_2}{\sqrt{\frac{MS_W}{n}}}$$

Here is the simple formula used to calculate HSD by hand. If a mean difference is greater than the HSD value, the difference is significant.

$$HSD = q_{crit}\sqrt{\frac{MS_W}{n}}$$

27.3.1.1 Doing HSD in R

The `emmeans` package provides a t-value, as with LSD, and corrects the p-values automatically. The computer can also correct for unequal sample sizes using the Tukey-Kramer method, while the formulas I gave you cannot.

Correcting a pairwise contrast to HSD (**note: You cannot technically apply HSD to a custom contrast, but you can use an MVT adjustment, which is very similar**).

Correcting LSD to HSD

```
# Fitted.Model<-emmeans(ANOVA.Table, ~Condition) #we already created this
pairs(Fitted.Model, adjust='tukey') %>% summary(infer = TRUE)
```

```
 contrast             estimate   SE df lower.CL upper.CL t.ratio p.value
 Distrupted - Helpful       -3 0.73 12  -4.9483   -1.052  -4.108  0.0038
 Distrupted - No            -1 0.73 12  -2.9483    0.948  -1.369  0.3866
 Helpful - No                2 0.73 12   0.0517    3.948   2.739  0.0441

Confidence level used: 0.95
Conf-level adjustment: tukey method for comparing a family of 3 estimates
P value adjustment: tukey method for comparing a family of 3 estimates
```

Note the message at the bottom: *P value adjustment: Tukey method for comparing a family of 3 estimates.* This corrects for three tests. We will need to be careful about this when we move to two-way ANOVAs. **Read the notes carefully because `emmeans` sometimes declines a request that does not make statistical sense.**

Correcting contrast to MVT

```
# Fitted.Model<-emmeans(ANOVA.Table, ~Condition) #we already created this
contrast(Fitted.Model, set1, adjust = "mvt")  %>% summary(infer = TRUE)
```

```
 contrast   estimate   SE df lower.CL upper.CL t.ratio p.value
 H1_No.VS.H        2 0.73 12   0.0528    3.947   2.739  0.0443
 H2_No.VS.D       -1 0.73 12  -2.9472    0.947  -1.369  0.3867
 H3_D.VS.H         3 0.73 12   1.0528    4.947   4.108  0.0036

Confidence level used: 0.95
Conf-level adjustment: mvt method for 3 estimates
P value adjustment: mvt method for 3 tests
```

27.3.2 Bonferroni Family

The logic of these tests is to lower α_{pc} based on the number of tests until $\alpha_{EW} = .05$, or another rate you set. These tests are extremely conservative and reduce power, although psychologists use them often. Methodologists have created many corrections that protect against Type I error without inflating Type II error as much in between-subjects designs.

27.3.2.1 Bonferroni Correction

This is the most conservative and least powerful approach.

$$\alpha_{pc} = \frac{\alpha_{EW}}{j}$$

For our study that means:

$$\alpha_{pc} = \frac{\alpha_{EW}}{j} = \frac{.05}{3} = .0167$$

Thus, we would run LSD tests with $\alpha_{pc} = .0167$ instead of .05.

27.3.2.1.1 Doing Bonferroni in R

`emmeans` calculates the contrast, or LSD, and corrects the p-values based on the number of tests in the family.

```
# Fitted.Model<-emmeans(ANOVA.Table, ~Condition) #we already created this
contrast(Fitted.Model, set1, adjust = "bon")%>% summary(infer = TRUE)
```

```
 contrast   estimate   SE df lower.CL upper.CL t.ratio p.value
 H1_No.VS.H        2 0.73 12  -0.0298     4.03   2.739  0.0539
 H2_No.VS.D       -1 0.73 12  -3.0298     1.03  -1.369  0.5880
 H3_D.VS.H         3 0.73 12   0.9702     5.03   4.108  0.0044

Confidence level used: 0.95
Conf-level adjustment: bonferroni method for 3 estimates
P value adjustment: bonferroni method for 3 tests
```

27.3.2.2 Sidak Correction

This approach is less conservative than Bonferroni, but still very conservative.

$$\alpha_{new} = 1 - (1 - \alpha_{pc})^{1/j}$$

```
contrast(Fitted.Model, set1, adjust = "sidak")  %>% summary(infer = TRUE)
```

```
 contrast   estimate   SE df lower.CL upper.CL t.ratio p.value
 H1_No.VS.H        2 0.73 12  -0.0231     4.02   2.739  0.0530
 H2_No.VS.D       -1 0.73 12  -3.0231     1.02  -1.369  0.4803
 H3_D.VS.H         3 0.73 12   0.9769     5.02   4.108  0.0043

Confidence level used: 0.95
Conf-level adjustment: sidak method for 3 estimates
P value adjustment: sidak method for 3 tests
```

27.3.2.3 Notes

There are many other types of corrections. They are most often applied to pairwise approaches using the familywise philosophy.

27.4 Effect Sizes for Contrasts

If you assume HOV, you can modify Cohen's d by using MS_W in these follow-up tests.

$$d = \frac{M_1 - M_2}{\sqrt{S_p^2}} = \frac{M_1 - M_2}{\sqrt{MS_w}}$$

You can extract MS_W from the ANOVA with this script. Make sure to change the object name to match your ANOVA.

```
SD.pooled = sqrt(ANOVA.Table$anova_table$MSE)
SD.pooled
```

```
[1] 1.154701
```

That value is your pooled SD, which you need for the next step.

Pass the estimated marginal means to the built-in `eff_size` function along with DF_W, the residual df from the model. You can also request a confidence interval around the effect size with `%>% summary()`.

27.4.0.1 Pairwise Effect Size

```
Con.with.d<-emmeans(Fitted.Model, ~ Condition,adjust='none')
eff_size(Con.with.d, sigma = SD.pooled, edf = 12, method='pairwise') %>% summary()
```

```
 contrast             effect.size    SE df lower.CL upper.CL
 Distrupted - Helpful      -2.598 0.825 12   -4.396   -0.800
 Distrupted - No           -0.866 0.657 12   -2.297    0.565
 Helpful - No               1.732 0.725 12    0.153    3.311

sigma used for effect sizes: 1.155
Confidence level used: 0.95
```

27.4.0.2 Custom Contrast Effect Size

Make sure to set `method = "identity"`.

```
Con.with.d.2<-contrast(Fitted.Model, set2, adjust = "none")
eff_size(Con.with.d.2, sigma = SD.pooled, edf = 12, method='identity') %>% summary()
```

```
 contrast effect.size    SE df lower.CL upper.CL
 H4              2.17 0.704 12    0.632      3.7

sigma used for effect sizes: 1.155
Confidence level used: 0.95
```

27.5 *A Priori* Power and Contrasts

Remember, *a priori* power is determined by α_{pc}, effect size, and sample size. Let's examine power for the ANOVA and the follow-up tests when we change alpha.

For power analysis with a one-way ANOVA, we need Cohen's f, which plays a role similar to d for t-tests.

$$f = \sqrt{\frac{K-1}{K}\frac{F}{n}}$$

ES	Value	Size
f	.1	Small effect
f	.25	Medium effect
f	.4	Large effect

$f_{observed} = 1.08$, a large effect for our study. Remember that these values may be inflated in small-N studies. For now, we will believe it and use it to calculate *post hoc* power, which does not predict our *a priori* power in the wild.

Note: You can approximate f from η^2 using $f = \sqrt{\frac{\eta^2}{1-\eta^2}}$, but it will not be bias-corrected like the formula above.

```r
library(pwr)
pwr.anova.test(k = 3, n = 5, f = 1.08, sig.level = 0.05, power = NULL)
```

```
     Balanced one-way analysis of variance power calculation

              k = 3
              n = 5
              f = 1.08
      sig.level = 0.05
          power = 0.9169511

NOTE: n is number in each group
```

We can convert f into d to estimate the overall effect size: $d = 2f$. **Note:** This is only a conceptual formula; Cohen provides different versions for different patterns of data.

$d = 2.16$

At $\alpha_{pc} = .05$:

```r
pwr.t.test(n = 5, d = 2.16, power = NULL, sig.level = .05,
               type = c("two.sample"), alternative = c("two.sided"))
```

```
     Two-sample t test power calculation

              n = 5
              d = 2.16
      sig.level = 0.05
          power = 0.8476039
    alternative = two.sided

NOTE: n is number in *each* group
```

We are above .80.

If we apply a Bonferroni correction, $\alpha_{pc} = .05/3 = .0167$:

```r
pwr.t.test(n = 5, d = 2.16, power = NULL, sig.level = .0167,
               type = c("two.sample"), alternative = c("two.sided"))
```

```
     Two-sample t test power calculation

              n = 5
              d = 2.16
      sig.level = 0.0167
          power = 0.6569859
```

```
    alternative = two.sided

NOTE: n is number in *each* group
```

The power dropped! Your ANOVA might find the effect while the Bonferroni-corrected tests miss it. This happens often. The next question people ask me is, which result is **real**, the ANOVA or the Bonferroni tests?

27.6 Suggestions for the Review Process

Follow-up testing is the hardest part of many analyses. There is often no single correct answer, and it is where reviewers will fight you the hardest.

> Reviewers often have their own perspectives on what they want to see and how they want to see it. Perhaps (a) you misunderstood your own hypothesis, (b) they misunderstood your hypothesis because you explained it badly or they simply did not read, or (c) one reviewer demands all comparisons while another demands that you remove them. Reviewer 1 versus Reviewer 3 on the same paper!
>
> They may be fine with what you tested but disagree with how you tested it. Perhaps (a) they want only a Bonferroni correction because they want to control Type I error, want to kill your effect cause they do not like it, cynical, I know, or were taught that LSD/HSD is the devil; (b) they want specialized contrasts when you used pairwise comparisons, or the reverse; or (c) they want you to use, or not use, the ANOVA error term in an LSD or contrast approach. If you violated assumptions, they may be right in specific cases, but they need to explain their rationale.

First, reviewers are not always right. However, assume that **YOU** did not clearly explain what you did. Rewrite in plain English what you tested and whether the tests were planned or unplanned. Use simple sentences. This is technical writing, not English literature. Make it clear **visually** and **verbally** what you did and why by guiding the reader with graphs or tables. Cite sources supporting your approach to follow-up testing. Do not assume that readers know what you did or why. In your response, explain how you changed the wording and thank the reviewer for helping you make it clearer.

Second, do not play games with readers. If $p = .07$ supports your hypothesis and you call it significant, but later call $p = .06$ nonsignificant because it contradicts your hypothesis, readers will notice. This is a huge red flag that you are biased at best or p-hacking at worst. Your paper might be rejected without an invitation to revise, and the editor and reviewers may remember you. Most psychology journals are not double-blind.

Third, do not p-hack. Do not hunt for a correction that makes your result significant after a more conservative test fails. If you get a reviewer who is paying attention, they will eat you alive! That lack of trust makes reviewers say to Bonferroni-correct everything. If you use a confirmatory approach, you can put the exploratory material in the appendix and apply Bonferroni there. Reviewers will trust you more when you show that you understand p-hacking and avoid drawing too many conclusions.

Finally, reviewers might be right. Thank the reviewer for the careful reading and correct your paper or withdraw it from the review process. Yes, it sucks, but that is why we have reviewers. If they do their job well, everyone benefits!

27.7 APA-Style Report

[Note: I cannot center everything in R, but you can render to Word and correct the formatting there. If you download the `.qmd` file, you will see my complete analysis.]

When people perform joint actions, their trust in one another increases (Launay, Dean, & Bailes, 2013). Feelings of affiliation also increase when joint action occurs synchronously (Hove & Risen, 2009), while poor synchronization decreases feelings of social affiliation (Demos et al., 2017). However, it remains unclear whether trust results from moving in time together (Hove & Risen, 2009) or simply sharing a task (Demos et al., 2012). To test this question, we designed a study with three conditions: helpful joint action, disrupted joint action, and no joint action. In the helpful condition, a confederate tried to remain synchronous with the participant. In the disrupted condition, the confederate performed the joint action asynchronously. In the control condition, the participant and confederate completed a task together but not jointly. We predicted that synchronous joint action would increase trust compared with no joint action or asynchronous movement.

Methods

Procedure

Two people entered the lab: a participant and a confederate whose status was unknown to the participant. They were asked to either (a) move chairs independently, (b) move a couch together while the confederate followed the participant's lead, or (c) move a couch together while the confederate hindered the participant. Moving a couch required the two people to coordinate their actions. After the study, participants rated how trustworthy their partner seemed from 1 to 7, with 7 indicating high trust.

Data Analysis

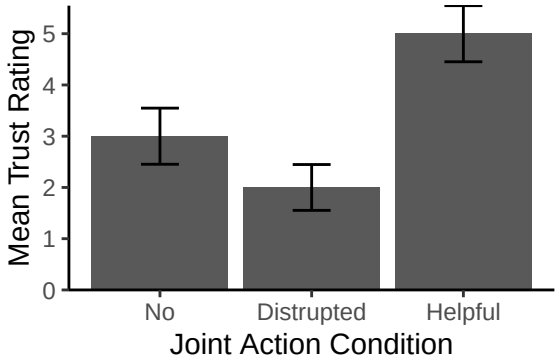
Analyses were conducted in R (4.2.1). The *afex* package (1.1-1; Singmann et al., 2018) was used to calculate the ANOVAs, and the *emmeans* package (1.8.0; Lenth, 2018) was used for follow-up testing. Data wrangling and plots were generated with the *tidyverse* package (1.3.2; Wickham et al., 2019). Planned comparisons were tested with linear contrasts. Unplanned contrasts were Sidak-corrected.

Results

A one-way between-subjects ANOVA was conducted to compare the effect of joint action on trust across the no, helpful, and disrupted joint-action conditions. There was a significant effect of joint action on trust, $F(2, 12) = 8.75, MS_W = 1.33, p < 0.01, \eta_g^2 = 0.59$. The planned contrast showed that trust was higher in the helpful joint-action condition (M = 5.00, SD = 1.22) than in the combined no and disrupted joint-action conditions (M = 2.50, SD = 1.18), $t(12)$ = 3.95, p = .0019, d = 4.33. As shown in Figure 1, the planned contrast between the disrupted and no joint-action conditions was not significant, $t(12)$ = 1.369, p = .20, d = -0.87. Sidak-corrected unplanned contrasts showed significantly higher trust in the helpful condition than in the disrupted condition, $t(12)$ = 4.11, p = .003, d = 2.6, and the no joint-action condition, $t(12)$ = 2.74, p = .04, d = 1.73. These results suggest that trust may result from synchronous action rather than joint action alone.

Figure 1

Mean trust ratings (1-7) across the three joint-action conditions. Error bars show +/- 1 SE.



27.8 References

Demos, A.P., Carter, D.J., Wanderley, M.M., & Palmer, C (2017). The unresponsive partner: Roles of social status, auditory feedback, and animacy in coordination of joint music performance. *Frontiers in Psychology*, 149(8).

Demos, A.P., Chaffin, R., Begosh, K. T., Daniels, J. R., & Marsh, K. L. (2012). Rocking to the beat: Effects of music and partner's movements on spontaneous interpersonal coordination. *Journal of Experimental Psychology: General*, 141(1), 49-53.

Hove, M. J., & Risen, J. L. (2009). It's all in the timing: Interpersonal synchrony increases affiliation. *Social Cognition*, 27(6), 949-960.

Launay, J., Dean, R. T., & Bailes, F. (2013). Synchronization can influence trust following virtual interaction. *Experimental Psychology*, 60(1), 53-63.

Lenth, R. (2018). emmeans: Estimated Marginal Means, aka Least-Squares Means. R Package Version 1.6.3. Available at: https://CRAN.R-project.org/package=emmeans.

Singmann, H., Bolker, B., Westfall, J., and Aust, F. (2021). afex: Analysis of Factorial Experiments. R Package Version 1.0-1. Available at: https://CRAN.R-project.org/package=afex.

28 Calculating the Two-Way Between-Subjects ANOVA

28.1 Design Example

Kardas and O'Brien (2018) reported that, with the proliferation of YouTube videos, people seem to think they can learn by *seeing* rather than *doing*. They "hypothesized that the more people merely watch others, the more they believe they can perform the skill themselves." They compared repeatedly watching the "tablecloth trick" with repeatedly reading or thinking about it. Participants assessed their own ability to perform the trick on a scale from 1 (I feel there is no chance at all I would succeed on this attempt) to 7 (I feel I would definitely succeed on this attempt). The study used a 3 (type of exposure: watch, read, think) x 2 (amount of exposure: low, high) between-subjects design. They collected N = 1,003 and found small effect sizes ($\eta^2 < .06$). To make this analysis manageable by hand, I will reduce the sample to five people per cell and inflate the effect size while preserving the pattern from Experiment 1.

Low Exposure Group	Watching	Reading	Thinking	**Row Means**
	1	3	3	
	3	2	3	
	3	3	2	
	4	1	3	
	2	4	1	
Cell Means	*2.6*	*2.6*	*2.4*	**2.53**
Cell SS	*5.2*	*5.2*	*3.2*	

High Exposure Group	Watching	Reading	Thinking	**Row Means**
	6	5	3	
	4	3	2	
	7	2	1	
	5	3	4	
	5	2	4	
Cell Means	*5.4*	*3.0*	*2.8*	**3.73**
Cell SS	*5.2*	*6.0*	*6.8*	
Column Means	**4.0**	**2.8**	**2.6**	G = 3.13

28.2 Logic of Two-Way ANOVA

We now have two factors, and we will test the main effects and interaction.

- Main effect (Factor A: Rows): The independent effect of Length of Exposure on Perceived Ability
- Main effect (Factor B: Columns): The independent effect of Type of Exposure on Perceived Ability
- Interaction: The joint effect of Length and Type of Exposure on Perceived Ability

28.2.1 Hypotheses

28.2.1.1 Main Effects

H_0: All groups have equal means.

H_1: At least one group differs from the others.

28.2.1.2 Interaction

H_0: There is no interaction between Factors A and B. All mean differences between treatment conditions are explained by the main effects.

H_1: There is an interaction between Factors A and B. The mean differences between treatment conditions are not what we would predict from the overall main effects of the two factors.

28.2.2 Logic of Variance Parsing

A one-way ANOVA has only one factor to test: $SS_T = SS_W + SS_B$

A two-way ANOVA has multiple effects to test: $SS_T = SS_W + (SS_{FactorA} + SS_{FactorB} + SS_{AxB})$

In a two-way ANOVA, $SS_B = SS_{FactorA} + SS_{FactorB} + SS_{AxB}$.

28.2.2.1 *F*-Ratio Logic

The omnibus F is the same as in a one-way ANOVA:

$$F = \frac{MS_B}{MS_W}$$

This still tells us whether at least one cell differs from another, but we must also calculate F-tests for Factors A and B and the A x B interaction.

$$F = \frac{MS_{FactorA}}{MS_W}$$

$$F = \frac{MS_{FactorB}}{MS_W}$$

$$F = \frac{MS_{AXB}}{MS_W}$$

28.3 Analysis Procedures

28.3.1 One-Way ANOVA Review (Simplified for Equal Sample Sizes)

With the formulas (formal equations)

Source	SS	DF	MS	F
Between	$n\sum_{i=1}^{k}(M_i - G)^2$	$K-1$	$\frac{SS_B}{df_B}$	$\frac{MS_B}{MS_W}$
Within	$\sum_{i=1}^{K}\sum_{j=1}^{n}(X_{i_j} - M_i)^2$	$N-K$	$\frac{SS_W}{df_W}$	
Total	$\sum_{j=1}^{N}(X_j - G)^2$	$N-1$		

With the formulas (conceptually simplified)

Source	SS	DF	MS	F
Between	$nSS_{treatment}$	$K-1$	$\frac{SS_B}{df_B}$	$\frac{MS_B}{MS_W}$
Within	$\sum SS_{within}$	$N-K$	$\frac{SS_W}{df_W}$	
Total	SS_{scores}	$N-1$		

Note: $SS_B + SS_W = SS_T$ & $df_B + df_W = df_T$

28.3.2 Two-Way ANOVA Review (Simplified for Equal Sample Sizes)

Source	SS	DF	MS	F
Between	SS_B	df_B	MS_B	F
$Factor\ A_{Row}$	SS_r	df_r	MS_r	F
$Factor\ B_{Col}$	SS_c	df_c	MS_c	F
$Factor\ AxB$	SS_{rxc}	df_{rxc}	MS_{rxc}	F
Within	SS_W	df_W	MS_W	
Total	SS_T	df_T		

With the formulas (formal equations)

Source	SS	DF	MS	F
Between	$n\left(\sum_{w=1}^{r}(M_r - G)^2 + \sum_{u=1}^{c}(M_c - G)^2\right)$	$rc-1$	$\frac{SS_B}{df_B}$	$\frac{MS_B}{MS_W}$
$A_{(Rows)}$	$n_r\sum_{w=1}^{r}(M_w - G)^2$	$r-1$	$\frac{SS_r}{df_r}$	$\frac{MS_r}{MS_W}$
$B_{(Col)}$	$n_c\sum_{u=1}^{c}(M_u - G)^2$	$c-1$	$\frac{SS_c}{df_c}$	$\frac{MS_c}{MS_W}$
AxB	$SS_B - SS_r - SS_c$	$(r-1)(c-1)$	$\frac{SS_{rxc}}{df_{rxc}}$	$\frac{MS_{rxc}}{MS_W}$
Within	$\sum_{w=1}^{r}\sum_{j=1}^{n}(X_{w_j} - M_w)^2 + \sum_{u=1}^{c}\sum_{j=1}^{n}(X_{u_j} - M_u)^2$	$nrc - rc$	$\frac{SS_w}{df_w}$	
Total	$\sum_{j=1}^{N}(X_j - G)^2$	$nrc-1$		

With the formulas (conceptually simplified)

Source	SS	DF	MS	F
Between	$nSS_{all\,cell\,means}$	$rc-1$	$\frac{SS_B}{df_B}$	$\frac{MS_B}{MS_W}$
$A_{(Rows)}$	$n_rSS_{row\,means}$	$r-1$	$\frac{SS_r}{df_r}$	$\frac{MS_r}{MS_W}$
$B_{(Cols)}$	$n_cSS_{col\,means}$	$c-1$	$\frac{SS_c}{df_c}$	$\frac{MS_c}{MS_W}$
AxB	$SS_B-SS_r-SS_c$	$(r-1)(c-1)$	$\frac{SS_{rxc}}{df_{rxc}}$	$\frac{MS_{rxc}}{MS_W}$
Within	$\sum SS_{within}$	$nrc-rc$	$\frac{SS_w}{df_w}$	
Total	SS_{scores}	$nrc-1$		

Note for SS: $SS_B+SS_W=SS_T$ and $SS_B=SS_r+SS_c+SS_{rxc}$.

Note for df: $df_B+df_W=df_T$ and $df_B=df_r+df_c+df_{rxc}$.

28.3.3 Effect Size

28.3.3.1 Eta-Squared

As with a one-way ANOVA, we will use eta-squared, the proportion of variance accounted for by the treatment. Remember that SS_B is the variation caused by the treatment, while SS_T is the variation due to treatment plus noise.

$$\eta_B^2=\frac{SS_B}{SS_B+SS_W}=\frac{SS_B}{SS_T}$$

$$\eta_r^2=\frac{SS_r}{SS_T}$$

$$\eta_c^2=\frac{SS_c}{SS_T}$$

$$\eta_{rxc}^2=\frac{SS_{rxc}}{SS_T}$$

28.3.3.2 Partial Eta-Squared

Eta-squared is the proportion of the total variance accounted for by an effect. The nice thing is that eta-squared values can add to 100%! However, each value depends on the number and magnitude of the other effects.

People often prefer to report partial eta-squared, η_p^2, which is the ratio of effect variance to effect plus error variance. This "controls" for the other effects and reports the effect relative to its noise rather than the total variance. Partial eta-squared is a ratio, while eta-squared is a proportion of the total. Thus, partial eta-squared values do not add to 100%.

$$\eta_{p_r}^2=\frac{SS_r}{SS_r+SS_w}$$

$$\eta_{p_c}^2=\frac{SS_c}{SS_c+SS_w}$$

$$\eta_{p_{rxc}}^2=\frac{SS_{rxc}}{SS_{rxc}+SS_w}$$

For a two-way ANOVA with manipulated between-subjects factors, this partial eta-squared formula equals the more modern generalized eta-squared, η_g^2, that R automatically generates. When a factor is measured rather than manipulated, such as gender, the formulas change slightly. We will discuss that next week along with ω_g^2 for bias correction.

28.4 Analysis in R

28.4.1 Build Our Data Frame (or Read a CSV)

I build the data frame by hand in long format.

```
n=5; r=2; c=3;
Example.Data<-data.frame(ID=seq(1:(n*r*c)),
          Level=c(rep("Low",(n*c)),rep("High",(n*c))),
          Type=rep(c(rep("Watching",n),rep("Reading",n),rep("Thinking",n)),2),
          Perceived.Ability = c(1,3,3,4,2,3,2,3,1,4,3,3,2,3,1,
                                6,4,7,5,5,5,3,2,3,2,3,2,1,4,4))

# Also, I want to re-set the order using the `factor` command and set the order I want.
Example.Data$Level<-factor(Example.Data$Level,
                           levels = c("Low","High"),
                           labels = c("Low","High"))

Example.Data$Type<-factor(Example.Data$Type,
                           levels = c("Watching","Reading","Thinking"),
                           labels = c("Watching","Reading","Thinking"))
```

28.4.2 Check the Means and SS Calculations with dplyr

```
library(dplyr)
Means.Table<-Example.Data %>%
  group_by(Level,Type) %>%
  summarise(N=n(),
            Means=mean(Perceived.Ability),
            SS=sum((Perceived.Ability-Means)^2),
            SD=sd(Perceived.Ability),
            SEM=SD/N^.5)
knitr::kable(Means.Table, digits = 2)
```

Level	Type	N	Means	SS	SD	SEM
Low	Watching	5	2.6	5.2	1.14	0.51
Low	Reading	5	2.6	5.2	1.14	0.51
Low	Thinking	5	2.4	3.2	0.89	0.40
High	Watching	5	5.4	5.2	1.14	0.51
High	Reading	5	3.0	6.0	1.22	0.55
High	Thinking	5	2.8	6.8	1.30	0.58

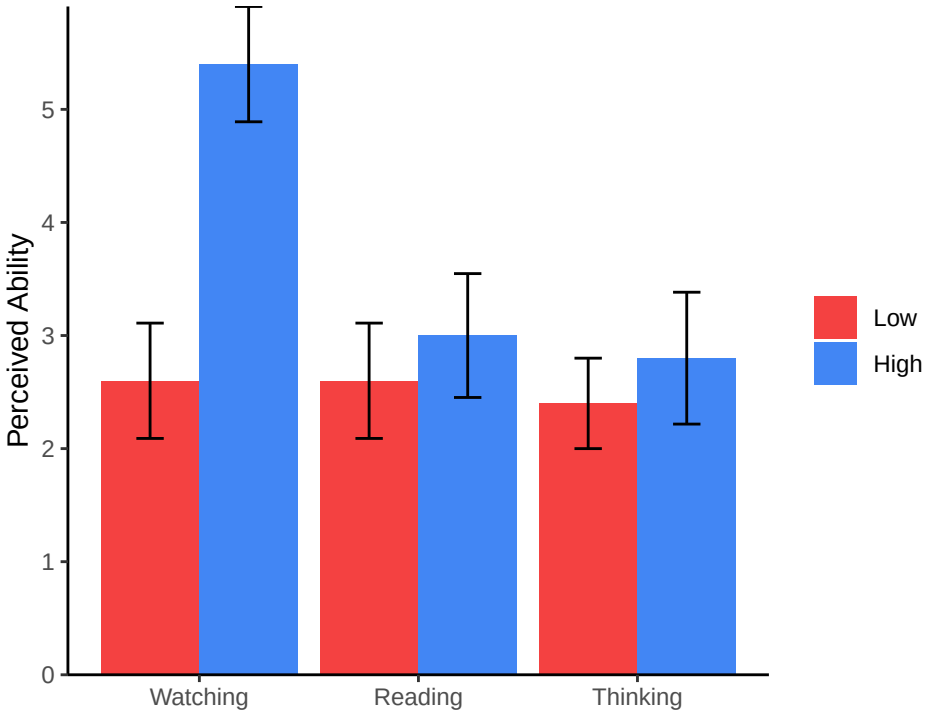
```
# Note remember knitr::kable makes it pretty, but you can just call `Means.Table`
```

28.4.3 Plot Study

Normally, we plot bar graphs with one SEM unit. I use the `Means.Table` object, which contains the means. We will apply some fancy code to make our plot more APA!

```
library(ggplot2)

Plot.1<-ggplot(Means.Table, aes(x = Type, y = Means, group=Level))+
  geom_col(aes(group=Level, fill=Level), position=position_dodge(width=0.9))+
  scale_y_continuous(expand = c(0, 0))+ # Forces plot to start at zero
  geom_errorbar(aes(ymax = Means + SEM, ymin= Means - SEM),
                position=position_dodge(width=0.9), width=0.25)+
  scale_fill_manual(values=c("#f44141","#4286f4"))+
  xlab('')+
  ylab('Perceived Ability')+
  theme_bw()+
  theme(panel.grid.major=element_blank(),
        panel.grid.minor=element_blank(),
        panel.border=element_blank(),
        axis.line=element_line(),
        legend.title=element_blank())
Plot.1
```



Which effects do you predict will be significant?

28.4.4 Run ANOVA

Again, we will use the `afex` package, which was developed to run SPSS-like Type III ANOVAs in R.

28.4.4.1 ANOVA-Style Output

- This style is close to our ANOVA source table, but it is formatted like a regression F-source table.
- You can ignore the intercept line (we will cover that next semester).
- Level = SS_r: Main effect of Level of Exposure (note R will give the order you called them. I put rows first in the formula below).
- Type = SS_c: Main effect of Type of Exposure
- Level:Type = SS_{rxc}: Interaction between Level and Type of Exposure
- Residual = SS_w: Residual in regression means leftover. This is the error term: chance.
- There is no total (you can do that with addition)

```
library(afex)

ANOVA.JA.Table<-aov_car(Perceived.Ability~Level*Type + Error(ID),
                  data=Example.Data, return='Anova')
ANOVA.JA.Table
```

```
Anova Table (Type III tests)

Response: dv
             Sum Sq Df  F value    Pr(>F)
(Intercept) 294.533  1 223.6962 1.157e-13 ***
Level        10.800  1   8.2025  0.008553 **
Type         11.467  2   4.3544  0.024353 *
Level:Type    9.600  2   3.6456  0.041446 *
Residuals    31.600 24
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

28.4.4.2 Nice-Style Output

- This style reports everything you need in APA format. Leave the return argument off or write `nice`.

```
library(afex)

Nice.JA.Table<-aov_car(Perceived.Ability~Level*Type + Error(ID),
                  data=Example.Data)
Nice.JA.Table
```

```
Anova Table (Type 3 tests)

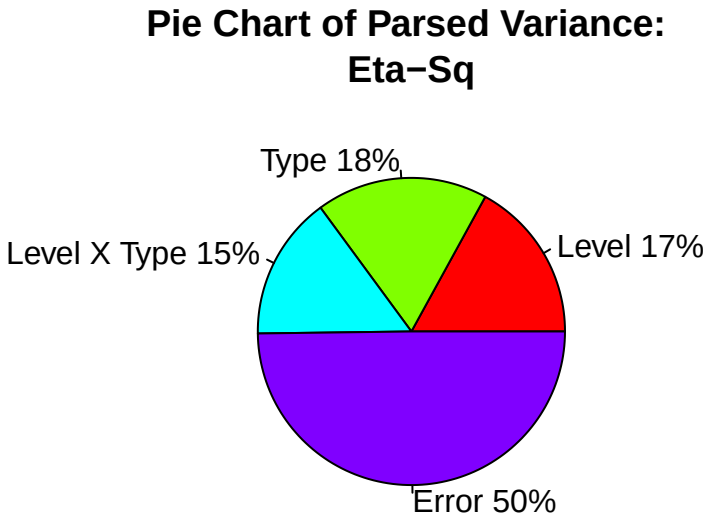
Response: Perceived.Ability
      Effect    df  MSE       F  ges p.value
1      Level 1, 24 1.32 8.20 ** .255    .009
2       Type 2, 24 1.32  4.35 * .266    .024
```

```
3 Level:Type 2, 24 1.32  3.65 * .233    .041
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1
```

28.4.5 Understanding η^2

η^2 is useful for understanding how your ANOVA "parsed" the variance, although it is not always the most useful effect size to report in papers.

Here we see that **Type**, **Level**, and **Type x Level** are "explained variance." Together, they explain about 50% of the variance. The error term is "unexplained variance," or the residual. Do not confuse η^2, shown in the chart below, with η_p^2.



28.5 Assumptions

We assume HOV and normality. We can test HOV using Levene's test, which is similar to the Brown-Forsythe test used for a one-way ANOVA but can be applied across multiple factors.

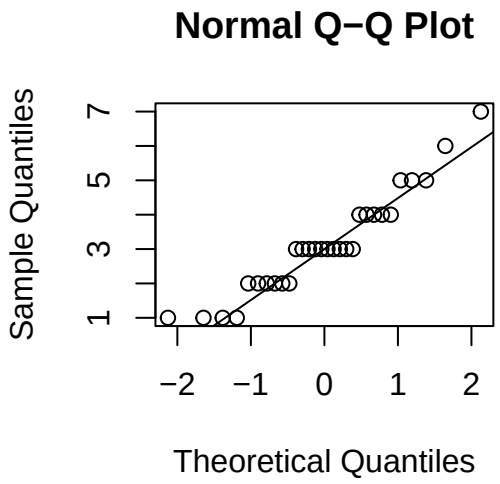
Using the `car` package, we can test for violations. As with Brown-Forsythe, you do NOT want this test to be significant.

```
library(car)
leveneTest(Perceived.Ability ~ Level*Type, data = Example.Data)
```

```
Levene's Test for Homogeneity of Variance (center = median)
      Df F value Pr(>F)
group  5  0.1171 0.9874
      24
```

We often examine Q-Q plots of the DV. If the data points deviate substantially from the line, that suggests a violation of normality.

```
qqnorm(Example.Data$Perceived.Ability)
qqline(Example.Data$Perceived.Ability)
```



The Shapiro-Wilk test evaluates normality statistically, but it can be very sensitive, especially in small samples. You do NOT want this test to be significant. In our case it is, but our sample is very small.

```
shapiro.test(Example.Data$Perceived.Ability)
```

```
    Shapiro-Wilk normality test

data:  Example.Data$Perceived.Ability
W = 0.92911, p-value = 0.04649
```

28.6 References

Kardas, M., & O'Brien, E. (2018). Easier seen than done: Merely watching others perform can foster an illusion of skill acquisition. *Psychological Science*, 29(4), 521-536.

29 Following Up the Two-Way Between-Subjects ANOVA

29.1 Design Example for Last Class

Kardas and O'Brien (2018) reported that, with the proliferation of YouTube videos, people seem to think they can learn by *seeing* rather than *doing*. They "hypothesized that the more people merely watch others, the more they believe they can perform the skill themselves." They compared repeatedly watching the "tablecloth trick" with repeatedly reading or thinking about it. Participants assessed their own ability to perform the trick on a scale from 1 (I feel there is no chance at all I would succeed on this attempt) to 7 (I feel I would definitely succeed on this attempt). The study used a 3 (type of exposure: watch, read, think) x 2 (amount of exposure: low, high) between-subjects design. They collected N = 1,003 and found small effect sizes ($\eta^2 < .06$). To make this analysis manageable by hand, I will reduce the sample to five people per cell and inflate the effect size while preserving the pattern from Experiment 1.

Low Exposure Group	Watching	Reading	Thinking	**Row Means**
	1	3	3	
	3	2	3	
	3	3	2	
	4	1	3	
	2	4	1	
Cell Means	*2.6*	*2.6*	*2.4*	**2.53**
Cell SS	*5.2*	*5.2*	*3.2*	

High Exposure Group	Watching	Reading	Thinking	**Row Means**
	6	5	3	
	4	3	2	
	7	2	1	
	5	3	4	
	5	2	4	
Cell Means	*5.4*	*3.0*	*2.8*	**3.73**
Cell SS	*5.2*	*6.0*	*6.8*	
Column Means	**4.0**	**2.8**	**2.6**	G = 3.13

29.1.1 ANOVA Result

```
library(afex)

Anova.Results<-aov_car(Perceived.Ability~Level*Type + Error(ID),
                  data=Example.Data)
Anova.Results
```

```
Anova Table (Type 3 tests)

Response: Perceived.Ability
      Effect    df  MSE       F  ges p.value
1      Level 1, 24 1.32 8.20 ** .255    .009
2       Type 2, 24 1.32  4.35 * .266    .024
3 Level:Type 2, 24 1.32  3.65 * .233    .041
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1
```

29.2 Follow-Up Logic

Like a one-way ANOVA, a two-way ANOVA tells us which factors have effects but not which levels differ. The best approach is the hybrid approach: use planned comparisons for confirmatory tests and test anything exploratory as conservatively as possible with unplanned comparisons. Clearly distinguish these approaches for your reader.

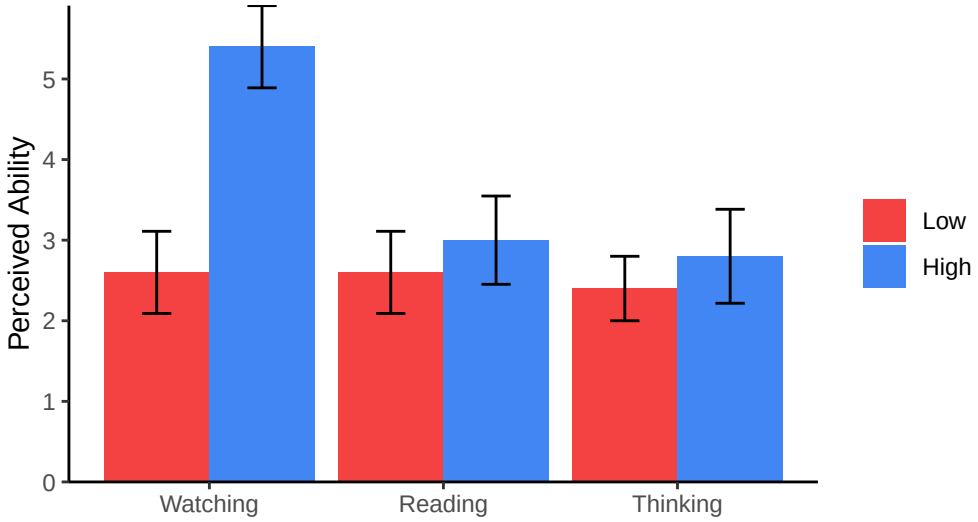
The challenge of a two-way ANOVA is **unpacking** a significant interaction. Every significant interaction must be explained by identifying which cells drove the effect. This is not always easy because the interaction may not match the predicted pattern.

29.2.1 Basic Rules

- **Do not follow up *F*-tests that were not significant.**
- Use the ANOVA error term for contrasts or protected *t*-tests.
- Planned: No multiple-comparison correction is necessary for fewer than three tests, but one can be applied if you want to be conservative or have many planned tests.
- Unplanned: Apply a multiple-comparison correction.
- Interactions: Conduct as FEW TESTS as possible to explain the interaction. Do not follow it up both ways.

29.2.2 Hypothesis from the Example Data

Kardas and O'Brien (2018) hypothesized that spending more time watching YouTube videos would make someone believe they could perform the tablecloth trick. We would therefore predict that the amount of exposure would matter only in the watching condition: the high-exposure group should have a higher mean than the low-exposure group for *watching*, with no low-versus-high difference for reading or thinking about the trick. In short, they predicted that only two cells would differ: Low versus High for Watching. *Note: A nonsignificant result means that we cannot reject the null. It does not mean that the groups are the same. Be careful when discussing nonsignificant results.*



29.2.2.1 Main Effects vs. the Interaction

If the interaction conflicts with the main effects, you may decide not to follow up the main effects or to explain them carefully. Kardas and O'Brien (2018) followed up only the interaction, not the main effects.

29.3 Planned Comparisons of the Interaction

29.3.1 Simple Effects with Two Levels

They want to test the **simple effects**, meaning the effect of one factor at each level of another factor, for Level of Exposure at each Type of Exposure. In other words: Low versus High for Watching, Low versus High for Reading, and Low versus High for Thinking.

As when we followed up the one-way ANOVA, we will use the df and error term from the omnibus ANOVA (df_W and MS_W). We will do this with the `emmeans` package. These tests can be reported as F-tests because we are essentially running one-way ANOVAs, or as t-values. Do not get confused: this is the same calculation used to follow up a one-way ANOVA, but now we are following up an interaction.

29.3.1.1 As *F*-Tests (One-Way ANOVAs)

Here is the old-fashioned, by-hand method. Run a one-way ANOVA at each level of one variable, such as *Type*, then recalculate the F- and p-values using the error term from the omnibus model. We also need to recalculate the effect size.

$$F = \frac{MS_{Level_{oneway}}}{MS_{W_{omnibus}}}$$

We need to subset the data to include only `Type == "Watching"`.

```
Example.Data.WatchingOnly<-subset(Example.Data, Type=="Watching")

One_way.Watching<-aov_car(Perceived.Ability~Level + Error(ID),
                  data=Example.Data.WatchingOnly)
One_way.Watching
```

```
Anova Table (Type 3 tests)

Response: Perceived.Ability
  Effect   df  MSE        F  ges p.value
1  Level 1, 8 1.30 15.08 ** .653    .005
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1
```

The problem is that the error term is incorrect. We need to extract SS_{levels}, convert it to $MS_{levels} = \frac{SS_L}{df_L}$, and divide again by the omnibus ANOVA error term (MS_W).

```
SS_L.Watch<-One_way.Watching$Anova$`Sum Sq`[2]
DF_L.Watch<-One_way.Watching$Anova$Df[2]
MS_L.Watch<- SS_L.Watch/DF_L.Watch

MS_W<-Anova.Results$anova_table$MSE[1]

F_Watch <- MS_L.Watch/ MS_W
P_Watch <- pf(F_Watch, 1, 24, lower.tail = FALSE)
```

Next, we recalculate the effect size by hand.

$$\eta^2_{p_{Levels}} = \frac{SS_L}{SS_L + SS_W}$$

```
SSw.watch<-Anova.Results$Anova$`Sum Sq`[5]

Eta_p_Watch <- SS_L.Watch/(SS_L.Watch+SSw.watch)
```

Simple effect for Watching, $F(1,24) = 14.886$, $p < 8 \times 10^{-4}$, $\eta_p^2 = 0.38$.

You would repeat this process two more times, once for each remaining level of TYPE. The fast way is through `emmeans`!

29.3.1.2 Fit the Model with `emmeans`

First, we must **cut** the ANOVA according to how we want to test the results:

```
library(emmeans)
library(dplyr)
Simple.Effects.By.Type<-emmeans(Anova.Results, ~Level|Type)
Simple.Effects.By.Type
```

```
Type = Watching:
 Level emmean    SE df lower.CL upper.CL
 Low      2.6 0.513 24     1.54     3.66
 High     5.4 0.513 24     4.34     6.46

Type = Reading:
 Level emmean    SE df lower.CL upper.CL
 Low      2.6 0.513 24     1.54     3.66
 High     3.0 0.513 24     1.94     4.06

Type = Thinking:
 Level emmean    SE df lower.CL upper.CL
 Low      2.4 0.513 24     1.34     3.46
 High     2.8 0.513 24     1.74     3.86

Confidence level used: 0.95
```

In this case, the F-tests equal the squared t-values. All we have to do is call `joint_tests()`. This automatically runs three one-way ANOVAs using the df and error term from the two-way ANOVA.

29.3.1.2.1 As *F*

```
One_way.By.Type<-joint_tests(Anova.Results, by = "Type")
One_way.By.Type
```

```
Type = Watching:
 model term df1 df2 F.ratio p.value
 Level        1  24  14.886  0.0008

Type = Reading:
 model term df1 df2 F.ratio p.value
 Level        1  24   0.304  0.5866

Type = Thinking:
 model term df1 df2 F.ratio p.value
 Level        1  24   0.304  0.5866
```

Notice that our by-hand values match.

29.3.1.2.2 As *t*-Values

In this case, we have two levels within each type of exposure, so we can report the tests as t-tests. The answers will be identical. Above, we have three subsets of the results. We can use `pairs()` to run three LSD tests, or protected t-tests, automatically using the df and error term from the two-way ANOVA.

```
pairs(Simple.Effects.By.Type,adjust='none') %>% summary(infer = TRUE)
```

```
Type = Watching:
 contrast   estimate    SE df lower.CL upper.CL t.ratio p.value
 Low - High     -2.8 0.726 24     -4.3     -1.3  -3.858  0.0008

Type = Reading:
 contrast   estimate    SE df lower.CL upper.CL t.ratio p.value
 Low - High     -0.4 0.726 24     -1.9      1.1  -0.551  0.5866

Type = Thinking:
 contrast   estimate    SE df lower.CL upper.CL t.ratio p.value
 Low - High     -0.4 0.726 24     -1.9      1.1  -0.551  0.5866

Confidence level used: 0.95
```

We can use the same code as before to calculate Cohen's d for the t-value. If we want η_p^2, we need to calculate it by hand. I cannot find a way to obtain it through the package yet.

Effect size:

```
SD.pooled = sqrt(Anova.Results$anova_table$MSE)[1]
eff_size(Simple.Effects.By.Type, sigma = SD.pooled, edf = 24, method='pairwise')
```

```
Type = Watching:
 contrast   effect.size    SE df lower.CL upper.CL
 Low - High      -2.440 0.724 24    -3.93   -0.946

Type = Reading:
 contrast   effect.size    SE df lower.CL upper.CL
 Low - High      -0.349 0.634 24    -1.66    0.961

Type = Thinking:
 contrast   effect.size    SE df lower.CL upper.CL
 Low - High      -0.349 0.634 24    -1.66    0.961

sigma used for effect sizes: 1.147
Confidence level used: 0.95
```

Note: *These are treated as three families of one test. Therefore, you cannot apply an FWER correction across them directly.*

29.3.1.2.3 As *t*-Values (Contrast Approach)

You can also call `contrast()` and obtain the same results. **Note**: *This will be useful as we add more levels.*

```
Set1 <- list(
  H1 = c(-1,1))

contrast(Simple.Effects.By.Type,Set1,adjust='none')
```

```
Type = Watching:
 contrast estimate    SE df t.ratio p.value
 H1            2.8 0.726 24   3.858  0.0008

Type = Reading:
 contrast estimate    SE df t.ratio p.value
 H1            0.4 0.726 24   0.551  0.5866

Type = Thinking:
 contrast estimate    SE df t.ratio p.value
 H1            0.4 0.726 24   0.551  0.5866
```

29.3.1.3 Which Should You Report?

F- or t-values? When you use the terms "simple effects" or "contrasts," people are primed to see F-values and generally do not ask for FWER corrections. When you use a pairwise approach, they may think you are conducting **unprotected** t-tests and ask to see an FWER correction.

29.3.2 Following Up the Interaction the Other Way

First and foremost, *ONLY UNPACK THE INTERACTION WITH ONE FAMILY OF SIMPLE EFFECTS*. Do not assume that you must also test it the other way: Watching versus Reading versus Thinking at Low Exposure, and Watching versus Reading versus Thinking at High Exposure. The authors did not conduct these follow-ups because they did not hypothesize them. They had already unpacked the ANOVA and identified the source of the interaction. Additional tests would inflate the FWER. A complex hypothesis might require follow-ups in both directions, but you should limit the simple effects to those needed to understand the interaction.

29.3.3 Simple Effects with Three Levels

They did not hypothesize follow-ups in the other direction, but we will conduct them as an example. We have three levels: Watching versus Reading versus Thinking at Low Exposure, and Watching versus Reading versus Thinking at High Exposure. In the old-school approach, we first run an F-test, or one-way ANOVA, at each level of Exposure. If the one-way ANOVA is significant, we follow it with pairwise comparisons or contrasts.

29.3.3.1 Fit the Model with `emmeans`

First, we must **cut** the ANOVA according to how we want to test the results:

```
One_way.Level<-joint_tests(Anova.Results, by = "Level")
One_way.Level
```

```
Level = Low:
 model term df1 df2 F.ratio p.value
 Type         2  24   0.051  0.9507

Level = High:
 model term df1 df2 F.ratio p.value
 Type         2  24   7.949  0.0022
```

Because there are three levels, you cannot test the entire simple effect with a single t-test. The package also does not easily provide effect sizes. Luckily, with between-subjects ANOVAs, it is easy to convert F-values to effect sizes. This conversion does NOT work the same way for more complex designs, such as mixed or repeated-measures ANOVAs.

You can convert the F-values into η_p^2 using the formula from your textbook.

$$\eta_p^2 = \frac{df_b F}{df_b F + df_w}$$

Because we have two F-values, R can give us two effect sizes!

```
Fs = One_way.Level$F.ratio
df_b = One_way.Level$df1
df_w =One_way.Level$df2

eta_p_exposure<- (Fs * df_b)/(Fs * df_b + df_w)
eta_p_exposure
```

```
[1] 0.004232014 0.398466089
```

Simple effect for Low Exposure, $F(2,24) = 0.051$, $p < 0.9507$, $\eta_p^2 = 0.0042$.

Simple effect for High Exposure, $F(2,24) = 7.949$, $p < 0.0022$, $\eta_p^2 = 0.3985$.

We have a simple effect at High Exposure. We need to follow it up to determine which levels differ. The correction depends on whether the comparison was planned or unplanned.

29.3.3.2 Follow-Up of the Significant Simple Effect

The code runs two families of three tests. You need to ignore the results for the Low Exposure group.

29.3.3.2.1 Specific Contrasts

Use the pairwise approach only for the significant simple effect you want to follow up, and apply an appropriate correction.

```
Simple.Effects.at.Level<-emmeans(Anova.Results, ~Type+Level)
Simple.Effects.at.Level
```

```
 Type     Level emmean    SE df lower.CL upper.CL
 Watching Low      2.6 0.513 24     1.54     3.66
 Reading  Low      2.6 0.513 24     1.54     3.66
 Thinking Low      2.4 0.513 24     1.34     3.46
 Watching High     5.4 0.513 24     4.34     6.46
 Reading  High     3.0 0.513 24     1.94     4.06
 Thinking High     2.8 0.513 24     1.74     3.86

Confidence level used: 0.95
```

```
Set2 <- list(
  High.EvsR = c(0,0,0,-1,1,0),
  High.WvsT = c(0,0,0,-1,0,1),
  High.RvsT = c(0,0,0,0,-1,1))

contrast(Simple.Effects.at.Level,Set2,adjust='MVT')
```

```
 contrast  estimate    SE df t.ratio p.value
 High.EvsR     -2.4 0.726 24  -3.307  0.0081
 High.WvsT     -2.6 0.726 24  -3.583  0.0041
 High.RvsT     -0.2 0.726 24  -0.276  0.9591

P value adjustment: mvt method for 3 tests
```

29.3.3.2.2 Simpler, but with Less Control

This corrects for three tests per family and is almost the same as MVT. The correction is applied separately within each family, so you need to ignore the Low Exposure group.

```
Simple.Effects.by.Level<-emmeans(Anova.Results, ~Type|Level)
pairs(Simple.Effects.by.Level,adjust='tukey')
```

```
Level = Low:
 contrast            estimate    SE df t.ratio p.value
 Watching - Reading       0.0 0.726 24   0.000  1.0000
 Watching - Thinking      0.2 0.726 24   0.276  0.9591
 Reading - Thinking       0.2 0.726 24   0.276  0.9591

Level = High:
 contrast            estimate    SE df t.ratio p.value
 Watching - Reading       2.4 0.726 24   3.307  0.0080
 Watching - Thinking      2.6 0.726 24   3.583  0.0041
 Reading - Thinking       0.2 0.726 24   0.276  0.9591

P value adjustment: tukey method for comparing a family of 3 estimates
```

29.3.3.2.3 "Complex" Linear Contrast

Here, you can merge and compare groups as before. If you have many tests, you can apply Bonferroni, Sidak, MVT, or FDR corrections within the family.

```
Simple.Effects.at.Level<-emmeans(Anova.Results, ~Type+Level)

Set3 <- list(
  WvsRT = c(0,0,0,1,-.5,-.5))

contrast(Simple.Effects.at.Level,Set3,adjust='none')
```

```
 contrast estimate    SE df t.ratio p.value
 WvsRT         2.5 0.628 24   3.978  0.0006
```

29.3.3.2.4 "Consecutive" Contrast

If the data were ordinal, you could test them in a logical order. If you have many tests, you can apply Bonferroni, Sidak, MVT, or FDR corrections within the family.

```
Consec.Set <- list(
  WvsR = c(-1,1,0),
  RvsT = c(0,-1,1))

contrast(Simple.Effects.by.Level,Consec.Set,adjust='none')
```

```
Level = Low:
 contrast estimate    SE df t.ratio p.value
 WvsR          0.0 0.726 24   0.000  1.0000
 RvsT         -0.2 0.726 24  -0.276  0.7852

Level = High:
 contrast estimate    SE df t.ratio p.value
 WvsR         -2.4 0.726 24  -3.307  0.0030
 RvsT         -0.2 0.726 24  -0.276  0.7852
```

```
# Also
# contrast(Simple.Effects.By.Level,'consec',adjust='none')
```

29.3.3.2.5 "Polynomial" Contrast

If the data were ordinal, you could test for a linear or curvilinear trend. If you have many tests, you can apply Bonferroni, Sidak, MVT, or FDR corrections within the family.

```
Poly.Set <- list(
  Linear = c(-1,0,1),
  Quad = c(-.5,1,-.5))

contrast(Simple.Effects.by.Level,Poly.Set,adjust='none')
```

```
Level = Low:
 contrast estimate    SE df t.ratio p.value
 Linear       -0.2 0.726 24  -0.276  0.7852
 Quad          0.1 0.628 24   0.159  0.8749

Level = High:
 contrast estimate    SE df t.ratio p.value
 Linear       -2.6 0.726 24  -3.583  0.0015
 Quad         -1.1 0.628 24  -1.750  0.0929
```

```
# Also
# contrast(Simple.Effects.By.Level,'poly',adjust='none')
```

29.4 Main-Effect Follow-Up

Remember, we would not follow this main effect because of the hypothesis we are testing, but you may follow up main effects in other cases. If the test is planned, use a less stringent correction. If it is unplanned, use a more conservative correction.

The code is the same as for a one-way ANOVA. Here, we can only follow up Type of Exposure. Again, we will use the df and error term from the two-way ANOVA.

```
Main.Effects.Type<-emmeans(Anova.Results, ~Type)
Main.Effects.Type
```

```
 Type     emmean    SE df lower.CL upper.CL
 Watching    4.0 0.363 24     3.25     4.75
 Reading     2.8 0.363 24     2.05     3.55
 Thinking    2.6 0.363 24     1.85     3.35

Results are averaged over the levels of: Level
Confidence level used: 0.95
```

You can run pairwise comparisons or any of the contrast approaches shown for interactions.

```
pairs(Main.Effects.Type,adjust='tukey')
```

```
 contrast            estimate    SE df t.ratio p.value
 Watching - Reading       1.2 0.513 24   2.338  0.0695
 Watching - Thinking      1.4 0.513 24   2.728  0.0304
 Reading - Thinking       0.2 0.513 24   0.390  0.9200

Results are averaged over the levels of: Level
P value adjustment: tukey method for comparing a family of 3 estimates
```

29.5 Unplanned Comparisons

This uses the same code as above, but you should apply conservative corrections. You can also force R to recognize additional tests within the family, but we will not cover that today.

29.6 References

Kardas, M., & O'Brien, E. (2018). Easier seen than done: Merely watching others perform can foster an illusion of skill acquisition. *Psychological Science*, 29(4), 521-536.

30 From the Paired *t*-Test to the One-Way Within-Subjects ANOVA

30.1 Types of Design

- Repeated: The same person is measured twice.
- Matched: "Doppelgangers we pretend are clones."
  - A. Natural matches, such as twins or siblings
  - B. Participants matched on other characteristics, such as IQ, age, gender, or other controls

30.1.1 Correlated Measures

- When participants repeat trials or complete multiple conditions, their scores tend to be **correlated**.
  - We want to control for individual variability.
  - In the formula below, we remove shared variance from the standard error using Pearson's r between conditions.

$$t = \frac{M_1 - M_2}{\sqrt{\frac{S_1^2+S_2^2}{n} - \frac{2rS_1S_2}{n}}}$$

30.1.1.1 Between-Subjects Assumption

Below is a covariance matrix for a between-subjects design with two levels, as in an independent-samples *t*-test. We assume HOV, meaning that the population variances are equal: $\sigma_1^2 = \sigma_2^2 = \sigma^2$. The cells show the correlations between groups. Note that ρ is the population correlation.

$$\mathbf{Cov} = \sigma^2 \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- Cell 1 = Group 1 with Group 1 ($\rho = 1$): The cell data are correlated with themselves.
- Cell 2 = Group 1 with Group 2 ($\rho = 0$): The groups are unrelated, so the correlation should be around zero.
- Cell 3 = Group 2 with Group 1 ($\rho = 0$): Same as Cell 2 because the matrix is symmetric.
- Cell 4 = Group 2 with Group 2 ($\rho = 1$): The cell data are correlated with themselves.

30.1.1.2 Within-Subjects Assumption

Below is a covariance matrix for a within-subjects design with two levels, as in a paired-samples t-test. We still assume HOV, meaning that the population variances are equal: $\sigma_1^2 = \sigma_2^2 = \sigma^2$. The cells show the correlations between conditions.

$$\mathbf{Cov} = \sigma^2 \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}$$

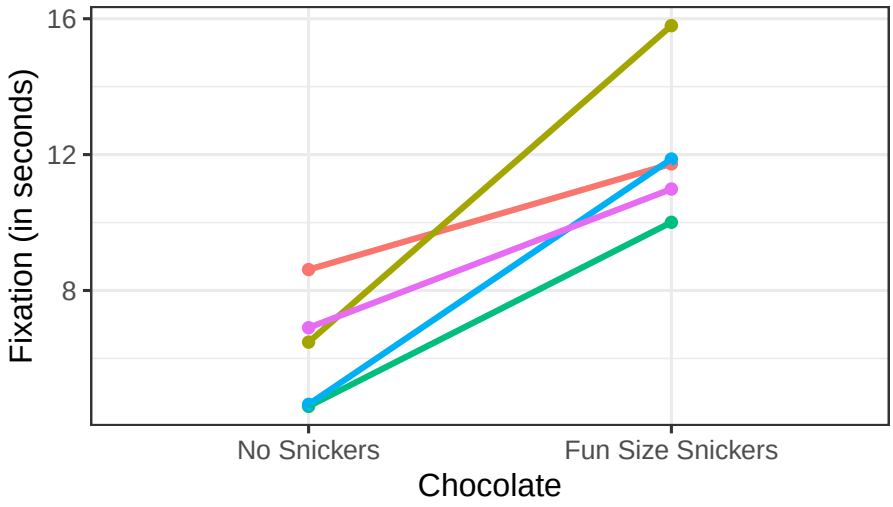
- Cell 1 = Group 1 with Group 1 ($\rho = 1$): The cell data are correlated with themselves.
- Cell 2 = Group 1 with Group 2 ($\rho > |0|$): Each person's scores are correlated across conditions and should NOT be zero.
- Cell 3 = Group 2 with Group 1 ($\rho > |0|$): Same as Cell 2 because the matrix is symmetric.
- Cell 4 = Group 2 with Group 2 ($\rho = 1$): The cell data are correlated with themselves.

30.2 One Repeated Factor: Two-Level Designs

Does eating chocolate during a lecture increase fixation on the lecture? We measure fixation as the mean time a student's eyes remain on the screen, in seconds per glance. We collect multiple trials per condition for each person and compare the means with a paired-samples t-test. The DV is mean fixation time. The IV is a two-level repeated factor: no chocolate in Lecture 1 versus one fun-size Snickers in Lecture 2. We simulate $n = 5$ students measured during both lectures, with means of $M_1 = 5$ and $M_2 = 10$ and $\sigma^2 = 4$. The responses across conditions are correlated at $\rho = .4$.

30.2.1 Plot of Simulation

- Spaghetti plot per person
- Each line represents a person's score from Lecture 1 (No Chocolate) to Lecture 2 (Chocolate).
- As you can see, most do better, while some do worse.



The correlation matrix:

Corr Matrix	None	Snickers
None	1	0.21
Snickers	0.21	1

The value of r is reasonably close to the $\rho = .4$ that we simulated.

Variance per cell:

Chocolate	Varience
No Snickers	2.849629
Fun Size Snickers	4.867386

The variances are not very homogeneous, but we have an N of only 5.

30.2.2 Conduct a Paired *t*-Test in R

We use the `t.test()` function with the argument `paired = TRUE`. First, we reshape the data so that each row contains one participant's scores in both conditions. This makes the pairing explicit. The df equals the number of participants minus 1.

```
DataSim1_Wide <- DataSim1 %>%
  dplyr::select(SS, Chocolate, Fixation) %>%
  pivot_wider(names_from = Chocolate, values_from = Fixation)

T1 <- with(
  DataSim1_Wide,
  t.test(`Fun Size Snickers`, `No Snickers`, paired = TRUE)
)
T1
```

```
    Paired t-test

data:  Fun Size Snickers and No Snickers
t = 5.2452, df = 4, p-value = 0.006318
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 2.743944 8.915821
sample estimates:
mean difference
       5.829882
```

30.2.2.1 APA-Style Report

The `apa` package can automatically convert our *t*-test result into APA format.

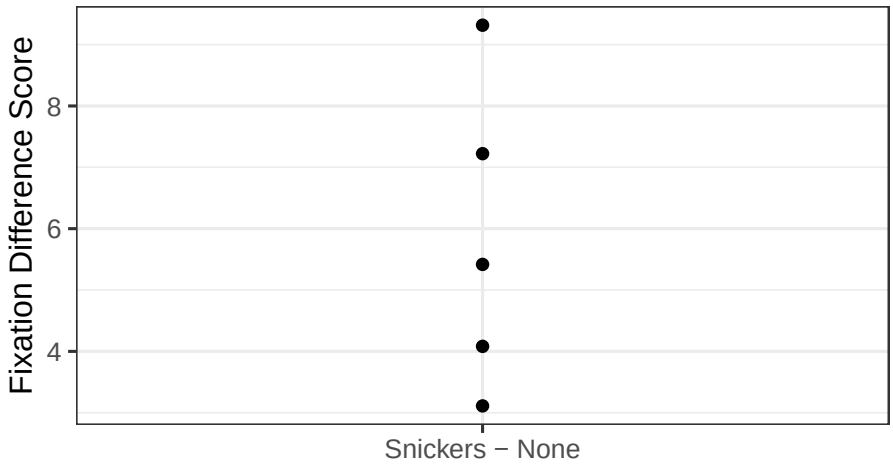
```
library(apa)
cat(apa(T1, format = "rmarkdown", print = FALSE))
```

$t(4) = 5.25$, $p = .006$, $d = 2.35$

30.2.3 Logic of the Paired *t*-Test

We remove **individual differences** because we care about how people **change** from one condition to another. Although the raw data look like the spaghetti plot above, the analysis removes these individual differences by calculating a difference score: *Fun-Size Snickers Condition - No Snickers Condition.*

Now that we have difference scores, we will plot the dots:



30.2.3.1 One-Sample *t*-Test on Difference Scores

Because we have one score per person, the difference between conditions, we can run a one-sample *t*-test to determine whether the difference scores differ from zero.

$$t = \frac{M - \mu}{\sqrt{S^2/n}}$$

```
T2<-t.test(x=DataSim2$diff, y = NULL, mu=0)
T2
```

```
    One Sample t-test

data:  DataSim2$diff
t = 5.2452, df = 4, p-value = 0.006318
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 2.743944 8.915821
sample estimates:
```

```
mean of x
 5.829882
```

Notice that the results are identical to those from the paired-samples t-test.

30.2.3.2 Simplified Paired *t*-Test Formula

To calculate this paired t-test by hand, calculate the difference scores (D), then find their mean and variance, or S. The result is identical to that from the more complex formula above, which removes individual differences using the Pearson correlation.

$$t = \frac{M_D - \mu_D}{\sqrt{S_D^2/n}}$$

30.2.4 Effect Size

There is considerable debate about calculating Cohen's d for paired designs. The simple formula looks like Cohen's d for a one-sample t-test, except that it uses difference scores:

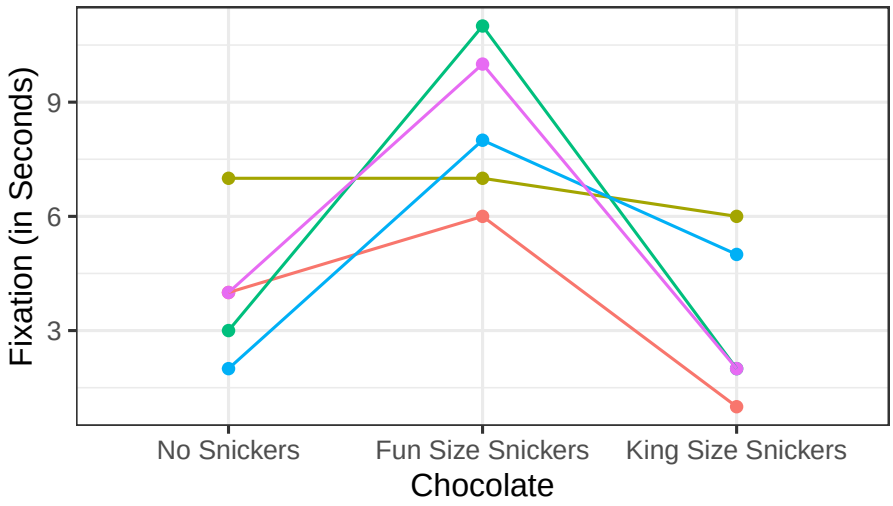
$$d = \frac{M_D}{S_D}$$

30.3 One Repeated Factor: Designs with Three or More Levels

> Does eating chocolate during a lecture increase fixation on the lecture? We measure fixation as the mean time a student's eyes remain on the screen, in seconds per glance. We collect multiple trials per condition for each person and compare the means with a repeated-measures ANOVA. The DV is mean fixation time. The IV is a three-level repeated factor: no chocolate in Lecture 1, one fun-size Snickers in Lecture 2, and one king-size Snickers in Lecture 3. We simulate $n = 5$ students measured during all three lectures, with means of $M_1 = 5$, $M_2 = 10$, and $M_3 = 3$, assuming HOV with $\sigma^2 = 4$. Responses across conditions are correlated at $\rho = .4$.

30.3.1 Plot of Simulation

- Spaghetti plot per person
- Each line represents a person's score across conditions



30.3.2 Repeated-Measures ANOVA Logic

In a repeated-measures ANOVA, we identify variance due to individual differences. We estimate it from the row sums, which are the sums of each participant's scores.

30.3.2.1 One-way ANOVA Table With the formulas (conceptually simplified)

Source	SS	DF	MS	F
Between	$SS_B = nSS_{treatment}$	$K-1$	$MS_B = \frac{SS_B}{df_B}$	$\frac{MS_B}{MS_W}$
Within	$SS_W = \sum SS_{within}$	$N-K$	$MS_W = \frac{SS_W}{df_W}$	
Total	$SS_T = SS_{scores}$	$N-1$		

Note: $SS_B + SS_W = SS_T$ and $df_B + df_W = df_T$.

30.3.2.2 One-way RM ANOVA Table With the formulas (conceptually simplified)

Source	SS	DF	MS	F
RM_1	$SS_{RM} = nSS_{all\,cell\,means}$	$c-1$	$MS_{RM} = \frac{SS_{RM}}{df_{RM}}$	$\frac{MS_{RM}}{MS_E}$
Within	$SS_w = \sum SS_{within}$			
$Subject\,[Sub]$	$SS_{sub} = \frac{1}{c} SS_{subjects:\,rows\,sums}$	$n-1$	$MS_{Sub} = \frac{SS_{sub}}{df_{sub}}$	
$Error[Sub\,x\,RM_1]$	$SS_E = SS_W - SS_{sub}$	$(n-1)(c-1)$	$MS_E = \frac{SS_E}{df_E}$	
Total	$SS_T = SS_{scores}$	$N-1$		

Note: $SS_{sub} + SS_E = SS_W$, $SS_B + SS_W = SS_T$, and $df_B + df_W = df_T$.

30.3.2.2.1 Explained vs. Unexplained Terms

Explained:

SS_{RM} = variance caused by the treatment. SS_{sub} = individual differences. We do not know why people differ, but we can parse this variance, so we have *explained it.*

Unexplained:

SS_E = variance within participants that cannot be explained. This is not individual-difference variance.

30.3.3 Assumptions

As with the paired *t*-test, we assume HOV ($\sigma_1^2 = \sigma_2^2 = \sigma_3^2$). More importantly, we assume that the correlations among the three conditions are equal ($\rho_1 = \rho_2 = \rho_3$). We call this *compound symmetry*:

$$\mathbf{Cov} = \sigma^2 \begin{bmatrix} 1 & \rho & \rho \\ \rho & 1 & \rho \\ \rho & \rho & 1 \end{bmatrix}$$

When HOV and compound symmetry are met, we satisfy the assumption of **homogeneity of covariance**.

30.3.3.1 Sphericity

We evaluate this assumption with Mauchly's test of sphericity, which tests the variances of the difference scores among all conditions.

$H_0 : \sigma_{2-1}^2 = \sigma_{3-1}^2 = \sigma_{3-2}^2$

$H_1 : \sigma_{2-1}^2 \neq \sigma_{3-1}^2 \neq \sigma_{3-2}^2$

When sphericity is violated, we reduce the df to lower the risk of Type I error. We will not cover the calculation in detail. The basic idea is that a value called ϵ is calculated from the degree of violation. The greater the violation, the larger the df correction. The old-school approach corrected only after rejecting the null in Mauchly's test. The modern approach always applies a sphericity correction, even when Mauchly's test is nonsignificant. R follows the second approach by default.

There are two methods for calculating ϵ. The most common and slightly more conservative is the Greenhouse-Geisser (GG) correction. A more powerful but less common approach is the Huynh-Feldt (HF) correction. R defaults to GG, as do psychologists, but you can request HF.

30.3.4 Effect Sizes

30.3.4.1 What SPSS and Your Book Suggest

Your book suggests an effect size that is very similar to η_p^2.

Remember in two-way ANOVA:

$$\eta_p^2 = \frac{SS_A}{SS_A + SS_W}$$

Because our error term is no longer SS_W, we now use SS_E.

$$\eta_p^2 = \frac{SS_{RM}}{SS_{RM} + SS_E}$$

30.3.4.2 R and Generalized Effect Size

Olejnik and Algina (2003) instead propose generalized eta-squared, which can be compared across within- and between-subjects designs and is what `afex` has been reporting.

$$\eta_g^2 = \frac{SS_{RM}}{SS_{RM} + SS_{sub} + SS_E} = \frac{SS_{RM}}{SS_T}$$

You can also calculate an unbiased generalized effect size by hand or use the Excel sheets.

$$\omega_g^2 = \frac{df_{RM}(MS_{RM} - MS_E)}{MS_{sub} + SS_T}$$

See Bakeman (2005) for additional issues concerning effect sizes in repeated-measures and mixed designs.

30.3.5 R Calculations

30.3.5.1 R Defaults

- `afex` calculates the repeated-measures ANOVA and automatically corrects for sphericity violations.
- `Error(Subject ID/RM factor)`: This tells `afex` that participants vary as a function of the repeated-measures condition.
- It reports η_g^2 assuming a manipulated treatment.

```
library(afex)
Model.1<-aov_car(Fixation~ Chocolate + Error(SS/Chocolate),
        data = DataSim4)
Model.1
```

```
Anova Table (Type 3 tests)

Response: Fixation
     Effect         df  MSE      F  ges p.value
1 Chocolate 1.69, 6.77 5.36 8.65 * .611    .015
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1

Sphericity correction method: GG
```

30.3.5.2 SPSS-Like Report

Use the following code to see results formatted like SPSS output:

```
aov_car(Fixation~ Chocolate + Error(SS/Chocolate),
        data = DataSim4, return="univariate")
```

```
Univariate Type III Repeated-Measures ANOVA Assuming Sphericity

            Sum Sq num Df Error SS den Df  F value    Pr(>F)
(Intercept)  405.6      1   13.733      4 118.1359 0.0004067 ***
Chocolate     78.4      2   36.267      8   8.6471 0.0100065 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1


Mauchly Tests for Sphericity

          Test statistic p-value
Chocolate        0.81747 0.73911


Greenhouse-Geisser and Huynh-Feldt Corrections
 for Departure from Sphericity

           GG eps Pr(>F[GG])
Chocolate 0.84565     0.0155 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

            HF eps Pr(>F[HF])
Chocolate 1.398289 0.01000649
```

This reports Mauchly's test, which is nonsignificant, along with the uncorrected df and p-value. It also shows the p-values for the GG and HF corrections, if applied.

30.3.5.3 Manually Select DF Correction & η_p^2

You can request `HF`, `GG`, or `none`. You can also request η_p^2 with `pes`.

```
aov_car(Fixation~ Chocolate + Error(SS/Chocolate),
        data = DataSim4,
        anova_table=list(correction = "HF", es='pes'))
```

```
Anova Table (Type 3 tests)

Response: Fixation
     Effect   df  MSE      F  pes p.value
```

```
1 Chocolate 2, 8 4.53 8.65 * .684    .010
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1

Sphericity correction method: HF
```

30.4 Challenges of Repeated Design

- Practice effects
- Carryover effects
- Order effects
- Fatigue

30.4.1 Solutions

- Counterbalancing
- Time between trials/distractor tasks
- Using a matched design

30.5 References

Bakeman, R. (2005). Recommended effect size statistics for repeated measures designs. *Behavior Research Methods*, 37(3), 379-384.

Olejnik, S., & Algina, J. (2003). Generalized eta and omega squared statistics: Measures of effect size for some common research designs. *Psychological Methods*, 8(4), 434-447. doi:10.1037/1082-989X.8.4.434

31 Two-Way Within-Subjects ANOVA: Analysis and Graphing

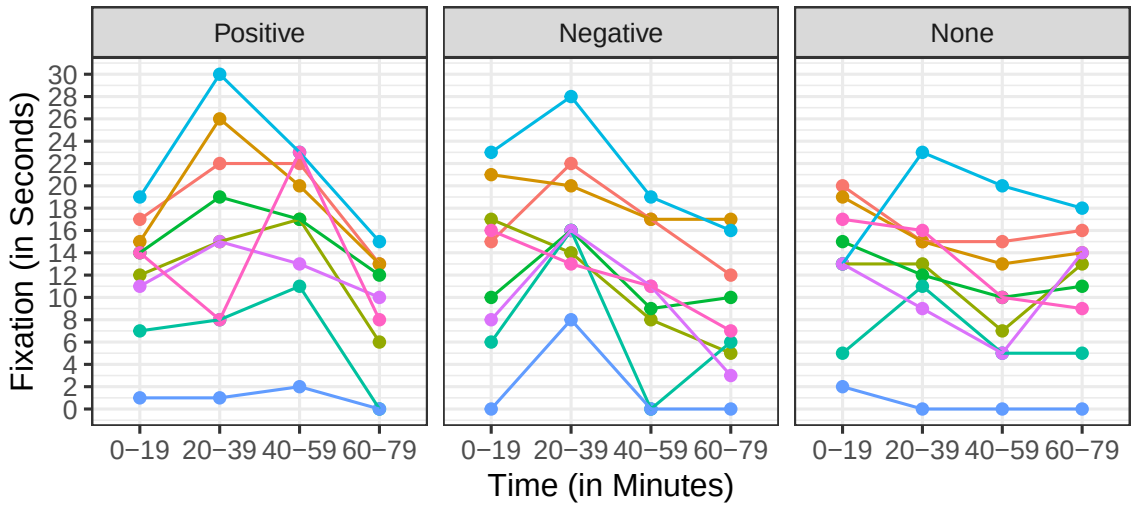
31.1 Two Repeated Factors

In the previous class, we saw that eating a small amount of chocolate increased fixation on the lecture. However, because we averaged fixation across the entire class, we do not know how long the effect lasts. To understand how chocolate modulates attention, we designed a fully within-subjects 4 x 3 study. We tracked graduate students' eyes during the first 20 minutes of class and measured their mean fixation time on the screen in seconds per glance. During the next 20 minutes, students received a fun-size Snickers to encourage them to pay attention. However, we were not sure how to "encourage" them. We could give them more Snickers as they spent more time looking at the screen, positive reinforcement, or encourage them to look back when they looked away for too long, negative reinforcement. The machine continued distributing Snickers for the rest of class based on the condition used during that lecture. We also needed a no-reinforcement control condition to understand how attention changed across the 80-minute lecture. Mean fixation time was recorded at the end of each 20-minute interval, producing four intervals. The order of the No, Positive, and Negative Reinforcement conditions was counterbalanced across lecture topics: Power Analysis, Bootstrapping Theory, and Probability Theory, which were matched on the thrillingness of the topic. We simulate $n = 9$ students. If classical learning theory is correct, positively rewarded students will pay more attention, while negatively rewarded students will learn that they receive more chocolate when they pay less attention. If graduate students are intrinsically motivated, negative reinforcement might not reduce attention as strongly as classical learning theory predicts. We also expect fatigue to reduce attention in all conditions.

31.1.1 Plot of Simulation

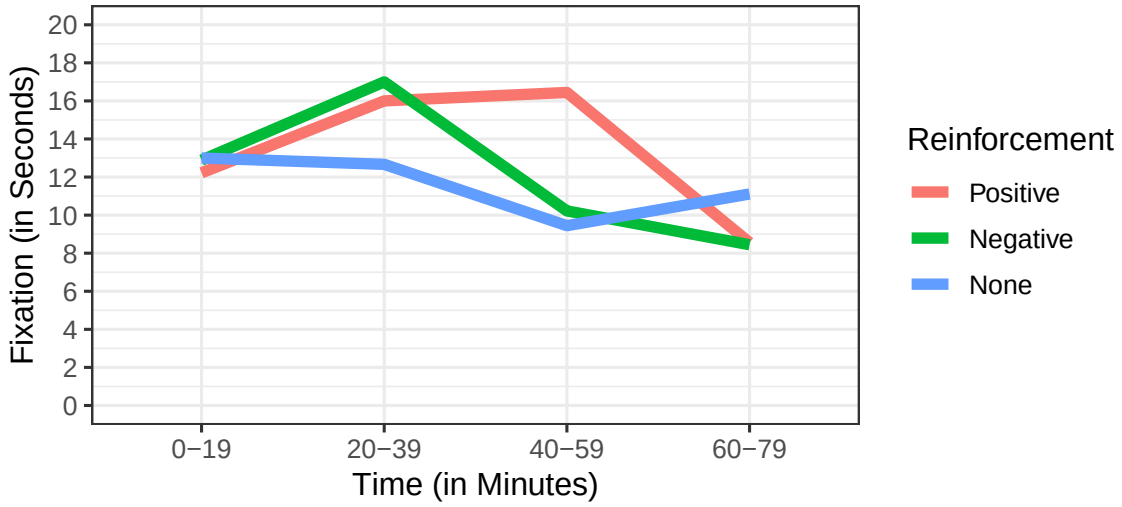
31.1.1.1 Spaghetti Plot

Each line represents a person's score across conditions.



31.1.1.2 Means Plot

For now, we will plot the means as lines without error bars. Error bars work differently in repeated-measures designs, and we will return to them later.



Are there main effects of Time or Reinforcement, and do Time and Reinforcement interact?

31.1.2 Two-Way Repeated-Measures ANOVA Logic

As in a two-way ANOVA, a two-way repeated-measures ANOVA has two main effects and an interaction. However, the error terms are more complicated. As in a one-way repeated-measures ANOVA, we identify variance due to individual differences. We estimate it from the row sums, which are the sums of each participant's scores.

31.1.2.1 One-way RM ANOVA Table With the formulas (conceptually simplified)

Source	SS	DF	MS	F
RM_1	$nSS_{Treat\,1\,cell\,means}$	$C-1$	$\frac{SS_{RM}}{df_{RM}}$	$\frac{MS_{RM_1}}{MS_{E_1}}$
Within	$\sum SS_{within}$			
$Subject\,[Sub]$	$\frac{1}{C}SS_{subjects:\,rows\,sums}$	$n-1$	$\frac{SS_{sub}}{df_{sub}}$	
$Error[Sub\,x\,RM_1]$	$SS_W - SS_{sub}$	$(n-1)(c-1)$	$\frac{SS_{E_1}}{df_{E_1}}$	
Total	SS_{scores}	$N-1$		

Note: $SS_{sub} + SS_E = SS_W$, $SS_{RM} + SS_W = SS_T$, and $df_{RM} + df_W = df_T$.

31.1.2.2 Two-way RM ANOVA Table With the formulas (conceptually simplified)

Source	SS	DF	MS	F
RM_1	$C_2nSS_{treat\,1\,cell\,means}$	C_1-1	$\frac{SS_{RM_1}}{df_{RM_1}}$	$\frac{MS_{RM_1}}{MS_{E_1}}$
RM_2	$C_1nSS_{treat\,2\,cell\,means}$	C_2-1	$\frac{SS_{RM_2}}{df_{RM_2}}$	$\frac{MS_{RM_2}}{MS_{E_2}}$
RM_{1x2}	$nSS_{all\,treat\,cells} - SS_{RM_1} - SS_{RM_2}$	$(C_1-1)(C_2-1)$	$\frac{SS_{RM_{1x2}}}{df_{RM_{1x2}}}$	$\frac{MS_{RM_{1x2}}}{MS_{E_{1x2}}}$
Within	$\sum SS_{within}$	$(n-1)(C_1C_2)$	$\frac{SS_w}{df_w}$	
$Subject\,[Sub]$	$\frac{1}{C_1}\frac{1}{C_2}SS_{subjects:\,rows\,sums}$	$n-1$	$\frac{SS_{sub}}{df_{sub}}$	
$Error_1[Sub\,x\,RM_1]$	$C_2SS_{treat\,1\,cells} - SS_{RM_1} - SS_{sub}$	$(n-1)(C_1-1)$	$\frac{SS_{E_1}}{df_{E_1}}$	
$Error_2[Sub\,x\,RM_2]$	$C_1SS_{treat\,2\,cells} - SS_{RM_2} - SS_{sub}$	$(n-1)(C_2-1)$	$\frac{SS_{E_2}}{df_{E_2}}$	
$Error_{1x2}[Sub\,x\,RM_{1x2}]$	$SS_w - SS_{sub} - SS_{E_1} - SS_{E_2}$	$(n-1)(C_1-1)(C_2-1)$	$\frac{SS_{E_{1x2}}}{df_{E_{1x2}}}$	
Total	SS_{scores}	$N-1$		

Note:

$SS_{sub} + SS_{E_1} + SS_{E_2} + SS_{E_{1x2}} = SS_W$

$SS_{RM1} + SS_{RM2} + SS_{RM1x2} + SS_W = SS_T$

$df_{RM1} + df_{RM2} + df_{RM1x2} + df_W = df_T$

31.1.2.2.1 Explained vs. Unexplained Terms

Explained:

- SS_{RM_1} = variance caused by Treatment 1.
- SS_{RM_2} = variance caused by Treatment 2.
- $SS_{RM_{1x2}}$ = variance caused by the Treatment 1 x Treatment 2 interaction.
- SS_{sub} = individual differences. We do not know why people differ, but we can parse this variance, so we have *explained it.*

Unexplained:

- SS_{E_1} = noise associated with RM_1.
- SS_{E_2} = noise associated with RM_2.
- $SS_{E_{1x2}}$ = noise associated with RM_{1x2}.

31.1.3 R ANOVA Calculations

31.1.3.1 R Defaults

- `afex` calculates the repeated-measures ANOVA and automatically corrects for sphericity violations.
- `Error(Subject ID/RM Factor 1 X RM Factor 2)`: This tells `afex` that participants vary as a function of the repeated-measures conditions.
- It reports η_g^2 assuming a manipulated treatment.

```
library(afex)
Model.1<-aov_car(Fixation~ Time*Reinforcement + Error(ID/Time*Reinforcement),
        data = DataSim2,include_aov = TRUE)
Model.1
```

```
Anova Table (Type 3 tests)

Response: Fixation
              Effect          df   MSE         F  ges p.value
1               Time 1.89, 15.09 13.76 18.04 *** .103   <.001
2      Reinforcement 1.87, 14.93  5.54    5.53 * .014    .017
3 Time:Reinforcement 3.80, 30.36 14.23  6.38 *** .078   <.001
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1

Sphericity correction method: GG
```

31.1.3.2 Assumptions

31.1.3.2.1 Sphericity

As in a one-way repeated-measures ANOVA, we examine Mauchly's test of sphericity for each term with more than two levels.

```
aov_car(Fixation~ Time*Reinforcement + Error(ID/Time*Reinforcement),
        data = DataSim2, return="univariate")
```

```
Univariate Type III Repeated-Measures ANOVA Assuming Sphericity

                    Sum Sq num Df Error SS den Df F value    Pr(>F)
(Intercept)        16428.0      1   3343.2      8 39.3112 0.0002405 ***
Time                 468.4      3    207.7     24 18.0412 2.410e-06 ***
Reinforcement         57.2      2     82.7     16  5.5323 0.0149196 *
Time:Reinforcement   344.6      6    432.2     48  6.3784 5.475e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1


Mauchly Tests for Sphericity
```

```
                   Test statistic p-value
Time                       0.36284 0.24003
Reinforcement              0.92799 0.76984
Time:Reinforcement         0.00899 0.18147


Greenhouse-Geisser and Huynh-Feldt Corrections
 for Departure from Sphericity

                    GG eps Pr(>F[GG])
Time                0.62881  0.0001184 ***
Reinforcement       0.93283  0.0173474 *
Time:Reinforcement 0.63258  0.0008870 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

                       HF eps   Pr(>F[HF])
Time                0.8166536 1.636504e-05
Reinforcement       1.2055774 1.491962e-02
Time:Reinforcement 1.2747975 5.475459e-05
```

31.1.3.2.2 Covariance Matrix

For most repeated-measures ANOVAs, we assume compound symmetry:

$$\mathbf{Cov} = \sigma^2 \begin{bmatrix} 1 & \rho & \rho & \rho \\ \rho & 1 & \rho & \rho \\ \rho & \rho & 1 & \rho \\ \rho & \rho & \rho & 1 \end{bmatrix}$$

However, as in the study we examined today, people's responses do not always have the same correlation across time. Think about a longitudinal study spanning weeks or months. Time introduces a problem called autoregression, or autocorrelation: what I do now affects what I do next. Remember that Time cannot be counterbalanced.

Many other covariance structures are possible, but not in a repeated-measures ANOVA. A repeated-measures ANOVA is a special case of a linear mixed model. Next semester, we will explore linear mixed models, which handle time more flexibly.

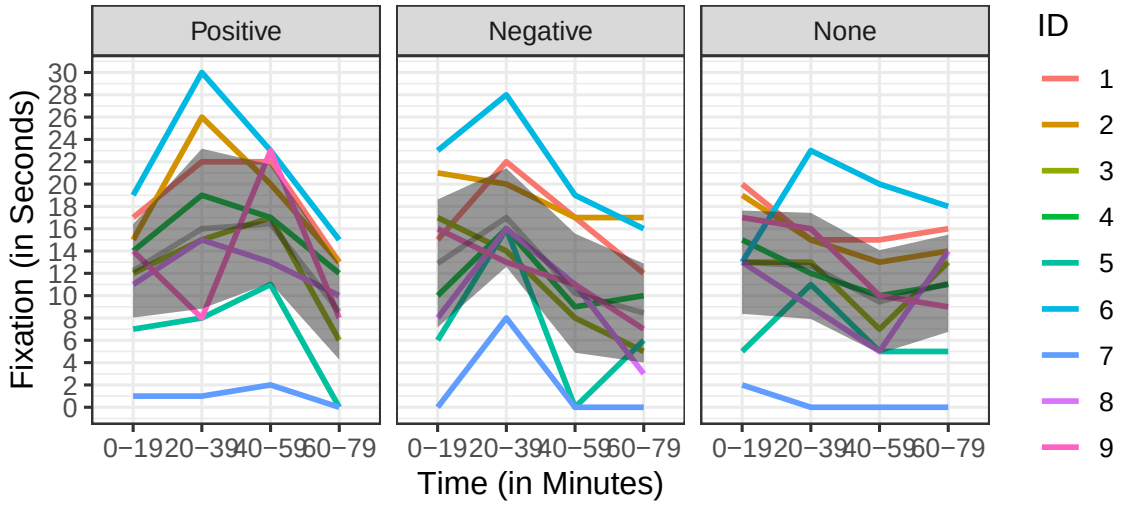
Here is a covariance matrix designed for time: first-order autoregressive, or AR(1).

$$\mathbf{Cov} = \sigma^2 \begin{bmatrix} 1 & \rho & \rho^2 & \rho^3 \\ \rho & 1 & \rho & \rho^2 \\ \rho^2 & \rho & 1 & \rho \\ \rho^3 & \rho^2 & \rho & 1 \end{bmatrix}$$

31.2 Plot Results

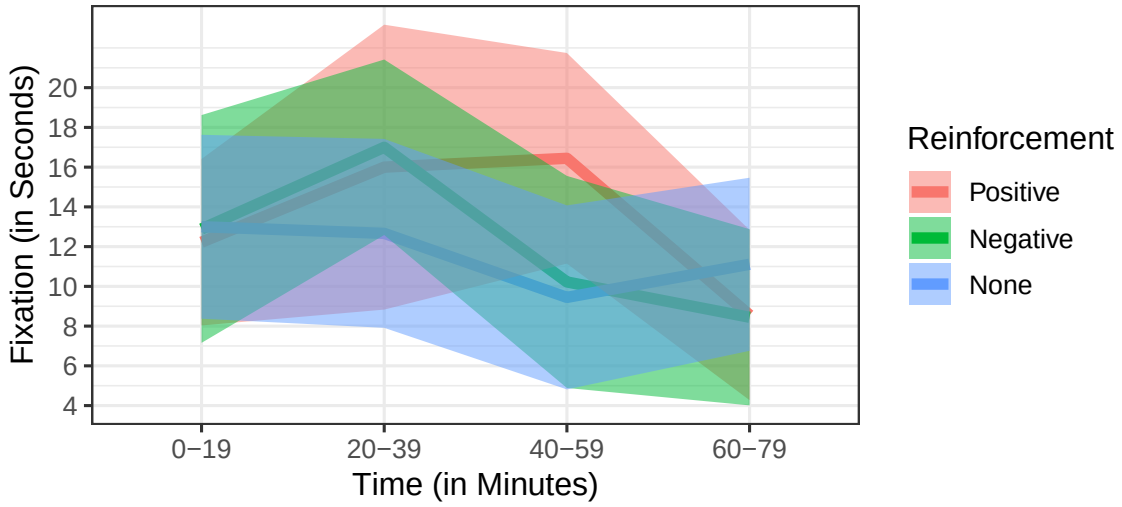
31.2.1 Problems with Error Bars

Within-subjects designs create several problems when we plot error bars to approximate significance visually. The main problem is that the error within each cell includes individual differences. The gray band over the data is the 95% confidence interval.



31.2.1.1 Uncorrected Error Bars

When we collapse the lines and plot the confidence intervals, they overlap completely. We might conclude that there are no significant differences between groups. The problem is that we have not removed the individual differences, which we can account for.

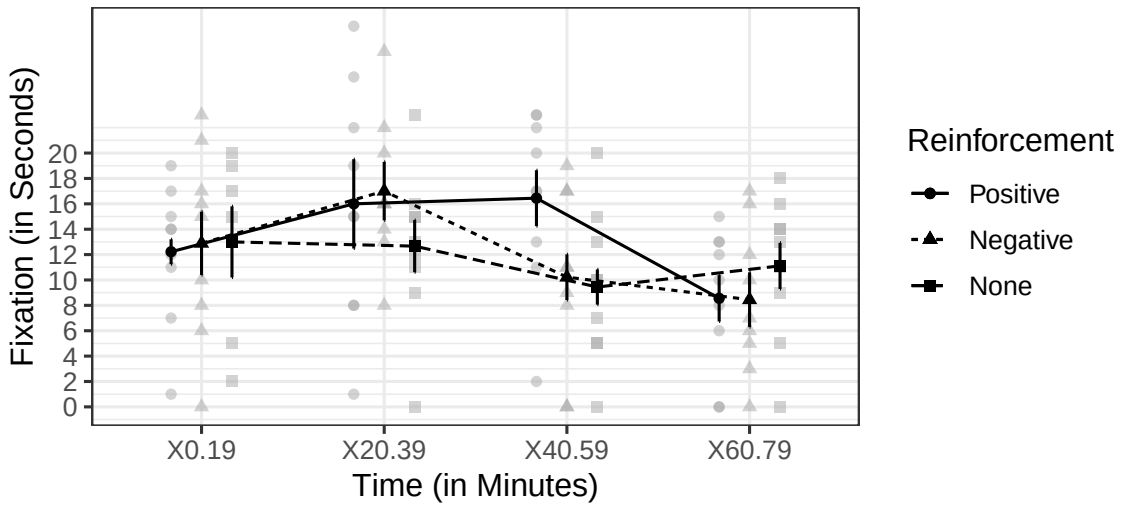


31.2.1.2 Within-Subjects Error Bars: Confidence Intervals

We will use Cousineau-Morey corrected error bars, which remove individual differences. See Figure 1 in Morey (2008) for a visual comparison of error-bar types and Cousineau and O'Brien (2014) for a review. First, we center each participant by removing their mean across all conditions. This makes each person's mean equal to 0 across conditions. Next, we recalculate the standard error or confidence interval from these centered means. Finally, we correct for bias because this method underestimates the variance. The correction is $\sqrt{\frac{J}{J-1}}$, where J is the number of within-subjects conditions. The corrected SE or confidence interval equals the value from the centered data multiplied by this correction.

The `afex_plot()` function now performs the Cousineau-Morey correction directly, so we no longer need the old custom helper functions. Set `error = "within"` for within-subject error bars. The default is a 95% confidence interval. Compare these corrected intervals with the uncorrected intervals above. The difference here is not massive.

```
Means.Within.CI <- afex_plot(
  Model.1,
  x = "Time",
  trace = "Reinforcement",
  error = "within",
  error_ci = TRUE
) +
  scale_y_continuous(breaks = seq(0, 20, 2)) +
  ylab("Fixation (in Seconds)") +
  xlab("Time (in Minutes)") +
  theme(legend.position = "right")
Means.Within.CI
```



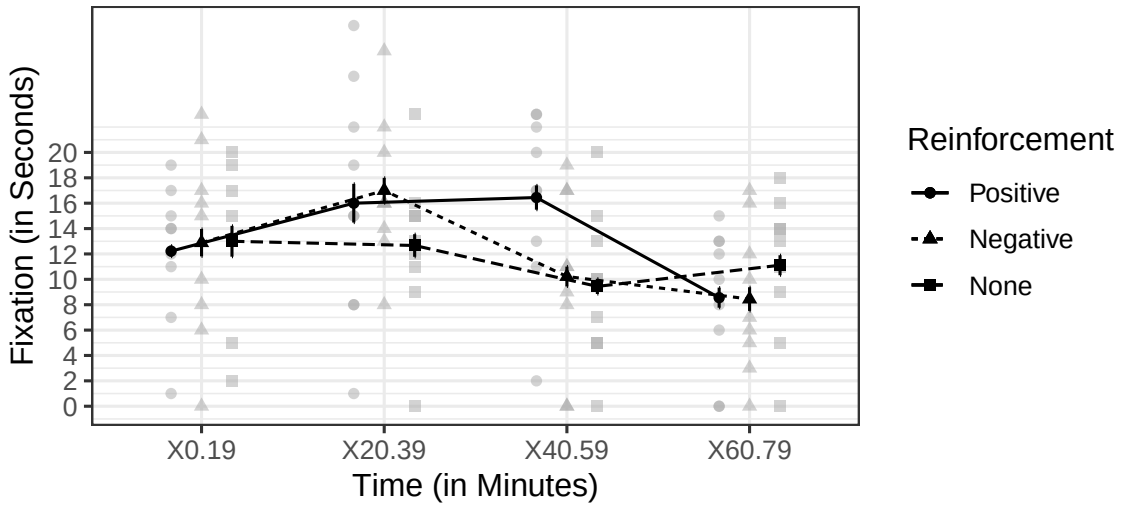
For corrected within-subjects error bars, you can visually approximate significance under suitable conditions (Cumming & Finch, 2005).

95% CIs:

- $p < .05$: when the proportional overlap is about .50
- $p < .01$: when the proportional overlap is about 0

31.2.1.3 Within-Subjects Error Bars: SE

```
Means.Within.SE <- afex_plot(
  Model.1,
  x = "Time",
  trace = "Reinforcement",
  error = "within",
  error_ci = FALSE
) +
  scale_y_continuous(breaks = seq(0, 20, 2)) +
  ylab("Fixation (in Seconds)") +
  xlab("Time (in Minutes)") +
  theme(legend.position = "right")
Means.Within.SE
```



SE bars:

- $p < .05$: when the gap is about one SE-bar length
- $p < .01$: when the gap is about two SE-bar lengths

31.3 References

Bakeman, R. (2005). Recommended effect size statistics for repeated measures designs. *Behavior Research Methods*, 37(3), 379-384.

Cousineau, D., & O'Brien, F. (2014). Error bars in within-subject designs: a comment on Baguley (2012). *Behavior Research Methods*, 46(4), 1149-1151.

Olejnik, S., & Algina, J. (2003). Generalized eta and omega squared statistics: Measures of effect size for some common research designs. *Psychological Methods*, 8(4), 434-447. doi:10.1037/1082-989X.8.4.434

32 Follow-Ups to the Two-Way Within-Subjects ANOVA

32.1 Follow-Up Testing

Following up main effects and interactions is mostly the same as in a two-way ANOVA. However, you have a few more decisions to make based on how conservative you want to be.

32.1.1 What Are Your Assumptions?

- Do you assume strong violations of sphericity? You can use two different approaches.
  - 1) Conduct only pairwise *t*-tests and ignore the omnibus repeated-measures ANOVA.
  - 2) Use a MANOVA, or multivariate ANOVA, for the error variance and conduct contrasts as usual. This is the default approach in `afex` as of version 1.0, matching **SPSS**.
- Do you assume no or only small violations of sphericity?

32.1.2 Assume Strong Violations of Sphericity: Classical Univariate Approach

Your book recommends not using the ANOVA error terms and instead running ONLY Bonferroni-corrected paired *t*-tests. This is the oldest approach and remains common. The problem is that it permits only pairwise analyses: **no contrasts of any kind**.

Pros:

- Provides the greatest protection against Type I error

Cons:

- MASSIVE inflation of Type II error
- You cannot conduct contrasts; you are limited to pairwise or stepwise tests.

32.1.2.1 Follow-up Interaction

It is best to examine simple effects. We will compare Reinforcement pairwise within each level of Time. This produces 12 tests and tells us when the reinforcement conditions separate at each time point.

Using `pairwise.t.test()`, we pass the arguments `paired = TRUE` and `p.adjust.method = "bonf"`. This is one family of tests, so $\alpha_{bon} = .05/12 = 0.0042$.

```
library(broom)
library(dplyr)
DataSim2 %>%
  group_by(Time) %>%
  do(
    tidy(
    pairwise.t.test(.$Fixation,.$Reinforcement,
                    paired=TRUE,
                    p.adj = "bonf")))
```

```
# A tibble: 12 x 4
# Groups:   Time [4]
   Time  group1   group2    p.value
   <fct> <chr>    <chr>       <dbl>
 1 0-19  Negative Positive 1
 2 0-19  None     Positive 1
 3 0-19  None     Negative 1
 4 20-39 Negative Positive 1
 5 20-39 None     Positive 0.392
 6 20-39 None     Negative 0.0159
 7 40-59 Negative Positive 0.00392
 8 40-59 None     Positive 0.000675
 9 40-59 None     Negative 1
10 60-79 Negative Positive 1
11 60-79 None     Positive 0.0512
12 60-79 None     Negative 0.323
```

32.1.2.1.1 Effect Sizes

Calculate Cohen's d, again ignoring the ANOVA error term. **The code is broken; I am working on it.**

32.1.3 Assume Strong Violations of Sphericity: Multivariate Approach

You can instead do what SPSS does by default for contrasts and assume violations of sphericity. This uses a multivariate analysis with smaller, more conservative df. We will not go deeply into univariate versus multivariate testing yet. This is a conservative approach.

Pros:

- You can apply an additional correction (HSD, MVT, Sidak, Holm, Bonferroni, etc.).
- You can do any type of contrast you want
- Provides better protection against Type I error

Cons:

- Can inflate Type II error

The error term and df do NOT come from the repeated-measures ANOVA. They come from a MANOVA, or multivariate analysis of variance. This approach treats the repeated measures as multiple but related measures from the same participant, such as three ways of measuring a similar construct. MANOVA "eats" all your df because we now use $df_E = n - 1$, making it less powerful overall and increasing Type II errors. MANOVA does not assume sphericity, although it accounts for covariance among measures. Running a MANOVA provides Pillai's trace, which ranges from 0 to 1, and an approximate F.

32.1.3.0.1 MANOVA Results vs. ANOVA

```
library(afex)
Model.1<-aov_car(Fixation~ Time*Reinforcement + Error(ID/Time*Reinforcement),
        data = DataSim2,include_aov = TRUE)
Model.1
```

```
Anova Table (Type 3 tests)

Response: Fixation
              Effect          df   MSE         F  ges p.value
1               Time 1.89, 15.09 13.76 18.04 *** .103   <.001
2      Reinforcement 1.87, 14.93  5.54    5.53 * .014    .017
3 Time:Reinforcement 3.80, 30.36 14.23  6.38 *** .078   <.001
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1

Sphericity correction method: GG
```

```
Model.M<-aov_car(Fixation~ Time*Reinforcement + Error(ID/Time*Reinforcement),
        data = DataSim2, return='Anova')
Model.M
```

```
Type III Repeated Measures MANOVA Tests: Pillai test statistic
                   Df test stat approx F num Df den Df    Pr(>F)
(Intercept)         1   0.83091   39.311      1      8 0.0002405 ***
Time                1   0.92902   26.177      3      6 0.0007611 ***
Reinforcement       1   0.65124    6.535      2      7 0.0250527 *
Time:Reinforcement  1   0.98753   39.605      6      3 0.0059995 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The interaction is no longer significant at $p < .05$, so we would not follow it up. However, people often do NOT examine the MANOVA results. They examine the repeated-measures ANOVA and then use the multivariate approach for follow-ups. It is logically inconsistent, but that is what SPSS users do and what many people are used to.

You do not need to run or inspect the MANOVA separately. Pass the original `afex` object to `emmeans` because it already contains the MANOVA.

32.1.3.0.2 Follow-Up Interaction

This example uses simple effects followed by pairwise comparisons.

```
library(emmeans)
Reinforcement.by.Time.M<-emmeans(Model.1,~Reinforcement|Time, model = "multivariate")
joint_tests(Reinforcement.by.Time.M, by = "Time")
```

```
Time = X0.19:
 model term    df1 df2 F.ratio p.value
 Reinforcement   2   8   0.480  0.6353

Time = X20.39:
 model term    df1 df2 F.ratio p.value
 Reinforcement   2   8   7.472  0.0148

Time = X40.59:
 model term    df1 df2 F.ratio p.value
 Reinforcement   2   8  20.677  0.0007

Time = X60.79:
 model term    df1 df2 F.ratio p.value
 Reinforcement   2   8   4.521  0.0486
```

Because only the middle two time points differ, we will examine the pairwise results within each family. You can also use any custom contrast you want. You must decide which correction to apply.

```
pairs(Reinforcement.by.Time.M, adjust ='tukey')
```

```
Time = X0.19:
 contrast            estimate    SE df t.ratio p.value
 Positive - Negative   -0.667 1.220  8  -0.544  0.8522
 Positive - None       -0.778 1.020  8  -0.759  0.7368
 Negative - None       -0.111 1.670  8  -0.067  0.9976

Time = X20.39:
 contrast            estimate    SE df t.ratio p.value
 Positive - Negative   -1.000 1.580  8  -0.632  0.8070
 Positive - None        3.333 1.980  8   1.684  0.2687
 Negative - None        4.333 1.140  8   3.792  0.0130

Time = X40.59:
 contrast            estimate    SE df t.ratio p.value
 Positive - Negative    6.222 1.290  8   4.829  0.0033
 Positive - None        7.000 1.110  8   6.332  0.0006
 Negative - None        0.778 1.050  8   0.740  0.7478

Time = X60.79:
 contrast            estimate    SE df t.ratio p.value
 Positive - Negative    0.111 1.230  8   0.090  0.9955
```

```
 Positive - None       -2.556 0.852  8  -3.001  0.0405
 Negative - None       -2.667 1.470  8  -1.812  0.2264

P value adjustment: tukey method for comparing a family of 3 estimates
```

32.1.4 Assume No or Small Violations of Sphericity: Univariate Approach

`emmeans` approximately uses the ANOVA error terms, although it actually calculates the pooled variance of the difference scores for each comparison. For the interaction, it calculates df using Satterthwaite approximations, which provide some correction for HOV. However, this does not control violations of sphericity. When sphericity is violated, this approach is anti-conservative and can inflate Type I error in follow-ups. When sphericity is not violated, this approach is equivalent to protected *t*-tests.

Pros:

- You can apply an additional correction (HSD, MVT, Sidak, Holm, Bonferroni, etc.).
- You can do any type of contrast you want

Cons:

- Can inflate Type I error, especially when sphericity is violated

32.1.4.1 Follow-Up Interaction

This example uses simple effects followed by pairwise comparisons.

```
library(emmeans)
Reinforcement.by.Time.UV<-emmeans(Model.1,~Reinforcement|Time,model = "univariate" )
joint_tests(Reinforcement.by.Time.UV, by = "Time")
```

```
Time = X0.19:
 model term    df1   df2 F.ratio p.value
 Reinforcement   2 61.38   0.198  0.8209

Time = X20.39:
 model term    df1   df2 F.ratio p.value
 Reinforcement   2 61.38   5.759  0.0051

Time = X40.59:
 model term    df1   df2 F.ratio p.value
 Reinforcement   2 61.38  16.467 <0.0001

Time = X60.79:
 model term    df1   df2 F.ratio p.value
 Reinforcement   2 61.38   2.546  0.0867
```

Only the middle two time points differ.

```
Pairs.UV<-pairs(Reinforcement.by.Time.UV, adjust ='tukey')
Pairs.UV
```

```
Time = X0.19:
 contrast            estimate   SE   df t.ratio p.value
 Positive - Negative   -0.667 1.34 61.4  -0.499  0.8722
 Positive - None       -0.778 1.34 61.4  -0.582  0.8304
 Negative - None       -0.111 1.34 61.4  -0.083  0.9962

Time = X20.39:
 contrast            estimate   SE   df t.ratio p.value
 Positive - Negative   -1.000 1.34 61.4  -0.748  0.7360
 Positive - None        3.333 1.34 61.4   2.493  0.0402
 Negative - None        4.333 1.34 61.4   3.241  0.0054

Time = X40.59:
 contrast            estimate   SE   df t.ratio p.value
 Positive - Negative    6.222 1.34 61.4   4.654 <0.0001
 Positive - None        7.000 1.34 61.4   5.235 <0.0001
 Negative - None        0.778 1.34 61.4   0.582  0.8304

Time = X60.79:
 contrast            estimate   SE   df t.ratio p.value
 Positive - Negative    0.111 1.34 61.4   0.083  0.9962
 Positive - None       -2.556 1.34 61.4  -1.911  0.1441
 Negative - None       -2.667 1.34 61.4  -1.994  0.1222

P value adjustment: tukey method for comparing a family of 3 estimates
```

32.1.4.1.1 Effect Sizes

Select the error term very carefully. If you follow up the interaction, use the interaction error term and match the edf to the df reported for the contrast. **This is not yet correct; I am still determining what to enter.**

```
SD.I = sqrt(Model.1$anova_table$MSE[3])
eff_size(Pairs.UV, sigma = SD.I, edf =62.8, method='identity') %>% summary()
```

```
Time = X0.19:
 contrast              effect.size    SE   df lower.CL upper.CL
 (Positive - Negative)     -0.1767 0.355 61.4   -0.886   0.5326
 (Positive - None)         -0.2061 0.355 61.4   -0.916   0.5034
 (Negative - None)         -0.0294 0.354 61.4   -0.738   0.6791

Time = X20.39:
 contrast              effect.size    SE   df lower.CL upper.CL
 (Positive - Negative)     -0.2650 0.355 61.4   -0.975   0.4451
 (Positive - None)          0.8835 0.363 61.4    0.158   1.6094
 (Negative - None)          1.1485 0.369 61.4    0.411   1.8861
```

```
Time = X40.59:
 contrast              effect.size    SE   df lower.CL upper.CL
 (Positive - Negative)      1.6492 0.384 61.4    0.882   2.4164
 (Positive - None)          1.8553 0.391 61.4    1.073   2.6374
 (Negative - None)          0.2061 0.355 61.4   -0.503   0.9157

Time = X60.79:
 contrast              effect.size    SE   df lower.CL upper.CL
 (Positive - Negative)      0.0294 0.354 61.4   -0.679   0.7380
 (Positive - None)         -0.6773 0.360 61.4   -1.396   0.0414
 (Negative - None)         -0.7068 0.360 61.4   -1.426   0.0129

sigma used for effect sizes: 3.773
Confidence level used: 0.95
```

32.1.4.1.2 Follow-Up of the Main Effect

This is a trend analysis of Time. You must calculate this BY HAND for the effect sizes to make sense.

```
Time.Main.Effect<-emmeans(Model.1,~Time,model = "univariate" )
Time.Main.Effect
```

```
 Time   emmean   SE   df lower.CL upper.CL
 X0.19   12.70 2.03 9.01     8.12     17.3
 X20.39  15.22 2.03 9.01    10.64     19.8
 X40.59  12.04 2.03 9.01     7.45     16.6
 X60.79   9.37 2.03 9.01     4.79     14.0

Results are averaged over the levels of: Reinforcement
Warning: EMMs are biased unless design is perfectly balanced
Confidence level used: 0.95
```

```
Poly.Time<-contrast(Time.Main.Effect, 'poly')
```

Effect sizes for the polynomial contrast: **I am still working on the code.**

32.2 Guidelines

- Graph your results with corrected error bars and examine spaghetti plots. This is the MOST important thing you can do in a repeated-measures design. You can see assumption violations, unusual participants or conditions, and participant-by-condition interactions. These patterns should guide your choice of follow-up tests and help you understand how the ANOVA parses the variance.
- Design repeated-measures studies with trend analyses in mind. Think ordinally when possible. When that is not possible, limit the number of levels you examine. Each additional level increases the chance of violating sphericity and inflating Type I error.

32.2.1 Follow-Ups

- When you use the more powerful univariate approach, report the packages used in your Method section.
- Apply an alpha correction and use a more conservative approach than you would for a between-subjects test.
- If you use the old-school method, avoid using the repeated-measures ANOVA error term for pairwise testing.
    - Yes, it buys power, but at a large cost of Type I error.
        * Conduct as few tests as possible so Bonferroni corrections do not destroy your power.

32.3 References

Bakeman, R. (2005). Recommended effect size statistics for repeated measures designs. *Behavior Research Methods*, 37(3), 379-384.

Cousineau, D., & O'Brien, F. (2014). Error bars in within-subject designs: a comment on Baguley (2012). *Behavior Research Methods*, 46(4), 1149-1151.

Olejnik, S., & Algina, J. (2003). Generalized eta and omega squared statistics: Measures of effect size for some common research designs. *Psychological Methods*, 8(4), 434-447. doi:10.1037/1082-989X.8.4.434

33 Two-Way Mixed-Design ANOVA: Analysis and Graphing

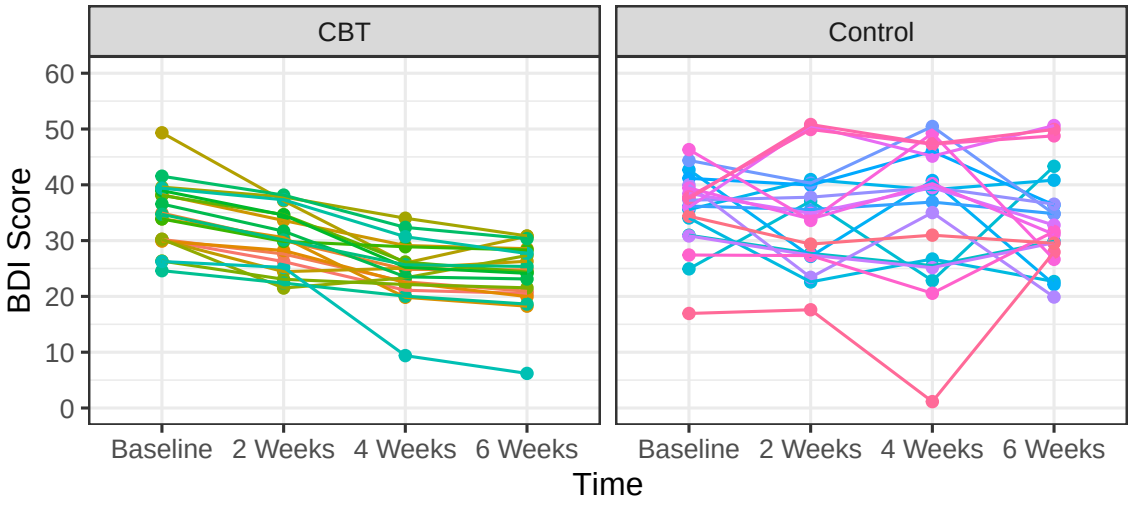
33.1 Mixed-Design Factors

You want to test the effectiveness of CBT therapy, compared with no therapy, for reducing depression scores. Clients and controls complete the Beck Depression Inventory (BDI) at baseline and every two weeks for six weeks. You assign 30 people with mild to severe depression, BDI scores from 20 to 50, to each between-subjects group: Control or CBT. You collect four BDI scores from each person, making Time a within-subjects factor. You expect BDI scores to drop by the second measurement and continue decreasing in the CBT group, while the Control group should show no change. Because the Control group receives no intervention and BDI scores can be unstable, you are concerned about assumption violations.

33.1.1 Plot of Simulation

33.1.1.1 Spaghetti Plot

Each line represents a person's score across conditions.



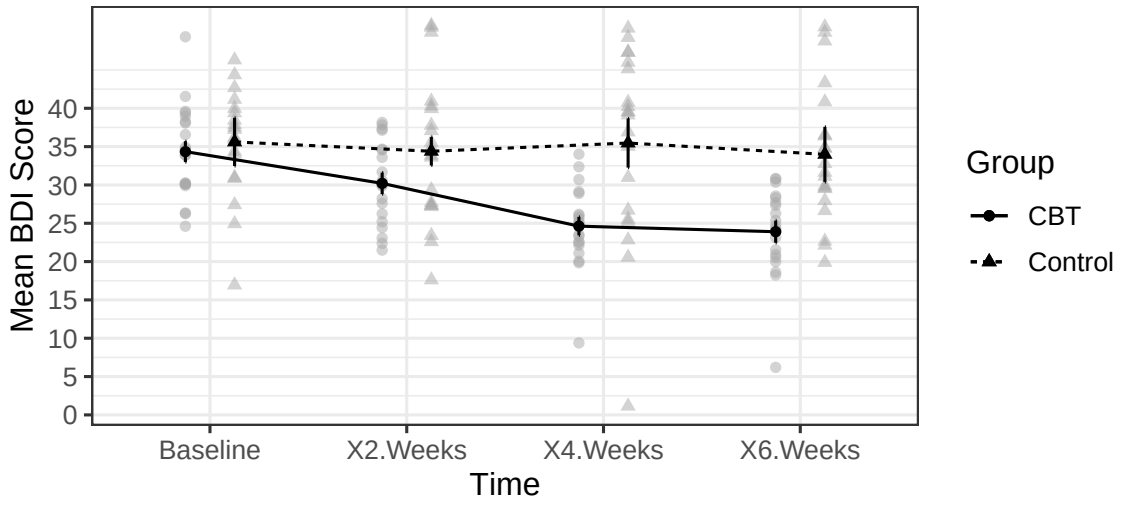
33.1.1.2 Means Plot

As in a repeated-measures ANOVA, we will use Cousineau-Morey corrected error bars, which remove individual differences. The current `afex_plot()` function performs this correction directly when we set `error = "within"`. Because `afex_plot()` uses a fitted `afex` model, we create the mixed-design ANOVA object here and use it again later.

```r
library(afex)

Mixed.1 <- aov_car(
  BDI ~ Time * Group + Error(ID / Time),
  data = DataSim2,
  include_aov = TRUE
)

Means.Within.CI <- afex_plot(
  Mixed.1,
  x = "Time",
  trace = "Group",
  error = "within",
  error_ci = TRUE
) +
  scale_y_continuous(breaks = seq(0, 40, 5)) +
  ylab("Mean BDI Score") +
  xlab("Time") +
  theme(legend.position = "right")
Means.Within.CI
```



These within-subject confidence intervals help compare time points within a group. Because this is a mixed design, they should not be used to compare the two groups at a single time point.

For corrected within-subjects error bars, you can visually approximate significance under suitable conditions (Cumming & Finch, 2005).

95% CIs:

- $p < .05$: when the proportional overlap is about .50
- $p < .01$: when the proportional overlap is about 0

Notice that you can also see problems with the variance.

33.1.2 Two-Way Mixed ANOVA Logic

Two-way mixed ANOVA table with conceptually simplified formulas:

Source	SS	DF	MS	F
$Between\,Subject$	$CSS_{row\,means}$	$nK-1$	$\frac{SS_{BS}}{df_{BS}}$	
$Group$	$CnSS_{Group\,means}$	$K-1$	$\frac{SS_G}{df_G}$	$\frac{MS_G}{MS_{W_G}}$
W_G	$SS_{BS}-SS_G$	$K(n-1)$	$\frac{SS_{W_G}}{df_{W_G}}$	
$Within\,Subject$	SS_T-SS_{BS}	$nK(C-1)$	$\frac{SS_w}{df_w}$	
RM	$KnSS_{RM\,means}$	$C-1$	$\frac{SS_{RM}}{df_{RM}}$	$\frac{MS_{RM}}{MS_{SXRM}}$
$GXRM$	$nSS_{Col\,means}-SS_G-SS_{RM}$	$(K-1)(C-1)$	$\frac{SS_{GXRM}}{df_{GXRM}}$	$\frac{MS_{GXRM}}{MS_{SXRM}}$
$Residual\,[Sub\,x\,RM]$	$SS_{WS}-SS_{RM}-SS_{GXRM}$	$K(n-1)(C-1)$	$\frac{SS_{SXRM}}{df_{SXRM}}$	
Total	SS_{scores}	$N-1$		

Notes:

- $SS_T = SS_{BS} + SS_{WS}$
  - $SS_{BS} = SS_G + SS_{W_G}$
  - $SS_{WS} = SS_{RM} + SS_{GXRM} + SS_{SXRM}$
- $df_T = df_{BS} + df_{WS}$
  - $df_{BS} = df_G + df_{W_G}$
  - $df_{WS} = df_{RM} + df_{GXRM} + df_{SXRM}$

33.1.2.1 Explained Terms

- SS_G = variance caused by Group.
- SS_{RM} = variance caused by the repeated treatment.
- SS_{GXRM} = variance caused by the Group x Treatment interaction.

33.1.2.2 Unexplained Terms

- SS_{W_G} = noise associated with Group.
- SS_{SXRM} = noise associated with Treatment.

33.1.3 R ANOVA Calculations

33.1.3.1 R Defaults

- `afex` calculates the repeated-measures ANOVA and automatically corrects for sphericity violations.
- `Error(Subject ID/RM Factor)`: This tells `afex` that participants vary as a function of the repeated-measures conditions.
- It reports η_g^2 assuming a manipulated group and treatment.

```
Mixed.1
```

```
Anova Table (Type 3 tests)

Response: BDI
      Effect          df    MSE         F  ges p.value
1      Group       1, 38 171.37  10.16 ** .157    .003
2       Time 1.92, 73.14  38.98 11.40 *** .084   <.001
3 Group:Time 1.92, 73.14  38.98  8.60 *** .064   <.001
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '+' 0.1 ' ' 1

Sphericity correction method: GG
```

33.1.3.2 Assumptions

33.1.3.2.1 Sphericity

As in a repeated-measures ANOVA, we examine Mauchly's test of sphericity for each term with more than two levels.

```
summary(Mixed.1, return="univariate")
```

```
Univariate Type III Repeated-Measures ANOVA Assuming Sphericity

            Sum Sq num Df Error SS den Df  F value    Pr(>F)
(Intercept) 159442      1   6511.9     38 930.4199 < 2.2e-16 ***
Group         1741      1   6511.9     38  10.1594  0.002871 **
Time           855      3   2850.8    114  11.3961 1.361e-06 ***
Group:Time     645      3   2850.8    114   8.5987 3.407e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1


Mauchly Tests for Sphericity

           Test statistic    p-value
Time              0.41054 4.3815e-06
Group:Time        0.41054 4.3815e-06


Greenhouse-Geisser and Huynh-Feldt Corrections
 for Departure from Sphericity

            GG eps Pr(>F[GG])
Time       0.64156  6.138e-05 ***
Group:Time 0.64156  0.0005214 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

              HF eps   Pr(>F[HF])
Time       0.6750962 4.288737e-05
```

```
Group:Time 0.6750962 4.031942e-04
```

Violations of sphericity are more common in mixed designs, so be cautious.

33.1.3.2.2 Covariance Matrix

We assume compound symmetry within each group:

$$\mathbf{Cov} = \sigma^2 \begin{bmatrix} 1 & \rho & \rho & \rho \\ \rho & 1 & \rho & \rho \\ \rho & \rho & 1 & \rho \\ \rho & \rho & \rho & 1 \end{bmatrix}$$

However, in the study we examined today, we have **two** covariance matrices, one for each group. We assume $COV_1 = COV_2$, so we must test whether the matrices are homogeneous. We can use Box's M.

33.1.3.2.3 Box's M

Box's M tests whether the covariance matrices are equal across groups, an assumption called homogeneity of covariance (**HOCV**). The R function requires us to spread the DV into columns based on the within-subjects factor. We will use `spread()` from the `tidyr` package: `dataset %>% spread(Within factor, DV)`.

```
library(tidyr); library(dplyr); library(knitr)
#Box's M
data_wide <- DataSim2 %>% spread(Time, BDI)
kable(head(data_wide))
```

ID	Group	Baseline	2 Weeks	4 Weeks	6 Weeks
1	CBT	30.12210	26.23478	21.10255	20.53913
2	CBT	30.20333	27.60115	22.61335	20.92678
3	CBT	34.93199	30.02745	24.84411	24.75778
4	CBT	29.90956	28.22758	22.47616	19.97385
5	CBT	33.91717	30.54910	19.83986	18.24787
6	CBT	38.24150	33.60338	29.12110	28.55372

Now that the data are wide, we index the DV columns, Columns 3 through 6. We pass these columns to `boxM()` from `heplots` and split them by group. We can view the covariance matrix for each group and the test results.

```
library(heplots)
BoxResult<-boxM(data_wide[,3:6],data_wide$Group)
BoxResult$cov
```

```
$CBT
          Baseline  2 Weeks  4 Weeks  6 Weeks
Baseline 36.80319 29.49647 21.26572 24.14393
2 Weeks  29.49647 28.71742 19.35184 17.08802
4 Weeks  21.26572 19.35184 28.18654 28.44126
6 Weeks  24.14393 17.08802 28.44126 33.06597
```

```
$Control
          Baseline  2 Weeks   4 Weeks   6 Weeks
Baseline 47.538709 25.48611  77.60549 -2.571358
2 Weeks  25.486106 87.61335  83.61910 75.609280
4 Weeks  77.605487 83.61910 150.55826 39.542095
6 Weeks  -2.571358 75.60928  39.54209 80.289598
```

```
BoxResult
```

```
 Box's M-test for Homogeneity of Covariance Matrices

data:  data_wide[, 3:6] by data_wide$Group
Chi-Sq (approx.) = 797.8098, df = 10, p-value = < 2.2e-16
```

Box's M does not work well with small samples and becomes overly sensitive in large samples. We can therefore set $\alpha = .001$. A significant result means that the covariance matrices are unequal, which creates a problem. It increases the rate at which we might commit a Type I error when examining the interaction: the treatment affected the covariance between trials differently across groups, not just the means as expected. Is the interaction due to mean differences or covariance differences?

33.1.3.2.4 HOV

Box's M is not a great test, so you should also check Levene's test, which centers the data on the mean, or the Brown-Forsythe test, which centers on the median, using the `car` package. Brown-Forsythe is more robust to violations of normality. The output will still say Levene's test when you center on the median. Both tests are sensitive to unequal group sizes. We can test HOV for Group and the Group x Treatment interaction. If interaction HOV is violated, that supports the conclusion from Box's M.

```
library(car)
# Brown-Forsythe test on Group Only
leveneTest(BDI~ Group, data=DataSim2,center=median)
```

```
Levene's Test for Homogeneity of Variance (center = median)
       Df F value  Pr(>F)
group   1   5.441 0.02093 *
      158
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Brown-Forsythe test on Group X Treatment Only
leveneTest(BDI~ Time*Group, data=DataSim2,center=median)
```

```
Levene's Test for Homogeneity of Variance (center = median)
       Df F value  Pr(>F)
group   7  2.6654 0.01251 *
      152
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

In this study, Box's M and the Brown-Forsythe test showed **HOCV** and **HOV** problems. We are in trouble. What can we do?

Solution? Cohen (2008) recommends that you **not** conduct the mixed ANOVA, meaning that you cannot test the interaction. You can conduct a one-way repeated-measures ANOVA within each group and an ANOVA or independent-samples *t*-test between groups after collapsing over the repeated-measures term. Is this extremely conservative approach justified?

33.1.4 How Serious Is the Problem?

I ran a simulation to examine Type I error rates under these violations. The simulation used 10,000 repetitions of a 2 (between) x 4 (within) design with a sample size of 30. All cells had a mean of 0, and the covariance matrices were random and differed across groups.

Type I Levels:

No HOCV violation: Interaction term in the ANOVA (uncorrected) = **.0465**

HOCV violation but no sphericity violation: Interaction term in the ANOVA (uncorrected) = **.059**

HOCV and sphericity violations: Interaction term in the ANOVA (uncorrected) = **.0616**

I then *followed up the significant interaction*, which was a Type I error, with **PAIRWISE** comparisons within each family. The table shows the proportion of simulations in which at least one follow-up test within a family was significant. These values should be high because the ANOVA already indicated that something was significant. They represent how often following a Type I interaction with additional tests produces a significant Type I follow-up.

Our expected alpha for 12 *within tests*: alpha error = 1-(1-.05)^12 = *.460*.

Our expected alpha for four *between tests*: alpha error = 1-(1-.05)^4 = *.185*.

Approach	Correction	Don't Violate HOCV	Violate HOCV	Violate HOCV + Sphericity
Univariate Within	None	.978	1	1
Univariate Within	Tukey	.557	.998	1
Univariate Within	Bonferroni	*.492*	.995	1
Multivariate Within	None	.976	1	1
Multivariate Within	Tukey	.557	.998	1
Multivariate Within	Bonferroni	*.480*	.991	.986
Univariate Between	None	.510	.844	.984
Univariate Between	Bonferroni	*.211*	.411	.411
Multivariate Between	None	.484	.842	.836
Multivariate Between	Bonferroni	*.206*	.392	.413
Ignore ANOVA Within	Bonferroni	*.273*	.338	.313
Ignore ANOVA Between	Bonferroni	*.206*	.364	.379

Ignore ANOVA Within = paired-samples *t*-tests Bonferroni-corrected within each group, producing six tests per family.

Ignore ANOVA Between = Welch t-tests Bonferroni-corrected within each time point, using .05/4.

This is why Cohen suggests that nonconservative follow-ups are problematic when HOCV is violated. In practice, almost no one avoids the interaction because it is usually the entire point of the study. But what happens to power?

Here is a simulation with 10,000 repetitions in which I had *88% power* to detect the interaction. The table shows power to detect a significant follow-up after a significant interaction. I include only simulations with a significant interaction, so these values should be very high.

Approach	Correction	Violate HOCV + Sphericity
Univariate Within	None	1
Univariate Within	Tukey	1
Univariate Within	Bonferroni	1
Multivariate Within	None	1
Multivariate Within	Tukey	1
Multivariate Within	Bonferroni	1
Univariate Between	None	.960
Univariate Between	Bonferroni	.740
Multivariate Between	None	.958
Multivariate Between	Bonferroni	.730
Ignore ANOVA Within	Bonferroni	1
Ignore ANOVA Between	Bonferroni	.700

If the interaction is NOT a Type I error, using a more conservative approach reduces power slightly.

33.1.5 Recommendations

- For Box's M, set $\alpha = .001$, double-check with Brown-Forsythe, and examine spaghetti plots when you have a small number of participants.
  - No violations:
    - The univariate and multivariate approaches should produce similar results. It is probably best to use a correction: Bonferroni, or Tukey if you are worried about losing too much power. You can apply the multivariate approach with a Bonferroni correction within each family, as SPSS does.
- When Box's M and Brown-Forsythe are violated:
  - Follow up ONLY the within-subjects variable, using the fewest tests needed to evaluate your predicted hypothesis and protect power.
    - If you need contrasts using the ANOVA error term, conduct only the necessary contrast and remember that it might not replicate.
    - If you need only pairwise tests, do NOT use the ANOVA error term. Conduct paired-samples t-tests and Bonferroni-correct the hell out of them. You can compare groups, but the Type I error rate is slightly higher and power is much lower, creating Type II problems.
    - The multivariate approach will NOT help, but your book is correct that you should be extremely careful.
  - **DO NOT DO ANY EXPLORATORY ANALYSIS; these results will all be Type I errors.**

Basically, if you designed the study well and have no violations, follow up as you would for a repeated-measures ANOVA using ***emmeans****. Focus on within-subjects follow-ups because they have greater power and therefore a lower Type II risk. If the violations look bad, think carefully about how to proceed.*

33.1.5.1 Example Follow-Up (Assuming No Violations)

33.1.5.1.1 Follow-Up Interaction

This example uses simple effects followed by a polynomial contrast.

```
library(emmeans)
Time.by.Group<-emmeans(Mixed.1,~Time|Group, model="multivariate")
```

The Control group changes over time, which we can follow up with polynomial, consecutive, or custom contrasts. We can index 1:3 to view only the CBT results.

```
contrast(Time.by.Group, 'poly')
```

```
Group = CBT:
 contrast  estimate   SE df t.ratio p.value
 linear     -36.928 5.17 38  -7.145 <0.0001
 quadratic    3.413 1.68 38   2.032  0.0491
 cubic        6.308 5.85 38   1.079  0.2876

Group = Control:
 contrast  estimate   SE df t.ratio p.value
 linear      -3.790 5.17 38  -0.733  0.4679
 quadratic   -0.257 1.68 38  -0.153  0.8792
 cubic       -4.856 5.85 38  -0.830  0.4116
```

33.2 References

Cousineau, D., & O'Brien, F. (2014). Error bars in within-subject designs: a comment on Baguley (2012). *Behavior Research Methods*, 46(4), 1149-1151.

Part V

Other Statistical Creatures

34 Pearson's Chi-Square

34.1 A Real Result in Two Numbers

Best (1979) watched 261 introductory psychology students take a multiple-choice exam and counted what happened every time somebody changed an answer. 27 changes went from right to wrong. 195 went from wrong to right.

> Do the observed counts differ from the counts we would expect by chance?

Categorical outcomes are everywhere in psychology. Did the participant comply or not, did the patient relapse or not, which of four options did they choose, did the answer change help or hurt. Whenever your outcome is a category instead of a quantity, this is the chapter you need. It is also the chapter where you get to watch a chance distribution get built from scratch, ten thousand fake students at a time, instead of being handed one and told to trust it.

34.2 Parametric vs. Non-Parametric

Statistical tests are often divided into parametric and nonparametric families. The difference concerns what the model assumes and what part of the data it compares.

34.2.1 Parametric Tests (everything up to this point)

Parametric tests describe populations with parameters such as means and variances. Particular tests make particular distributional and error assumptions; they do not all demand that the raw scores themselves be perfectly normal. They are powerful when their assumptions are reasonable.

You already know the ingredients, and by now you know the tests: the mean and SD from Part 1 are what every t-test, F-test, and regression you have run since then is built out of. That is the family we just watched fail on two counts.

34.2.2 Nonparametric Tests (this chapter and the next)

Nonparametric tests make fewer or different distributional assumptions and often ask questions about counts, signs, or ranks:

- They usually do not require normally distributed raw scores
- Work with ordinal or nominal data
- Also useful when parametric assumptions are violated
- Instead of means, they compare counts/frequencies or rank observations

They are not assumption-free. Independence, sampling, measurement, and the design still matter. The next chapter examines that entire nonparametric family in more detail.

34.2.3 Chi-Square

Pearson's chi-square is often placed under the "nonparametric" umbrella because it does not model means with a normal error distribution. More precisely, it is a test for **categorical counts**. It is not the emergency version of a t-test to use whenever normality looks annoyed.

Rather than asking: "How far is this group mean from another group mean?"

Chi-square asks: "Do the observed counts differ from what we would expect by chance?"

In technical terms, chi-square hypothesis testing asks whether an observed pattern is consistent with a chance-based null hypothesis.

- We compare observed data to expected values under null hypothesis (H_0), which means we expect no difference from chance.
- Large discrepancies lead us to reject H_0 and provide support for the alternative hypothesis (H_1).

Note: You already know this logic from the t-test chapters. Chi-square runs the same play with counts instead of means.

Chi-square is especially useful when:

- Outcomes are categories (yes/no, group A/B/C, correct/incorrect)
- Means and standard deviations either don't exist or don't make sense

There are two types of chi-square:

- One-Way Pearson's Chi-Square [Goodness of Fit]
- Two-Way Pearson's Chi-Square [Test of Independence]

34.3 One-Way Pearson's Chi-Square [Goodness of Fit]

Back to the two numbers from the start of the chapter, this time with the right tool. You have been told: "Stay with your first answer on a multiple-choice test." *So is changing answers more likely to be harmful?*

> Best studied the responses of 261 students in an introductory psychology course (Best 1979). He recorded the number of right-to-wrong (27), wrong-to-right (195), and wrong-to-wrong (39) answer changes for each student. Note: We will ignore wrong-to-wrong as he did in his first analysis.

Frequency	Right-to-Wrong	Wrong-to-Right
Observed	27	195

Total of 222 observations

We will use the **One-Way Chi-Squared** [Goodness of fit]. It asks whether the relative frequencies observed in the categories of a sample frequency distribution are in agreement with the relative frequencies hypothesized to be true in the population.

34.3.1 Hypothesis for Goodness of fit

There are two versions (but it does not change the math, just how you talk about it):

- No preference: there will be no preference between different categories (e.g., do people prefer Coke or Pepsi)
- No difference from a known population: This type hypothesis compares two populations (or representative samples)

If people were making chance guesses from a population of people who don't have any knowledge about material on the test (version 2 of the hypothesis):

- $H_0 : P_{right-to-wrong} = .5, P_{wrong-to-right} = .5$
- $H_1 : H_0$ is False

Note: The probability you set can be whatever you want, but across all cells, they must add to 1. When you do not know what probability should be from a theoretical standpoint, you can simply do 1/Number of cells.

34.3.2 Find Expected Frequencies

If we expect by chance people would have gotten $p = .50$ on wrong-to-right and right to wrong, $f_e = pn$

$F_{e_{right-to-wrong}} = .5 * 222 = 111$ $F_{e_{wrong-to-right}} = .5 * 222 = 111$

Frequency	Right-to-Wrong	Wrong-to-Right
Observed	27	195
Expected	111	111

Note: These tables are often shown in papers (sometimes as proportions).

34.3.3 Goodness of Fit Calculation by Hand

$$\chi^2 = \sum \frac{(f_o - f_e)^2}{f_e}$$

$\chi^2 = \frac{(27-111)^2}{111} + \frac{(195-111)^2}{111} = 127.14$

Note: Never enter proportions into the chi-square. You must work with frequencies.

One thing to notice before we go on. The expected frequency here is 111, which is exactly the mean the *t*-test printed at the top of the chapter. The *t*-test found the right number. It just had no idea what the number was for: it treated 111 as the center of the data, when 111 is the thing the data are supposed to be compared *against*.

34.3.4 Test Against Chance Distribution

χ^2 values close to zero are probably "chance," and values that are larger are farther from chance. Wait. How do I know that?

Look at our f_o values. If the f_o values were close to the f_e values, then the total value we calculate would be very close to zero.

For example, if we actually found this pattern instead:

Frequency	Right-to-Wrong	Wrong-to-Right
Observed	111	111
Expected	111	111

$\chi^2 = \frac{(111-111)^2}{111} + \frac{(111-111)^2}{111} = 0$

While it's very rare for the observed frequencies to be exactly what we expect by chance, it's often really close to zero.

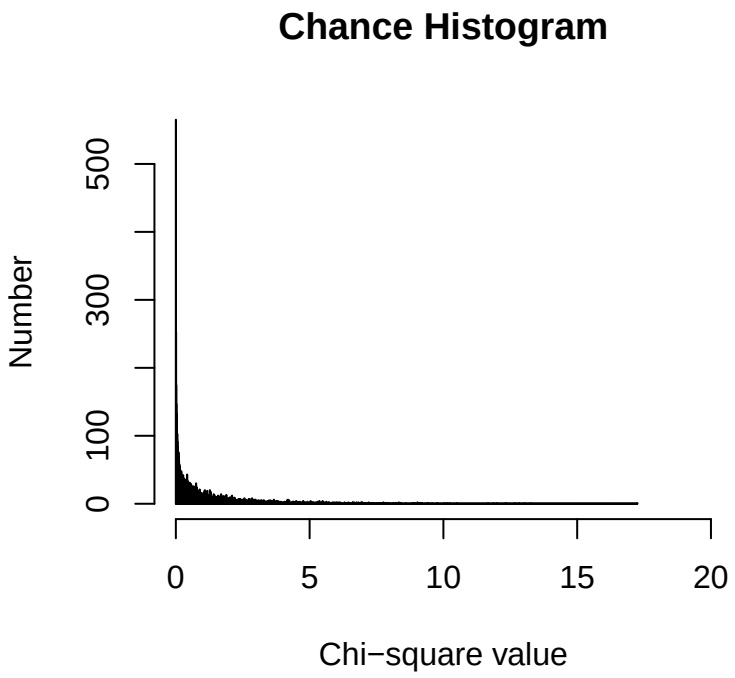
To get a sense of how often it would be close to zero, imagine we created a new test and the multiple-choice question said:

> "Please select an answer, then erase it and select something else. There is one right answer."

That's all you get. No actual knowledge is being tested. It's a pure game of chance! In other words, people are totally guessing and are forced to change their answer.

Sometimes, purely by chance, they will go from Right-to-Wrong or Wrong-to-Right (and we will ignore all the Wrong-to-Wrong cases). What would that look like?

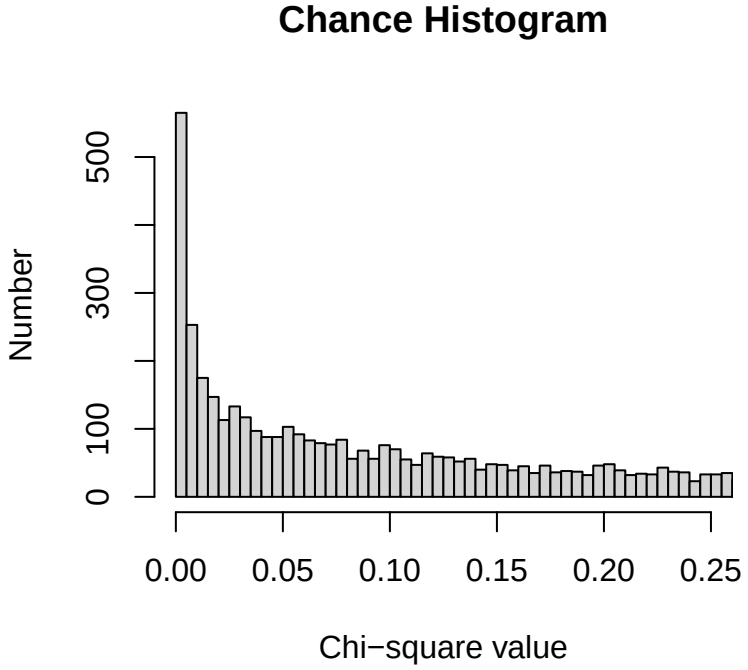
I can run 10,000 subjects to get a sense of what that might look like.



It looks like we get zero and values close to zero a lot, but also, purely by chance, we can get values all the way out to about 17.25. So even by chance, we can get a value that's "far" from zero.

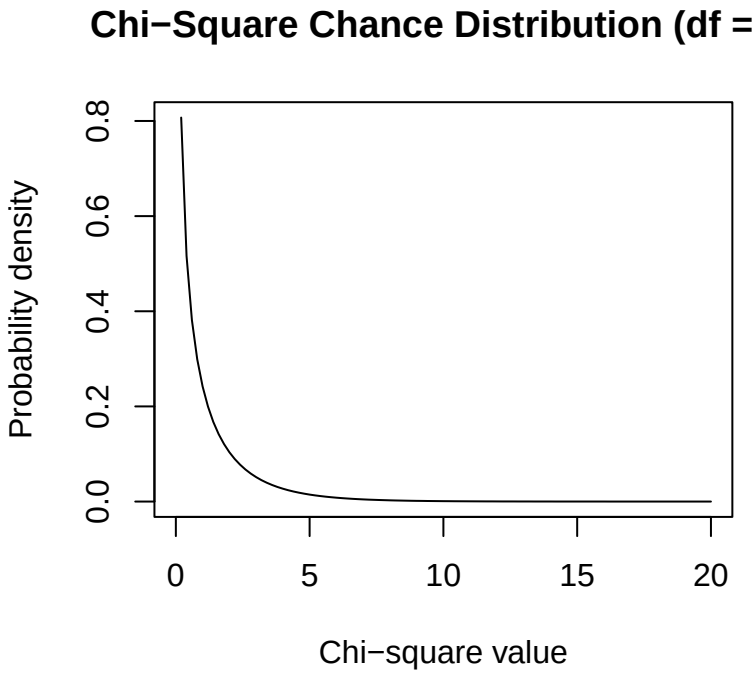
But does that happen often?

Before we answer that question, let's zoom in closer to zero so we can see what's going on more clearly.



So how far from zero is still "chance"?

To answer that question, we would need more than 10,000 people. In fact, we would need the entire population. That would be impossible, but luckily, someone has done the math-magic to figure out what that would look like:



Earlier, we looked at frequencies: how often different chi-square values occurred in a simulation.

Here, we switch to a probability density, which describes how likely different chi-square values are under the null hypothesis.

- The x-axis shows possible chi-square values.
- The y-axis shows relative likelihood (how often it will occur), not counts.

The important idea is that:

- Values near zero are very likely under chance
- Larger values are increasingly unlikely under chance

The total area under the curve equals 1, representing all possible outcomes under the null hypothesis.

34.3.5 How do we set chance?

How extreme does a chi-square value need to be before we stop believing it's just chance?

One very common rule is the 95% / 5% rule.

> We say: "If the null hypothesis is true, then 95% of the time our chi-square values should fall below some cutoff."

That cutoff defines the critical value. Any value beyond that point is rare enough that we start to doubt the chance explanation.

This does not mean:

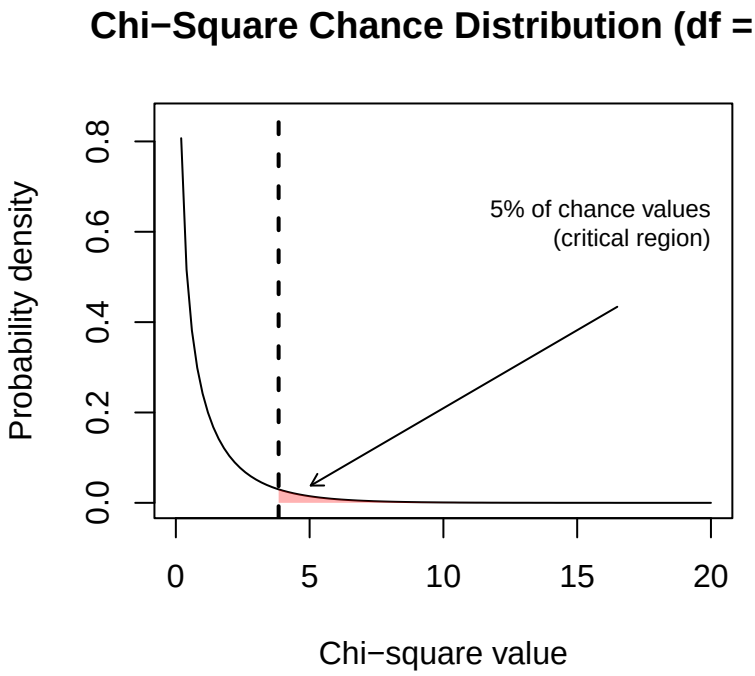
- That chance is impossible past that value
- Or that we are 100% certain the result is "real"

It means:

- About 5% of the time, even when the null hypothesis is true, we will still see values this extreme by chance alone
- We accept that risk in exchange for having a decision rule

So when we say a result is "unlikely due to chance," what we really mean is:

> "If this were just chance, we would expect to see something this extreme only about 5% of the time."



The distribution changes as a function of the degrees of freedom [df is the number of cells - 1]. Degrees of freedom tell us how much independent information is available to estimate variability in the data. If we have a different number of conditions, we have a different distribution to compare it to.

We get the crit = 3.84. Any χ^2 we calculate above that value means it has only a 5% probability of being a chance result.

34.3.6 Our results?

We ask where is our chi-square value relative to this chance distribution given our $df = 1$. We calculated 127.14, and that is in the tail (the small part, well above chance value). We can use R or a table from a book to get the critical values for alpha (p) =.05. We don't need to make the graph every time as someone did this already using calculus and has given us all the values with a simple function in R called `qchisq`.

This function answers the question:

- "What chi-square value cuts off a certain percentage of chance outcomes?"
  - `p = .05`
- We are defining the 5% region, the part of the distribution that happens only about 5% of the time under chance.
- `df = 1`
  - This tells R which chi-square distribution to use.
    - With two categories, the degrees of freedom is 1.
- lower.tail = FALSE
  - This is important:

* FALSE means we want the upper tail of the distribution

We get the crit $= 3.84 < 127.14$ (experimental value we got), so we reject the null hypothesis (H_0) and provide evidence for the alternative (H_1).

34.3.7 Report in APA

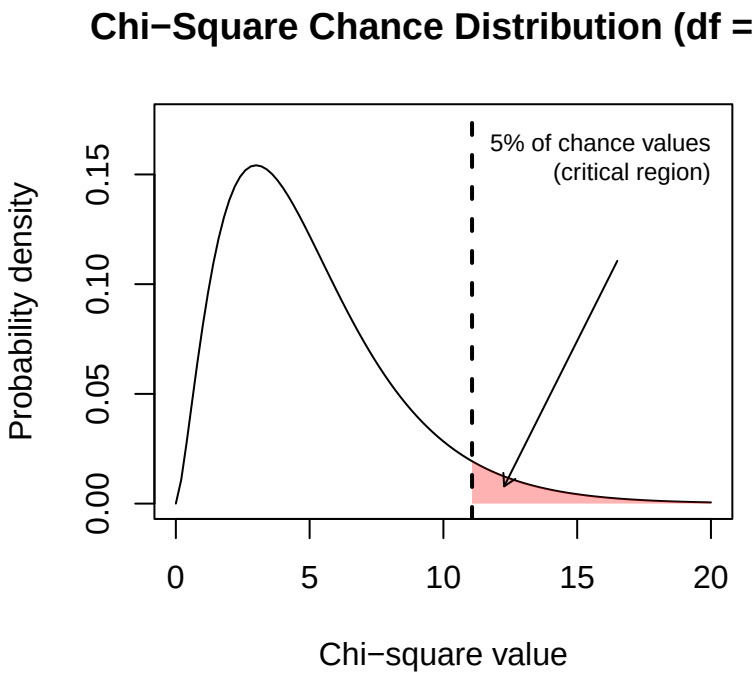
$\chi^2(df, N) = X.XX$, $p = .XXX$

Since our p-value was below .05, we rejected the null. Report the exact p-value to two or three decimals, and only fall back on $p < .001$ when it is smaller than that. Ours is far smaller than that.

Students were significantly more likely to change an answer from wrong to right (87.84%) than from right to wrong (12.16%), $\chi^2(1, N = 222) = 127.14$, $p < .001$.

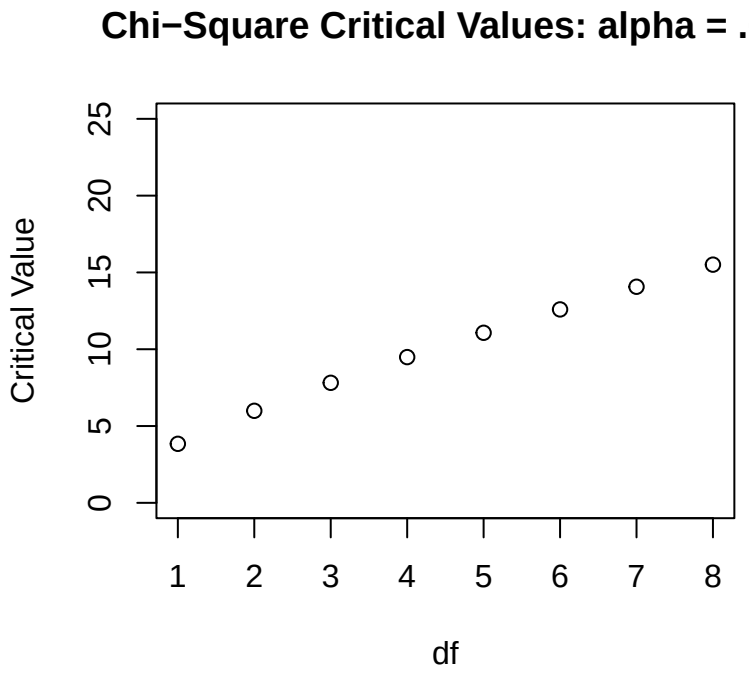
34.3.8 Impact of higher DF

Note: shape of chance distribution changes as we have more df (say 6 cells to compare)



As you can see the distribution gets more spread out. So with larger DF our critical value increases

```
ChiCrit<-qchisq(.05, 1:8, lower.tail = FALSE)
plot(ChiCrit, main="Chi-Square Critical Values: alpha = .05",
  xlab="df", ylab="Critical Value", xlim=c(1,8), ylim=c(0,25))
```



34.3.9 Goodness of Fit Calculation by Code

The way above is the hard way, and it shows you understand what the computer is doing. Doing one-way chi-squares in R is really easy!

34.3.10 Step 1: Build your Table

```
MCQ.Study <- as.table(rbind(c(27, 195)))
dimnames(MCQ.Study) <- list(Freq = c("Observed"),
                            Action = c("RtoW", "WtoR"))
MCQ.Study
```

```
          Action
Freq       RtoW WtoR
  Observed   27  195
```

34.3.11 Step 2: Calculate Chi-Square

```
MCQ.chi <- chisq.test(MCQ.Study, p=c(.5, .5))
MCQ.chi
```

```
    Chi-squared test for given probabilities
```

```
data:  MCQ.Study
X-squared = 127.14, df = 1, p-value < 2.2e-16
```

R prints the same 127.14 we ground out by hand, which is the point of doing it by hand once.

Again, in APA format, we would say,

Students were significantly more likely to change an answer from wrong to right (87.84%) than from right to wrong (12.16%), $\chi^2(1, N = 222) = 127.14$, $p < .001$.

34.4 Two-Way Pearson's Chi-Square [Test of Independence]

"Maybe we should only change our answers on easy items, but on hard items, we should trust ourselves."

> Best (1979) also examined the 1670 total number of responses across the 261 students and divided the items into easy/difficult in an introductory psychology course. He recorded the number of right-to-wrong, wrong-to-right, and wrong-to-wrong (39) answer changes for each student. Again we will ignore wrong-to-wrong (17% of the responses) for simplicity.

Frequency	Right to Wrong	Wrong to Right
Easy	97	411
Difficult	251	620

We need to do a Two-Way Chi-Square [Test of Independence]. It asks whether observed frequencies reflect the independence of two qualitative variables. Compares the actual observed frequencies of some phenomenon (in our sample) with the frequencies we would expect if there were no relationship at all between the two variables in the larger (sampled) population. We ask if the two variables are independent: knowledge of the value of one variable provides no information about the value of another variable.

34.4.1 Hypothesis for Test of Independence

The null hypothesis is that for each, the value obtained for one variable is not related to or influenced by the second variable. This idea can be expressed in two versions:

- **Version 1:** A single sample is measured on two separate variables. For example, knowing your personality will help predict your color preference (alternative). Knowing your personality will NOT help you predict your color preference (null) [Personality and color preference are measured per person]
  - Null: The variables are not related.
  - Alternative: The variables are related.
- **Version 2:** Two (or more) samples are measured and compared to see if there are differences. For example, the same proportions of extroverts (group 1) and introverts (group 2) prefer the same color (null). If the proportions are different for the two groups, then you have the alternative hypothesis
  - Null: The samples are independent
  - Alternative: The samples are not independent

Here we are working with Version 1 (one group taking two types of items)

34.4.2 Null Hypothesis (H_0)

- Answer-changing behavior is independent of item difficulty.
- The proportion of right-to-wrong and wrong-to-right changes is the same for easy and difficult items

Knowing whether an item is easy or difficult provides no information about how answers are changed

34.4.3 Alternative Hypothesis (H_1)

Answer-changing behavior depends on item difficulty.

- Students are more likely to benefit from changing answers on easy items
- On difficult items, answer changes are less likely to improve performance

Knowing whether an item is easy or difficult does provide information about how answers are changed

Simply put:

- H_0 : The item difficulty is not related to how a person changes their answer
- H_1 : The item difficulty is related to how a person changes their answer

34.4.4 Find Expected Frequencies

This is a little more complex than the goodness of fit test

34.4.5 Find row/col sums

Frequency	Right to Wrong	Wrong to Right	Sum
Easy	97	411	**508**
Difficult	251	620	**871**
Sum	**348**	**1031**	**1379**

34.4.6 Step 2: Expected frequencies per cell

$$f_e = \frac{col\,total * row\,total}{overall\,total}$$

Frequency	Right to Wrong	Wrong to Right	Sum
Easy	97 (128.1972)	411 (379.8028)	**508**
Difficult	251 (219.8028)	620 (651.1972)	**871**
Sum	**348**	**1031**	**1379**

34.4.7 Test of Independence Calculation by Hand

$$\chi^2 = \sum \frac{(f_o - f_e)^2}{f_e}$$

$\chi^2 = \frac{(97-128.1972)^2}{128.1972} + \frac{(411-379.8028)^2}{379.8028} + \frac{(251-219.8028)^2}{219.8028} + \frac{(620-651.1972)^2}{651.1972}$

$\chi^2 = 16.08$

34.4.8 Test Against Distribution

$df = (Row - 1)(Col - 1)$ $df = (2-1)(2-1) = 1$

Given our $df = 1$. We calculated 16.08 we can use R or a table to get the critical values for alpha = .05

We get the crit = 3.84 < 16.08, so we reject the null.

34.4.9 Report in APA

$\chi^2(df, N) = X.XX$, $p = .XXX$, $V = .XX$

In APA format:

Item difficulty was related to how students changed their answers, $\chi^2(1, N = 1379) = 16.08$, $p < .001$.

One flag before you meet the code. That hand calculation used no continuity correction, and R's default for a 2×2 table does. So the output a few sections down will print a slightly smaller number than this one. Nothing is broken; we sort it out when it happens.

[That is all we can say statistically. Normally people present a table in percentages to unpack this result, **but** You have to decide if you want to do row or column percentages. People do not present both]. I think the row percent make more sense in this case we want to compare across item difficulty.

% Row	Right to Wrong	Wrong to Right	%
Easy	19.1%	80.9%	100%
Difficult	28.8%	71.2%	100%
Col Total	25.2%	74.8%	100%

34.4.10 Test of Independence Calculation by Code

Again the code for R is very easy, so I will show you how it works:

34.4.11 Step 1: Build Your table

```
MCQ.Study.2 <- as.table(rbind(c(97, 411), c(251, 620)))
dimnames(MCQ.Study.2) <- list(Difficulty = c("Easy","Diff"),
                              Change = c("RtoW", "WtoR"))

MCQ.Study.2
```

```
          Change
Difficulty RtoW WtoR
      Easy   97  411
      Diff  251  620
```

34.4.12 Step 2: Calculate Chi-Square

In tests of independence, R calculates the expected frequencies for you. For a 2×2 table, `chisq.test()` applies **Yates's continuity correction** by default when `correct = TRUE`. The correction tries to improve a continuous chi-square approximation to discrete counts, but it can be overly conservative. It is not a ritual that must always be performed. Use it deliberately, report whether you used it, and consider Fisher's exact test when expected counts are sparse.

$$\chi^2_{Yates} = \sum \frac{(|f_o - f_e| - .5)^2}{f_e}$$

Note: You can view the observed and expected frequencies. For larger sparse tables, `chisq.test(..., simulate.p.value = TRUE)` can estimate a p-value by simulation.

```
MCQ.chi.2 <- chisq.test(MCQ.Study.2,correct = TRUE)
MCQ.chi.2
```

```
    Pearson's Chi-squared test with Yates' continuity correction

data:  MCQ.Study.2
X-squared = 15.566, df = 1, p-value = 7.968e-05
```

There is the mismatch we promised. R printed 15.57; we calculated 16.08 by hand. That gap is Yates, and it is the entire gap. Turn the correction off and R lands right back on the hand calculation:

```
chisq.test(MCQ.Study.2, correct = FALSE)
```

```
    Pearson's Chi-squared test

data:  MCQ.Study.2
X-squared = 16.077, df = 1, p-value = 6.082e-05
```

Neither number is the wrong one. They answer the same question with slightly different arithmetic, and both reject the null here. What matters is that you say which one you ran, because a reader cannot tell 15.57 from 16.08 by staring at it.

34.4.13 Row Proportion Code

```
prop.table(MCQ.Study.2,1)
```

```
          Change
Difficulty      RtoW      WtoR
      Easy 0.1909449 0.8090551
      Diff 0.2881745 0.7118255
```

34.4.14 Col Proportion Code

```
prop.table(MCQ.Study.2,2)
```

```
          Change
Difficulty      RtoW      WtoR
      Easy 0.2787356 0.3986421
      Diff 0.7212644 0.6013579
```

34.4.15 Effect Size: Because an Asterisk Is Not a Personality

The chi-square test tells us whether the observed table is difficult to explain under the null hypothesis. It does **not** tell us whether the association is large enough to matter. With enough observations, a microscopic association can earn an asterisk and begin acting important at faculty meetings.

For a 2×2 table, **phi** (ϕ) is:

$$\phi = \sqrt{\frac{\chi^2}{N}}$$

For a table of any size, use **Cramer's V**:

$$V = \sqrt{\frac{\chi^2}{N \min(r-1, c-1)}}$$

Here, r and c are the numbers of rows and columns. Cramer's V ranges from 0, meaning no association, to 1, meaning perfect association. Do not automatically translate `.10`, `.30`, and `.50` into small, medium, and large without thinking about the design and research area. Those labels are rough landmarks, not sacred commandments delivered on stone tablets by Cohen.

```
chi_uncorrected <- chisq.test(MCQ.Study.2, correct = FALSE)
n_total <- sum(MCQ.Study.2)

phi <- sqrt(as.numeric(chi_uncorrected$statistic) / n_total)
cramers_v <- sqrt(
  as.numeric(chi_uncorrected$statistic) /
    (n_total * min(dim(MCQ.Study.2) - 1))
```

```
)

c(phi = phi, cramers_v = cramers_v)
```

```
      phi cramers_v
0.1079744 0.1079744
```

For these data, $V = .11$: there is an association, but it is not a gigantic one. The asterisk has not transformed into a flaming sword.

34.4.16 Odds Ratios for a 2-by-2 Table

An **odds ratio** compares the odds of an outcome between two groups. In our table, the odds of changing an easy item from right to wrong rather than wrong to right are 97/411. For difficult items, those odds are 251/620.

$$OR = \frac{97/411}{251/620} = 0.58$$

An odds ratio of 1 would mean equal odds. Here, the odds of a right-to-wrong rather than wrong-to-right change were about 42% lower for easy items than for difficult items. If you reverse the comparison, the odds for difficult items are about $1/0.58 = 1.71$ times those for easy items. Odds ratios are direction-sensitive little creatures, so always say which group and outcome are in the numerator.

Base R gives us the odds ratio and its confidence interval with Fisher's exact test:

```
fisher_result <- fisher.test(MCQ.Study.2)
fisher_result$estimate
fisher_result$conf.int
```

```
odds ratio
 0.5831962
[1] 0.4421001 0.7654197
attr(,"conf.level")
[1] 0.95
```

The estimated odds ratio is 0.58, 95% CI [0.44, 0.77].

34.4.17 Power for Chi-Square

Power asks how often a study would detect an association of a specified size if that association really exists. Small samples give fickle chance more room to cause trouble. A weak association plus twelve people and one undergraduate who clicked every response while watching TikTok is not a sturdy research program.

For chi-square tests, Cohen's w is commonly used in power calculations. In a 2×2 table it is equivalent to phi and Cramer's V. The function below uses only base R. It finds the total sample size needed for 80% power when $w = .1062$, approximately the association observed here.

```r
chisq_power <- function(n, w, df, alpha = .05) {
  critical_value <- qchisq(1 - alpha, df = df)
  1 - pchisq(critical_value, df = df, ncp = n * w^2)
}

required_n <- uniroot(
  function(n) chisq_power(n, w = .1062, df = 1) - .80,
  interval = c(2, 10000)
)$root

ceiling(required_n)
```

```
[1] 696
```

The answer is about 696 total observations. That number is not permission to choose an effect size after seeing the results. Plan power using the smallest effect worth detecting, prior evidence, or a defensible range of possible effects.

34.4.18 Reporting the Result

A compact report can include the test, effect size, and a table of interpretable percentages:

> Item difficulty was associated with answer-change direction, $\chi^2(1, N = 1379) = 15.57$, $p < .001$, $V = .11$. Compared with difficult items, easy items had lower odds of being changed from right to wrong rather than wrong to right, $OR = 0.58$, 95% CI [0.44, 0.77].

The chi-square value above uses Yates's correction, matching the test already run in this chapter. The effect-size calculation uses the ordinary Pearson chi-square. State your choices so readers do not have to conduct forensic accounting on your results section.

34.4.19 Which Cells Are Doing the Work?

A significant chi-square says the table as a whole is hard to explain under the null. It does not say which part of the table caused the trouble. The fitted object already knows, and you do not have to compute anything: `chisq.test()` stores the standardized residuals.

```r
MCQ.chi.2$stdres
```

```
          Change
Difficulty      RtoW      WtoR
      Easy -4.009616  4.009616
      Diff  4.009616 -4.009616
```

Read those like z-scores, one per cell. Each is the gap between observed and expected divided by its own standard error, so anything past about ± 2 is a cell pulling real weight. The sign tells you the direction: positive means more cases landed there than chance predicted, negative means fewer. Easy items came up short on right-to-wrong changes and long on wrong-to-right ones, which is the association the omnibus test detected, now with an address.

All four values here are the same size because a 2×2 table has one degree of freedom: the four cells are one fact told four times. Residuals earn their keep on bigger tables, where a significant omnibus test can be hiding behind one rogue

cell out of twenty. That is also where you should start counting, because twenty cells inspected is twenty comparisons, and twenty comparisons is a family. See Multiple Comparisons.

34.4.20 Chi-Square Issues

- Chi-square works with **counts**, not percentages, means, or the emotional intensity with which you typed the numbers.
- The observations must be independent. One person should not appear in several cells unless the design and model explicitly handle repeated observations.
- The approximation becomes unreliable when expected counts are very small. A common guideline is that no expected count should be below 1 and no more than 20% should be below 5, but this is a guideline rather than a commandment. For a small 2×2 table, use Fisher's exact test. For larger sparse tables, a simulated p-value may be useful.
- Chi-square cannot be negative because every discrepancy is squared. It can equal zero when every observed count equals its expected count, an event normally witnessed immediately before the Probability Gods demand another sacrifice.
- A cell contributes more when its observed and expected counts differ by a large amount **relative to the expected count**.
- A significant omnibus test says that some association exists. It does not identify every cell responsible for it. Standardized residuals are the first place to look, and follow-up comparisons then create a multiple-testing problem, which is what Multiple Comparisons was for, before the asterisks multiply and form a union.

! Counts First; Interpretation Second

A chi-square test asks whether the observed **counts** differ from the counts expected under the null model. After that test, inspect the percentages, residuals, and effect size to learn where the association lives and whether anybody should care.

35 Nonparametric Tests: When the Data Refuse to Behave

35.1 What Makes a Test Nonparametric?

Many familiar tests describe data using parameters such as means and variances and make assumptions about how scores behave. **Nonparametric tests** make fewer or different assumptions, often by analyzing counts, signs, or ranks instead of the original scores.

This does **not** mean they are assumption-free. No statistical test arrives free of assumptions, calories, or consequences. Nonparametric tests still care about the design, independence, what was measured, and what question the test actually answers.

They are especially useful when:

- The outcome is nominal or ordinal.
- A few extreme scores are dragging the mean across the room by its ankles.
- The sample is too small to trust an approximation that needs large samples.
- Ranks or directions of change answer the research question better than raw-score differences.

Do not select a nonparametric test merely because a normality test produced an asterisk. With large samples, normality tests detect tiny, harmless departures. With small samples, they may miss serious ones. Look at the data, consider the design, and decide what feature of the outcome you actually want to compare.

35.2 The Very Short Decision Guide

Research question	Design	Useful test
Is one category more common than a specified probability predicts?	One binary variable	Exact binomial test
Are two binary variables associated in a small table?	Independent observations, usually 2×2	Fisher's exact test
Do two independent groups tend to have different scores?	Two separate groups	Wilcoxon rank-sum / Mann–Whitney test
Did paired scores tend to change?	Two scores from each person	Sign test or Wilcoxon signed-rank test
Do three or more independent groups differ?	Separate groups	Kruskal–Wallis test
Do three or more repeated conditions differ?	Same people in every condition	Friedman test

The table is a map, not an oracle. A map can get you to the correct neighborhood. It cannot stop you from driving into a volcano because Google said the road continued.

35.3 Tests for Nominal Outcomes

35.3.1 Fisher's Exact Test

Pearson's chi-square uses an approximation that works well when expected counts are reasonably large. When a 2×2 table is tiny or sparse, **Fisher's exact test** calculates an exact p-value under fixed margins.

Suppose sixteen students volunteer for an escape-room study. Half receive calm-breathing instructions. Half are pursued by a research assistant holding a foam chainsaw and repeatedly yelling, "This was approved by the IRB!" The outcome is whether they escape before the timer expires.

```
escape_table <- matrix(
  c(1, 7,
    6, 2),
  nrow = 2,
  byrow = TRUE,
  dimnames = list(
    Training = c("Calm breathing", "Foam chainsaw"),
    Escaped = c("Yes", "No")
  )
)

escape_table
```

```
                Escaped
Training         Yes No
  Calm breathing   1  7
  Foam chainsaw    6  2
```

```
chisq.test(escape_table)$expected
```

```
                Escaped
Training         Yes  No
  Calm breathing 3.5 4.5
  Foam chainsaw  3.5 4.5
```

```
fisher.test(escape_table)
```

```

    Fisher's Exact Test for Count Data

data:  escape_table
p-value = 0.04056
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 0.0009187486 0.9194607531
sample estimates:
odds ratio
0.06173059
```

Fisher's test is not merely "chi-square for small samples," but that is its most common job in an introductory workflow. Unlike Pearson's large-sample approximation, it obtains the p-value from the possible tables with the same margins. It also returns an odds-ratio estimate and confidence interval.

Do not automatically use Fisher's test for every table. For larger tables it can become computationally expensive, and the exact question depends on how the margins are treated. When expected counts are healthy, Pearson's chi-square is usually perfectly happy doing its job.

35.3.2 The Exact Binomial Test

The **binomial test** handles a binary outcome when trials are independent and the probability under the null hypothesis is specified.

Suppose 18 of 24 students report hearing colors after consuming a legally and ethically questionable quantity of caffeine. If "yes" and "no" were equally likely, the null probability would be .50.

```
binom.test(
  x = 18,
  n = 24,
  p = .50,
  alternative = "two.sided"
)
```

```
    Exact binomial test

data:  18 and 24
number of successes = 18, number of trials = 24, p-value = 0.02266
alternative hypothesis: true probability of success is not equal to 0.5
95 percent confidence interval:
 0.5328872 0.9022696
sample estimates:
probability of success
                  0.75
```

The p-value asks how unusual 18 or a result at least this far from the expected 12 would be if the true probability were .50. The confidence interval estimates the probability of hearing colors. Neither number establishes that caffeine **caused** chromatic audition. Perhaps these students already heard mauve every Thursday. Design still matters.

35.3.3 The Sign Test

The **sign test** is a binomial test applied to paired differences. It ignores how large each change is and records only its direction:

- `+` means the score moved in the predicted direction.
- `-` means it moved in the opposite direction.
- Ties are removed because they have no direction.

Suppose students rate exam panic before and after receiving a one-page study guide. Lower scores are better.

```
panic_before <- c(8, 7, 9, 6, 8, 7, 9, 5, 8, 6, 7, 9, 8, 6, 7, 8)
panic_after  <- c(5, 6, 7, 6, 4, 5, 6, 4, 7, 5, 7, 5, 6, 7, 5, 6)

change <- panic_before - panic_after
non_ties <- change[change != 0]
n_improved <- sum(non_ties > 0)

binom.test(n_improved, length(non_ties), p = .50)
```

```
    Exact binomial test

data:  n_improved and length(non_ties)
number of successes = 13, number of trials = 14, p-value = 0.001831
alternative hypothesis: true probability of success is not equal to 0.5
95 percent confidence interval:
 0.6613155 0.9981932
sample estimates:
probability of success
             0.9285714
```

The sign test is wonderfully stubborn: a one-point improvement and a seven-point improvement both count as one plus. That makes it resistant to extreme magnitudes, but it also throws away useful information. This is the statistical equivalent of judging a chainsaw and a butter knife only by whether each object is pointy.

35.4 Tests Based on Ranks

Ranking replaces the original scores with their ordered positions. The smallest score gets the lowest rank, the next score gets the next rank, and tied scores share ranks. Extreme raw values therefore lose much of their power to terrorize the analysis.

However, a rank test does not automatically become "a test of medians." Under additional assumptions, especially similarly shaped distributions, it may be interpreted as a location or median comparison. Without those assumptions, the safer question is whether scores from one condition tend to be higher or lower than scores from the other.

35.4.1 Two Independent Groups: Wilcoxon Rank-Sum

The **Wilcoxon rank-sum test**, also called the **Mann–Whitney U test**, compares two independent groups.

```
fear_data <- data.frame(
  training = factor(rep(c("Breathing", "Foam chainsaw"), each = 10)),
  fear = c(
    3, 4, 2, 5, 4, 3, 6, 2, 4, 5,
    7, 8, 6, 9, 7, 10, 8, 6, 9, 7
  )
)
```

```r
aggregate(fear ~ training, data = fear_data, median)
```

```
       training fear
1     Breathing  4.0
2 Foam chainsaw  7.5
```

```r
wilcox.test(
  fear ~ training,
  data = fear_data,
  exact = FALSE,
  conf.int = TRUE
)
```

```
    Wilcoxon rank sum test with continuity correction

data:  fear by training
W = 1, p-value = 0.0002243
alternative hypothesis: true location shift is not equal to 0
95 percent confidence interval:
 -5.000009 -2.999983
sample estimates:
difference in location
                    -4
```

`exact = FALSE` asks for the normal approximation. Psychology data tie constantly, and with tied ranks R cannot compute an exact p-value: it warns you and falls back to the approximation anyway. We are just telling it upfront so nobody panics at a red message. Every rank test in this chapter does the same thing for the same reason.

The groups are independent because each student appears in only one training condition. If Alex secretly reuses the same students because recruitment is annoying, the design becomes paired and the analysis must change. Laziness is not a covariance structure.

35.4.2 Reading the Line Everyone Skips

That output has a line labeled **difference in location**, and it is the most useful number on the page. It is the **Hodges–Lehmann estimate**, and it is defined by something you can do by hand: pair every Breathing score with every Foam chainsaw score, take all one hundred differences, and take the median of those.

```r
breathing <- fear_data$fear[fear_data$training == "Breathing"]
chainsaw  <- fear_data$fear[fear_data$training == "Foam chainsaw"]

all_pairwise_differences <- outer(breathing, chainsaw, "-")

c(
  n_pairs = length(all_pairwise_differences),
  hodges_lehmann = median(all_pairwise_differences)
)
```

```
 n_pairs hodges_lehmann
     100             -4
```

Same number R printed. A typical Breathing student sits about four fear points below a typical chainsaw student, and the confidence interval says the data are consistent with a shift somewhere between three and five points. That is an effect size **in the units of the thing you measured**, which is what item 4 of the reporting checklist below is asking for. Report it with the group medians and IQRs and you have satisfied items 3 and 4 in one breath.

Note that this is a shift estimate, not a difference of the two medians. Here they happen to be close, but they are different quantities and only one of them comes with an interval.

If a reviewer wants a standardized effect size instead, the rank-based analogue of a correlation is the **rank-biserial correlation**:

```
library(effectsize)

rank_biserial(fear ~ training, data = fear_data)
```

```
r (rank biserial) |         95% CI
-----------------------------------
-0.98             | [-0.99, -0.94]
```

It runs from -1 to 1 and answers "how often does a score from one group beat a score from the other." At -0.98 these two groups barely overlap, which you can see by looking at the raw numbers: the highest Breathing score is a 6 and the lowest chainsaw score is a 6. Standardized values are handy for meta-analysis and for reviewers who want a number they can compare across studies. For everyone else, four fear points is easier to understand than -0.98.

35.4.3 Two Paired Conditions: Wilcoxon Signed-Rank

The **Wilcoxon signed-rank test** uses both the direction and rank of the absolute paired differences. It is more informative than the sign test, but it assumes the distribution of paired differences is reasonably symmetric.

Use the two-vector interface in modern R:

```
wilcox.test(
  x = panic_before,
  y = panic_after,
  paired = TRUE,
  exact = FALSE,
  conf.int = TRUE
)
```

```
    Wilcoxon signed rank test with continuity correction

data:  panic_before and panic_after
V = 102, p-value = 0.001881
alternative hypothesis: true location shift is not equal to 0
95 percent confidence interval:
 0.9999786 2.4999692
```

```
sample estimates:
(pseudo)median
      1.500002
```

This output labels its estimate **(pseudo)median** rather than "difference in location," which is R being fussy but correct. It is the same Hodges–Lehmann idea applied to one set of paired differences: average every difference with every other difference, including itself, and take the median of those averages. Read it the same way. Panic dropped by about a point and a half, and the interval runs from 1 to 2.5.

Do **not** write `wilcox.test(score ~ condition, paired = TRUE)`. The formula method does not safely identify which observation should be paired with which. Giving the two aligned vectors makes the pairing explicit and prevents R from responding with the statistical equivalent of "I have no idea which student is which."

35.5 Three or More Conditions

35.5.1 Independent Groups: Kruskal–Wallis

The **Kruskal–Wallis test** extends rank-based comparison to three or more independent groups. It is the common nonparametric relative of a one-way independent-groups ANOVA.

```r
coping_data <- data.frame(
  strategy = factor(rep(c("Breathing", "Pep talk", "Statistics ritual"), each = 8)),
  panic = c(
    4, 5, 3, 6, 4, 5, 2, 5,
    6, 7, 5, 8, 6, 7, 5, 6,
    8, 9, 7, 10, 8, 9, 7, 9
  )
)

kruskal.test(panic ~ strategy, data = coping_data)
```

```
    Kruskal-Wallis rank sum test

data:  panic by strategy
Kruskal-Wallis chi-squared = 17.343, df = 2, p-value = 0.0001714
```

A significant result says that the groups do not all behave alike. It does not say which pairs differ. Follow-up pairwise Wilcoxon tests need a multiple-comparison correction:

```r
pairwise.wilcox.test(
  x = coping_data$panic,
  g = coping_data$strategy,
  p.adjust.method = "holm",
  exact = FALSE
)
```

```
    Pairwise comparisons using Wilcoxon rank sum test with continuity correction

data:  coping_data$panic and coping_data$strategy

                  Breathing Pep talk
Pep talk          0.0093    -
Statistics ritual 0.0026    0.0093

P value adjustment method: holm
```

Holm's method controls the familywise error rate and is generally preferable to applying the plain Bonferroni correction to every comparison with equal enthusiasm.

35.5.2 Repeated Conditions: Friedman

The **Friedman test** compares three or more repeated conditions. Each row below is one student; each column is something the student was asked to confront.

```
fear_matrix <- matrix(
  c(
    2, 6, 9,
    3, 7, 8,
    1, 5, 9,
    4, 6, 10,
    2, 7, 9,
    3, 5, 8,
    2, 6, 10,
    4, 7, 9
  ),
  nrow = 8,
  byrow = TRUE,
  dimnames = list(
    Student = paste0("S", 1:8),
    Stimulus = c("Duck", "Reviewer 2", "Exam")
  )
)

fear_matrix
```

```
       Stimulus
Student Duck Reviewer 2 Exam
     S1    2          6    9
     S2    3          7    8
     S3    1          5    9
     S4    4          6   10
     S5    2          7    9
     S6    3          5    8
     S7    2          6   10
```

```
     S8    4         7    9
```

```
friedman.test(fear_matrix)
```

```

    Friedman rank sum test

data:  fear_matrix
Friedman chi-squared = 16, df = 2, p-value = 0.0003355
```

The blocking variable is the student: every student contributes one score to every condition. Follow-up paired Wilcoxon tests can compare condition pairs, again with corrected p-values.

```
pair_names <- combn(colnames(fear_matrix), 2, simplify = FALSE)

raw_p <- vapply(pair_names, function(pair) {
  wilcox.test(
    fear_matrix[, pair[1]],
    fear_matrix[, pair[2]],
    paired = TRUE,
    exact = FALSE
  )$p.value
}, numeric(1))

data.frame(
  comparison = vapply(pair_names, paste, collapse = " vs. ", FUN.VALUE = character(1)),
  raw_p = raw_p,
  holm_p = p.adjust(raw_p, method = "holm")
)
```

```
           comparison      raw_p     holm_p
1 Duck vs. Reviewer 2 0.01298403 0.03895209
2       Duck vs. Exam 0.01356012 0.03895209
3 Reviewer 2 vs. Exam 0.01356012 0.03895209
```

35.6 What These Tests Do Not Fix

Nonparametric tests cannot rescue:

- Dependent observations analyzed as independent.
- A confounded design.
- A measure that does not represent the construct.
- Selective deletion of inconvenient participants.
- Twenty-seven unplanned analyses followed by a sudden claim that the only significant one was "the hypothesis all along."

They can also have lower power than a well-chosen parametric model when that model's assumptions are reasonable. More importantly, rank tests change the question by discarding some information about raw distances. Sometimes that is exactly what you want. Sometimes it is like buying noise-canceling headphones and then complaining that you cannot hear the fire alarm.

35.7 Reporting Nonparametric Results

Report enough information for a reader to understand both the result and the data:

1. Name the test and the design.
2. Report the test statistic, degrees of freedom when applicable, and p-value.
3. Describe each condition with counts, proportions, medians, and interquartile ranges as appropriate.
4. Include an effect size and confidence interval when a defensible one is available.
5. State how ties, zero differences, exact p-values, and multiple comparisons were handled.

The test is only one sentence in the argument. The data, design, effect size, uncertainty, and actual psychological question still have to show up for work.

35.8 The Short Story

- **Nonparametric does not mean assumption-free.** It means fewer assumptions, or different ones. Independence, sampling, and measurement still apply, and no test has ever been forgiven for a broken design.
- **These tests change the question.** Counts, signs, and ranks are not stand-ins for means. A rank test asks whether scores from one condition tend to be higher, which is a different sentence from "the means differ" and should be written as a different sentence.
- **Pick by design and question, not by a normality test at 2 a.m.** Normality tests find harmless departures in big samples and miss serious ones in small samples, which is exactly backwards from what you wanted.
- **The decision table above is the map.** Design and outcome type get you to the right test. They do not get you to a defensible study.
- **Three or more conditions means follow-ups, and follow-ups mean a family.** Kruskal–Wallis and Friedman are omnibus tests. Correct the pairwise comparisons; Holm is the sensible default.
- **Report medians and IQRs, the Hodges–Lehmann shift with its interval, and how you handled ties.** The shift is in the units you measured, which beats a standardized coefficient for everyone except a meta-analyst.

> ⚠ Nonparametric Does Not Mean Assumption-Free
>
> Rank and exact tests make different assumptions and often answer different questions from their parametric cousins. Choose one because it fits the design, scale, and scientific question, not because the data offended a normality test at 2:00 a.m.

36 Advanced: Bootstrapping, the Jackknife, and Resampling

36.1 Before the Bootstrap: Why We Needed Resampling

Many familiar statistics come with a theoretical sampling distribution. If the assumptions behave, we can use that distribution to calculate a standard error, confidence interval, or test statistic. Unfortunately, statistics such as medians, indirect effects, complicated effect sizes, and estimates from strange models may not arrive with a pleasant little formula attached.

The resampling solution is wonderfully rude: make the computer imitate repeated sampling. We use the data we observed to create many slightly different samples, calculate the statistic in every sample, and watch how much it moves. The movement gives us information about uncertainty.

This idea sounds obvious now because your laptop can perform thousands of resamples before you finish complaining about the assignment. It was less obvious when a computer occupied a room, demanded punch cards, and had less memory than a suspicious toaster.

> ⚠ Resampling Still Has Assumptions
>
> Resampling replaces some mathematical work with computation. It does not remove the need to understand the design, the estimator, or the assumptions. If you resample the wrong unit 10,000 times, you obtain a very stable estimate of the wrong uncertainty.

36.2 A Very Short History of People Deleting and Reusing Data

Resampling did not appear from a statistical volcano fully formed.

- **The jackknife:** Quenouille developed leave-one-out calculations to study bias. Tukey extended the method to variance estimation and gave it the name *jackknife*: a general-purpose statistical tool that could be carried around and used when needed.
- **The bootstrap:** Efron's 1979 paper was titled *Bootstrap Methods: Another Look at the Jackknife.* The name referred to pulling oneself up by one's bootstraps: using the sample itself to learn about repeated samples when the population is unavailable.
- **The computer changes everything:** The jackknife was designed when computation was expensive. The bootstrap became practical because computers could repeatedly resample and recalculate statistics. By 2003, Efron described the bootstrap as a method with roots in jackknife work on bias and variance, but with a much wider reach (Efron 2003).

The history matters because the jackknife and bootstrap are not two unrelated approaches. They answer the same basic question in different ways: **how sensitive is our estimate to the particular observations we happened to collect?**

36.3 The Jackknife: Remove One Person at a Time

Suppose our sample has n observations and the statistic from all observations is $\hat{\theta}$. The jackknife removes observation i, calculates the statistic again, and calls the result $\hat{\theta}_{(i)}$. After doing this for every observation, we have n leave-one-out estimates.

The average leave-one-out estimate is

$$\overline{\theta}_{(.)} = \frac{1}{n}\sum_{i=1}^{n}\hat{\theta}_{(i)}$$

The jackknife estimate of bias is

$$\widehat{Bias}_{jack} = (n-1)\left(\overline{\theta}_{(.)} - \hat{\theta}\right)$$

and the jackknife standard error is

$$SE_{jack} = \sqrt{\frac{n-1}{n}\sum_{i=1}^{n}\left(\hat{\theta}_{(i)} - \overline{\theta}_{(.)}\right)^2}$$

That standard error asks how much the estimate changes when each person temporarily vanishes. This is a statistical disappearance, not a proposal for managing annoying group members.

36.3.1 A Jackknife Example by Hand

Suppose 12 students report how many minutes they delayed before opening an assignment. One student reports 190 minutes because procrastination became a full-time position.

```
Delay.Small <- c(12, 18, 21, 23, 27, 31, 34, 38, 42, 49, 65, 190)

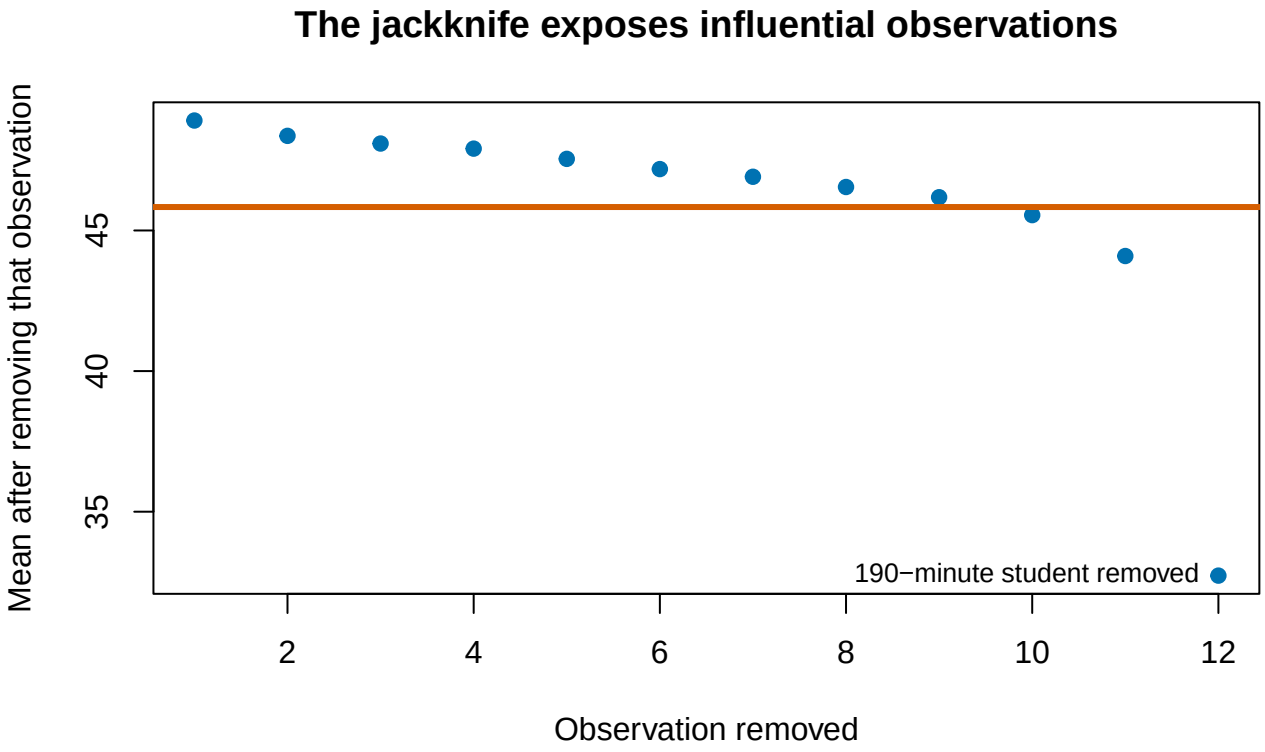
OriginalMean <- mean(Delay.Small)
JackMeans <- sapply(seq_along(Delay.Small), function(i) {
  mean(Delay.Small[-i])
})

JackMean <- mean(JackMeans)
JackBias <- (length(Delay.Small) - 1) * (JackMean - OriginalMean)
JackSE <- sqrt(
  (length(Delay.Small) - 1) / length(Delay.Small) *
    sum((JackMeans - JackMean)^2)
)

c(
  OriginalMean = OriginalMean,
  JackknifeBias = JackBias,
  JackknifeSE = JackSE
)
```

```
 OriginalMean JackknifeBias   JackknifeSE
     45.83333       0.00000      13.76471
```

```r
plot(
  seq_along(JackMeans), JackMeans,
  pch = 19, col = "#0072B2",
  xlab = "Observation removed",
  ylab = "Mean after removing that observation",
  main = "The jackknife exposes influential observations"
)
abline(h = OriginalMean, lwd = 3, col = "#D55E00")
# pos = 2 puts the label to the LEFT of the point. At pos = 3 it sat above the
# last point, which is hard against the right edge, and got clipped.
text(12, JackMeans[12], labels = "190-minute student removed", pos = 2, cex = .8)
```



The last leave-one-out estimate moves far more than the others. The jackknife therefore does more than generate a standard error. It also shows which observations are carrying the estimate around like exhausted undergraduate research assistants.

36.3.2 Pseudovalues and Bias Correction

The jackknife pseudovalue for observation i is

$$PV_i = n\hat{\theta} - (n-1)\hat{\theta}_{(i)}$$

The mean of the pseudovalues is the jackknife bias-corrected estimate:

$$\hat{\theta}_{jack} = n\hat{\theta} - (n-1)\overline{\theta}_{(\cdot)}$$

Pseudovalues transform the leave-one-out results so they behave more like observation-level contributions to the estimate. You will encounter them in older methods and in some specialized modern procedures. You do not need to place them under your pillow and hope they become intuitive overnight.

36.3.3 Where the Jackknife Struggles

The jackknife works especially well for smooth statistics such as means, regression coefficients, and correlations. It can behave badly for statistics that change abruptly when a case is removed. The sample median is the famous example (Singh and Xie 2008).

```
OriginalMedian <- median(Delay.Small)
JackMedians <- sapply(seq_along(Delay.Small), function(i) {
  median(Delay.Small[-i])
})

c(
  OriginalMedian = OriginalMedian,
  UniqueJackknifeMedians = length(unique(JackMedians)),
  JackknifeSE = sqrt(
    (length(Delay.Small) - 1) / length(Delay.Small) *
      sum((JackMedians - mean(JackMedians))^2)
  )
)
```

```
        OriginalMedian UniqueJackknifeMedians            JackknifeSE
             32.500000               2.000000               4.974937
```

Removing one observation may leave the median unchanged or make it jump between only a few values. That choppy behavior can make the leave-one-out approximation poor. The bootstrap usually gives the median more room to wander because each resample can omit several observations and repeat several others.

36.4 The Bootstrap: Reuse the Sample with Replacement

Bootstrapping asks what would happen if we repeatedly sampled from a population resembling the one we observed. Because we do not possess the population, the **empirical distribution** of the sample stands in for it. Each observed case receives probability $1/n$, and we draw a new sample of size n **with replacement** (Efron and Tibshirani 1993; Efron 2003).

With replacement means that after selecting a participant, we return that participant before the next draw. Some participants appear several times; others disappear entirely. This is statistically acceptable and ethically preferable to cloning the undergraduates.

36.4.1 The Plug-In Principle

Let F represent the unknown population distribution and let $\widehat{F}$ represent the empirical distribution created from our sample. If the population quantity is

$$\theta = t(F)$$

we estimate it by plugging in the empirical distribution:

$$\hat{\theta} = t(\widehat{F})$$

The bootstrap then draws $F_1^*, F_2^*, \ldots, F_B^*$ from $\widehat{F}$ and calculates

$$\hat{\theta}_b^* = t(F_b^*)$$

for each bootstrap sample b. This is the **plug-in principle**: replace the unknown population with the best estimate we have of it, then study what repeated sampling from that estimate does (Efron 2003).

> **What the Bootstrap Treats as the Population**
>
> The observed sample temporarily plays the role of the population. If the sample is biased, excludes important groups, or was collected with a design that we ignore, bootstrapping reproduces that problem thousands of times with impressive efficiency.

36.4.2 Why About 63.2% of Cases Appear

In one bootstrap sample, the probability that a particular case is never selected is

$$\left(1 - \frac{1}{n}\right)^n$$

As n grows, this approaches $e^{-1} \approx .368$. Therefore, a typical bootstrap sample contains about $1 - .368 = .632$, or 63.2%, of the original cases at least once. The remaining positions are duplicates. Nothing is missing from R; repetition is the entire point.

36.5 Bootstrapping a Skewed Median

Suppose students report how many minutes they procrastinated before opening an assignment. The distribution is strongly right-skewed because several students began procrastinating during the previous semester.

```
set.seed(343)
Delay <- round(rlnorm(55, meanlog = log(35), sdlog = .8))
median(Delay)
```

```
[1] 30
```

```r
mean(Delay)
```

```
[1] 40.61818
```

We can show one bootstrap sample before asking R to perform the process thousands of times.

```r
set.seed(344)
OneBootstrapSample <- sample(
  Delay,
  size = length(Delay),
  replace = TRUE
)

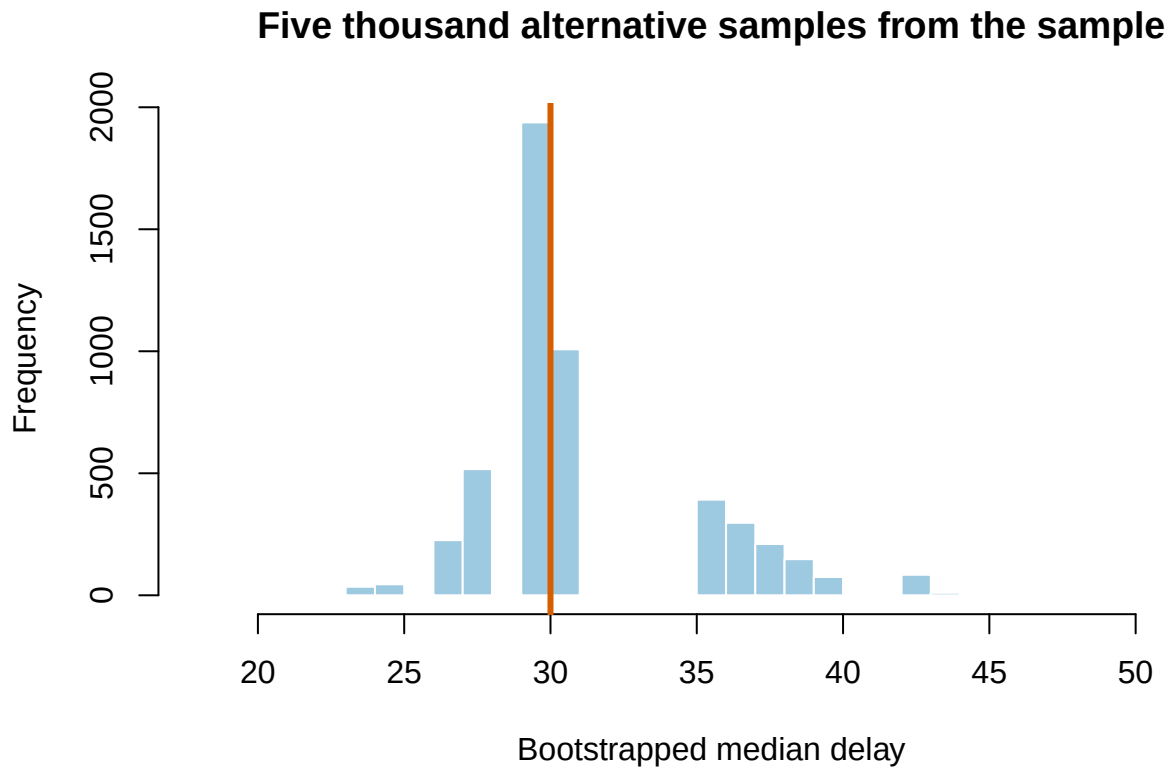
c(
  OriginalMedian = median(Delay),
  BootstrapMedian = median(OneBootstrapSample),
  UniqueCasesRepresented = length(unique(OneBootstrapSample))
)
```

```
        OriginalMedian         BootstrapMedian UniqueCasesRepresented
                    30                      30                      24
```

Now repeat the resampling process $B = 5{,}000$ times.

```r
set.seed(345)
B <- 5000
BootMedian <- replicate(
  B,
  median(sample(Delay, size = length(Delay), replace = TRUE))
)

hist(
  BootMedian, breaks = 30,
  col = "#9ECAE1", border = "white",
  xlab = "Bootstrapped median delay",
  main = "Five thousand alternative samples from the sample"
)
abline(v = median(Delay), col = "#D55E00", lwd = 3)
```



The bootstrap standard error is the standard deviation of the bootstrap estimates:

$$SE_{boot} = \sqrt{\frac{1}{B-1}\sum_{b=1}^{B}(\hat{\theta}_b^* - \overline{\theta}^*)^2}$$

The bootstrap estimate of bias is

$$\widehat{Bias}_{boot} = \overline{\theta}^* - \hat{\theta}$$

```
BootSE <- sd(BootMedian)
BootBias <- mean(BootMedian) - median(Delay)

c(
  ObservedMedian = median(Delay),
  BootstrapBias = BootBias,
  BootstrapSE = BootSE
)
```

```
ObservedMedian  BootstrapBias    BootstrapSE
     30.000000       1.661800       3.846814
```

36.6 Bootstrap Confidence Intervals: More Than One Flavor

There is no single thing called *the bootstrap confidence interval*. Several methods use the bootstrap distribution differently, and they do not have identical accuracy.

36.6.1 Normal Interval

The normal interval uses the bootstrap standard error in a familiar symmetric interval:

$$\hat{\theta} \pm z_{1-\alpha/2} SE_{boot}$$

It is simple, but symmetry may be a bad approximation when the bootstrap distribution is skewed.

36.6.2 Percentile Interval

The percentile interval uses the bootstrap quantiles directly:

$$[q^*_{\alpha/2}, q^*_{1-\alpha/2}]$$

For a 95% interval, take the 2.5th and 97.5th percentiles. This is intuitive, but intuition and good coverage are not always roommates.

36.6.3 Basic Interval

The basic interval reflects the bootstrap errors around the original estimate:

$$[2\hat{\theta} - q^*_{.975}, 2\hat{\theta} - q^*_{.025}]$$

If the bootstrap estimates tend to fall above the observed estimate, the basic interval reflects that pattern below it.

36.6.4 Studentized or Bootstrap-t Interval

The studentized method bootstraps a standardized pivot:

$$t^*_b = \frac{\hat{\theta}^*_b - \hat{\theta}}{SE^*_b}$$

It then uses quantiles of the t^*_b distribution to construct the interval. This can achieve better coverage, but every bootstrap sample needs its own standard-error estimate. That may require a second bootstrap inside the first bootstrap, because apparently the Probability Gods enjoy nesting dolls.

36.6.5 BCa Interval

The **bias-corrected and accelerated** interval adjusts the percentile cutoffs using two quantities:

- z_0, which describes median bias in the bootstrap distribution.
- a, the acceleration constant, which describes how rapidly the standard error changes with the true parameter.

The acceleration term is commonly estimated from jackknife values. The jackknife therefore returns inside the bootstrap, wearing a fake mustache and pretending it never left. BCa intervals are second-order accurate under suitable conditions and often useful for skewed statistics, but they can be unstable when the sample is tiny or the statistic is poorly behaved (Efron 2003).

```
median_statistic <- function(data, indices) {
  median(data[indices])
}

set.seed(346)
BootObject <- boot::boot(
  data = Delay,
  statistic = median_statistic,
  R = 5000
)

boot::boot.ci(
  BootObject,
  type = c("norm", "basic", "perc", "bca")
)
```

```
BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
Based on 5000 bootstrap replicates

CALL :
boot::boot.ci(boot.out = BootObject, type = c("norm", "basic",
    "perc", "bca"))

Intervals :
Level      Normal              Basic
95%   (20.73, 35.81 )   (20.00, 33.00 )

Level     Percentile            BCa
95%   (27, 40 )   (21, 30 )
Calculations and Intervals on Original Scale
Warning : BCa Intervals used Extreme Quantiles
Some BCa intervals may be unstable
```

> **!** Coverage Has Two Tails
>
> A nominal 95% interval should miss the true value about 2.5% of the time below and 2.5% above across repeated samples. An interval that misses 5% entirely on one side still has 95% total coverage, but it is systematically off-center. Better bootstrap intervals try to improve both overall coverage and tail balance (Efron 2003).

> ⚠ A 95% Confidence Interval Is Not a 95% Probability Statement About This Finished Interval
>
> The long-run-procedure interpretation applies to bootstrap intervals too, and the evidence says almost nobody holds onto it: see the Standard Error chapter for the depressing Hoekstra et al. numbers (Hoekstra et al. 2014). Reread that callout if the word *confidence* has started whispering probability statements to you.

36.7 Bootstrapping a Regression Slope

For independent observations, a **case bootstrap** or **pairs bootstrap** resamples rows. Each resampled participant keeps their outcome and all predictors attached.

```
set.seed(347)
RegressionData <- data.frame(
  Sleep = runif(90, 3, 9)
)
RegressionData$Memory <-
  42 + 4 * RegressionData$Sleep +
  rnorm(90, 0, RegressionData$Sleep^3 / 30)

OriginalModel <- lm(Memory ~ Sleep, data = RegressionData)
OriginalSlope <- coef(OriginalModel)["Sleep"]

BootSlopes <- replicate(5000, {
  rows <- sample(seq_len(nrow(RegressionData)), replace = TRUE)
  coef(lm(Memory ~ Sleep, data = RegressionData[rows, ]))["Sleep"]
})

c(
  OriginalSlope = OriginalSlope,
  BootstrapSE = sd(BootSlopes),
  quantile(BootSlopes, c(.025, .975))
)
```

```
OriginalSlope.Sleep         BootstrapSE               2.5%              97.5%
          3.9546071           0.8179124           2.2972744           5.5003347
```

Look closely at that `rnorm()` call before we go on. The standard deviation of the noise is `Sleep^3 / 30`, which is about 1 point for a student who slept three hours and about 24 points for one who slept nine. **I built the heteroscedasticity in on purpose**, and it is aggressive on purpose too. Everything that follows is a demonstration rather than a catalog: watch what that one design choice does to each flavor of bootstrap.

The row bootstrap is not the only regression bootstrap.

36.7.1 Case or Pairs Bootstrap

Resample complete rows (Y_i, X_i). This preserves the relationship among all variables within each case and allows the observed predictor values to vary across bootstrap samples. It is a defensible default when cases were independently sampled from a population.

36.7.2 Residual Bootstrap

Fit the model, hold X fixed, resample centered residuals, and create

$$Y_i^* = \widehat{Y}_i + e_i^*$$

This treats the design matrix as fixed and assumes the residuals are exchangeable. Ordinary residual resampling is therefore questionable under heteroscedasticity because small-variance and large-variance residuals are thrown into one communal bucket.

```
set.seed(348)
FittedValues <- fitted(OriginalModel)
CenteredResiduals <- residuals(OriginalModel) - mean(residuals(OriginalModel))

ResidualSlopes <- replicate(3000, {
  YStar <- FittedValues + sample(CenteredResiduals, replace = TRUE)
  coef(lm(YStar ~ RegressionData$Sleep))[2]
})

sd(ResidualSlopes)
```

```
[1] 0.6773271
```

36.7.3 Wild Bootstrap

A wild bootstrap keeps each residual attached to its predictor location but multiplies it by a random weight with mean zero and variance one:

$$Y_i^* = \widehat{Y}_i + e_i w_i$$

This preserves the way residual size changes across X better than shuffling residuals. Wild-bootstrap ideas grew from work on resampling regression under heteroscedasticity (Wu 1986).

```
set.seed(349)
WildSlopes <- replicate(3000, {
  Weights <- sample(c(-1, 1), nrow(RegressionData), replace = TRUE)
  YStar <- FittedValues + residuals(OriginalModel) * Weights
  coef(lm(YStar ~ RegressionData$Sleep))[2]
})

sd(WildSlopes)
```

```
[1] 0.8277079
```

36.7.4 The Three Flavors, Side by Side

Three standard errors for the same slope in the same data. Put them next to the ordinary `lm()` standard error, and next to the HC3 sandwich estimator from the diagnostics chapter, which is designed for exactly this disease:

```
# sandwich: heteroscedasticity-robust covariance. See the Advanced Regression
# Diagnostics chapter, where HC3 is derived.
library(sandwich)

round(c(
  `Ordinary lm() SE`   = summary(OriginalModel)$coefficients["Sleep", "Std. Error"],
  `HC3 sandwich SE`    = sqrt(vcovHC(OriginalModel, type = "HC3")["Sleep", "Sleep"]),
  `Case bootstrap`     = sd(BootSlopes),
  `Residual bootstrap` = sd(ResidualSlopes),
  `Wild bootstrap`     = sd(WildSlopes)
), 3)
```

```
  Ordinary lm() SE    HC3 sandwich SE     Case bootstrap Residual bootstrap
             0.678              0.851              0.818              0.677
    Wild bootstrap
             0.828
```

The three bootstraps do not disagree by accident, and they do not disagree at random. **The case and wild bootstraps land next to HC3. The residual bootstrap lands on the ordinary `lm()` standard error**: 0.677 against 0.678. Three thousand resamples bought a number we already had, which is a slightly humiliating result for a method whose whole job was to improve on it.

That is the communal bucket doing exactly what we said it would do. Resampling centered residuals and reattaching them to arbitrary rows destroys the link between where a case sits on X and how noisy it is; every case ends up with the same average noise, which is the homoscedasticity assumption sneaking back in through a side door. The case bootstrap keeps each residual welded to its own row because it resamples whole rows. The wild bootstrap keeps it welded by flipping the residual's sign in place instead of moving it.

So the flavor is not a stylistic preference. Under heteroscedasticity, one of these three will quietly report the wrong uncertainty, and it will do it with exactly the same calm confidence as the other two.

36.7.5 Parametric Bootstrap

A parametric bootstrap simulates new outcomes from the fitted probability model. For a Gaussian regression, we might simulate

$$Y_i^* \sim N(\widehat{Y}_i, \hat{\sigma}^2)$$

For logistic regression, we might simulate binary outcomes using each fitted probability. This method can be efficient when the model is correct. If the model is wrong, the simulation faithfully reenacts the wrong model thousands of times.

36.8 Preserve the Dependence Structure

If observations are repeated, nested, clustered, or ordered in time, resampling individual rows usually destroys the design.

- **Cluster bootstrap:** Resample whole participants, classrooms, families, clinics, or other independently sampled clusters.
- **Multilevel bootstrap:** Resample at more than one level when the sampling process justifies it.
- **Block bootstrap:** Resample adjacent blocks of a time series so short-range dependence remains intact (Singh and Xie 2008).
- **Model-based bootstrap:** Simulate from a fitted mixed model or time-series model while preserving its assumed structure.

! Resample the Unit That Was Sampled

If trials are nested within people, treating trials as independent resampling units manufactures a fictional sample size. If entire classrooms were sampled, resampling individual students pretends the classroom did not exist. Preserve the design. The bootstrap is flexible, not psychic.

36.9 Bootstrap Estimation Is Not Automatically a Hypothesis Test

Resampling the observed data estimates uncertainty around the observed statistic. A hypothesis test asks what results would occur under a null model. Those are different jobs.

To test a null hypothesis, we may need to:

- Center outcomes so the null value is true.
- Permute group labels under exchangeability.
- Simulate from a restricted model that imposes the null.
- Compare full and reduced models using a parametric bootstrap.

⚠ A Bootstrap Distribution Is Not Automatically a Null Distribution

Dividing a bootstrap estimate by its bootstrap standard error and feeding the result to an ordinary t table is not a universal bootstrap test. First decide what must be true under the null, then create resamples in which that null is actually true.

36.10 How Many Replications?

A few hundred replications may be enough to teach the idea or debug code. Standard errors commonly need at least a few thousand. Confidence-interval endpoints depend on tail quantiles and often need more. Run enough replications that repeating the analysis with a different seed does not change the reported conclusion or the meaningful digits.

The number of bootstrap replications B controls **Monte Carlo error** in the calculation. It does not increase the original sample size. More replications do not repair a biased sample, weak measurement, or a resampling scheme that ignores the design. Ten thousand wrong resamples are wrong with excellent numerical stability.

36.11 The Bootstrap Is Not Magic

The bootstrap is useful when:

- A theoretical sampling distribution is hard to derive.
- A statistic has a skewed or otherwise awkward sampling distribution.
- We need standard errors or intervals for medians, indirect effects, complex effect sizes, or model predictions.
- We want to see how sensitive an estimate is to sampling variation.

The bootstrap cannot manufacture information that was not in the original data. It struggles when:

- The sample is extremely small.
- Rare but important parts of the population are absent.
- The estimator sits on a boundary or behaves discontinuously.
- The resampling plan does not preserve clustering or time dependence.
- The model is badly misspecified.

Efron called attention to both the power and the limits of the plug-in principle: the empirical distribution is a powerful stand-in for the population, but it can inherit and sometimes amplify the sample's pathologies (Efron 2003). In technical language, garbage in, garbage out. In Alex language, the computer cannot rescue a study that was thrown into the volcano before the analysis began.

36.12 The Short Story

- **The jackknife is leave-one-out.** Remove observation i, recompute, repeat. It gives you bias, a standard error, and a picture of which cases are carrying the estimate. It fails on statistics that jump when a case is removed, and the median is the standing example.
- **The bootstrap is resample-with-replacement**, treating the sample as a stand-in for the population. That buys you a sampling distribution for statistics that never came with a formula.
- **Resample the unit that was sampled.** Rows if rows were sampled, clusters if clusters were sampled, blocks if the data are a time series. Getting this wrong manufactures a fictional sample size.
- **Pick the interval flavor to fit the estimator.** Percentile is intuitive, BCa is usually better, normal assumes a symmetry you may not have.
- **Pick the regression flavor to fit the errors.** Under heteroscedasticity the residual bootstrap will hand you back the ordinary standard error you were trying to escape. Case and wild will not.
- B **controls Monte Carlo error, not information.** More replications stabilize the calculation. They do not add participants, fix a biased sample, or repair a resampling plan that ignores the design.
- **Estimation is not null-hypothesis testing.** A bootstrap distribution is centered on your estimate, not on the null. To test, first make the null true, then resample.

Part VI

When R Inevitably Betrays You

37 Reporting Models Without Making the Reader Solve a Puzzle

37.1 The Analysis Is Not Finished When R Stops Printing

A results section should let a reader reconstruct the statistical argument without opening your R session, reading your mind, or contacting the undergraduate who disappeared with the codebook.

Report:

- What was analyzed.
- How variables were coded.
- Which model was fitted and why.
- Estimates in interpretable units.
- Uncertainty.
- Effect size or model fit.
- Diagnostics and consequential deviations.
- What the result supports and what it does not.

> **!** Lead with the Scientific Result
>
> "A regression was significant" is about software output. "Each additional hour of sleep predicted about four more memory points" is about the research question. Start with the second sentence.

37.2 One Reproducible Example

```
set.seed(343)
n <- 180
ReportData <- data.frame(
  Sleep = runif(n, 3, 9),
  Caffeine = pmax(rnorm(n, 180, 85), 0),
  Condition = factor(sample(c("Control", "Training"), n, replace = TRUE))
)

ReportData$Memory <-
  38 + 3.8 * ReportData$Sleep -
  .018 * ReportData$Caffeine +
  5.5 * (ReportData$Condition == "Training") +
  rnorm(n, 0, 7)

ReportModel <- lm(Memory ~ Sleep + Caffeine + Condition, data = ReportData)
```

37.3 Descriptives Before Models

Describe the sample and variables needed to understand the model. Do not create a 47-column Table 1 containing every variable collected since the lab opened.

```
Descriptives <- data.frame(
  Variable = c("Memory", "Sleep", "Caffeine"),
  Mean = c(mean(ReportData$Memory), mean(ReportData$Sleep), mean(ReportData$Caffeine)),
  SD = c(sd(ReportData$Memory), sd(ReportData$Sleep), sd(ReportData$Caffeine)),
  Minimum = c(min(ReportData$Memory), min(ReportData$Sleep), min(ReportData$Caffeine)),
  Maximum = c(max(ReportData$Memory), max(ReportData$Sleep), max(ReportData$Caffeine))
)

knitr::kable(Descriptives, digits = 2, caption = "Descriptive statistics")
```

Table 37.1: Descriptive statistics

Variable	Mean	SD	Minimum	Maximum
Memory	59.41	9.98	35.72	87.23
Sleep	5.81	1.71	3.03	8.89
Caffeine	187.00	77.65	0.00	385.28

37.4 Extract Results from the Model Object

Never type changing results manually into a table when R can extract them. Manual transcription creates opportunities to report the coefficient from Model 4, the p-value from Model 6, and the confidence interval from a model no one remembers fitting.

```
CoefficientTable <- broom::tidy(ReportModel, conf.int = TRUE)
knitr::kable(
  CoefficientTable,
  digits = 3,
  caption = "Regression predicting memory performance"
)
```

Table 37.2: Regression predicting memory performance

term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	38.130	2.487	15.330	0.000	33.221	43.038
Sleep	3.600	0.319	11.283	0.000	2.970	4.229
Caffeine	-0.013	0.007	-1.845	0.067	-0.027	0.001
ConditionTraining	5.064	1.092	4.637	0.000	2.909	7.219

For mixed models, use `broom.mixed::tidy()` because fixed effects, random-effect parameters, and model components require more supervision.

37.4.1 The Lazy One-Line Version

`broom::tidy()` plus `kable()` is the version you should understand, because it shows you that a table is just a data frame you built on purpose. When you need a publication-ready table right now, `sjPlot` (already the plotting package for this course) does the whole thing in one line:

```
# sjPlot: model tables and plots. tab_model() writes HTML, so this chunk only
# runs for the web version of the book.
library(sjPlot)

tab_model(ReportModel)
```

It gives you estimates, confidence intervals, p-values, R^2, and N without you typing a single number. Use it. Just remember that a beautiful table of a bad model is still a bad model: the checklist in the next section governs what belongs in the table, no matter who typeset it.

37.5 What Belongs in a Regression Table?

At minimum:

- Predictor names readers understand.
- Unstandardized estimates.
- Standard errors or confidence intervals.
- Test statistics and p-values when required.
- Sample size.
- Model R^2 and adjusted R^2 for ordinary regression.

Standardized coefficients may supplement, not automatically replace, coefficients in meaningful original units.

> **Asterisks Are Not a Results Section**
>
> Stars can help scan a table. They do not describe magnitude, uncertainty, practical importance, or whether the model made sense. Never make the reader infer the entire conclusion from celestial punctuation.

37.6 APA Rules You Lose Points For

Every chapter in this book has shown you an APA write-up. None of them stopped to say what the typographic rules actually are, and those rules are what you get points taken off for. Here they are in one place.

Rule	Wrong	Right
Statistical symbols are italic	t(176) = 11.28	$t(176) = 11.28$
No leading zero when the number cannot exceed 1	$p = 0.067$, $r = 0.42$	$p = .067$, $r = .42$
Keep the leading zero when it can	$b = .60$, $M = .48$	$b = 0.60$, $M = 0.48$
Two decimals for most statistics	$b = 3.5997$	$b = 3.60$
Exact p, two or three decimals, with $p <$ .001 as the floor	$p < .05$	$p = .067$, or $p < .001$

Rule	Wrong	Right
Degrees of freedom in parentheses, attached to the symbol	$F = 53.83$, $df = 3, 176$	$F(3, 176) = 53.83$
Spaces around =, < and >	p=.067	$p = .067$
Confidence intervals in square brackets, comma between	CI = (2.97-4.23)	95% CI [2.97, 4.23]
Sample size travels with the statistic that needs it	$\chi^2 = 16.08$	$\chi^2(1, N = 1379) = 16.08$

The italic rule is the one people lose points on, and the logic is simpler than it looks: **a Latin letter standing in for a statistic leans; a Greek letter does not.** So M, SD, SE, N, n, p, r, t, F, b, d, df, OR and R^2 are all italic, while α, β, χ^2 and σ stay upright. The one Latin exception worth memorizing is **CI**, which is an abbreviation of two English words rather than a symbol, and stays upright.

Inside math it is handled for you: write `$t(98)$` or `$F(1, 97)$` and LaTeX italicizes the letter itself. Outside math you have to ask, which is why this book writes `*t*-test` rather than `t-test`.

Every write-up example in this book follows these rules. When your poster does not, the TA will find it, and the TA will be correct.

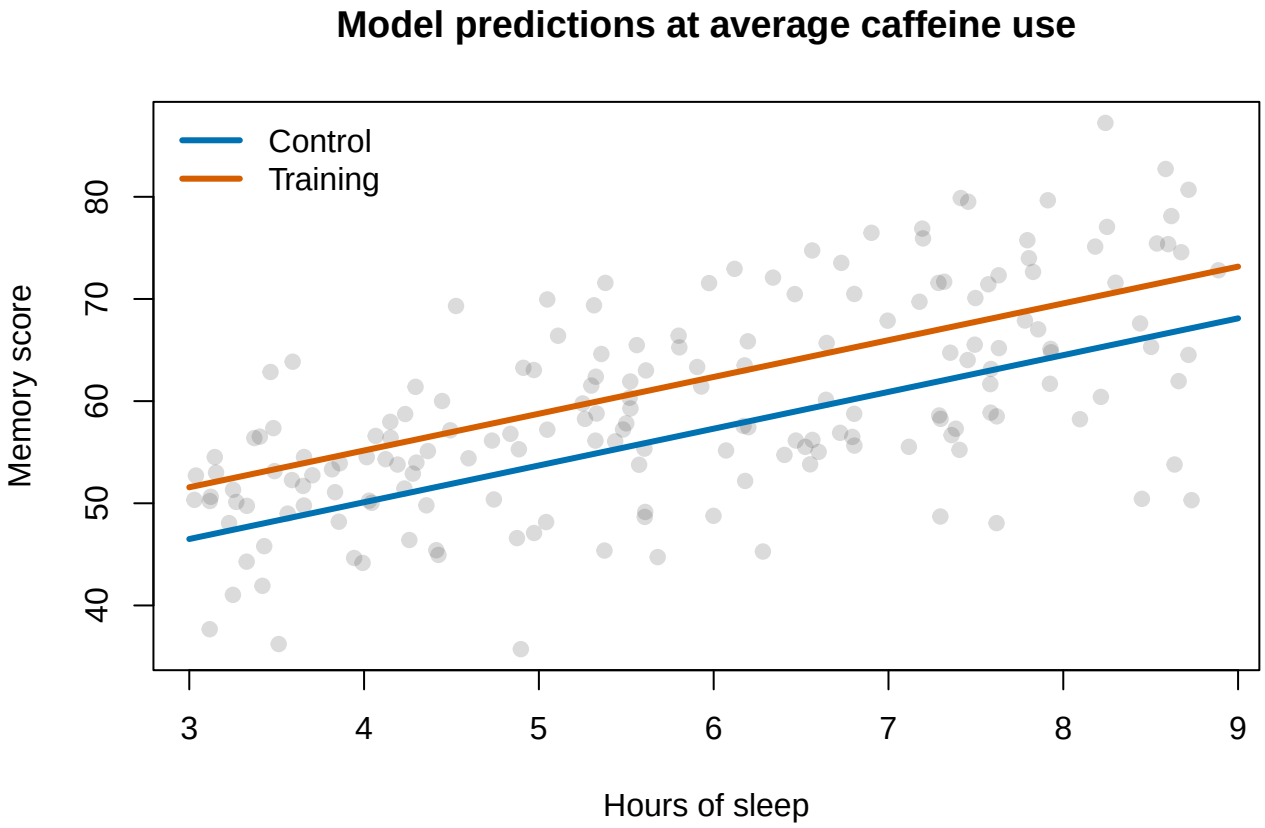
37.7 Plot Model-Based Predictions

```
PredictionData <- expand.grid(
  Sleep = seq(3, 9, length.out = 100),
  Caffeine = mean(ReportData$Caffeine),
  Condition = levels(ReportData$Condition)
)

PredictionData$Predicted <- predict(ReportModel, newdata = PredictionData)

plot(
  ReportData$Sleep, ReportData$Memory,
  pch = 19, col = rgb(.2, .2, .2, .18),
  xlab = "Hours of sleep", ylab = "Memory score",
  main = "Model predictions at average caffeine use"
)

colors <- c("#0072B2", "#D55E00")
for (i in seq_along(levels(ReportData$Condition))) {
  group <- levels(ReportData$Condition)[i]
  d <- PredictionData[PredictionData$Condition == group, ]
  lines(d$Sleep, d$Predicted, lwd = 3, col = colors[i])
}
legend("topleft", legend = levels(ReportData$Condition),
       col = colors, lwd = 3, bty = "n")
```



Figures should show the comparison the model actually tested. Label axes in meaningful units, show uncertainty when it matters, and do not use a bar chart to hide a continuous distribution.

37.8 Example Write-Up

> A multiple regression examined whether sleep, caffeine consumption, and training condition predicted memory scores, $R^2 = .48$, adjusted $R^2 = .47$, $F(3, 176) = 53.83$, $p < .001$. Holding caffeine and condition constant, each additional hour of sleep predicted an estimated 3.60-point increase in memory performance, 95% CI [2.97, 4.23], $t(176) = 11.28$, $p < .001$. Training participants scored 5.06 points higher than control participants at the same sleep and caffeine values, 95% CI [2.91, 7.22], $t(176) = 4.64$, $p < .001$. Caffeine showed a small negative adjusted association that was not statistically distinguishable from zero, b = -0.013 points per milligram, or about 1.3 points lower per 100 mg, 95% CI [-0.027, 0.001], $t(176)$ = -1.84, p = .067. Unstandardized coefficients and 95% confidence intervals appear in Table 2.

Every number in that paragraph was computed inline from `ReportModel`. Nothing was typed. If the model changes, the paragraph changes with it, which is the entire trick.

Underneath the numbers there is a template you can reuse for any coefficient: **claim in units, then the statistics in parentheses, then what it does and does not support.** Interpret the coefficients that answer the research question rather than reciting every cell of the table aloud.

Notice what the caffeine sentence does. The estimate is negative, and it is also compatible with zero, so the sentence says both things and does not promote a non-significant coefficient to a finding by describing it confidently. That sentence is the one reviewers read most carefully.

A real write-up also ends with a sentence about diagnostics: what you checked, and whether anything you found changed the interpretation. See the Advanced Regression Diagnostics chapter for what belongs in it.

37.9 Reproducibility Checklist

- Use a script or Quarto document.
- Set random seeds for simulations.
- Record package versions when reproducibility is important.
- Keep raw data unchanged.
- Document exclusions and transformations.
- Generate tables and figures from fitted objects.
- Store enough code to reproduce every reported number.

> **Future You Is an Unreliable Collaborator**
>
> Future You will not remember why `Model.Final.2.REAL.useThisOne` excludes participants 14 and 88. Write the reason now. Future You is busy, confused, and probably blaming Past You with considerable justification.

37.10 The Short Story

- **Lead with the scientific result, not the software.** "Each additional hour of sleep predicted about 3.6 more memory points" is a finding. "The regression was significant" is a status update.
- **Report the estimate, its uncertainty, an effect size or model fit, and the limits.** A coefficient with no interval is half a result.
- **Generate every number from the fitted object.** Inline code and `broom::tidy()` cost less than one round of manual transcription, and they cannot report Model 4's coefficient beside Model 6's p-value.
- **Say what the result does not support.** A coefficient compatible with zero gets described as compatible with zero.
- **Follow the formatting rules above.** Italics, leading zeros, exact p, df in parentheses. They are free points.
- **Leave a reproducible trail**, because Future You is an unreliable collaborator who will not remember why participant 88 is missing.

38 R Help and Review: Tidyverse Without Tears

38.1 Why Is This at the End of the Book?

This is not a second statistics course hiding in the closet. It is a compact reference for the R operations you will repeatedly forget five minutes after learning them. That is normal. Professional R users also look things up constantly; they have merely developed more dignified facial expressions while doing it.

The **tidyverse** is a collection of R packages designed around consistent ways of storing, transforming, summarizing, and plotting data. We will focus on two:

- `dplyr` for manipulating data.
- `ggplot2` for making graphs.

Install them once on a computer:

```
install.packages(c("dplyr", "ggplot2"))
```

Load them again whenever you begin a new R session. This chunk really does run, and it is what makes every other chunk in the chapter work:

```
library(dplyr)
library(ggplot2)
```

Installing is moving the furniture into your apartment. Loading is taking the furniture out of the closet so you can sit on it. Do not reinstall the sofa every morning.

38.2 Caffeine Study

We will simulate the data a completely fictional study, where 160 students consume different amounts of caffeine. We record sleep, anxiety, hatred of group projects, and whether the student begins hearing colors. No students actually consumed these doses. The IRB has stopped returning our calls for unrelated reasons.

```
set.seed(343)

n_students <- 160

lab_students <- data.frame(
  student_id = seq_len(n_students),
  caffeine_mg = sample(c(0, 100, 200, 400), n_students, replace = TRUE),
  sleep_hours = round(pmin(10, pmax(2, rnorm(n_students, 6.2, 1.4))), 1),
  group_project_hatred = sample(1:10, n_students, replace = TRUE)
```

```
)

color_probability <- plogis(
  -4 + .010 * lab_students$caffeine_mg - .20 * (lab_students$sleep_hours - 6)
)

lab_students$hears_colors <- factor(
  ifelse(rbinom(n_students, 1, color_probability) == 1, "Yes", "No"),
  levels = c("No", "Yes")
)

lab_students$anxiety <- round(pmin(
  10,
  pmax(
    0,
    4 + .009 * lab_students$caffeine_mg -
      .30 * lab_students$sleep_hours +
      .12 * lab_students$group_project_hatred +
      rnorm(n_students, 0, 1.2)
  )
), 1)

head(lab_students)
```

```
  student_id caffeine_mg sleep_hours group_project_hatred hears_colors anxiety
1          1           0         4.8                     6           No     1.8
2          2         400         7.1                     9           No     6.2
3          3         200         3.4                     5           No     2.5
4          4         100         6.9                     6           No     1.9
5          5         400         7.0                     1          Yes     5.3
6          6         400         5.6                     5           No     4.7
```

Because `set.seed(343)` fixes the random-number sequence, everyone gets the same fictional students. The Probability Gods are still fickle, but for this example we temporarily confiscated their dice.

38.3 Data Frames

A data frame is a rectangular collection of variables:

- Each **row** is one observational unit, here one student.
- Each **column** is one variable.
- Each **cell** is one student's value on one variable.

This structure is often called **tidy data** when each variable has its own column, each observation has its own row, and each value has its own cell.

Useful first inspections include:

```
dim(lab_students)
```

```
[1] 160   6
```

```
names(lab_students)
```

```
[1] "student_id"           "caffeine_mg"          "sleep_hours"
[4] "group_project_hatred" "hears_colors"         "anxiety"
```

```
str(lab_students)
```

```
'data.frame':   160 obs. of  6 variables:
 $ student_id          : int  1 2 3 4 5 6 7 8 9 10 ...
 $ caffeine_mg         : num  0 400 200 100 400 400 400 100 100 200 ...
 $ sleep_hours         : num  4.8 7.1 3.4 6.9 7 5.6 8.4 5.6 3.8 6.5 ...
 $ group_project_hatred: int  6 9 5 6 1 5 1 8 10 6 ...
 $ hears_colors        : Factor w/ 2 levels "No","Yes": 1 1 1 1 2 1 1 1 1 1 ...
 $ anxiety             : num  1.8 6.2 2.5 1.9 5.3 4.7 6.5 5.4 4.5 5.8 ...
```

```
summary(lab_students)
```

```
   student_id      caffeine_mg     sleep_hours    group_project_hatred
 Min.   :  1.00   Min.   :  0.0   Min.   :2.000   Min.   : 1.0
 1st Qu.: 40.75   1st Qu.:100.0   1st Qu.:5.600   1st Qu.: 3.0
 Median : 80.50   Median :150.0   Median :6.400   Median : 5.0
 Mean   : 80.50   Mean   :176.2   Mean   :6.379   Mean   : 5.5
 3rd Qu.:120.25   3rd Qu.:200.0   3rd Qu.:7.200   3rd Qu.: 8.0
 Max.   :160.00   Max.   :400.0   Max.   :9.600   Max.   :10.0
 hears_colors    anxiety
 No :144      Min.   :0.000
 Yes: 16      1st Qu.:2.875
              Median :4.200
              Mean   :4.247
              3rd Qu.:5.500
              Max.   :8.500
```

`str()` is especially useful. It reveals whether R thinks a variable is numeric, character, or a factor before you spend 45 minutes accusing the regression model of personal betrayal.

38.4 The Native Pipe

Modern R uses `|>` to pass the result of one step into the next step:

```
lab_students |>
  head(3)
```

```
  student_id caffeine_mg sleep_hours group_project_hatred hears_colors anxiety
1          1           0         4.8                    6           No     1.8
2          2         400         7.1                    9           No     6.2
3          3         200         3.4                    5           No     2.5
```

Read it as **"and then."**

Start with `lab_students`, **and then** keep highly caffeinated students, **and then** select the variables we want.

```
lab_students |>
  filter(caffeine_mg >= 200) |>
  select(student_id, caffeine_mg, anxiety, hears_colors) |>
  head()
```

```
  student_id caffeine_mg anxiety hears_colors
1          2         400     6.2           No
2          3         200     2.5           No
3          5         400     5.3          Yes
4          6         400     4.7           No
5          7         400     6.5           No
6         10         200     5.8           No
```

Older R code often uses `%>%`, the pipe from `magrittr` and `dplyr`. It is still valid, you will see it in published code (including my own), and the ancient inscriptions found beneath our Behavioral Science Building at UIC close to where we keep the chained demons. For everyday `dplyr` work the two mean the same thing, so when you find `%>%` on Stack Overflow you can read it as `|>` and carry on. This book uses `|>` everywhere, including the earlier chapters, which were swept so you would never have to wonder whether the difference mattered.

38.5 The Five Verbs You Will Use Constantly

38.5.1 `select()`: Keep Columns

```
small_data <- lab_students |>
  select(student_id, caffeine_mg, sleep_hours, anxiety)

head(small_data)
```

```
  student_id caffeine_mg sleep_hours anxiety
1          1           0         4.8     1.8
2          2         400         7.1     6.2
3          3         200         3.4     2.5
4          4         100         6.9     1.9
5          5         400         7.0     5.3
6          6         400         5.6     4.7
```

Remove columns with a minus sign:

```
lab_students |>
  select(-student_id) |>
  head()
```

```
  caffeine_mg sleep_hours group_project_hatred hears_colors anxiety
1           0         4.8                    6           No     1.8
2         400         7.1                    9           No     6.2
3         200         3.4                    5           No     2.5
4         100         6.9                    6           No     1.9
5         400         7.0                    1          Yes     5.3
6         400         5.6                    5           No     4.7
```

38.5.2 `filter()`: Keep Rows

```
chromatic_students <- lab_students |>
  filter(hears_colors == "Yes")

nrow(chromatic_students)
```

```
[1] 16
```

Combine conditions with `&` for **and** and `|` for **or**:

```
lab_students |>
  filter(caffeine_mg >= 200 & sleep_hours < 6) |>
  head()
```

```
  student_id caffeine_mg sleep_hours group_project_hatred hears_colors anxiety
1          3         200         3.4                    5           No     2.5
2          6         400         5.6                    5           No     4.7
3         11         200         5.1                    9           No     5.8
4         14         400         5.9                    9           No     8.5
5         17         200         5.0                    1           No     3.8
6         23         200         4.3                    3           No     4.1
```

Inside `filter()`, a comma means the same thing as `&`: every condition must be true. So `filter(caffeine_mg >= 200, sleep_hours < 6)` returns exactly the rows above. You will see both, including in the solutions at the end of this chapter, and neither is more correct than the other.

Use `==` to test equality. One `=` assigns an argument; two `==` ask whether values are equal. This tiny distinction has destroyed entire afternoons.

38.5.3 `mutate()`: Create or Change Columns

```
lab_students <- lab_students |>
  mutate(
    caffeine_per_hour_sleep = caffeine_mg / sleep_hours,
    extremely_caffeinated = caffeine_mg >= 400
  )

head(lab_students)
```

```
  student_id caffeine_mg sleep_hours group_project_hatred hears_colors anxiety
1          1           0         4.8                    6           No     1.8
2          2         400         7.1                    9           No     6.2
3          3         200         3.4                    5           No     2.5
4          4         100         6.9                    6           No     1.9
5          5         400         7.0                    1          Yes     5.3
6          6         400         5.6                    5           No     4.7
  caffeine_per_hour_sleep extremely_caffeinated
1                 0.00000                FALSE
2                56.33803                 TRUE
3                58.82353                FALSE
4                14.49275                FALSE
5                57.14286                 TRUE
6                71.42857                 TRUE
```

`mutate()` keeps the existing variables and adds or replaces columns. The expression is calculated once per row.

38.5.4 `arrange()`: Sort Rows

```
lab_students |>
  arrange(desc(anxiety)) |>
  select(student_id, caffeine_mg, sleep_hours, anxiety) |>
  head(10)
```

```
   student_id caffeine_mg sleep_hours anxiety
1          14         400         5.9     8.5
2         152         400         5.6     8.0
3          72         400         8.0     7.9
4         130         400         5.0     7.9
5         129         400         8.1     7.8
6         145         400         7.1     7.8
7         114         400         5.6     7.6
8          22         200         6.4     7.5
9          73         400         5.4     7.2
10        158         400         6.4     7.2
```

These are the students most likely to email at 2:13 a.m. with the subject line `QUICK QUESTION!!!!!`.

38.5.5 `summarise()`: Collapse Many Rows into Answers

```
lab_students |>
  summarise(
    n = n(),
    mean_anxiety = mean(anxiety),
    sd_anxiety = sd(anxiety),
    median_sleep = median(sleep_hours),
    proportion_hearing_colors = mean(hears_colors == "Yes")
  )
```

```
    n mean_anxiety sd_anxiety median_sleep proportion_hearing_colors
1 160       4.2475   1.863315          6.4                       0.1
```

Unlike `mutate()`, `summarise()` reduces many rows to a smaller summary. `n()` counts rows. A logical statement such as `hears_colors == "Yes"` becomes `TRUE` or `FALSE`; its mean is therefore the proportion that is `TRUE`.

38.6 Group, Then Summarize

`group_by()` tells later verbs to operate separately within categories.

```
caffeine_summary <- lab_students |>
  group_by(caffeine_mg) |>
  summarise(
    n = n(),
    mean_anxiety = mean(anxiety),
    sd_anxiety = sd(anxiety),
    mean_sleep = mean(sleep_hours),
    proportion_hearing_colors = mean(hears_colors == "Yes"),
    .groups = "drop"
  )

caffeine_summary
```

```
# A tibble: 4 x 6
  caffeine_mg     n mean_anxiety sd_anxiety mean_sleep proportion_hearing_colors
        <dbl> <int>        <dbl>      <dbl>      <dbl>                      <dbl>
1           0    36         2.48       1.45       6.23                     0.0278
2         100    44         4.01       1.33       6.16                     0
3         200    41         4.18       1.36       6.47                     0.0732
4         400    39         6.22       1.28       6.66                     0.308
```

`.groups = "drop"` removes the grouping after the summary. This prevents the invisible ghost of a previous grouping decision from possessing later analyses.

For simple frequency tables, `count()` is faster:

```
lab_students |>
  count(caffeine_mg, hears_colors)
```

```
  caffeine_mg hears_colors  n
1           0           No 35
2           0          Yes  1
3         100           No 44
4         200           No 38
5         200          Yes  3
6         400           No 27
7         400          Yes 12
```

For the same summaries across several columns, use `across()`:

```
lab_students |>
  group_by(caffeine_mg) |>
  summarise(
    across(
      c(sleep_hours, anxiety, group_project_hatred),
      list(mean = mean, sd = sd),
      .names = "{.col}_{.fn}"
    ),
    .groups = "drop"
  )
```

```
# A tibble: 4 x 7
  caffeine_mg sleep_hours_mean sleep_hours_sd anxiety_mean anxiety_sd
        <dbl>            <dbl>          <dbl>        <dbl>      <dbl>
1           0             6.23           1.57         2.48       1.45
2         100             6.16           1.13         4.01       1.33
3         200             6.47           1.29         4.18       1.36
4         400             6.66           1.26         6.22       1.28
# i 2 more variables: group_project_hatred_mean <dbl>,
#   group_project_hatred_sd <dbl>
```

38.7 Reading a Real Data File

Quarto renders the book from the project directory. Use a project-relative path instead of `setwd()`:

```
caffeine_data <- read.csv("data/caffeine.csv")
```

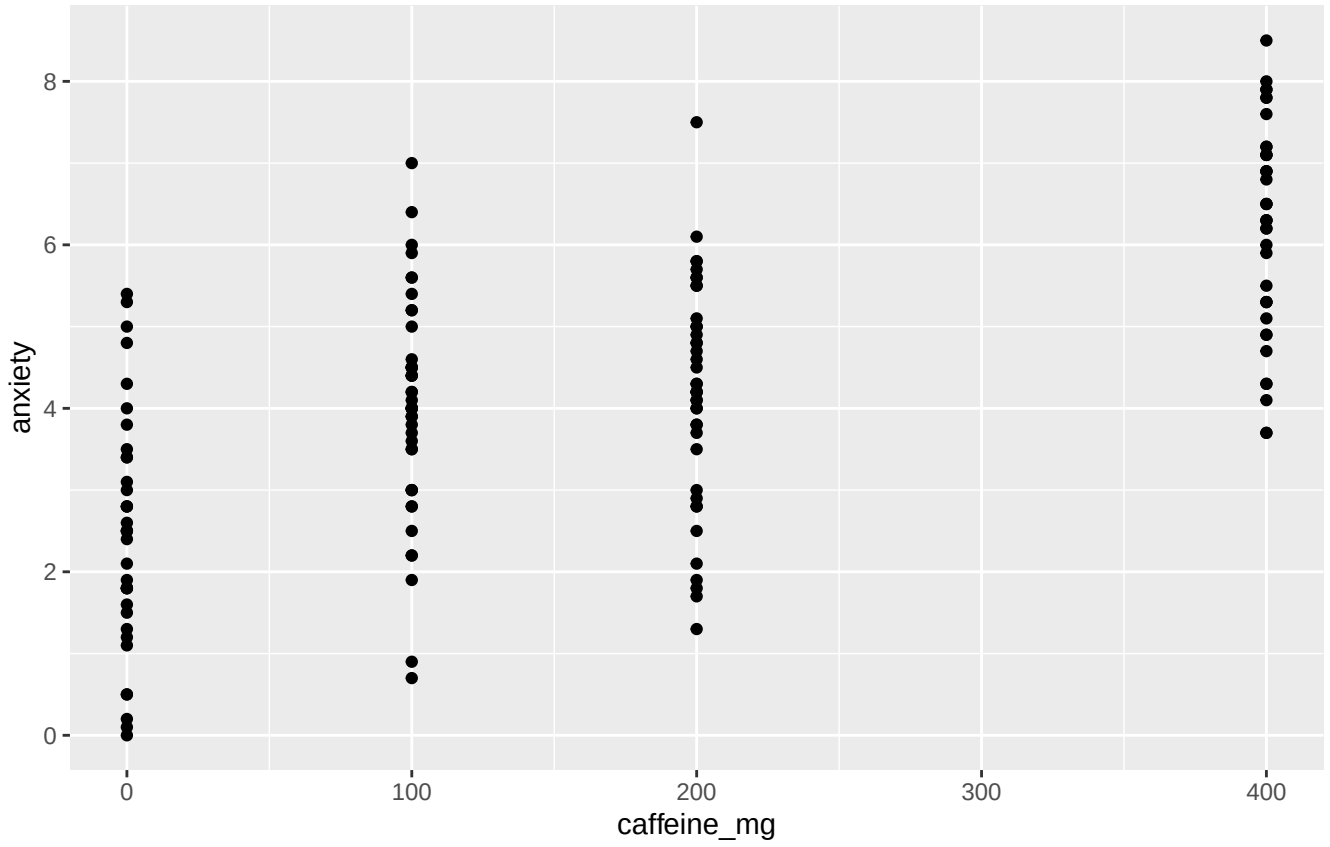
This says the file lives in a folder named `data` inside the project. Hard-coding a path to one person's Dropbox guarantees that the code will fail on another computer, next semester, or immediately after Alex reorganizes his folders during an avoidable burst of optimism.

38.8 ggplot2: Build a Graph in Layers

A `ggplot` usually has three core ingredients:

1. **Data**: the data frame.
2. **Aesthetics**: variables mapped to position, color, shape, or size with `aes()`.
3. **Geometries**: the marks placed on the graph, such as points, bars, or boxes.

```
ggplot(
  data = lab_students,
  mapping = aes(x = caffeine_mg, y = anxiety)
) +
  geom_point()
```



The + adds layers to a ggplot. It is not interchangeable with the pipe. The pipe moves data through operations; the plus sign adds components to a graph. R is very committed to making punctuation carry emotional weight.

38.8.1 Scatterplot with a Model Line

```
ggplot(lab_students, aes(x = caffeine_mg, y = anxiety, color = hears_colors)) +
  geom_jitter(width = 12, height = 0, alpha = .65, size = 2) +
  geom_smooth(method = "lm", se = TRUE, color = "black") +
  scale_color_manual(values = c("No" = "#1F4E79", "Yes" = "#D55E00")) +
```

```
  labs(
    x = "Caffeine (mg)",
    y = "Anxiety rating",
    color = "Hears colors",
    title = "At some point the espresso begins explaining the wallpaper"
  ) +
  theme_minimal(base_size = 12)
```

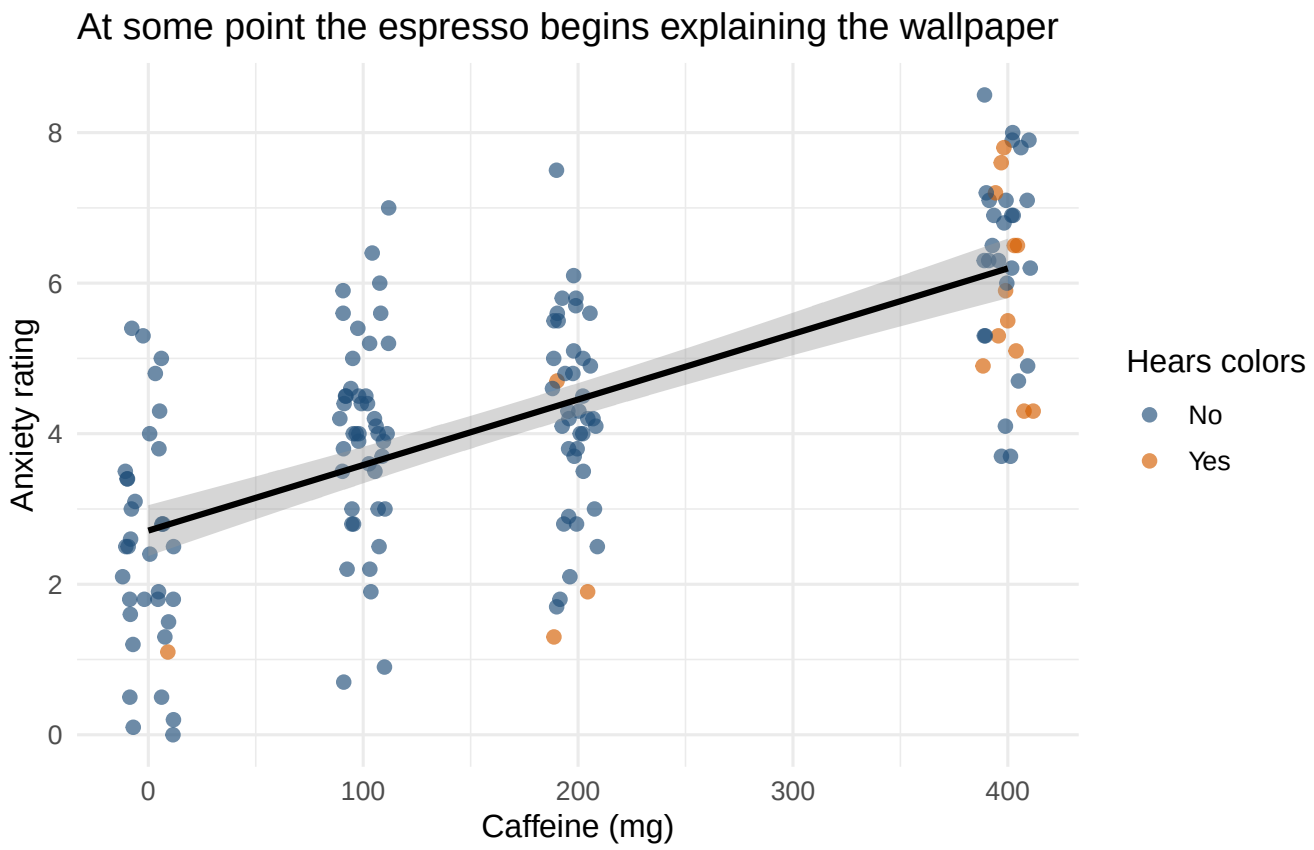


Figure 38.1: Anxiety across caffeine doses. Jitter separates students who otherwise occupy the same caffeine value.

`geom_jitter()` moves overlapping points by a tiny random amount horizontally so we can see them. It does not change the caffeine condition stored in the data.

38.8.2 Boxplots, Correctly Explained

```
ggplot(lab_students, aes(x = factor(caffeine_mg), y = anxiety)) +
  geom_boxplot(fill = "#9ECAE1", color = "#1F4E79", outlier.color = "#D55E00") +
  labs(
    x = "Caffeine (mg)",
    y = "Anxiety rating",
    title = "Boxes: useful, compact, and frequently lied about"
  ) +
  theme_minimal(base_size = 12)
```

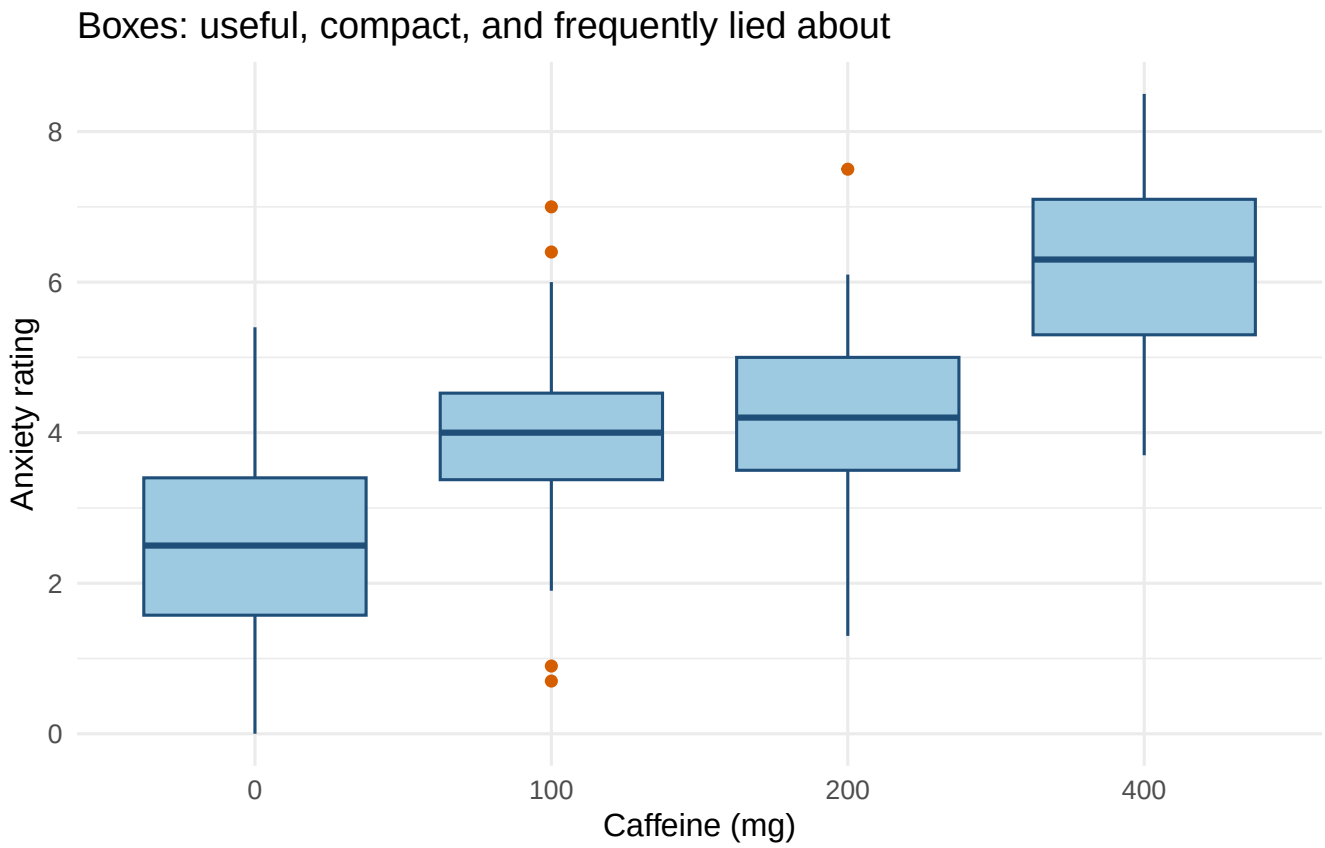


Figure 38.2: Anxiety distributions at each caffeine dose.

The line inside each box is the **median**. The bottom and top of the box are the first and third quartiles, so the box contains the middle **50%** of scores. Its height is the interquartile range (IQR). The whiskers ordinarily extend to the most extreme observed values within 1.5 IQRs of the box; points beyond them are plotted separately.

Those separate points are not automatically errors, criminals, or volunteers for deletion. They are observations that deserve inspection and an explanation.

38.8.3 Plotting a Summary

```
ggplot(caffeine_summary, aes(x = factor(caffeine_mg), y = proportion_hearing_colors)) +
  geom_col(fill = "#CC79A7", width = .70) +
  scale_y_continuous(limits = c(0, 1)) +
  labs(
    x = "Caffeine (mg)",
    y = "Proportion hearing colors",
    title = "Eventually teal starts whispering"
  ) +
  theme_minimal(base_size = 12)
```

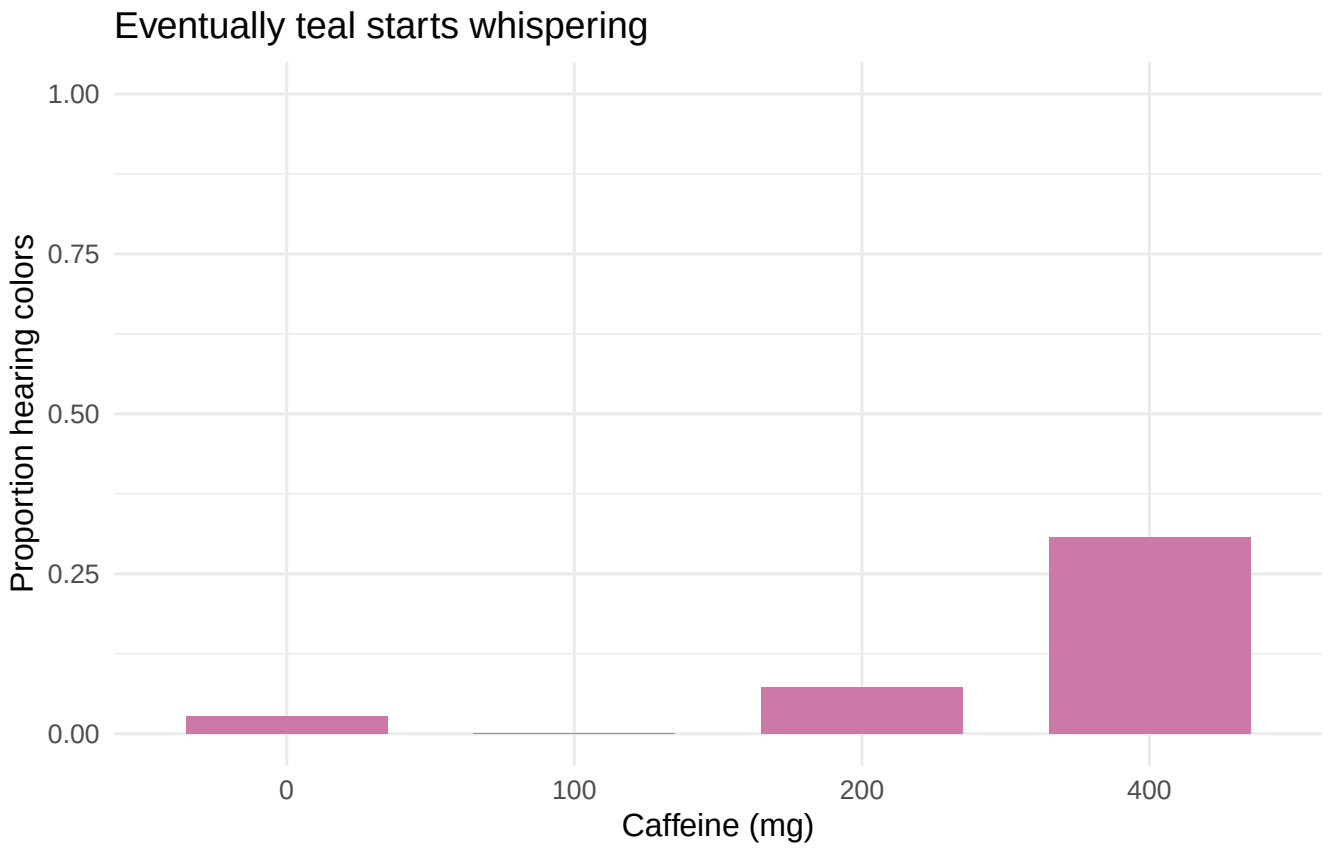


Figure 38.3: Simulated proportion of students reporting chromatic audition by caffeine dose.

`geom_col()` uses the y-values already calculated in `caffeine_summary`. `geom_bar()` would count raw rows instead. This difference is small, annoying, and worth remembering.

38.9 A Compact Workflow

Most routine work follows this sequence:

```
analysis_data <- lab_students |>
  filter(!is.na(anxiety)) |>
  mutate(caffeine_group = factor(caffeine_mg)) |>
  select(student_id, caffeine_group, sleep_hours, anxiety, hears_colors)

analysis_summary <- analysis_data |>
  group_by(caffeine_group) |>
  summarise(
    n = n(),
    mean = mean(anxiety),
    sd = sd(anxiety),
    .groups = "drop"
  )

analysis_summary
```

```
# A tibble: 4 x 4
  caffeine_group     n  mean    sd
  <fct>          <int> <dbl> <dbl>
1 0                 36  2.48  1.45
2 100               44  4.01  1.33
3 200               41  4.18  1.36
4 400               39  6.22  1.28
```

1. Start with the original data.
2. Filter only with a defensible rule.
3. Create or recode needed variables.
4. Select the variables used in the analysis.
5. Check and summarize the result.
6. Graph the data before asking a model to explain them.

Keep the original data unchanged and create a new analysis object. Overwriting the only copy of a variable is how `Final_Data_REAL_v7_USE_THIS_ONE.csv` enters the world.

38.10 Practice: Make the Caffeine Data Confess

Try these before looking at the solutions:

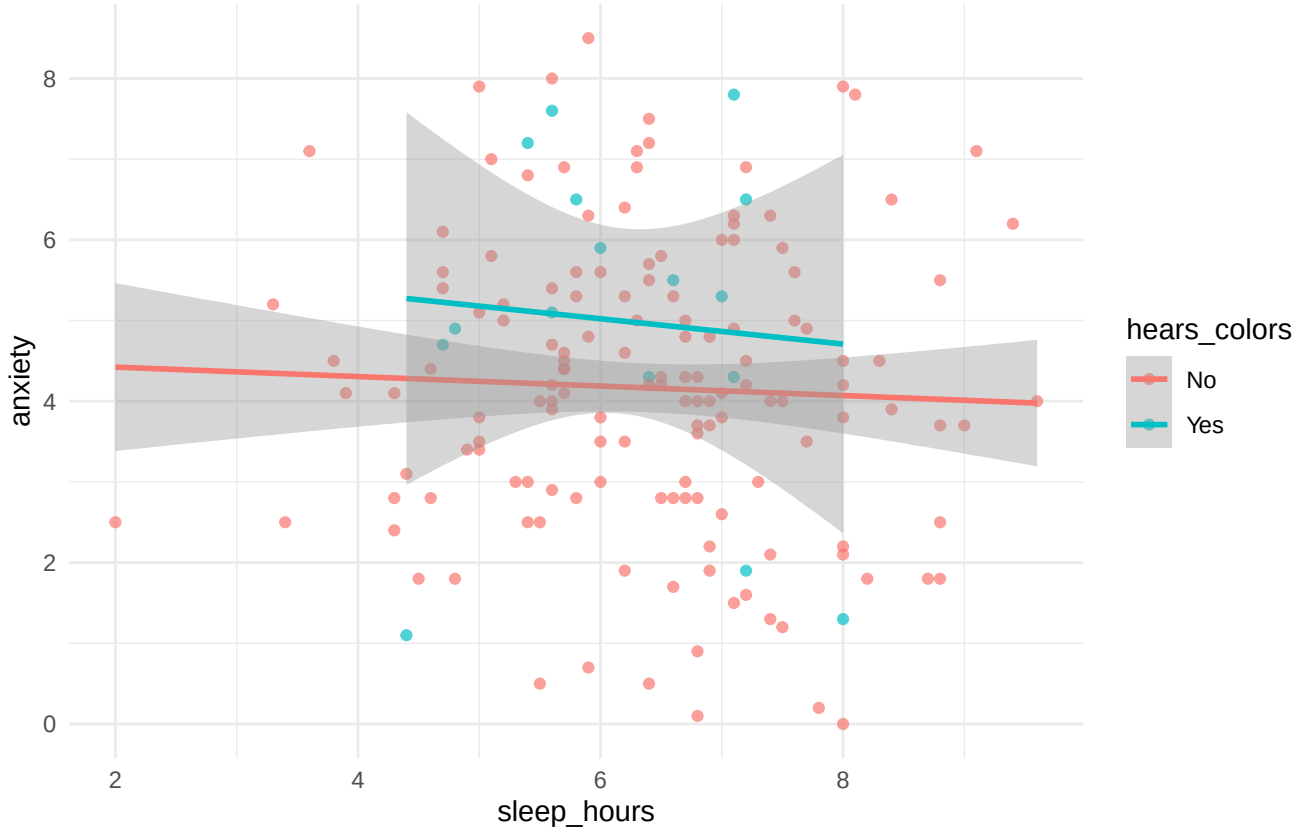
1. Keep students who slept fewer than six hours and consumed at least 200 mg of caffeine.
2. For each caffeine dose, calculate sample size, median anxiety, IQR of anxiety, and the proportion hearing colors.
3. Create a scatterplot of sleep and anxiety, coloring points by whether the student hears colors.
4. Find the ten students with the highest caffeine-per-hour-of-sleep values.

Solutions, because suffering has limits

```
sleep_deprived_and_caffeinated <- lab_students |>
  filter(sleep_hours < 6, caffeine_mg >= 200)

practice_summary <- lab_students |>
  group_by(caffeine_mg) |>
  summarise(
    n = n(),
    median_anxiety = median(anxiety),
    iqr_anxiety = IQR(anxiety),
    proportion_hearing_colors = mean(hears_colors == "Yes"),
    .groups = "drop"
  )

ggplot(lab_students, aes(x = sleep_hours, y = anxiety, color = hears_colors)) +
  geom_point(alpha = .70) +
  geom_smooth(method = "lm", se = TRUE) +
  theme_minimal()
```




```
most_concerning_students <- lab_students |>
  arrange(desc(caffeine_per_hour_sleep)) |>
  select(student_id, caffeine_mg, sleep_hours, caffeine_per_hour_sleep) |>
  head(10)

practice_summary
```

```
# A tibble: 4 x 5
  caffeine_mg     n median_anxiety iqr_anxiety proportion_hearing_colors
        <dbl> <int>          <dbl>       <dbl>                     <dbl>
1           0    36            2.5        1.82                    0.0278
```

38.11 R Will Let You Do Anything

Write code as a sequence of small, named decisions. Inspect the result after each important transformation. Make graphs before interpreting models. When something fails, read the error message from the bottom up and identify the first object or argument R says it cannot find.

Do not confuse code that runs with an analysis that makes sense. R will cheerfully calculate the mean of participant ID numbers, regress anxiety on alphabetical order, and draw a confidence interval around almost any mistake you can encode. The software is obedient. You still have to be the adult in the room, which seems unfair but is apparently how science works.

References

Abelson, Robert P. 1995. *Statistics as Principled Argument*. Lawrence Erlbaum Associates.

Aiken, Leona S., and Stephen G. West. 1991. *Multiple Regression: Testing and Interpreting Interactions*. Sage.

Ang, L. H., and Masitah Shahrill. 2014. "Identifying Students' Specific Misconceptions in Learning Probability." *International Journal of Probability and Statistics* 3 (2): 23–29.

Benjamini, Yoav, and Yosef Hochberg. 1995. "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing." *Journal of the Royal Statistical Society: Series B (Methodological)* 57 (1): 289–300. https://doi.org/10.1111/j.2517-6161.1995.tb02031.x.

Bennett, Craig M., Abigail A. Baird, Michael B. Miller, and George L. Wolford. 2010. "Neural Correlates of Interspecies Perspective Taking in the Post-Mortem Atlantic Salmon: An Argument for Proper Multiple Comparisons Correction." *Journal of Serendipitous and Unexpected Results* 1 (1): 1–5.

Best, John B. 1979. "Item Difficulty and Answer Changing." *Teaching of Psychology* 6 (4): 228–30. https://doi.org/10.1207/s15328023top0604_13.

Button, Katherine S., John P. A. Ioannidis, Claire Mokrysz, et al. 2013. "Power Failure: Why Small Sample Size Undermines the Reliability of Neuroscience." *Nature Reviews Neuroscience* 14 (5): 365–76. https://doi.org/10.1038/nrn3475.

Cohen, Jacob. 1962. "The Statistical Power of Abnormal-Social Psychological Research: A Review." *The Journal of Abnormal and Social Psychology* 65 (3): 145–53. https://doi.org/10.1037/h0045186.

Cohen, Jacob. 1988. *Statistical Power Analysis for the Behavioral Sciences*. 2nd ed. Lawrence Erlbaum Associates.

Cohen, Jacob. 1992. "A Power Primer." *Psychological Bulletin* 112 (1): 155–59. https://doi.org/10.1037/0033-2909.112.1.155.

Cohen, Jacob. 1994. "The Earth Is Round ($p < .05$)." *American Psychologist* 49 (12): 997–1003. https://doi.org/10.1037/0003-066X.49.12.997.

Cohen, Jacob, Patricia Cohen, Stephen G. West, and Leona S. Aiken. 2003. *Applied Multiple Regression/Correlation Analysis for the Behavioral Sciences*. 3rd ed. Lawrence Erlbaum Associates.

Efron, Bradley. 2003. "Second Thoughts on the Bootstrap." *Statistical Science* 18 (2): 135–40. https://doi.org/10.1214/ss/1063994968.

Efron, Bradley, and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman; Hall/CRC.

Errington, Timothy M., Maya Mathur, Courtney K. Soderberg, et al. 2021. "Investigating the Replicability of Preclinical

Cancer Biology." *eLife* 10: e71601. https://doi.org/10.7554/eLife.71601.

Fox, John, and Sanford Weisberg. 2019. *An r Companion to Applied Regression*. Third. Sage. https://www.john-fox.ca/Companion/.

Friedrich, Robert J. 1982. "In Defense of Multiplicative Terms in Multiple Regression Equations." *American Journal of Political Science* 26 (4): 797–833. https://doi.org/10.2307/2110973.

Gelman, Andrew, and Hal Stern. 2006. "The Difference Between 'Significant' and 'Not Significant' Is Not Itself Statistically Significant." *The American Statistician* 60 (4): 328–31. https://doi.org/10.1198/000313006X152649.

Hedges, Larry V. 1981. "Distribution Theory for Glass's Estimator of Effect Size and Related Estimators." *Journal of Educational Statistics* 6 (2): 107–28. https://doi.org/10.3102/10769986006002107.

Hoekstra, Rink, Richard D. Morey, Jeffrey N. Rouder, and Eric-Jan Wagenmakers. 2014. "Robust Misinterpretation of Confidence Intervals." *Psychonomic Bulletin & Review* 21 (5): 1157–64. https://doi.org/10.3758/s13423-013-0572-3.

Hoenig, John M., and Dennis M. Heisey. 2001. "The Abuse of Power: The Pervasive Fallacy of Power Calculations for Data Analysis." *The American Statistician* 55 (1): 19–24. https://doi.org/10.1198/000313001300339897.

Holm, Sture. 1979. "A Simple Sequentially Rejective Multiple Test Procedure." *Scandinavian Journal of Statistics* 6 (2): 65–70.

Ioannidis, John P. A. 2005. "Why Most Published Research Findings Are False." *PLoS Medicine* 2 (8): e124. https://doi.org/10.1371/journal.pmed.0020124.

Ioannidis, John P. A. 2012. "Why Science Is Not Necessarily Self-Correcting." *Perspectives on Psychological Science* 7 (6): 645–54. https://doi.org/10.1177/1745691612464056.

Kicinski, Michal, David A. Springate, and Evangelos Kontopantelis. 2015. "Publication Bias in Meta-Analyses from the Cochrane Database of Systematic Reviews." *Statistics in Medicine* 34 (20): 2781–93. https://doi.org/10.1002/sim.6525.

Kuhn, Thomas S. 1962. *The Structure of Scientific Revolutions*. University of Chicago Press.

Lane-Getaz, Sharon J. 2005. "Summary of p-Value Survey Research." *Proceedings of the 2005 United States Conference on Teaching Statistics*, 13–17.

Lehmann, E. L. 1999. "'Student' and Small-Sample Theory." *Electronic Journal of Probability* 4 (11): 1–33.

Long, J. Scott, and Laurie H. Ervin. 2000. "Using Heteroscedasticity Consistent Standard Errors in the Linear Regression Model." *The American Statistician* 54 (3): 217–24. https://doi.org/10.1080/00031305.2000.10474549.

Lüdecke, Daniel, Mattan S. Ben-Shachar, Indrajeet Patil, Philip Waggoner, and Dominique Makowski. 2021. "Performance: An R Package for Assessment, Comparison and Testing of Statistical Models." *Journal of Open Source Software* 6 (60): 3139. https://doi.org/10.21105/joss.03139.

MacCallum, Robert C., Shaobo Zhang, Kristopher J. Preacher, and Derek D. Rucker. 2002. "On the Practice of Dichotomization of Quantitative Variables." *Psychological Methods* 7 (1): 19–40. https://doi.org/10.1037/1082-989X.7.1.19.

Marcoci, Alexandru, David P. Wilkinson, Ans Vercammen, Bonnie C. Wintle, et al. 2025. "Predicting the Replicability of Social and Behavioural Science Claims in COVID-19 Preprints." *Nature Human Behaviour* 9: 287–304. https://doi.org/10.1038/s41562-024-01961-1.

Marek, Scott, Brenden Tervo-Clemmens, Finnegan J. Calabro, et al. 2022. "Reproducible Brain-Wide Association Studies Require Thousands of Individuals." *Nature* 603: 654–60. https://doi.org/10.1038/s41586-022-04492-9.

Meehl, Paul E. 1978. "Theoretical Risks and Tabular Asterisks: Sir Karl, Sir Ronald, and the Slow Progress of Soft Psychology." *Journal of Consulting and Clinical Psychology* 46 (4): 806–34. https://doi.org/10.1037/0022-006X.46.4.806.

Meehl, Paul E. 1990a. "Appraising and Amending Theories: The Strategy of Lakatosian Defense and Two Principles That Warrant It." *Psychological Inquiry* 1 (2): 108–41. https://doi.org/10.1207/s15327965pli0102_1.

Meehl, Paul E. 1990b. "Why Summaries of Research on Psychological Theories Are Often Uninterpretable." *Psychological Reports* 66 (1): 195–244. https://doi.org/10.2466/pr0.1990.66.1.195.

Michell, Joel. 1997. "Quantitative Science and the Definition of Measurement in Psychology." *British Journal of Psychology* 88 (3): 355–83. https://doi.org/10.1111/j.2044-8295.1997.tb02641.x.

Miller, Lulu. 2020. *Why Fish Don't Exist: A Story of Loss, Love, and the Hidden Order of Life*. Simon & Schuster.

Nature Neuroscience. 2024. "Reducing Publication Bias with Registered Reports." *Nature Neuroscience* 27: 1635. https://doi.org/10.1038/s41593-024-01762-9.

Open Science Collaboration. 2015. "Estimating the Reproducibility of Psychological Science." *Science* 349 (6251): aac4716. https://doi.org/10.1126/science.aac4716.

Perugini, Marco, Marcello Gallucci, and Giulio Costantini. 2014. "Safeguard Power as a Protection Against Imprecise Power Estimates." *Perspectives on Psychological Science* 9 (3): 319–32. https://doi.org/10.1177/1745691614528519.

Peters, Michael, Bruno Laeng, K. Latham, M. Jackson, R. Zaiyouna, and C. Richardson. 1995. "A Redrawn Vandenberg and Kuse Mental Rotations Test: Different Versions and Factors That Affect Performance." *Brain and Cognition* 28 (1): 39–58. https://doi.org/10.1006/brcg.1995.1032.

Singh, Kesar, and Minge Xie. 2008. "Bootstrap: A Statistical Method." Unpublished manuscript.

Stevens, S. S. 1946. "On the Theory of Scales of Measurement." *Science* 103 (2684): 677–80. https://doi.org/10.1126/science.103.2684.677.

Student. 1908. "The Probable Error of a Mean." *Biometrika* 6 (1): 1–25. https://doi.org/10.1093/biomet/6.1.1.

U.S. Census Bureau. 2024. *Historical Income Tables: Households*. https://www.census.gov/data/tables/time-series/demo/income-poverty/cps-hinc/hinc-01.2024.html.

White, Halbert. 1980. "A Heteroskedasticity-Consistent Covariance Matrix Estimator and a Direct Test for Heteroskedasticity." *Econometrica* 48 (4): 817–38. https://doi.org/10.2307/1912934.

Wu, Chien-Fu Jeff. 1986. "Jackknife, Bootstrap and Other Resampling Methods in Regression Analysis." *The Annals of Statistics* 14 (4): 1261–95. https://doi.org/10.1214/aos/1176350142.

Yeager, David S., Paul Hanselman, Gregory M. Walton, Jared S. Murray, et al. 2019. "A National Experiment Reveals Where a Growth Mindset Improves Achievement." *Nature* 573: 364–69. https://doi.org/10.1038/s41586-019-1466-y.

Zeileis, Achim. 2004. "Econometric Computing with HC and HAC Covariance Matrix Estimators." *Journal of Statistical Software* 11 (10): 1–17. https://doi.org/10.18637/jss.v011.i10.

Zeileis, Achim, and Torsten Hothorn. 2002. "Diagnostic Checking in Regression Relationships." *R News* 2 (3): 7–10. https://CRAN.R-project.org/doc/Rnews/.